BOAZ BARAK

INTRODUCTION TO THEORETICAL COMPUTER SCIENCE

TEXTBOOK IN PREPARATION.
AVAILABLE ON HTTPS://INTROTCS.ORG

Text available on https://github.com/boazbk/tcs - please post any issues there - thank you!

This version was compiled on Wednesday 6th March, 2019 22:03

Copyright © 2019 Boaz Barak

This work is licensed under a Creative Commons "Attribution-NonCommercial-NoDerivatives 4.0 International" license.



To Ravit, Alma and Goren.

Contents

Preface 19

Preliminaries 27

0 Introduction 29

1 Mathematical Background 45

2 Computation and Representation 83

I Finite computation 119

3 Defining computation 121

4 Syntactic sugar, and computing every function 151

5 Code as data, data as code 169

II Uniform computation 195

6 Loops and infinity 197

7 Equivalent models of computation 225

8 Universality and uncomputability 261

9 Restricted computational models 291

10 Is every theorem provable? 325

III Efficient algorithms 343

11 Efficient computation 345

12 Modeling running time 363

13 Polynomial-time reductions 391

14 NP, NP completeness, and the Cook-Levin Theorem 405

15 What if P equals NP? 423

16 Space bounded computation 445

IV Randomized computation 447

17 Probability Theory 101 449

18 Probabilistic computation 465

19 Modeling randomized computation 477

V Advanced topics 497

20 Cryptography 499

21 Proofs and algorithms 525

22 Quantum computing 527

VI Appendices 557

Contents (detailed)

Preface **19**

0.1 To the student . . . 21

0.1.1 Is the effort worth it? . . . 21

0.2 To potential instructors . . . 22

0.3 Acknowledgements . . . 24

Preliminaries **27**

0 Introduction **29**

0.1 Extended Example: A faster way to multiply . . . 32

0.1.1 Beyond Karatsuba's algorithm . . . 35

0.2 Algorithms beyond arithmetic . . . 37

0.3 On the importance of negative results. . . . 38

0.4 Roadmap to the rest of this book . . . 39

0.4.1 Dependencies between chapters . . . 40

0.5 Exercises . . . 41

0.6 Bibliographical notes . . . 43

1 Mathematical Background **45**

1.1 A mathematician's apology . . . 45

1.2 This chapter: a reader's manual . . . 46

1.3 A quick overview of mathematical prerequisites . . . 46

1.4 Reading mathematical texts . . . 47

1.4.1 Definitions . . . 48

1.4.2 Assertions: Theorems, lemmas, claims . . . 50

1.4.3 Proofs . . . 50

1.4.4 Example: Defining a one to one function . . . 51

1.5 Basic discrete math objects . . . 52

1.5.1 Sets . . . 52

1.5.2 Sets in Python (optional) . . . 53

1.5.3 Special sets . . . 54

1.5.4 Functions . . . 55

1.5.5 Graphs . . . 58

1.5.6 Logic operators and quantifiers. . . . 60

1.5.7 Quantifiers for summations and products 61
1.5.8 Parsing formulas: bound and free variables . . . 61
1.5.9 Asymptotics and Big-O notation 63
1.5.10 Some "rules of thumb" for Big-O notation 65
1.6 Proofs 65
1.6.1 Proofs and programs 66
1.6.2 Proof writing style 66
1.6.3 Patterns in proofs 67
1.7 Extended example: Topological Sorting 69
1.7.1 Mathematical induction 71
1.7.2 Proving the result by induction 72
1.7.3 Minimality and uniqueness 74
1.8 Notation and conventions 76
1.8.1 Conventions 77
1.9 Exercises 79
1.10 Bibliographical notes 81

2 Computation and Representation 83
2.1 Defining representations 84
2.1.1 Representing natural numbers. 85
2.1.2 Implementing the representation in python: . . . 87
2.1.3 Meaning of representation 88
2.2 Representing more objects 89
2.2.1 Representing (potentially negative) integers . . . 89
2.2.2 Representing rational numbers 90
2.3 Representing real numbers 91
2.3.1 Can we represent reals *exactly*? 92
2.3.2 { .definition title = "The set $\{0,1\}^\infty$" } 93
2.4 Beyond numbers 97
2.4.1 Finite representations 97
2.4.2 Prefix-free encoding 98
2.4.3 Making representations prefix-free 100
2.4.4 "Proof by Python" (optional) 102
2.4.5 Representing letters and text 104
2.4.6 Representing vectors, matrices, images 107
2.4.7 Representing graphs 107
2.4.8 Representing lists 108
2.4.9 Notation 108
2.5 Defining computational tasks 109
2.5.1 Distinguish functions from programs! 111
2.6 Exercises 113
2.7 Bibliographical notes 117

I Finite computation **119**

3 Defining computation **121**
3.1 Defining computation . . . 121
3.2 Boolean formulas with AND, OR, and NOT. . . . 123
3.2.1 Extended example: Computing *XOR* from *AND*,*OR*,*NOT*. . . . 125
3.2.2 Informally defining "basic operations" and "algorithms" . . . 127
3.3 Physical implementations of computing devices. . . . 129
3.3.1 Transistors . . . 129
3.3.2 Logical gates from transistors . . . 130
3.3.3 Biological computing . . . 130
3.3.4 Cellular automata and the game of life . . . 130
3.3.5 Neural networks . . . 131
3.3.6 The marble computer . . . 131
3.4 Boolean Circuits . . . 132
3.5 Equivalence of circuits and straight-line programs . . . 133
3.5.1 "Proof by Python" . . . 136
3.6 The NAND function . . . 138
3.6.1 NAND Circuits . . . 139
3.6.2 More examples of NAND circuits (optional) . . . 141
3.6.3 The NAND-CIRC Programming language . . . 143
3.7 Equivalence of all these models . . . 146
3.7.1 Circuits with other gate sets (optional) . . . 147
3.8 Exercises . . . 148
3.9 Biographical notes . . . 149

4 Syntactic sugar, and computing every function **151**
4.1 Some examples syntactic sugar . . . 152
4.1.1 Constants . . . 152
4.1.2 Functions / Macros . . . 152
4.1.3 Example: Computing Majority via NAND's . . . 153
4.1.4 Conditional statements . . . 153
4.2 Extended example: Addition and Multiplication (optional) . . . 155
4.3 The LOOKUP function . . . 156
4.3.1 Constructing a NAND-CIRC program for *LOOKUP* . . . 157
4.4 Computing *every* function . . . 159
4.4.1 Proof of NAND's Universality . . . 160
4.4.2 Improving by a factor of n (optional) . . . 162
4.5 The class $SIZE_{n,m}(T)$ . . . 163
4.6 Exercises . . . 165

4.7 Bibliographical notes . . . 167

5 Code as data, data as code **169**
5.1 A NAND interpreter in NAND . . . 170
5.1.1 Efficient universal programs . . . 171
5.1.2 Concrete representation for NAND-CIRC programs . . . 172
5.1.3 A NAND interpeter in "pseudocode" . . . 173
5.1.4 A NAND interpreter in Python . . . 175
5.1.5 Constructing the NAND interpreter in NAND . . . 176
5.2 A Python interpreter in NAND (discussion) . . . 178
5.3 Counting programs, and lower bounds on the size of NAND-CIRC programs . . . 179
5.4 Size hierarchy theorem (advanced, optional) . . . 182
5.5 The physical extended Church-Turing thesis (discussion) . . . 183
5.5.1 Attempts at refuting the PECTT . . . 186
5.6 Finite computation recap . . . 191
5.7 Exercises . . . 191
5.8 Bibliographical notes . . . 193

II Uniform computation **195**

6 Loops and infinity **197**
6.1 Turing Machines . . . 199
6.1.1 Example: A Turing machine for palindromes . . . 200
6.1.2 Turing machines: a formal definition . . . 202
6.1.3 Computable functions . . . 204
6.1.4 Infinite loops and partial functions . . . 205
6.2 Turing machines as programming languages . . . 206
6.2.1 The NAND-TM Programming language . . . 207
6.2.2 Sneak peak: NAND-TM vs Turing machines . . . 209
6.2.3 Examples . . . 210
6.3 Equivalence of Turing machines and NAND-TM programs . . . 212
6.3.1 Specification vs implementation (again) . . . 215
6.4 NAND-TM syntactic sugar . . . 215
6.4.1 Well formed programs: The NAND-TM style manual . . . 217
6.5 Uniformity, and NAND vs NAND-TM (discussion) . . . 221
6.6 Exercises . . . 222
6.7 Bibliographical notes . . . 223

7 Equivalent models of computation **225**

7.1 RAM machines and NAND-RAM . . . 225
7.2 The gory details (optional) . . . 228
7.2.1 Indexed access in NAND-TM . . . 228
7.2.2 Two dimensional arrays in NAND-TM . . . 230
7.2.3 All the rest . . . 230
7.3 Turing equivalence (discussion) . . . 230
7.3.1 The "Best of both worlds" paradigm . . . 232
7.3.2 Let's talk about abstractions. . . . 232
7.3.3 Definition of "Algorithm" . . . 234
7.4 Lambda calculus and functional programming languages . . . 235
7.4.1 Applying functions to functions . . . 236
7.4.2 Obtaining multi-argument functions via Currying 236
7.4.3 Formal description of the λ calculus. . . . 237
7.4.4 Functions as first class objects . . . 239
7.5 The "Enhanced" λ calculus . . . 239
7.5.1 Enhanced λ expressions . . . 242
7.5.2 Enhanced λ calculus is Turing-complete . . . 243
7.6 The pure λ calculus . . . 245
7.6.1 List processing . . . 247
7.6.2 The Y combinator, or recursion without recursion 248
7.6.3 Infinite loops in the λ calculus . . . 251
7.7 More Turing-complete computational models . . . 251
7.7.1 Parallel algorithms and cloud computing . . . 251
7.7.2 Game of life, tiling and cellular automata . . . 252
7.7.3 Configurations of Turing machines and one dimensional cellular automata . . . 252
7.7.4 Turing completeness and equivalence, a formal definition (optional) . . . 256
7.8 The Church-Turing Thesis (discussion) . . . 257
7.8.1 Different models of computation . . . 258
7.9 Exercises . . . 259
7.10 Bibliographical notes . . . 259

8 Universality and uncomputability 261
8.1 Universality or a self-circular evaluator . . . 262
8.2 Is every function computable? . . . 267
8.3 The Halting problem . . . 269
8.3.1 Is the Halting problem really hard? (discussion) 271
8.3.2 A direct proof of the uncomputability of *HALT* (optional) . . . 272
8.4 Reductions . . . 274
8.4.1 Example: Halting on the zero problem . . . 276

8.5 Rice's Theorem and the impossibility of general software verification . . . 278
8.5.1 Rice's Theorem . . . 280
8.5.2 Halting and Rice's Theorem for other Turing-complete models . . . 284
8.5.3 Is software verification doomed? (discussion) . . . 285
8.6 Exercises . . . 287
8.7 Bibliographical notes . . . 289

9 Restricted computational models **291**
9.1 Turing completeness as a bug . . . 291
9.2 Regular expressions . . . 293
9.3 Deterministic finite automata, and efficient matching of regular expressions (optional) . . . 298
9.3.1 Matching regular expressions using constant memory . . . 301
9.3.2 Deterministic Finite Automata . . . 302
9.3.3 Regular functions are closed under complement . 305
9.4 Limitations of regular expressions . . . 305
9.5 Other semantic properties of regular expressions . . . 309
9.6 Context free grammars . . . 311
9.6.1 Context-free grammars as a computational model 313
9.6.2 The power of context free grammars . . . 315
9.6.3 Limitations of context-free grammars (optional) 317
9.7 Semantic properties of context free languages . . . 318
9.7.1 Uncomputability of context-free grammar equivalence (optional) . . . 319
9.8 Summary of semantic properties for regular expressions and context-free grammars . . . 322
9.9 Exercises . . . 323
9.10 Bibliographical notes . . . 324

10 Is every theorem provable? **325**
10.1 Hilbert's Program and Gödel's Incompleteness Theorem 326
10.2 Quantified integer statements . . . 330
10.3 Diophantine equations and the MRDP Theorem . . . 332
10.4 Hardness of quantified integer statements . . . 333
10.4.1 Step 1: Quantified mixed statements and computation histories . . . 334
10.4.2 Step 2: Reducing mixed statements to integer statements . . . 337
10.5 Exercises . . . 339
10.6 Bibliographical notes . . . 340

III Efficient algorithms **343**

11 Efficient computation **345**
11.1 Problems on graphs . . . 346
11.1.1 Finding the shortest path in a graph . . . 347
11.1.2 Finding the longest path in a graph . . . 349
11.1.3 Finding the minimum cut in a graph . . . 350
11.1.4 Finding the maximum cut in a graph . . . 353
11.1.5 A note on convexity . . . 353
11.2 Beyond graphs . . . 354
11.2.1 The 2SAT problem . . . 354
11.2.2 The 3SAT problem . . . 355
11.2.3 Solving linear equations . . . 355
11.2.4 Solving quadratic equations . . . 356
11.3 More advanced examples . . . 357
11.3.1 Determinant of a matrix . . . 357
11.3.2 The permanent (mod 2) problem . . . 358
11.3.3 The permanent (mod 3) problem . . . 358
11.3.4 Finding a zero-sum equilibrium . . . 359
11.3.5 Finding a Nash equilibrium . . . 359
11.3.6 Primality testing . . . 359
11.3.7 Integer factoring . . . 360
11.4 Our current knowledge . . . 360
11.5 Lecture summary . . . 361
11.6 Exercises . . . 361
11.7 Bibliographical notes . . . 361
11.8 Further explorations . . . 362
11.9 Acknowledgements . . . 362

12 Modeling running time **363**
12.1 Formally defining running time . . . 364
12.1.1 Nice time bounds . . . 365
12.1.2 Non-boolean and partial functions (optional) . . . 367
12.2 Efficient simulation of RAM machines: NAND-RAM vs NAND-TM . . . 368
12.3 Efficient universal machine: a NAND-RAM interpreter in NAND-RAM . . . 371
12.4 Time hierarchy theorem . . . 375
12.5 Unrolling the loop: Uniform vs non uniform computation . . . 377
12.5.1 Algorithmic transformation of NAND-TM to NAND and "Proof by Python" (optional) . . . 380
12.5.2 The class $\mathbf{P}_{/\mathbf{poly}}$ . . . 383
12.5.3 Simulating NAND with NAND-TM? . . . 384

12.5.4 Uniform vs. Nonuniform computation: A recap . 385
12.6 Extended Church-Turing Thesis 386
12.7 Exercises 388
12.8 Bibliographical notes 389
12.9 Further explorations 389
12.10 Acknowledgements 389

13 Polynomial-time reductions **391**
13.0.1 Decision problems 392
13.1 Reductions 392
13.2 Some example reductions 394
13.2.1 Reducing 3SAT to quadratic equations 394
13.3 The independent set problem 397
13.4 Reducing Independent Set to Maximum Cut 399
13.5 Reducing 3SAT to Longest Path 401
13.6 Exercises 402
13.7 Bibliographical notes 403
13.8 Further explorations 403
13.9 Acknowledgements 403

14 NP, NP completeness, and the Cook-Levin Theorem **405**
14.1 The class **NP** 405
14.1.1 Examples of **NP** functions 407
14.1.2 Basic facts about **NP** 409
14.2 From **NP** to 3SAT: The Cook-Levin Theorem 411
14.2.1 What does this mean? 412
14.2.2 The Cook-Levin Theorem: Proof outline 413
14.3 The *NANDSAT* Problem, and why it is **NP** hard. 414
14.4 The 3*NAND* problem 416
14.5 From 3*NAND* to 3*SAT* 418
14.6 Wrapping up 419
14.7 Exercises 420
14.8 Bibliographical notes 421
14.9 Further explorations 421
14.10 Acknowledgements 421

15 What if P equals NP? **423**
15.1 Search-to-decision reduction 424
15.2 Optimization 426
15.2.1 Example: Supervised learning 430
15.2.2 Example: Breaking cryptosystems 430
15.3 Finding mathematical proofs 431
15.4 Quantifier elimination (advanced) 432
15.4.1 Application: self improving algorithm for 3*SAT* . 435
15.5 Approximating counting problems (advanced, optional) 435

15.6 What does all of this imply? . . . 437
15.7 Can **P** $\neq$ **NP** be neither true nor false? . . . 439
15.8 Is **P** $=$ **NP** "in practice"? . . . 440
15.9 What if **P** $\neq$ **NP**? . . . 441
15.10 Exercises . . . 442
15.11 Bibliographical notes . . . 443
15.12 Further explorations . . . 443
15.13 Acknowledgements . . . 443

16 Space bounded computation **445**
16.1 Lecture summary . . . 445
16.2 Exercises . . . 445
16.3 Bibliographical notes . . . 445
16.4 Further explorations . . . 445
16.5 Acknowledgements . . . 445

IV Randomized computation **447**

17 Probability Theory 101 **449**
17.1 Random coins . . . 449
17.1.1 Random variables . . . 452
17.1.2 Distributions over strings . . . 453
17.1.3 More general sample spaces. . . . 454
17.2 Correlations and independence . . . 454
17.2.1 Independent random variables . . . 456
17.2.2 Collections of independent random variables. . . . 457
17.3 Concentration . . . 458
17.3.1 Chebyshev's Inequality . . . 459
17.3.2 The Chernoff bound . . . 460
17.4 Lecture summary . . . 461
17.5 Exercises . . . 461
17.6 Bibliographical notes . . . 464

18 Probabilistic computation **465**
18.1 Finding approximately good maximum cuts. . . . 466
18.1.1 Amplification . . . 467
18.1.2 Two-sided amplification . . . 469
18.1.3 What does this mean? . . . 469
18.1.4 Solving SAT through randomization . . . 470
18.1.5 Bipartite matching. . . . 472
18.2 Exercises . . . 475
18.3 Bibliographical notes . . . 475
18.4 Further explorations . . . 476
18.5 Acknowledgements . . . 476

19 Modeling randomized computation **477**
19.0.1 An alternative view: random coins as an "extra input" . . . 478
19.0.2 Amplification . . . 480
19.1 **BPP** and **NP** completeness . . . 481
19.2 The power of randomization . . . 483
19.2.1 Solving **BPP** in exponential time . . . 483
19.2.2 Simulating randomized algorithms by circuits or straight-line programs. . . . 483
19.3 Derandomization . . . 485
19.3.1 Pseudorandom generators . . . 486
19.3.2 From existence to constructivity . . . 488
19.3.3 Usefulness of pseudorandom generators . . . 489
19.4 **P** = **NP** and **BPP** vs **P** . . . 490
19.5 Non-constructive existence of pseudorandom generators (advanced, optional) . . . 493
19.6 Exercises . . . 496
19.7 Bibliographical notes . . . 496
19.8 Further explorations . . . 496
19.9 Acknowledgements . . . 496

V Advanced topics **497**

20 Cryptography **499**
20.1 Classical cryptosystems . . . 500
20.2 Defining encryption . . . 502
20.3 Defining security of encryption . . . 503
20.4 Perfect secrecy . . . 505
20.4.1 Example: Perfect secrecy in the battlefield . . . 506
20.4.2 Constructing perfectly secret encryption . . . 507
20.5 Necessity of long keys . . . 508
20.6 Computational secrecy . . . 510
20.6.1 Stream ciphers or the "derandomized one-time pad" . . . 512
20.7 Computational secrecy and **NP** . . . 514
20.8 Public key cryptography . . . 516
20.8.1 Defining public key encryption . . . 518
20.8.2 Diffie-Hellman key exchange . . . 519
20.9 Other security notions . . . 520
20.10 Magic . . . 521
20.10.1 Zero knowledge proofs . . . 521
20.10.2 Fully homomorphic encryption . . . 521
20.10.3 Multiparty secure computation . . . 522

20.11 Exercises . . . 523
20.12 Bibliographical notes . . . 523
20.13 Further explorations . . . 524
20.14 Acknowledgements . . . 524

21 Proofs and algorithms **525**
21.1 Lecture summary . . . 525
21.2 Exercises . . . 525
21.3 Bibliographical notes . . . 525
21.4 Further explorations . . . 525
21.5 Acknowledgements . . . 526

22 Quantum computing **527**
22.1 The double slit experiment . . . 528
22.2 Quantum amplitudes . . . 528
22.3 Bell's Inequality . . . 531
22.4 Quantum weirdness . . . 532
22.5 Quantum computing and computation - an executive summary. . . . 533
22.6 Quantum systems . . . 535
22.6.1 Quantum amplitudes . . . 537
22.6.2 Recap . . . 538
22.7 Analysis of Bell's Inequality (optional) . . . 538
22.8 Quantum computation . . . 540
22.8.1 Quantum circuits . . . 541
22.8.2 QNAND-CIRC programs (optional) . . . 544
22.8.3 Uniform computation . . . 544
22.9 Physically realizing quantum computation . . . 545
22.10 Shor's Algorithm: Hearing the shape of prime factors . . . 546
22.10.1 Period finding . . . 547
22.10.2 Shor's Algorithm: A bird's eye view . . . 547
22.11 Quantum Fourier Transform (advanced, optional) . . . 550
22.11.1 Quantum Fourier Transform over the Boolean Cube: Simon's Algorithm . . . 551
22.11.2 From Fourier to Period finding: Simon's Algorithm (advanced, optional) . . . 553
22.11.3 From Simon to Shor (advanced, optional) . . . 553
22.12 Exercises . . . 555
22.13 Bibliographical notes . . . 555
22.14 Further explorations . . . 556
22.15 Acknowledgements . . . 556

VI Appendices **557**

Preface

> *"We make ourselves no promises, but we cherish the hope that the unobstructed pursuit of useless knowledge will prove to have consequences in the future as in the past" ... "An institution which sets free successive generations of human souls is amply justified whether or not this graduate or that makes a so-called useful contribution to human knowledge. A poem, a symphony, a painting, a mathematical truth, a new scientific fact, all bear in themselves all the justification that universities, colleges, and institutes of research need or require"*, Abraham Flexner, The Usefulness of Useless Knowledge, 1939.

> *"I suggest that you take the hardest courses that you can, because you learn the most when you challenge yourself... CS 121 I found pretty hard."*, Mark Zuckerberg, 2005.

This is a textbook for an undergraduate introductory course on Theoretical Computer Science. The educational goals of this course are to convey the following:

- That computation but arises in a variety of natural and human-made systems, and not only in modern silicon-based computers.
- Similarly, beyond being an extremely important *tool*, computation also serves as a useful *lens* to describe natural, physical, mathematical and even social concepts.
- The notion of *universality* of many different computational models, and the related notion of the duality between *code* and *data*.
- The idea that one can precisely define a mathematical model of computation, and then use that to prove (or sometimes only conjecture) lower bounds and impossibility results.
- Some of the surprising results and discoveries in modern theoretical computer science, including the prevalence of NP-completeness, the power of interaction, the power of randomness on one hand

and the possibility of derandomization on the other, the ability to use hardness "for good" in cryptography, and the fascinating possibility of quantum computing.

I hope that following this course, students would be able to recognize computation, with both its power and pitfalls, as it arises in various settings, including seemingly "static" content or "restricted" formalisms such as macros and scripts. They should be able to follow through the logic of *proofs* about computation, including the central concept of a *reduction* as well understand "self-referential" proofs (such as diagonalization-based proofs that involve programs given their own code as input). Students should understand that some problems are *inherently intractable*, and be able to recognize the potential for intractability when they are faced with a new problem. While this course only touches on cryptography, students should understand the basic idea of how we can use computational hardness for cryptographic purposes. However, more than any specific skill, this course aims to introduce students to a new way of thinking of computation as an object in its own right and to illustrate how this new way of thinking leads to far-reaching insights and applications.

My aim in writing this text is to try to convey these concepts in the simplest possible way and try to make sure that the formal notation and model help elucidate, rather than obscure, the main ideas. I also tried to take advantage of modern students' familiarity (or at least interest!) in programming, and hence use (highly simplified) programming languages to describe our models of computation. That said, this course does not assume fluency with any particular programming language, but rather only some familiarity with the general *notion* of programming. We will use programming metaphors and idioms, occasionally mentioning specific programming languages such as *Python*, *C*, or *Lisp*, but students should be able to follow these descriptions even if they are not familiar with these languages.

Proofs in this course, including the existence of a universal Turing Machine, the fact that every finite function can be computed by some circuit, the Cook-Levin theorem, and many others, are often constructive and algorithmic, in the sense that they ultimately involve transforming one program to another. While it is possible to follow these proofs without seeing the code, I do think that having access to the code, and the ability to play around with it and see how it acts on various programs, can make these theorems more concrete for the students. To that end, an accompanying website (which is still work in progress) allows executing programs in the various computational models we define, as well as see constructive proofs of some of the theorems.

0.1 TO THE STUDENT

This book can be challenging, mainly because it brings together a variety of ideas and techniques in the study of computation. There are quite a few technical hurdles to master, whether it is following the diagonalization argument in proving the Halting Problem is undecidable, combinatorial gadgets in NP-completeness reductions, analyzing probabilistic algorithms, or arguing about the adversary to prove the security of cryptographic primitives.

The best way to engage with this material is to read these notes **actively**, so make sure you have a pen ready. While reading, I encourage you to stop and think about the following:

- When I state a theorem, stop and take a shot at proving it on your own *before* reading the proof. You will be amazed by how much you can understand a proof better even after only 5 minutes of attempting it on your own.
- When reading a definition, make sure that you understand what the definition means, and what are natural examples of objects that satisfy it and objects that do not. Try to think of the motivation behind the definition, and whether there are other natural ways to formalize the same concept.
- Actively notice which questions arise in your mind as you read the text, and whether or not they are answered in the text.

As a general rule, it is more important that you understand the **definitions** than the **theorems**, and it is more important that you understand a **theorem statement** than its **proof**. After all, before you can prove a theorem, you need to understand what it states, and to understand what a theorem is about, you need to know the definitions of the objects involved. Whenever a proof of a theorem is at least somewhat complicated, I provide a "proof idea." Feel free to skip the actual proof in a first reading, focusing only on the proof idea.

This book contains some code snippets, but this is by no means a programming course. You don't need to know how to program to follow this material. The reason we use code is that it is a *precise* way to describe computation. Particular implementation details are not as important to us, and so we will emphasize code readability at the expense of considerations such as error handling, encapsulation, etc. that can be extremely important for real-world programming.

0.1.1 Is the effort worth it?

This is not an easy book, and you might reasonably wonder why should you spend the effort in learning this material. A traditional

justification for a "Theory of Computation" course is that you might encounter these concepts later on in your career. Perhaps you will come across a hard problem and realize it is NP complete, or find a need to use what you learned about regular expressions. This might very well be true, but the main benefit of this course is not in teaching you any practical tool or technique, but instead in giving you a *different way of thinking*: an ability to recognize computational phenomena even when they occur in non-obvious settings, a way to model computational tasks and questions, and to reason about them.

Regardless of any use you will derive from this book. I believe learning this material is important because it contains concepts that are both beautiful and fundamental. The role that *energy* and *matter* played in the 20th century is played in the 21st by *computation* and *information*, not just as tools for our technology and economy, but also as the basic building blocks we use to understand the world. This course will give you a taste of some of the theory behind those, and hopefully spark your curiosity to study more.

0.2 TO POTENTIAL INSTRUCTORS

I wrote this book for my Harvard course, but I hope that other lecturers will find it useful as well. To some extent, it is similar in content to "Theory of Computation" or "Great Ideas" courses such as those taught at CMU or MIT.

The most significant difference between our approach and more traditional ones (such as Hopcroft and Ullman's [HU69; HU79] and Sipser's [Sip97]) is that we do not start with *finite automata* as our initial computational model. Instead, our initial computational model is *Boolean Circuits*.[1] We believe that Boolean Circuits are more fundamental to the theory of computing (and even its practice!) than automata. In particular, Boolean Circuits are a prerequisite for many concepts that one would want to teach in a modern course on Theoretical Computer Science, including cryptography, quantum computing, derandomization, attempts at proving $\mathbf{P} \neq \mathbf{NP}$, and more. Even in cases where Boolean Circuits are not strictly required, they can often offer significant simplifications (as in the case of the proof of the Cook-Levin Theorem).

[1] An earlier book that starts with circuits as the initial model is John Savage's [Sav98].

Furthermore, I believe there are pedagogical reasons to start with Boolean circuits as opposed to finite automata. Boolean circuits are a more natural model of computation, and one that corresponds more closely to computing in Silicon, making the connection to practice more immediate to the students. Finite functions are arguably easier to grasp than infinite ones, as we can fully write down their truth table. The theorem that *every* finite function can be computed by some Boolean circuit is both simple enough and important enough to serve

as an excellent starting point for this course. Moreover, many of the main conceptual points of the theory of computation, including the notions of the duality between *code* and *data*, and the idea of *universality*, can already be seen in this context.

After Boolean circuits, we move on to Turing machines and prove results such as the existence of a universal Turing machine, the uncomputability of the halting problem, and Rice's Theorem. Automata are discussed after we see Turing machines and undecidability, as an example for a *restricted computational model* where problems such as determining halting can be effectively solved.

While this is not our motivation, the order we present circuits, Turing machines, and automata roughly corresponds to the chronological order of their discovery. Boolean algebra goes back to Boole's and DeMorgan's works in the 1840s [Boo47; De 47] (though the definition of Boolean circuits and the connection to physical computation was given 90 years later by Shannon [Sha38]). Alan Turing defined what we now call "Turing Machines" in the 1930s [Tur37], while finite automata were introduced in the 1943 work of McCulloch and Pitts [MP43] but only really understood in the seminal 1959 work of Rabin and Scott [RS59].

More importantly, while models such as finite-state machines, regular expressions, and context-free grammars are incredibly important for practice, the main applications for these models (whether it is for parsing, for analyzing properties such as *liveness* and *safety*, or even for software defined routing tables) rely crucially on the fact that these are *tractable* models for which we can effectively answer *semantic questions*. This practical motivation can be better appreciated *after* students see the undecidability of semantic properties of general computing models.

The fact that we start with circuits makes proving the Cook-Levin Theorem much easier. In fact, our proof of this theorem can (and is) done using a handful of lines of Python. Combining this proof with the standard reductions (which are also implemented in Python) allows students to appreciate visually how a question about computation can be mapped into a question about (for example) the existence of an independent set in a graph.

Some other differences between this book and previous texts are the following:

1. For measuring *time complexity*, we use the standard RAM machine model used (implicitly) in algorithms courses, rather than Turing machines. While these two models are of course polynomially equivalent, and hence make no difference for the definitions of the classes **P**, **NP**, and **EXP**, our choice makes the distinction between

notions such as $O(n)$ or $O(n^2)$ time more meaningful. This choice also ensures that these finer-grained time complexity classes correspond to the informal definitions of linear and quadratic time that students encounter in their algorithms lectures (or their whiteboard coding interviews..).

2. We use the terminology of *functions* rather than *languages*. That is, rather than saying that a Turing Machine M *decides a language* $L \subseteq \{0,1\}^*$, we say that it *computes a function* $F : \{0,1\}^* \to \{0,1\}$. The terminology of "languages" arises from Chomsky's work [Cho56], but it is often more confusing than illuminating. The language terminology also makes it cumbersome to discuss concepts such as algorithms that compute functions with more than one bit of output (including basic tasks such as addition, multiplication, etc..). The fact that we use functions rather than languages means we have to be extra vigilant about students distinguishing between the *specification* of a computational task (e.g., the *function*) and its *implementation* (e.g., the *program*). On the other hand, this point is so important that it is worth repeatedly emphasizing and drilling into the students, regardless of the notation used. The book does mention the language terminology and reminds of it occasionally, to make it easier for students to consult outside resources.

Reducing the time dedicated to finite automata and context-free languages allows instructors to spend more time on topics that a modern course in the theory of computing needs to touch upon. These include randomness and computation, the interactions between *proofs* and *programs* (including Gödel's incompleteness theorem, interactive proof systems, and even a bit on the λ-calculus and the Curry-Howard correspondence), cryptography, and quantum computing.

This book contains sufficient detail to enable its use for self-study. Toward that end, every chapter starts with a list of learning objectives, ends with a recap, and is peppered with "pause boxes" which encourage students to stop and work out an argument or make sure they understand a definition before continuing further.

Section 0.4 contains a "roadmap" for this book, with descriptions of the different chapters, as well as the dependency structure between them. This can help in planning a course based on this book.

0.3 ACKNOWLEDGEMENTS

This text is continually evolving, and I am getting input from many people, for which I am deeply grateful. Thanks to Scott Aaronson, Michele Amoretti, Marguerite Basta, Sam Benkelman, Jarosław Błasiok, Emily Chan, Christy Cheng, Michelle Chiang, Daniel Chiu, Chi-Ning Chou, Michael Colavita, Robert Darley Waddilove, Juan Esteller,

Leor Fishman, William Fu, Piotr Galuszka, Mark Goldstein, Alexander Golovnev, Chan Kang, Nina Katz-Christy, Estefania Lahera, Allison Lee, Ondřej Lengál, Raymond Lin, Emma Ling, Alex Lombardi, Lisa Lu, Aditya Mahadevan, Jacob Meyerson, George Moe, Hamish Nicholson, Sandip Nirmel, Sebastian Oberhoff, Thomas Orton, Pablo Parrilo, Juan Perdomo, Aaron Sachs, Abdelrhman Saleh, Brian Sapozhnikov, Peter Schäfer, Josh Seides, Alaisha Sharma, Noah Singer, Matthew Smedberg, Hikari Sorensen, Alec Sun, Everett Sussman, Marika Swanberg, Garrett Tanzer, Sarah Turnill, Salil Vadhan, Patrick Watts, Ryan Williams, Licheng Xu, Wanqian Yang, Elizabeth Yeoh-Wang, Josh Zelinsky, Grace Zhang, and Jessica Zhu for helpful feedback.

If you have any comments or suggestions, please do post them on the GitHub repository https://github.com/boazbk/tcs.

Salil Vadhan co-taught with me the first iteration of this course and gave me a tremendous amount of useful feedback and insights during this process. Michele Amoretti and Marika Swanberg carefully read several chapters of this text and gave extremely helpful detailed comments.

Thanks to Anil Ada, Venkat Guruswami, and Ryan O'Donnell for helpful tips from their experience in teaching CMU 15-251. Juan Esteller and Gabe Montague initially implemented the NAND* programming languages in OCaml and Javascript.

I am using many open source software packages in the production of these notes for which I am grateful. In particular, I am thankful to Donald Knuth and Leslie Lamport for LaTeX and to John MacFarlane for Pandoc. David Steurer wrote the original scripts to produce this text. The current version uses Sergio Correia's panflute. The templates for the LaTeX and HTML versions are derived from Tufte LaTeX, Gitbook and Bookdown. Thanks to Amy Hendrickson for some LaTeX consulting. I used the Jupyter project to write the supplemental code snippets.

Finally, I would like to thank my family: my wife Ravit, and my children Alma and Goren. Working on this book (and the corresponding course) took so much of my time that Alma wrote an essay for her fifth-grade class saying that "universities should not pressure professors to work too much." I'm afraid all I have to show for this effort is 600 pages of ultra-boring mathematical text.

PRELIMINARIES

Learning Objectives:

- **Introduce and motivate the study of computation for its own sake, irrespective of particular implementations.**
- **The notion of an *algorithm* and some of its history.**
- **Algorithms as not just *tools*, but also *ways of thinking and understanding*.**
- **Taste of Big-O analysis and the surprising creativity in the design of efficient algorithms.**

0
Introduction

> *"Computer Science is no more about computers than astronomy is about telescopes"*, attributed to Edsger Dijkstra. [1]

> *"Hackers need to understand the theory of computation about as much as painters need to understand paint chemistry."* , Paul Graham 2003. [2]

> *"The subject of my talk is perhaps most directly indicated by simply asking two questions: first, is it harder to multiply than to add? and second, why?...I (would like to) show that there is no algorithm for multiplication computationally as simple as that for addition, and this proves something of a stumbling block."*, Alan Cobham, 1964

The origin of much of science and medicine can be traced back to the ancient Babylonians. However, the Babylonians' most significant contribution to humanity was arguably the invention of the *place-value number system*. The place-value system represents any number using a collection of digits, whereby the *position* of the digit is used to determine its value, as opposed to a system such as Roman numerals, where every symbol has a fixed numerical value regardless of position. For example, the distance to the moon is 238,900 of our miles or 259,956 Roman miles. The latter quantity, expressed in standard Roman numerals is

MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMM

[1] This quote is typically read as disparaging the importance of actual physical computers in Computer Science, but note that telescopes are absolutely essential to astronomy as they provide us with the means to connect theoretical predictions with actual experimental observations.

[2] To be fair, in the following sentence Graham says "you need to know how to calculate time and space complexity and about Turing completeness". This book includes these topics, as well as others such as NP-hardness, randomization, cryptography, quantum computing, and more.

```
MMMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMDCCCCLVI
```

Writing the distance to the sun in Roman numerals would require about 100,000 symbols: a 50-page book just containing this single number!

The above means that for someone who thinks of numbers in an additive system like Roman numerals, quantities like the distance to the moon or sun are not merely large- they are *unspeakable*: cannot be expressed or even grasped. It's no wonder that Eratosthenes, who was the first person to calculate the earth's diameter (up to about ten percent error) and Hipparchus who was the first to calculate the distance to the moon, did not use a Roman-numeral type system but rather the Babylonian sexadecimal (i.e., base 60) place-value system.

The Babylonians also invented the precursors of the "standard algorithms" that we were all taught in elementary school for adding and multiplying numbers.[3] These algorithms and their variants have been of course essential to people throughout history working with abaci, papyrus, or pencil and paper, but in our computer age, do they still serve any purpose beyond torturing third graders?

[3] For more on the actual algorithms the Babylonians used, see Knuth's paper and Neugebauer's classic book.

To answer this question, let us try to see in what sense is the standard digit by digit multiplication algorithm "better" than the straightforward implementation of multiplication as iterated addition. Let's start by more formally describing both algorithms:

Algorithm 0.1 — Multiplication via repeated addition.
Input: Non-negative integers x, y
Operation:

1. Let $result \leftarrow 0$.
2. For $i = 1, \ldots, y$: set $result \leftarrow result + x$
3. Output $result$

Algorithm 0.2 — Grade-school multiplication.
Input: Non-negative integers x, y.
Operation:

1. Let n be number of digits of y, and set $result \leftarrow 0$.
2. For $i = 0, \ldots, n-1$: set $result \leftarrow result + 10^i \times y_i \times x$, where y_i is the i-th digit of y (i.e. $y = 10^0 y_0 + 10^1 y_1 + \cdots + y_{n-1} 10^{n-1}$)
3. Output $result$.

Both algorithms assume that we already know how to add numbers, and the second one also assumes that we can multiply a number by a power of 10 (which is, after all, a simple shift) as well as multiply by a single-digit (which like addition, is done by multiplying each digit and propagating carries).

Suppose that x and y are two numbers of n decimal digits each. Adding two such numbers takes at least n single-digit additions (depending on how many times we need to use a "carry"), and so adding x to itself y times as per Algorithm 0.1 will take at least $n \cdot y$ single-digit additions. In contrast, the grade-school algorithm (i.e., Algorithm 0.2) reduces this problem to taking n products of x with a single-digit (which require up to $2n$ single-digit operations each, depending on carries) and then adding all of those together (total of n additions, which, again depending on carries, would cost at most $2n^2$ single-digit operations) for a total of at most $4n^2$ single-digit operations. How much faster would $4n^2$ operations be than $n \cdot y$? Also, would this make any difference in a modern computer?

Let us consider the case of multiplying 64-bit or 20-digit numbers.[4] That is the task of multiplying two numbers x, y that are between 10^{19} and 10^{20}. Since in this case $n = 20$, the grade-school algorithm (Algorithm 0.2) would use at most $4n^2 = 1600$ single-digit operations, while repeated addition (Algorithm 0.1) would require at least $n \cdot y \geq 20 \cdot 10^{19}$ single-digit operations. To understand the difference, consider that a human being can perform a single-digit operation in about 2 seconds, requiring just under an hour to complete the calculation of $x \times y$ using the grade-school algorithm. In contrast, even though it is more than a billion times faster than a human, a modern PC that computes $x \times y$ using naïve iterated addition would require about $10^{20}/10^9 = 10^{11}$ seconds (which is more than three millennia!) to compute the same result.

[4] This is a typical size in several programming languages; for example the `long` data type in the Java programming language, and (depending on architecture) the `long` or `long long` types in C.



Remark 0.3 — Value vs. length of a number.. It is important to distinguish between the *value* of a number, and the *length of its representation* (i.e., the number of digits it has). There is a big difference between the two: having 1,000,000,000 dollars is not the same as having 10 dollars! When talking about running time of algorithms, "less is more", and so an algorithm that runs in time proportional to the *number of digits* of an input number (or even the number of digit squared) is much preferred to an algorithm that runs in time proportional to the *value* of the input number.

We see that computers have not made algorithms obsolete. On the contrary, the vast increase in our ability to measure, store, and

communicate data has led to much higher demand for developing better and more sophisticated algorithms that can allow us to make better decisions based on these data. We also see that in no small extent the notion of *algorithm* is independent of the actual computing device that executes it. The digit-by-digit multiplication algorithm is vastly better than iterated addition, regardless whether the technology we use to implement it is a silicon-based chip, or a third grader with pen and paper.

Theoretical computer science is concerned with the *inherent* properties of algorithms and computation; namely, those properties that are *independent* of current technology. We ask some questions that were already pondered by the Babylonians, such as "what is the best way to multiply two numbers?", but also questions that rely on cutting-edge science such as "could we use the effects of quantum entanglement to factor numbers faster?".

In Computer Science parlance, a scheme such as the decimal (or sexadecimal) positional representation for numbers is known as a *data structure*, while operations on such representations are known as *algorithms*. Data structures and algorithms have enabled amazing applications that have transformed human society, but their importance goes beyond their practical utility. Structures from computer science, such as bits, strings, graphs, and even the notion of a program itself, as well as concepts such as universality and replication, have not just found (many) practical uses but contributed a new language and a new way to view the world.

0.1 EXTENDED EXAMPLE: A FASTER WAY TO MULTIPLY

Once you think of the standard digit-by-digit multiplication algorithm, it seems that it is clearly the "right" way to multiply numbers. Indeed, in 1960, the famous mathematician Andrey Kolmogorov organized a seminar at Moscow State University in which he conjectured that every algorithm for multiplying two n digit numbers would require a number of basic operations that is proportional to n^2.[5] In other words, Kolmogorov conjectured that in any multiplication algorithm, doubling the number of digits would *quadruple* the number of basic operations required.

A young student named Anatoly Karatsuba was in the audience, and within a week he found an algorithm that requires only about $Cn^{1.6}$ operations for some constant C. Such a number becomes much smaller than n^2 as n grows.[6] Amazingly, Karatsuba's algorithm is based on a faster way to multiply *two-digit* numbers.

Suppose that $x, y \in [100] = \{0, \ldots, 99\}$ are a pair of two-digit numbers. Let's write $\overline{x}$ for the "tens" digit of x, and $\underline{x}$ for the "ones" digit, so that $x = 10\overline{x} + \underline{x}$, and write similarly $y = 10\overline{y} + \underline{y}$ for $\overline{y}, \underline{y} \in [10]$.

[5] That is, he conjectured that the number of operations would be at least some n^2/C operations for some constant C or, using "Big-O notation", $\Omega(n^2)$ operations. See the *mathematical background* chapter for a precise definition of Big-O notation.

[6] At the time of this writing, the standard Python multiplication implementation switches from the elementary school algorithm to Karatsuba's algorithm when multiplying numbers larger than 1000 bits long.

The grade-school algorithm for multiplying x and y is illustrated in Fig. 1.

The grade-school algorithm works by transforming the task of multiplying a pair of two-digit number into *four* single-digit multiplications via the formula

$$(10\overline{x}+\underline{x}) \times (10\overline{y}+\underline{y}) = 100\overline{x}\overline{y} + 10(\overline{x}\underline{y} + \underline{x}\overline{y}) + \underline{x}\underline{y} \quad (1)$$

Karatsuba's algorithm is based on the observation that we can express Eq. (1) also as

$$(10\overline{x}+\underline{x})\times(10\overline{y}+\underline{y}) = (100-10)\overline{x}\overline{y}+10\left[(\overline{x}+\underline{x})(\overline{y}+\underline{y})\right]-(10-1)\underline{x}\underline{y} \quad (2)$$

which reduces multiplying the two-digit number x and y to computing the following three "simple" products: $\overline{x}\overline{y}$, $\underline{x}\underline{y}$ and $(\overline{x}+\underline{x})(\overline{y}+\underline{y})$.[7]

Of course, if all we wanted was to multiply two digit numbers, we wouldn't need any clever algorithms. It turns out that we can repeatedly apply the same idea, and use them to multiply 4-digit numbers, 8-digit numbers, 16-digit numbers, and so on and so forth. If we used the grade-school approach, then our cost for doubling the number of digits would be to *quadruple* the number of multiplications, which for $n = 2^\ell$ digits would result in about $4^\ell = n^2$ operations. In contrast, in Karatsuba's approach doubling the number of digits only *triples* the number of operations, which means that for $n = 2^\ell$ digits we require about $3^\ell = n^{\log_2 3} \sim n^{1.58}$ operations.

Specifically, we use a *recursive* strategy as follows:

Algorithm 0.4 — Karatsuba multiplication.
Input: nonnegative integers x, y each of at most n digits
Operation:

1. If $n \leq 2$ then return $x \cdot y$ (using a constant number of single-digit multiplications).
2. Otherwise, let $m = \lfloor n/2 \rfloor$, and write $x = 10^m\overline{x} + \underline{x}$ and $y = 10^m\overline{y} + \underline{y}$. [8]
3. Use *recursion* to compute $A = \overline{x}\overline{y}$, $B = \underline{x}\underline{y}$ and $C = (\overline{x} + \underline{x})(\overline{y}+\underline{y})$. Note that all the numbers will have at most $m+1$ digits.
4. Return $(10^n - 10^m) \cdot A + 10^m \cdot B + (1 - 10^m) \cdot C$

To understand why the output will be correct, first note that for $n > 2$, it will always hold that $m < n - 1$, and hence the recursive calls will always be for multiplying numbers with a smaller number of digits, and (since eventually we will get to single or double digit numbers)

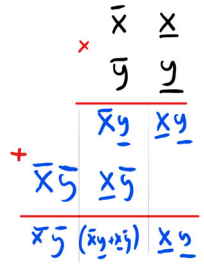


Figure 1: The grade-school multiplication algorithm illustrated for multiplying $x = 10\overline{x} + \underline{x}$ and $y = 10\overline{y} + \underline{y}$. It uses the formula $(10\overline{x} + \underline{x}) \times (10\overline{y} + \underline{y}) = 100\overline{x}\overline{y} + 10(\overline{x}\underline{y} + \underline{x}\overline{y}) + \underline{x}\underline{y}$.

[7] The term $(\overline{x} + \underline{x})(\overline{y} + \underline{y})$ is not exactly a single-digit multiplication as $\overline{x} + \underline{x}$ and $\overline{y} + \underline{y}$ are numbers between 0 and 18 and not between 0 and 9. As we'll see, it turns out that this does not make much of a difference, since when we use this algorithm recursively on n-digit numbers, this term will have at most $\lceil n/2 \rceil + 1$ digits, which is essentially half the number of digits as the original input.

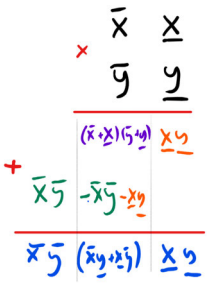


Figure 2: Karatsuba's multiplication algorithm illustrated for multiplying $x = 10\overline{x} + \underline{x}$ and $y = 10\overline{y} + \underline{y}$. We compute the three orange, green and purple products $\underline{x}\underline{y}$, $\overline{x}\overline{y}$ and $(\overline{x} + \underline{x})(\overline{y} + \underline{y})$ and then add and subtract them to obtain the result.

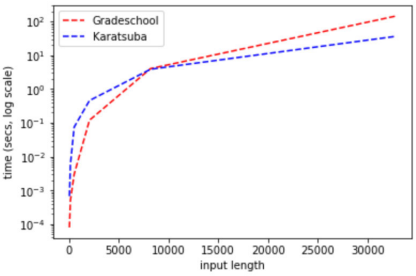


Figure 3: Running time of Karatsuba's algorithm vs. the grade-school algorithm. (Python implementation available online.) Note the existence of a "cutoff" length, where for sufficiently large inputs Karatsuba becomes more efficient than the grade-school algorithm. The precise cutoff location varies by implementation and platform details, but will always occur eventually.

the algorithm will indeed terminate. Now, since $x = 10^m\overline{x} + \underline{x}$ and $y = 10^m\overline{y} + \underline{y}$,

$$x \times y = 10^n\overline{x} \cdot \overline{y} + 10^m(\overline{xy} + \underline{xy}) + \underline{xy} \; . \tag{3}$$

Rearranging the terms we see that

$$x \times y = 10^n\overline{x} \cdot \overline{y} + 10^m \left[(\overline{x} + \underline{x})(\overline{y} + \underline{y}) - \underline{xy} - \overline{xy}\right] + \underline{xy} \; , \tag{4}$$

which equals $(10^n - 10^m) \cdot A + 10^m \cdot B + (1 - 10^m) \cdot C$, the value returned by the algorithm.

The key observation is that (4) reduces the task of computing the product of two n-digit numbers to computing *three* products of m-digit numbers where $m = \lfloor n/2 \rfloor + 1$ is roughly equal to $n/2$ (in fact, m is always at most $n/2 + 1$). Specifically, we can compute $x \times y$ from the three products $\overline{xy}$, $\underline{xy}$ and $(\overline{x} + \underline{x})(\overline{y} + \underline{y}))$, using a constant number (in fact eight) of additions, subtractions, and multiplications by 10^n or $10^{\lfloor n/2 \rfloor}$. (Multiplication by a power of ten can be done very efficiently as it corresponds to simply shifting the digits.) Intuitively this means that as the number of digits *doubles*, the cost of performing a multiplication via Karatsuba's algorithm *triples* instead of quadrupling, as happens in the naive algorithm.

In particular, by iteratively repeating this reasoning, we can see that multiplying numbers of $n = 2^\ell$ digits via Karatsuba's Algorithm will cost about $3^\ell = n^{\log_2 3} \sim n^{1.585}$ operations. Exercise 0.4 shows that the number of single-digit operations that Karatsuba's algorithm uses for multiplying n digit integers is at most $O(n^{\log_2 3})$ (see also Fig. 2).

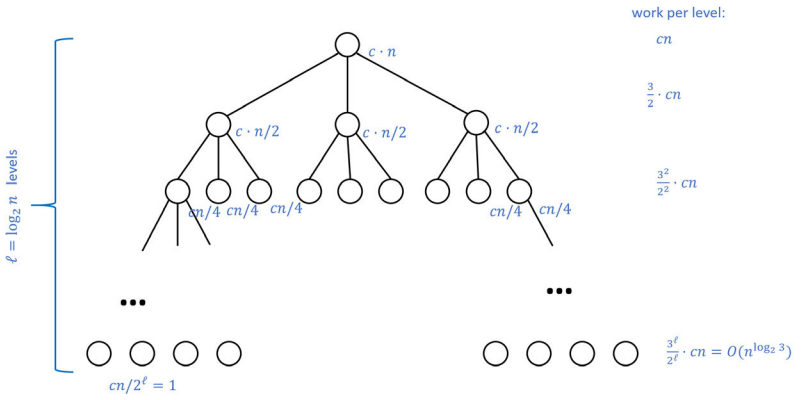


Figure 4: Karatsuba's algorithm reduces an n-bit multiplication to three $n/2$-bit multiplications, which in turn are reduced to nine $n/4$-bit multiplications and so on. We can represent the computational cost of all these multiplications in a 3-ary tree of depth $\log_2 n$, where at the root the extra cost is cn operations, at the first level the extra cost is $c(n/2)$ operations, and at each of the 3^i nodes of level i, the extra cost is $c(n/2^i)$. The total cost is $cn\sum_{i=0}^{\log_2 n}(3/2)^i \leq 10cn^{\log_2 3}$ by the formula for summing a geometric series.

[8] Recall that for a number x, $\lfloor x \rfloor$ is obtained by "rounding down" x to the largest integer smaller or equal to x.

R

Remark 0.5 — Ceilings, floors, and rounding. One of the benefits of using Big-O notation is that we can allow ourselves to be a little looser with issues such as rounding numbers. For example, the natural way to describe Karatsuba's algorithm's running time is via the following recursive equation

$$T(n) = 3T(n/2) + O(n) \ . \tag{5}$$

But if n is not even then we cannot recursively invoke the algorithm on $n/2$-digit integers. Rather, the true recursive equation is

$$T(n) = 3T(\lfloor n/2 \rfloor + 1) + O(n) \ . \tag{6}$$

However, you can verify that the difference between (5) and (6) is insignificant when we are only concerned with "Big Oh analysis". These kinds of tricks work not just in the context of multiplication algorithms but in many other cases as well. Thus most of the time we can safely ignore "rounding issues" in the lengths of the input in the analysis of algorithms.

0.1.1 Beyond Karatsuba's algorithm

Toom and Cook extended in the ideas of Karatsuba to get even faster multiplication algorithms, but this was not the end of the line. In 1971, Schönhage and Strassen improved on the Toom-Cook algorithm using the *Fast Fourier Transform*; their idea was to somehow treat integers as "signals" and do the multiplication more efficiently by moving to the Fourier domain.[9] The latest asymptotic improvement was given by Fürer in 2007 (though it only starts beating the Schönhage-Strassen algorithm for truly astronomical numbers). Yet, despite all this progress, we still don't know whether or not there is an $O(n)$ time algorithm for multiplying two n digit numbers!

[9] The *Fourier transform* is a central tool in mathematics and engineering, used in a great many applications. If you have not seen it yet, you will hopefully encounter it at some point in your studies.

R

Remark 0.6 — Matrix Multiplication (advanced note). (This book contains many "advanced" or "optional" notes and sections. These may assume background that not every student has, and can be safely skipped over as none of the future parts depends on them.)

Ideas similar to Karatsuba's can be used to speed up *matrix* multiplications as well. Matrices are a powerful way to represent linear equations and operations, widely used in a great many applications of scientific computing, graphics, machine learning, and many many more.

One of the basic operations one can do with two matrices is to *multiply* them. For example, if $x = \begin{pmatrix} x_{0,0} & x_{0,1} \\ x_{1,0} & x_{1,1} \end{pmatrix}$ and $y = \begin{pmatrix} y_{0,0} & y_{0,1} \\ y_{1,0} & y_{1,1} \end{pmatrix}$ then the product of x and y is the matrix $\begin{pmatrix} x_{0,0}y_{0,0} + x_{0,1}y_{1,0} & x_{0,0}y_{0,1} + x_{0,1}y_{1,1} \\ x_{1,0}y_{0,0} + x_{1,1}y_{1,0} & x_{1,0}y_{0,1} + x_{1,1}y_{1,1} \end{pmatrix}$. You can see that we can compute this matrix by *eight* products of numbers.

Now suppose that n is even and x and y are a pair of $n \times n$ matrices which we can think of as each composed of four $(n/2) \times (n/2)$ blocks $x_{0,0}, x_{0,1}, x_{1,0}, x_{1,1}$ and $y_{0,0}, y_{0,1}, y_{1,0}, y_{1,1}$. Then the formula for the matrix product of x and y can be expressed in the same way as above, just replacing products $x_{a,b}y_{c,d}$ with *matrix* products, and addition with matrix addition. This means that we can use the formula above to give an algorithm that *doubles* the dimension of the matrices at the expense of increasing the number of operation by a factor of 8, which for $n = 2^\ell$ results in $8^\ell = n^3$ operations.

In 1969 Volker Strassen noted that we can compute the product of a pair of two-by-two matrices using only *seven* products of numbers by observing that each entry of the matrix xy can be computed by adding and subtracting the following seven terms: $t_1 = (x_{0,0} + x_{1,1})(y_{0,0} + y_{1,1})$, $t_2 = (x_{0,0} + x_{1,1})y_{0,0}$, $t_3 = x_{0,0}(y_{0,1} - y_{1,1})$, $t_4 = x_{1,1}(y_{0,1} - y_{0,0})$, $t_5 = (x_{0,0} + x_{0,1})y_{1,1}$, $t_6 = (x_{1,0} - x_{0,0})(y_{0,0} + y_{0,1})$, $t_7 = (x_{0,1} - x_{1,1})(y_{1,0} + y_{1,1})$. Indeed, one can verify that $xy = \begin{pmatrix} t_1 + t_4 - t_5 + t_7 & t_3 + t_5 \\ t_2 + t_4 & t_1 + t_3 - t_2 + t_6 \end{pmatrix}$.

Using this observation, we can obtain an algorithm such that doubling the dimension of the matrices results in increasing the number of operations by a factor of 7, which means that for $n = 2^\ell$ the cost is $7^\ell = n^{\log_2 7} \sim n^{2.807}$. A long sequence of work has since improved this algorithm, and the current record has running time about $O(n^{2.373})$. However, unlike the case of integer multiplication, at the moment we don't know of any algorithm for matrix multiplication that runs in time linear or even close to linear in the size of the input matrices (e.g., an $O(n^2 polylog(n))$ time algorithm). People have tried to use group representations, which can be thought of as generalizations of the Fourier transform, to obtain faster algorithms, but this effort has not yet succeeded.

0.2 ALGORITHMS BEYOND ARITHMETIC

The quest for better algorithms is by no means restricted to arithmetical tasks such as adding, multiplying or solving equations. Many *graph algorithms*, including algorithms for finding paths, matchings, spanning trees, cuts, and flows, have been discovered in the last several decades, and this is still an intensive area of research. (For example, the last few years saw many advances in algorithms for the *maximum flow* problem, borne out of unexpected connections with electrical circuits and linear equation solvers.) These algorithms are being used not just for the "natural" applications of routing network traffic or GPS-based navigation, but also for applications as varied as drug discovery through searching for structures in gene-interaction graphs to computing risks from correlations in financial investments.

Google was founded based on the *PageRank* algorithm, which is an efficient algorithm to approximate the "principal eigenvector" of (a dampened version of) the adjacency matrix of the web graph. The *Akamai* company was founded based on a new data structure, known as *consistent hashing*, for a hash table where buckets are stored at different servers. The *backpropagation algorithm*, which computes partial derivatives of a neural network in $O(n)$ instead of $O(n^2)$ time, underlies many of the recent phenomenal successes of learning deep neural networks. Algorithms for solving linear equations under sparsity constraints, a concept known as *compressed sensing*, have been used to drastically reduce the amount and quality of data needed to analyze MRI images. This made a critical difference for MRI imaging of cancer tumors in children, where previously doctors needed to use anesthesia to suspend breath during the MRI exam, sometimes with dire consequences.

Even for classical questions, studied through the ages, new discoveries are still being made. For example, for the question of determining whether a given integer is prime or composite, which has been studied since the days of Pythagoras, efficient probabilistic algorithms were only discovered in the 1970s, while the first deterministic polynomial-time algorithm was only found in 2002. For the related problem of actually finding the factors of a composite number, new algorithms were found in the 1980s, and (as we'll see later in this course) discoveries in the 1990s raised the tantalizing prospect of obtaining faster algorithms through the use of quantum mechanical effects.

Despite all this progress, there are still many more questions than answers in the world of algorithms. For almost all natural problems, we do not know whether the current algorithm is the "best", or whether a significantly better one is still waiting to be discovered. As

we already saw, even for the classical problem of multiplying numbers we have not yet answered the age-old question of **"is multiplication harder than addition?"** .

But at least we now know the right way to *ask* it.

0.3 ON THE IMPORTANCE OF NEGATIVE RESULTS.

Finding better multiplication algorithms is undoubtedly a worthwhile endeavor. But why is it important to prove that such algorithms *don't* exist? What useful applications could possibly arise from an impossibility result?

One motivation is pure intellectual curiosity. After all, this is a question even Archimedes could have been excited about. Another reason to study impossibility results is that they correspond to the fundamental limits of our world. In other words, they are *laws of nature*. In physics, the impossibility of building a *perpetual motion machine* corresponds to the *law of conservation of energy*. The impossibility of building a heat engine beating Carnot's bound corresponds to the second law of thermodynamics, while the impossibility of faster-than-light information transmission is a cornerstone of special relativity.

In mathematics, while we all learned the solution for quadratic equations in high school, the impossibility of generalizing this to equations of degree five or more gave birth to *group theory*. Another example of an impossibility result comes from geometry. For two millennia, mathematicians tried to show that Euclid's fifth axiom or "postulate" could be derived from the first four. (This fifth postulate was known as the "parallel postulate", and roughly speaking it states that every line has a unique parallel line of each distance.) It was shown to be impossible using constructions of so called "non-Euclidean geometries", which turn out to be crucial for the theory of general relativity.[10]

In an analogous way, impossibility results for computation correspond to "computational laws of nature" that tell us about the fundamental limits of any information processing apparatus, whether based on silicon, neurons, or quantum particles.[11] Moreover, computer scientists have recently been finding creative approaches to *apply* computational limitations to achieve certain useful tasks. For example, much of modern Internet traffic is encrypted using the RSA encryption scheme, which relies on its security on the (conjectured) impossibility of efficiently factoring large integers. More recently, the Bitcoin system uses a digital analog of the "gold standard" where, instead of using a precious metal, new currency is obtained by "mining" solutions for computationally difficult problems.

[10] It is fine if you have not yet encountered many of the above examples of impossiblity results. I hope however that they spark your curiosity! See the "Biliographical Notes" section (Section 0.6) for some references.

[11] Indeed, some exciting recent research is focused on trying to use computational complexity to shed light on fundamental questions in physics such understanding black holes and reconciling general relativity with quantum mechanics.



Lecture Recap

- The history of algorithms goes back thousands of years; they have been essential much of human progress and these days form the basis of multi-billion dollar industries, as well as life-saving technologies.
- There is often more than one algorithm to achieve the same computational task. Finding a faster algorithm can often make a much bigger difference than improving computing hardware.
- Better algorithms and data structures don't just speed up calculations, but can yield new qualitative insights.
- One question we will study is to find out what is the *most efficient* algorithm for a given problem.
- To show that an algorithm is the most efficient one for a given problem, we need to be able to *prove* that it is *impossible* to solve the problem using a smaller amount of computational resources.

0.4 ROADMAP TO THE REST OF THIS BOOK

Often, when we try to solve a computational problem, whether it is solving a system of linear equations, finding the top eigenvector of a matrix, or trying to rank Internet search results, it is enough to use the "I know it when I see it" standard for describing algorithms. As long as we find some way to solve the problem, we are happy and don't care so much about formal descriptions of the algorithm. But when we want to answer a question such as "does there *exist* an algorithm to solve the problem P?" we need to be much more precise.

In particular, we will need to (**1**) define exactly what it means to solve P, and (**2**) define exactly what an algorithm is. Even (**1**) can sometimes be non-trivial but (**2**) is particularly challenging; it is not at all clear how (and even whether) we can encompass all potential ways to design algorithms. We will consider several simple *models of computation*, and argue that, despite their simplicity, they do capture all "reasonable" approaches to achieve computing, including all those that are currently used in modern computing devices.

Once we have these formal models of computation, we can try to obtain *impossibility results* for computational tasks, showing that some problems *can not be solved* (or perhaps can not be solved within the resources of our universe). Archimedes once said that given a fulcrum and a long enough lever, he could move the world. We will see how *reductions* allow us to leverage one hardness result into a slew of a great many others, illuminating the boundaries between

the computable and uncomputable (or tractable and intractable) problems.

Later in this course we will go back to examining our models of computation, and see how resources such as randomness or quantum entanglement could potentially change the power of our model. In the context of probabilistic algorithms, we will see a glimpse of how randomness has become an indispensable tool for understanding computation, information, and communication. We will also see how computational difficulty can be an asset rather than a hindrance, and be used for the "derandomization" of probabilistic algorithms. The same ideas also show up in *cryptography*, which has undergone not just a technological but also an intellectual revolution in the last few decades, much of it building on the foundations that we explore in this course.

Theoretical Computer Science is a vast topic, branching out and touching upon many scientific and engineering disciplines. This book provides a very partial (and biased) sample of this area. More than anything, I hope I will manage to "infect" you with at least some of my love for this field, which is inspired and enriched by the connection to practice, but is also deep and beautiful regardless of applications.

0.4.1 Dependencies between chapters

This book is divided into the following parts ,see Fig. 5.

- **Preliminaries:** Introduction, mathematical background, and representing objects as strings.
- **Part I: Finite computation:** Boolean circuits / straight-line programs. Universal gate sets, existence of a circuit for every function, representing circuits as strings and the universal circuit, and the counting lower bound.
- **Part II: Uniform computation:** Turing machines / programs with loops. Equivalence of models (including RAM machines and λ calculus), universal Turing machine, uncomputable functions the Halting problem and Rice's Theorem, Gödel's incompleteness theorem, restricted models (regular and context free languages).
- **Part III: Efficient computation:** Definition of running time, time hierarchy theorem, **P** and **NP**, **NP** completeness, space bounded computation.
- **Part IV: Randomized computation:** Probability, randomized algorithms, **BPP**, amplification, **BPP** $\subseteq \mathbf{P}_{/poly}$, pseudrandom generators and derandomization.

- **Part V: Advanced topics:** Cryptography, proofs and algorithms (interactive and zero knowledge proofs, Curry-Howard correspondence), quantum computing.

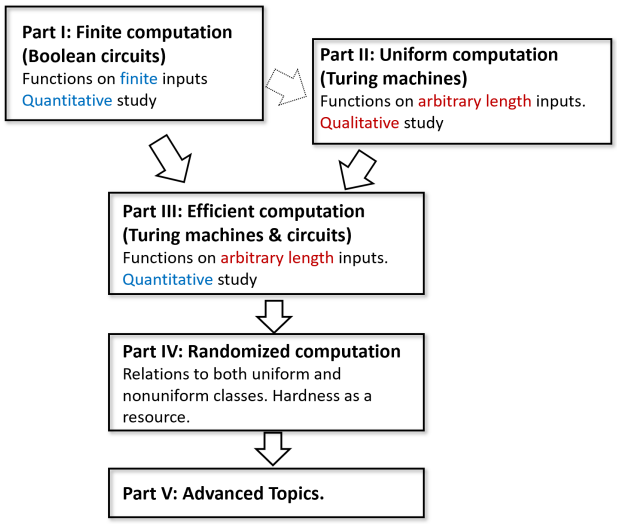


Figure 5: The dependency structure of the different parts. Part I introduces the model of Boolean circuits to study *finite functions* with an emphasis on *quantitative* questions (how many gates to compute a function). Part II introduces the model of Turing machines to study functions that have *unbounded input lengths* with an emphasis on *qualitative* questions (is this function computable or not). Much of Part II does not depend on Part I, as Turing machines can be used as the first computational model. Part III depends on both parts as it introduces a *quantitative* study of functions with unbounded input length. The more advancec parts IV (randomized computation) and V (advanced topics) rely on the material of Parts I, II and III.

The book largely proceeds in linear order, with each chapter building on the previous ones, with the following exceptions:

- Part II (Uniform Computation) does not have a strong dependency on Part I (Finite computation) and it should be possible to teach them in the reverse order.
- All chapters in Part V (Advanced topics) are independent of one another and can be covered in any order.
- Chapter 10 (Gödel's incompleteness theorem), Chapter 9 (Restricted computational models), and Chapter 16 (Space bounded computation), are not used in following chapters. Hence you can choose whether to cover or skip any of them them.

A course based on this book can use all of Parts I, II, and III (possibly skipping over some or all of Chapter 10, Chapter 9 or Chapter 16), and then either cover all or some of Part IV (randomized computation), and add a "sprinkling" of advanced topics from Part V based on student or instructor interest.

0.5 EXERCISES

Exercise 0.1 Rank the significance of the following inventions in speeding up multiplication of large (that is 100-digit or more) numbers. That is, use "back of the envelope" estimates to order them in terms of the speedup factor they offered over the previous state of affairs.

a. Discovery of the grade-school digit by digit algorithm (improving upon repeated addition)

b. Discovery of Karatsuba's algorithm (improving upon the digit by digit algorithm)

c. Invention of modern electronic computers (improving upon calculations with pen and paper).

■

Exercise 0.2 The 1977 Apple II personal computer had a processor speed of 1.023 Mhz or about 10^6 operations per seconds. At the time of this writing the world's fastest supercomputer performs 93 "petaflops" (10^{15} floating point operations per second) or about 10^{18} basic steps per second. For each one of the following running times (as a function of the input length n), compute for both computers how large an input they could handle in a week of computation, if they run an algorithm that has this running time:

a. n operations.

b. n^2 operations.

c. $n \log n$ operations.

d. 2^n operations.

e. $n!$ operations.

■

Exercise 0.3 — Usefulness of algorithmic non-existence. In this chapter we mentioned several companies that were founded based on the discovery of new algorithms. Can you give an example for a company that was founded based on the *non existence* of an algorithm? See footnote for hint.[12]

■

Exercise 0.4 — Analysis of Karatsuba's Algorithm. a. Suppose that $T_1, T_2, T_3, \ldots$ is a sequence of numbers such that $T_2 \leq 10$ and for every n, $T_n \leq 3T_{\lfloor n/2 \rfloor + 1} + Cn$ for some $C \geq 1$. Prove that $T_n \leq 20Cn^{\log_2 3}$ for every $n > 2$.[13]

b. Prove that the number of single-digit operations that Karatsuba's algorithm takes to multiply two n digit numbers is at most $1000n^{\log_2 3}$.

■

[12] As we will see in Chapter Chapter 20, almost any company relying on cryptography needs to assume the *non existence* of certain algorithms. In particular, RSA Security was founded based on the security of the RSA cryptosystem, which presumes the *non existence* of an efficient algorithm to compute the prime factorization of large integers.

[13] **Hint:** Use a proof by induction - suppose that this is true for all n's from 1 to m and prove that this is true also for $m + 1$.

Exercise 0.5 Implement in the programming language of your choice functions `Gradeschool_multiply(x,y)` and `Karatsuba_multiply(x,y)` that take two arrays of digits `x` and `y` and return an array representing the product of `x` and `y` (where `x` is identified with the number `x[0]+10*x[1]+100*x[2]+...` etc..) using the grade-school algorithm and the Karatsuba algorithm respectively. At what number of digits does the Karatsuba algorithm beat the grade-school one?

■

Exercise 0.6 — Matrix Multiplication (optional, advanced). In this exercise, we show that if for some $\omega > 2$, we can write the product of two $k \times k$ real-valued matrices A, B using at most k^ω multiplications, then we can multiply two $n \times n$ matrices in roughly n^ω time for every large enough n.

To make this precise, we need to make some notation that is unfortunately somewhat cumbersome. Assume that there is some $k \in \mathbb{N}$ and $m \leq k^\omega$ such that for every $k \times k$ matrices A, B, C such that $C = AB$, we can write for every $i, j \in [k]$:

$$C_{i,j} = \sum_{\ell=0}^{m} \alpha_{i,j}^{\ell} f_\ell(A) g_\ell(B) \tag{7}$$

for some linear functions $f_0, \ldots, f_{m-1}, g_0, \ldots, g_{m-1} : \mathbb{R}^{n^2} \to \mathbb{R}$ and coefficients $\{\alpha_{i,j}^{\ell}\}_{i,j \in [k], \ell \in [m]}$. Prove that under this assumption for every $\epsilon > 0$, if n is sufficiently large, then there is an algorithm that computes the product of two $n \times n$ matrices using at most $O(n^{\omega+\epsilon})$ arithmetic operations.[14]

■

[14] *Hint:* Start by showing this for the case that $n = k^t$ for some natural number t, in which case you can do so recursively by breaking the matrices into $k \times k$ blocks.

0.6 BIBLIOGRAPHICAL NOTES

For a brief overview of what we'll see in this book, you could do far worse than read Bernard Chazelle's wonderful essay on the Algorithm as an Idiom of modern science. The book of Moore and Mertens [MM11] gives a wonderful and comprehensive overview of the theory of computation, including much of the content discussed in this chapter and the rest of this book. Aaronson's book [Aar13] is another great read that touches upon many of the same themes.

Many of the algorithms we mention in this chapter are covered in algorithms textbooks such as those by Cormen, Leiserson, Rivert, and Stein [Cor+09], Kleinberg and Tardos [KT06], and Dasgupta, Papadimitriou and Vazirani [DPV08].

The story of Karatsuba's discovery of his multiplication algorithm is recounted by him in [Kar95]. As mentioned above, further improvements were made by Toom and Cook [Too63; Coo66], Schönhage

and Strassen [SS71], and Fürer [Für07]. These last papers crucially rely on the *Fast Fourier transform* algorithm. The interesting of the (re)discovery of this algorithm by John Tukey in the context of the cold war is recounted in [Coo87]. (We say re-discovery because it later turned out that the algorithm dates back to Gauss [HJB85].) The Fast Fourier Transform is covered in some of the books mentioned below, and there are also online available lectures such as Jeff Erickson's. See also this popular article by David Austin. Fast *matrix* multiplication was discovered by Strassen [Str69], and since then this has been an active area of research. [Blä13] is a recommended self-contained survey of this area.

The *Backpropagation* algorithm for fast differentiation of neural networks was invented by Werbos [Wer74]. The *Pagerank* algorithm was invented by Larry Page and Sergey Brin [Pag+99]. It is closely related to the *HITS* algorithm of Kleinberg [Kle99]. The *Akamai* company was founded based on the *consistent hashing* data structure described in [Kar+97]. *Compressed sensing* has a long history but two foundational papers are [CRT06; Don06]. [Lus+08] gives a survey of applications of compressed sensing to MRI; see also this popular article by Ellenberg [Ell10]. The deterministic polynomial-time algorithm for testing primality was given by Agrawal, Kayal, and Saxena [AKS04].

We alluded briefly to classical impossibility results in mathematics, including the impossibility of proving Euclid's fifth postulate from the other four, impossibility of trisecting an angle with a straightedge and compass and the impossibility of solving a quintic equation via radicals. A geometric proof of the impossibility of angle trisection (one of the three geometric problems of antiquity, going back to the ancient Greeks) is given in this blog post of Tao. The book of Mario Livio [Liv05] covers some of the background and ideas behind these impossibility results.

Learning Objectives:

- Recall basic mathematical notions such as sets, functions, numbers, logical operators and quantifiers, strings, and graphs.
- Rigorously define Big-O notation.
- Proofs by induction.
- Practice with reading mathematical *definitions*, *statements*, and *proofs*.
- Transform an intuitive argument into a rigorous proof.

1
Mathematical Background

> *"When you have mastered numbers, you will in fact no longer be reading numbers, any more than you read words when reading books. You will be reading meanings."*, W. E. B. Du Bois

> *"I found that every number, which may be expressed from one to ten, surpasses the preceding by one unit: afterwards the ten is doubled or tripled ... until a hundred; then the hundred is doubled and tripled in the same manner as the units and the tens ... and so forth to the utmost limit of numeration."*, Muhammad ibn Mūsā al-Khwārizmī, 820, translation by Fredric Rosen, 1831.

In this chapter, we review some of the mathematical concepts that we will use in this course. Most of these are not very complicated, but do require some practice and exercise to get comfortable with. If you have not previously encountered some of these concepts, there are several excellent freely-available resources online that cover them, see the "Bibliographical Notes" section (Section 1.10).

1.1 A MATHEMATICIAN'S APOLOGY

Before explaining the math background, perhaps I should explain why is this course so "mathematically heavy". After all, this is supposed to be a course about *computation*; one might think we should be talking mostly about *programs*, rather than more "mathematical" objects such as *sets*, *functions*, and *graphs*, and doing more *coding* on an actual computer than writing mathematical proofs with pen and paper. So, why are we doing so much math in this course? Is it just some form of hazing? Perhaps a revenge of the "math nerds" against the "hackers"?

At the end of the day, mathematics is simply a language for modeling concepts in a precise and unambiguous way. In this course, we

will be mostly interested in the concept of *computation*. For example, we will look at questions such as *"is there an efficient algorithm to find the prime factors of a given integer?"*.[1] To even *phrase* such a question, we need to give a precise *definition* of the notion of an *algorithm*, and of what it means for an algorithm to be *efficient*. Also, if the answer to this or similar questions turns out to be *negative*, then this cannot be shown by simply writing and executing some code. After all, there is no empirical experiment that will prove the *nonexistence* of an algorithm. Thus, our only way to show this type of *negative result* is to use *mathematical proofs*. So you can see why our main tools in this course will be mathematical proofs and definitions.

[1] Actually, scientists currently do not know the answer to this question, but we will see that settling it in either direction has very interesting applications touching on areas as far apart as Internet security and quantum mechanics.

1.2 THIS CHAPTER: A READER'S MANUAL

Depending on your background, you can approach this chapter in two different ways:

- If you already have taken some proof-based courses, and are very familiar with the notions of discrete mathematics, you can take a quick look at Section 1.3 to see the main tools we will use, Section 1.8 for our notation and conventions, and then skip ahead to the rest of this book. Alternatively, you can sit back, relax, and read this chapter just to get familiar with our notation, as well as to enjoy (or not) my philosophical musings and attempts at humor. You might also want to start brushing up on *discrete probability*, which we'll use later in this course.
- If your background is not as extensive, you should lean forward, and read this chapter with a pen and paper handy, making notes and working out derivations as you go along. We cannot fit a semester-length discrete math course in a single chapter, and hence will be necessarily brief in our discussions. Thus you might want to occasionally pause to watch some discrete math lectures, read some of the resources mentioned in Section 1.10, and do some exercises to make sure you internalized the material.

1.3 A QUICK OVERVIEW OF MATHEMATICAL PREREQUISITES

The main notions we will use in this course are the following:

- **Proofs:** First and foremost, this course will involve a heavy dose of formal mathematical reasoning, which includes mathematical *definitions*, *statements*, and *proofs*.
- **Sets:** The basic set *relationships* of membership ($\in$), containment ($\subseteq$), and set *operations*, principally union, intersection, set difference and Cartesian product ($\cup, \cap, \setminus$ and $\times$).

- **Tuples and strings:** The set Σ^k of length-k strings/lists over elements in Σ, where Σ is some finite set which is called the *alphabet* (quite often $\Sigma = \{0,1\}$). We use Σ^* for the set of all strings of finite length.
- **Some special sets:** The set $\mathbb{N}$ of natural numbers. We will index from zero in this course and so write $\mathbb{N} = \{0,1,2,...\}$. We will use $[n]$ for the set $\{0,1,2,\ldots,n-1\}$. We use $\{0,1\}^*$ for the set of all binary strings and $\{0,1\}^n$ for the set of strings of length n. If x is a string of length n, then we refer to its coordinate by $x_0,\ldots,x_{n-1}$.
- **Functions:** The *domain* and *codomain* of a function, properties such as being *one-to-one* (also known as *injective*) or *onto* (also known as *surjective*) functions, as well as *partial functions* (that, unlike standard or "total" functions, are not necessarily defined on all elements of their domain).
- **Logical operations:** The operations AND, OR, and NOT $(\wedge,\vee,\neg)$ and the quantifiers "there exists" and "for all" ($\exists$,$\forall$).
- **Basic combinatorics:** Notions such as $\binom{n}{k}$ (the number of k-sized subsets of a set of size n).
- **Graphs:** Undirected and directed graphs, connectivity, paths, and cycles.
- **Big-O notation:** $O, o, \Omega, \omega, \Theta$ notation for analyzing asymptotic growth of functions.
- **Discrete probability:** Later on in we will use *probability theory*, and specifically probability over *finite* samples spaces such as tossing n coins, including notions such as *random variables*, *expectation*, and *concentration*. We will only use probability theory in the second half of this text, and will review it beforehand. However, probabilistic reasoning is a subtle (and extremely useful!) skill, and it's always good to start early in acquiring it.

In the rest of this chapter we briefly review the above notions. This is partially to remind the reader and reinforce material that might not be fresh in your mind, and partially to introduce our notation and conventions which might occasionally differ from those you've encountered before.

1.4 READING MATHEMATICAL TEXTS

Mathematical texts, such as this book, can often be intimidating at first, as they use a lot of mathematical jargon that can seem dense and confusing to the uninitiated. Mathematicians use jargon for the same

reason that it is used in many other professions such engineering, law, medicine, etc. We want to make terms *precise* and introduce shorthand for concepts that are frequently reused. Mathematical text tends to "pack a lot of punch" per sentence, and so the key is to read it slowly and carefully, parsing each symbol at a time.



Remark 1.1 — Don't be scared of Jargon!. Mathematical Jargon can be kind of scary initially. A statement such as $\forall_{x\in\mathbb{R},\epsilon>0}\exists_{a,b\in\mathbb{N}}|x-\frac{a}{b}|<\epsilon$ might be quite frightening to the uninitiated. But as you get more experience in reading mathematical texts, you will see that the jargon is no longer an issue, and you spend most of your time grappling with the actual ideas in the text. This is no different than programming. For example, even the code of the simple "Hello World" program in Java can be quite intimidating to newcomers, containing great many terms that look scary and inscrutable, but as you work in Java, you quickly get used to these quirks of jargon.

```
public class HelloWorld
{
^^Ipublic static void main(String[] args) {
^^I^^ISystem.out.println("Hello World!");
^^I}
}
```

Dealing with mathematical text is in many ways not so different from dealing with any other complex text, whether it's a legal argument, a philosophical treatise, an English Renaissance play, or even the source code of an operating system. You should not expect it to be clear in a first reading, but you need not despair. Rather you should engage with the text, trying to figure out both the high level intentions as well as the underlying details. Luckily, compared to philosophers or even programmers, mathematicians have a greater discipline of introducing definitions in linear order, making sure that every new concept is defined only in terms of previously defined notions.

The basic components of a mathematical text are **definitions**, **assertions** and **proofs**.

1.4.1 Definitions

Mathematicians often define new concepts in terms of old concepts. Here is an example of a definition:

Definition 1.2 — Perfect Square. A natural number n is a *perfect square* if there exists a natural number k such that $n = k \cdot k$.

Definition 1.2 uses the concept of a natural number, and the concept of multiplication, to define the new term "perfect square".

When you see a new definition or concept, always ask yourself the following questions:

1. What is the intuitive notion that this definition aims at modeling?
2. How is the new concept defined in terms of other concepts?
3. Which of these prior concepts am I already familiar with, and which ones do I still need to look up?
4. Can I come up with examples of objects that satisfy this definition? and examples of objects that do not?

As you read through the rest of this chapter and this text, try to ask yourself the questions 1-4 above every time that you encounter a new definition.

P

You can try this out for the definition of a "perfect square". Can you come up with examples of numbers that are perfect squares, and examples of numbers that are not perfect squares? How do you *prove* that a number is *not* a perfect square.

You can also try to consider more refined questions such as finding out if there exists a perfect square that is the sum of two other perfect squares. You can also try to see how you would define notions such as being a "perfect cube". (Can you try to think if there is a perfect cube which is the sum of two other perfect cubes?)

R

Remark 1.3 — Ifs and equalities in definitions. The meaning of "if" is subtly different in the context of a *definition* than it is in the context of a standalone mathematical statement. When we say that we define n to be a perfect square if $n = k^2$ for some $k \in \mathbb{N}$, we are *defining* the new property "perfect square" to be equivalent to the property that $n = k^2$ for some $k \in \mathbb{N}$. Since this is the *definition* of this property, it means that n is in fact a perfect square *if and only if* $m = k^2$ for some $k \in \mathbb{N}$. We could have also said that we define n to be a perfect square *if and only if* (or use the abbreviation *iff*) $n = k^2$ for some $k \in \mathbb{N}^2$. We will sometimes use "iff" instead of "if" in definitions when it helps clarity, but since mathematicians typically use "if" in such cases, we will often follow this convention as well.

Similarly, the equality symbol "=" plays a subtly different role in the context of a definition than it does elsewhere. When we say something like "we define $y = \sqrt{x}$" what we really mean is that we are *assigning* the value $\sqrt{x}$ to y, as opposed to *asserting* that y equals $\sqrt{x}$. To compare with programming languages, in the context of a definition, the equality sign $=$ plays the role analogous to the assignment operator = in languages such as C, Python, or Javascript, while in other contexts it plays the role analogous to the "equal to" operator ==.

A definition does not have to occur only in the context of a formal "Definition" environment. Whenever you see text such as "we define X as", "let Y be", or "we denote by Z", this is a sign that a new term is being defined.

1.4.2 Assertions: Theorems, lemmas, claims

These are true statements about the concepts that we defined. For example, here is a true statement:

Lemma 1.4 For every perfect square a and perfect square b, $a \cdot b$ is a perfect square.

Deciding whether to call a particular statement a "Theorem", a "Lemma" or a "Claim" is a judgement call, and does not make a mathematical difference. All three correspond to true statements which can be proven. The difference is that a *Theorem* refers to a significant result, that we would want to remember and highlight. A *Lemma* often refers to a technical result, that is not necessarily important in its own right, but can be often very useful in proving other theorems. A *Claim* is a "throw away" statement, that we need to use in order to prove some other bigger results, but do not care so much about for its own sake.

1.4.3 Proofs

These are the justifications that demonstrates that our theorems. We discuss proofs more in Section 1.6 below, but the mathematical standard of proof is very high. Namely we need an "airtight" argument that demonstrates that the statement is true beyond a shadow of a doubt. For example, just checking that **??** is true for the $a = 25$ and $b = 4$ (in which case $a \cdot b = 100$ is indeed a perfect square) doesn't cut it. In contrast, here is a valid proof for **??**:

Proof. Let a and b be perfect squares. Then by the definition of a perfect square, there exist natural numbers a', b' such that $a = a' \cdot a'$ and $b = b' \cdot b'$. Define $c = a \cdot b$. Then by the above

$$c = (a' \cdot a') \cdot (b' \cdot b') = (a' \cdot b') \cdot (a' \cdot b') \tag{1.1}$$

.[2]

Since a', b' are natural numbers, $c' = a' \cdot b'$ is a natural number as well and hence (1.1) implies that $c = c' \cdot c'$ is a perfect square.

■

As mentioned in the preface, as a general rule, it is more important you understand the **definitions** than the **theorems**, and it is more important you understand a **theorem statement** than its **proof**.

1.4.4 Example: Defining a one to one function

Here is a mathematical definition which you may have encountered in the past (and will encounter again shortly):

Definition 1.5 — One to one function. A function $f : S \to T$ is *one to one* if for every two elements $x, x' \in S$, if $x \neq x'$ then $f(x) \neq f(x')$.

This definition captures a simple concept, but even so it uses quite a bit of notation. When reading this definition, or any other piece of mathematical text, it is often useful to annotate it with a pen as you're going through it, as in Fig. 1.1. For every identifier you encounter (for example f, S, T, x, x' in this case), make sure that you realize what sort of object is it: is it a set, a function, an element, a number, a gremlin? Make sure you understand how the identifiers are *quantified*. For example, in Definition 1.5 there is a *universal* or "for all" (sometimes denotes by $\forall$) quantifier over pairs (x, x') of distinct elements in S. Finally, and most importantly, make sure that aside from being able to parse the definition formally, you also have an intuitive understanding of what is it that the text is actually saying. For example, Definition 1.5 says that a one to one function is a function where every output is obtained by a unique input.

Reading mathematical texts in this way takes time, but it gets easier with practice. Moreover, this is one of the most transferable skills you could take from this course. Our world is changing rapidly, not just in the realm of technology, but also in many other human endeavors, whether it is medicine, economics, law or even culture. Whatever your future aspirations, it is likely that you will encounter texts that use new concepts that you have not seen before (for semi-random recent examples from current "hot areas", see Fig. 1.2 and Fig. 1.3). Being able to internalize and then apply new definitions can be hugely important. It is a skill that's much easier to acquire in the relatively

[2] The last inequality follows from the associativity and commutativity of multiplication; i.e., the fact that $(x \cdot y) \cdot z = x \cdot (y \cdot z)$ and $x \cdot y = y \cdot x$ for every numbers x, y, z). These are the kind of properties that in a course of this level we can assume that the reader knows and can "fill in the blank", and hence do not need to state explicitly. However, if you have any doubt, always err on the side of supplying more, rather than less, detail.

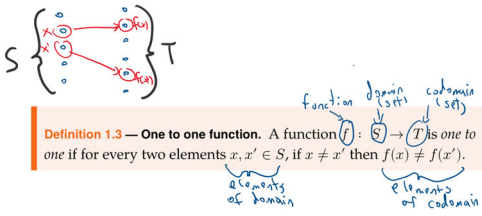


Figure 1.1: An annotated form of Definition 1.5, marking which type is every object, and with a doodle explaining what the definition says.

safe and stable context of a mathematical course, where one at least has the guarantee that the concepts are fully specified, and you have access to your teaching staff for questions.

1.5 BASIC DISCRETE MATH OBJECTS

We now quickly review some of the mathematical objects (the "basic data structures" of mathematics, if you will) we use in this course.

1.5.1 Sets

A *set* is an unordered collection of objects. For example, when we write $S = \{2, 4, 7\}$, we mean that S denotes the set that contains the numbers 2, 4, and 7. (We use the notation "$2 \in S$" to denote that 2 is an element of S.) Note that the set $\{2, 4, 7\}$ and $\{7, 4, 2\}$ are identical, since they contain the same elements. Also, a set either contains an element or does not contain it – there is no notion of containing it "twice" – and so we could even write the same set S as $\{2, 2, 4, 7\}$ (though that would be a little weird). The *cardinality* of a finite set S, denoted by $|S|$, is the number of elements it contains.[3] So, in the example above, $|S| = 3$. A set S is a *subset* of a set T, denoted by $S \subseteq T$, if every element of S is also an element of T. (We can also describe this by saying that T is a *superset* of S.) For example, $\{2, 7\} \subseteq \{2, 4, 7\}$. The set that contains no elements is known as the *empty set* and it is denoted by $\emptyset$.

We can define sets by either listing all their elements or by writing down a rule that they satisfy such as

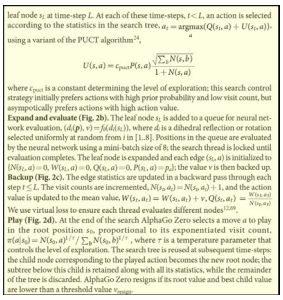
$$\text{EVEN} = \{x \ : \ x = 2y \text{ for some non-negative integer } y\} \ . \tag{1.2}$$

Of course there is more than one way to write the same set, and often we will use intuitive notation listing a few examples that illustrate the rule. For example, we can also define EVEN as

$$\text{EVEN} = \{0, 2, 4, ...\} \ . \tag{1.3}$$

Note that a set can be either finite (such as the set $\{2, 4, 7\}$) or infinite (such as the set EVEN). Also, the elements of a set don't have to be numbers. We can talk about the sets such as the set $\{a, e, i, o, u\}$ of all the vowels in the English language, or the set { `New York`, `Los Angeles`, `Chicago`, `Houston`, `Philadelphia`, `Phoenix`, `San Antonio`, `San Diego`, `Dallas` } of all cities in the U.S. with population more than one million per the 2010 census. A set can even have other sets as elements, such as the set $\{\emptyset, \{1, 2\}, \{2, 3\}, \{1, 3\}\}$ of all even-sized subsets of $\{1, 2, 3\}$.

Operations on sets: The *union* of two sets S, T, denoted by $S \cup T$, is the set that contains all elements that are either in S *or* in T. The



leaf node s_L at time-step L. At each of these time-steps, $t < L$, an action is selected according to the statistics in the search tree, $a_t = \underset{a}{\text{argmax}}(Q(s_t, a) + U(s_t, a))$, using a variant of the PUCT algorithm[24],

$$U(s,a) = c_{\text{puct}} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

where c_{puct} is a constant determining the level of exploration; this search control strategy initially prefers actions with high prior probability and low visit count, but asymptotically prefers actions with high action value.

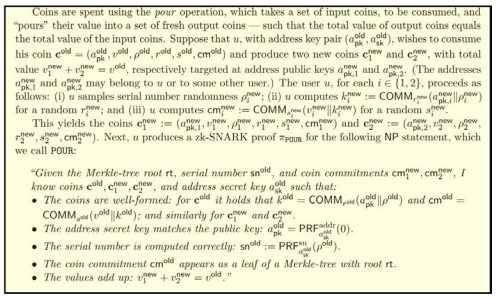
Expand and evaluate (Fig. 2b). The leaf node s_L is added to a queue for neural network evaluation, $(d(\boldsymbol{p}), v) = f_\theta(d_i(s_L))$, where d_i is a dihedral reflection or rotation selected uniformly at random from i in $[1...8]$. Positions in the queue are evaluated by the neural network using a mini-batch size of 8; the search thread is locked until evaluation completes. The leaf node is expanded and each edge (s, a) is initialized to $\{N(s_L, a) = 0, W(s_L, a) = 0, Q(s_L, a) = 0, P(s_L, a) = p_a\}$; the value v is then backed up.

Backup (Fig. 2c). The edge statistics are updated in a backward pass through each step $t \leq L$. The visit counts are incremented, $N(s_t, a_t) = N(s_t, a_t) + 1$, and the action value is updated to the mean value, $W(s_t, a_t) = W(s_t, a_t) + v$, $Q(s_t, a_t) = \frac{W(s_t, a_t)}{N(s_t, a_t)}$.

We use virtual loss to ensure each thread evaluates different nodes[12,69].

Play (Fig. 2d). At the end of the search AlphaGo Zero selects a move a to play in the root position s_0, proportional to its exponentiated visit count, $\pi(a|s_0) = N(s_0, a)^{1/\tau} / \sum_b N(s_0, b)^{1/\tau}$, where τ is a temperature parameter that controls the level of exploration. The search tree is reused at subsequent time-steps: the child node corresponding to the played action becomes the new root node; the subtree below this child is retained along with all its statistics, while the remainder of the tree is discarded. AlphaGo Zero resigns if its root value and best child value are lower than a threshold value v_{resign}.

Figure 1.2: A snippet from the "methods" section of the "AlphaGo Zero" paper by Silver et al, *Nature*, 2017.



Coins are spent using the *pour* operation, which takes a set of input coins, to be consumed, and "pours" their value into a set of fresh output coins — such that the total value of output coins equals the total value of the input coins. Suppose that u, with address key pair $(a^{\text{old}}_{\text{pk}}, a^{\text{old}}_{\text{sk}})$, wishes to consume his coin $\mathsf{c}^{\text{old}} = (a^{\text{old}}_{\text{pk}}, v^{\text{old}}, \rho^{\text{old}}, r^{\text{old}}, s^{\text{old}}, \mathsf{cm}^{\text{old}})$ and produce two new coins $\mathsf{c}^{\text{new}}_1$ and $\mathsf{c}^{\text{new}}_2$, with total value $v^{\text{new}}_1 + v^{\text{new}}_2 = v^{\text{old}}$, respectively targeted at address public keys $a^{\text{new}}_{\text{pk},1}$ and $a^{\text{new}}_{\text{pk},2}$. (The addresses $a^{\text{new}}_{\text{pk},1}$ and $a^{\text{new}}_{\text{pk},2}$ may belong to u or to some other user.) The user u, for each $i \in \{1, 2\}$, proceeds as follows: (i) u samples serial number randomness ρ^{new}_i; (ii) u computes $k^{\text{new}}_i := \mathsf{COMM}_{r^{\text{new}}_i}(a^{\text{new}}_{\text{pk},i} \| \rho^{\text{new}}_i)$ for a random r^{new}_i; and (iii) u computes $\mathsf{cm}^{\text{new}}_i := \mathsf{COMM}_{s^{\text{new}}_i}(v^{\text{new}}_i \| k^{\text{new}}_i)$ for a random s^{new}_i.

This yields the coins $\mathsf{c}^{\text{new}}_1 := (a^{\text{new}}_{\text{pk},1}, v^{\text{new}}_1, \rho^{\text{new}}_1, r^{\text{new}}_1, s^{\text{new}}_1, \mathsf{cm}^{\text{new}}_1)$ and $\mathsf{c}^{\text{new}}_2 := (a^{\text{new}}_{\text{pk},2}, v^{\text{new}}_2, \rho^{\text{new}}_2, r^{\text{new}}_2, s^{\text{new}}_2, \mathsf{cm}^{\text{new}}_2)$. Next, u produces a zk-SNARK proof $\pi_{\texttt{POUR}}$ for the following NP statement, which we call `POUR`:

"*Given the Merkle-tree root* rt, *serial number* sn^{old}, *and coin commitments* $\mathsf{cm}^{\text{new}}_1, \mathsf{cm}^{\text{new}}_2$, *I know coins* $\mathsf{c}^{\text{old}}, \mathsf{c}^{\text{new}}_1, \mathsf{c}^{\text{new}}_2$, *and address secret key* $a^{\text{old}}_{\text{sk}}$ *such that:*

- *The coins are well-formed: for* c^{old} *it holds that* $k^{\text{old}} = \mathsf{COMM}_{r^{\text{old}}}(a^{\text{old}}_{\text{pk}} \| \rho^{\text{old}})$ *and* $\mathsf{cm}^{\text{old}} = \mathsf{COMM}_{s^{\text{old}}}(v^{\text{old}} \| k^{\text{old}})$; *and similarly for* $\mathsf{c}^{\text{new}}_1$ *and* $\mathsf{c}^{\text{new}}_2$.
- *The address secret key matches the public key:* $a^{\text{old}}_{\text{pk}} = \mathsf{PRF}^{\text{addr}}_{a^{\text{old}}_{\text{sk}}}(0)$.
- *The serial number is computed correctly:* $\mathsf{sn}^{\text{old}} := \mathsf{PRF}^{\text{sn}}_{a^{\text{old}}_{\text{sk}}}(\rho^{\text{old}})$.
- *The coin commitment* cm^{old} *appears as a leaf of a Merkle-tree with root* rt.
- *The values add up:* $v^{\text{new}}_1 + v^{\text{new}}_2 = v^{\text{old}}$."

Figure 1.3: A snippet from the "Zerocash" paper of Ben-Sasson et al, that forms the basis of the cryptocurrency startup Zcash.

[3] Later in this course we will discuss how to extend the notion of cardinality to *infinite* sets.

intersection of S and T, denoted by $S \cap T$, is the set of elements that are both in S *and* in T. The *set difference* of S and T, denoted by $S \setminus T$ (and in some texts also by $S - T$), is the set of elements that are in S but *not* in T.

Tuples, lists, strings, sequences: A *tuple* is an *ordered* collection of items. For example $(1, 5, 2, 1)$ is a tuple with four elements (also known as a 4-tuple or quadruple). Since order matters, this is not the same tuple as the 4-tuple $(1, 1, 5, 2)$ or the 3-tuple $(1, 5, 2)$. A 2-tuple is also known as a *pair*. We use the terms *tuples* and *lists* interchangeably. A tuple where every element comes from some finite set Σ (such as $\{0, 1\}$) is also known as a *string*. Analogously to sets, we denote the *length* of a tuple T by $|T|$. Just like sets, we can also think of infinite analogues of tuples, such as the ordered collection $(1, 2, 4, 9, ...)$ of all perfect squares. Infinite ordered collections are known as *sequences*; we might sometimes use the term "infinite sequence" to emphasize this, and use "finite sequence" as a synonym for a tuple.[4]

[4] We can identify a sequence $(a_0, a_1, a_2, ...)$ of elements in some set S with a *function* $A : \mathbb{N} \to S$ (where $a_n = A(n)$ for every $n \in \mathbb{N}$). Similarly, we can identify a k-tuple $(a_0, ..., a_{k-1})$ of elements in S with a function $A : [k] \to S$.

Cartesian product: If S and T are sets, then their *Cartesian product*, denoted by $S \times T$, is the set of all ordered pairs (s, t) where $s \in S$ and $t \in T$. For example, if $S = \{1, 2, 3\}$ and $T = \{10, 12\}$, then $S \times T$ contains the 6 elements $(1, 10), (2, 10), (3, 10), (1, 12), (2, 12), (3, 12)$. Similarly if S, T, U are sets then $S \times T \times U$ is the set of all ordered triples (s, t, u) where $s \in S$, $t \in T$, and $u \in U$. More generally, for every positive integer n and sets $S_0, ..., S_{n-1}$, we denote by $S_0 \times S_1 \times \cdots \times S_{n-1}$ the set of ordered n-tuples $(s_0, ..., s_{n-1})$ where $s_i \in S_i$ for every $i \in \{0, ..., n-1\}$. For every set S, we denote the set $S \times S$ by S^2, $S \times S \times S$ by S^3, $S \times S \times S \times S$ by S^4, and so on and so forth.

1.5.2 Sets in Python (optional)

To get more comfortable with sets, one can also play with the `set` data structure in Python:[5]

[5] The `set` data structure only corresponds to *finite* sets; infinite sets are much more cumbersome to handle in programming languages, though mechanisms such as Python generators and lazy evaluation in general can be helpful.

```
A = { 7 , 10 , 12}
B = {12 , 8 , 5 }
print(A==B)
# False
print(A=={10,7,7,12})
# True

def intersect(S,T):
    return {x for x in S if x in T}

print(intersect(A,B))
# {12}
```

```
def contains(S,T):
    return all({x in T for x in S})

print(contains(A,B))
# False

print(contains({2,8,8,12},{12,8,2,34}))
# True

def product(S,T):
    return {(s,t) for s in  S for t in  T}

print(product(A,B))
# {(10, 8), (10, 5), (7, 12), (12, 12), (10, 12), (12, 5),
 ↪  (7, 5), (7, 8), (12, 8)}
```

1.5.3 Special sets

There are several sets that we will use in this course time and again, and so find it useful to introduce explicit notation for them. For starters we define

$$\mathbb{N} = \{0, 1, 2, ...\} \tag{1.4}$$

to be the set of all *natural numbers*, i.e., non-negative integers. For any natural number n, we define the set $[n]$ as $\{0, \ldots, n-1\} = \{k \in \mathbb{N} : k < n\}$.[6]

We will also occasionally use the set $\mathbb{Z} = \{\ldots, -2, -1, 0, +1, +2, ...\}$ of (negative and non-negative) *integers*,[7] as well as the set $\mathbb{R}$ of *real* numbers. (This is the set that includes not just the integers, but also fractional and even irrational numbers; e.g., $\mathbb{R}$ contains numbers such as $+0.5$, $-\pi$, etc.) We denote by $\mathbb{R}_+$ the set $\{x \in \mathbb{R} : x > 0\}$ of *positive* real numbers. This set is sometimes also denoted as $(0, \infty)$.

[6] We start our indexing of both $\mathbb{N}$ and $[n]$ from 0, while many other texts index those sets from 1. Starting from zero or one is simply a convention that doesn't make much difference, as long as one is consistent about it.

[7] The letter Z stands for the German word "Zahlen", which means *numbers*.

Strings: Another set we will use time and again is

$$\{0,1\}^n = \{(x_0, \ldots, x_{n-1}) \ : \ x_0, \ldots, x_{n-1} \in \{0,1\}\} \tag{1.5}$$

which is the set of all n-length binary strings for some natural number n. That is $\{0,1\}^n$ is the set of all n-tuples of zeroes and ones. This is consistent with our notation above: $\{0,1\}^2$ is the Cartesian product $\{0,1\} \times \{0,1\}$, $\{0,1\}^3$ is the product $\{0,1\} \times \{0,1\} \times \{0,1\}$ and so on.

We will write the string $(x_0, x_1, \ldots, x_{n-1})$ as simply $x_0 x_1 \cdots x_{n-1}$ and so for example

$$\{0,1\}^3 = \{000, 001, 010, 011, 100, 101, 110, 111\} \ . \tag{1.6}$$

For every string $x \in \{0,1\}^n$ and $i \in [n]$, we write x_i for the ith coordinate of x. If x and y are strings, then xy denotes their *concatenation*. That is, if $x \in \{0,1\}^n$ and $y \in \{0,1\}^m$, then xy is equal to the string $z \in \{0,1\}^{n+m}$ such that for $i \in [n]$, $z_i = x_i$ and for $i \in \{n, \ldots, n+m-1\}$, $z_i = y_{i-n}$.

We will also often talk about the set of binary strings of *all* lengths, which is

$$\{0,1\}^* = \{(x_0, \ldots, x_{n-1}) \ : \ n \in \mathbb{N} \ , \ , x_0, \ldots, x_{n-1} \in \{0,1\}\} \ . \tag{1.7}$$

Another way to write this set is as

$$\{0,1\}^* = \{0,1\}^0 \cup \{0,1\}^1 \cup \{0,1\}^2 \cup \cdots \tag{1.8}$$

or more concisely as

$$\{0,1\}^* = \cup_{n \in \mathbb{N}} \{0,1\}^n \ . \tag{1.9}$$

The set $\{0,1\}^*$ contains also the "string of length 0" or "the empty string", which we will denote by "".[8]

[8] In using this notation we follow the convention of many programming languages. Other texts sometimes use ϵ or λ to denote the empty string. However, this doesn't matter much since we will rarely encounter this "edge case".

Generalizing the star operation: For every set Σ, we define

$$\Sigma^* = \cup_{n \in \mathbb{N}} \Sigma^n \ . \tag{1.10}$$

For example, if $\Sigma = \{a, b, c, d, \ldots, z\}$ then Σ^* denotes the set of all finite length strings over the alphabet a-z.

Concatenation: As mentioned in Section 1.5.3, the *concatenation* of two strings $x \in \Sigma^n$ and $y \in \Sigma^m$ is the $(n+m)$-length string xy obtained by writing y after x.

1.5.4 Functions

If S and T are nonempty sets, a *function* F mapping S to T, denoted by $F : S \to T$, associates with every element $x \in S$ an element $F(x) \in T$. The set S is known as the *domain* of F and the set T is known as the *codomain* of F. The *image* of a function F is the set $\{F(x) \mid x \in S\}$ which is the subset of F's codomain consisting of all output elements that are mapped from some input.[9] As in the case of sets, we can write a function either by listing the table of all the values it gives for elements in S or by using a rule. For example if $S = \{0,1,2,3,4,5,6,7,8,9\}$ and $T = \{0,1\}$, then the table below defines a function $F : S \to T$. Note that this function is the same as the function defined by the rule $F(x) = (x \mod 2)$.[10]

[9] Some texts use *range* to denote the image of a function, while other texts use *range* to denote the codomain of a function. Hence we will avoid using the term "range" altogether.

[10] For two natural numbers x and a, $x \mod a$ (where mod is shorthand for "modulo") denotes the *remainder* of x when it is divided by a. That is, it is the number r in $\{0, \ldots, a-1\}$ such that $x = ak + r$ for some integer k. We sometimes also use the notation $x = y \pmod a$ to denote the assertion that $x \mod a$ is the same as $y \mod a$.

Table 1.1: An example of a function.

Input	Output
0	0
1	1
2	0
3	1
4	0
5	1
6	0
7	1
8	0
9	1

If $F : S \to T$ satisfies that $F(x) \neq F(y)$ for all $x \neq y$ then we say that F is *one-to-one* (also known as an *injective* function or simply an *injection*). If F satisfies that for every $y \in T$ there is some $x \in S$ such that $F(x) = y$ then we say that F is *onto* (also known as a *surjective* function or simply a *surjection*). A function that is both one-to-one and onto is known as a *bijective* function or simply a *bijection*. A bijection from a set S to itself is also known as a *permutation* of S. If $F : S \to T$ is a bijection then for every $y \in T$ there is a unique $x \in S$ s.t. $F(x) = y$. We denote this value x by $F^{-1}(y)$. Note that F^{-1} is itself a bijection from T to S (can you see why?).

Giving a bijection between two sets is often a good way to show they have the same size. In fact, the standard mathematical definition of the notion that "S and T have the same cardinality" is that there exists a bijection $f : S \to T$. In particular, the cardinality of a set S is defined to be n if there is a bijection from S to the set $\{0, \ldots, n-1\}$. As we will see later in this course, this is a definition that can generalizes to defining the cardinality of *infinite* sets.

Partial functions: We will sometimes be interested in *partial* functions from S to T. A partial function is allowed to be undefined on some subset of S. That is, if F is a partial function from S to T, then for every $s \in S$, either there is (as in the case of standard functions) an element $F(s)$ in T, or $F(s)$ is undefined. For example, the partial function $F(x) = \sqrt{x}$ is only defined on non-negative real numbers. When we want to distinguish between partial functions and standard (i.e., non-partial) functions, we will call the latter *total* functions. When we say "function" without any qualifier then we mean a *total* function.

The notion of partial functions is a strict generalization of functions, and so every function is a partial function, but not every partial function is a function. (That is, for every nonempty S and T, the set

of partial functions from S to T is a proper superset of the set of total functions from S to T.) When we want to emphasize that a function f from A to B might not be total, we will write $f : A \to_p B$. We can think of a partial function F from S to T also as a total function from S to $T \cup \{\perp\}$ where $\perp$ is some special "failure symbol", and so instead of saying that F is undefined at x, we can say that $F(x) = \perp$.

Basic facts about functions: Verifying that you can prove the following results is an excellent way to brush up on functions:

- If $F : S \to T$ and $G : T \to U$ are one-to-one functions, then their *composition* $H : S \to U$ defined as $H(s) = G(F(s))$ is also one to one.
- If $F : S \to T$ is one to one, then there exists an onto function $G : T \to S$ such that $G(F(s)) = s$ for every $s \in S$.
- If $G : T \to S$ is onto then there exists a one-to-one function $F : S \to T$ such that $G(F(s)) = s$ for every $s \in S$.
- If S and T are finite sets then the following conditions are equivalent to one another: **(a)** $|S| \leq |T|$, **(b)** there is a one-to-one function $F : S \to T$, and **(c)** there is an onto function $G : T \to S$.[11]



You can find the proofs of these results in many discrete math texts, including for example, Section 4.5 in the Lehman-Leighton-Meyer notes. However, I strongly suggest you try to prove them on your own, or at least convince yourself that they are true by proving special cases of those for small sizes (e.g., $|S| = 3, |T| = 4, |U| = 5$).

Let us prove one of these facts as an example:

Lemma 1.6 If S, T are non-empty sets and $F : S \to T$ is one to one, then there exists an onto function $G : T \to S$ such that $G(F(s)) = s$ for every $s \in S$.

Proof. Let S, T and $F : S \to T$ be as in the Lemma's statement, and choose some $s_0 \in S$. We will define the function $G : T \to S$ as follows: for every $t \in T$, if there is some $s \in S$ such that $F(s) = t$ then set $G(t) = s$ (the choice of s is well defined since by the one-to-one property of F, there cannot be two distinct s, s' that both map to t). Otherwise, set $G(t) = s_0$. Now for every $s \in S$, by the definition of G, if $t = F(s)$ then $G(t) = G(F(s)) = s$. Moreover, this also shows that G is *onto*, since it means that for every $s \in S$ there is some t (namely $t = F(s)$) such that $G(t) = s$.

■

[11] This is actually true even for *infinite* S and T: in that case **(b)** is the commonly accepted *definition* for $|S| \leq |T|$.

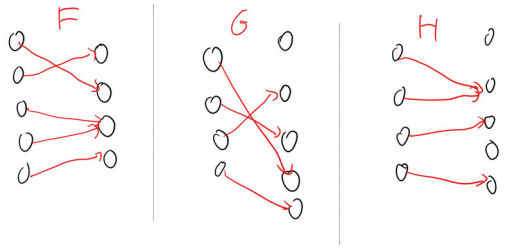


Figure 1.4: We can represent finite functions as a directed graph where we put an edge from x to $f(x)$. The *onto* condition corresponds to requiring that every vertex in the codomain of the function has in-degree *at least* one. The *one-to-one* condition corresponds to requiring that every vertex in the codomain of the function has in-degree *at most* one. In the examples above F is an onto function, G is one to one, and H is neither onto nor one to one.

1.5.5 Graphs

Graphs are ubiquitous in Computer Science, and many other fields as well. They are used to model a variety of data types including social networks, scheduling constraints, road networks, deep neural nets, gene interactions, correlations between observations, and a great many more. The formal definitions of graphs are below for the sake of completeness, but if you have not seen graphs before in a course, I urge you to read up on them in one of the sources mentioned in Section 1.10.[12]

Graphs come in two basic flavors: *undirected* and *directed*.[13]

Definition 1.7 — Undirected graphs. An *undirected graph* $G = (V, E)$ consists of a set V of *vertices* and a set E of edges. Every edge is a size two subset of V. We say that two vertices $u, v \in V$ are *neighbors*, if the edge $\{u, v\}$ is in E.

Given this definition, we can define several other properties of graphs and their vertices. We define the *degree* of u to be the number of neighbors u has. A *path* in the graph is a tuple $(u_0, \ldots, u_k) \in V^k$, for some $k > 0$ such that u_{i+1} is a neighbor of u_i for every $i \in [k]$. A *simple path* is a path $(u_0, \ldots, u_{k-1})$ where all the u_i's are distinct. A *cycle* is a path $(u_0, \ldots, u_k)$ where $u_0 = u_k$. We say that two vertices $u, v \in V$ are *connected* if either $u = v$ or there is a path from $(u_0, \ldots, u_k)$ where $u_0 = u$ and $u_k = v$. We say that the graph G is *connected* if every pair of vertices in it is connected.

Here are some basic facts about undirected graphs. We give some informal arguments below, but leave the full proofs as exercises. (The proofs can also be found in most basic texts on graph theory.)

Lemma 1.8 In any undirected graph $G = (V, E)$, the sum of the degrees of all vertices is equal to twice the number of edges.

Lemma 1.8 can be shown by seeing that every edge $\{u, v\}$ contributes twice to the sum of the degrees (once for u and the second time for v.)

Lemma 1.9 The connectivity relation is *transitive*, in the sense that if u is connected to v, and v is connected to w, then u is connected to w.

Lemma 1.9 can be shown by simply attaching a path of the form $(u, u_1, u_2, \ldots, u_{k-1}, v)$ to a path of the form $(v, u'_1, \ldots, u'_{k'-1}, w)$ to obtain the path $(u, u_1, \ldots, u_{k-1}, v, u'_1, \ldots, u'_{k'-1}, w)$ that connects u to w.

Lemma 1.10 For every undirected graph $G = (V, E)$ and connected pair u, v, the shortest path from u to v is simple. In particular, for every connected pair there exists a simple path that connects them.

[12] One such recommended resource is this lecture of Berkeley CS70.

[13] It is possible, and sometimes useful, to think of an undirected graph as the special case of an directed graph that has the special property that for every pair u, v either both the edges (u, v) and (v, u) are present or neither of them is. However, in many settings there is a significant difference between undirected and directed graphs, and so it's typically best to think of them as separate categories.

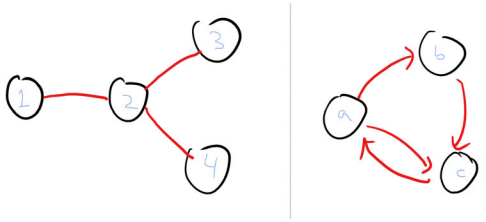


Figure 1.5: An example of an undirected and a directed graph. The undirected graph has vertex set $\{1, 2, 3, 4\}$ and edge set $\{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$. The directed graph has vertex set $\{a, b, c\}$ and the edge set $\{(a, b), (b, c), (c, a), (a, c)\}$.

Lemma 1.10 can be shown by "shortcutting" any non simple path of the form $(u, u_1, \ldots, u_{i-1}, w, u_{i+1}, \ldots, u_{j-1}, w, u_{j+1}, \ldots, u_{k-1}, v)$ where the same vertex w appears in both the i-th and j-position, to obtain the shorter path $(u, u_1, \ldots, u_{i-1}, w, u_{j+1}, \ldots, u_{k-1}, v)$.



If you haven't seen these proofs before, it is indeed a great exercise to transform the above informal argument into fully rigorous proofs.

Definition 1.11 — Directed graphs. A *directed graph* $G = (V, E)$ consists of a set V and a set $E \subseteq V \times V$ of *ordered pairs* of V. We sometimes denote the edge (u, v) also as $u \to v$. If the edge $u \to v$ is present in the graph then we say that v is an *out-neighbor* of u and u is an *in-neighbor* of v.

A directed graph might contain both $u \to v$ and $v \to u$ in which case u will be both an in-neighbor and an out-neighbor of v and vice versa. The *in-degree* of u is the number of in-neighbors it has, and the *out-degree* of v is the number of out-neighbors it has. A *path* in the graph is a tuple $(u_0, \ldots, u_k) \in V^k$, for some $k > 0$ such that u_{i+1} is an out-neighbor of u_i for every $i \in [k]$. As in the undirected case, a *simple path* is a path $(u_0, \ldots, u_{k-1})$ where all the u_i's are distinct and a *cycle* is a path $(u_0, \ldots, u_k)$ where $u_0 = u_k$. One type of directed graphs we often care about is *directed acyclic graphs* or *DAGs*, which, as their name implies, are directed graphs without any cycles:

Definition 1.12 — Directed Acyclic Graphs. We say that $G = (V, E)$ is a *directed acyclic graph* (*DAG*) if it is a directed graph and there does not exist a list of vertices $u_0, u_1, \ldots, u_k \in V$ such that $u_0 = u_k$ and for every $i \in [k]$, the edge $u_i \to u_{i+1}$ is in E.

The lemmas we mentioned above have analogs for directed graphs. We again leave the proofs (which are essentially identical to their undirected analogs) as exercises for the reader:

Lemma 1.13 In any directed graph $G = (V, E)$, the sum of the in-degrees is equal to the sum of the out-degrees, which is equal to the number of edges.

Lemma 1.14 In any directed graph G, if there is a path from u to v and a path from v to w, then there is a path from u to w.

Lemma 1.15 For every directed graph $G = (V, E)$ and a pair u, v such that there is a path from u to v, the *shortest path* from u to v is simple.

R

Remark 1.16 — Graph terminology. The word *graph* in the sense above was coined by the mathematician Sylvester in 1878 in analogy with the chemical graphs used to visualize molecules. There is an unfortunate confusion between this tern and the more common usage of the word "graph" as a way to plot data, and in particular a plot of some function $f(x)$ as a function of x. One way to relate these two notions is to identify every function $f : A \to B$ with the directed graph G_f over the vertex set $V = A \cup B$ such that G_f contains the edge $x \to f(x)$ for every $x \in A$.

In a graph G_f constructed in this way, every vertex in A has out-degree equal to one. If the function f is *one to one* then every vertex in B has in-degree at most one. If the function f is *onto* then every vertex in B has in-degree at least one. if f is a bijection then every vertex in B has in-degree exactly equal to one.

R

Remark 1.17 — Labeled graphs (optional). For some applications one needs to consider *labeled graphs*, where the vertices or edges have some *labels* (which can be numbers, strings, or members of some other set) associated with them. We can think of such a graph as having an associated (possibly partial) *labelling function* $L : V \cup E \to \mathcal{L}$, where $\mathcal{L}$ is the set of potential labels. However we will typically not refer explicitly to this labeling function and simply say things such as "vertex v has the label α".

1.5.6 Logic operators and quantifiers.

If P and Q are some statements that can be true or false, then P AND Q (denoted as $P \wedge Q$) is the statement that is true if and only if both P *and* Q are true, and P OR Q (denoted as $P \vee Q$) is the statement that is true if and only if either P *or* Q is true. The *negation* of P, denoted as $\neg P$ or $\overline{P}$, is the statement that is true if and only if P is false.

Suppose that $P(x)$ is a statement that depends on some *parameter* x (also sometimes known as an *unbound* variable) in the sense that for every instantiation of x with a value from some set S, $P(x)$ is either true or false. For example, $x > 7$ is a statement that is not a priori true or false, but does become true or false whenever we instantiate x with some real number. In such case we denote by $\forall_{x \in S} P(x)$ the statement that is true if and only if $P(x)$ is true *for every* $x \in S$.[14] We denote by $\exists_{x \in S} P(x)$ the statement that is true if and only if *there exists* some $x \in S$ such that $P(x)$ is true.

[14] In this book, we place the variable that is bound by a quantifier in a subscript and so write $\forall_{x \in S} P(x)$. Some other texts do not use this subscript notation and so will write the same statement as $\forall x \in S\ P(x)$.

For example, the following is a formalization of the true statement that there exists a natural number n larger than 100 that is not divisible by 3:

$$\exists_{n\in\mathbb{N}}(n > 100) \wedge (\forall_{k\in N} k + k + k \neq n) \ . \tag{1.11}$$

For sufficiently large n. One expression that we will see come up time and again in this book is the claim that some statement $P(n)$ is true "for sufficiently large n". What this means is that there exists an integer N_0 such that $P(n)$ is true for every $n > N_0$. We can formalize this as $\exists_{N_0\in\mathbb{N}} \forall_{n>N_0} P(n)$.

1.5.7 Quantifiers for summations and products

The following shorthands for summing up or taking products of several numbers are often convenient. If $S = \{s_0, \ldots, s_{n-1}\}$ is a finite set and $f : S \to \mathbb{R}$ is a function, then we write $\sum_{x\in S} f(x)$ as shorthand for

$$f(s_0) + f(s_1) + f(s_2) + \ldots + f(s_{n-1}) \ , \tag{1.12}$$

and $\prod_{x\in S} f(x)$ as shorthand for

$$f(s_0) \cdot f(s_1) \cdot f(s_2) \cdot \ldots \cdot f(s_{n-1}) \ . \tag{1.13}$$

For example, the sum of the squares of all numbers from 1 to 100 can be written as

$$\sum_{i\in\{1,\ldots,100\}} i^2 \ . \tag{1.14}$$

Since summing up over intervals of integers is so common, there is a special notation for it, and for every two integers $a \leq b$, $\sum_{i=a}^{b} f(i)$ denotes $\sum_{i\in S} f(i)$ where $S = \{x \in \mathbb{Z} : a \leq x \leq b\}$. Hence we can write the sum (1.14) also as

$$\sum_{i=1}^{100} i^2 \ . \tag{1.15}$$

1.5.8 Parsing formulas: bound and free variables

In mathematics, as in coding, we often have symbolic "variables" or "parameters". It is important to be able to understand, given some formula, whether a given variable is *bound* or *free* in this formula. For example, in the following statement n is free but a and b are bound by the $\exists$ quantifier:

$$\exists_{a,b\in\mathbb{N}}(a \neq 1) \wedge (a \neq n) \wedge (n = a \times b) \tag{1.16}$$

Since n is free, it can be set to any value, and the truth of the statement (1.16) depends on the value of n. For example, if $n = 8$ then (1.16) is true, but for $n = 11$ it is false. (Can you see why?)

The same issue appears when parsing code. For example, in the following snippet from the C++ programming language

```
for (int i=0 ; i<n ; i=i+1) {
    printf("*");
}
```

the variable `i` is bound to the `for` operator but the variable `n` is free.

The main property of bound variables is that we can *rename* them (as long as the new name doesn't conflict with another used variable) without changing the meaning of the statement. Thus for example the statement

$$\exists_{x,y\in\mathbb{N}}(x \neq 1) \wedge (x \neq n) \wedge (n = x \times y) \tag{1.17}$$

is *equivalent* to (1.16) in the sense that it is true for exactly the same set of n's.

Similarly, the code

```
for (int j=0 ; j<n ; j=j+1) {
    printf("*");
}
```

produces the same result as the code above that used `i` instead of `j`.



Remark 1.18 — Aside: mathematical vs programming notation. Mathematical notation has a lot of similarities with programming language, and for the same reasons. Both are formalisms meant to convey complex concepts in a precise way. However, there are some cultural differences. In programming languages, we often try to use meaningful variable names such as `NumberOfVertices` while in math we often use short identifiers such as n. (Part of it might have to do with the tradition of mathematical proofs as being hand-written and verbally presented, as opposed to typed up and compiled.)

One consequence of that is that in mathematics we often end up reusing identifiers, and also "run out" of letters and hence use Greek letters too, as well as distinguish between small and capital letters. Similarly, mathematical notation tends to use quite a lot of "overloading", using operators such as $+$ for a great variety of objects (e.g., real numbers, matrices, finite field elements, etc..), and assuming that the meaning can be inferred from the context.

Both fields have a notion of "types", and in math we often try to reserve certain letters for variables of a particular type. For example, variables such as i, j, k, ℓ, m, n will often denote integers, and ϵ will often denote a small positive real number (see Section 1.8 for more on these conventions). When reading or writing mathematical texts, we usually don't have the advantage of a "compiler" that will check type safety for us. Hence it is important to keep track of the type of each variable, and see that the operations that are performed on it "make sense".
Kun's book [Kun18] contains an extensive discussion on the similarities and differences between the cultures of mathematics and programming.

1.5.9 Asymptotics and Big-O notation

> "$\log\log\log n$ *has been proved to go to infinity, but has never been observed to do so.*", Anonymous, quoted by Carl Pomerance (2000)

It is often very cumbersome to describe precisely quantities such as running time and is also not needed, since we are typically mostly interested in the "higher order terms". That is, we want to understand the *scaling behavior* of the quantity as the input variable grows. For example, as far as running time goes, the difference between an n^5-time algorithm and an n^2-time one is much more significant than the difference between an $100n^2 + 10n$ time algorithm and an $10n^2$ time algorithm. For this purpose, O-notation is extremely useful as a way to "declutter" our text and focus our attention on what really matters. For example, using O-notation, we can say that both $100n^2 + 10n$ and $10n^2$ are simply $\Theta(n^2)$ (which informally means "the same up to constant factors"), while $n^2 = o(n^5)$ (which informally means that n^2 is "much smaller than" n^5).[15]

Generally (though still informally), if F, G are two functions mapping natural numbers to non-negative reals, then "$F = O(G)$" means that $F(n) \leq G(n)$ if we don't care about constant factors, while "$F = o(G)$" means that F is much smaller than G, in the sense that no matter by what constant factor we multiply F, if we take n to be large enough then G will be bigger (for this reason, sometimes $F = o(G)$ is written as $F \ll G$). We will write $F = \Theta(G)$ if $F = O(G)$ and $G = O(F)$, which one can think of as saying that F is the same as G if we don't care about constant factors. More formally, we define Big-O notation as follows:

[15] While Big-O notation is often used to analyze running time of algorithms, this is by no means the only application. At the end of the day, Big-O notation is just a way to express asymptotic inequalities between functions on integers. It can be used regardless of whether these functions are a measure of running time, memory usage, or any other quantity that may have nothing to do with computation.

Definition 1.19 — Big-O notation. For $F, G : \mathbb{N} \to \mathbb{R}_+$, we define $F = O(G)$ (sometimes also written as $F \leq O(G)$) if there exist numbers $a, N_0 \in \mathbb{N}$ such that $F(n) \leq a \cdot G(n)$ for every $n > N_0$. [16] We define $F = \Omega(G)$ (sometimes also writtenas $F \leq \Omega(G)$) if $G = O(F)$.

We write $F = o(G)$ if for every $\epsilon > 0$ there is some N_0 such that $F(n) < \epsilon G(n)$ for every $n > N_0$. We write $F = \omega(G)$ if $G = o(F)$. We write $F = \Theta(G)$ if $F = O(G)$ and $G = O(F)$.

We can also use the notion of *limits* to define Big- and Little-O notation. You can verify that $F = o(G)$ (or, equivalently, $G = \omega(F)$) if and only if $\lim_{n\to\infty} \frac{F(n)}{G(n)} = 0$. Similarly, if the limit $\lim_{n\to\infty} \frac{F(n)}{G(n)}$ exists and is a finite number then $F = O(G)$. If you are familiar with the notion of *supremum*, then you can verify that $F = O(G)$ if and only if $\limsup_{n\to\infty} \frac{F(n)}{G(n)} < \infty$.

[16] Recall that $\mathbb{R}_+$, which is also sometimes denoted as $(0, \infty)$, is the set of positive real numbers, so the above is just a way of saying that F and G's outputs are always positive numbers.

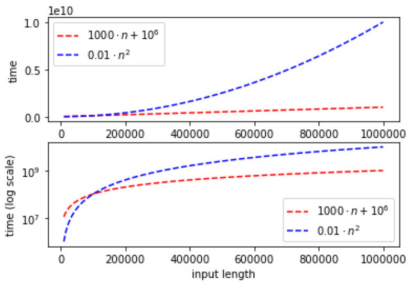


Figure 1.6: If $F(n) = o(G(n))$ then for sufficiently large n, $F(n)$ will be smaller than $G(n)$. For example, if Algorithm A runs in time $1000 \cdot n + 10^6$ and Algorithm B runs in time $0.01 \cdot n^2$ then even though B might be more efficient for smaller inputs, when the inputs get sufficiently large, A will run *much* faster than B.

Remark 1.20 — Big-O and equality. Using the equality sign for O-notation is extremely common, but is somewhat of a misnomer, since a statement such as $F = O(G)$ really means that F is in the set $\{G' : \exists_{N,c} \text{ s.t. } \forall_{n>N} G'(n) \leq cG(n)\}$.

If anything, it would have made more sense use *inequalities* and write $F \leq O(G)$ and $F \geq \Omega(G)$, reserving equality for $F = \Theta(G)$, and so we will sometimes use this notation, but since the equality notation is quite firmly entrenched we will often stick to it as well. (Some texts use the notation $F \in O(G)$ instead of $F = O(G)$, but we will not use this notation in this book.)

Despite the misleading equality sign, you should remember that a statement such as $F = O(G)$ means that F is "at most" G in some rough sense when we ignore constants, and a statement such as $F = \Omega(G)$ means that F is "at least" G in the same rough sense.

It's often convenient to use "anonymous functions" in the context of O-notation, and also to emphasize the input parameter to the function. For example, when we write a statement such as $F(n) = O(n^3)$, we mean that $F = O(G)$ where G is the function defined by $G(n) = n^3$. Chapter 7 in Jim Apsnes' notes on discrete math provides a good summary of O notation; see also this tutorial for a gentler and more programmer-oriented introduction.

1.5.10 Some "rules of thumb" for Big-O notation

There are some simple heuristics that can help when trying to compare two functions F and G:

- Multiplicative constants don't matter in O-notation, and so if $F(n) = O(G(n))$ then $100F(n) = O(G(n))$.
- When adding two functions, we only care about the larger one. For example, for the purpose of O-notation, $n^3 + 100n^2$ is the same as n^3, and in general in any polynomial, we only care about the larger exponent.
- For every two constants $a, b > 0$, $n^a = O(n^b)$ if and only if $a \leq b$, and $n^a = o(n^b)$ if and only if $a < b$. For example, combining the two observations above, $100n^2 + 10n + 100 = o(n^3)$.
- Polynomial is always smaller than exponential: $n^a = o(2^{n^\epsilon})$ for every two constants $a > 0$ and $\epsilon > 0$ even if ϵ is much smaller than a. For example, $100n^{100} = o(2^{\sqrt{n}})$.
- Similarly, logarithmic is always smaller than polynomial: $(\log n)^a$ (which we write as $\log^a n$) is $o(n^\epsilon)$ for every two constants $a, \epsilon > 0$. For example, combining the observations above, $100n^2 \log^{100} n = o(n^3)$.

In most (though not all!) cases we use O-notation, the constants hidden by it are not too huge and so on an intuitive level, you can think of $F = O(G)$ as saying something like $F(n) \leq 1000G(n)$ and $F = \Omega(G)$ as saying something $F(n) \geq 0.001G(n)$.

1.6 PROOFS

Many people think of mathematical proofs as a sequence of logical deductions that starts from some axioms and ultimately arrives at a conclusion. In fact, some dictionaries define proofs that way. This is not entirely wrong, but in reality a mathematical proof of a statement X is simply an argument that convinces the reader that X is true beyond a shadow of a doubt.

To produce such a proof you need to:

1. Understand precisely what X means.
2. Convince *yourself* that X is true.
3. Write your reasoning down in plain, precise and concise English (using formulas or notation only when they help clarity).

In many cases, Step 1 is the most important one. Understanding what a statement means is often more than halfway towards understanding why it is true. In Step 3, to convince the reader beyond a shadow of a doubt, we will often want to break down the reasoning to "basic steps", where each basic step is simple enough to be "self evident". The combination of all steps yields the desired statement.

1.6.1 Proofs and programs

There is a great deal of similarity between the process of writing *proofs* and that of writing *programs*, and both require a similar set of skills. Writing a *program* involves:

1. Understanding what is the *task* we want the program to achieve.
2. Convincing *yourself* that the task can be achieved by a computer, perhaps by planning on a whiteboard or notepad how you will break it up to simpler tasks.
3. Converting this plan into code that a compiler or interpreter can understand, by breaking up each task into a sequence of the basic operations of some programming language.

In programs as in proofs, step 1 is often the most important one. A key difference is that the reader for proofs is a human being and for programs is a compiler.[17] Thus our emphasis is on *readability* and having a *clear logical flow* for the proof (which is not a bad idea for programs as well...). When writing a proof, you should think of your audience as an intelligent but highly skeptical and somewhat petty reader, that will "call foul" at every step that is not well justified.

[17] This difference might be eroding with time, as more proofs are being written in a *machine verifiable form* and progress in artificial intelligence allows expressing programs in more human friendly ways, such as "programming by example". Interestingly, much of the progress in automatic proof verification and proof assistants relies on a much deeper correspondence between *proofs* and *programs*, see Chapter 21.

1.6.2 Proof writing style

A mathematical proof is a piece of writing, but it is a specific genre of writing with certain conventions and preferred styles. As in any writing, practice makes perfect, and it is also important to revise your drafts for clarity.

In a proof for the statement X, all the text between the words **"Proof:"** and **"QED"** should be focused on establishing that X is true. Digressions, examples, or ruminations should be kept outside these two words, so they do not confuse the reader. The proof should have a clear logical flow in the sense that every sentence or equation in it should have some purpose and it should be crystal-clear to the reader what this purpose is. When you write a proof, for every equation or sentence you include, ask yourself:

1. Is this sentence or equation stating that some statement is true?
2. If so, does this statement follow from the previous steps, or are we going to establish it in the next step?

3. What is the *role* of this sentence or equation? Is it one step towards proving the original statement, or is it a step towards proving some intermediate claim that you have stated before?

4. Finally, would the answers to questions 1-3 be clear to the reader? If not, then you should reorder, rephrase or add explanations.

Some helpful resources on mathematical writing include this handout by Lee, this handout by Hutching, as well as several of the excellent handouts in Stanford's CS 103 class.

1.6.3 Patterns in proofs

> *"If it was so, it might be; and if it were so, it would be; but as it isn't, it ain't. That's logic."*, Lewis Carroll, *Through the looking-glass*.

Just like in programming, there are several common patterns of proofs that occur time and again. Here are some examples:

Proofs by contradiction: One way to prove that X is true is to show that if X was false then we would get a contradiction as a result. Such proofs often start with a sentence such as "Suppose, towards a contradiction, that X is false" and end with deriving some contradiction (such as a violation of one of the assumptions in the theorem statement). Here is an example:

Lemma 1.21 There are no natural numbers a, b such that $\sqrt{2} = \frac{a}{b}$.

Proof. Suppose, towards the sake of contradiction that this is false, and so let $a \in \mathbb{N}$ be the smallest number such that there exists some $b \in \mathbb{N}$ satisfying $\sqrt{2} = \frac{a}{b}$. Squaring this equation we get that $2 = a^2/b^2$ or $a^2 = 2b^2$ $(*)$. But this means that a^2 is *even*, and since the product of two odd numbers is odd, it means that a is even as well, or in other words, $a = 2a'$ for some $a' \in \mathbb{N}$. Yet plugging this into $(*)$ shows that $4a'^2 = 2b^2$ which means $b^2 = 2a'^2$ is an even number as well. By the same considerations as above we gat that b is even and hence $a/2$ and $b/2$ are two natural numbers satisfying $\frac{a/2}{b/2} = \sqrt{2}$, contradicting the minimality of a.

■

Proofs of a universal statement: Often we want to prove a statement X of the form "Every object of type O has property P." Such proofs often start with a sentence such as "Let o be an object of type O" and end by showing that o has the property P. Here is a simple example:

Lemma 1.22 For every natural number $n \in N$, either n or $n + 1$ is even.

Proof. Let $n \in N$ be some number. If $n/2$ is a whole number then we are done, since then $n = 2(n/2)$ and hence it is even. Otherwise, $n/2 + 1/2$ is a whole number, and hence $2(n/2 + 1/2) = n + 1$ is even. ■

Proofs of an implication: Another common case is that the statement X has the form "A implies B". Such proofs often start with a sentence such as "Assume that A is true" and end with a derivation of B from A. Here is a simple example:

Lemma 1.23 If $b^2 \geq 4ac$ then there is a solution to the quadratic equation $ax^2 + bx + c = 0$.

Proof. Suppose that $b^2 \geq 4ac$. Then $d = b^2 - 4ac$ is a non-negative number and hence it has a square root s. Thus $x = (-b + s)/(2a)$ satisfies

$$\begin{aligned} ax^2 + bx + c &= a(-b+s)^2/(4a^2) + b(-b+s)/(2a) + c \\ &= (b^2 - 2bs + s^2)/(4a) + (-b^2 + bs)/(2a) + c \,. \end{aligned} \tag{1.18}$$

■

Rearranging the terms of (1.18) we get

$$s^2/(4a) + c - b^2/(4a) = (b^2 - 4ac)/(4a) + c - b^2/(4a) = 0 \tag{1.19}$$

Proofs of equivalence: If a statement has the form "A if and only if B" (often shortened as "A iff B") then we need to prove both that A implies B and that B implies A. We call the implication that A implies B the "only if" direction, and the implication that B implies A the "if" direction.

Proofs by combining intermediate claims: When a proof is more complex, it is often helpful to break it apart into several steps. That is, to prove the statement X, we might first prove statements X_1,X_2, and X_3 and then prove that $X_1 \wedge X_2 \wedge X_3$ implies X.[18]

[18] As mentioned below, $\wedge$ denotes the logical AND operator.

Proofs by case distinction: This is a special case of the above, where to prove a statement X we split into several cases $C_1, \ldots, C_k$, and prove that **(a)** the cases are *exhaustive*, in the sense that *one* of the cases C_i must happen and **(b)** go one by one and prove that each one of the cases C_i implies the result X that we are after.

Proofs by induction: We discuss induction and give an example in Section 1.7.1 below. We can think of such proofs as a variant of the above, where we have an unbounded number of intermediate claims $X_0, X_2, \ldots, X_k$, and we prove that X_0 is true, as well that X_0 implies X_1, and that $X_0 \wedge X_1$ implies X_2, and so on and so forth. The website for CMU course 15-251 contains a useful handout on potential pitfalls when making proofs by induction.

Without loss of generality (w.l.o.g): This term can be initially quite confusing to students. It is essentially a way to simplify proofs by case distinctions. The idea is that if Case 1 is equal to Case 2 up to a change of variables or a similar transformation, then the proof of Case 1 will also imply the proof of case 2. It is always a statement that should be viewed with suspicion. Whenever you see it in a proof, ask yourself if you understand *why* the assumption made is truly without loss of generality, and when you use it, try to see if the use is indeed justified. When writing a proof, sometimes it might be easiest to simply repeat the proof of the second case (adding a remark that the proof is very similar to the first one).

R

Remark 1.24 — Hierarchical Proofs (optional). Mathematical proofs are ultimately written in English prose. The well-known computer scientist Leslie Lamport argues that this is a problem, and proofs should be written in a more formal and rigorous way. In his manuscript he proposes an approach for *structured hierarchical proofs*, that have the following form:

- A proof for a statement of the form "If A then B" is a sequence of numbered claims, starting with the assumption that A is true, and ending with the claim that B is true.
- Every claim is followed by a proof showing how it is derived from the previous assumptions or claims.
- The proof for each claim is itself a sequence of subclaims.

The advantage of Lamport's format is that it is very clear for every sentence in the proof what is the role that it plays. It is also much easier to transform such proofs into machine-checkable forms. The disadvantage is that such proofs can be more tedious to read and write, with less differentiation between the important parts of the arguments versus the more routine ones.

1.7 EXTENDED EXAMPLE: TOPOLOGICAL SORTING

In this section we will prove the following: every directed acyclic graph (DAG, see Definition 1.12) can be arranged in layers so that for all directed edges $u \to v$, the layer of v is larger than the layer of u. This result is known as topological sorting and is used in many applications, including task scheduling, build systems, software package management, spreadsheet cell calculations, and many others (see Fig. 1.7). In fact, we will also use it ourselves later on in this book.

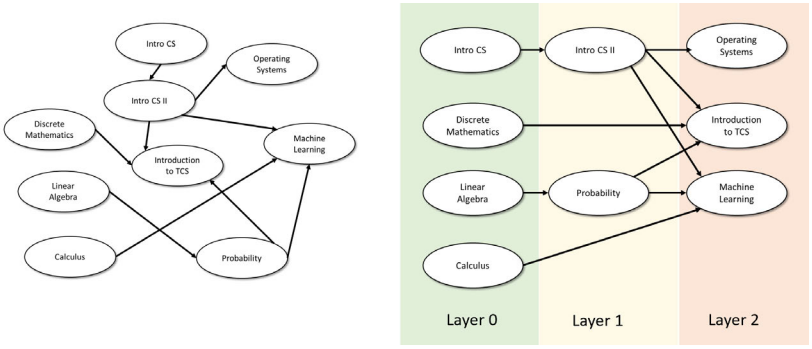


Figure 1.7: An example of *topological sorting*. We consider a directed graph corresponding to a prerequisite graph of the courses in some Computer Science program. The edge $u \to v$ means that the course u is a prerequisite for the course v. A *layering* or "topological sorting" of this graph is the same as mapping the courses to semesters so that if we decide to take the course v in semester $f(v)$, then we have already taken all the prerequisites for v (i.e., its in-neighbors) in prior semesters.

We start with the following definition. A *layering* of a directed graph is a way to assign for every vertex v a natural number (corresponding to its layer), such that v's in-neighbors are in lower-numbered layers than v, and v's out-neighbors are in higher-numbered layers. The formal definition is as follows:

Definition 1.25 — Layering of a DAG. Let $G = (V, E)$ be a directed graph. A *layering* of G is a function $f : V \to \mathbb{N}$ such that for every edge $u \to v$ of G, $f(u) < f(v)$.

In this section we prove that a directed graph is acylic if and only if it has a valid layering.

Theorem 1.26 — Topological Sort. Let G be a directed graph. Then G is acyclic if and only if there exists a layering f of G.

To prove such a theorem, we need to first understand what it means. Since it is an "if and only if" statement, Theorem 1.26 corresponds to two statements:

Lemma 1.27 For every directed graph G, if G is acyclic then it has a layering.

Lemma 1.28 For every directed graph G, if G has a layering, then it is acyclic.

To prove Theorem 1.26 we need to prove both Lemma 1.27 and Lemma 1.28. Lemma 1.28 is actually not that hard to prove. Intuitively, if G contains a *cycle*, then it cannot be the case that all edges on the cycle increase in layer number, since if we travel along the cycle at some point we must come back to the place we started from. The formal proof is as follows:

Proof. Let $G = (V, E)$ be a directed graph and let $f : \mathbb{N} \to \mathbb{N}$ be a layering of G as per Definition 1.25 . Suppose, towards the sake of

contradiction, that G is not acyclic, and hence there exists some cycle $u_0, u_1, \ldots, u_k$ such that $u_0 = u_k$ and for every $i \in [k]$ the edge $u_i \to u_{i+1}$ is present in G. Since f is a layering, for every $i \in [k]$, $f(u_i) < f(u_{i+1})$, which means that

$$f(u_0) < f(u_1) < \cdots < f(u_k) \tag{1.20}$$

but this is a contradiction since $u_0 = u_k$ and hence $f(u_0) = f(u_k)$. ■

Lemma 1.27 corresponds to the more difficult (and useful) direction. To prove it, we need to show how given an arbitrary DAG G, we can come up with a layering of the vertices of G so that all edges "go up".



If you have not seen the proof of this theorem before (or don't remember it), this would be an excellent point to pause and try to prove it yourself. One way to do it would be to describe an *algorithm* that on input a directed acyclic graph G on n vertices and $n - 2$ or fewer edges, constructs an array F of length n such that for every edge $u \to v$ in the graph $F[u] < F[v]$.

1.7.1 Mathematical induction

There are several ways to prove Lemma 1.27. One approach to do is to start by proving it for small graphs, such as graphs with 1,2 or 3 vertices (see **??**), for which we can check all the cases, and then try to extend the proof for larger graphs. The technical term for this proof approach is *proof by induction*.

Induction is simply an application of the self-evident Modus Ponens rule that says that if

(a) P is true
and

(b) P implies Q
then Q is true.

In the setting of proofs by induction we typically have a statement $Q(k)$ that is parameterized by some integer k, and we prove that **(a)** $Q(0)$ is true, and **(b)** For every $k > 0$, if $Q(0), \ldots, Q(k-1)$ are all true then $Q(k)$ is true.[19] By applying Modus Ponens, we can deduce from **(a)** and **(b)** that $Q(1)$ is true. Once we did so, since we now know that both $Q(0)$ and $Q(1)$ are true, then we can use this and **(b)** to deduce (again using Modus Ponens) that $Q(2)$ is true. We can repeat the same reasoning again and again to obtain that $Q(k)$ is true for every

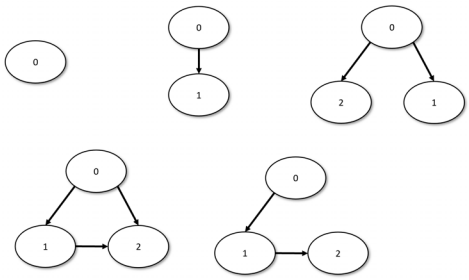


Figure 1.8: Some examples of DAGs of one, two and three vertices, and valid ways to assign layers to the vertices.

[19] Usually proving **(b)** is the hard part, though there are examples where the "base case" **(a)** is quite subtle.

k. The statement **(a)** is called the "base case", while **(b)** is called the "inductive step". The assumption in **(b)** that $Q(i)$ holds for $i < k$ is called the "inductive hypothesis".[20]

[20] The form of induction described here is sometimes called "strong induction" as opposed to "weak induction" where we replace **(b)** by the statement **(b')** that if $Q(k-1)$ is true then $Q(k)$ is true. Weak induction can be thought of as the special case of strong induction where we don't use the assumption that $Q(0), \ldots, Q(k-2)$ are true.



Remark 1.29 — Induction and recursion. Proofs by inductions are closely related to algorithms by recursion. In both cases we reduce solving a larger problem to solving a smaller instance of itself. In a recursive algorithm to solve some problem P on an input of length k we ask ourselves "what if someone handed me a way to solve P on instances smaller than k?". In an inductive proof to prove a statement Q parameterized by a number k, we ask ourselves "what if I already knew that $Q(k')$ is true for $k' < k$". Both induction and recursion are crucial concepts for this course and Computer Science at large (and even other areas of inquiry, including not just mathematics but other sciences as well). Both can be initially (and even post-initially) confusing, but with time and practice they become clearer. For more on proofs by induction and recursion, you might find the following Stanford CS 103 handout, this MIT 6.00 lecture or this excerpt of the Lehman-Leighton book useful.

1.7.2 Proving the result by induction

There are several ways to use induction to prove Lemma 1.27 by induction. We will use induction on the number n of vertices, and so we will define the statement $Q(n)$ as follows:

> $Q(n)$ is *"For every DAG $G = (V, E)$ with n vertices, there is a layering of G."*

The statement $Q(1)$ is trivial, since in this case G contains no edge, and we can simply use the layering $f(v) = 0$ (where v is the single vertex of G). Thus it will suffice to prove the following: *for every $n > 1$, if $Q(n-1)$ is true then $Q(n)$ is true.*

To do so, we need to somehow find a way, given a graph G of n vertices, to reduce the task of finding a layering for G into the task of finding a layering for some other graph G' of $n-1$ vertices. The idea is that we will find a *source* of G: a vertex v that has no in-neighbors. We can then assign to v the layer 0, and layer the remaining vertices using the inductive hypothesis in layers $1, 2, \ldots$.

The above is the intuition behind the proof, but when writing the proof, we use the benefit of hindsight, and try to streamline what was a messy journey into a linear and easy-to-follow flow of logic that starts with the word **"Proof:"** and ends with **"QED"** or the symbol ■.[21] All our discussions, examples and digressions can be very in-

[21] QED stands for "quod erat demonstrandum", which is "What was to be demonstrated." or "The very thing it was required to have shown." in Latin.

sightful, but we keep them outside the space delimited between these two words, where (as described by this excellent handout) "every sentence must be load bearing". Just like we do in programming, we can break the proof into little "subroutines" or "functions" (known as *lemmas* or *claims* in math language), which will be smaller statements that help us prove the main result. However, it should always be crystal-clear to the reader in what stage we are of the proof. Just like it should always be clear to which function a line of code belongs to, it should always be clear whether an individual sentence is part of a proof of some intermediate result, or is part of the argument showing that this intermediate result implies the theorem. Sometimes we highlight this partition by noting after each occurrence of **"QED"** to which lemma or claim it belongs.

We now present the formal proof.

Proof of Lemma 1.27. Let $G = (V, E)$ be a DAG and $n = |V|$ be the number of its vertices. We prove the lemma by induction on n. The base case is $n = 1$. In this case $V = \{v\}$ for a single vertex v, and we can use the layering defined as $f(v) = 0$. For the case of $n > 1$, we make the inductive hypothesis that every DAG G' of at most $n - 1$ vertices has a layering.

We make the following claim:

Claim: G must contain a vertex v of in-degree zero.

Proof of Claim: Suppose otherwise that every vertex $v \in V$ has an in-neighbor. Let v_0 be some vertex of G, let v_1 be an in-neighbor of v_0, v_2 be an in-neighbor of v_1, and continue in this way for n steps until we construct a list $v_0, v_1, \ldots, v_n$ such that for every $i \in [n]$, v_{i+1} is an in-neighbor of v_i, or in other words the edge $v_{i+1} \to v_i$ is present in the graph. Since there are only n vertices in this graph, one of the $n + 1$ vertices in this sequence must repeat itself, and so there exists $i < j$ such that $v_i = v_j$. But then the sequence $v_j \to v_{j-1} \to \cdots \to v_i$ is a cycle in G, contradicting our assumption that it is acyclic. (**QED Claim**)

Given the claim, we can let v_0 be some vertex of in-degree zero in G, and let G' be the graph obtained by removing v_0 from G. G' has $n - 1$ vertices and hence per the inductive hypothesis has a layering $f' : (V \setminus \{v_0\}) \to \mathbb{N}$. We define $f : V \to \mathbb{N}$ as follows:

$$f(v) = \begin{cases} f'(v) + 1 & v \neq v_0 \\ 0 & v = v_0 \end{cases} . \tag{1.21}$$

We claim that f is a valid layering, namely that for every edge $u \to v$, $f(u) < f(v)$. To prove this, we split into cases:

- **Case 1:** $u \neq v_0$, $v \neq v_0$. In this case the edge $u \to v$ exists in the graph G' and hence by the inductive hypothesis $f'(u) < f'(v)$ which implies that $f'(u) + 1 < f'(v) + 1$.

- **Case 2:** $u = v_0$, $v \neq v_0$. In this case $f(u) = 0$ and $f(v) = f'(v) + 1 > 0$.
- **Case 3:** $u \neq v_0$, $v = v_0$. This case can't happen since v_0 does not have in-neighbors.

Thus f is a valid layering for G which completes the proof.[22] ■

[22] If we were being really pedantic we would have a fourth case for $u = v_0$ and $v = v_0$ but that would correspond to a self loop and we always assume our graphs do not have those unless explicitly stated otherwise.



Reading a proof is no less of an important skill than producing one. In fact, just like understanding code, it is a highly non-trivial skill in itself. Therefore I strongly suggest that you re-read the above proof, asking yourself at every sentence whether the assumption it makes are justified, and whether this sentence truly demonstrates what it purports to achieve. Another good habit is to ask yourself when reading a proof for every variable you encounter (such as u, i, G', f' etc. in the above proof) the following questions: **(1)** What *type* of variable is it? is it a number? a graph? a vertex? a function? and **(2)** What do we know about it? Is it an arbitrary member of the set? Have we shown some facts about it?, and **(3)** What are we *trying* to show about it?.

1.7.3 Minimality and uniqueness

Theorem 1.26 guarantees that for every DAG $G = (V, E)$ there exists some layering $f : V \to \mathbb{N}$ but this layering is not necessarily *unique*. For example, if $f : V \to \mathbb{N}$ is a valid layering of the graph then so is the function f' defined as $f'(v) = 2 \cdot f(v)$. However, it turns out that the *minimal* layering is unique. A minimal layering is one where every vertex is given the absolute smallest layer number that we possibly can. We now formally define minimality and state the uniqueness theorem:

Theorem 1.30 — Minimal layering is unique. Let $G = (V, E)$ be a DAG. We say that a layering $f : V \to \mathbb{N}$ is *minimal* if for every vertex $v \in V$, if v has no in-neighbors then $f(v) = 0$ and if v has in-neighbors then there exists an in-neighbor u of v such that $f(u) = f(v) - 1$.

For every layering $f, g : V \to \mathbb{N}$ of G, if both f and g are minimal then $f = g$.

The definition of minimality in Theorem 1.30 implies that for every vertex $v \in V$, we cannot move it to a lower layer without making the layering invalid. If v is a source (i.e., has in-degree zero) then a minimal layering f must put it in layer 0, and for every other v, if

$f(v) = i$, then we cannot modify this to set $f(v) \leq i - 1$ since there is an-neighbor u of v satisfying $f(u) = i - 1$. What Theorem 1.30 says is that a minimal layering f is *unique* in the sense that every other minimal layering is equal to f.

Proof Idea:

The idea is to prove the theorem by induction on the layers. If f and g are minimal then they must agree on the source vertices, since both f and g should assign these vertices to layer 0. We can then show that if f and g agree up to layer $i - 1$, then the minimality property implies that they need to agree in layer i as well. In the actual proof we use a small trick to save on writing. Rather than proving the statement that $f = g$ (or in other words that $f(v) = g(v)$ for every $v \in V$), we prove the weaker statement that $f(v) \leq g(v)$ for every $v \in V$. (This is a weaker statement since the condition that $f(v)$ is lesser or equal than to $g(v)$ is implied by the condition that $f(v)$ is equal to $g(v)$.) However, since f and g are just labels we give to two minimal layerings, by simply changing the names "f" and "g" the same proof also shows that $g(v) \leq f(v)$ for every $v \in V$ and hence that $f = g$.

⋆

Proof of Theorem 1.30. Let $G = (V, E)$ be a DAG and $f, g : V \to \mathbb{N}$ be two minimal valid layering of G. We will prove that for every $v \in V$, $f(v) \leq g(v)$. Since we didn't assume anything about f, g except their minimality, the same proof will imply that for every $v \in V$, $g(v) \leq f(v)$ and hence that $f(v) = g(v)$ for every $v \in V$, which is what we needed to show.

We will prove that $f(v) \leq g(v)$ for every $v \in V$ by induction on $i = f(v)$. The case $i = 0$ is immediate: since in this case $f(v) = 0$, $g(v)$ must be at least $f(v)$. For the case $i > 0$, by the minimality of f, if $f(v) = i$ then there must exist some in-neighbor u of v such that $f(u) = i - 1$. By the induction hypothesis we get that $g(u) \geq i - 1$, and since g is a valid layering it must hold that $g(v) > g(u)$ which means that $g(v) \geq i = f(v)$.

■

P

The proof of Theorem 1.30 is fully rigorous, but is written in a somewhat terse manner. Make sure that you read through it and understand *why* this is indeed an airtight proof of the Theorem's statement.

1.8 NOTATION AND CONVENTIONS

Most of the notation we use in this book is standard and is used in most mathematical texts. The main points where we diverge are:

- We index the natural numbers $\mathbb{N}$ starting with 0 (though many other texts, especially in computer science, do the same).
- We also index the set $[n]$ starting with 0, and hence define it as $\{0,\ldots,n-1\}$. In other texts it is often defined as $\{1,\ldots,n\}$. Similarly, we index coordinates of our strings starting with 0, and hence a string $x \in \{0,1\}^n$ is written as $x_0 x_1 \cdots x_{n-1}$.
- If n is a natural number then 1^n does *not* equal the number 1 but rather this is the length n string $11\cdots1$ (that is a string of n ones). Similarly, 0^n refers to the length n string $00\cdots0$.
- *Partial* functions are functions that are not necessarily defined on all inputs. When we write $f : A \to B$ this means that f is a *total* function unless we say otherwise. When we want to emphasize that f can be a partial function, we will sometimes write $f : A \to_p B$.
- As we will see later on in the course, we will mostly describe our computational problems in the terms of computing a *Boolean function* $f : \{0,1\}^* \to \{0,1\}$. In contrast, many other textbooks refer to the same task as *deciding a language* $L \subseteq \{0,1\}^*$. These two viewpoints are equivalent, since for every set $L \subseteq \{0,1\}^*$ there is a corresponding function F such that $F(x) = 1$ if and only if $x \in L$. Computing *partial functions* corresponds to the task known in the literature as a solving a *promise problem*.[23]
- We use $\lceil x \rceil$ and $\lfloor x \rfloor$ for the "ceiling" and "floor" operators that correspond to "rounding up" or "rounding down" a number to the nearest integer. We use $(x \mod y)$ to denote the "remainder" of x when divided by y. That is, $(x \mod y) = x - y\lfloor x/y \rfloor$. In context when an integer is expected we'll typically "silently round" the quantities to an integer. For example, if we say that x is a string of length $\sqrt{n}$ then this means that x is of length $\lceil \sqrt{n} \rceil$. (We round up for the sake of convention, but in most such cases, it will not make a difference whether we round up or down.)
- Like most Computer Science texts, we default to the logarithm in base two. Thus, $\log n$ is the same as $\log_2 n$.
- We will also use the notation $f(n) = poly(n)$ as a short hand for $f(n) = n^{O(1)}$ (i.e., as shorthand for saying that there are some constants a, b such that $f(n) \leq a \cdot n^b$ for every sufficiently large n). Similarly, we will use $f(n) = polylog(n)$ as shorthand for

[23] Because the language notation is so prevalent in other textbooks, we will occasionally remind the reader of this correspondence.

$f(n) = poly(\log n)$ (i.e., as shorthand for saying that there are some constants a, b such that $f(n) \leq a \cdot (\log n)^b$ for every sufficiently large n).

- As in often the case in mathematical literature, we use the apostrophe character to enrich our set of identifier. Typically if x denotes some object, then x', x'', etc. will denote other objects of the same type.

- To save on "cognitive load" we will often use round constants such as $10, 100, 1000$ in the statements of both theorems and problem set questions. When you see such a "round" constant, you can typically assume that it has no special significance and was just chosen arbitrarily. For example, if you see a theorem of the form "Algorithm A takes at most $1000 \cdot n^2$ steps to compute function F on inputs of length n" then probably the number 1000 is an abitrary sufficiently large constant, and one could prove the same theorem with a bound of the form $c \cdot n^2$ for a constant c that is smaller than 1000. Similarly, if a problem-set question asks you to prove that some quantity is at least $n/100$, it is quite possible that in truth the quantity is at least n/d for some constant d that is smaller than 100.

1.8.1 Conventions

Like programming, mathematics is full of *variables*. Whenever you see a variable, it is always important to keep track of what is its *type* (e.g., whether the variable is a number, a string, a function, a graph, etc.). To make this easier, we try to stick to certain conventions and consistently use certain identifiers for variables of the same type. Some of these conventions are listed in Section 1.8.1 below. These conventions are not immutable laws and we might occasionally deviate from them. Also, such conventions do not replace the need to explicitly declare for each new variable the type of object that it denotes.

Table 1.2: Conventions for identifiers in this book

Identifier	*Often denotes object of type*
i,j,k,ℓ,m,n	Natural numbers (i.e., in $\mathbb{N} = \{0, 1, 2, ...\}$)
ϵ, δ	Small positive real numbers (very close to 0)
x, y, z, w	Typically strings in $\{0,1\}^*$ though sometimes numbers or other objects. We often identify an object with its representation as a string.
G	A *graph*. The set of G's vertices is typically denoted by V. Often $V = [n]$. The set of G's edges is typically denoted by E.
S	Set

Identifier	*Often denotes object of type*
f, g, h	Functions. We often (though not always) use lowercase identifiers for *finite functions*, which map $\{0,1\}^n$ to $\{0,1\}^m$ (often $m = 1$).
F, G, H	Infinite (unbounded input) functions mapping $\{0,1\}^*$ to $\{0,1\}^*$ or $\{0,1\}^*$ to $\{0,1\}^m$ for some m. Based on context, the identifiers G, H are sometimes used to denote functions and sometimes graphs.
A, B, C	Boolean circuits
M, N	Turing machines
P, Q	Programs
T	A function mapping $\mathbb{N}$ to $\mathbb{N}$ that corresponds to a time bound.
c	A positive number (often an unspecified constant; e.g., $T(n) = O(n)$ corresponds to the existence of c s.t. $T(n) \leq c \cdot n$ every $n > 0$). We sometimes use a, b in a similar way.
Σ	Finite set (often used as the *alphabet* for a set of strings).



Lecture Recap

- The basic "mathematical data structures" we'll need are *numbers*, *sets*, *tuples*, *strings*, *graphs* and *functions*.
- We can use basic objects to define more complex notions. For example, *graphs* can be defined as a list of *pairs*.
- Given precise *definitions* of objects, we can state unambiguous and precise *statements*. We can then use mathematical *proofs* to determine whether these statements are true or false.
- A mathematical proof is not a formal ritual but rather a clear, precise and "bulletproof" argument certifying the truth of a certain statement.
- Big-O notation is an extremely useful formalism to suppress less significant details and allow us to focus on the high level behavior of quantities of interest.
- The only way to get comfortable with mathematical notions is to apply them in the contexts of solving problems. You should expect to need to go back time and again to the definitions and notation in this lecture as you work through problems in this course.

1.9 EXERCISES

Exercise 1.1 — Logical expressions. a. Write a logical expression $\varphi(x)$ involving the variables x_0, x_1, x_2 and the operators $\wedge$ (AND), $\vee$ (OR), and $\neg$ (NOT), such that $\varphi(x)$ is true if the majority of the inputs are *True*.

b. Write a logical expression $\varphi(x)$ involving the variables x_0, x_1, x_2 and the operators $\wedge$ (AND), $\vee$ (OR), and $\neg$ (NOT), such that $\varphi(x)$ is true if the sum $\sum_{i=0}^{2} x_i$ (identifying "true" with 1 and "false" with 0) is *odd*.

■

Exercise 1.2 — Quantifiers. Use the logical quantifiers $\forall$ (for all), $\exists$ (there exists), as well as $\wedge, \vee, \neg$ and the arithmetic operations $+, \times, =, >, <$ to write the following:

a. An expression $\varphi(n, k)$ such that for every natural numbers n, k, $\varphi(n, k)$ is true if and only if k divides n.

b. An expression $\varphi(n)$ such that for every natural number n, $\varphi(n)$ is true if and only if n is a power of three.

■

Exercise 1.3 Describe the following statement in English words: $\forall_{n\in\mathbb{N}}\exists_{p>n}\forall a, b \in \mathbb{N}(a \times b \neq p) \vee (a = 1)$.

■

Exercise 1.4 — Set construction notation. Describe in words the following sets:

a. $S = \{x \in \{0,1\}^{100} : \forall_{i\in\{0,\ldots,99\}} x_i = x_{99-i}\}$

b. $T = \{x \in \{0,1\}^* : \forall_{i,j\in\{2,\ldots,|x|-1\}} i \cdot j \neq |x|\}$

■

Exercise 1.5 — Existence of one to one mappings. For each one of the following pairs of sets (S, T), prove or disprove the following statement: there is a one to one function f mapping S to T.

a. Let $n > 10$. $S = \{0,1\}^n$ and $T = [n] \times [n] \times [n]$.

b. Let $n > 10$. S is the set of all functions mapping $\{0,1\}^n$ to $\{0,1\}$. $T = \{0,1\}^{n^3}$.

c. Let $n > 100$. $S = \{k \in [n] \mid k \text{ is prime}\}$, $T = \{0,1\}^{\lceil \log n - 1 \rceil}$.

■

Exercise 1.6 — Inclusion Exclusion. a. Let A, B be finite sets. Prove that $|A \cup B| = |A| + |B| - |A \cap B|$.

b. Let $A_0, \ldots, A_{k-1}$ be finite sets. Prove that $|A_0 \cup \cdots \cup A_{k-1}| \geq \sum_{i=0}^{k-1} |A_i| - \sum_{0 \leq i < j < k} |A_i \cap A_j|$.

c. Let $A_0, \ldots, A_{k-1}$ be finite subsets of $\{1, \ldots, n\}$, such that $|A_i| = m$ for every $i \in [k]$. Prove that if $k > 100n$, then there exist two distinct sets A_i, A_j s.t. $|A_i \cap A_j| \geq m^2/(10n)$.

■

Exercise 1.7 Prove that if S, T are finite and $F : S \to T$ is one to one then $|S| \leq |T|$.

■

Exercise 1.8 Prove that if S, T are finite and $F : S \to T$ is onto then $|S| \geq |T|$.

■

Exercise 1.9 Prove that for every finite S, T, there are $(|T|+1)^{|S|}$ partial functions from S to T.

■

Exercise 1.10 Suppose that $\{S_n\}_{n \in \mathbb{N}}$ is a sequence such that $S_0 \leq 10$ and for $n > 1$ $S_n \leq 5S_{\lfloor \frac{n}{5} \rfloor} + 2n$. Prove by induction that $S_n \leq 100n \log n$ for every n.

■

Exercise 1.11 Prove that for every undirected graph G of 100 vertices, if every vertex has degree at most 4, then there exists a subset S of at 20 vertices such that no two vertices in S are neighbors of one another.

■

Exercise 1.12 — O-notation. For every pair of functions F, G below, determine which of the following relations holds: $F = O(G)$, $F = \Omega(G)$, $F = o(G)$ or $F = \omega(G)$.

a. $F(n) = n$, $G(n) = 100n$.

b. $F(n) = n$, $G(n) = \sqrt{n}$.

c. $F(n) = n \log n$, $G(n) = 2^{(\log(n))^2}$.

d. $F(n) = \sqrt{n}$, $G(n) = 2^{\sqrt{\log n}}$

e. $F(n) = \binom{n}{\lceil 0.2n \rceil}$, $G(n) = 2^{0.1n}$ (where $\binom{n}{k}$ is the number of k-sized subsets of a set of size n) and $g(n) = 2^{0.1n}$.[24]

[24] *Hint:* one way to do this is to use Stirling's approximation for the factorial function..

■

Exercise 1.13 Give an example of a pair of functions $F, G : \mathbb{N} \to \mathbb{N}$ such that neither $F = O(G)$ nor $G = O(F)$ holds.

■

Exercise 1.14 Prove that for every undirected graph G on n vertices, if G has at least n edges then G contains a cycle.

■

Exercise 1.15 Prove that for every undirected graph G of 1000 vertices, if every vertex has degree at most 4, then there exists a subset S of at least 200 vertices such that no two vertices in S are neighbors of one another.

■

1.10 BIBLIOGRAPHICAL NOTES

The section heading "A Mathematician's Apology", refers of course to Hardy's classic book [Har41]. Even when Hardy is wrong, he is very much worth reading.

There are many online sources for the mathematical background needed for this book. In particular, the lecture notes for MIT 6.042 "Mathematics for Computer Science" [LLM18] are extremely comprehensive, and videos and assignments for this course are available online. Similarly, Berkeley CS 70: "Discrete Mathematics and Probability Theoru" has extensive lecture notes online. The book of Rosen [Ros19] also covers much of this material. See also Jim Aspens' online book [Asp18].

The book by Lewis and Zax [LZ19], as well as the online book of Fleck [Fle18], give a more gentle overview of the much of the same material. Solow's book [Sol14] is a good gentle introduction to proof reading and writing. Kun's book [Kun18] gives an introduction to mathematics aimed at readers with programming background. Stanford's CS 103 course has a wonderful collections of handouts on mathematical proof techniques and discrete mathematics.

Learning Objectives:

- *Representing* an object as a string (often of zeroes and ones).
- Examples of representations for common objects such as numbers, vectors, lists, graphs.
- Prefix-free representations.
- Distinguish between *specification* and *implementation*, or equivalently between *algorithms/programs* and *mathematical functions*.

2
Computation and Representation

> *"The alphabet was a great invention, which enabled men to store and to learn with little effort what others had learned the hard way – that is, to learn from books rather than from direct, possibly painful, contact with the real world."*, B.F. Skinner

> *"The name of the song is called 'HADDOCK'S EYES.'"* [said the Knight]
>
> *"Oh, that's the name of the song, is it?"* Alice said, trying to feel interested.
>
> *"No, you don't understand,"* the Knight said, looking a little vexed. *"That's what the name is CALLED. The name really is 'THE AGED AGED MAN.'"*
>
> *"Then I ought to have said 'That's what the SONG is called'?"* Alice corrected herself.
>
> *"No, you oughtn't: that's quite another thing! The SONG is called 'WAYS AND MEANS': but that's only what it's CALLED, you know!"*
>
> *"Well, what IS the song, then?"* said Alice, who was by this time completely bewildered.
>
> *"I was coming to that,"* the Knight said. *"The song really IS 'A-SITTING ON A GATE': and the tune's my own invention."*
>
> Lewis Carroll, *Through the looking glass*

To a first approximation, __computation_ _ is a process that maps an *input* to an *output*.

When discussing computation, it is essential to separate the question of **what** is the task we need to perform (i.e., the *specification*) from the question of **how** we achieve this task (i.e., the *implementation*). For example, as we've seen, there is more than one way to achieve the computational task of computing the product of two integers.

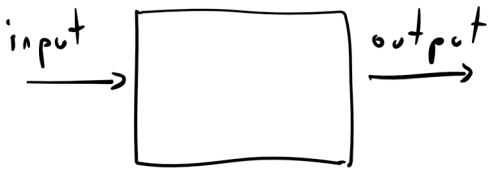


Figure 2.1: Our basic notion of *computation* is some process that maps an input to an output

In this chapter we focus on the **what** part, namely defining computational tasks. For starters, we need to define the inputs and outputs. A priori, defining all potential inputs and outputs seems challenging, since computation today is applied to a wide variety of objects. We do not compute merely on numbers, but also on texts, images, videos, connection graphs of social networks, MRI scans, gene data, and even other programs. We will represent all these objects as **strings of zeroes and ones**, that is objects such as 0011101 or 1011 or any other finite list of 1's and 0's.

Today, we are so used to the notion of digital representation that we are not surprised by the existence of such an encoding. Indeed, at the time of writing, the full contents of the English Wikipedia, including all the text and media, can be encoded in a string in $\{0,1\}^n$ for $n \sim 10^{12}$ (i.e., about 100 Gigabytes). However, this is in fact a deep insight with significant implications. Many animals can convey a particular fear or desire, but what is unique about humans is *language*: we use a finite collection of basic symbols to describe a potentially unlimited range of experiences. Language allows transmission of information over both time and space and enables societies that span a great many people and accumulate a body of shared knowledge over time.

Over the last several decades, we have seen a revolution in what we can represent and convey in digital form. We can capture experiences with almost perfect fidelity, and disseminate it essentially instantaneously to an unlimited audience. Moreover, once information is in digital form, we can *compute* over it, and gain insights from data that were not accessible in prior times. At the heart of this revolution is the simple but profound observation that we can represent an unbounded variety of objects using a finite set of symbols (and in fact using only the two symbols `0` and `1`).[1]

In later lectures, we will often fall back on taking this representation for granted, and hence write something like "program P takes x as input" when x might be a number, a vector, a graph, or any other objects, when we really mean that P takes as input the *representation* of x as a binary string. However, in this chapter we will dwell a bit more on how we can construct such representations.



Figure 2.2: We represent numbers, texts, images, networks and many other objects using strings of zeroes and ones. Writing the zeroes and ones themselves in green font over a black background is optional.

[1] There is nothing "holy" about using zero and one as the basic symbols, and we can (indeed sometimes people do) use any other finite set of two or more symbols as the fundamental "alphabet". We use zero and one in this course mainly because it simplifies notation.

2.1 DEFINING REPRESENTATIONS

Every time we store numbers, images, sounds, databases, or other objects on a computer, what we actually store in memory is the *representation* of these objects. Moreover, the idea of representation is not restricted to digital computers. When we write down text or a music sheet, we are *representing* ideas or experiences as sequences of symbols (which might as well be strings of zeroes and ones). Even our brain

does not store the actual sensory inputs we experience, but rather only our *representation* of them.

To use objects such as numbers, images, graphs, or others as inputs for computation, we need to define precisely how to represent these objects as binary strings. A *representation scheme* is a way to map an object x to a binary string $E(x) \in \{0,1\}^*$. For example, a representation scheme for natural numbers is a function $E : \mathbb{N} \to \{0,1\}^*$. Of course, we cannot merely represent all numbers as the string "0011" (for example). A minimal requirement is that if two numbers x and x' are different then they would be represented by different strings. That is, E is *one to one* (later on in this book we might need representations with additional properties, such as being easy to compute).

2.1.1 Representing natural numbers.

We start with the most simple setting of a representation scheme, which is the task of representing natural numbers. Over the years people have represented numbers in a variety of ways, including Roman numerals, the decimal system, as well as many other ways.

There are a great many ways to represent natural numbers as binary strings. For example, we could represent a number x by breaking it apart to its decimal digits, and represent each one as a binary string corresponding to its graphical representation (see Fig. 2.3). However, for the sake of concreteness, we will use the *binary basis* as our default representation of natural numbers as strings. For example, will represent the number six as the string $y = 110$, which satisfies $y_0 \cdot 2^2 + y_1 \cdot 2^1 + y_2 \cdot 2^0 = 6$, and similarly we represent the number thirty-five as the string $y' = 100011$ which satisfies $\sum_{i=0}^{5} y_i \cdot 2^{|y|-i} = 35$.[2]

Table 2.1: Representing numbers in the binary basis. The lefthand column contains representations of natural numbers in the decimal basis, while the righthand column contains representations of the same numbers in the binary basis. Note that in both representations the leftmost (i.e., most significant) digit is never equal to zero (unless we represent the natural number zero).

Number (decimal representation)	Number (binary representation)
503	111110111
53	110101
40	101000
16	10000
40	101000
801	1100100001
111	1101111
389	110000101

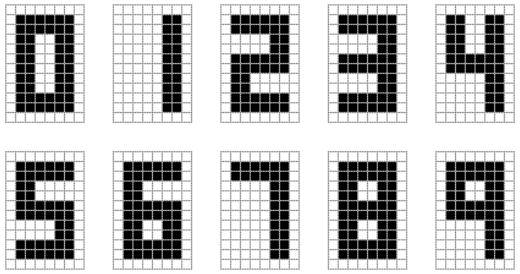


Figure 2.3: Representing each one the digits $0, 1, 2, \ldots, 9$ as a 12×8 bitmap image, which can be thought of as a string in $\{0,1\}^{96}$. Using this scheme we can represent a natural number x of n decimal digits as a string in $\{0,1\}^{96n}$. Image taken from blog post of A. C. Andersen.

[2] We could have equally well reversed the order so as to represent 35 by the string $y = 110001$ satisfying $\sum_i y_i \cdot 2^i = 35$. Such low level choices will not make a difference in this course. A related (though not identical) distinction is the Big Endian vs. Little Endian representation for integers in computing architectures.

Number (decimal representation)	Number (binary representation)
3750	111010100110
506	111111010

Formally, our representation is given by the following theorem:

Theorem 2.1 — Binary representation of natural numbers. There exists a one-to-one function $NtS : \mathbb{N} \to \{0,1\}^*$. [3]

[3] NtS stands for "numbers to strings".

Proof Idea:

This theorem is proven using the standard binary representation mentioned above. There are also many sources online that describe the binary representation, see also the Python code below.

⋆

Proof of Theorem 2.1. If x is even then the least significant binary digit of x is zero, while if x is odd, then the least significant binary digit is one. In either case, $\lfloor x/2 \rfloor$ corresponds to the number obtained when we "chop off" the least significant digit. Hence we can define NtS recursively as follows:

$$NtS(x) = \begin{cases} "" & x = 0 \\ NTS(\lfloor x/2 \rfloor) parity(x) & x > 0 \end{cases} \tag{2.1}$$

where $parity(x)$ is defined to equal 1 if x is odd and to equal 0 if x is even. (The function NtS is well defined since for every $x > 0$, $\lfloor x/2 \rfloor < x$.) If α and β are two strings, then $\alpha\beta$ corresponds to the string obtained by *concatenating* α and β (that is, the string γ of length $|\alpha| + |\beta|$ obtained by writing α first and then writing β). Thus in the definition above $NTS(\lfloor x/2 \rfloor) parity(x)$ corresponds to the *concetenation* of the string $NTS(\lfloor x/2 \rfloor)$ with the (one bit long) string $parity(x)$.

It can be shown (though we omit the proof since it is slightly tedious, and can be easily found in many online resources) that if $y = NtS(x)$ then $x = \sum_{i=0}^{n} y_i \cdot 2^{n-i}$ where $n = |y| - 1$. In particular, this gives a way to recover (or *decode*) the original x from the output $y = NtS(x)$ which means that NtS is one to one.

The above representation uses the empty string "" to represent the number 0. However, we can also represent this number using the string 0 as well. This choice will not make any difference for our purposes.

■

2.1.2 Implementing the representation in python:

In the *Python* programming language, we can compute the above encoding and decoding functions as follows:

```
from math import floor, log
def NtS(x):
    if x<1: return ""
    return NtS(floor(x/2))+str(x % 2)

print(NtS(236))
# 11101100

print(int2bits(19))
# 10011

def StN(y):
    x = 0
    n = len(y)-1
    for i in range(n+1):
        x += int(y[i])*(2**(n-i))
    return x

print(StN(NtS(246)))
# 236
```

We can also implement NtS non recursively as follows:

```
def NtS(x):
    def list2string(L): return "".join([str(e) for e in L])
    n = floor(log2(x))  # largest power of two smaller
     ↪ than x
    return list2string([ floor(x / 2**(n-i)) % 2 for i in
     ↪ range(n+1)])
```



Remark 2.2 — Programming examples. In this book, we will often illustrate our points by using programming languages to present certain computations. Our examples will be relatively short, and our point will always be to emphasize that certain computations can be achieved concretely, rather than focusing on any particular language features. We often use Python, but this choice is somewhat arbitrary. Indeed, one of the messages of this course is that all programming languages are in a certain precise sense *equivalent* to one another, and hence we could have just as well used JavaScript, C, COBOL, Visual Basic or even BrainF*ck.

This book is *not* about programming, and it is absolutely OK if you are not familiar with Python or do not follow the fine details of code examples such as the above. You might still find it instructive to try to parse these examples, with the help of websites such as Google or StackOverflow. In particular, the non-recursive implementation of the function `NtS` above uses the fact that the binary representation of a natural number x is the list $(\lfloor \frac{x}{2^i} \rfloor \mod 2)_{i=0,\ldots,\lfloor \log_2 x \rfloor}$, which in Python-speak is written as `[ floor(x / 2**i ) % 2 for i in range(floor(log2(x))+1)]`.

2.1.3 Meaning of representation

It is natural for us to think of 236 as the "actual" number, and of 11101100 as "merely" its representation. However, for most Europeans in the middle ages CCXXXVI would be the "actual" number and 236 (if they have even heard about it) would be the weird Hindu-Arabic positional representation.[4] When our AI robot overlords materialize, they will probably think of 11101100 as the "actual" number and of 236 as "merely" a representation that they need to use when they give commands to humans.

So what is the "actual" number? This is a question that philosophers of mathematics have pondered over the generations. Plato argued that mathematical objects exist in some ideal sphere of existence (that to a certain extent is more "real" than the world we perceive via our senses, as this latter world is merely the shadow of this ideal sphere). Thus in Plato's vision, the symbols 236 are merely notation for some ideal object, that, in homage to the late musician, we can refer to as "the number commonly represented by 236".

The Austrian philosopher Ludwig Wittgenstein, on the other hand, argued that mathematical objects do not exist at all, and the only things that exist are the actual splotches on paper that make up 236, 00110111 or CCXXXVI. In Wittgenstein's view, mathematics is merely about formal manipulation of symbols that do not have any inherent meaning. You can think of the "actual" number as (somewhat recursively) "that thing which is common to 236, 00110111 and CCXXXVI and all other past and future representations that are meant to capture the same object".

While reading this book, you are free to choose your own philosophy of mathematics, as long as you maintain the distinction between the mathematical objects themselves and the particular choice of representing them, whether as splotches of ink, pixels on a screen, zeroes and one, or any other form.

[4] While the Babylonians already invented a positional system much earlier, the decimal positional system we use today was invented by Indian mathematicians around the third century. It was taken up by Arab mathematicians in the 8th century. It was mainly introduced to Europe in the 1202 book *"Liber Abaci"* by Leonardo of Pisa, also known as Fibonacci, but did not displace Roman numerals in common usage until the 15th century.

2.2 REPRESENTING MORE OBJECTS

We now discuss how we can represent many other objects as strings, going far beyond natural numbers. In many instances, choosing the "right" string representation for a piece of data is highly nontrivial, and finding the "best" one (e.g., most compact, best fidelity, most efficiently manipulable, robust to errors, most informative features, etc.) is the object of intense research. But for now, we focus on presenting some simple representations for various objects that we would like to use as inputs and outputs for computation.

2.2.1 Representing (potentially negative) integers

Now that we can represent natural numbers, we can represent the full set of *integers* (i.e., members of the set $\mathbb{Z} = \{\ldots,-3,-2,-1,0,+1,+2,+3,\ldots\}$) by adding one more bit that represents the sign. If $x \in \mathbb{Z}$, then we can represent it by the string $NtS(|x|)\sigma$ where σ equals to 0 is $x \geq 0$ and σ equals to 1 if $x < 0$.

Thus the string $y \in \{0,1\}^*$ will represent the number

$$x = (-1)^{y_{n+1}} \left(\sum_{i=0}^{n-1} y_i \cdot 2^{n-i} \right) \tag{2.2}$$

where $n = |y| - 2$. Formally, the above can be shown to give a one to one function $ZtS : \mathbb{Z} \to \{0,1\}^*$ that maps the integers into strings.

The decoding function of a representation scheme is always *onto* since every object must be represented by some string. However, the decoding function is not always *one to one*. For example, in this particular representation scheme, the two strings 1 and 0 both represent the number zero (since they can be thought of as representing -0 and $+0$ respectively, can you see why?). We can also allow a *partial* decoding function for representations. For example, in the representation above there is no number that is represented by the empty string. But this is still a fine representation, since the decoding partial function is onto and the encoding function is the one-to-one total function $E : \mathbb{Z} \to \{0,1\}^*$ which maps an integer of the form $a \times k$, where $a \in \{\pm 1\}$ and $k \in \mathbb{N}$ to the bit $(-1)^a$ concatenated with the binary representation of k. That is, every integer can be represented as a string, and every two distinct integers have distinct representations.

R

Remark 2.3 — Interpretation and context. Given a string $y \in \{0,1\}^*$, how do we know if it's "supposed" to represent a (nonnegative) natural number or a (potentially negative) integer? For that matter, even if we know y is "supposed" to be an integer, how do we know what representation scheme it uses? The short answer is that we do not necessarily know this

information, unless it is supplied from the context.[5] We can treat the same string y as representing a natural number, an integer, a piece of text, an image, or a green gremlin. Whenever we say a sentence such as "let n be the number represented by the string y," we will assume that we are fixing some canonical representation scheme such as the ones above. The choice of the particular representation scheme will rarely matter, except that we want to make sure to stick with the same one for consistency.

[5] In programming language, the compiler or interpreter determines the representation of the sequence of bits corresponding to a variable based on the variable's *type*.

2.2.2 Representing rational numbers

We can represent a rational number of the form a/b by representing the two numbers a and b (again, this is not a unique representation, but this is fine). However, merely concatenating the representations of a and b will not work.[6] For example, recall that we represent 4 as 100 and 43 as 101011, but the concatenation 100101011 of these strings is also the concatenation of the representation 10010 of 18 and the representation 1011 of 11. Hence, if we used such simple concatenation then we would not be able to tell if the string 100101011 is supposed to represent $4/43$ or $18/11$.[7]

[6] Recall that the *concatenation* of two strings x and y is the string of length $|x| + |y|$ obtained by writing y after x.

[7] The above assumes we use the simple binary representation of natural numbers as strings. If we want to handle negative numbers, then we should add the sign bit as well, though it would not make any qualitative difference to this discussion.

The way to tackle this is to find a general representation for *pairs* of numbers. If we were using a pen and paper, we would just use a separator such as the symbol $\|$ to represent, for example, the pair consisting of the numbers represented by $(0,1)$ and $(1,1,0,0,0,1)$ as the length-9 string s "$01\|110001$". Using such separators is similar to the way we use spaces and punctuation to separate words in English. By adding a little redundancy, we can do just that in the digital domain. The idea is that we can map the three element set $\Sigma = \{0,1,\|\}$ to the four element set $\{0,1\}^2$ via the one-to-one map that takes 0 to 00, 1 to 11 and $\|$ to 01.

■ **Example 2.4 — Representing a rational number as a string.** Consider the rational number $r = 19/236$. In our convention, we represent 19 as the string 10011 and 236 as the string 11101100, and so we could represent r as the *pair* of strings $(10011, 11101100)$. We can then represent this pair as the length 14 string $10011\|11101100$ over the alphabet $\{0,1,\|\}$. Now, applying the map $0 \mapsto 00, 1 \mapsto 11, \| \mapsto 01$, we can represent the latter string as the length 28 string $s = 11000011110111111110011110000$ over the alphabet $\{0,1\}$. So we represent the rational number $r = 19/36$ be the binary string $s = 11000011110111111110011110000$.

More generally, we obtained a representation of the non-negative rational numbers as binary strings by composing the following representations:

1. Representing a non-negative rational number as a pair of natural numbers.
2. Representing a natural number by a string via the binary representation. (We can use the representation of integers to handle rational numbers that can be negative.)
3. Combining 1 and 2 to obtain a representation of a rational number as a pair of strings.
4. Representing a pair of strings over $\{0,1\}$ as a single string over $\Sigma = \{0,1,\|\}$.
5. Representing a string over Σ as a longer string over $\{0,1\}$.

The same idea can be used to represent triples, quadruples, and generally all tuples of strings as a single string (can you see why?). Indeed, this is one instance of a very general principle that we use time and again in both the theory and practice of computer science (for example, in Object Oriented programming):

Big Idea 1 If we can represent objects of type T as strings, then we can also represent more complex objects built out of T as strings (such as pairs or lists of elements in T, nested lists, and so on and so forth).

We will come back to this point when we discuss *prefix free encoding* in Section 2.4.2.

2.3 REPRESENTING REAL NUMBERS

The set of *real numbers* $\mathbb{R}$ contains all numbers including positive, negative, and fractional, as well as *irrational* numbers such as π or e. Every real number can be approximated by a rational number, and thus we can represent every real number x by a rational number a/b that is very close to x. For example, we can represent π by $22/7$ within an error of about 10^{-3}. If we want a smaller error (e.g., about 10^{-4}) then we can use $311/99$, and so on and so forth.

The above representation of real numbers via rational numbers that approximate them is a fine choice for a representation scheme. However, typically in computing applications, it is more common to use the *floating point representation scheme* to represent real numbers. In the floating point representation scheme we represent x by the

pair (a, b) of (positive or negative) integers of some prescribed sizes (determined by the desired accuracy) such that $a \times 2^b$ is closest to x.[8]

The reader might be (rightly) worried about this issue of approximation. In many (though not all) computational applications, one can make the accuracy tight enough so that this does not affect the final result, though sometimes we do need to be careful. This representation is called "floating point" because we can think of the number a as specifying a sequence of binary digits, and b as describing the location of the "binary point" within this sequence. The use of floating representation is the reason why in many programming systems printing the expression `0.1+0.2` will result in `0.30000000000000004` and not `0.3`, see here, here and here for more (see also Fig. 2.4).

Floating-point bugs can sometimes be no joking matter. A floating point error has been implicated in the explosion of the Ariane 5 rocket, a bug that cost more than 370 million dollars, and the failure of a U.S. Patriot missile to intercept an Iraqi Scud missile, costing 28 lives. Floating point is often problematic in financial applications as well.

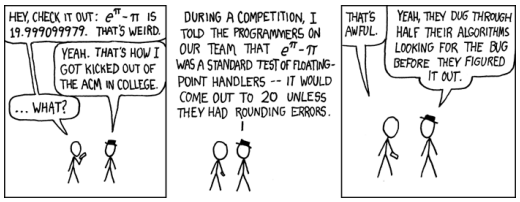


Figure 2.4: XKCD cartoon on floating-point arithmetic.

2.3.1 Can we represent reals *exactly*?

Given the issues with floating point representation, we could ask whether we could represent real numbers *exactly* as strings. Unfortunately, the following theorem shows that this cannot be done:

Theorem 2.5 — Reals are uncountable. There is no one-to-one function $RtS : \mathbb{R} \to \{0,1\}^*$. [9]

Theorem 2.5 was proven by Georg Cantor in 1874.[10] This result (and the theory around it) was quite shocking to mathematicians at the time. By showing that there is no one-to-one map from $\mathbb{R}$ to $\{0,1\}^*$ (or $\mathbb{N}$), Cantor showed that these two infinite sets have "different forms of infinity" and that the set of real numbers $\mathbb{R}$ is in some sense "bigger" than the infinite set $\{0,1\}^*$. The notion that there are "shades of infinity" was deeply disturbing to mathematicians and philosophers at the time. The philosopher Ludwig Wittgenstein (whom we mentioned before) called Cantor's results "utter nonsense" and "laughable." Others thought they were even worse than that. Leopold Kronecker called Cantor a "corrupter of youth," while Henri Poincaré said that Cantor's ideas "should be banished from mathematics once and for all." The tide eventually turned, and these days Cantor's work is universally accepted as the cornerstone of set theory and the foundations of mathematics. As David Hilbert said in 1925, *"No one shall expel us from the paradise which Cantor has created for us."* As we will see later in this book, Cantor's ideas also play a huge role in the theory of computation.

[9] RtS stands for "reals to strings".

[10] Cantor used the set $\mathbb{N}$ rather than $\{0,1\}^*$, but one can show that these two results are equivalent using the one-to-one maps between those two sets, see Exercise 2.12. Saying that there is no one-to-one map from $\mathbb{R}$ to $\mathbb{N}$ is equivalent to saying that there is no onto map $NtR : \mathbb{N} \to \mathbb{R}$ or, in other words, that there is no way to "count" all the real numbers as $NtR(0), NtR(1), NtR(2), \ldots$. For this reason Theorem 2.5 is known as the *uncountability of the reals*.

Now that we have discussed the theorem's importance, let us see the proof. The idea behind the proof is to do the following::

1. Define some infinite set $\mathcal{X}$ for which it is easier for us to prove that $\mathcal{X}$ is not countable: that is there is no one-to-one function from $\mathcal{X}$ to $\{0,1\}^*$.

2. Prove that there *is* a one-to-one function G mapping $\mathcal{X}$ to $\mathbb{R}$.

These two facts together imply Cantor's Theorem. We can show this using a "proof by contradiction." If we assume (towards the sake of contradiction) that there exists some one-to-one F mapping $\mathbb{R}$ to $\{0,1\}^*$ then the function $x \mapsto F(G(x))$ obtained by composing F with the function G from Step 2 above would be a one-to-one function from $\mathcal{X}$ to $\{0,1\}^*$, which contradicts what we proved in Step 1!

To turn this idea into a proof of Theorem 2.5 we need to:

- Define the set $\mathcal{X}$.
- Prove that there is no one-to-one function from $\mathcal{X}$ to $\{0,1\}^*$
- Prove that there *is* a one-to-one function from $\mathcal{X}$ to $\mathbb{R}$.

We now proceed to do precisely that. Namely, we will present a definition for a certain set and then state and prove two lemmas that show that this set satisfies our two desired properties.

2.3.2 { .definition title = "The set $\{0,1\}^\infty$" }

We define $\{0,1\}^\infty$ to be the set $\{f \mid f : \mathbb{N} \to \{0,1\}\}$.

That is, $\{0,1\}^\infty$ is a set of *functions*, and a function f is in $\{0,1\}^infty$ iff its domain is $\mathbb{N}$ and its codomain is $\{0,1\}$.[11] The set $\{0,1\}^\infty$ will play the role of $\mathcal{X}$ above. Namely, we will prove the following two results about it:

Lemma 2.6 There does not exist a one-to-one map $FtS : \{0,1\}^\infty \to \{0,1\}^*$.[12]

Lemma 2.7 There *does* exist a one-to-one map $FtR : \{0,1\}^\infty \to \mathbb{R}$.[13]

As we've seen above, Lemma 2.6 and Lemma 2.7 together imply Theorem 2.5. To repeat the argument more formally, suppose, for the sake of contradiction, that there did exist a one-to-one function $RtS : \mathbb{R} \to \{0,1\}^*$. By Lemma 2.7, there exists a one-to-one function $FtR : \{0,1\}^\infty \to \mathbb{R}$. Thus, under this assumption, since the composition of two one-to-one functions is one-to-one (see Exercise 2.11), the function $FtS : \{0,1\}^\infty \to \{0,1\}^*$ defined as $FtS(f) = RtS(FtR(f))$ will be one to one, contradicting Lemma 2.6. See **??** for a graphical illustration of this argument.

[11] We can also think of $\{0,1\}^\infty$ as the set of all infinite *sequences* of bits, since a function $f : \mathbb{N} \to \{0,1\}$ can be identified with the sequence $(f(0), f(1), f(2), ...)$.

[12] FtS stands for "functions to strings".

[13] FtR stands for "functions to reals."

Now all that is left is to prove these two lemmas. We start by proving Lemma 2.6 which is really the heart of Theorem 2.5.

Proof. We will prove that there does not exist an *onto* function $StF : \{0,1\}^* \to \{0,1\}^\infty$. This will imply the lemma since for every two sets A and B, there exists an onto function from A to B if and only if there exists a one-to-one function from B to A (see Lemma 1.6).

Let $StF : \{0,1\}^* \to \{0,1\}^\infty$ be any function mapping $\{0,1\}^*$ to $\{0,1\}^\infty$. We will prove that StF is *not* onto by showing that there exists some $f^* \in \{0,1\}^\infty$ that is *not* in the image of the function StF. Namely, $StF(x) \neq f^*$ for every $x \in \{0,1\}^*$.

The construction of f^* is short but subtle. For every number $n \in \mathbb{N}$, we let $x(n)$ be the string obtained by representing n in the binary basis and "chopping off" its most significant digit. We define $f^*(n)$ as follows:

$$f^*(n) = 1 - StF(x(n))(n) \tag{2.3}$$

As computer scientists, let's first verify that (2.3) "type checks". First of all, f^* is a member of $\{0,1\}^\infty$ and so for every $n \in \mathbb{N}$, $f^*(n)$ should be a bit in $\{0,1\}$, and so for (2.3) to "type check" we need its right-hand side to also be a bit. For every n, $x(n)$ is a string in $\{0,1\}^*$ and so $StF(x(n))$ is a function $g \in \{0,1\}^i nfty$. If we apply the function $g = StF(x(n))$ to n we get a bit $b \in \{0,1\}$ and so $1-b$ is indeed also a bit as we needed it to be.

Now we want to prove that for every $x \in \{0,1\}^*$, $StF(x) \neq f^*$. Indeed, suppose (towards a contradiction) that there did exist some $x \in \{0,1\}^*$ such that

$$StF(x) = f^* \ . \tag{2.4}$$

Then, if we let n be the number whose binary representation is $1x$, we see that one the one hand by (2.4) $f^*(n) = StF(x)(n)$ but on the other hand (since $x(n) = x$) by (2.3)

$$f^*(n) = 1 - StF(x)(n) \ . \tag{2.5}$$

We obtained that $f^*(n)$ is equal to both $StF(x)(n)$ and to one minus the same quantity which is clearly a contradiction!

■

P

The proof of Lemma 2.7 is rather subtle, and worth re-reading a second or third time. It is known as the "diagonal" argument, as the construction of f^* can be thought of as going over the diagonal elements of a table that in the n-th row and m-column contains $StF(x)(m)$ where x is the string such that $n(x) = n$, see **??**. We will use the diagonal argument again several times later on in this book.

R

Remark 2.8 — Generalizing beyond strings and reals. Lemma 2.6 doesn't really have much to do with the natural numbers or the strings. An examination of the proof shows that it really shows that for *every* set S, there is no one-to-one map $F : \{0,1\}^S \to S$ where $\{0,1\}^S$ denotes the set $\{f \mid f : S \to \{0,1\}\}$ of all Boolean functions with domain S. Since we can identify a subset $V \subseteq S$ with its characteristic function $f = 1_V$ (i.e., $1_V(x) = 1$ iff $x \in V$), we can think of $\{0,1\}^S$ also as the set of all *subsets* of S. This subset is sometimes called the *power set* of S.

The proof of Lemma 2.6 can be generalized to show that there is no one-to-one map between a set and its power set. In particular, it means that the set $\{0,1\}^{\mathbb{R}}$ is "even bigger" than $\mathbb{R}$. Cantor used these ideas to construct an infinite hierarchy of shades of infinity. The number of such shades turns out to be much larger than $|\mathbb{N}|$ or even $|\mathbb{R}|$. He denoted the cardinality of $\mathbb{N}$ by $\aleph_0$ and denoted the next largest infinite number by $\aleph_1$. ($\aleph$ is the first letter in the Hebrew alphabet.) Cantor also made the continuum hypothesis that $|\mathbb{R}| = \aleph_1$. We will come back to the fascinating story of this hypothesis later on in this book. This lecture of Aaronson mentions some of these issues (see also this Berkeley CS 70 lecture).

To complete the proof of Theorem 2.5, we need to show Lemma 2.7. This requires some calculus background but is otherwise straightforward. The idea is that we can construct a one-to-one map from $\{0,1\}^\infty$ to the real numbers by mapping the function $f : \mathbb{N} \to \{0,1\}$ to the number that has the infinite decimal expansion $f(0).f(1)f(2)f(3)f(4)f(5)\ldots$ (i.e., the number between 0 and 2 that is $\sum_{i=0}^{\infty} f(i)10^{-i}$). We will now do this more formally. If you have not had much experience with limits of a real series before, then the formal proof below might be a little hard to follow. This part is not the core of Cantor's argument, nor are such limits important to the remainder of this book, so feel free to also just take Lemma 2.7 on faith and skip the proof.i

Proof Idea:

As discussed above, we define $FtR(f)$ to be the number between 0 and 2 whose decimal expansion is $f(0).f(1)f(2)\ldots$, or in other words $FtR(f) = \sum_{i=0}^{\infty} f(i) \cdot 10^{-i}$. To prove that FtR is one to one, we need to show that if $f \neq g$ then $FtR(f) \neq FtR(g)$. To do that we let $k \in \mathbb{N}$ be the first input on which f and g disagree. Then the numbers $FtR(f)$ and $FtR(g)$ agree in the first $k-2$ digits following the decimal point and disagree in the $k-1$-th digit. One can then calculate and verify

that this means that $|FtR(f) - FtR(g)| > 0.5 \cdot 10^{-k}$ which in particular means that these two numbers are distinct from one another.[14]

⋆

Proof of Lemma 2.7. For every $f \in \{0,1\}^{\infty}$ and $n \in \mathbb{N}$, we define $S(f)_n = \sum_{i=0}^{n} f(i)10^{-i}$. It is a known result in calculus (whose proof we will not repeat here) that for every $f : \mathbb{N} \to \{0,1\}$, the sequence $(S(f)_n)_{n=0}^{\infty}$ has a *limit*. In other words, for every f there is a value $\alpha(f) \in \mathbb{R}$ such that for every $\epsilon > 0$, if n is sufficiently large then $|S(f)_n - \alpha| < \epsilon$. The value $\alpha(f)$ is denoted by $\sum_{i=0}^{\infty} f(i) \cdot 10^{-i}$. We define the function FtR by setting $FtR(f) = \alpha(f)$.

We define $FtR(f)$ to be this value $x(f)$. In other words, we define

$$FtR(f) = \sum_{i=0}^{\infty} f(i) \cdot 10^{-i} \tag{2.6}$$

which will be a number between 0 and 2.

To show that FtR is one to one, we need to show that $FtR(f) \neq FtR(g)$ for every distinct $f, g : \mathbb{N} \to \{0,1\}$. Let $f \neq g$ be such functions. Since f and g are distinct, there must be some input on which they differ, and we define k to be the smallest such input. That is, $k \in \mathbb{N}$ is the smallest number for which $f(k) \neq g(k)$. We will show that $|FtR(f) - FtR(g)| > 0.5 \cdot 10^{-k}$. This will complete the proof since in particular it implies that $FtR(f) \neq FtR(g)$.

Since $f(k) \neq g(k)$, we can assume without loss of generality that $f(k) = 0$ and $g(k) = 1$ (otherwise, if $f(k) = 1$ and $g(k) = 1$, then we can simply switch the roles of f and g). Define $S = \sum_{i=0}^{k-1} 10^{-i} \cdot f(i) = \sum_{i=0}^{k-1} 10^{-i} \cdot g(i)$ (the equality holds since f and g agree up to $k-1$). Now, since $g(k) = 1$, we can write

$$FtR(g) = \sum_{i=0}^{\infty} g(i)10^{-i} \geq S + g(k)10^{-k} = S + 10^{-k} \,. \tag{2.7}$$

On the other hand, since $f(k) = 0$,

$$FtR(f) = \sum_{i=0}^{\infty} f(i)10^{-i} = S + \sum_{i=k+1}^{\infty} f(i)10^{-i} \leq S + 10^{-(k-1)} \sum_{j=0}^{\infty} 10^{-j} \,. \tag{2.8}$$

(Using the trivial equality $f(j) \leq 1$ for every j.)

Now $\sum_{j=0}^{\infty} 10^{-j}$ is simply the number $1.11111\ldots = 11/9$, and hence we get that

$$FtR(f) \leq S + 11/9 \cdot 10^{-k-1} = S + \tfrac{11}{90} \cdot 10^{-k} < S + 0.2 \cdot 10^{-k} \tag{2.9}$$

while $FtR(g) \geq S + 10^{-k}$ which means the difference between them is larger than $0.5 \cdot 10^{-k}$.

■

[14] You could wonder why we can't immediately deduce that two numbers that differ in a digit are not the same. The issue is that we have to be a little more careful when talking about infinite expansions. For example, the number half has two decimal expansions 0.5 and $0.49999\cdots$. However, this issue does not come up if (as in our case) we restrict attention only to numbers with decimal expansions that do not involve the digit 9.

2.4 BEYOND NUMBERS

We can of course represent objects other than numbers as binary strings. Let us give a general definition for a *representation scheme*. Such a scheme for representing objects from some set $\mathcal{O}$ consists of an *encoding* function that maps an object in $\mathcal{O}$ to a string, and a *decoding* function that decodes a string back to an object in $\mathcal{O}$. Formally, we make the following definition:

Definition 2.9 — String representation. Let $\mathcal{O}$ be some set. A *representation scheme* for $\mathcal{O}$ is a pair of functions E, D where $E : \mathcal{O} \to \{0,1\}^*$ is a total one-to-one function, $D : \{0,1\}^* \to_p \mathcal{O}$ is a (possibly partial) function, and such that D and E satisfy that $D(E(o)) = o$ for every $o \in \mathcal{O}$. E is known as the *encoding* function and D is known as the *decoding* function.

Note that the condition $D(E(o)) = o$ for every $o \in \mathcal{O}$ implies that D is *onto* (can you see why?). It turns out that to construct a representation scheme we only need to find an *encoding* function. That is, every one-to-one encoding function has a corresponding decoding function, as shown in the following lemma:

Lemma 2.10 Suppose that $E : \mathcal{O} \to \{0,1\}^*$ is one-to-one. Then there exists a function $D : \{0,1\}^* \to \mathcal{O}$ such that $D(E(o)) = o$ for every $o \in \mathcal{O}$.

Proof. Let o_0 be some arbitrary element of O. For every $x \in \{0,1\}^*$, there exists either zero or a single $o \in \mathcal{O}$ such that $E(o) = x$ (otherwise E would not be one-to-one). We will define $D(x)$ to equal o_0 in the first case and this single object o in the second case. By definition $D(E(o)) = o$ for every $o \in \mathcal{O}$. ■

R

Remark 2.11 —. While the decoding function of a representation scheme can in general be a *partial* function, the proof of Lemma 2.10 implies that every representation scheme has a *total* decoding function. This observation can sometimes be useful.

2.4.1 Finite representations

If $\mathcal{O}$ is *finite*, then we can represent every object in o as a string of length at most some number n. What is the value of n? Let us denote by $\{0,1\}^{\leq n}$ the set $\{x \in \{0,1\}^* : |x| \leq n\}$ of strings of length at

most n. The size of $\{0,1\}^{\leq n}$ is equal to

$$|\{0,1\}^0|+|\{0,1\}^1|+|\{0,1\}^2|+\cdots+|\{0,1\}^n| = \sum_{i=0}^{n} 2^i = 2^{n+1}-1 \ . \quad (2.10)$$

using the standard formula for summing a geometric progression.

To obtain a representation of objects in $\mathcal{O}$ as strings of length at most n we need to come up with a one-to-one function from $\mathcal{O}$ to $\{0,1\}^{\leq n}$. We can do so, if and only if $|\mathcal{O}| \leq 2^{n+1}-1$ as is implied by the following lemma:

Lemma 2.12 For every two finite sets S, T, there exists a one-to-one $E : S \to T$ if and only if $|S| \leq |T|$.

Proof. Let $k = |S|$ and $m = |T|$ and so write the elements of S and T as $S = \{s_0, s_1, \ldots, s_{k-1}\}$ and $T = \{t_0, t_1, \ldots, t_{m-1}\}$. We need to show that there is a one-to-one function $E : S \to T$ iff $k \leq m$. For the "if" direction, if $k \leq m$ we can simply define $E(s_i) = t_i$ for every $i \in [k]$. Clearly for $i \neq j$, $t_i = E(s_i) \neq E(s_j) = t_j$, and hence this function is one-to-one. In the other direction, suppose that $k > m$ and $E : S \to T$ is some function. Then E cannot be one-to-one. Indeed, for $i = 0, 1, \ldots, m-1$ let us "mark" the element $t_j = E(s_i)$ in T. If t_j was marked before, then we have found two objects in S mapping to the same element t_j. Otherwise, since T has m elements, when we get to $i = m-1$ we mark all the objects in T. Hence, in this case, $E(s_m)$ must map to an element that was already marked before.[15] ■

[15] This observation is sometimes known as the "Pigeon Hole Principle": the principle that if you have a pigeon coop with m holes, and $k > m$ pigeons, then there must be two pigeons in the same hole.

2.4.2 Prefix-free encoding

When showing a representation scheme for rational numbers, we used the "hack" of encoding the alphabet $\{0, 1, \|\}$ to represent tuples of strings as a single string. This turns out to be a special case of the general paradigm of *prefix-free* encoding. The idea is the following: if our representation has the property that no string x representing an object o is a *prefix* (i.e., an initial substring) of a string y representing a different object o', then we can represent a *lists* of objects by merely concatenating the representations of all the list members. For example, because in English every sentence ends with a punctuation mark such as a period, exclamation, or question mark, we can represent a list of sentences (i.e., a paragraph) by merely concatenating the sentences one after the other.

It turns out that we can transform *every* representation to a prefix-free form. This justifies Big Idea 1, and allows us to transform a representation scheme for objects of a type T to a representation scheme of *lists* of objects of the type T. By repeating the same technique, we can also represent lists of lists of objects of type T, and so on and so forth. But first let us formally define prefix-freeness:

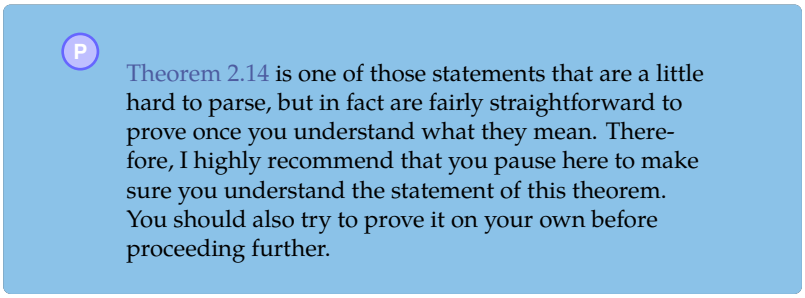
Definition 2.13 — Prefix free encoding. For two strings y, y', we say that y is a *prefix* of y' if $|y| \leq |y'|$ and for every $i < |y'|$, $y'_i = y_i$.

Let $E : \mathcal{O} \to \{0,1\}^*$ be a function. We say that E *prefix-free* if there does not exist a distinct pair of objects $o, o' \in \mathcal{O}$ such that $E(o)$ is a prefix of $E(o')$.

Recall that for every set $\mathcal{O}$, the set $\mathcal{O}^*$ consists of all finite length tuples (i.e., *lists*) of elements in $\mathcal{O}$. The following theorem shows that if E is a prefix-free encoding of $\mathcal{O}$ then by concatenating encodings we can obtain a valid (i.e., one-to-one) representation of $\mathcal{O}^*$:

Theorem 2.14 — Prefix-free implies tuple encoding. Suppose that $E : \mathcal{O} \to \{0,1\}^*$ is prefix-free. Then the following map $\overline{E} : \mathcal{O}^* \to \{0,1\}^*$ is one to one, for every $o_0, \ldots, o_{k-1} \in \mathcal{O}^*$, we define

$$\overline{E}(o_0, \ldots, o_{k-1}) = E(o_0)E(o_1)\cdots E(o_{k-1}) \; . \tag{2.11}$$



Theorem 2.14 is one of those statements that are a little hard to parse, but in fact are fairly straightforward to prove once you understand what they mean. Therefore, I highly recommend that you pause here to make sure you understand the statement of this theorem. You should also try to prove it on your own before proceeding further.



Figure 2.7: If we have a prefix-free representation of each object then we can concatenate the representations of k objects to obtain a representation for the tuple $(o_0, \ldots, o_{k-1})$.

Proof Idea:

The idea behind the proof is simple. Suppose that for example we want to decode a triple (o_0, o_1, o_2) from its representation $x = E'(o_0, o_1, o_2) = E(o_0)E(o_1)E(o_2)$. We will do so by first finding the first prefix x_0 of x such is a representation of some object. Then we will decode this object, remove x_0 from x to obtain a new string x', and continue onwards to find the first prefix x_1 of x' and so on and so forth (see Exercise 2.8). The prefix-freeness property of E will ensure that x_0 will in fact be $E(o_0)$, x_1 will be $E(o_1)$ etc.

⋆

Proof of Theorem 2.14. We now show the formal proof. Suppose, towards the sake of contradiction that there exist two distinct tuples $(o_0, \ldots, o_{k-1})$ and $(o'_0, \ldots, o'_{k'-1})$ such that

$$\overline{E}(o_0, \ldots, o_{k-1}) = \overline{E}(o'_0, \ldots, o'_{k'-1}) \; . \tag{2.12}$$

We will denote the string $\overline{E}(o_0, \ldots, o_{k-1})$ by $\overline{x}$.

Let i be the first coordinate such that $o_i \neq o'_i$. (If $o_i = o'_i$ for all i then, since we assume the two tuples are distinct, one of them must be larger than the other. In this case we assume without loss of generality that $k' > k$ and let $i = k$.) In the case that $i < k$, we see that the string $\overline{x}$ can be written in two different ways:

$$\overline{x} = \overline{E}(o_0, \ldots, o_{k-1}) = x_0 \cdots x_{i-1} E(o_i) E(o_{i+1}) \cdots E(o_{k-1}) \tag{2.13}$$

and

$$\overline{x} = \overline{E}(o'_0, \ldots, o'_{k'-1}) = x_0 \cdots x_{i-1} E(o'_i) E(o'_{i+1}) \cdots E(o'_{k-1}) \tag{2.14}$$

where $x_j = E(o_j) = E(o'_j)$ for all $j < i$. Let $\overline{y}$ be the string obtained after removing the prefix $x_0 \cdots x_{i-i}$ from $\overline{x}$. We see that $\overline{y}$ can be written as both $\overline{y} = E(o_i)s$ for some string $s \in \{0,1\}^*$ and as $\overline{y} = E(o'_i)s'$ for some $s' \in \{0,1\}^*$. But this means that one of $E(o_i)$ and $E(o'_i)$ must be a prefix of the other, contradicting the prefix-freeness of E.

In the case that $i = k$ and $k' > k$, we get a contradiction in the following way. In this case

$$\overline{x} = E(o_0) \cdots E(o_{k-1}) = E(o_0) \cdots E(o_{k-1}) E(o'_k) \cdots E(o'_{k'-1}) \tag{2.15}$$

which means that $E(o'_k) \cdots E(o'_{k'-1})$ must correspond to the empty string "". But in such a case $E(o'_k)$ must be the empty string, which in particular is the prefix of any other string, contradicting the prefix-freeness of E.

■

R

Remark 2.15 — Prefix freeness of list representation. Even if the representation E of objects in $\mathcal{O}$ is prefix free, this does not mean that our representation $\overline{E}$ of *lists* of such objects will be prefix free as well. In fact, it won't be: for every three objects o, o', o'' the representation of the list (o, o') will be a prefix of the representation of the list (o, o', o''). However, as we see in Lemma 2.16 below, we can transform *every* representation into prefix-free form, and so will be able to use that transformation if needed to represent lists of lists, lists of lists of lists, and so on and so forth.

2.4.3 Making representations prefix-free

Some natural representations are prefix-free. For example, every *fixed output length* representation (i.e., one-to-one function $E : \mathcal{O} \to \{0,1\}^n$)

is automatically prefix-free, since a string x can only be a prefix of an equal-length x' if x and x' are identical. Moreover, the approach we used for representing rational numbers can be used to show the following:

Lemma 2.16 Let $E : \mathcal{O} \to \{0,1\}^*$ be a one-to-one function. Then there is a one-to-one prefix-free encoding $\overline{E}$ such that $|\overline{E}(o)| \leq 2|E(o)| + 2$ for every $o \in \mathcal{O}$.



For the sake of completeness, we will include the proof below, but it is a good idea for you to pause here and try to prove it on your own, using the same technique we used for representing rational numbers.

Proof of Lemma 2.16. Define the function $PF : \{0,1\}^* \to \{0,1\}^*$ as follows $PF(x) = x_0x_0x_1x_1 \ldots x_{n-1}x_{n-1}01$ for every $x \in \{0,1\}^*$. If $E : \mathcal{O} \to \{0,1\}^*$ is the (potentially not prefix-free) representation for $\mathcal{O}$, then we transform it into a prefix-free representation $\overline{E} : \mathcal{O} \to \{0,1\}^*$ by defining $\overline{E}(o) = PF(E(o))$.

To prove the lemma we need to show that **(1)** $\overline{E}$ is one-to-one and **(2)** $\overline{E}$ is prefix-free. In fact **(2)** implies **(1)**, since if $\overline{E}(o)$ is never a prefix of $\overline{E}(o')$ for every $o \neq o'$ then in particular $\overline{E}$ is one-to-one. Now suppose, toward a contradiction, that there are $o \neq o'$ in $\mathcal{O}$ such that $\overline{E}(o)$ is a prefix of $\overline{E}(o')$. (That is, if $y = \overline{E}(o)$ and $y' = \overline{E}(o')$, then $y_j = y'_j$ for every $j < |y|$.)

Define $x = E(o)$ and $x' = E(o')$. Note that since E is one-to-one, $x \neq x'$. (Recall that two strings x, x' are distinct if they either differ in length or have at least one distinct coordinate.) Under our assumption, $|PF(x)| \leq |PF(x')|$, and since by construction $|PF(x)| = 2|x| + 2$, it follows that $|x| \leq |x'|$. If $|x| = |x'|$ then, since $x \neq x'$, there must be a coordinate $i \in \{0, \ldots, |x| - 1\}$ such that $x_i \neq x'_i$. But since $PF(x)_{2i} = x_i$, we get that $PF(x)_{2i} \neq PF(x')_{2i}$ and hence $\overline{E}(o) = PF(x)$ is *not* a prefix of $\overline{E}(o') = PF(x')$. Otherwise (if $|x| \neq |x'|$) then it must be that $|x| < |x'|$, and hence if $n = |x|$, then $PF(x)_{2n} = 0$ and $PF(x)_{2n+1} = 1$. But since $n < |x'|$, $PF(x')_{2n}, PF(x')_{2n+1}$ is equal to either 00 or 11, and in any case we get that $\overline{E}(o) = PF(x)$ is not a prefix of $\overline{E}(o') = PF(x')$.

■

The proof of Lemma 2.16 is not the only or even the best way to transform an arbitrary representation into prefix-free form. In fact, we can even obtain a more efficient transformation satisfying $|E'(o)| \leq |o| + O(\log |o|)$. We leave proving this as an exercise (see Exercise 2.9).

2.4.4 "Proof by Python" (optional)

The proofs of Theorem 2.14 and Lemma 2.16 are *constructive* in the sense that they give us:

- A way to transform the encoding and decoding functions of any representation of an object O to an encoding and decoding functions that are prefix-free;
- A way to extend prefix-free encoding and decoding of single objects to encoding and decoding of *lists* of objects by concatenation.

Specifically, we could transform any pair of Python functions `encode` and `decode` to functions `pfencode` and `pfdecode` that correspond to a prefix-free encoding and decoding. Similarly, given `pfencode` and `pfdecode` for single objects, we can extend them to encoding of lists. Let us show how this works for the case of the `NtS` and `StN` functions we defined above.

We start with the "Python proof" of Lemma 2.16: a way to transform an arbitrary representation into one that is *prefix free*. The function `prefixfree` below takes as input a pair of encoding and decoding functions, and returns a triple of functions containing *prefix-free* encoding and decoding functions, as well as a function that checks whether a string is a valid encoding of an object.

```
# takes functions encode and decode mapping
# objects to lists of bits and vice versa,
# and returns functions pfencode and pfdecode that
# maps objects to lists of bits and vice versa
# in a prefix-free way.
# Also returns a function pfvalid that says
# whether a list is a valid encoding
def prefixfree(encode, decode):
    def pfencode(o):
        L = encode(o)
        return [L[i//2] for i in range(2*len(L))]+[0,1]
    def pfdecode(L):
        return decode([L[j] for j in range(0,len(L)-2,2)])
    def pfvalid(L):
        return (len(L) % 2 == 0 ) and L[-2:]==[0,1]

    return pfencode, pfdecode, pfvalid

pfNtS, pfStN , pfvalidN = prefixfree(NtS,StN)

NtS(234)
# 11101010
```

```
pfNtS(234)
# 111111001100110001
pfStN(pfNtS(234))
# 234
pfvalidM(pfNtS(234))
# true
```



Note that Python function `prefixfree` above takes two *Python functions* as input and outputs three Python functions as output. [16] You don't have to know Python in this course, but you do need to get comfortable with the idea of functions as mathematical objects in their own right, that can be used as inputs and outputs of other functions.

We now show a "Python proof" of Theorem 2.14. Namely, we show a function `represlists` that takes as input a prefix-free representation scheme (implemented via encoding, decoding, and validity testing functions) and outputs a representation scheme for *lists* of such objects. If we want to make this representation prefix-free then we could fit it into the function `prefixfree` above.

```
# Takes functions pfencode, pfdecode and pfvalid,
# and returns functions encodelists, decodelists
# that can encode and decode
# lists of the objects respectively
def represlists(pfencode,pfdecode,pfvalid):

    def encodelist(L):
        """Gets list of objects, encodes it as list of
        ↪ bits"""
        return "".join([pfencode(obj) for obj in L])

    def decodelist(S):
        """Gets lists of bits, returns lists of objects"""
        i=0; j=1 ; res = []
        while j<=len(S):
            if pfvalid(S[i:j]):
                res += [pfdecode(S[i:j])]
                i=j
            j+= 1
        return res

    return encodelist,decodelist
```

[16] When it's not too awkward, we use the term "Python function" or "subroutine" to distinguish between such snippets of python programs and mathematical functions. However, in comments in python source we use "functions" to denote python functions, just as we use "integers" to denote python int objects.

```
LtS , StL = represlists(pfNtS,pfStN,pfvalidN)

LtS([234,12,5])
# 11111100110011000111110000011100l101
StL(LtS([234,12,5]))
# [234, 12, 5]
```

2.4.5 Representing letters and text

We can represent a letter or symbol by a string, and then if this representation is prefix-free, we can represent a sequence of symbols by merely concatenating the representation of each symbol. One such representation is the ASCII that represents 128 letters and symbols as strings of 7 bits. Since the ASCII representation is fixed-length, it is automatically prefix-free (can you see why?). Unicode is representation of (at the time of this writing) about 128,000 symbols as numbers (known as *code points*) between 0 and $1,114,111$. There are several types of prefix-free representations of the code points, a popular one being UTF-8 that encodes every codepoint into a string of length between 8 and 32.

■ **Example 2.17 — The Braille representation.** The *Braille system* is another way to encode letters and other symbols as binary strings. Specifically, in Braille, every letter is encoded as a string in $\{0,1\}^6$, which is written using indented dots arranged in two columns and three rows, see Fig. 2.8. (Some symbols require more than one six-bit string to encode, and so Braille uses a more general prefix-free encoding.)

The Braille system was invented in 1821 by Louis Braille when he was just 12 years old (though he continued working on it and improving it throughout his life). Braille was a French boy that lost his eyesight at the age of 5 as the result of an accident.

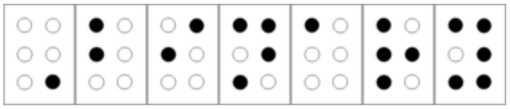


Figure 2.8: The word "Binary" in "Grade 1" or "uncontracted" Unified English Braille. This word is encoded using seven symbols since the first one is a modifier indicating that the first letter is capitalized.

■ **Example 2.18 — Representing objects in C (optional).** We can use programming languages to probe how our computing environment represents various values. This is easiest to do in "unsafe" programming languages such as `C` that allow direct access to the memory.

Using a simple `C` program we have produced the following representations of various values. One can see that for integers, multiplying by 2 corresponds to a "left shift" inside each byte. In contrast, for floating point numbers, multiplying by two corre-

sponds to adding one to the exponent part of the representation. A negative number is represented using the two's complement approach. Strings are represented in a prefix-free form by ensuring that a zero byte is at their end.

```
int      2    : 00000010 00000000 00000000 00000000
int      4    : 00000100 00000000 00000000 00000000
int      513  : 00000001 00000010 00000000 00000000
long     513  : 00000001 00000010 00000000 00000000
 ↪  00000000 00000000 00000000 00000000
int      -1   : 11111111 11111111 11111111 11111111
int      -2   : 11111110 11111111 11111111 11111111
string   Hello: 01001000 01100101 01101100 01101100
 ↪  01101111 00000000
string   abcd : 01100001 01100010 01100011 01100100
 ↪  00000000
float    33.0 : 00000000 00000000 00000100 01000010
float    66.0 : 00000000 00000000 10000100 01000010
float    132.0: 00000000 00000000 00000100 01000011
double   132.0: 00000000 00000000 00000000 00000000
 ↪  00000000 10000000 01100000 01000000
```

If you are curious, the code for this program (which you can run here) is the following:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *bytes(void *p,int n){
  int i;
  int j;
  char *a = (char *) p;
  char *s = malloc(9*n+2);
  s[9*n] = '\n';
  s[9*n+1] = 0;


  j = 0;
  for(i=0;i< n*8;i++){
    s[j] = a[i/8] & (128 >> (i % 8)) ? '1' : '0';
    if (i% 8 == 7) { s[++j] = ' '; }
    ++j;
  }
```

```
  return s;
}

void printint(int a) {
  printf("%-8s %-5d: %s", "int", a,
   ↪  bytes(&a,sizeof(int)));
}

void printlong(long a) {
  printf("%-8s %-5d: %s", "long", a,
   ↪  bytes(&a,sizeof(long)));
}


void printstring(char *s) {
  printf("%-8s %-5s: %s", "string", s,
   ↪  bytes(s,strlen(s)+1));
}


void printfloat(float f) {
  printf("%-8s %-5.1f: %s", "float", f,
   ↪  bytes(&f,sizeof(float)));
}

void printdouble(double f) {
  printf("%-8s %-5.1f: %s", "double", f,
   ↪  bytes(&f,sizeof(double)));
}



int main(void) {

  printint(2);
  printint(4);
  printint(513);
  printlong(513);


  printint(-1);

  printint(-2);
```

```
  printstring("Hello");

  printstring("abcd");

  printfloat(33);
  printfloat(66);
  printfloat(132);
  printdouble(132);


  return 0;
}
```

2.4.6 Representing vectors, matrices, images

Once we can represent numbers and lists of numbers, then we can also represent *vectors* (which are just lists of numbers). Similarly, we can represent lists of lists, and thus, in particular, can represent *matrices*. To represent an image, we can represent the color at each pixel by a list of three numbers corresponding to the intensity of Red, Green and Blue.[17] Thus an image of n pixels would be represented by a list of n such length-three lists. A video can be represented as a list of images.[18]

[17] We can restrict to three primary colors since (most) humans only have three types of cones in their retinas. We would have needed 16 primary colors to represent colors visible to the Mantis Shrimp.

[18] Of course these representations are rather wasteful and much more compact representations are typically used for images and videos, though this will not be our concern in this book.

2.4.7 Representing graphs

A *graph* on n vertices can be represented as an $n \times n$ *adjacency* matrix whose $(i,j)^{th}$ entry is equal to 1 if the edge (i,j) is present and is equal to 0 otherwise. That is, we can represent an n vertex directed graph $G = (V,E)$ as a string $A \in \{0,1\}^{n^2}$ such that $A_{i,j} = 1$ iff the edge $\overrightarrow{i\,j} \in E$. We can transform an undirected graph to a directed graph by replacing every edge $\{i,j\}$ with both edges $\overrightarrow{i\,j}$ and $\overleftarrow{i\,j}$

Another representation for graphs is the *adjacency list* representation. That is, we identify the vertex set V of a graph with the set $[n]$ where $n = |V|$, and represent the graph $G = (V,E)$ as a list of n lists, where the i-th list consists of the out-neighbors of vertex i. The difference between these representations can be significant for some applications, though for us would typically be immaterial.

Once again, we can also define these encoding and decoding functions in python:

```
from graphviz import Graph

# get n by n matrix (as list of n lists)
# return graph corresponding to it
def matrix2graph(M):
```

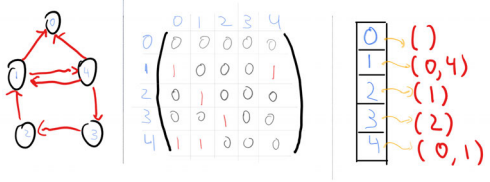
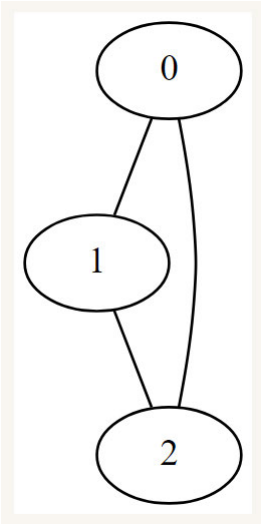


Figure 2.9: Representing the graph $G = (\{0,1,2,3,4\}, \{(1,0),(4,0),(1,4),(4,1),(2,1),(3,2),(4,3)\})$ in the adjacency matrix and adjacency list representations.

```
    G = Graph(); n = len(M)
    for i in range(n):
        G.node(str(i)) # add vertex i
        for j in range(n):
            G.node(str(j))
            if M[i][j]: G.edge(str(i),str(j))
            # if M[i][j] is nonzero then add edge between
             ↪  i and j
    return G

matrix2graph([[0,1,0],[0,0,1],[1,0,0]])
```



2.4.8 Representing lists

If we have a way of representing objects from a set $\mathcal{O}$ as binary strings, then we can represent lists of these objects by applying a prefix-free transformation. Moreover, we can use a trick similar to the above to handle *nested* lists. The idea is that if we have some representation $E : \mathcal{O} \to \{0,1\}^*$, then we can represent nested lists of items from $\mathcal{O}$ using strings over the five element alphabet $\Sigma = \{$ `0`,`1`,`[`,`]`,`,` $\}$. For example, if o_1 is represented by `0011`, o_2 is represented by `10011`, and o_3 is represented by `00111`, then we can represent the nested list $(o_1,(o_2,o_3))$ as the string `"[0011,[10011,00111]]"` over the alphabet Σ. By encoding every element of Σ itself as a three-bit string, we can transform any representation for objects $\mathcal{O}$ into a representation that enables representing (potentially nested) lists of these objects.

2.4.9 Notation

We will typically identify an object with its representation as a string. For example, if $F : \{0,1\}^* \to \{0,1\}^*$ is some function that maps strings to strings and x is an integer, we might make statements such

as "$F(x)+1$ is prime" to mean that if we represent x as a string $\underline{x}$ and let $\underline{y} = F(\underline{x})$, then the integer y represented by the string $\underline{y}$ satisfies that $y+1$ is prime. (You can see how this convention of identifying objects with their representation can save us a lot of cumbersome formalism.) Similarly, if x, y are some objects and F is a function that takes strings as inputs, then by $F(x,y)$ we will mean the result of applying F to the representation of the ordered pair (x,y). We will use the same notation to invoke functions on k-tuples of objects for every k.

This convention of identifying an object with its representation as a string is one that we humans follow all the time. For example, when people say a statement such as "17 is a prime number", what they really mean is that the integer whose decimal representation is the string "17", is prime.

When we say

A is an algorithm that computes the multiplication function on natural numbers.

what we really mean is that

A is an algorithm that computes the function $F : \{0,1\}^* \to \{0,1\}^*$ such that for every pair $a, b \in \mathbb{N}$, if $x \in \{0,1\}^*$ is a string representing the pair (a,b) then $F(x)$ will be a string representing their product $a \cdot b$.

2.5 DEFINING COMPUTATIONAL TASKS

Abstractly, a *computational process* is some process that takes an input which is a string of bits and produces an output which is a string of bits. This transformation of input to output can be done using a modern computer, a person following instructions, the evolution of some natural system, or any other means.

In future chapters, we will turn to mathematically defining computational processes, but, as we discussed above, at the moment we focus on *computational tasks*. That is, we focus on the **specification** and not the **implementation**. Again, at an abstract level, a computational task can specify any relation that the output needs to have with the input. However, for most of this book, we will focus on the simplest and most common task of *computing a function*. Here are some examples:

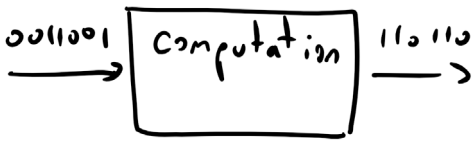


Figure 2.10: A computational process

- Given (a representation) of two integers x, y, compute the product $x \times y$. Using our representation above, this corresponds to computing a function from $\{0,1\}^*$ to $\{0,1\}^*$. We have seen that there is more than one way to solve this computational task, and in fact, we still do not know the best algorithm for this problem.

- Given (a representation of) an integer z, compute its *factorization*; i.e., the list of primes $p_1 \leq \cdots \leq p_k$ such that $z = p_1 \cdots p_k$. This again corresponds to computing a function from $\{0,1\}^*$ to $\{0,1\}^*$. The gaps in our knowledge of the complexity of this problem are even longer.

- Given (a representation of) a graph G and two vertices s and t, compute the length of the shortest path in G between s and t, or do the same for the *longest* path (with no repeated vertices) between s and t. Both these tasks correspond to computing a function from $\{0,1\}^*$ to $\{0,1\}^*$, though it turns out that there is a vast difference in their computational difficulty.

- Given the code of a Python program, determine whether there is an input that would force it into an infinite loop. This task corresponds to computing a *partial* function from $\{0,1\}^*$ to $\{0,1\}$ since not every string corresponds to a syntactically valid Python program. We will see that we *do* understand the computational status of this problem, but the answer is quite surprising.

- Given (a representation of) an image I, decide if I is a photo of a cat or a dog. This correspond to computing some (partial) function from $\{0,1\}^*$ to $\{0,1\}$.

R

Remark 2.19 — Boolean functions and languages. An important special case of computational tasks corresponds to computing *Boolean* functions, whose output is a single bit $\{0,1\}$. Computing such functions corresponds to answering a YES/NO question, and hence this task is also known as a *decision problem*. Given any function $F : \{0,1\}^* \to \{0,1\}$ and $x \in \{0,1\}^*$, the task of computing $F(x)$ corresponds to the task of deciding whether or not $x \in L$ where $L = \{x : F(x) = 1\}$ is known as the *language* that corresponds to the function F. [19] Hence many texts refer to such as computational task as *deciding a language*.

For every particular function F, there can be several possible *algorithms* to compute F. We will be interested in questions such as:

- For a given function F, can it be the case that *there is no algorithm* to compute F?

- If there is an algorithm, what is the best one? Could it be that F is "effectively uncomputable" in the sense that every algorithm for computing F requires a prohibitively large amount of resources?

[19] The language terminology is due to historical connections between the theory of computation and formal linguistics as developed by Noam Chomsky.

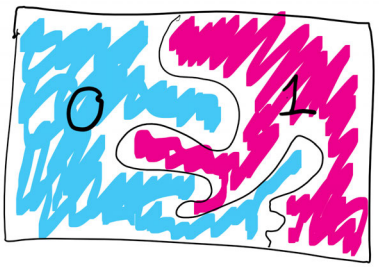


Figure 2.11: A subset $L \subseteq \{0,1\}^*$ can be identified with the function $F : \{0,1\}^* \to \{0,1\}$ such that $F(x) = 1$ if $x \in L$ and $F(x) = 0$ if $x \notin L$. Functions with a single bit of output are called *Boolean functions*, while subsets of strings are called *languages*. The above shows that the two are essentially the same object, and we can identify the task of deciding membership in L (known as *deciding a language* in the literature) with the task of computing the function F.

- If we cannot answer this question, can we show equivalence between different functions F and F' in the sense that either they are both easy (i.e., have fast algorithms) or they are both hard?

- Can a function being hard to compute ever be a *good thing*? Can we use it for applications in areas such as cryptography?

In order to do that, we will need to mathematically define the notion of an *algorithm*, which is what we will do in Chapter 3.

2.5.1 Distinguish functions from programs!

You should always watch out for potential confusions between **specifications** and **implementations** or equivalently between **mathematical functions** and **algorithms/programs**. It does not help that programming languages (Python included) use the term *"functions"* to denote (parts of) *programs*. This confusion also stems from thousands of years of mathematical history, where people typically defined functions by means of a way to compute them.

For example, consider the multiplication function on natural numbers. This is the function $MULT : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ that maps a pair (x, y) of natural numbers to the number $x \cdot y$. As we mentioned, it can be implemented in more than one way:

```
def mult1(x,y):
    res = 0
    while y>0:
        res += x
        y   -= 1
    return res

def mult2(x,y):
    a = NtS(x)
    b = NtS(y)
    res = 0
    res = [0]*(len(a)+len(b))
    for i in range(len(a)):
        for j in range(len(b)):
            res += int(a[len(a)-i])*int(b[len(b)-
             ↪  j])*(10**(i+j))
    return res

print(mult1(12,7))
# 84
print(mult2(12,7))
# 84
```

Both `mult1` and `mult2` produce the same output given the same pair of inputs. (Though `mult1` will take far longer to do so when the numbers become large.) Hence, even though these are two different *programs*, they compute the same *mathematical function*. This distinction between a *program* or *algorithm* A, and the *function* F that A *computes* will be absolutely crucial for us in this course (see also Fig. 2.12).

Big Idea 2 A *function* is not the same as a *program*. A program *computes* a function.

Distinguishing *functions* from *programs* (or other ways for computing, including *circuits* and *machines*) is a crucial theme for this course. For this reason, this is often a running theme in questions that I (and many other instructors) assign in homeworks and exams (hint, hint..).

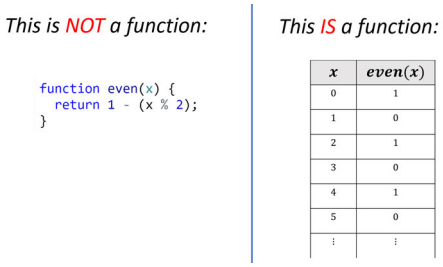


Figure 2.12: A *function* is a mapping of inputs to outputs. A *program* is a set of instructions on how to obtain an output given an input. A program *computes* a function, but it is not the same as a function, popular programming language terminology notwithstanding.

R

Remark 2.20 — Computation beyond functions (advanced, optional). Functions capture quite a lot of computational tasks, but one can consider more general settings as well. For starters, we can and will talk about *partial* functions, that are not defined on all inputs. When computing a partial function, we only need to worry about the inputs on which the function is defined. Another way to say it is that we can design an algorithm for a partial function F under the assumption that someone "promised" us that all inputs x would be such that $F(x)$ is defined (as otherwise, we do not care about the result). Hence such tasks are also known as *promise problems*.

Another generalization is to consider *relations* that may have more than one possible admissible output. For example, consider the task of finding any solution for a given set of equations. A *relation* R maps a string $x \in \{0,1\}^*$ into a *set of strings* $R(x)$ (for example, x might describe a set of equations, in which case $R(x)$ would correspond to the set of all solutions to x). We can also identify a relation R with the set of pairs of strings (x, y) where $y \in R(x)$. A computational process solves a relation if for every $x \in \{0,1\}^*$, it outputs some string $y \in R(x)$.

Later in this book, we will consider even more general tasks, including *interactive* tasks, such as finding a good strategy in a game, tasks defined using probabilistic notions, and others. However, for much of this book, we will focus on the task of computing a function, and often even a *Boolean* function, that has only a single bit of output. It turns out that a great deal of the theory of computation can be studied in the context of

this task, and the insights learned are applicable in the more general settings.



Lecture Recap

- We can represent essentially every object we want to compute on using binary strings.
- A representation scheme for a set of objects $\mathcal{O}$ is a one-to-one map from $\mathcal{O}$ to $\{0,1\}^*$.
- A basic computational task is the task of *computing a function* $F : \{0,1\}^* \to \{0,1\}^*$. This task encompasses not just arithmetical computations such as multiplication, factoring, etc. but a great many other tasks arising in areas as diverse as scientific computing, artificial intelligence, image processing, data mining and many many more.
- We will study the question of finding (or at least giving bounds on) what is the *best* algorithm for computing F for various interesting functions F.

2.6 EXERCISES

Exercise 2.1 Which one of these objects can be represented by a binary string?

a. An integer x

b. An undirected graph G.

c. A directed graph H

d. All of the above.

Exercise 2.2 — More compact than ASCII representation. The ASCII encoding can be used to encode a string of n English letters as a $7n$ bit binary string, but in this exercise, we ask about finding a more compact representation for strings of English lowercase letters.

1. Prove that there exists a representation scheme (E, D) for strings over the 26-letter alphabet $\{a, b, c, \ldots, z\}$ as binary strings such that for every $n > 0$ and length-n string $x \in \{a, b, \ldots, z\}^n$, the representation $E(x)$ is a binary string of length at most $4.8n + 1000$. In other words, prove that for every n, there exists a one-to-one function $E : \{a, b, \ldots, z\}^n \to \{0,1\}^{\lfloor 4.8n+1000 \rfloor}$.

2. Prove that there exists *no* representation scheme for strings over the alphabet $\{a, b, \ldots, z\}$ as binary strings such that for every length-n

string $x \in \{a, b, \ldots, z\}^n$, the representation $E(x)$ is a binary string of length $\lfloor 4.6n + 1000 \rfloor$. In other words, prove that there exists some $n > 0$ such that there is no one-to-one function $E : \{a, b, \ldots, z\}^n \rightarrow \{0,1\}^{\lfloor 4.6n+1000 \rfloor}$

3. Python's `bz2.compress` function is a mapping from strings to strings, which uses the *lossless* (and hence *one to one*) bzip2 algorithm for compression. After converting to lowercase, and truncating spaces and numbers, the text of Tolstoy's "War and Peace" contains $n = 2,517,262$. Yet, if we run `bz2.compress` on the string of the text of "War and Peace" we get a string of length $k = 6,274,768$ bits, which is only $2.49n$ (and in particular much smaller than $4.6n$). Explain why this does not contradict your answer to the previous question.

4. Interestingly, if we try to apply `bz2.compress` on a *random* string, we get much worse performance. In my experiments, I got a ratio of about 4.78 between the number of bits in the output and the number of characters in the input. However, one could imagine that one could do better and that there exists a company called "Pied Piper" with an algorithm that can losslessly compress a string of n random lowercase letters to fewer than $4.6n$ bits.[20] Show that this is not the case by proving that for *every* $n > 100$ and one to one function $Encode : \{a, \ldots, z\}^n \rightarrow \{0,1\}^*$, if we let Z be the random variable $|Encode(x)|$ (i.e., the length of $Encode(x)$) for x chosen uniformly at random from the set $\{a, \ldots, z\}^n$, then the expected value of Z is at least $4.6n$.

[20] Actually that particular fictional company uses a metric that focuses more on compression *speed* then *ratio*, see here and here.

■

Exercise 2.3 — Representing graphs: upper bound. Show that there is a string representation of directed graphs with vertex set $[n]$ and degree at most 10 that uses at most $1000n \log n$ bits. More formally, show the following. Suppose we define for every $n \in \mathbb{N}$, the set G_n as the set containing all directed graphs (with no self loops) over the vertex set $[n]$ where every vertex has degree at most 10. Then, prove that for every sufficiently large n, there exists a one-to-one function $E : G_n \rightarrow \{0,1\}^{\lfloor 1000n \log n \rfloor}$.

■

Exercise 2.4 — Representing graphs: lower bound. 1. Define S_n to be the set of one-to-one and onto functions mapping $[n]$ to $[n]$. Prove that there is a one-to-one mapping from S_n to G_{2n}, where G_{2n} is the set defined in Exercise 2.3 above.

2. In this question you will show that one cannot improve the representation of Exercise 2.3 to length $o(n \log n)$. Specifically, prove

for every sufficiently large $n \in \mathbb{N}$ there is *no* one-to-one function $E : G_n \to \{0,1\}^{\lfloor 0.001 n \log n \rfloor + 1000}$.

■

Exercise 2.5 — Multiplying in different representation. Recall that the grade-school algorithm for multiplying two numbers requires $O(n^2)$ operations. Suppose that instead of using decimal representation, we use one of the following representations $R(x)$ to represent a number x between 0 and $10^n - 1$. For which one of these representations you can still multiply the numbers in $O(n^2)$ operations?

a. The standard binary representation: $B(x) = (x_0, \ldots, x_k)$ where $x = \sum_{i=0}^{k} x_i 2^i$ and k is the largest number s.t. $x \geq 2^k$.

b. The reverse binary representation: $B(x) = (x_k, \ldots, x_0)$ where x_i is defined as above for $i = 0, \ldots, k-1$.

c. Binary coded decimal representation: $B(x) = (y_0, \ldots, y_{n-1})$ where $y_i \in \{0,1\}^4$ represents the ith decimal digit of x mapping 0 to 0000, 1 to 0001, 2 to 0010, etc. (i.e. 9 maps to 1001)

d. All of the above.

■

Exercise 2.6 Suppose that $R : \mathbb{N} \to \{0,1\}^*$ corresponds to representing a number x as a string of x 1's, (e.g., $R(4) = 1111$, $R(7) = 1111111$, etc.). If x, y are numbers between 0 and $10^n - 1$, can we still multiply x and y using $O(n^2)$ operations if we are given them in the representation $R(\cdot)$?

■

Exercise 2.7 Recall that if F is a one-to-one and onto function mapping elements of a finite set U into a finite set V then the sizes of U and V are the same. Let $B : \mathbb{N} \to \{0,1\}^*$ be the function such that for every $x \in \mathbb{N}$, $B(x)$ is the binary representation of x.

a. Prove that $x < 2^k$ if and only if $|B(x)| \leq k$.

b. Use a. to compute the size of the set $\{y \in \{0,1\}^* : |y| \leq k\}$ where $|y|$ denotes the length of the string y.

c. Use a. and b. to prove that $2^k - 1 = 1 + 2 + 4 + \cdots + 2^{k-1}$.

■

Exercise 2.8 — Prefix-free encoding of tuples. Suppose that $F : \mathbb{N} \to \{0,1\}^*$ is a one-to-one function that is *prefix-free* in the sense that there is no $a \neq b$ s.t. $F(a)$ is a prefix of $F(b)$.

a. Prove that $F_2 : \mathbb{N} \times \mathbb{N} \to \{0,1\}^*$, defined as $F_2(a,b) = F(a)F(b)$ (i.e., the concatenation of $F(a)$ and $F(b)$) is a one-to-one function.

b. Prove that $F_* : \mathbb{N}^* \to \{0,1\}^*$ defined as $F_*(a_1,\ldots,a_k) = F(a_1)\cdots F(a_k)$ is a one-to-one function, where $\mathbb{N}^*$ denotes the set of all finite-length lists of natural numbers.

■

Exercise 2.9 — More efficient prefix-free transformation. Suppose that $F : O \to \{0,1\}^*$ is some (not necessarily prefix-free) representation of the objects in the set O, and $G : \mathbb{N} \to \{0,1\}^*$ is a prefix-free representation of the natural numbers. Define $F'(o) = G(|F(o)|)F(o)$ (i.e., the concatenation of the representation of the length $F(o)$ and $F(o)$).

a. Prove that F' is a prefix-free representation of O.

b. Show that we can transform any representation to a prefix-free one by a modification that takes a k bit string into a string of length at most $k + O(\log k)$.

c. Show that we can transform any representation to a prefix-free one by a modification that takes a k bit string into a string of length at most $k + \log k + O(\log\log k)$.[21]

[21] Hint: Think recursively how to represent the length of the string.

■

Exercise 2.10 — Kraft's Inequality. Suppose that $S \subseteq \{0,1\}^n$ is some finite prefix-free set.

a. For every $k \leq n$ and length-k string $x \in S$, let $L(x) \subseteq \{0,1\}^n$ denote all the length-n strings whose first k bits are $x_0,\ldots,x_{k-1}$. Prove that **(1)** $|L(x)| = 2^{n-|x|}$ and **(2)** If $x \neq x'$ then $L(x)$ is disjoint from $L(x')$.

b. Prove that $\sum_{x\in S} 2^{-|x|} \leq 1$.

c. Prove that there is no prefix-free encoding of strings with less than logarithmic overhead. That is, prove that there is no function $PF : \{0,1\}^* \to \{0,1\}^*$ s.t. $|PF(x)| \leq |x| + 0.9\log|x|$ for every $x \in \{0,1\}^*$ and such that the set $\{PF(x) : x \in \{0,1\}^*\}$ is prefix-free. The factor 0.9 is arbitrary; all that matters is that it is less than 1.

■

Exercise 2.11 — Composition of one-to-one functions. Prove that for every two one-to-one functions $F : S \to T$ and $G : T \to U$, the function $H : S \to U$ defined as $H(x) = G(F(x))$ is one to one.

■

Exercise 2.12 — Natural numbers and strings. 1. We have shown that the natural numbers can be represented as strings. Prove that the other direction holds as well: that there is a one-to-one map $StN : \{0,1\}^* \to \mathbb{N}$. ($StN$ stands for "strings to numbers.")

2. Recall that Cantor proved that there is no one-to-one map $RtN : \mathbb{R} \to \mathbb{N}$. Show that Cantor's result implies Theorem 2.5.

■

Exercise 2.13 — Map lists of integers to a number. Recall that for every set S, the set S^* is defined as the set of all finite sequences of members of S (i.e., $S^* = \{(x_0, \ldots, x_{n-1}) \mid n \in \mathbb{N} \ , \ \forall_{i \in [n]} x_i \in S\}$). Prove that there is a one-one-map from $\mathbb{Z}^*$ to $\mathbb{N}$ where $\mathbb{Z}$ is the set of $\{\ldots, -3, -2, -1, 0, +1, +2, +3, \ldots\}$ of all integers.

■

2.7 BIBLIOGRAPHICAL NOTES

The study of representing data as strings mostly follows under the purview of *information theory*, as covered in the classic textbook of Cover and Thomas [CT06]. Representations are also studied in the field of *data structures design*, as covered in texts such as [Cor+09].

The idea that we should separate the *definition* or *specification* of a function from its *implementation* or *computation* might seem "obvious," but it took quite a lot of time for mathematicians to arrive at this viewpoint. Historically, a function F was identified by rules or formulas showing how to derive the output from the input. As we discuss in greater depth in Chapter 8, in the 1800s this somewhat informal notion of a function started "breaking at the seams," and eventually mathematicians arrived at the more rigorous definition of a function as an arbitrary assignment of input to outputs. While many functions may be described (or computed) by one or more formulas, today we do not consider that to be an essential property of functions, and also allow functions that do not correspond to any "nice" formula.

We have mentioned that all representations of the real numbers are inherently *approximate*. Thus an important endeavor is to understand what guarantees we can offer on the approximation quality of the output of an algorithm, as a function of the approximation quality of the inputs. This question is known as the question of determining the numerical stability of given equations. The Floating Points Guide website contains an extensive description of the floating point representation, as well the many ways in which it could subtly fail, see also the website 0.30000000000000004.com.

Dauben [Dau90] gives a biography of Cantor with emphasis on the development of his mathematical ideas. [Hal60] is a classic textbook on set theory, including also Cantor's theorem. Cantor's Theo-

rem is also covered in many texts on discrete mathematics, including [**LehmanLeightonMeyer** ; **LewisZax19**].

The adjacency matrix representation of graphs is not merely a convenient way to map a graph into a binary string, but it turns out that many natural notions and operations on matrices are useful for graphs as well. (For example, Google's PageRank algorithm relies on this viewpoint.) The notes of Spielman's course are an excellent source for this area, known as *spectral graph theory*. We will return to this view much later in this book when we talk about *random walks*.

Gromov's and Pomerantz's quotes are taken from Doron Zeilberger's page.

I

FINITE COMPUTATION

Learning Objectives:

- See that computation can be precisely modeled.
- Learn the computational model of *Boolean circuits / straight-line programs*.
- See the NAND operation and also why the specific choice of NAND is not important.
- Examples of computing in the physical world.
- Equivalence of circuits and programs.

3
Defining computation

> *"there is no reason why mental as well as bodily labor should not be economized by the aid of machinery"*, Charles Babbage, 1852

> *"If, unwarned by my example, any man shall undertake and shall succeed in constructing an engine embodying in itself the whole of the executive department of mathematical analysis upon different principles or by simpler mechanical means, I have no fear of leaving my reputation in his charge, for he alone will be fully able to appreciate the nature of my efforts and the value of their results."*, Charles Babbage, 1864

> *"To understand a program you must become both the machine and the program."*, Alan Perlis, 1982

People have been computing for thousands of years, with aids that include not just pen and paper, but also abacus, slide rulers, various mechanical devices, and modern electronic computers. A priori, the notion of computation seems to be tied to the particular mechanism that you use. You might think that the "best" algorithm for multiplying numbers will differ if you implement it in *Python* on a modern laptop than if you use pen and paper. However, as we saw in the introduction (Chapter 0), an algorithm that is asymptotically better would eventually beat a worse one regardless of the underlying technology. This gives us hope for a *technology independent* way of defining computation, which is what we will do in this chapter.

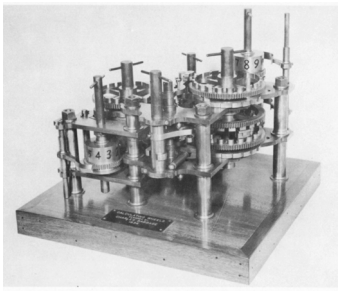


Figure 3.1: Calculating wheels by Charles Babbage. Image taken from the Mark I 'operating manual'

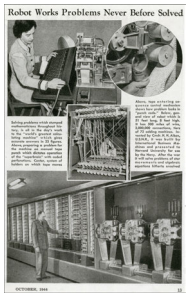


Figure 3.2: A 1944 *Popular Mechanics* article on the Harvard Mark I computer.

3.1 DEFINING COMPUTATION

The name "algorithm" is derived from the Latin transliteration of Muhammad ibn Musa al-Khwarizmi's name. Al-Khwarizmi was a Persian scholar during the 9th century whose books introduced the western world to the decimal positional numeral system, as well as to

the solutions of linear and quadratic equations (see Fig. 3.3). However Al-Khwarizmi's descriptions of algorithms were rather informal by today's standards. Rather than use "variables" such as x, y, he used concrete numbers such as 10 and 39, and trusted the reader to be able to extrapolate from these examples.[1]

Here is how al-Khwarizmi described the algorithm for solving an equation of the form $x^2 + bx = c$:[2]

> [*How to solve an equation of the form*] *"roots and squares are equal to numbers": For instance "one square , and ten roots of the same, amount to thirty-nine dirhems" that is to say, what must be the square which, when increased by ten of its own root, amounts to thirty-nine? The solution is this: you halve the number of the roots, which in the present instance yields five. This you multiply by itself; the product is twenty-five. Add this to thirty-nine' the sum is sixty-four. Now take the root of this, which is eight, and subtract from it half the number of roots, which is five; the remainder is three. This is the root of the square which you sought for; the square itself is nine.*

For the purposes of this book, we will need a much more precise way to describe algorithms. Fortunately (or is it unfortunately?), at least at the moment, computers lag far behind school-age children in learning from examples. Hence in the 20th century people have come up with exact formalisms for describing algorithms, namely *programming languages*. Here is al-Khwarizmi's quadratic equation solving algorithm described in the *Python* programming language:[3]

```
from math import sqrt
#Pythonspeak to enable use of the sqrt function to compute
 ↪  square roots.

def solve_eq(b,c):
    # return solution of x^2 + bx = c following Al
     ↪  Khwarizmi's instructions
    # Al Kwarizmi demonstrates this for the case b=10 and
     ↪  c= 39

    val1 = b/2.0 # "halve the number of the roots"
    val2 = val1*val1 # "this you multiply by itself"
    val3 = val2 + c # "Add this to thirty-nine"
    val4 = sqrt(val3) # "take the root of this"
    val5 = val4 - val1 # "subtract from it half the number
     ↪  of roots"
```

[1] Indeed, extrapolation from examples is still the way most of us first learn algorithms such as addition and multiplication, see Fig. 3.4)

[2] Translation from "The Algebra of Ben-Musa", Fredric Rosen, 1831.



Figure 3.3: Text pages from Algebra manuscript with geometrical solutions to two quadratic equations. Shelfmark: MS. Huntington 214 fol. 004v-005r

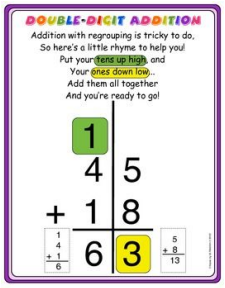


Figure 3.4: An explanation for children of the two digit addition algorithm

[3] This is not a programming textbook, and it is absolutely fine if you don't know Python. Still the code below should be fairly self-explanatory.

```
    return val5  # "This is the root of the square which
     ↪  you sought for"

# Test: solve x^2 + 10*x = 39
print(solve_eq(10,39))
# 3.0
```

We can define algorithms informally as follows:

> **Informal definition of an algorithm:** An *Algorithm* is a set of instructions of how to compute an output from an input by following a sequence of "elementary steps".
>
> An algorithm A *computes* a function F if for every input x, if we follow the instructions of A on the input x, we obtain the output $F(x)$.

In this chapter we will use an ultra-simple "programming language" to give a *formal* (that is, *precise*) definition of algorithms. (In fact, our programming language will be so simple that it is hardly worthy of this name.) However, it will take us some time to get there. We will start by discussing what are "elementary operations" and also how do we map a description of an algorithm into an actual physical process that produces an output from an input in the real world.

3.2 BOOLEAN FORMULAS WITH AND, OR, AND NOT.

An algorithm breaks down a complex calculation into a series of simpler steps. These steps can be executed in a variety of different ways, including:

- Writing down symbols on a piece of paper
- Modifying the current flowing on electrical wires.
- Binding a protein to a strand of DNA
- Response to a stimulus by a member of a collection (e.g., a bee in a colony, a trader in a market).

To formally define algorithms, let us try to "err on the side of simplicity" and model our "basic steps" as truly minimal. For example, here are some very simple functions:

- $OR : \{0,1\}^2 \to \{0,1\}$ defined as

$$OR(a,b) = \begin{cases} 0 & a = b = 0 \\ 1 & \text{otherwise} \end{cases} \tag{3.1}$$

- $AND : \{0,1\}^2 \to \{0,1\}$ defined as

$$AND(a,b) = \begin{cases} 1 & a=b=1 \\ 0 & \text{otherwise} \end{cases} \tag{3.2}$$

- $NOT : \{0,1\} \to \{0,1\}$ defined as $NOT(a) = 1-a$.

The *AND*, *OR* and *NOT* functions are the basic logical operators used in logic and many computer system. Each one of these functions takes either one or two single bits as input, and produces a single bit as output. Clearly, it cannot get much more basic than that. However, the power of computation comes from *composing* simple building blocks together.

Here is an example. Consider the function $MAJ : \{0,1\}^3 \to \{0,1\}$ that is defined as follows:

$$MAJ(x) = \begin{cases} 1 & x_0 + x_1 + x_2 \geq 2 \\ 0 & \text{otherwise} \end{cases} . \tag{3.3}$$

That is, for every $x \in \{0,1\}^3$, $MAJ(x) = 1$ if and only if the majority (i.e., at least two out of the three) of x's coordinates are equal to 1. Can you come up with a formula involving *AND*, *OR* and *NOT* to compute *MAJ*?



It is useful for you to pause at this point and work out the formula for yourself. As a hint, although it is needed to compute some functions, you will not need to use the *NOT* operator to compute *MAJ*.

Let us first try to rephrase $MAJ(x)$ in words: "$MAJ(x) = 1$ if and only if there exists some pair of distinct coordinates i, j such that both x_i and x_j are equal to 1." In other words it means that $MAJ(x) = 1$ iff *either* both $x_0 = 1$ *and* $x_1 = 1$, *or* both $x_1 = 1$ *and* $x_2 = 1$, *or* both $x_0 = 1$ *and* $x_2 = 1$. Since the *OR* of three conditions c_0, c_1, c_2 can be written as $OR(c_0, OR(c_1, c_2))$, we can now translate this into a formula as follows:

$$MAJ(x_0,x_1,x_2) = OR\left(\, AND(x_0,x_1) \;,\; OR(AND(x_1,x_2) \;,\; AND(x_0,x_2))\,\right) . \tag{3.4}$$

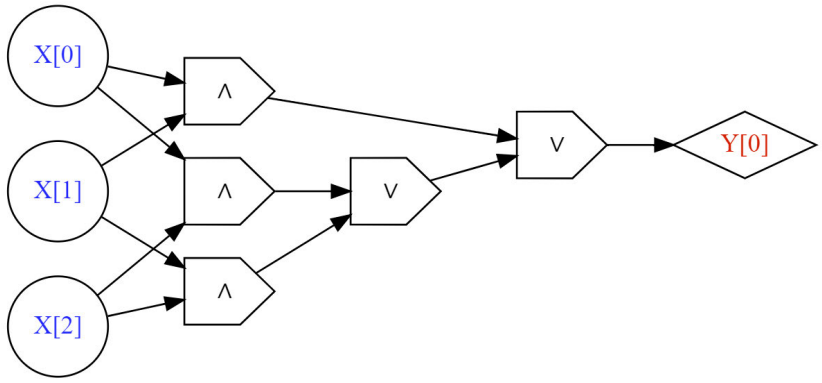
It is common to use $a \vee b$ for $OR(a,b)$ and $a \wedge b$ for $AND(a,b)$, as well as write $a \vee b \vee c$ as shorthand for $(a \vee b) \vee c$. ($NOT(a)$ is often written as either $\neg a$ or $\overline{a}$; we will use both notations in this book.) With this notation, (3.4) can also be written as

$$MAJ(x_0,x_1,x_2) = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_0 \wedge x_3) . \tag{3.5}$$

We can also write (3.4) in a "programming language" format, expressing it as a set of instructions for computing *MAJ* given the basic operations AND, OR, NOT:

```
def MAJ(X[0],X[1],X[2]):
    firstpair  = AND(X[0],X[1])
    secondpair = AND(X[1],X[2])
    thirdpair  = AND(X[0],X[2])
    temp       = OR(secondpair,thirdpair)
    return OR(firstpair,temp)
```

Yet a third way to describe the same computation is by a *Boolean circuit*. Think of having *wires* that can carry a signal that is either the value 0 or 1. [4] An *OR gate* is a gadget that has two incoming wires and one outgoing wires, and is designed so that if the signals on the incoming wires are a and b respectively (for $a, b \in \{0,1\}$), then the signal on the outgoing wire will be $OR(a,b)$. AND and NOT gates are defined similarly. Using this, we can express (3.4) as a circuit as well:



[4] In practice, this is often implemented by electric potential or *voltage* on a wire, where for example voltage above a certain level is interpreted as a logical value of 1, and below a certain level is interpreted as a logical value of 0.

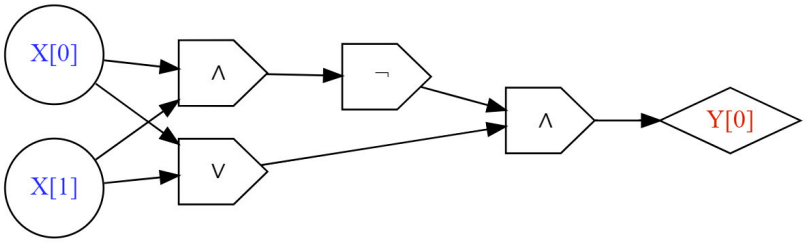
3.2.1 Extended example: Computing *XOR* from *AND*,*OR*,*NOT*.

Let us see how we can obtain a different function from the same building blocks. Define $XOR : \{0,1\}^2 \to \{0,1\}$ to be the function $XOR(a,b) = a + b \mod 2$. That is, $XOR(0,0) = XOR(1,1) = 0$ and $XOR(1,0) = XOR(0,1) = 1$. We claim that we can construct *XOR* using only *AND*, *OR*, and *NOT*.

Here is an algorithm to compute $XOR(a,b)$ using AND, NOT, OR as basic operations:

1. Compute $w1 = AND(a,b)$
2. Compute $w2 = NOT(w1)$
3. Compute $w3 = OR(a,b)$
4. Output $AND(w2,w3)$

We can also express this algorithm as a circuit:



Last but not least, we can also express it in a programming language. Specifically, the following is a *Python* program that computes the *XOR* function:

```
def AND(a,b): return a*b
def OR(a,b):  return 1-(1-a)*(1-b)
def NOT(a): return 1-a

def XOR(a,b):
    w1 = AND(a,b)
    w2 = NOT(w1)
    w3 = OR(a,b)
    return AND(w2,w3)

print([f"XOR({a},{b})={XOR(a,b)}" for a in [0,1] for b in
 ↪  [0,1]])
# ['XOR(0,0)=0', 'XOR(0,1)=1', 'XOR(1,0)=1', 'XOR(1,1)=0']
```

:::

Solved Exercise 3.1 — Compute XOR on three bits of input. Let $XOR_3 : \{0,1\}^3 \to \{0,1\}$ be the function defined as $XOR_3(a,b,c) = a+b+c \mod 2$. That is, $XOR_3(a,b,c) = 1$ if $a+b+c$ is odd, and $XOR_3(a,b,c) = 0$ otherwise. Show that you can compute XOR_3 using AND, OR, and NOT. You can express it as a forumla, use a programming language such as Python, or use a Boolean circuit.

■

Solution:

Addition modulo two satisfies the similar properties such as associativity, commutativity, etc.. as standard addition. This means that, if we define $a \oplus b$ to equal $a + b \mod 2$, then

$$XOR_3(a,b,c) = (a \oplus b) \oplus c \tag{3.6}$$

or in other words

$$XOR_3(a,b,c) = XOR(XOR(a,b),c) \ . \tag{3.7}$$

Since we know how to compute *XOR* using AND, OR, and NOT, we can compose this to compute XOR_3 using the same building blocks. In Python this corresponds to the following program:

```
def XOR3(a,b,c):
    w1 = AND(a,b)
    w2 = NOT(w1)
    w3 = OR(a,b)
    w4 = AND(w2,w3)
    w5 = AND(w4,c)
    w6 = NOT(w5)
    w7 = OR(w4,c)
    return AND(w6,w7)

# Let's test this out
print([f"XOR3({a},{b},{c})={XOR3(a,b,c)}" for a in [0,1]
 ↪  for b in [0,1] for c in [0,1]])
# ['XOR3(0,0,0)=0', 'XOR3(0,0,1)=1', 'XOR3(0,1,0)=1',
 ↪  'XOR3(0,1,1)=0', 'XOR3(1,0,0)=1', 'XOR3(1,0,1)=0',
 ↪  'XOR3(1,1,0)=0', 'XOR3(1,1,1)=1']
```

■

Try to generalize the above examples to obtain a way to compute $XOR_n : \{0,1\}^n \rightarrow \{0,1\}$ for every n using at most $4n$ basic steps involving applications of a function in $\{AND, OR, NOT\}$ to outputs or previously computed values.

3.2.2 Informally defining "basic operations" and "algorithms"

We've seen that we can obtain at least some examples of interesting functions by composing together applications of *AND*, *OR*, and *NOT*. This suggest we can use *AND*, *OR*, and *NOT* as our "basic operations", hence obtaining the following definition of an "algorithm":

Semi-formal definition of an algorithm: An *algorithm* consists of a sequence of steps of the form "compute a new value by applying *AND*, *OR*, or *NOT* to previously computed values".

An algorithm A *computes* a function F if for every input x to F, if we feed x as input to the algorithm, the value computed in its last step is $F(x)$.

There are several concerns that are raised by this definition:

1. First and foremost, this definition is indeed too informal. We do not specify exactly what each step does, nor what it means to "feed x as input".

2. Second, the choice of *AND*, *OR* or *NOT*? Why not *XOR* and *MAJ*? Why not allow operations like addition and multiplication? What about any other logical constructions such `if/then` or `while`?

3. Third, do we even know that this definition has anything to do with actual computing? If someone gave us a description of such an algorithm, could we use it to actually compute the function in the real world?



These concerns will to a large extent guide us in the upcoming chapters. Thus you would be well advised to re-read the above informal definition and see what you think about these issues.

A large part of this course will be devoted to addressing the above issues. We will see that:

1. We can make the definition of an algorithm fully formal, and so give a precise mathematical meaning to statements such as "Algorithm A computes function F".

2. While the choice of *AND*/*OR*/*NOT* is arbitrary, and we could just as well chose some other functions, we will also see this choice does not matter much. We will see that the we would obtain the same computational power if we used instead for addition and multiplication, and essentially every other operation that could be reasonably thought of as a basic step.

3. It turns out that we can and do compute such "*AND*/*OR*/*NOT* based algorithms" in the real world. First of all, such an algorithm is clearly well specified, and so can be executed by a human with a pen and paper. Second, there are a variety of ways to *mechanize* this computation. We've already seen that we can write Python code that corresponds to following such a list of instructions. But in fact we can directly implement operations such as *AND*, *OR*, *NOT* etc.. via electronic signals using components known as *transistors*. This is how modern electronic computers operate.

In the remainder of this chapter, and the rest of this book, we will begin to answer some of these questions. We will see more examples of the power of simple operations to compute more complex operations including addition, multiplication, sorting and more. We will

also discuss how to *physically implement* simple operations such as *AND*, *OR* and *NOT* using a variety of technologies.

3.3 PHYSICAL IMPLEMENTATIONS OF COMPUTING DEVICES.

Computation is an abstract notion, that is distinct from its physical *implementations*. While most modern computing devices are obtained by mapping logical gates to semi-conductor based transistors, over history people have computed using a huge variety of mechanisms, including mechanical systems, gas and liquid (known as *fluidics*), biological and chemical processes, and even living creatures (e.g., see Fig. 3.5 or this video for how crabs or slime mold can be used to do computations).

In this section we will review some of these implementations, both so you can get an appreciation of how it is possible to directly translate NAND-CIRC programs to the physical world, without going through the entire stack of architecture, operating systems, compilers, etc. as well as to emphasize that silicon-based processors are by no means the only way to perform computation. Indeed, as we will see much later in this course, a very exciting recent line of works involves using different media for computation that would allow us to take advantage of *quantum mechanical effects* to enable different types of algorithms.

3.3.1 Transistors

A *transistor* can be thought of as an electric circuit with two inputs, known as *source* and *gate* and an output, known as the *sink*. The gate controls whether current flows from the source to the sink. In a *standard transistor*, if the gate is "ON" then current can flow from the source to the sink and if it is "OFF" then it can't. In a *complementary transistor* this is reversed: if the gate is "OFF" then current can flow from the source to the sink and if it is "ON" then it can't.

There are several ways to implement the logic of a transistor. For example, we can use faucets to implement it using water pressure (e.g. Fig. 3.6).[5] However, the standard implementation uses electrical current. One of the original implementations used *vacuum tubes*. As its name implies, a vacuum tube is a tube containing nothing (i.e., vacuum) and where a priori electrons could freely flow from source (a wire) to the sink (a plate). However, there is a gate (a grid) between the two, where modulating its voltage can block the flow of electrons.

Early vacuum tubes were roughly the size of lightbulbs (and looked very much like them too). In the 1950's they were supplanted by *transistors*, which implement the same logic using *semiconductors* which are materials that normally do not conduct electricity but whose conductivity can be modified and controlled by inserting impurities ("doping") and an external electric field (this is known as the

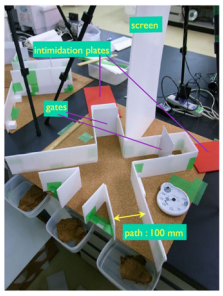


Figure 3.5: Crab-based logic gates from the paper "Robust soldier-crab ball gate" by Gunji, Nishiyama and Adamatzky. This is an example of an AND gate that relies on the tendency of two swarms of crabs arriving from different directions to combine to a single swarm that continues in the average of the directions.

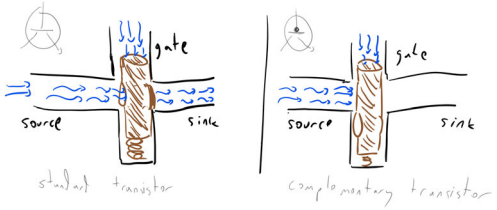


Figure 3.6: We can implement the logic of transistors using water. The water pressure from the gate closes or opens a faucet between the source and the sink.

[5] This might seem as merely a curiosity but there is a field known as fluidics concerned with implementing logical operations using liquids or gasses. Some of the motivations include operating in extreme environmental conditions such as in space or a battlefield, where standard electronic equipment would not survive.

field effect). In the 1960's computers were started to be implemented using *integrated circuits* which enabled much greater density. In 1965, Gordon Moore predicted that the number of transistors per circuit would double every year (see Fig. 3.7), and that this would lead to "such wonders as home computers —or at least terminals connected to a central computer— automatic controls for automobiles, and personal portable communications equipment". Since then, (adjusted versions of) this so-called "Moore's law" has been running strong, though exponential growth cannot be sustained forever, and some physical limitations are already becoming apparent.

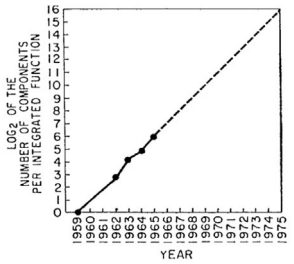


Figure 3.7: The number of transistors per integrated circuits from 1959 till 1965 and a prediction that exponential growth will continue at least another decade. Figure taken from "Cramming More Components onto Integrated Circuits", Gordon Moore, 1965

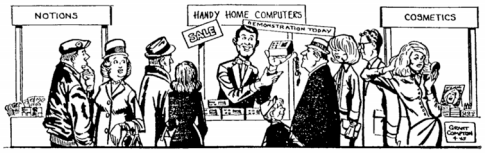


Figure 3.8: Gordon Moore's cartoon "predicting" the implications of radically improving transistor density.

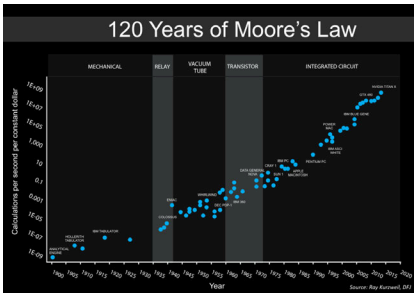


Figure 3.9: The exponential growth in computing power over the last 120 years. Graph by Steve Jurvetson, extending a prior graph of Ray Kurzweil.

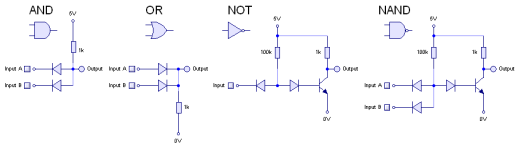


Figure 3.10: Implementing logical gates using transistors. Figure taken from Rory Mangles' website.

3.3.2 Logical gates from transistors

We can use transistors to implement various Boolean functions such as *AND*, *OR*, *NOT*, and *NAND*. For each a two-input gate $G : \{0,1\}^2 \to \{0,1\}$, such an implementation would be a system with two input wires x, y and one output wire z, such that if we identify high voltage with "1" and low voltage with "0", then the wire z will equal to "1" if and only if applying G to the values of the wires x and y is 1 (see Fig. 3.10 and Fig. 3.11). This means that there exists a AND/OR/NOT circuit to compute a function $g : \{0,1\}^n \to \{0,1\}^m$, then we can compute g in the physical world using transistors as well.

3.3.3 Biological computing

Computation can be based on biological or chemical systems. For example the *lac* operon produces the enzymes needed to digest lactose only if the conditions $x \wedge (\neg y)$ hold where x is "lactose is present" and y is "glucose is present". Researchers have managed to create transistors, and from them the NAND function and other logic gates, based on DNA molecules (see also Fig. 3.12). One motivation for DNA computing is to achieve increased parallelism or storage density; another is to create "smart biological agents" that could perhaps be injected into bodies, replicate themselves, and fix or kill cells that were damaged by a disease such as cancer. Computing in biological systems is not restricted of course to DNA. Even larger systems such as flocks of birds can be considered as computational processes.

3.3.4 Cellular automata and the game of life

Cellular automata is a model of a system composed of a sequence of *cells*, which of which can have a finite state. At each step, a cell updates its state based on the states of its *neighboring cells* and some simple rules. As we will discuss later in this course, cellular automata such as Conway's "Game of Life" can be used to simulate computation gates (see Fig. 3.13).

3.3.5 Neural networks

One computation device that we all carry with us is our own *brain*. Brains have served humanity throughout history, doing computations that range from distinguishing prey from predators, through making scientific discoveries and artistic masterpieces, to composing witty 280 character messages. The exact working of the brain is still not fully understood, but it seems that to a first approximation it can be modeled by a (very large) *neural network*.

A neural network can be thought of as a Boolean circuit that instead of *AND*/*OR*/*NOT* uses some other gates as the basic basis. For example, one particular basis we can use are *threshold gates*. For every vector $w = (w_0, \ldots, w_{k-1})$ of integers and integer t (some or all of whom could be negative), the *threshold function corresponding to* w, t is the function $T_{w,t} : \{0,1\}^k \to \{0,1\}$ that maps $x \in \{0,1\}^k$ to 1 if and only if $\sum_{i=0}^{k-1} w_i x_i \geq t$. For example, the threshold function $T_{w,t}$ corresponding to $w = (1,1,1,1,1)$ and $t = 3$ is simply the majority function MAJ_5 on $\{0,1\}^5$. For example the negation of AND (known as *NAND*) corresponds to the threshold function corresponding to $w = (-1,-1)$ and $t = -1$, since $NAND(x_0, x_1) = 1$ if and only if $x_0 + x_1 \leq 1$ or equivalently, $-x_0 - x_1 \geq -1$.[6]

Threshold gates can be thought of as an approximation for *neuron cells* that make up the core of human and animal brains. To a first approximation, a neuron has k inputs and a single output and the neurons "fires" or "turns on" its output when those signals pass some threshold.

3.3.6 The marble computer

We can implement computation using many other physical media, without need for any electronic, biological, or chemical components. Many suggestions for *mechanical* computers have been put forward, starting with Charles Babbage's 1837 plan for a mechanical "Analytical Engine".

As one example, Fig. 3.14 shows a simple implementation of a NAND gate using marbles going through pipes. We represent a logical value in $\{0,1\}$ by a pair of pipes, such that there is a marble flowing through exactly one of the pipes. We call one of the pipes the "0 pipe" and the other the "1 pipe", and so the identity of the pipe containing the marble determines the logical value. A NAND gate would correspond to some mechanical object with two pairs of incoming pipes and one pair of outgoing pipes, such that for every $a, b \in \{0,1\}$, if two marble are rolling toward the object in the a pipe of the first pair and the b pipe of the second pair, then a marble will roll out of the object in the $NAND(a,b)$-pipe of the outgoing pair.

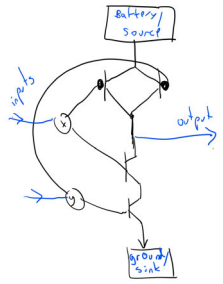


Figure 3.11: Implementing a NAND gate using transistors.

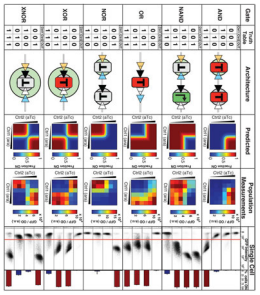


Figure 3.12: Performance of DNA-based logic gates. Figure taken from paper of Bonnet et al, Science, 2013.

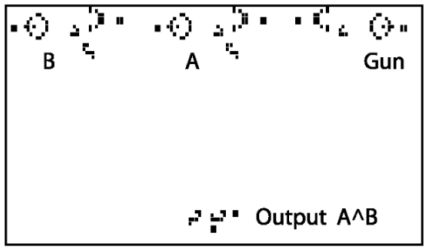


Figure 3.13: An AND gate using a "Game of Life" configuration. Figure taken from Jean-Philippe Rennard's paper.

[6] Threshold is just one example of gates that can used by neural networks. More generally, a neural network is often described as operating on signals that are real numbers, rather than 0/1 values, and where the output of a gate on inputs $x_0, \ldots, x_{k-1}$ is obtained by applying $f(\sum_i w_i x_i)$ where $f : \mathbb{R} \to \mathbb{R}$ is an an activation function such as rectified linear unit (ReLU), Sigmoid, or many others. However, for the purpose of our discussion, all of the above are equivalent. In particular we can reduce the real case to the binary case by a real number in the binary basis, and multiplying the weight of the bit corresponding to the ith digit by 2^i.

As shown in Fig. 3.14, we can achieve such a NAND gate in a fairly straightforward way, together with a gadget that ensures that at most one marble flows in each wire. Such NAND gates can be combined together to form for every n-input NAND circuit P a physical computer that simulates P in the sense that if the marbles are placed in its ingoing pipes according to some input $x \in \{0,1\}^n$, then eventually marbles will come out of its outgoing pipes according to the output $P(x)$.[7] In fact, there is even a commercially-available educational game that uses marbles as a basis of computing, see Fig. 3.16.

3.4 BOOLEAN CIRCUITS

As we've seen above, a *Boolean Circuit* is obtained by connecting AND, OR, and NOT gates via wires so as to produce an output from an input. Formally, we define a circuit as a directed acyclic graph (DAG). The vertices of the graph correspond to the gates and inputs of the circuit, and the edges of the graph correspond to the wires. Therefore we can:

- Formally define a Boolean circuit as a mathematical object.
- Formally define what it means for a circuit C computing a function f.
- So that statements such as "There is a Boolean circuit of 5 gates that computes the function $MAJ : \{0,1\}^3 \to \{0,1\}$" become mathematical theorems with a full formal proof.

We now proceed to do so.

Definition 3.1 — Boolean Circuits. Let n, m, s be positive integers with $s \geq m$. A *Boolean circuit* with n inputs, m outputs, and s gates, is a labeled directed acyclic graph (DAG) $G = (V, E)$ with $s+n$ vertices satisfying the following properties:

- Exactly n of the vertices have no in-neighbors. These vertices are known as *inputs* and are labeled with the n labels `X[0]`, ..., `X[`$n-1$`]`.
- The other s vertices are known as *gates*. Each one of them is labeled with $\wedge$, $\vee$ or $\neg$. Gates labeled with $\wedge$ or $\vee$ have two in-neighbors. Gates labeled with $\neg$ have one in-neighbor. We will allow parallel edges (and so for example an AND gate can have both its in-neighbors be the same vertex).

[7] If our circuit uses the same value as input to more than one gate then we will need also a "copying gadget", that given input $a \in \{0,1\}$ outputs two copies of a. However, such a gadget is easy to construct using the same ideas, and we leave doing so as an exercise for the reader.

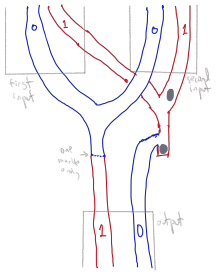


Figure 3.14: A physical implementation of a NAND gate using marbles. Each wire in a Boolean circuit is modeled by a pair of pipes representing the values 0 and 1 respectively, and hence a gate has four input pipes (two for each logical input) and two output pipes. If one of the input pipes representing the value 0 has a marble in it then that marble will flow to the output pipe representing the value 1. (The dashed line represent a gadget that will ensure that at most one marble is allowed to flow onward in the pipe.) If both the input pipes representing the value 1 have marbles in them, then the first marble will be stuck but the second one will flow onwards to the output pipe representing the value 0.

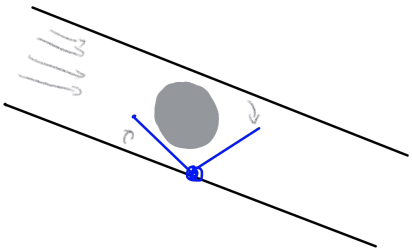


Figure 3.15: A "gadget" in a pipe that ensures that at most one marble can pass through it. The first marble that passes causes the barrier to lift and block new ones.

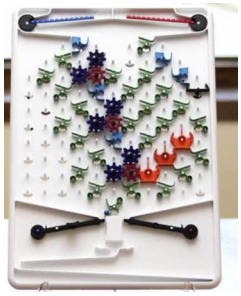


Figure 3.16: The game "Turing Tumble" contains an imlementation of logical gates using marbles.

- Exactly m of the gates are also labeled with the m labels `Y[0]`, ..., `Y[`$m-1$`]` (in addition to their label $\wedge/vee/\neg$). These are known as *outputs*.

If C is a circuit with n inputs and m outputs, and $x \in \{0,1\}^n$, then we can compute the output of C on the input x in the natural way: assign the input vertices `X[0]` , `X[`$n-1$`]` the values $x_0, \ldots, x_{n-1}$, apply each gate on the values of its in-neighbors, and then output the values that correspond to the output vertices. Formally, this is defined as follows:

Definition 3.2 — Computing a function via a Boolean circuit. Let C be a Boolean circuit with n inputs and m outputs. For every $x \in \{0,1\}^n$, the *output* of C on the input x, denoted by $C(x)$, is defined as the result of the following process:

We let $h : V \to \mathbb{N}$ be a *minimal layering* of C (see Theorem 1.30). We let L be the maximum layer of h, and for $\ell = 0, 1, \ldots, L$ we do the following:

- For every v in the ℓ-th layer (i.e., v such that $h(v) = \ell$) do:
  - If v is an input vertex labeled with `X[`i`]` for some $i \in [n]$, then we assign to v the value x_i. [8]
  - If v is a gate vertex labeled with $\wedge$ and with two in-neighbors u, w then we assign to v the AND of the values assigned to u and w. [9]
  - If v is a gate vertex labeled with $\vee$ and with two in-neighbors u, w then we assign to v the OR of the values assigned to u and w.
  - If v is a gate vertex labeled with $\neg$ and with one in-neighbor u then we assign to v the negation of the value assigned to u.
- The result of this process is the value $y \in \{0,1\}^m$ such that for every $j \in [m]$, y_j is the value assigned to the vertex with label `Y[`j`]`.

Let $f : \{0,1\}^n \to \{0,1\}^m$. We say that the circuit C *computes* f if for every $x \in \{0,1\}^n$, $C(x) = f(x)$.

[8] Since h is a minimal layering, all the input vertices will be in the 0-th layer.

[9] Note that since u, w are in-neighbors of v, they are in lower layer than v, and hence their value has already been assigned.

3.5 EQUIVALENCE OF CIRCUITS AND STRAIGHT-LINE PROGRAMS

We have seen two ways to describe how to compute a function f using AND, OR and NOT:

- A *Boolean circuit*, defined in Definition 3.1, computes f by connecting via wires AND, OR, and NOT gates to the inputs.
- We can also describe such a computation using a *straight-line program* that has lines of the form `foo = AND(bar,blah)`, `foo = OR(bar,blah)` and `foo = NOT(bar)` where `foo`, `bar` and `blah` are variable names. (We call this a *straight-line program* since it contains no loops or if/then statements.)

We will now formally define the AON-CIRC programming language ("AON" stands for AND/OR/NOT) which has the above operations, and show that it is equivalent to Boolean circuits.

Definition 3.3 — AON-CIRC Programming language. An *AON-CIRC program* is a string of lines of the form `foo = AND(bar,blah)`, `foo = OR(bar,blah)` and `foo = NOT(bar)` where `foo`, `bar` and `blah` are variable names. [10] Variables of the form `X[`i`]` are known as *input* variables, and variables of the form `Y[`j`]` are known as *output* variables. In every line, the variables on the righthand side of the assignment operators must either be input variables or variables that have already been assigned a value before.

If an AON-CIRC program P has input variables `X[0]`,...,`X[`$n-1$`]` and output variables `Y[0]`,..., `Y[`$m-1$`]` then for every $x \in \{0,1\}^n$, we define the *output of P on input x*, denoted by $P(x)$, to be the value $y \in \{0,1\}^m$ corresponding to the values of the output variables `Y[0]` ,..., `Y[`$m-1$`]` in the execution of P where we initialize the input variables `X[0]`,...,`X[`$n-1$`]` to the values $x_0, \ldots, x_{n-1}$.

We say that such an AON-CIRC program P *computes* a function $f : \{0,1\}^n \to \{0,1\}^m$ if $P(x) = f(x)$ for every $x \in \{0,1\}^n$.

Note that we can easily write code to evaluate an AON-CIRC program on an input of our choice

```
def EVAL(code,X):
    """Evaluate code on input X."""
    n,m = numinout(code) # helper function - get number of
     ↪  inputs and outputs of program by searching for
     ↪  substrings of form X[i] and Y[j]

    vtable = { f"X[{i}]":int(X[i]) for i in range(n)}
    # table of variable values, initially only contains
     ↪  input variables

    for line in code.split("\n"):
        if not line: continue
        foo,op,bar,blah = parseline(line,2)
```

[10] We follow the common programming languages convention of using names such as `foo`, `bar`, `baz`, `blah` as stand-ins for generic identifiers. A variable identifier in our programming language can be any combination of letters, numbers, underscores, and brackets. The appendix contains a full formal specification of our programming language.

```
        # helper function - split "foo = OP(,blah)" to list
         ↪  ["foo","OP","bar","blah"]
        # 2 is num of arguments to expect: blah is empty
         ↪  if it's missing
        if op=="NOT": vtable[foo] = NOT(vtable[bar])
        if op=="AND": vtable[foo] =
         ↪  AND(vtable[bar],vtable[blah])
        if op=="OR": vtable[foo] =
         ↪  OR(vtable[bar],vtable[blah])

    return [vtable[f"Y[{j}]"] for j in range(m)]
```

It turns out that AON-CIRC programs and Boolean circuits have exactly the same power:

Theorem 3.4 — Equivalence of circuits and straight-line programs. Let $f : \{0,1\}^n \to \{0,1\}^m$ and $s \geq m$ be some number. Then f is computable by a Boolean circuit of s gates if and only if f is computable by an AON-CIRC program of s lines.

Proof Idea:

The idea is simple - AON-CIRC program and Boolean circuits are just different ways of describing the exact same computational process. For example, an AND gate in a Boolean circuit corresponding to computing the AND of two previously-computed values. In a AON-CIRC program this will correspond to the line that stores in a variable the AND of two previously-computed variables.

⋆



This proof is simple at heart, but all the details it contains can make it a little cumbersome to read. You might be better off trying to work it out yourself before reading it. We will also show a "proof by Python" that might help in clarifying these details.

Proof of Theorem 3.4. Let $f : \{0,1\}^n \to \{0,1\}^m$. Since the theorem is an "if and only if" statement, to prove it we need to show both directions: translating an AON-CIRC program that computes f into a circuit that computes f, and translating a circuit that computes f into an AON-CIRC program that does so.

We start with the first direction. Let P be an s line AON-CIRC that computes f. We define a circuit C as follows: the circuit will have n inputs and s gates. For every $i \in [s]$, if the i-th line has the form `foo`

= AND(bar,blah) then the i-th gate in the circuit will be an AND gate that is connected to gates j and k where j and k correspond to the last lines before i where the variables bar and blah (respectively) where written to. (For example, if $i = 57$ and the last line bar was written to is 35 and the last line blah was written to is 17 then the two in-neighbors of gate 57 will be gates 35 and 17.) If either bar or blah is an input variable then we connect the gate to the corresponding input vertex instead. If foo is an output variable of the form Y[j] then we add the same label the corresponding gate to mark it as an output gate. We do the analogous operations if the i-th line involves an OR or a NOT operation (except that we use an OR, or a NOT, gate, and in the latter case have only one in-neighbor instead of two). For every input $x \in \{0,1\}^n$, if we run the program P on x, then the value written that is computed in the i-th line is exactly the value that will be assigned to the i-th gate if we evaluate the circuit C on x. Hence $C(x) = P(x)$ for every $x \in \{0,1\}^n$.

For the other direction, let C be a circuit of s gates and n inputs that computes the function f. We sort the gates according to a topological order and write them as $v_0, \ldots, v_{s-1}$. We now can create a program P of s lines as follows. For every $i \in [s]$, if v_i is an AND gate with in-neighbors v_j, v_k then we will add a line to P of the form temp_i = AND(temp_j,temp_k), unless one of the vertices is an input vertex or an output gate, in which case we change this to the form X[.] or Y[.] appropriately. Because we work in topological ordering, we are guaranteed that the in-neighbors v_j and v_k correspond to variables that have already been assigned a value. We do the same for OR and NOT gate. Once again, one can verify that for every input x, the value $P(x)$ will equal $C(x)$ and hence the program computes the same function as the circuit.

■

3.5.1 "Proof by Python"

The proof of Theorem 3.4 is *constructive*. It yields a way of transforming an AON-CIRC program into an equivalent Boolean circuit and vice versa. Below is the code that carries out this transformation. (It uses some "helper" functions and objects: see our GitHub repository for the full implementation.) **??** shows an example of the result of this transformation.

```
def circuit2prog(C):
    """Transform circuit to a program."""

    code = ""
    def key2var(key):
```

```
        """Helper function: translate  key identifying a
         ↪  node into a variable name"""
        if key[:6]=="input_": return f"X[{key[6:]}]"
        elif key in C.outputs: return
         ↪  f"Y[{C.outputs[key]}]"
        return key

    # every gate is translated into a line
    for (key,n) in C.nodes.items():
        # we assume nodes are in topological ordering,
         ↪  otherwise  should layer first
        if n[0]!="GATE": continue  # ignore input (non
         ↪  gate) nodes
        args = ",".join(map(key2var,C.in_neighbors[key]))
        code += f"{key2var(key)} =
         ↪  {n[1].__name__}({args})\n"

    return code

def prog2circuit(code,gateset=None):
    """Transform a straight-line program into a circuit.
       Takes as input the basic gates one uses (otherwise
 ↪  use all functions currently defined)"""
    if not gateset: gateset = globals()
    n,m = numinout(code) # helper function - extract
     ↪  number of inputs and outputs from code
    C = Circuit(n) # create circuit with n inputs

    nodes = { f"X[{i}]" : C.X[i] for i in range(n) }
    # initially we have n nodes corresponding to n inputs.

    for line in code.split("\n"): # every line is
     ↪  translated to a new gate
        if not line: continue
        foo,op,bar,blah = parseline(line,2)
        # parseline takes "foo = OP(bar,blah)" to the list
         ↪  ["foo","OP","bar","blah"]
        if blah: g =
         ↪  C.gate(gateset[op],nodes[bar],nodes[blah])
        else: g = C.gate(gateset[op],nodes[bar])
        nodes[foo] = g
        if foo[0]=="Y": C.output(g,int(foo[2:-1]))

    return C
```

3.6 THE NAND FUNCTION

Here is another function we can compute using AND, OR, NOT. The $NAND$ function maps $\{0,1\}^2$ to $\{0,1\}$ and is defined as

$$NAND(a,b) = \begin{cases} 0 & a=b=1 \\ 1 & \text{otherwise} \end{cases} \tag{3.8}$$

As its name implies, $NAND$ is the NOT of AND (i.e., $NAND(a,b) = NOT(AND(a,b))$), and so we can clearly compute $NAND$ using AND and NOT. Interestingly, the opposite direction also holds:

Theorem 3.5 — NAND computes AND,OR,NOT. We can compute AND, OR, and NOT by composing only the $NAND$ function.

Proof. We start with the following observation. For every $a \in \{0,1\}$, $AND(a,a) = a$. Hence, $NAND(a,a) = NOT(AND(a,a)) = NOT(a)$. This means that $NAND$ can compute NOT, and since by the principle of "double negation", $AND(a,b) = NOT(NOT(AND(a,b)))$ this means that we can use $NAND$ to compute AND as well. Once we can compute AND and NOT, we can compute OR using the so called "De Morgan's Law": $OR(a,b) = NOT(AND(NOT(a), NOT(b)))$ (which can also be written as $a \vee b = \overline{\overline{a} \wedge \overline{b}}$) for every $a, b \in \{0,1\}$. ■

P

Theorem 3.5's proof is very simple, but you should make sure that **(i)** you understand the statement of the theorem, and **(ii)** you follow its proof completely. In particular, you should make sure you understand why De Morgan's law is true.

R

Remark 3.6 — Verify NAND's universality by Python (optional). If you are so inclined, you can also verify the proof of Theorem 3.5 by Python:

```
def NAND(a,b): return 1-a*b

def ORwithNAND(a,b):
    return NAND(NAND(a,a),NAND(b,b))

print([f"Test {a},{b}: {ORwithNAND(a,b)==OR(a,b)}"
 ↪  for a in [0,1] for b in [0,1]])
# ['Test 0,0: True', 'Test 0,1: True', 'Test 1,0:
 ↪  True', 'Test 1,1: True']
```

Solved Exercise 3.2 — Compute majority with NAND. Let $MAJ : \{0,1\}^3 \to \{0,1\}$ be the function that on input a, b, c outputs 1 iff $a + b + c \geq 2$. Show how to compute *MAJ* using a composition of *NAND*'s.

Solution:

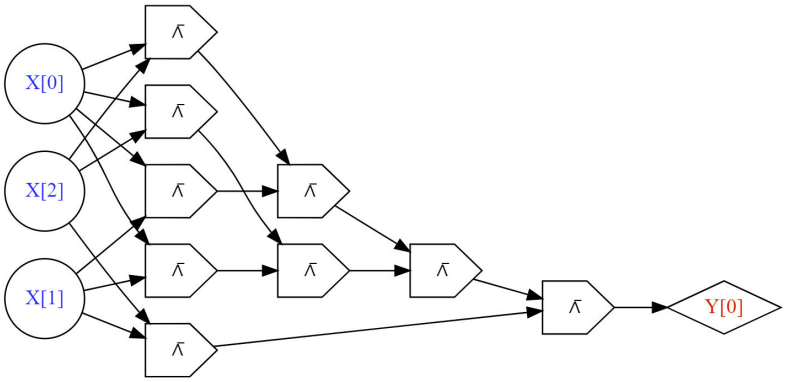
Recall that (3.4) stated that

$$MAJ(x_0, x_1, x_2) = OR\left(\, AND(x_0, x_1) \;,\; OR(AND(x_1, x_2) \;,\; AND(x_0, x_2))\,\right) \;. \tag{3.9}$$

We can use Theorem 3.5 to replace all the occurrences of *AND* and *OR* with *NAND*'s. Specifically, we can use the equivalence $AND(a,b) = NOT(NAND(a,b))$, $OR(a,b) = NAND(NOT(a), NOT(b))$, and $NOT(a) = NAND(a,a)$ to replace the righthand side of (3.9) with an expression involving only *NAND*, yielding that $MAJ(a,b,c)$ is equivalent the (somewhat unwieldy) expression

$$\begin{aligned} NAND\Big(\, & NAND\Big(\, NAND(NAND(a,b), NAND(a,c)), \\ & NAND(NAND(a,b), NAND(a,c))\,\Big), \\ & NAND(b,c)\,\Big) \end{aligned} \tag{3.10}$$

This corresponds to the following circuit with *NAND* gates:



3.6.1 NAND Circuits

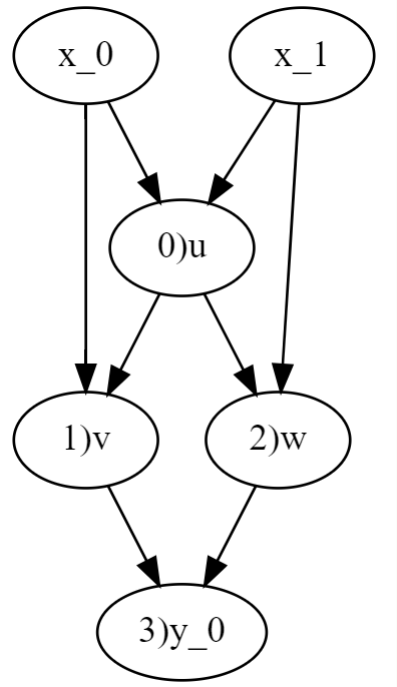
We can define *NAND Circuits* to be circuits in which all the gates are NAND operations. Such a circuit again corresponds to a directed acyclic graph (DAG) but it is even simpler than general Boolan circuits: all the gates correspond to the same function (i.e., NAND) and all of them have in-degree exactly two. Despite their simplicity, NAND circuits can be quite powerful.

■ **Example 3.7 — $NAND$ circuit for XOR.** Recall the *XOR* function which maps $x_0, x_1 \in \{0,1\}$ to $x_0 + x_1 \mod 2$. We have seen in **??** that we can compute this function using *AND*, *OR*, and *NOT*, and so by Theorem 3.5 we can compute it using only *NAND*'s. However, the following is a direct construction of computing *XOR* by a sequence of NAND operations:

1. Let $u = NAND(x_0, x_1)$.
2. Let $v = NAND(x_0, u)$
3. Let $w = NAND(x_1, u)$.
4. The *XOR* of x_0 and x_1 is $y_0 = NAND(v, w)$.

(We leave it to you to verify that this algorithm does indeed compute *XOR*.)

We can also represent this algorithm graphically as a circuit:



In fact, we can show the following theorem:

Theorem 3.8 — NAND is a universal operation. For every Boolean circuit C of s gates, there exists a NAND circuit C' of at most $3s$ gates that computes the same function as C.

Proof Idea:

The idea of the proof is to just replace every *AND*, *OR* and *NOT* gate with their NAND implementation following the proof of Theorem 3.5.

⋆

Proof of Theorem 3.8. If C is a Boolean circuit, then since, as we've seen in the proof of Theorem 3.5, for every $a, b \in \{0,1\}$

- $NOT(a) = NAND(a,a)$
- $AND(a,b) = NAND(NAND(a,b), NAND(a,b))$
- $OR(a,b) = NAND(NAND(a,a), NAND(b,b))$

we can replace every gate of C with at most three *NAND* gates to obtain an equivalent circuit C'. The resulting circuit will have at most $3s$ gates.

■

Big Idea 3 Once we have shown that two models such AND/OR/NOT circuits and NAND circuits are *computationally equivalent*, we can translate between one model to the other freely. Therefore we can always choose the model that is most convenient for the task at hand.

3.6.2 More examples of NAND circuits (optional)

Here are some more sophisticated examples of NAND circuits

■ **Example 3.9 — $NAND$ circuit for incrementing.** Consider the task of computing, given as input a string $x \in \{0,1\}^n$ that represents a natural number $X \in \mathbb{N}$, the representation of $X + 1$. That is, we want to compute the function $INC_n : \{0,1\}^n \to \{0,1\}^{n+1}$ such that for every $x_0, \ldots, x_{n-1}$, $INC_n(x) = y$ which satisfies $\sum_{i=0}^{n} y_i 2^i = \left(\sum_{i=0}^{n-1} x_i 2^i\right) + 1$. (For simplicity of notation in this example we will use the representation where the least significant digit is first rather than last.)

The increment operation can be very informally described as follows: *"Add 1 to the least significant bit and propagate the carry"*. A little more precisely, in the case of the binary representation, to obtain the increment of x, we scan x from the least significant bit onwards, and flip all 1's to 0's until we encounter a bit equal to 0, in which case we flip it to 1 and stop. (Please verify you understand why this is the case.)

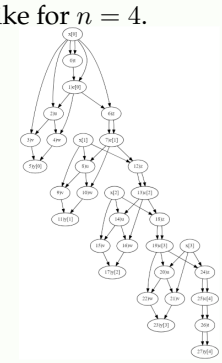
Thus we can compute the increment of $x_0, \ldots, x_{n-1}$ by doing the following:

1. Set $c_0 = 1$ (we pretend we have a "carry" of 1 initially)
2. For $i = 0, \ldots, n-1$ do the following:

a. Let $y_i = XOR(x_i, c_i)$.
b. If $c_i = x_i = 1$ then $c_{i+1} = 1$, else $c_{i+1} = 0$.

3. Set $y_n = c_n$.

The above is a very precise description of an algorithm to compute the increment operation, and can be easily transformed into *Python* code that performs the same computation, but it does not seem to directly yield a NAND circuit to compute this. However, we can transform this algorithm line by line to a NAND circuit. For example, since for every a, $NAND(a, NOT(a)) = 1$, we can replace the initial statement $c_0 = 1$ with $c_0 = NAND(x_0, NAND(x_0, x_0))$. We already know how to compute *XOR* using NAND, so line 2.a can be replaced by some NAND operations. Next, we can write line 2.b as simply saying $c_{i+1} = AND(y_i, x_i)$, or in other words $c_{i+1} = NAND(NAND(y_i, x_i), NAND(y_i, x_i))$. Finally, the assignment $y_n = c_n$ can be written as $y_n = NAND(NAND(c_n, c_n), NAND(c_n, c_n))$. Combining these observations yields for every $n \in \mathbb{N}$, a *NAND* circuit to compute INC_n. For example, this is how this circuit looks like for $n = 4$.



■ **Example 3.10 — Addition using NANDs.** Once we have the increment operation, we can certainly compute addition by repeatedly incrementing (i.e., compute $x + y$ by performing $INC(x)$ y times). However, that would be quite inefficient and unnecessary. With the same idea of keeping track of carries we can implement the "grade-school" algorithm for addition to compute the function $ADD_n : \{0,1\}^{2n} \to \{0,1\}^{n+1}$ that on input $x \in \{0,1\}^{2n}$ outputs the

binary representation of the sum of the numbers represented by $x_0, \ldots, x_{n-1}$ and $x_{n+1}, \ldots, x_n$:

1. Set $c_0 = 0$.
2. For $i = 0, \ldots, n-1$:
   i. Let $y_i = x_i + x_{n+i} + c_i \pmod 2$.
   ii. If $x_i + x_{n+i} + c_i \geq 2$ then $c_{i+1} = 1$.
3. Let $y_n = c_n$

Once again, this can be translated into a NAND circuit. To transform Step 2.b to a NAND circuit we use the fact (shown in Solved Exercise 3.2) that the function $MAJ_3 : \{0,1\}^3 \to \{0,1\}$ can be computed using *NAND*s.

3.6.3 The NAND-CIRC Programming language

Just like we did for Boolean circuits, we can define a programming language analog of NAND circuits. It is even simpler than the AON-CIRC language since we only have a single operation. We define the *NAND-CIRC Programming Language* to be a programming language where every line has the following form:

```
foo = NAND(bar,blah)
```

where `foo`, `bar` and `blah` are variable identifiers.

■ **Example 3.11 — Our first NAND-CIRC program.** Here is an example of a NAND-CIRC program:

```
u = NAND(X[0],X[1])
v = NAND(X[0],u)
w = NAND(X[1],u)
Y[0] = NAND(v,w)
```



Do you know what function this program computes?
Hint: you have seen it before.

We can formally define the notion of computation by a NAND-CIRC program in the natural way:

Definition 3.12 — Computing by a NAND-CIRC program. Let $f : \{0,1\}^n \to \{0,1\}^m$ be some function, and let P be a NAND-CIRC program. We say that P *computes* the function F if:

1. P has n input variables `X[0]`, ... ,`X[`$n-1$`]` and m output variables `Y[0]`,...,`Y[`$m-1$`]`.

2. For every $x \in \{0,1\}^n$, if we execute P when we assign to `X[0]`, ... ,`X[`$n-1$`]` the values $x_0, \ldots, x_{n-1}$, then at the end of the execution, the output variables `Y[0]`,...,`Y[`$m-1$`]` have the values $y_0, \ldots, y_{m-1}$ where $y = f(x)$.

As before we can show that NAND circuits are equivalent to NAND-CIRC programs (see Fig. 3.18):

Theorem 3.13 — NAND circuits and straight-line program equivalence. For every $f : \{0,1\}^n \rightarrow \{0,1\}^m$ and $s \geq m$, f is computable by a NAND-CIRC program of s lines if and only if f is computable by a NAND circuit of s gates.

We omit the proof of Theorem 3.13 since it follows along exactly the same lines as the equivalence of Boolean circuits and AON-CIRC program (Theorem 3.4). Given Theorem 3.13 and Theorem 3.8, we know that we can translate every s-line AON-CIRC program P into an equivalent NAND-CIRC program of at most $3s$ lines. In fact, this translation can be easily done by replacing every line of the form `foo = AND(bar,blah)`, `foo = OR(bar,blah)` or `foo = NOT(bar)` with the equivalent 1-3 lines that use the `NAND` operation.

Here is a Here is a "proof by code": a simple Python program that translates an input AON-CIRC program into an equivalent NAND-CIRC program:

```
def AON2NAND(code):
    """Translate an AON-CIRC program to an equivalent
     ↪ NAND-CIRC program"""
    output = ""
    counter = 0
    for line in code.split("\n"):
        if not line: continue
        foo,op,bar,blah = parseline(line,2)
        if op=="NOT":
            output += f"{foo} = NAND({bar},{bar})\n"
        if op=="AND":
            output += f"temp_{counter} =
             ↪ NAND({bar},{blah})\n"
            output += f"{foo} =
             ↪ NAND(temp_{counter},temp_{counter})\n"
            counter +=1
        if op=="OR":
```

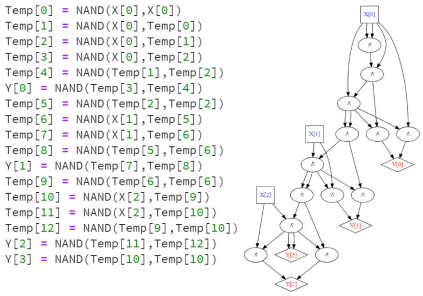


Figure 3.18: The NAND code and the corresponding circuit for a program to compute the *increment* function that maps a string $x \in \{0,1\}^3$ (which we think of as a number in $[7]$) to the string $y \in \{0,1\}^4$ that represents $x+1$. Note how every line in the program corresponds to a gate in the circuit.

```
            output += f"temp_{counter} =
             ↪  NAND({bar},{bar})\n"
            output += f"temp_{counter+1} =
             ↪  NAND({blah},{blah})\n"
            output += f"{foo} =
             ↪  NAND(temp_{counter},temp_{counter+1})\n"
            counter +=2
    return output

# The AON-CIRC code
t1      = AND(X[0],X[1])
notx0   = NOT(X[0])
t2      = AND(notx0,X[2])
Y[0]    = OR(t1,t2)

# will be translated into the NAND-CIRC code
temp_0 = NAND(X[0],X[1])
t1 = NAND(temp_0,temp_0)
notx0 = NAND(X[0],X[0])
temp_1 = NAND(notx0,X[2])
t2 = NAND(temp_1,temp_1)
temp_2 = NAND(t1,t1)
temp_3 = NAND(t2,t2)
Y[0] = NAND(temp_2,temp_3)
```



Remark 3.14 — Is the NAND-CIRC programming language Turing Complete? (optional note). You might have heard of a term called "Turing Complete" that is sometimes used to describe programming languages. (If you haven't, feel free to ignore the rest of this remark: we will encounter this term later in this course and define it properly.) If so, you might wonder if the NAND-CIRC programming language has this property. The answer is **no**, or perhaps more accurately, the term is not really applicable for the NAND-CIRC programming language. The reason is that, by design, the NAND-CIRC programming language can only compute *finite* functions $F : \{0,1\}^n \to \{0,1\}^m$ that take a fixed number of input bits and produce a fixed number of outputs bits. The term "Turing Complete" is really only applicable to programming languages for *infinite* functions that can take inputs of arbitrary length. We will come back to this distinction later on in the course.

3.7 EQUIVALENCE OF ALL THESE MODELS

If we put together Theorem 3.4, Theorem 3.8, and Theorem 3.13 we obtain the following result:

Theorem 3.15 — Equivalence between models of finite computation. For every sufficiently large s, n, m and $f : \{0,1\}^n \to \{0,1\}^m$, the following conditions are all equivalent to one another:

- f can be computed by a Boolean circuit (with $\wedge, \vee, \neg$ gates) of at most $O(s)$ gates.
- f can be computed by an AON-CIRC straight-line program of at most $O(s)$ lines.
- f can be computed by a NAND circuit of at most $O(s)$ gates.
- f can be computed by a NAND-CIRC straight-line program of at most $O(s)$ lines.

By "$O(s)$" we mean that the bound is at most $c \cdot s$ where c is a constant that is independent of n. For example, if f can be computed by a Boolean circuit of s gates, then it can be computed by a NAND-CIRC program of at most $3s$ lines, and if f can be computed by a NAND circuit of s gates, then it can be computed by an AON-CIRC program of at most $2s$ lines.

Proof Idea:

We omit the formal proof since it just involved putting together Theorem 3.4, Theorem 3.8, and Theorem 3.13. We can translate a program/circuit that compute f in one model into a program/circuit that computes f in another model by increasing the lines/gates by at most a constant factor (in fact this constant factor is at most 3).

⋆

Theorem 3.4 is actually a special case of a more general result. We can consider even more general models of computation, where instead of AND/OR/NOT or NAND, we use other operations (see Section 3.7.1 below). It turns out that Boolean circuits are equivalent in power to such models as well. The fact that all these different ways to define computation lead to equivalent models shows that we are "on the right track". It justifies the seemingly arbitrary choices that we've made of using AND/OR/NOT or NAND as our basic operations, since these choices do not affect the computational model of our power.

R

Remark 3.16 — Moving freely between circuits and programs. Equivalence results such as Theorem 3.15 mean that we can easily translate between Boolean circuits, NAND circuits, NAND-CIRC programs and the like. We will use this ability later on in this book, often shifting to the most convenient formulation without making a big deal about it. Hence we will not worry too much about the distinction between, for example, Boolean circuits and NAND-CIRC programs.

In contrast, we will continue to take special care to distinguish between *circuits/programs* and *functions* (recall Big Idea 2). A function corresponds to a *specification* of a computational task, and it is a fundamentally different object than a program or a circuit, which corresponds to the *implementation* of the task.

3.7.1 Circuits with other gate sets (optional)

There is nothing special about AND/OR/NOT or NAND. For every set of functions $\mathcal{G} = \{G_0, \ldots, G_{k-1}\}$, we can define a notion of circuits that use elements of $\mathcal{G}$ as gates, and a notion of a "$\mathcal{G}$ programming language" where every line involves assigning to a variable `foo` the result of applying some $G_i \in \mathcal{G}$ to previously defined or input variables. Specifically, we can make the following definition:

Definition 3.17 — General straight-line programs. Let $\mathcal{F} = \{f_0, \ldots, f_{t-1}\}$ be a finite collection of Boolean functions, such that $f_i : \{0,1\}^{k_i} \rightarrow \{0,1\}$ for some $k_i \in \mathbb{N}$. An $\mathcal{F}$ *program* is a sequence of lines, each of which assigns to some variable the result of applying some $f_i \in \mathcal{F}$ to k_i other variables. As above, we use `X[`i`]` and `Y[`j`]` to denote the input and output variables.

AON-CIRC programs correspond to $\{AND, OR, NOT\}$ programs, NAND-CIRC programs corresponds to $\mathcal{F}$ programs for the set $\mathcal{F}$ that only contains the *NAND* function, but we can also $\{XOR, 0, 1\}$ programs, or use any other set. We can also define $\mathcal{F}$ *circuits*, which will be directed graphs in which the *gates* corresponds to applying a function $f_i \in \mathcal{F}$, and will each have k_i incoming wires and a single outgoing wire.[11] As in Theorem 3.4, we can show that $\mathcal{F}$ circuits and $\mathcal{F}$ programs are equivalent. We have seen that for $\mathcal{F} = \{AND, OR, NOT\}$, the resulting circuits/programs are equivalent in power to the NAND-CIRC programming language, as we can compute *NAND* using *AND*/*OR*/*NOT* and vice versa.

This turns out to be a special case of a general phenomena— the *universality* of *NAND* and other gate sets — that we will explore more in depth later in this book. For example, the following set is *universal*:

[11] There is a minor technical complication when using gates corresponding to *non symmetric* functions. A function $f : \{0,1\}^k \rightarrow \{0,1\}$ is *symmetric* if reordering its inputs does not make a difference to the output. For example, the functions *NAND*, *AND*, *OR* are symmetric. If we consider circuits with gates that are non-symmetric functions, then we need to label each wire entering a gate as to which parameter of the function it correspond to.

$\mathcal{F} = \{IF, ZERO, ONE\}$ where $ZERO : \{0,1\} \to \{0,1\}$ and $ONE : \{0,1\} \to \{0,1\}$ are the constant zero and one functions,[12] and $IF : \{0,1\}^3 \to \{0,1\}$ is the function that on input (a,b,c) outputs b if $a = 1$ and c otherwise. There are also some sets $\mathcal{F}$ that are more restricted in power, for example it can be shown that if we use only AND or OR gates (without NOT) then we do *not* get an equivalent model of comutation. **??** (which we highly recommend) covers several examples of universal and non-universal gate sets.

[12] One can also define these functions as taking a length zero input. This makes no difference for the computational power of the model.



Lecture Recap

- An *algorithm* is a recipe for performing a computation as a sequence of "elementary" or "simple" operations.
- One candidate definition for "elementary" operations is the set *AND*, *OR* and *NOT*.
- Another candidate definition for an "elementary" operation is the *NAND* operation. It is an operation that is easily implementable in the physical world in a variety of methods including by electronic transistors.
- We can use *NAND* to compute many other functions, including majority, increment, and others.
- There are other equivalent choices, including the set $\{AND, OR, NOT\}$.
- We can formally define the notion of a function $F : \{0,1\}^n \to \{0,1\}^m$ being computable using the *NAND-CIRC Programming language*.
- For every set of basic operations, the notions of being computable by a circuit and being computable by a straight-line program are equivalent.

3.8 EXERCISES

Exercise 3.1 — OR,NOT is universal. Prove that the set $\{OR, NOT\}$ is *universal*, in the sense that one can compute NAND from it.

■

Exercise 3.2 — AND,OR is not universal. Prove that for every n-bit input circuit C that contains only AND, and OR gates, as well as gates that compute the constant functions 0 and 1, C is *monotone*, in the sense that if $x, x' \in \{0,1\}^n$, $x_i \leq x'_i$ for every $i \in [n]$, then $C(x) \leq C(x')$.

Conclude that the set $\{AND, OR, 0, 1\}$ is *not* universal.

■

Exercise 3.3 — XOR is not universal. Prove that for every n-bit input circuit C that contains only XOR, gates, as well as gates that compute the constant functions 0 and 1, C is *affine or linear modulo two*, in the sense

that there exists some $a \in \{0,1\}^n$ and $b \in \{0,1\}$ such that for every $x \in \{0,1\}^n$, $C(x) = \sum_{i=0}^{n-1} a_i x_i + b \mod 2$.

Conclude that the set $\{XOR, 0, 1\}$ is *not* universal.

■

Exercise 3.4 — MAJ,NOT is universal. Prove that $\{MAJ, NOT\}$ is a universal set of gates.

■

Exercise 3.5 — NOR is universal. Let $NOR : \{0,1\}^2 \to \{0,1\}$ defined as $NOR(a,b) = NOT(OR(a,b))$. Prove that $\{NOR\}$ is a universal set of gates.

■

Exercise 3.6 — Lookup is universal. Prove that $\{LOOKUP_1, 0, 1\}$ is a universal set of gates where 0 and 1 are the constant functions $LOOKUP_1 : \{0,1\}^3 \to \{0,1\}$ satisfies $LOOKUP_1(a,b,c)$ equals a if $c = 0$ and equals b if $c = 1$.

■

Exercise 3.7 — Bound on universal basis size (challenge). Prove that for every subset B of the functions from $\{0,1\}^k$ to $\{0,1\}$, if B is universal then there is a B-circuit of at most $O(k)$ gates to compute the *NAND* function (you can start by showing that there is a B circuit of at most $O(k^{16})$ gates).[13]

[13] Thanks to Alec Sun for solving this problem.

■

Exercise 3.8 — Threshold using NANDs. Prove that there is some constant c such that for every $n > 1$, and integers $a_0, \ldots, a_{n-1}, b \in \{-2^n, -2^n + 1, \ldots, -1, 0, +1, \ldots, 2^n\}$, there is a NAND circuit with at most cn^4 gates that computes the *threshold* function $f_{a_0,\ldots,a_{n-1},b} : \{0,1\}^n \to \{0,1\}$ that on input $x \in \{0,1\}^n$ outputs 1 if and only if $\sum_{i=0}^{n-1} a_i x_i > b$.

■

Exercise 3.9 — Majority with NANDs efficiently. Prove that there is some constant c such that for every $n > 1$, there is a NAND circuit of at most $c \cdot n$ gates that computes the function $MAJ_n : \{0,1\}^n \to \{0,1\}$ is the majority function on n input bits. That is $MAJ_n(x) = 1$ iff $\sum_{i=0}^{n-1} x_i > n/2$.[14]

[14] *Hint:* One approach to solve this is using recursion and analyzing it using the so called "Master Theorem".

■

3.9 BIOGRAPHICAL NOTES

Charles Babbage (1791-1871) was a visionary scientist, mathematician, and inventor (see [Swa02; CM00]). More than a century before the invention of modern electronic computers, Babbage realized that computation can be in principle mechanized. His first design for a mechanical computer was the *difference engine* that was designed to do

polynomial interpolation. He then designed the *analytical engine* which was a much more general machine and the first prototype for a programmable general purpose computer. Unfortunately, Babbage was never able to complete the design of his prototypes. One of the earliest people to realize the engine's potential and far reaching implications was Ada Lovelace (see the notes to Chapter 6).

Boolean algebra was first investigated by Boole and DeMorgan in the 1840's [Boo47; De 47] but the definition of Boolean circuits and connection to electrical relay circuits was given in Shannon's Masters Thesis [Sha38]. (Howard Gardener called Shannon's thesis "possibly the most important, and also the most famous, master's thesis of the [20th] century".) Savage's book [Sav98], like this one, introduces the theory of computation starting with Boolean circuits as the first model. Jukna's book [Juk12] contains a modern exposition of Boolean circuits. See also Wegener's book [Weg87].

The NAND function was shown to be universal by Sheffer [She13] (though apparently this was shown even earlier by Peirce, see [**Peirce1976** ; **Burks1978charles**]). Whitehead and Russell used NAND as the basis for their logic in their magnum opus *Principia Mathematica* [WR12]. In her Ph.D thesis, Ernst [Ern09] investigates empirically the minimal NAND circuits for various functions. Nissan and Shocken's book [NS05] builds a computing system starting from NAND gates and ending with high level programs games ("NAND to Tetris"); see also the website nandtotetris.org.

Learning Objectives:

- Get comfort with syntactic sugar or automatic translation of higher level logic to low level gates.
- Learn proof of major result: every finite function can be computed by a Boolean circuit.
- Start thinking *quantitatively* about number of lines required for computation.

4

Syntactic sugar, and computing every function

> *"[In 1951] I had a running compiler and nobody would touch it because, they carefully told me, computers could only do arithmetic; they could not do programs."*, Grace Murray Hopper, 1986.

> *"Syntactic sugar causes cancer of the semicolon."*, Alan Perlis, 1982.

The NAND-CIRC programing language is pretty much as "bare bones" as programming languages come. After all, it only has a single operation. But, it turns out we can implement some "added features" on top of it. That is, we can show how we can implement those features using the underlying mechanisms of the language.

For example, some calculations can more convenient to express using AND, OR and NOT than with NAND, but we can always translate code such as

```
foo = OR(bar,blah)
```

into the valid NAND-CIRC code:

```
notbar  = NAND(bar,bar)
notblah = NAND(blah,blah)
foo     = NAND(notbar,notblah)
```

Thus in describing NAND-CIRC programs we can (and will) allow ourselves to use operations such as OR, with the understanding that in actual programs we will replace every line of the first form with the three lines of the second form. In programming language parlance this is known as *"syntactic sugar"*, since we are not changing the definition of the language, but merely introducing some convenient notational shortcuts.[1] We will use several such "syntactic sugar" constructs to make our descriptions of NAND-CIRC programs shorter and simpler. However, these descriptions are merely shorthand for

[1] This concept is also known as "macros" or "meta-programming" and is sometimes implemented via a preprocessor or macro language in a programming language or a text editor. One modern example is the Babel JavaScript syntax transformer, that converts JavaScript programs written using the latest features into a format that older Browsers can accept. It even has a plug-in architecture, that allows users to add their own syntactic sugar to the language.

the equivalent standard or "sugar free" NAND-CIRC program that is obtained after removing the use of all these constructs. In particular, when we say that a function f has an s-line NAND-CIRC program, we mean a *standard* NAND-CIRC program, that does not use any syntactic sugar.

4.1 SOME EXAMPLES SYNTACTIC SUGAR

Here are some examples of "syntactic sugar" that we can use in constructing NAND-CIRC programs. This is not an exhaustive list - if you find yourself needing to use an extra feature in your NAND-CIRC program then you can just show how to implement it based on the existing ones. Going over examples for syntatic sugar can be a little tedious, but we do it for two reasons:

1. To convince you that despite its seeming simplicity and limitations, the NAND-CIRC programming language is actually quite powerful and can capture many of the fancy programming constructs such as `if` statements and function definitions that exists in more fashionable languages.
2. So you can realize how lucky you are to be taking a theory of computation course and not a compilers course... `:)`

4.1.1 Constants

We can create variables `zero` and `one` that have the values 0 and 1 respectively by adding the lines

```
temp = NAND(X[0],X[0])
one  = NAND(temp,X[0])
zero = NAND(one,one)
```

Note that since for every $x \in \{0,1\}$, $NAND(x,\overline{x}) = 1$, the variable `one` will get the value 1 regardless of the value of x_0, and the variable `zero` will get the value $NAND(1,1) = 0$.

4.1.2 Functions / Macros

Another staple of almost any programming language is the ability to execute *functions*. However, we can achieve the same effect as (non recursive) functions using the time honored technique of "copy and paste". That is, we can replace code which defines a macro

```
def Func(a,b):
    function_code
    return c
some_code
f = Func(e,d)
some_more_code
```

with the following code where we "paste" the code of `Func`

```
some_code
function_code'
some_more_code
```

and where `function_code'` is obtained by replacing all occurrences of `a` with `d`,`b` with `e`, `c` with `f`. When doing that we will need to ensure that all other variables appearing in `function_code'` don't interfere with other variables. We can always do so by renaming variables to new names that were not used before.

4.1.3 Example: Computing Majority via NAND's

Function definitions allow us to express NAND-CIRC programs much more cleanly and succinctly. For example, because we can compute AND,OR, NOT using NANDs, we can compute the *Majority* function as well.

```
def NOT(a): return NAND(a,a)
def AND(a,b): return NOT(NAND(a,b))
def OR(a,b): return NAND(NOT(a),NOT(b))

def MAJ(a,b,c):
    return OR(OR(AND(a,b),AND(b,c)),AND(a,c))

print(MAJ(0,1,1))
# 1
```

This is certainly much more pleasant than the full NAND alternative:

```
Temp[0] = NAND(X[0],X[1])
Temp[1] = NAND(Temp[0],Temp[0])
Temp[2] = NAND(X[1],X[2])
Temp[3] = NAND(Temp[2],Temp[2])
Temp[4] = NAND(Temp[1],Temp[1])
Temp[5] = NAND(Temp[3],Temp[3])
Temp[6] = NAND(Temp[4],Temp[5])
Temp[7] = NAND(X[0],X[2])
Temp[8] = NAND(Temp[7],Temp[7])
Temp[9] = NAND(Temp[6],Temp[6])
Temp[10] = NAND(Temp[8],Temp[8])
Y[0] = NAND(Temp[9],Temp[10])
```

4.1.4 Conditional statements

Another sorely missing feature in NAND is a conditional statement such as the `if/then` constructs that are found in many programming

languages. However, using functions, we can obtain an ersatz if/then construct. First we can compute the function $IF : \{0,1\}^3 \rightarrow \{0,1\}$ such that $IF(a,b,c)$ equals b if $a = 1$ and c if $a = 0$.



Before reading onwards, try to see how you could compute the *IF* function using *NAND*'s. Once you you do that, see how you can use that to emulate `if/then` types of constructs.

The *IF* function can be implemented from NANDs as follows (see Exercise 4.2):

```
def IF(cond,a,b):
    notcond = NAND(cond,cond)
    temp = NAND(b,notcond)
    temp1 = NAND(a,cond)
    return NAND(temp,temp1)


print(IF(0,1,0))
# 0
print(IF(1,1,0))
# 1
```

The *IF* function is also known as the *multiplexing* function, since $cond$ can be thought of as a switch that controls whether the output is connected to a or b.

Using the *IF* function, we can implement conditionals in NAND. The idea is that we replace code of the form

```
if (condition):  assign blah to variable foo
```

with code of the form

```
foo   = IF(condition,blah, foo)
```

that assigns to `foo` its old value when `condition` equals 0, and assign to `foo` the value of `blah` otherwise. More generally we can replace code of the form

```
if (cond):
    a = ...
    b = ...
    c = ...
```

with code of the form

```
temp_a = ...
temp_b = ...
temp_c = ...
a = IF(cond,temp_a,a)
b = IF(cond,temp_b,b)
c = IF(cond,temp_c,c)
```

4.2 EXTENDED EXAMPLE: ADDITION AND MULTIPLICATION (OPTIONAL)

Using "syntactic sugar", we can write the integer addition function as follows:[2]

[2] We use here least-significant-digit first convention for simplicity of notation.

```
# Add two n-bit integers
def ADD(A,B):
    Result = [0]*(n+1)
    Carry  = [0]*(n+1)
    Carry[0] = zero(A[0])
    for i in range(n):
        Result[i] = XOR(Carry[i],XOR(A[i],B[i]))
        Carry[i+1] = MAJ(Carry[i],A[i],B[i])
    Result[n] = Carry[n]
    return Result

ADD([1,1,1,0,0],[1,0,0,0,0]);;
# [0, 0, 0, 1, 0, 0]
```

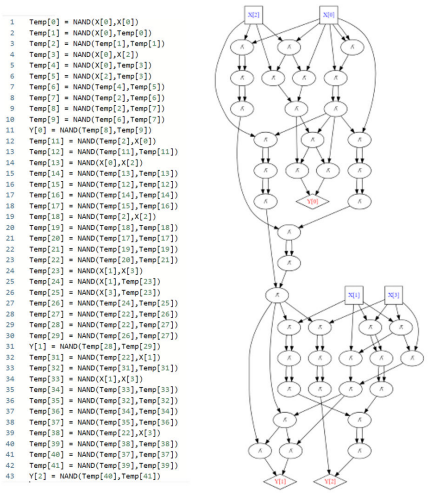
where `zero` is the constant zero function, and `MAJ` and `XOR` correspond to the majority and XOR functions respectively.

In the above we used the *loop* `for i in range(n)` but we can expand this out by simply repeating the code n times, replacing the value of `i` with $0, 1, 2, \ldots, n-1$. The crucial point is that (unlike most programming languages) we do not allow the number of times the loop is executed to depend on the input, and so it is always possible to "expand out" the loop by simply copying the code the requisite number of times.

By expanding out all the features, for every value of n we can translate the above program into a standard ("sugar free") NAND-CIRC program. Fig. 4.1 depicts what we get for $n = 2$.

By going through the above program carefully and accounting for the number of gates, we can see that it yields a proof of the following theorem (see also Fig. 4.2):

Theorem 4.1 — Addition using NAND-CIRC programs. For every $n \in \mathbb{N}$, let $ADD_n : \{0,1\}^{2n} \to \{0,1\}^{n+1}$ be the function that, given



```
Temp[0] = NAND(X[0],X[0])
Temp[1] = NAND(X[0],Temp[0])
Temp[2] = NAND(Temp[1],Temp[1])
Temp[3] = NAND(X[0],X[2])
Temp[4] = NAND(X[0],Temp[3])
Temp[5] = NAND(X[2],Temp[3])
Temp[6] = NAND(Temp[4],Temp[5])
Temp[7] = NAND(Temp[2],Temp[6])
Temp[8] = NAND(Temp[2],Temp[7])
Temp[9] = NAND(Temp[6],Temp[7])
Y[0] = NAND(Temp[8],Temp[9])
Temp[11] = NAND(Temp[2],X[0])
Temp[12] = NAND(Temp[11],Temp[11])
Temp[13] = NAND(X[0],X[2])
Temp[14] = NAND(Temp[13],Temp[13])
Temp[15] = NAND(Temp[13],Temp[12])
Temp[16] = NAND(Temp[14],Temp[14])
Temp[17] = NAND(Temp[15],Temp[16])
Temp[18] = NAND(Temp[2],X[2])
Temp[19] = NAND(Temp[18],Temp[18])
Temp[20] = NAND(Temp[17],Temp[17])
Temp[21] = NAND(Temp[19],Temp[19])
Temp[22] = NAND(Temp[20],Temp[21])
Temp[23] = NAND(X[1],X[3])
Temp[24] = NAND(X[1],Temp[23])
Temp[25] = NAND(X[3],Temp[23])
Temp[26] = NAND(Temp[24],Temp[25])
Temp[27] = NAND(Temp[22],Temp[26])
Temp[28] = NAND(Temp[22],Temp[27])
Temp[29] = NAND(Temp[26],Temp[27])
Y[1] = NAND(Temp[28],Temp[29])
Temp[31] = NAND(Temp[22],X[1])
Temp[32] = NAND(Temp[31],Temp[31])
Temp[33] = NAND(X[1],X[3])
Temp[34] = NAND(Temp[33],Temp[33])
Temp[35] = NAND(Temp[32],Temp[32])
Temp[36] = NAND(Temp[34],Temp[34])
Temp[37] = NAND(Temp[35],Temp[36])
Temp[38] = NAND(Temp[22],X[3])
Temp[39] = NAND(Temp[38],Temp[38])
Temp[40] = NAND(Temp[37],Temp[37])
Temp[41] = NAND(Temp[39],Temp[39])
Y[2] = NAND(Temp[40],Temp[41])
```

Figure 4.1: The NAND-CIRC program and corresponding NAND circuit for adding two-digit binary numbers that are obtained by "expanding out" all the syntactic sugar. The program/circuit has 43 lines/-gates which is by no means necessary. It is possible to add n bit numbers using $9n$ NAND gates, see Exercise 4.5.

$x, x' \in \{0,1\}^n$ computes the representation of the sum of the numbers that x and x' represent. Then there is a constant $c \leq 30$ such that for every n there is a NAND-CIRC program of at most c lines computing ADD_n. [3]

Once we have addition, we can use the grade-school algorithm to obtain multiplication as well, thus obtaining the following theorem:

Theorem 4.2 — Multiplication using NAND-CIRC programs. For every n, let $MULT_n : \{0,1\}^{2n} \to \{0,1\}^{2n}$ be the function that, given $x, x' \in \{0,1\}^n$ computes the representation of the product of the numbers that x and x' represent. Then there is a constant c such that for every n, there is a NAND-CIRC program of at most cn^2 that computes the function $MULT_n$.

We omit the proof, though in Exercise 4.7 we ask you to supply a "constructive proof" in the form of a program (in your favorite programming language) that on input a number n, outputs the code of a NAND-CIRC program of at most $1000n^2$ lines that computes the $MULT_n$ function. In fact, we can use Karatsuba's algorithm to show that there is a NAND-CIRC program of $O(n^{\log_2 3})$ lines to compute $MULT_n$ (and one can even get further asymptotic improvements using the newer algorithms).

[3] The value of c can be improved to 9, see Exercise 4.5.

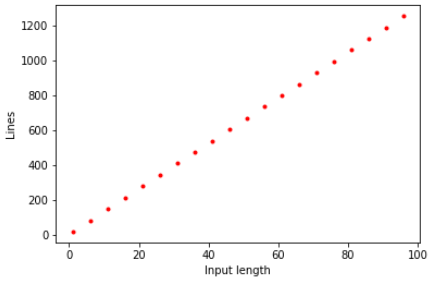


Figure 4.2: The number of lines in our NAND-CIRC program to add two n bit numbers, as a function of n, for n's between 1 and 100. This is not the most efficient program for this task, but the important point is that it has the form $O(n)$.

4.3 THE LOOKUP FUNCTION

We have seen that NAND-CIRC programs can add and multiply numbers. But can they compute other type of functions, that have nothing to do with arithmetic? Here is one example:

Definition 4.3 — Lookup function. For every k, the *lookup* function $LOOKUP_k : \{0,1\}^{2^k+k} \to \{0,1\}$ is defined as follows: For every $x \in \{0,1\}^{2^k}$ and $i \in \{0,1\}^k$,

$$LOOKUP_k(x,i) = x_i \tag{4.1}$$

where x_i denotes the ith entry of x, using the binary representation to identify i with a number in $\{0,\ldots,2^k-1\}$.

The function $LOOKUP_1 : \{0,1\}^3 \to \{0,1\}$ maps $(x_0,x_1,i) \in \{0,1\}^3$ to x_i. It is actually the same as the *IF*/*MUX* function we have seen above, that has a 4 line NAND-CIRC program. However, can we compute higher levels of *LOOKUP*? This turns out to be the case:

Theorem 4.4 — Lookup function. For every k, there is a NAND-CIRC program that computes the function $LOOKUP_k : \{0,1\}^{2^k+k} \to \{0,1\}$. Moreover, the number of lines in this program is at most $4 \cdot 2^k$.

As a corollary, for every $k > 0$, $LOOKUP_k$ can be computed by a Boolean circuit (with AND, OR and NOT gates) of at most $8 \cdot 2^k$.

4.3.1 Constructing a NAND-CIRC program for *LOOKUP*

We now prove Theorem 4.4. The idea is actually quite simple. Consider the function $LOOKUP_3 : \{0,1\}^{2^3+3} \to \{0,1\}$ that takes a an input of $8+3=11$ bits and output a single bit. We can write this function in pseudocode as follows:

```
def
 ↪ LOOKUP_3(X[0],X[1],X[2],X[3],X[4],X[5],X[6],X[7],i[0],i[1],i[8]):
    if i == (0,0,0): return X[0]
    if i == (0,0,1): return X[1]
    if i == (0,1,0): return X[2]
    ...
    if i == (1,1,1): return X[7]
```

A condition such as `i==(0,1,0)` can be expanded out to the AND of `NOT(i[0])`, `i[1]` and `NOT(i[2])` and each one of these `AND` and `NOT` gates can be then translated into `NAND`. The above can yield a proof of a version of Theorem 4.4 with a slightly larger number of gates, but if we are a little more careful we can prove the theorem with the number of gates as stated.

Specifically, we will prove Theorem 4.4 by induction. We will do so by induction. That is, we show how to use a NAND-CIRC program for computing $LOOKUP_k$ to compute $LOOKUP_{k+1}$. For the case $k =$

1, $LOOKUP_1$ is the same as `IF` for which we given a NAND-CIRC program with four line.

Now let us consider the case of $k = 2$. Given input $x = (x_0, x_1, x_2, x_3)$ for $LOOKUP_2$ and an index $i = (i_0, i_1)$, if the most significant bit i_0 of the index is 0 then $LOOKUP_2(x, i)$ will equal x_0 if $i_1 = 0$ and equal x_1 if $i_1 = 1$. Similarly, if the most significant bit i_0 is 1 then $LOOKUP_2(x, i)$ will equal x_2 if $i_1 = 0$ and will equal x_3 if $i_1 = 1$. Another way to say this is that we can write $LOOKUP_2$ as follows:

```
def LOOKUP2(X[0],X[1],X[2],X[3],i[0],i[1]):
    if i[0]==1:
        return LOOKUP1(X[2],X[3],i[1])
    else:
        return LOOKUP1(X[0],X[1],i[1])
```

or in other words,

```
def LOOKUP2(X[0],X[1],X[2],X[3],i[0],i[1]):
    a = LOOKUP1(X[2],X[3],i[1])
    b = LOOKUP1(X[0],X[1],i[1])
    return IF( i[0],a,b)
```

Similarly, we can write

```
def
 ↪ LOOKUP3(X[0],X[1],X[2],X[3],X[4],X[5],X[6],X[7],i[0],i[1],i[2]):
    a = LOOKUP2(X[3],X[4],X[5],X[6],i[1],i[2])
    a = LOOKUP2(X[0],X[1],X[2],X[3],i[1],i[2])
    return IF( i[0],a,b)
```

and so on and so forth. Generally, we can compute $LOOKUP_k$ using two invocations of $LOOKUP_{k-1}$ and one invocation of *IF*, which yields the following lemma:

Lemma 4.5 — Lookup recursion. For every $k \geq 2$, $LOOKUP_k(x_0, \ldots, x_{2^k-1}, i_0, \ldots, i_{k-1})$ is equal to

$$IF(i_0, LOOKUP_{k-1}(x_0, \ldots, x_{2^{k-1}-1}, i_1, \ldots, i_{k-1}), LOOKUP_{k-1}(x_{2^{k-1}}, \ldots, x_{2^k-1}, i_1, \ldots, i_{k-1})) \tag{4.2}$$

Proof. If the most significant bit i_0 of i is zero, then the index i is in $\{0, \ldots, 2^{k-1} - 1\}$ and hence we can perform the lookup on the "first half" of x and the result of $LOOKUP_k(x, i)$ will be the same as $a = LOOKUP_{k-1}(x_0, \ldots, x_{2^{k-1}-1}, i_1, \ldots, i_{k-1})$. On the other hand, if this most significant bit i_0 is equal to 1, then the index is in $\{2^{k-1}, \ldots, 2^k - 1\}$, in which case the result of $LOOKUP_k(x, i)$ is the same as $b = LOOKUP_{k-1}(x_{2^{k-1}}, \ldots, x_{2^k-1}, i_1, \ldots, i_{k-1})$. Thus we can compute

$LOOKUP_k(x, i)$ by first computing a and b and then outputting $IF(i_{k-1}, a, b)$.

■

Lemma 4.5 directly implies Theorem 4.4. We prove by induction on k that there is a NAND-CIRC program of at most $4 \cdot 2^k$ lines for $LOOKUP_k$. For $k = 1$ this follows by the four line program for *IF* we've seen before. For $k > 1$, we use the following pseudocode

```
a = LOOKUP_(k-1)(X[0],...,X[2^(k-1)-1],i[1],...,i[k-1])
b = LOOKUP_(k-1)(X[2^(k-1)],...,Z[2^(k-1)],i[1],...,i[k-
 ↪  1])
return  IF(i[k-1],a,b)
```

If we let $L(k)$ be the number of lines required for $LOOKUP_k$, then the above shows that

$$L(k) \leq 2L(k-1) + 4 \,. \tag{4.3}$$

which solves for $L(k) \leq 4(2^k - 1)$. (See Fig. 4.3 for a plot of the actual number of lines in our implementation of $LOOKUP_k$.)

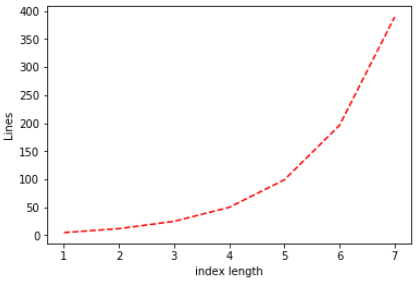


Figure 4.3: The number of lines in our implementation of the LOOKUP_k function as a function of k (i.e., the length of the index). The number of lines in our implementation is roughly $3 \cdot 2^k$.

4.4 COMPUTING *EVERY* FUNCTION

At this point we know the following facts about NAND-CIRC programs:

1. They can compute at least some non trivial functions.
2. Coming up with NAND-CIRC programs for various functions is a very tedious task.

Thus I would not blame the reader if they were not particularly looking forward to a long sequence of examples of functions that can be computed by NAND-CIRC programs. However, it turns out we are not going to need this, as we can show in one fell swoop that NAND-CIRC programs can compute *every* finite function:

Theorem 4.6 — Universality of NAND. There exists some constant $c > 0$ such that for every $n, m > 0$ and function $f : \{0,1\}^n \to \{0,1\}^m$, there is a NAND circuit with at most $c \cdot m2^n$ gates that computes the function f.

Note that up to constants, the models of NAND circuits, NAND-CIRC programs, AON-CIRC programs, and Boolean circuits, are all equivalent to one another, and hence **??** holds for all these models.

R

Remark 4.7 — Improved bound. As we'll see in the proof, the constant c will be smaller than 10. In fact, with a tighter proof, we can even shave an extra factor of n, as well as optimize the constant, to obtain the following stronger result:

Lemma 4.8 For every $\epsilon > 0$, $m \in \mathbb{N}$ and sufficiently large n, if $f : \{0,1\}^n \to \{0,1\}^m$ then f can be computed by a NAND circuit of at most

$$(1+\epsilon)\frac{m \cdot 2^n}{n} \tag{4.4}$$

gates.

We will not prove Lemma 4.8 in this book, but discuss how to obtain a bound of the form $O(\frac{m \cdot 2^n}{n})$ in Section 4.4.2. See also the biographical notes.

Big Idea 4 *Every* finite function can be computed by a large enough Boolean circuit.

4.4.1 Proof of NAND's Universality

To prove Theorem 4.6, we need to give a NAND circuit, or equivalently a NAND-CIRC program, for *every* possible function. We will restrict our attention to the case of Boolean functions (i.e., $m = 1$). In Exercise 4.9 you will show how to extend the proof for all values of m. A function $F : \{0,1\}^n \to \{0,1\}$ can be specified by a table of its values for each one of the 2^n inputs. For example, the table below describes one particular function $G : \{0,1\}^4 \to \{0,1\}$:[4]

Table 4.1: An example of a function $G : \{0,1\}^4 \to \{0,1\}$. {#table-functiong}

Input (x)	Output ($G(x)$)
0000	1
1000	1
0100	0
1100	0
0010	1
1010	0
0110	0
1110	1
0001	0
1001	0
0101	0

[4] In case you are curious, this is the function that computes the digits of π in the binary basis. Note that as per the convention of this course, if we think of strings as numbers then we right them with the least significant digit first.

Input (x)	Output ($G(x)$)
1101	0
0011	1
1011	1
0111	1
1111	1

We can see that for every $x \in \{0,1\}^4$, $G(x) = LOOKUP_4(1100100100001111, x)$. Therefore the following is NAND "pseudocode" to compute G:

```
G0000 = 1
G1000 = 1
G0100 = 0
...
G0111 = 1
G1111 = 1
Y[0] = LOOKUP_4(G0000,G1000,...,G1111,
                X[0],X[1],X[2],X[3])
```

We can translate this pseudocode into an actual NAND-CIRC program by adding three lines to define variables `zero` and `one` that are initialized to 0 and 1 repsectively, and then replacing a statement such as `Gxxx = 0` with `Gxxx = NAND(one,one)` and a statement such as `Gxxx = 1` with `Gxxx = NAND(zero,zero)`. The call to `LOOKUP_4` will be replaced by the NAND-CIRC program that computes $LOOKUP_4$, plugging in the appropriate inputs.

There was nothing about the above reasoning that was particular to the function G of **??**. Given *every* function $F : \{0,1\}^n \to \{0,1\}$, we can write a NAND-CIRC program that does the following:

1. Initialize 2^n variables of the form `F00...0` till `F11...1` so that for every $z \in \{0,1\}^n$, the variable corresponding to z is assigned the value $F(z)$.

2. Compute $LOOKUP_n$ on the 2^n variables initialized in the previous step, with the index variable being the input variables `X[0]`,...,`X[`$2^n - 1$`]`. That is, just like in the pseudocode for `G` above, we use `Y[0] = LOOKUP(F00..00,...,F11..1,X[0],..,x[`$n-1$`])`

The total number of lines in the program will be 2^n plus the $4 \cdot 2^n$ lines that we pay for computing $LOOKUP_n$. This completes the proof of Theorem 4.6.

R

Remark 4.9 — Result in perspective. While Theorem 4.6 seems striking at first, in retrospect, it is perhaps not that surprising that every finite function can be computed with a NAND-CIRC program. After all, a finite function $F : \{0,1\}^n \to \{0,1\}^m$ can be represented by simply the list of its outputs for each one of the 2^n input values. So it makes sense that we could write a NAND-CIRC program of similar size to compute it. What is more interesting is that *some* functions, such as addition and multiplication, have a much more efficient representation: one that only requires $O(n^2)$ or even smaller number of lines.

4.4.2 Improving by a factor of n (optional)

As discussed in Remark 4.7, by being a little more careful, we can improve the bound of Theorem 4.6 and show that every function $F : \{0,1\}^n \to \{0,1\}^m$ can be computed by a NAND-CIRC program of at most $O(m2^n/n)$ lines. As before, it is enough to prove the case that $m = 1$.

The idea is to use the technique known as *memoization*. Let $k = \log(n - 2\log n)$ (the reasoning behind this choice will become clear later on). For every $a \in \{0,1\}^{n-k}$ we define $F_a : \{0,1\}^k \to \{0,1\}$ to be the function that maps $w_0, \ldots, w_{k-1}$ to $F(a_0, \ldots, a_{n-k-1}, w_0, \ldots, w_{k-1})$.

On input $x = x_0, \ldots, x_{n-1}$, we can compute $F(x)$ as follows: First we compute a 2^{n-k} long string P whose a^{th} entry (identifying $\{0,1\}^{n-k}$ with $[2^{n-k}]$) equals $F_a(x_{n-k}, \ldots, x_{n-1})$. One can verify that $F(x) = \mathit{LOOKUP}_{n-k}(P, x_0, \ldots, x_{n-k-1})$. Since we can compute LOOKUP_{n-k} using $O(2^{n-k})$ lines, if we can compute the string P (i.e., compute variables `P_0`, ..., `P_`$2^{n-k}-1$) using T lines, then we can compute F in $O(2^{n-k}) + T$ lines.

The trivial way to compute the string P would be to use $O(2^k)$ lines to compute for every a the map $x_0, \ldots, x_{k-1} \mapsto F_a(x_0, \ldots, x_{k-1})$ as in the proof of Theorem 4.6. Since there are 2^{n-k} a's, that would be a total cost of $O(2^{n-k} \cdot 2^k) = O(2^n)$ which would not improve at all on the bound of Theorem 4.6. However, a more careful observation shows that we are making some *redundant* computations. After all, there are only 2^{2^k} distinct functions mapping k bits to one bit. If a and a' satisfy that $F_a = F_{a'}$ then we don't need to spend 2^k lines computing both $F_a(x)$ and $F_{a'}(x)$ but rather can only compute the variable `P_`a and then copy `P_`a to `P_`a' using $O(1)$ lines. Since we have 2^{2^k} unique functions, we can bound the total cost to compute P by $O(2^{2^k}2^k) + O(2^{n-k})$.

Now it just becomes a matter of calculation. By our choice of k, $2^k = n - 2\log n$ and hence $2^{2^k} = \frac{2^n}{n^2}$. Since $n/2 \leq 2^k \leq n$, we can bound the total cost of computing $F(x)$ (including also the additional

$O(2^{n-k})$ cost of computing $LOOKUP_{n-k}$) by $O(\frac{2^n}{n^2} \cdot n) + O(2^n/n)$, which is what we wanted to prove.

4.5 THE CLASS $SIZE_{n,m}(T)$

We now make a fundamental definition, we let $SIZE_{n,m}(s)$ denote the set of all functions from $\{0,1\}^n$ to $\{0,1\}^m$ that can be computed by NAND circuits of at most s gates (or equivalently, by NAND-CIRC programs of at most s lines):

Definition 4.10 — Size class. Let $n, m, s \in \mathbb{N}$ be numbers with $s \geq m$. The set $SIZE_{n,m}(s)$ denotes the set of all functions $f : \{0,1\}^n \to \{0,1\}^m$ such that there exists a NAND circuit of at most s gates that computes f. We denote by $SIZE_n(s)$ the set $SIZE_{n,1}(s)$.

Fig. 4.4 depicts the sets $SIZE_{n,1}(s)$, note that $SIZE_{n,m}(s)$ is a set of *functions*, not of *programs!* (asking if a program or a circuit is a member of $SIZE_{n,m}(s)$ is a *category error* as in the sense of Fig. 4.5).

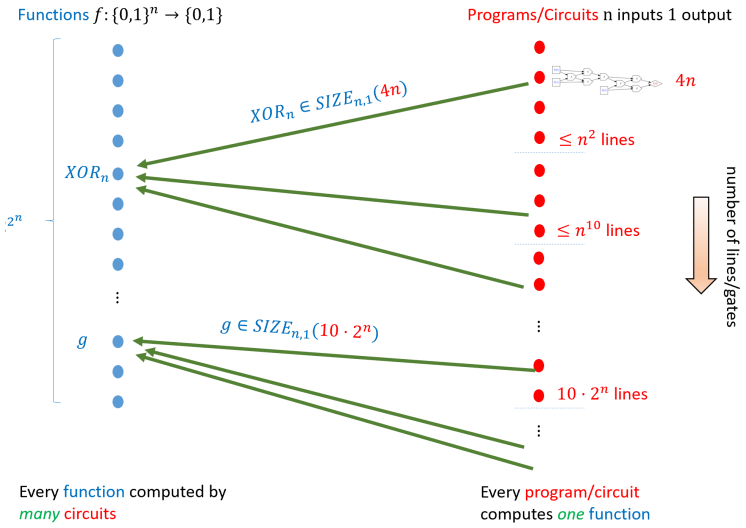


Figure 4.4: There are 2^{2^n} functions mapping $\{0,1\}^n$ to $\{0,1\}$, and an infinite number of circuits with n bit inputs and a single bit of output. Every circuit computes one function, but every function can be computed by many circuits. We say that $f \in SIZE_{n,1}(s)$ if the smallest circuit that computes f has s or fewer gates. For example $XOR_n \in SIZE_{n,1}(4n)$. Theorem 4.6 shows that *every* function g is computable by some circuit of at most $c \cdot 2^n/n$ gates, and hence $SIZE_{n,1}(c \cdot 2^n/n)$ corresponds to the set of *all* functions from $\{0,1\}^n$ to $\{0,1\}$.

While we defined $SIZE_{n,m}(s)$ with respect to NAND gates, we would get essentially the same class if we defined it with respect to AND/OR/NOT gates:

Lemma 4.11 Let $SIZE^{AON}_{n,m,s}$ denote the set of all functions $f : \{0,1\}^n \to \{0,1\}^m$ that can be computed by an AND/OR/NOT Boolean circuit of at most s gates. Then,

$$SIZE_{n,m}(s/2) \subseteq SIZE^{AON}_{n,m}(s) \subseteq SIZE_{n,m}(3s) \tag{4.5}$$

Proof. If f can be computed by a NAND circuit of at most $s/2$ gates, then by replacing each NAND with the two gates NOT and AND, we

can obtain an AND/OR/NOT Boolean circuit of at most s gates that computes f. On the other hand, if f can be computed by a Boolean AND/OR/NOT circuit of at most s gates, then by Theorem 3.8 it can be computed by a NAND circuit of at most $3s$ gates.

■

The results we've seen before can be phrased as showing that $ADD_n \in SIZE_{2n,n+1}(100n)$ and $MULT_n \in SIZE_{2n,2n}(10000n^{\log_2 3})$. Theorem 4.6 shows that $SIZE_{n,m}(4m2^n)$ is equal the set of all functions from $\{0,1\}^n$ to $\{0,1\}^m$. See Fig. 5.4.

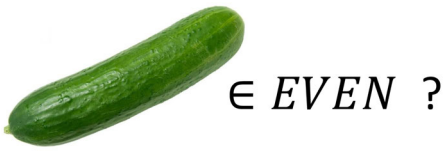


Figure 4.5: A "category error" is a question such as "is a cucumber even or odd?" which does not even make sense. In this book the category errors one needs to watch out for are confusing *functions* and *programs* (i.e., confusing *specifications* and *implementations*). If C is a circuit or program, then asking if $C \in SIZE_{n,1}(s)$ is a category error, since $SIZE_{n,1}(s)$ is a set of *functions* and not programs or circuits.

P

Note that $SIZE_{n,m}(s)$ does **not** correspond to a set of programs! Rather, it is a set of *functions* (see Fig. 4.5). This distinction between *programs* and *functions* will be crucial for us in this course. You should always remember that while a program *computes* a function, it is not *equal* to a function. In particular, as we've seen, there can be more than one program to compute the same function.

R

Remark 4.12 — Finite vs infinite functions. A NAND-CIRC program P can only compute a function with a certain number n of inputs and a certain number m of outputs. Hence for example there is no single NAND-CIRC program that can compute the increment function $INC : \{0,1\}^* \to \{0,1\}^*$ that maps a string x (which we identify with a number via the binary representation) to the string that represents $x+1$. Rather for every $n > 0$, there is a NAND-CIRC program P_n that computes the restriction INC_n of the function INC to inputs of length n. Since it can be shown that for every $n > 0$ such a program P_n exists of length at most $10n$, $INC_n \in SIZE(10n)$ for every $n > 0$.

If $T : \mathbb{N} \to \mathbb{N}$ and $F : \{0,1\}^* \to \{0,1\}^*$, we will sometimes slightly abuse notation and write $F \in SIZE(T(n))$ to indicate that for every n the restriction $F_{\restriction n}$ of F to inputs in $\{0,1\}^n$ is in $SIZE(T(n))$. Hence we can write $INC \in SIZE(10n)$. We will come back to this issue of finite vs infinite functions later in this course.

Solved Exercise 4.1 — $SIZE$ closed under complement.. In this exercise we prove a certain "closure property" of the class $SIZE_n(s)$. That is, we show that if f is in this class then (up to some small additive term) so is the complement of f, which is the function $g(x) = 1 - f(x)$.

Prove that there is a constant c such that for every $f : \{0,1\}^n \to \{0,1\}$ and $s \in \mathbb{N}$, if $f \in SIZE_n(s)$ then $1 - f \in SIZE_n(s + c)$.

Solution:

If $f \in SIZE(s)$ then there is an s-line NAND-CIRC program P that computes f. We can rename the variable `Y[0]` in P to a variable `temp` and add the line

```
Y[0] = NAND(temp,temp)
```

at the very end to obtain a program P' that computes $1 - f$.



Lecture Recap

- We can define the notion of computing a function via a simplified "programming language", where computing a function F in T steps would correspond to having a T-line NAND-CIRC program that computes F.
- While the NAND-CIRC programming only has one operation, other operations such as functions and conditional execution can be implemented using it.
- Every function $f : \{0,1\}^n \to \{0,1\}^m$ can be computed by a circuit of at most $O(m2^n)$ gates (and in fact at most $O(m2^n/n)$ gates).
- Sometimes (or maybe always?) we can translate an *efficient* algorithm to compute f into a circuit that computes f with a number of gates comparable to the number of steps in this algorithm.

4.6 EXERCISES

Exercise 4.1 — Pairing. This exercise asks you to give a one-to-one map from $\mathbb{N}^2$ to $\mathbb{N}$. This can be useful to implement two-dimensional arrays as "syntacic sugar" in programming languages that only have one-dimensional array.

1. Prove that the map $F(x,y) = 2^x 3^y$ is a one-to-one map from $\mathbb{N}^2$ to $\mathbb{N}$.

2. Show that there is a one-to-one map $F : \mathbb{N}^2 \to \mathbb{N}$ such that for every x, y, $F(x,y) \leq 100 \cdot \max\{x,y\}^2 + 100$.

3. For every k, show that there is a one-to-one map $F : \mathbb{N}^k \to \mathbb{N}$ such that for every $x_0, \ldots, x_{k-1} \in \mathbb{N}$, $F(x_0, \ldots, x_{k-1}) \leq 100 \cdot (x_0 + x_1 + \ldots + x_{k-1} + 100k)^k$.

Exercise 4.2 — Computing MUX. Prove that the NAND-CIRC program below computes the function MUX (or $LOOKUP_1$) where $MUX(a,b,c)$ equals a if $c=0$ and equals b if $c=1$:

```
t = NAND(X[2],X[2])
u = NAND(X[0],t)
v = NAND(X[1],X[2])
Y[0] = NAND(u,v)
```

Exercise 4.3 — At least two / Majority. Give a NAND-CIRC program of at most 6 lines to compute $MAJ : \{0,1\}^3 \rightarrow \{0,1\}$ where $MAJ(a,b,c) = 1$ iff $a+b+c \geq 2$.

Exercise 4.4 — Conditional statements. In this exercise we will show that even though the NAND-CIRC programming language does not have an `if .. then .. else ..` statement, we can still implement it. Suppose that there is an s-line NAND-CIRC program to compute $f : \{0,1\}^n \rightarrow \{0,1\}$ and an s'-line NAND-CIRC program to compute $f' : \{0,1\}^n \rightarrow \{0,1\}$. Prove that there is a program of at most $s + s' + 10$ lines to compute the function $g : \{0,1\}^{n+1} \rightarrow \{0,1\}$ where $g(x_0,\ldots,x_{n-1},x_n)$ equals $f(x_0,\ldots,x_{n-1})$ if $x_n = 0$ and equals $f'(x_0,\ldots,x_{n-1})$ otherwise.

Exercise 4.5 — Half and full adders.

1. A *half adder* is the function $HA : \{0,1\}^2 :\rightarrow \{0,1\}^2$ that corresponds to adding two binary bits. That is, for every $a,b \in \{0,1\}$, $HA(a,b) = (e,f)$ where $2e + f = a + b$. Prove that there is a NAND circuit of at most five NAND gates that computes HA.

2. A *full adder* is the function $FA : \{0,1\}^3 \rightarrow \{0,1\}$ that takes in two bits and a "carry" bit and outputs their sum. That is, for every $a,b,c \in \{0,1\}$, FA(a,b,c) = (e,f)$ such that $2e + f = a + b + c$. Prove that there is a NAND circuit of at most nine NAND gates that computes FA.

3. Prove that if there is a NAND circuit of c gates that computes FA, then there is a circuit of cn gates that computes ADD_n where (as in Theorem 4.1) $ADD_n : \{0,1\}^{2n} \rightarrow \{0,1\}^n$ is the function that outputs the addition of two input n-bit numbers. See footnote for hint.[5]

4. Show that for every n there is a NAND-CIRC program to compute ADD_n with at most $9n$ lines.

[5] Use a "cascade" of adding the bits one after the other, starting with the least significant digit, just like in the elementary-school algorithm.

Exercise 4.6 — Addition. Write a program using your favorite programming language that on input an integer n, outputs a NAND-CIRC program that computes ADD_n. Can you ensure that the program it outputs for ADD_n has fewer than $10n$ lines?

Exercise 4.7 — Multiplication. Write a program using your favorite programming language that on input an integer n, outputs a NAND-CIRC program that computes $MULT_n$. Can you ensure that the program it outputs for $MULT_n$ has fewer than $1000 \cdot n^2$ lines?

Exercise 4.8 — Efficient multiplication (challenge). Write a program using your favorite programming language that on input an integer n, outputs a NAND-CIRC program that computes $MULT_n$ and has at most $10000n^{1.9}$ lines.[6] What is the smallest number of lines you can use to multiply two 2048 bit numbers?

[6] **Hint:** Use Karatsuba's algorithm

Exercise 4.9 — Multibit function. Prove that

a. If there is an s-line NAND-CIRC program to compute $f : \{0,1\}^n \to \{0,1\}$ and an s'-line NAND-CIRC program to compute $f' : \{0,1\}^n \to \{0,1\}$ then there is an $s + s'$-line program to compute the function $g : \{0,1\}^n \to \{0,1\}^2$ such that $g(x) = (f(x), f'(x))$.

b. For every function $f : \{0,1\}^n \to \{0,1\}^m$, there is a NAND-CIRC program of at most $10m \cdot 2^n$ lines that computes f.

4.7 BIBLIOGRAPHICAL NOTES

See Jukna's and Wegener's books [Juk12; Weg87] for much more extensive discussion on circuits. Shannon showed that every Boolean function can be computed by a circuit of exponential size [Sha38]. The improved bound of $c \cdot 2^n/n$ (with the optimal value of c for many bases) is due to Lupanov [Lup58]. An exposition of this for the case of NAND is given in Chapter 4 of his book [Lup84]. (Thanks to Sasha Golovnev for tracking down this reference!)

Learning Objectives:

- Understand one of the most important concepts in computing: duality between code and data.
- Build up comfort in moving between different representations of programs.
- Follow the construction of a "universal NAND-CIRC program" that can evaluate other NAND-CIRC programs given their representation.
- See and understand the proof of a major result that compliments the result of the last chapter: some functions require an *exponential* number of gates to compute.
- Understand the *physical extended Church-Turing thesis* that NAND-CIRC programs capture *all* feasible computation in the physical world, and its physical and philosophical implications.

5
Code as data, data as code

> *"The term code script is, of course, too narrow. The chromosomal structures are at the same time instrumental in bringing about the development they foreshadow. They are law-code and executive power - or, to use another simile, they are architect's plan and builder's craft - in one."* , Erwin Schrödinger, 1944.

> *"A mathematician would hardly call a correspondence between the set of 64 triples of four units and a set of twenty other units,"universal", while such correspondence is, probably, the most fundamental general feature of life on Earth"*, Misha Gromov, 2013

A NAND-CIRC program can be thought of as simply a sequence of symbols, each of which can be encoded with zeros and ones using (for example) the ASCII standard. Thus we can represent every NAND-CIRC program (and hence also every Boolean circuit) as a binary string. This statement seems obvious but it is actually quite profound. It means that we can treat circuits or NAND-CIRC programs both as instructions to carrying computation and also as *data* that could potentially be input to other computations.

Big Idea 5 A *program* is a piece of text, and so it can be fed as input to other programs.

This correspondence between *code* and *data* is one of the most fundamental aspects of computing. It underlies the notion of *general purpose* computers, that are not pre-wired to compute only one task, and it is also the basis of our hope for obtaining *general* artificial intelligence. This concept finds immense use in all areas of computing, from scripting languages to machine learning, but it is fair to say that we haven't yet fully mastered it. Indeed many security exploits involve

cases such as "buffer overflows" when attackers manage to inject code where the system expected only "passive" data (see Fig. 5.1). The idea of code as data reaches beyond the realm of electronic computers. For example, DNA can be thought of as both a program and data (in the words of Schrödinger, who wrote before DNA's discovery a book that inspired Watson and Crick, it is both "architect's plan and builder's craft").

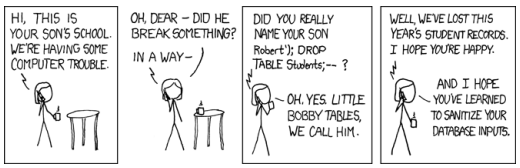


Figure 5.1: As illustrated in this xkcd cartoon, many exploits, including buffer overflow, SQL injections, and more, utilize the blurry line between "active programs" and "static strings".

5.1 A NAND INTERPRETER IN NAND

Since we can represent programs as strings, we can also think of a program as an input to a function. In particular, for every choice of a representation scheme for NAND-CIRC programs, the following is a well defined mathematical function:

$$EVAL(P,x) = \begin{cases} P(x) & |x| = \text{no. of } P\text{'s inputs} \\ 0 & \text{otherwise} \end{cases} \tag{5.1}$$

where P and x are strings in $\{0,1\}^*$, and we denote by $P(x)$ the output of the program represented by the string P on the input x.

P

The above is one of those observations that are simultaneously both simple and profound. Please make sure that you understand (**1**) how for every fixed choice of representing programs as strings, the function *EVAL* above is well defined, and (**2**) what this function actually does.

EVAL takes strings of *arbitrary length*, and hence cannot be computed by a NAND-CIRC program, since such programs take inputs of finite length. However, for every fixed number of lines s, inputs n, and outputs m, we can define $EVAL_{s,n,m}$ to be the function that on input a string P representing an s-line n-input m-output program, and a string $x \in \{0,1\}^n$, output the value $P(x)$.[1] Specifically, we will fix some representation scheme for NAND-CIRC programs and let $S(s)$ denote the number of bits needed to represent programs of s lines. Then $EVAL_{s,n,m}$ takes inputs of length $S+n$ (that is a string $P \in \{0,1\}^S$ representing the program and a string $x \in \{0,1\}^n$ which corresponds to the input) and outputs the string $P(x) \in \{0,1\}^m$.

One of the first examples of *self circularity* we will see in this book is the following theorem:

[1] If $P \in \{0,1\}^S$ is a string that does not describe a valid program then we don't care what $EVAL_{s,n,m}(P,x)$ is. For concreteness we can define $EVAL_{s,n,m}(P,x)$ to equal 0^m in such a case.

Theorem 5.1 — Bounded Universality of NAND-CIRC programs. For every $s, n, m \in \mathbb{N}$ with $s \geq m$ there is a NAND-CIRC program $U_{s,n,m}$ that computes the function $EVAL_{s,n,m}$.

That is, the NAND-CIRC program $U_{s,n,m}$ takes the description of *any other NAND-CIRC program* P (of the right length and input-s/outputs) and *any input* x, and computes the result of evaluating the program P on the input x. Given the equivalence between NAND-CIRC programs and Boolean circuits, we can also think of $U_{s,n,m}$ as a circuit that takes as input the description of other circuits and their inputs, and returns their evaluation, see Fig. 5.2.

We call this NAND-CIRC program $U_{s,n,m}$ that computes $EVAL_{s,n,m}$ a *bounded universal program* (or a *universal circuit*). "Universal" stands for the fact that this is a *single program* that can evaluate *arbitrary* code, where "bounded" stands for the fact that $U_{s,n,m}$ only evaluates programs of bounded size. Of course this limitation is inherent for the NAND-CIRC programming language, since a program of s lines (or, equivalently, a circuit of s gates) can take at most $2s$ inputs. Later, in Chapter 6, we will introduce the concept of *loops*, that allows to escape this limitation.

Of course to fully specify $EVAL_{s,n,m}$, we need to fix a precise representation scheme for NAND-CIRC programs as binary strings. We can simply use the ASCII representation, though below we will choose a more convenient representation. But regardless of the choice of representation, Theorem 5.1 is an immediate corollary of Theorem 4.6, which states that *every* finite function, and so in particular the function $EVAL_{S,n,m}$ above, can be computed by *some* NAND-CIRC program.

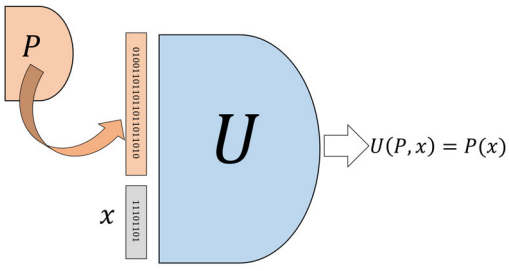


Figure 5.2: A *universal circuit* U is a circuit that gets as input the description of an arbitrary (smaller) circuit C as a binary string, and an input x, and outputs the string $C(x)$ which is the evaluation of C on x. We can also think of U as a straight-line program that gets as input the code of a straight-line program C and an input x, and outputs $C(x)$.

P

Once again, Theorem 5.1 is subtle but important. Make sure you understand what this theorem means, and why it is a corollary of Theorem 4.6.

5.1.1 Efficient universal programs

It turns out that we don't even need to pay that much of an overhead for universality. Namely, the size of U needs only be *polynomial* in the size of the input program.

Theorem 5.2 — Efficient bounded universality of NAND-CIRC programs. For every $s, n, m \in \mathbb{N}$ there is a NAND-CIRC program of at most $O(s^2 \log s)$ lines that computes the function $EVAL_{S,n,m} : \{0,1\}^{S+n} \to \{0,1\}^m$ defined above.

Unlike Theorem 5.1, Theorem 5.2 is not a trivial corollary of the fact that every finite function can be computed by some circuit. Proving Theorem 5.1 requires us to present a concrete NAND-CIRC program for the $EVAL_{s,n,m}$ function. We will do so in several stages.

1. First, we will spell out precisely how to represent NAND-CIRC programs as strings. We can prove Theorem 5.2 using the ASCII representation, but a "cleaner" representation will turn out to be more convenient.

2. Then, we will show how we can write a program to compute $EVAL_{s,n,m}$ in *Python*.[2]

3. Finally, we will show how we can transform this Python program into a NAND-CIRC program.

[2] We will not use much about Python, and a reader that has familiarity with programming in any language should be able to follow along.

5.1.2 Concrete representation for NAND-CIRC programs

A NAND-CIRC program is simply a sequence of lines of the form

```
blah = NAND(baz,boo)
```

There is of course nothing special about the particular names we use for variables. Although they would be harder to read, we could write all our programs using only working variables such as `temp_0`, `temp_1` etc. Therefore, our representation for NAND-CIRC programs will ignore the actual names of the variables, and just associate a *number* with each variable. Thus we will encode a *line* of the program as just a triple of numbers. If the line has the form `foo = NAND(bar,blah)` then we encode it with the triple (i, j, k) where i is the number corresponding to the variable `foo` and j and k are the numbers corresponding to `bar` and `blah` respectively.

More concretely, we use the set $[t] = \{0, 1, \ldots, t-1\}$ as our set of variables. The first n numbers $\{0, \ldots, n-1\}$ will correspond to the *input* variables, the last m numbers $\{t-m, \ldots, t-1\}$ will correspond to the *output* variables, and the intermediate numbers $\{n, \ldots, t-m-1\}$ will correspond to the remaining "workspace" variables.

This motivates the following definition:

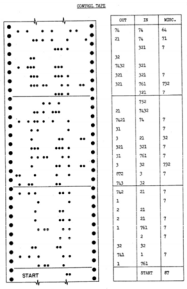


Figure 5.3: In the Harvard Mark I computer, a program was represented as a list of triples of numbers, which were then encoded by perforating holes in a control card.

Definition 5.3 — List of tuples representation. Let P be a NAND-CIRC program of n inputs, m outputs, and s lines, and let t be the number of distinct variables used by P. The *list of tuples representation of* P is the triple (n, m, L) where L is a list of triples of the form (i, j, k) for $i, j, k \in [t]$.

We assign a number for variable of P as follows:

- For every $i \in [n]$, the variable `X[`i`]` is assigned the number i.

- For every $j \in [m]$, the variable `Y[j]` is assigned the number $t-m+j$.
- Every other variable is assigned a number in $\{n, n+1, \ldots, t-m-1\}$ in the order of which it appears.

The list of tuples representation will be our default choice for representing NAND-CIRC programs, and since "list of tuples representation" is a bit of a mouthful, we will often call this simply the *representation* for a program P.

■ **Example 5.4 — Representing the XOR program.** Our favorite NAND-CIRC program, the XOR program:

```
u = NAND(X[0],X[1])
v = NAND(X[0],u)
w = NAND(X[1],u)
Y[0] = NAND(v,w)
```

is represented as the tuple $(2, 1, L)$ where $L = ((2, 0, 1), (3, 0, 2), (4, 1, 2), (5, 3, 4))$. That is, the variables `X[0]` and `X[1]` are given the indices 0 and 1 respectively, the variables `u`,`v`,`w` are given the indices $2, 3, 4$ respectively, and the variable `Y[0]` is given the index 5.

Transforming a NAND-CIRC program from its representation as code to the representation as a list of tuples is a fairly straightforward programming exercise, and in particular can be done in a few lines of *Python*.[3] Note that this representation loses information such as the particular names we used for the variables, but this is OK since these names do not make a difference to the functionality of the program. To represent these s triples numbers as a string requires $O(s \log s)$ bits, as each number in $[t]$ can be represented in the binary basis using $\lceil \log t \rceil$ bits, and $t \leq 3s$.[4] For a fixed s, n, m, we can represent the list of triples of a program with n inputs, m outputs, and s lines by simply concatenating the representation of the $3s$ numbers, with each represented as a string of length $\lceil \log 3s \rceil$ using the binary basis. Hence we can think of $EVAL_{s,n,m}$ as mapping $\{0,1\}^{3s\lceil \log 3s \rceil + n}$ to $\{0,1\}^m$.

[3] If you're curious what these few lines are, see our GitHub repository.

[4] The maximum value t can take is $s+n$ and since every line touches at most two inputs, we can assume that $s \geq n/2$ or $n \leq 2s$ as otherwise there would be an input that is not used in any line of the program.

5.1.3 A NAND interpeter in "pseudocode"

To prove Theorem 5.2 it suffices to give a NAND-CIRC program of $O(s^2 \log s) \leq O((s \log s)^2)$ lines that can evaluate NAND-CIRC programs of s lines. Let us start by thinking how we would evaluate such programs if we weren't restricted to the NAND operations. That is, let us describe informally an *algorithm* that on input n, m, s, a list of

triples L, and a string $x \in \{0,1\}^n$, evaluates the program represented by (n, m, L) on the string x.



It would be highly worthwhile for you to stop here and try to solve this problem yourself. For example, you can try thinking how you would write a program `NANDEVAL(n,m,s,L,x)` that computes this function in the programming language of your choice.

Here is a description of such an algorithm:

Algorithm 5.5 — Eval NAND-CIRC programs.
Input: Numbers n, m and a list L of s triples of numbers in $[t]$ for some $t \leq 3s$, as well as a string $x \in \{0,1\}^n$.
Goal: Evaluate the program represented by (n, m, L) on the input $x \in \{0,1\}^n$.
Operation:

1. We will create a *dictionary* data structure `Vartable` that for every $i \in [t]$ stores a bit. We will assume we have the operations `GET(Vartable,i)` which restore the bit corresponding to `i`, and the operation `UPDATE(Vartable,i,b)` which update the bit corresponding to `i` with the value `b`. (More concretely, we will write this as `Vartable = UPDATE(Vartable,i,b)` to emphasize the fact that the state of the data structure changes, and to keep our convention of using functions free of "side effects".)
2. We will initialize the table by setting the i-th value of `Vartable` to x_i for every $i \in [n]$.
3. We will go over the list L in order, and for every triple (i, j, k) in L, we let a be `GET(Vartable,`j`)`, b be `GET(Vartable,`k`)`, and then set the value corresponding to i to the NAND of a and b. That is, let `Vartable = UPDATE(Vartable,`i`,NAND(`a`,`b`))`.
4. Finally, we output the value `GET(Vartable,`$t-m+j$`)` for every $j \in [m]$.



Please make sure you understand this algorithm and why it does produce the right value.

5.1.4 A NAND interpreter in Python

To make things more concrete, let us see how we implement the above algorithm in the *Python* programming language. We will construct a function NANDEVAL that on input n, m, L, x will output the result of evaluating the program represented by (n, m, L) on x.[5]

```
def NANDEVAL(n,m,L,X):
    # Evaluate a NAND-CIRC program from its list of triple
     ↪ representation.
    s = len(L) # number of lines
    t = max(max(a,b,c) for (a,b,c) in L)+1 # maximum index
     ↪ appearing + 1

    Vartable = [0] * t # we'll simply use an array to
     ↪ store data

    def GET(V,i): return V[i]
    def UPDATE(V,i,b):
        V[i]=b
        return V

    # load input values to Vartable:
    for i in range(n): Vartable = UPDATE(Vartable,i,X[i])

    # Run the program
    for (i,j,k) in L:
        a = GET(Vartable,j)
        b = GET(Vartable,k)
        c = NAND(a,b)
        Vartable = UPDATE(Vartable,i,c)

    # Return outputs Vartable[t-m],
     ↪ Vartable[t-m+1],....,Vartable[t-1]
    return [GET(Vartable,t-m+j) for j in range(m)]

# Test on XOR (2 inputs, 1 output)
L = ((2, 0, 1), (3, 0, 2), (4, 1, 2), (5, 3, 4))
print(NANDEVAL(2,1,L,(0,1))) # XOR(0,1)
# [1]
print(NANDEVAL(2,1,L,(1,1))) # XOR(1,1)
# [0]
```

Accessing an element of the array Vartable at a given index takes a constant number of basic operations. Hence (since $n, m \leq s$ and $t \leq 3s$), the program above will use $O(s)$ basic operations.[6]

[5] To keep things simple, we will not worry about the case that L does not represent a valid program of n inputs and m outputs. Also, there is nothing special about Python. We could have easily presented a corresponding function in JavaScript, C, OCaml, or any other programming language.

[6] Python does not distinguish between lists and arrays, but allows constant time random access to an indexed elements to both of them. One could argue that if we allowed programs of truly unbounded length (e.g., larger than 2^{64}) then the price would not be constant but logarithmic in the length of the array/lists, but the difference between $O(s)$ and $O(s \log s)$ will not be important for our discussions.

5.1.5 Constructing the NAND interpreter in NAND

We now turn to describing the proof of Theorem 5.2. To do this, it is of course not enough to give a Python program. Rather, we need to show how we compute the function $EVAL_{s,n,m}$ itself using a NAND-CIRC program. In other words, our job is to transform, for every s, n, m, the Python code above to a NAND-CIRC program $U_{s,n,m}$ that computes the function $EVAL_{s,n,m}$.

P

Before reading further, try to think how *you* could give a "constructive proof" of Theorem 5.2. That is, think of how you would write, in the programming language of your choice, a function `universal(s,n,m)` that on input s, n, m outputs the code for the NAND-CIRC program $U_{s,n,m}$ such that $U_{s,n,m}$ computes $EVAL_{s,n,m}$. Note that there is a subtle but crucial difference between this function and the Python `NANDEVAL` program described above. Rather than actually evaluating a given program P on some input w, the function `universal` should output the *code* of a NAND-CIRC program that computes the map $(P, x) \mapsto P(x)$.

Our construction will follow very closely the Python implementation of `EVAL` above. We will use variables `Vartable[0]`,...,`Vartable[`$2^\ell-1$`]`, where $\ell = \lceil \log 3s \rceil$ to store our variables. However, NAND doesn't have integer-valued variables, so we cannot write code such as `Vartable[i]` for some variable `i`. However, we *can* implement the function `GET(Vartable,i)` that outputs the `i`-th bit of the array `Vartable`. Indeed, this is nothing but the function `LOOKUP` that we have seen in Theorem 4.4!

P

Please make sure that you understand why `GET` and `LOOKUP` are the same function.

We saw that we can compute `LOOKUP` on arrays of size 2^ℓ in time $O(2^\ell)$, which will be $O(s)$ for our choice of ℓ.

To compute the `update` function on input `V,i,b`, we need to scan the array `V`, and for $j \in [2^\ell]$, have our j-th output be `V[`j`]` unless j is equal to `i`, in which case the j-th output is `b`. We can do this as follows:

1. For every $j \in [2^\ell]$, there is an $O(\ell)$ line NAND-CIRC program to compute the function $EQUALS_j : \{0,1\}^\ell \to \{0,1\}$ that on input i outputs 1 if and only if i is equal to (the binary representation of) j. (We leave verifying this as Exercise 5.2 and Exercise 5.3.)

2. We have seen that we can compute the function $IF : \{0,1\}^3 \to \{0,1\}$ such that $IF(a,b,c)$ equals b if $a = 1$ and c if $a = 0$.

Together, this means that we can compute UPDATE as follows:

```
def UPDATE(V,i,b):
    # update a 2**ell length array at location i to the
     ↪ value b
    for j in range(2**ell): # j = 0,1,2,...,2^ell -1
        a = EQUALS_j(i)
        Y[j] = IF(a,b,V[j])
    return Y
```

Once we can compute GET and UPDATE, the rest of the implementation amounts to "book keeping" that needs to be done carefully, but is not too insightful, and hence we omit the full details.

Since the loop over j in UPDATE is run 2^ℓ times, and computing EQUALS_j takes $O(\ell)$ lines, the total number of lines to compute UPDATE is $O(2^\ell \cdot \ell) = O(s \log s)$. Since we run this function s times, the total number of lines for computing $EVAL_{s,n,m}$ is $O(s^2 \log s)$. This completes (up to the omitted details) the proof of Theorem 5.2.



Remark 5.6 — Improving to quasilinear overhead (advanced optional note). The NAND-CIRC program above is less efficient that its Python counterpart, since NAND does not offer arrays with efficient random access. Hence for example the LOOKUP operation on an array of s bits takes $\Omega(s)$ lines in NAND even though it takes $O(1)$ steps (or maybe $O(\log s)$ steps, depending how we count) in *Python*.

It turns out that it is possible to improve the bound of Theorem 5.2, and evaluate s line NAND-CIRC programs using a NAND-CIRC program of $O(s \log s)$ lines. The key is to consider the description of NAND-CIRC programs as circuits, and in particular as directed acyclic graphs (DAGs) of bounded in degree. A universal NAND-CIRC program U_s for s line programs will correspond to a *universal graph* H_s for such s vertex DAGs. We can think of such as graph U_s as fixed "wiring" for communication network, that should be able to accommodate any arbitrary pattern of communication between s vertices (where this pattern corresponds to an s line NAND-CIRC program). It turns out that there exist such efficient routing networks exist that allow embedding any s vertex circuit inside a universal graph of size $O(s \log s)$, see the bibliographical notes Section 5.8 for more on this issue.

5.2 A PYTHON INTERPRETER IN NAND (DISCUSSION)

To prove Theorem 5.2 we essentially translated every line of the Python program for `EVAL` into an equivalent NAND snippet. It turns out that none of our reasoning was specific to the particular function *EVAL*. It is possible to translate *every* Python program into an equivalent `NAND` program of comparable efficiency.[7] Actually doing so requires taking care of many details and is beyond the scope of this book, but let me try to convince you why you should believe it is possible in principle.

[7] More concretely, if the Python program takes $T(n)$ operations on inputs of length at most n then we can find a NAND-CIRC program of $O(T(n) \log T(n))$ lines that agrees with the Python program on inputs of length n.

For starters, one can can use CPython (the reference implementation for Python), to evaluate every Python program using a `C` program. We can combine this with a C compiler to transform a Python program to various flavors of "machine language". So, to transform a Python program into an equivalent NAND-CIRC program, it is enough to show how to transform a *machine language* program into an equivalent NAND-CIRC program. One minimalistic (and hence convenient) family of machine languages is known as the *ARM architecture* which powers many mobile devices including essentially all Android devices.[8] There are even simpler machine languages, such as the LEG acrhitecture for which a backend for the LLVM compiler was implemented (and hence can be the target of compiling any of large and growing list of languages that this compiler supports). Other examples include the TinyRAM architecture (motivated by interactive proof systems that we will discuss in Chapter 21) and the teaching-oriented Ridiculously Simple Computer architecture. Going one by one over the instruction sets of such computers and translating them to NAND snippets is no fun, but it is a feasible thing to do. In fact, ultimately this is very similar to the transformation that takes place in converting our high level code to actual silicon gates that are not so different from the operations of a NAND-CIRC program. Indeed, tools such as MyHDL that transform "Python to Silicon" can be used to convert a Python program to a NAND-CIRC program.

[8] ARM stands for "Advanced RISC Machine" where RISC in turn stands for "Reduced instruction set computer".

The NAND-CIRC programming language is just a teaching tool, and by no means do I suggest that writing NAND-CIRC programs, or compilers to NAND, is a practical, useful, or enjoyable activity. What I do want is to make sure you understand why it *can* be done, and to have the confidence that if your life (or at least your grade) depended on it, then you would be able to do this. Understanding how programs in high level languages such as Python are eventually transformed into concrete low-level representation such as NAND is fundamental to computer science.

The astute reader might notice that the above paragraphs only outlined why it should be possible to find for every *particular* Python-

computable function f, a *particular* comparably efficient NAND-CIRC program P that computes f. But this still seems to fall short of our goal of writing a "Python interpreter in NAND" which would mean that for every parameter n, we come up with a *single* NAND-CIRC program $UNIV_n$ such that given a description of a Python program P, a particular input x, and a bound T on the number of operations (where the length of P, x and the magnitude of T are all at most n) would return the result of executing P on x for at most T steps. After all, the transformation above would transform every Python program into a different NAND-CIRC program, but would not yield "one NAND-CIRC program to rule them all" that can evaluate every Python program up to some given complexity. However, it turns out that it is enough to show such a transformation for a single Python program. The reason is that we can write a Python interpreter *in Python*: a Python program U that takes a bit string, interprets it as Python code, and then runs that code. Hence, we only need to show a NAND-CIRC program U^* that computes the same function as the particular Python program U, and this will give us a way to evaluate *all* Python programs.

What we are seeing time and again is the notion of *universality* or *self reference* of computation, which is the sense that all reasonably rich models of computation are expressive enough that they can "simulate themselves". The importance of this phenomena to both the theory and practice of computing, as well as far beyond it, including the foundations of mathematics and basic questions in science, cannot be overstated.

5.3 COUNTING PROGRAMS, AND LOWER BOUNDS ON THE SIZE OF NAND-CIRC PROGRAMS

One of the consequences of our representation is the following:

Theorem 5.7 — Counting programs. For every n, m, s with $s \geq m, s \geq n/2$,

$$|SIZE_{n,m}(s)| \leq 2^{O(s \log s)}. \tag{5.2}$$

That is, there are at most $2^{O(s \log s)}$ functions computed by NAND-CIRC programs of at most s lines.

Moreover, the implicit constant in the $O(\cdot)$ notation in Theorem 5.7 is at most 10.[9]

[9] By this we mean that for all sufficiently large s, $|Size(s)| \leq 2^{10 s \log s}$.

Proof Idea:

The idea behind the proof is that, as we've seen, we can represent every s line program by a binary string of $O(s \log s)$ bits. Therefore the number of functions computed by s-line programs cannot be larger

than the number of such strings, which is $2^{O(s \log s)}$. In the actual proof, given below, we count the number of representations a little more carefully, talking directly about triples rather than binary strings, although the idea remains the same.

⋆

Proof of Theorem 5.7. Every NAND-CIRC program P with s lines has at most $3s$ variables. Hence, using our canonical representation, P can be represented by the numbers n, m of P's inputs and outputs, as well as by the list L of s triples of natural numbers, each of which is smaller or equal to $3s$.

If two programs compute distinct functions then they have distinct representations. (Make sure you understand why the above statement is true!) This means that our representation scheme yields a *one to one function* from $SIZE_{n,m}(s)$ to the set $[3s]^{3s}$ of all length-s lists of triples of numbers in $[3s]$.[10] Therefore

$$|SIZE_{n,m}(s)| \leq (3s)^{3s} = 2^{3s \log(3s)} = 2^{3s \log s + 3 \log 3s} \ . \tag{5.3}$$

Since $s = o(s \log s)$, for sufficiently large n, this bound will be smaller than $2^{4s \log s}$.

■

[10] Strictly speaking, the lists have length *at most* s, but we can always add "dummy instructions" to a program with smaller than s lines that would not modify its input/output behavior but ensure that its length is exactly s.

R

Remark 5.8 — Counting by ASCII representation. We can also establish Theorem 5.7 directly from the ASCII representation of the source code. Since an s-line NAND-CIRC program has at most $3s$ distinct variables, we can change all the non input/output variables of such a program to have the form `Temp[`i`]` for i between 0 and $3s - 1$ without changing the function that it computes. This means that after removing extra whitespaces, every line of such a program (which will be of the form form `var = NAND(var',var'')` for variable identifiers which will be either `X[###]`,`Y[###]` or `Temp[###]` where `###` is some number smaller than $3s$) will require at most, say, $20 + 3 \log_{10}(3s) \leq O(\log s)$ characters. Since each one of those characters can be encoded using seven bits in the ASCII representation, we see that the number of functions computed by s-line NAND-CIRC programs is at most $2^{O(s \log s)}$.

A function mapping $\{0,1\}^2$ to $\{0,1\}$ can be identified with the table of its four values on the inputs $00, 01, 10, 11$; a function mapping $\{0,1\}^3$ to $\{0,1\}$ can be identified with the table of its eight values on the inputs $000, 001, 010, 011, 100, 101, 110, 111$. More generally, every function $F : \{0,1\}^n \to \{0,1\}$ can be identified with the table of its 2^n

values on the inputs $\{0,1\}^n$. Hence the number of functions mapping $\{0,1\}^n$ to $\{0,1\}$ is equal to the number of such tables which (since we can choose either 0 or 1 for every row) is exactly 2^{2^n}. Note that this is *double exponential* in n, and hence even for small values of n (e.g., $n = 10$) the number of functions from $\{0,1\}^n$ to $\{0,1\}$ is truly astronomical.[11] This has the following important corollary:

[11] "Astronomical" here is an understatement: there are much fewer than $2^{2^{10}}$ stars, or even particles, in the observable universe.

Theorem 5.9 — Counting argument lower bound. There is a function $F : \{0,1\}^n \to \{0,1\}$ such that the shortest NAND-CIRC program to compute F requires $2^n/(100n)$ lines.

Proof. Suppose, towards the sake of contradiction, that every function $F : \{0,1\}^n \to \{0,1\}$ can be computed by a NAND-CIRC program of at most $s = 2^n/(100n)$ lines. Then the by Theorem 5.7 the total number of such functions would be at most $2^{10s \log s} \leq 2^{10 \log s \cdot 2^n/(100n)}$. Since $\log s = n - \log(100n) \leq n$ this means that the total number of such functions would be at most $2^{2^n/10}$, contradicting the fact that there are 2^{2^n} of them.

■

We have seen before that *every* function mapping $\{0,1\}^n$ to $\{0,1\}$ can be computed by an $O(2^n/n)$ line program. We now see that this is tight in the sense that some functions do require such an astronomical number of lines to compute.

Big Idea 6 Some functions $f : \{0,1\}^n \to \{0,1\}$ *cannot* be computed by a Boolean circuit using a fewer than exponential number of gates.

In fact, as we explore in the exercises below, this is the case for *most* functions. Hence functions that can be computed in a small number of lines (such as addition, multiplication, finding short paths in graphs, or even the *EVAL* function) are the exception, rather than the rule.

R

Remark 5.10 — Advanced note: more efficient representation. The list of triples is not the shortest representation for NAND-CIRC programs. We have seen that every NAND-CIRC program of s lines and n inputs can be represented by a directed graph of $s + n$ vertices, of which n have in-degree zero, and the s others have in-degree at most two. Using the adjacency list representation, such a graph can be represented using roughly $2s \log(s+n) \leq 2s(\log s + O(1))$ bits. Using this representation we can reduce the implicit constant in Theorem 5.7 arbitrarily close to 2.

5.4 SIZE HIERARCHY THEOREM (ADVANCED, OPTIONAL)

By Theorem 4.6 the class $SIZE_n(4 \cdot 2^n)$ contains *all* functions from $\{0,1\}^n$ to $\{0,1\}$. In fact, as discussed in Section 4.4.2 this can be improved to show that there is some constant C such that $SIZE_n(C \cdot 2^n/n)$ contains all functions from $\{0,1\}^n$ to $\{0,1\}$ (in fact, $C = 4$ will do). On the other hand, Theorem 5.9 shows that if $c > 0$ is small enough ($c = 1/4$ will do) then $SIZE_n(c2^n/n)$ does *not* contain all functions. Thus, these two results together show that there exists some constants $C > c > 0$ such that for every sufficiently large n,

$$SIZE_n(c2^n/n) \subsetneq SIZE_n(C2^n/n) \; . \tag{5.4}$$

That is, the set of functions that can be computed using $c2^n/n$ gates is a *strict subset* of the set of functions that can be computed using $C2^n/n$ gates.

In fact, we can use the same results to show a more general result: as a general rule, if we increase our "budget" of gates by a constant factor, then we can compute new functions:

Theorem 5.11 — Size Hierarchy Theorem. There exists some constant C such that for *every* $n \leq s \leq 2^n/(4n)$, there exists some function f that *can not* be computed using s gates but *can* be computed using $C \cdot s$ gates.

Proof Idea:

The idea is to "scale down" the result of Theorem 5.9. We set ℓ to be such that s is about exponential in ℓ, and so that Cs gates are enough to compute all functions on ℓ bits but s gates are not enough to compute some function $g : \{0,1\}^\ell \to \{0,1\}$. We can then let $f : \{0,1\}^n \to \{0,1\}$ be a function that ignores all but the first input ℓ bits, and returns the result of applying g to these bits.

⋆

Proof of **????**. Let s be as in the theorem statement, and let ℓ be the largest integer such that $10s \geq 2^\ell/(10\ell)$. (By this choice $10s < 2^{\ell-1}/(10\ell-1)$ which means that $5s < 2^\ell/(10\ell)$.) Let $g : \{0,1\}^\ell \to \{0,1\}$ be a function outside $SIZE_\ell(2^\ell/(10\ell)) \supseteq SIZE_\ell(5s)$ (the existence of such a function is guaranteed by Theorem 5.9). We let $f : \{0,1\}^n \to \{0,1\}$ be the function defined as $f(x_0, \ldots, x_{n-1}) = g(x_0, \ldots, x_{\ell-1})$. We claim that there is some constant C such that:

- $f \in SIZE_n(C \cdot s)$.
- $f \notin SIZE_n(s)$.

For the former, by the Theorem 4.6 (more precisely its strengthened variant discussed in Section 4.4.2), if C' is sufficiently large then $SIZE_\ell(C' \cdot 2^\ell/\ell)$ contains *all* functions mapping $\{0,1\}^\ell$ to $\{0,1\}$. Therefore we can compute g, and hence f as well, using a circuit of at most $C' \cdot 2^\ell/\ell$ gates which is at most Cs for $C = 10C'$.

For the latter, suppose towards the sake of contradiction that f *can* be computed by an n-input NAND-CIRC program P of at most s lines. Now consider the following program P' which is obtained by P as follows:

1. We add three lines to ensure access to the variable `zero` that is always equal to 0.

2. We replace every instance in which P refers to a variable of the form `X[`i`]` for $i > \ell$ with the variable `zero`.

The new program P' only takes ℓ inputs and has at most $s + 3$ lines. Moreover, for every $x \in \{0,1\}^\ell$, $P'(x) = P(x0^{n-\ell})$. This means that under our assumption that P computes f, $P'(x) = f(x0^{n-\ell}) = g(x)$ for every $x \in \{0,1\}^\ell$. But this contradicts the assumption that $g \notin SIZE_\ell(5s)$.

■

R

Remark 5.12 — Explicit functions. While the size hierarchy theorem guarantees that there exists *some* function that *can* be computed using, for example, n^2 gates, but not using $100n$ gates, we do not know of any explicit example of such a function. While we suspect that integer multiplication is such an example, we do not have any proof that this is the case.

5.5 THE PHYSICAL EXTENDED CHURCH-TURING THESIS (DISCUSSION)

We've seen that NAND gates can be implemented using very different systems in the physical world. What about the reverse direction? Can NAND-CIRC programs simulate any physical computer?

We can take a leap of faith and stipulate that NAND-CIRC programs do actually encapsulate *every* computation that we can think of. Such a statement (in the realm of infinite functions, which we'll encounter in Chapter 6) is typically attributed to Alonzo Church and Alan Turing, and in that context is known as the *Church Turing Thesis*. As we will discuss in future lectures, the Church-Turing Thesis is not a mathematical theorem or conjecture. Rather, like theories in physics,

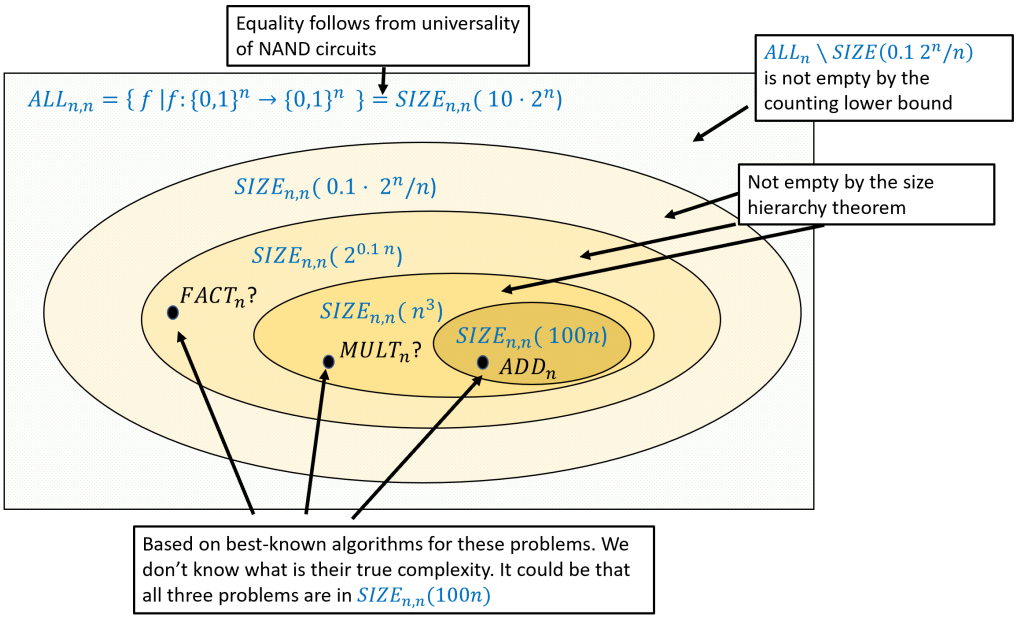


Figure 5.4: An illustration of some of what we know about the size complexity classes (not to scale!). This figure depicts classes of the form $SIZE_{n,n}(s)$ but the state of affairs for other size complexity classes such as $SIZE_{n,1}(s)$ is similar. We know by Theorem 4.6 (with the improvement of Section 4.4.2) that all functions mapping n bits to n bits can be computed by a circuit of size $c \cdot 2^n$ for $c \leq 10$, while on the other hand the counting lower bound (Theorem 5.9, see also Exercise 5.4) shows that *some* such functions will require $0.1 \cdot 2^n$, and the size hierarchy theorem (Theorem 5.11) shows the existence of functions in $SIZE(S) \setminus SIZE(s)$ whenever $s = o(S)$, see also Exercise 5.5. We also consider some specific examples: addition of two $n/2$ bit numbers can be done in $O(n)$ lines, while we don't know of such a program for *multiplying* two n bit numbers, though we do know it can be done in $O(n^2)$ and in fact even better size. In the above $FACTOR_n$ corresponds to the inverse problem of multiplying- finding the *prime factorization* of a given number. At the moment we do not know of any circuit a polynomial (or even sub-exponential) number of lines that can compute $FACTOR_n$.

the Church-Turing Thesis is about mathematically modelling the real world. In the context of finite functions, we can make the following informal hypothesis or prediction:

> *If a function $F : \{0,1\}^n \rightarrow \{0,1\}^m$ can be computed in the physical world using s amount of "physical resources" then it can be computed by a Boolean circuit program of roughly s gates.*

We call this hypothesis the **"Physical Extended Church-Turing Thesis"** or *PECTT* for short. A priori it might seem rather extreme to hypothesize that our meager NAND model captures all possible physical computation. But yet, in more than a century of computing technologies, no one has yet built any scalable computing device that challenges this hypothesis.

We now discuss the "fine print" of the PECTT in more detail, as well as the (so far unsuccessful) challenges that have been raised against it. There is no single universally-agreed-upon formalization of "roughly s physical resources", but we can approximate this notion by considering the size of any physical computing device and the time it takes to compute the output, and ask that any such device can be simulated by a Boolean circuit with a number of gates that is a polynomial (with not too large exponent) in the size of the system and the time it takes it to operate.

In other words, we can phrase the PECTT as stipulating that any function that can be computed by a device of that takes as a certain volume V of space and requires t time to complete the computation, must be computable by a Boolean circuit with a number of gates that is polynomial in V and t.

The exact polynomial is not clear but it is generally accepted that if $f : \{0,1\}^n \rightarrow \{0,1\}$ is an *exponentially hard* function, in the sense that it has no NAND-CIRC program of fewer than, say, $2^{n/2}$ lines, then a demonstration of a physical device that can compute in the real world f for moderate input lengths (e.g., $n = 500$) would be a violation of the PECTT.



Remark 5.13 — Advanced note: making PECTT concrete (advanced optional). We can attempt at a more exact phrasing of the PECTT as follows. Suppose that Z is a physical system that accepts n binary stimuli and has a binary output, and can be enclosed in a sphere of volume V. We say that the system Z *computes* a function $f : \{0,1\}^n \rightarrow \{0,1\}$ within t seconds if whenever we set the stimuli to some value $x \in \{0,1\}^n$, if we measure the output after t seconds then we obtain $f(x)$.

One can then phrase the PECTT as stipulating that if there exists such a system Z that computes F within t seconds, then there exists a NAND-CIRC program that computes F and has at most $\alpha(Vt)^2$ lines, where α is some normalization constant. [12] In particular, suppose that $f : \{0,1\}^n \to \{0,1\}$ is a function that requires $2^n/(100n) > 2^{0.8n}$ lines for any NAND-CIRC program (such a function exists by Theorem 5.9). Then the PECTT would imply that either the volume or the time of a system that computes F will have to be at least $2^{0.2n}/\sqrt{\alpha}$. Since this quantity grows *exponentially* in n, it is not hard to set parameters so that even for moderately large values of n, such a system could not fit in our universe.

To fully make the PECTT concrete, we need to decide on the units for measuring time and volume, and the normalization constant α. One conservative choice is to assume that we could squeeze computation to the absolute physical limits (which are many orders of magnitude beyond current technology). This corresponds to setting $\alpha = 1$ and using the Planck units for volume and time. The *Planck length* ℓ_P (which is, roughly speaking, the shortest distance that can theoretically be measured) is roughly 2^{-120} meters. The *Planck time* t_P (which is the time it takes for light to travel one Planck length) is about 2^{-150} seconds. In the above setting, if a function F takes, say, 1KB of input (e.g., roughly 10^4 bits, which can encode a 100 by 100 bitmap image), and requires at least $2^{0.8n} = 2^{0.8 \cdot 10^4}$ NAND lines to compute, then any physical system that computes it would require either volume of $2^{0.2 \cdot 10^4}$ Planck length cubed, which is more than 2^{1500} meters cubed or take at least $2^{0.2 \cdot 10^4}$ Planck Time units, which is larger than 2^{1500} seconds. To get a sense of how big that number is, note that the universe is only about 2^{60} seconds old, and its observable radius is only roughly 2^{90} meters. The above discussion suggests that it is possible to *empirically falsify* the PECTT by presenting a smaller-than-universe-size system that computes such a function. [13]

[13] We can also consider variants where we use surface area instead of volume, or take (Vt) to a different power than 2. However, none of these choices makes a qualitative difference to the discussion below.

[13] There are of course several hurdles to refuting the PECTT in this way, one of which is that we can't actually test the system on all possible inputs. However, it turns out that we can get around this issue using notions such as *interactive proofs* and *program checking* that we might encounter later in this book. Another, perhaps more salient problem, is that while we know many hard functions exist, at the moment there is *no single explicit function* $F : \{0,1\}^n \to \{0,1\}$ for which we can *prove* an $\omega(n)$ (let alone $\Omega(2^n/n)$) lower bound on the number of lines that a NAND-CIRC program needs to compute it.

5.5.1 Attempts at refuting the PECTT

One of the admirable traits of mankind is the refusal to accept limitations. In the best case this is manifested by people achieving long-standing "impossible" challenges such as heavier-than-air flight, putting a person on the moon, circumnavigating the globe, or even resolving Fermat's Last Theorem. In the worst case it is manifested by people continually following the footsteps of previous failures to try to do proven-impossible tasks such as build a perpetual motion machine, trisect an angle with a compass and straightedge, or refute Bell's in-

equality. The Physical Extended Church Turing thesis (in its various forms) has attracted both types of people. Here are some physical devices that have been speculated to achieve computational tasks that cannot be done by not-too-large NAND-CIRC programs:

- **Spaghetti sort:** One of the first lower bounds that Computer Science students encounter is that sorting n numbers requires making $\Omega(n \log n)$ comparisons. The "spaghetti sort" is a description of a proposed "mechanical computer" that would do this faster. The idea is that to sort n numbers $x_1, \ldots, x_n$, we could cut n spaghetti noodles into lengths $x_1, \ldots, x_n$, and then if we simply hold them together in our hand and bring them down to a flat surface, they will emerge in sorted order. There are a great many reasons why this is not truly a challenge to the PECTT hypothesis, and I will not ruin the reader's fun in finding them out by her or himself.

- **Soap bubbles:** One function $F : \{0,1\}^n \rightarrow \{0,1\}$ that is conjectured to require a large number of NAND lines to solve is the *Euclidean Steiner Tree* problem. This is the problem where one is given m points in the plane $(x_1, y_1), \ldots, (x_m, y_m)$ (say with integer coordinates ranging from 1 till m, and hence the list can be represented as a string of $n = O(m \log m)$ size) and some number K. The goal is to figure out whether it is possible to connect all the points by line segments of total length at most K. This function is conjectured to be hard because it is *NP complete* - a concept that we'll encounter later in this course - and it is in fact reasonable to conjecture that as m grows, the number of NAND lines required to compute this function grows *exponentially* in m, meaning that the PECTT would predict that if m is sufficiently large (such as few hundreds or so) then no physical device could compute F. Yet, some people claimed that there is in fact a very simple physical device that could solve this problem, that can be constructed using some wooden pegs and soap. The idea is that if we take two glass plates, and put m wooden pegs between them in the locations $(x_1, y_1), \ldots, (x_m, y_m)$ then bubbles will form whose edges touch those pegs in the way that will minimize the total energy which turns out to be a function of the total length of the line segments. The problem with this device of course is that nature, just like people, often gets stuck in "local optima". That is, the resulting configuration will not be one that achieves the absolute minimum of the total energy but rather one that can't be improved with local changes. Aaronson has carried out actual experiments (see Fig. 5.5), and saw that while this device often is successful for three or four pegs, it starts yielding suboptimal results once the number of pegs grows beyond that.

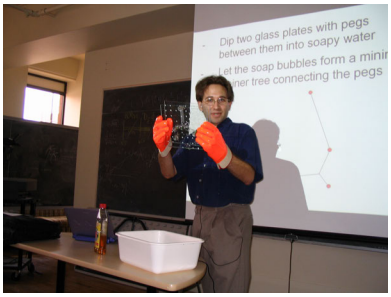


Figure 5.5: Scott Aaronson tests a candidate device for computing Steiner trees using soap bubbles.

- **DNA computing.** People have suggested using the properties of DNA to do hard computational problems. The main advantage of DNA is the ability to potentially encode a lot of information in relatively small physical space, as well as compute on this information in a highly parallel manner. At the time of this writing, it was demonstrated that one can use DNA to store about 10^{16} bits of information in a region of radius about milimiter, as opposed to about 10^{10} bits with the best known hard disk technology. This does not posit a real challenge to the PECTT but does suggest that one should be conservative about the choice of constant and not assume that current hard disk + silicon technologies are the absolute best possible.[14]

- **Continuous/real computers.** The physical world is often described using continuous quantities such as time and space, and people have suggested that analog devices might have direct access to computing with real-valued quantities and would be inherently more powerful than discrete models such as NAND machines. Whether the "true" physical world is continuous or discrete is an open question. In fact, we do not even know how to precisely *phrase* this question, let alone answer it. Yet, regardless of the answer, it seems clear that the effort to measure a continuous quantity grows with the level of accuracy desired, and so there is no "free lunch" or way to bypass the PECTT using such machines (see also this paper). Related to that are proposals known as "hypercomputing" or "Zeno's computers" which attempt to use the continuity of time by doing the first operation in one second, the second one in half a second, the third operation in a quarter second and so on.. These fail for a similar reason to the one guaranteeing that Achilles will eventually catch the tortoise despite the original Zeno's paradox.

- **Relativity computer and time travel.** The formulation above assumed the notion of time, but under the theory of relativity time is in the eye of the observer. One approach to solve hard problems is to leave the computer to run for a lot of time from *his* perspective, but to ensure that this is actually a short while from *our* perspective. One approach to do so is for the user to start the computer and then go for a quick jog at close to the speed of light before checking on its status. Depending on how fast one goes, few seconds from the point of view of the user might correspond to centuries in computer time (it might even finish updating its Windows operating system!). Of course the catch here is that the energy required from the user is proportional to how close one needs to get to the speed of light. A more interesting proposal is to use time travel via *closed timelike curves* (*CTCs*). In this case we could run an arbitrarily long

[14] We were extremely conservative in the suggested parameters for the PECTT, having assumed that as many as $\ell_P^{-2} 10^{-6} \sim 10^{61}$ bits could potentially be stored in a milimeter radius region.

computation by doing some calculations, remembering the current state, and the travelling back in time to continue where we left off. Indeed, if CTCs exist then we'd probably have to revise the PECTT (though in this case I will simply travel back in time and edit these notes, so I can claim I never conjectured it in the first place...)

- **Humans.** Another computing system that has been proposed as a counterexample to the PECTT is a 3 pound computer of about 0.1m radius, namely the human brain. Humans can walk around, talk, feel, and do others things that are not commonly done by NAND-CIRC programs, but can they compute partial functions that NAND-CIRC programs cannot? There are certainly computational tasks that *at the moment* humans do better than computers (e.g., play some video games, at the moment), but based on our current understanding of the brain, humans (or other animals) have no *inherent* computational advantage over computers. The brain has about 10^{11} neurons, each operating in a speed of about 1000 operations per seconds. Hence a rough first approximation is that a Boolean circuit of about 10^{14} gates could simulate one second of a brain's activity.[15] Note that the fact that such a circuit (likely) exists does not mean it is easy to *find* it. After all, constructing this circuit took evolution billions of years. Much of the recent efforts in artificial intelligence research is focused on finding programs that replicate some of the brain's capabilities and they take massive computational effort to discover, these programs often turn out to be much smaller than the pessimistic estimates above. For example, at the time of this writing, Google's neural network for machine translation has about 10^4 nodes (and can be simulated by a NAND-CIRC program of comparable size). Philosophers, priests and many others have since time immemorial argued that there is something about humans that cannot be captured by mechanical devices such as computers; whether or not that is the case, the evidence is thin that humans can perform computational tasks that are inherently impossible to achieve by computers of similar complexity.[16]

- **Quantum computation.** The most compelling attack on the Physical Extended Church Turing Thesis comes from the notion of *quantum computing*. The idea was initiated by the observation that systems with strong quantum effects are very hard to simulate on a computer. Turning this observation on its head, people have proposed using such systems to perform computations that we do not know how to do otherwise. At the time of this writing, Scalable quantum computers have not yet been built, but it is a fascinating possibility, and one that does not seem to contradict any known law of nature. We will discuss quantum computing in much more

[15] This is a very rough approximation that could be wrong to a few orders of magnitude in either direction. For one, there are other structures in the brain apart from neurons that one might need to simulate, hence requiring higher overhead. On the other hand, it is by no mean clear that we need to fully clone the brain in order to achieve the same computational tasks that it does.

[16] There are some well known scientists that have advocated that humans have inherent computational advantages over computers. See also this.

detail in Chapter 22. Modeling quantum computation involves extending the model of Boolean circuits into *Quantum circuits* that hvae one more (very special) gate. However, the main take away is that while quantum computing does suggest we need to amend the PECTT, it does *not* require a complete revision of our worldview. Indeed, almost all of the content of this course remains the same whether the underlying computational model is Boolean circuits or quantum circuits.



Remark 5.14 — PECTT in practice. While even the precise phrasing of the PECTT, let alone understanding its correctness, is still a subject of research, some variant of it is already implicitly assumed in practice. A statement such as "this cryptosystem provides 128 bits of security" really means that (**a**) it is conjectured that there is no Boolean circuit (or, equivalently, a NAND gate) of size much smaller than 2^{128} that can break the system, [17] and (**b**) we assume that no other physical mechanism can do better, and hence it would take roughly a 2^{128} amount of "resources" to break the system.



Lecture Recap

- We can think of programs both as describing a *process*, as well as simply a list of symbols that can be considered as *data* that can be fed as input to other programs.
- We can write a NAND-CIRC program that evaluates arbitrary NAND-CIRC programs (or equivalently a circuit that evaluates other circuits). Moreover, the efficiency loss in doing so is not too large.
- We can even write a NAND-CIRC program that evaluates programs in other programming languages such as Python, C, Lisp, Java, Go, etc.
- By a leap of faith, we could hypothesize that the number of gates in the smallest circuit that computes a function f captures roughly the amount of physical resources required to compute f. This statement is known as the *Physical Extended Church-Turing Thesis* (*PECTT*).
- Boolean circuits (or equivalently AON-CIRC or NAND-CIRC programs) capture a surprisingly wide array of computational models. The strongest currently known challenge to the PECTT comes from the potential for using quantum mechanical effects to speed-up computation, a model known as *quantum computers*.

[17] We say "conjectured" and not "proved" because, while we can phrase such a statement as a precise mathematical conjecture, at the moment we are unable to *prove* such a statement for any cryptosystem. This is related to the **P** vs **NP** question we will discuss in future chapters.

5.6 FINITE COMPUTATION RECAP

The main take-aways from Chapter 3, Chapter 4, and Chapter 5 are:

- We can formally define the notion of a function $f : \{0,1\}^n \rightarrow \{0,1\}^m$ being computed using s basic operations. Whether these operations are AND/OR/NOT, NAND, or some other universal basis does not make much difference. We can describe such a computation either using a *circuit* or using a *straight-line program*.

- *Every* function $f : \{0,1\}^n \rightarrow \{0,1\}^m$ can be computed using a circuit of *at most* $O(m \cdot 2^n/n)$ gates. *Some* functions require *at least* $\Omega(m \cdot 2^n/n)$ gates. We define $SIZE_{n,m}(s)$ to be the set of functions from $\{0,1\}^n$ to $\{0,1\}^m$ that can be computed using at most s gates.

- We can describe a circuit/program P as a string. For every s, there is a *universal* circuit/program U_s that can evaluate programs of length s given their description as strings. We use this representation also to prove that some functions cannot be computed by circuit of smaller-than-exponential size.

- If there is a circuit of s gates that computes a function f, then we can build a physical device to compute f using about s components (such as transistors). If f is a function for which *every* circuit requires at least s gates then the "Physical Extended Church-Turing Thesis" postulates that *every* physical device to compute f will require about s "physical resources". The main challenge to the PECTT is *quantum computing* which we will discuss in Chapter 22.

5.7 EXERCISES

Exercise 5.1 Which one of the following statements is false:

a. There is an $O(s^3)$ line NAND-CIRC program that given as input program P of s lines in the list-of-tuples representation computes the output of P when all its input are equal to 1.

b. There is an $O(s^3)$ line NAND-CIRC program that given as input program P of s characters encoded as a string of $7s$ bits using the ASCII encoding, computes the output of P when all its input are equal to 1.

c. There is an $O(\sqrt{s})$ line NAND-CIRC program that given as input program P of s lines in the list-of-tuples representation computes the output of P when all its input are equal to 1.

■

Exercise 5.2 — Equals function. For every $k \in \mathbb{N}$, show that there is an $O(k)$ line NAND-CIRC program that computes the function $EQUALS_k : \{0,1\}^{2k} \to \{0,1\}$ where $EQUALS(x,x') = 1$ if and only if $x = x'$.

■

Exercise 5.3 — Equal to constant function. For every $k \in \mathbb{N}$ and $x' \in \{0,1\}^k$, show that there is an $O(k)$ line NAND-CIRC program that computes the function $EQUALS_{x'} : \{0,1\}^k \to \{0,1\}$ that on input $x \in \{0,1\}^k$ outputs 1 if and only if $x = x'$.

■

Exercise 5.4 — Counting lower bound for multibit functions. Prove that there exist a number $\epsilon > 0$ such that for every n, m there exists a function $f : \{0,1\}^n \to \{0,1\}^m$ that requires at least $\epsilon m \cdot 2^n/n$ NAND gates to compute. See footnote for hint.[18]

[18] How many functions from $\{0,1\}^n$ to $\{0,1\}^m$ exist?

■

Exercise 5.5 — Size hierarchy theorem for multibit functions. Prove that there exists a number C such that for every n, m and $s < m \cdot 2^n/(Cn)$ there exists a function $f \in SIZE_{n,m}(C \cdot s) \setminus SIZE_{n,m}(s)$. See footnote for hint.[19]

[19] Follow the proof of Theorem 5.11, replacing the use of the counting argument with Exercise 5.4.

■

Exercise 5.6 — Random functions are hard. Suppose $n > 1000$ and that we choose a function $F : \{0,1\}^n \to \{0,1\}$ at random, choosing for every $x \in \{0,1\}^n$ the value $F(x)$ to be the result of tossing an independent unbiased coin. Prove that the probability that there is a $2^n/(1000n)$ line program that computes F is at most 2^{-100}.[20]

[20] **Hint:** An equivalent way to say this is that you need to prove that the set of functions that can be computed using at most $2^n/(1000n)$ has fewer than $2^{-100}2^{2^n}$ elements. Can you see why?

■

Exercise 5.7 The following is a tuple representing a NAND program:
$(3,1,((3,2,2),(4,1,1),(5,3,4),(6,2,1),(7,6,6),(8,0,0),(9,7,8),(10,5,0),(11,9,10))$.

1. Write a table with the eight values $P(000)$, $P(001)$, $P(010)$, $P(011)$, $P(100)$, $P(101)$, $P(110)$, $P(111)$ in this order.

2. Describe what the programs does in words.

■

Exercise 5.8 — EVAL with XOR. For every sufficiently large n, let $E_n : \{0,1\}^{n^2} \to \{0,1\}$ be the function that takes an n^2-length string that encodes a pair (P,x) where $x \in \{0,1\}^n$ and P is a NAND program of n inputs, a single output, and at most $n^{1.1}$ lines, and returns the output of P on x.[21] That is, $E_n(P,x) = P(x)$.

Prove that for every sufficiently large n, there *does not exist* an XOR circuit C that computes the function E_n, where a XOR circuit has the *XOR* gate as well as the constants 0 and 1 (see Exercise 3.3). That is, prove that there is some constant n_0 such that for every $n > n_0$ and

[21] Note that if n is big enough, then it is easy to represent such a pair using n^2 bits, since we can represent the program using $O(n^{1.1}\log n)$ bits, and we can always pad our representation to have exactly n^2 length.

XOR circuit C of n^2 inputs and a single output, there exists a pair (P, x) such that $C(P, x) \neq E_n(P, x)$.

■

5.8 BIBLIOGRAPHICAL NOTES

The *EVAL* function is usually known as *universal circuit*. The implementation we describe is not the most efficient known. Valiant [Val76] first showed a universal circuit of $O(n \log n)$ size where n is the size of the input. Universal circuits have seen in recent years new motivations due to their applications for cryptography, see [LMS16; GKS17] .

While we've seen that "most" functions mapping n bits to one bit require circuits of exponential size $\Omega(2^n/n)$, we actually do not know of any *explicit* function for which we can *prove* that it requires, say, at least n^{100} or even $100n$ size. At the moment, strongest such lower bound we know is that there are quite simple and explicit n-variable functions that require at least $(5 - o(1))n$ lines to compute, see this paper of Iwama et al as well as this more recent work of Kulikov et al. Proving lower bounds for restricted models of circuits is an extremely interesting research area, for which Jukna's book [Juk12] (see also Wegener [Weg87]) provides very good introduction and overview.

Scott Aaronson's blog post on how information is physical is a good discussion on issues related to the physical extended Church-Turing Physics. Aaronson's survey on NP complete problems and physical reality is also a great source for some of these issues, though might be easier to read after we reach Chapter 14 on **NP** and **NP**-completeness.

II

UNIFORM COMPUTATION

Learning Objectives:

- Learn the model of *Turing machines*, which can compute functions of *arbitrary input lengths*.
- See a programming-language description of Turing machines, using NAND-TM programs, which add *loops* and *arrays* to NAND-CIRC.
- See some basic syntactic sugar and equivalence of variants of Turing machines and NAND-TM programs.

6
Loops and infinity

> *"The bounds of arithmetic were however outstepped the moment the idea of applying the [punched] cards had occurred; and the Analytical Engine does not occupy common ground with mere"calculating machines."" ... In enabling mechanism to combine together general symbols, in successions of unlimited variety and extent, a uniting link is established between the operations of matter and the abstract mental processes of the most abstract branch of mathematical science. "*, Ada Augusta, countess of Lovelace, 1843 [1]

> *"What is the difference between a Turing machine and the modern computer? It's the same as that between Hillary's ascent of Everest and the establishment of a Hilton hotel on its peak."* , Alan Perlis, 1982.

[1] Translation of "Sketch of the Analytical Engine" by L. F. Menabrea, Note A.

The model of Boolean circuits (or equivalently, the NAND-CIRC programming language) has one very significant drawback: a Boolean circuit can only compute a *finite* function f, and in particular the number of inputs of f is always smaller than (twice) the number of gates of the circuit.[2]

This does not capture our intuitive notion of an algorithm as a *single recipe* to compute a potentially infinite function. For example, the standard elementary school multiplication algorithm is a *single* algorithm that multiplies numbers of all lengths, but yet we cannot express this algorithm as a single NAND-CIRC program, but rather need a different NAND-CIRC program for every input length (see Fig. 6.1).

Let us consider the case of the simple *parity* or *XOR* function $XOR : \{0,1\}^* \rightarrow \{0,1\}$, where $XOR(x)$ equals 1 iff the number of 1's in x is odd. As simple as it is, the *XOR* function cannot be computed by a NAND-CIRC program. Rather, for every n, we can compute XOR_n (the restriction of *XOR* to $\{0,1\}^n$) using a different NAND-CIRC

[2] Since very NAND gate has two inputs, the number of inputs that an s-gate NAND circuit can depend on is at most $2s$. However, the conceptual point of being only able to handle a finite number of inputs holds for any model of circuits or straight-line programming language.

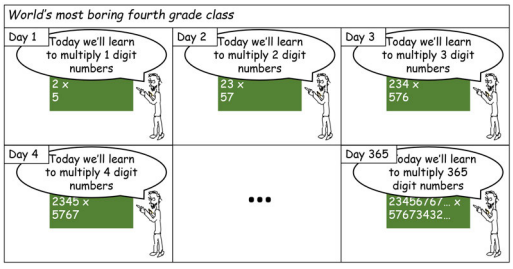


Figure 6.1: Once you know how to multiply multi-digit numbers, you can do so for every number n of digits, but if you had to describe multiplication using NAND-CIRC programs or Boolean circuits, you would need a different program/circuit for every length n of the input.

program. For example, here is the NAND-CIRC program to compute XOR_5: (see also Fig. 6.2)

```
Temp[0] = NAND(X[0],X[1])
Temp[1] = NAND(X[0],Temp[0])
Temp[2] = NAND(X[1],Temp[0])
Temp[3] = NAND(Temp[1],Temp[2])
Temp[4] = NAND(X[2],Temp[3])
Temp[5] = NAND(X[2],Temp[4])
Temp[6] = NAND(Temp[3],Temp[4])
Temp[7] = NAND(Temp[5],Temp[6])
Temp[8] = NAND(Temp[7],X[3])
Temp[9] = NAND(Temp[7],Temp[8])
Temp[10] = NAND(X[3],Temp[8])
Temp[11] = NAND(Temp[9],Temp[10])
Temp[12] = NAND(Temp[11],X[4])
Temp[13] = NAND(Temp[11],Temp[12])
Temp[14] = NAND(X[4],Temp[12])
Y[0] = NAND(Temp[13],Temp[14])
```

This is rather repetitive, and more importantly, does not capture the fact that there is a *single* algorithm to compute the parity on all inputs. Typical programming language use the notion of *loops* to express such an algorithm, and so we might have wanted to use code such as:

```
# s is the "running parity", initialized to 0
while i<len(X):
    u = NAND(s,X[i])
    v = NAND(s,u)
    w = NAND(X[i],u)
    s = NAND(v,w)
    i+= 1
Y[0] = s
```

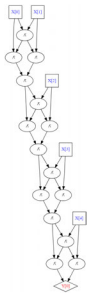


Figure 6.2: The circuit for computing the XOR of 5 bits. Note how it merely repeats four times the circuit to compute the XOR of 2 bits.

In this chapter we will show how we can extend the NAND-CIRC programming language so that it can capture these kinds of constructs. We will see two ways to do so:

- *Turing machines*, invented by Alan Turing in 1936, are an hypothetical abstract device that can yields a finite description of an algorithm that can handle arbitrarily long inputs.

- The *NAND-TM Programming language* extends NAND-CIRC with the notion of *loops* and *arrays* to allow a finite program that can compute a function with arbitrarily long inputs.

It turns out that these two models are *equivalent*, and in fact they are equivalent to a great many other computational models including programming languages you may be familiar with such as C, Java, Python, Javascript, OCaml, and so on and so forth. This notion, known as *Turing equivalence* or *Turing completeness*, will be discussed in Chapter 7. We start off by presenting Turing machines and then show their equivalence to NAND-TM, though it is also possible to present these in the opposite order.



Figure 6.3: Aside from his many other achievements, Alan Turing was an excellent long distance runner who just fell shy of making England's olympic team. A fellow runner once asked him why he punished himself so much in training. Alan said "I have such a stressful job that the only way I can get it out of my mind is by running hard; it's the only way I can get some release."

6.1 TURING MACHINES

> *"Computing is normally done by writing certain symbols on paper. We may suppose that this paper is divided into squares like a child's arithmetic book.. The behavior of the [human] computer at any moment is determined by the symbols which he is observing, and of his 'state of mind' at that moment... We may suppose that in a simple operation not more than one symbol is altered."*,
>
> *"We compare a man in the process of computing ... to a machine which is only capable of a finite number of configurations... The machine is supplied with a 'tape' (the analogue of paper) ... divided into sections (called 'squares') each capable of bearing a 'symbol' "*, Alan Turing, 1936

The "granddaddy" of all models of computation is the *Turing Machine*, which is the standard model of computation in most textbooks.[3] Turing machines were defined in 1936 by Alan Turing in an attempt to formally capture all the functions that can be computed by human "computers" (see Fig. 6.4) that follow a well-defined set of rules, such as the standard algorithms for addition or multiplication.



Figure 6.4: Until the advent of electronic computers, the word "computer" was used to describe a person that performed calculations. These human computers were absolutely essential to many achievements including mapping the stars, breaking the Enigma cipher, and the NASA space mission. Photo taken from from [Sob17].

Turing thought of such a person as having access to as much "scratch paper" as they need. For simplicity we can think of this scratch paper as a one dimensional piece of graph paper (or *tape*, as it is commonly referred to), which is divided to "cells", where each "cell" can hold a single symbol (e.g., one digit or letter, and more generally some element of a finite *alphabet*). At any point in time, the person can read from and write to a single cell of the paper, and based on the contents can update his/her finite mental state, and/or move to the cell immediately to the left or right of the current one.

Thus, Turing modeled such a computation by a "machine" that maintains one of k states, and at each point can read and write a single symbol from some alphabet Σ (containing $\{0,1\}$) from its "work tape" (see Fig. 6.6). To perform computation using this machine, we write the input $x \in \{0,1\}^n$ on the tape, and the goal of the machine is to ensure that at the end of the computation, the value $F(x)$ will be



Figure 6.5: Steam-powered Turing Machine mural, painted by CSE grad students the University of Washington on the night before spring qualifying examinations, 1987. Image from https://www.cs.washington.edu/building/art/SPTM.

[3] This definitional choice does not make much difference since, as we show here, NAND-TM programs are equivalent to Turing machines in their computing power.

written on the tape. Specifically, a computation of a Turing Machine M with k states and alphabet Σ on input $x \in \{0,1\}^*$ proceeds as follows:

- Initially the machine is at state 0 (known as the "starting state") and the tape is initialized to $\triangleright, x_0, \ldots, x_{n-1}, \varnothing, \varnothing, \ldots$.[4]
- The location i to which the machine points to is set to 0.
- At each step, the machine reads the symbol $\sigma = T[i]$ that is in the ith location of the tape, and based on this symbol and its state s decides on:
  - What symbol σ' to write on the tape
  - Whether to move **L**eft (i.e., $i \leftarrow i-1$), **R**ight (i.e., $i \leftarrow i+1$), **S**tay in place, or **H**alt the computation.
  - What is going to be the new state $s \in [k]$
- The set of rules the Turing machine follows is known as its *transition function*.
- When the machine halts then its output is obtained by reading off the tape from the second location (just after the $\triangleright$) onwards, stopping at the first point where the symbol is not 0 or 1.

[4] We use the symbol $\triangleright$ to denote the beginning of the tape, and the symbol $\varnothing$ to denote an empty cell. Hence we will assume that Σ contains these symbols, along with 0 and 1.

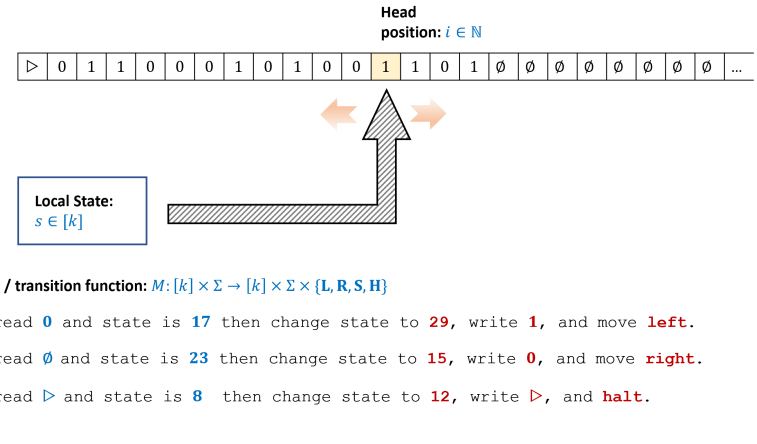


Figure 6.6: A Turing machine has access to a *tape* of unbounded length. At each point in the execution, the machine can read a single symbol of the tape, and based on that and its current state, write a new symbol, update the tape, decide whether to move left, right, stay, or halt.

6.1.1 Example: A Turing machine for palindromes

Let PAL (for *palindromes*) be the function that on input $x \in \{0,1\}^*$, outputs 1 if and only if x is an (even length) *palindrome*, in the sense that $x = w_0 \cdots w_{n-1} w_{n-1} w_{n-2} \cdots w_0$ for some $n \in \mathbb{N}$ and $w \in \{0,1\}^n$.

We now show a Turing Machine M that computes PAL. To specify M we need to specify **(i)** M's tape alphabet Σ which should contain at

least the symboles 0,1, $\triangleright$ and $\varnothing$, and **(ii)** M's *transition function* which determines what action M takes when it reads a given symbol while it is in a particular state.

In our case, M will use the alphabet $\{0, 1, \triangleright, \varnothing, \times\}$ and will have $k = 14$ states. Though the states are simply numbers between 0 and $k - 1$, for convenience we will give them the following labels:

State	Label
0	`START`
1	`RIGHT_0`
2	`RIGHT_1`
3	`LOOK_FOR_0`
4	`LOOK_FOR_1`
5	`RETURN`
6	`REJECT`
7	`ACCEPT`
8	`OUTPUT_0`
9	`OUTPUT_1`
10	`0_AND_BLANK`
11	`1_AND_BLANK`
12	`BLANK_AND_STOP`

We describe the operation of our Turing Machine M in words:

- M starts in state `START` and will go right, looking for the first symbol that is 0 or 1. If we find $\varnothing$ before we hit such a symbol then we will move to the `OUTPUT_1` state that we describe below.

- Once M found such a symbol $b \in \{0, 1\}$, M deletes b from the tape by writing the $\times$ symbol, it enters either the `RIGHT_0` or `RIGHT_1` mode according to the value of b and starts moving rightwards until it hits the first $\varnothing$ or $\times$ symbol.

- Once we found this symbol we into the state `LOOK_FOR_0` or `LOOK_FOR_1` depending on whether we were in the state `RIGHT_0` or `RIGHT_1` and make one left move.

- In the state `LOOK_FOR_`b, we check whether the value on the tape is b. If it is, then we delete it by changing its value to $\times$, and move to the state `RETURN`. Otherwise, we change to the `OUTPUT_0` state.

- The `RETURN` state means we go back to the beginning. Specifically, we move leftward until we hit the first symbol that is not 0 or 1, in which case we change our state to `START`.

- The `OUTPUT_`b states mean that we are going to output the value b. In both these states we go left until we hit $\triangleright$. Once we do so, we make a right step, and change to the `1_AND_BLANK` or `0_AND_BLANK` states respectively. In the latter states, we write the corresponding value, and then move right and change to the `BLANK_AND_STOP` state, in which we write $\varnothing$ to the tape and halt.

The above description can be turned into a table describing for each one of the $13 \cdot 5$ combination of state and symbol, what the Turing machine will do when it is in that state and it reads that symbol. This table is known as the *transition function* of the Turing machine.

6.1.2 Turing machines: a formal definition

The formal definition of Turing machines is as follows:

Definition 6.1 — Turing Machine. A (one tape) *Turing machine* with k states and alphabet $\Sigma \supseteq \{0,1,\triangleright,\varnothing\}$ is represented by a *transition function* $\delta_M : [k] \times \Sigma \to [k] \times \Sigma \times \{\mathsf{L},\mathsf{R},\mathsf{S},\mathsf{H}\}$.

For every $x \in \{0,1\}^*$, the *output* of M on input x, denoted by $M(x)$, is the result of the following process:

- We initialize T to be the sequence $\triangleright, x_0, x_1, \ldots, x_{n-1}, \varnothing, \varnothing, \ldots$, where $n = |x|$. (That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \varnothing$ for $i > n$.)
- We also initialize $i = 0$ and $s = 0$.
- We then repeat the following process:
  1. Let $(s', \sigma', D) = \delta_M(s, T[i])$.
  2. Set $s \to s'$, $T[i] \to \sigma'$.
  3. If $D = \mathsf{R}$ then set $i \to i+1$, if $D = \mathsf{L}$ then set $i \to \max\{i-1, 0\}$. (If $D = \mathsf{S}$ then we keep i the same.)
  4. If $D = \mathsf{H}$ then halt.
- The *result* of the process, which we denote by $M(x)$, is the string $T[1], \ldots, T[m]$ where $m > 0$ is the smallest integer such that $T[m+1] \notin \{0,1\}$. If the process never ends then we denote the result by $\perp$.



You should make sure you see why this formal definition corresponds to our informal description of a Turing Machine. To get more intuition on Turing Machines, you can play with some of the online avail-

able simulators such as Martin Ugarte's, Anthony Morphett's, or Paul Rendell's.

R

Remark 6.2 — Transition functions vs computed functions, comparison to Sipser. One should not confuse the *transition function* of a Turing machine M with the function that the machine computes. The transition function is a *finite* function, with $k|\Sigma|$ inputs and $4k|\Sigma|$ outputs. (Can you see why?) The machine can compute an *infinite* function F that takes as input a string $x \in \{0,1\}^*$ of arbitrary length and might also produce an arbitrary length string as output.

In our formal definition, we identified the machine with its transition function since the transition function tells us everything we need to know about the Turing machine, and hence serves as a good mathematical representation of it. This choice of representation is somewhat arbitrary, and is based on our convention that the state space is always the numbers $\{0, \ldots, k - 1\}$, where we use 0 as our starting. Other texts use different conventions and so their mathematical definition of a Turing machine might look superficially different, but these definitions describe the same computational process and has the same computational powers.

For example, Sipser's text [Sip97] allows a more general set of states Q and allow to designate arbitrary elements of Q as starting and halting states, though by simple relabeling of the states one can see that this has no effect on the computational power of the model. Sipser also restricts attention to Turing machines that output only a single bit. In such cases, it is convenient to have *two halting states*: one of them is designated as the "0 halting state" (often known as the *rejecting state*) and the other as the "1 halting state" (often known as the *accepting state*). Thus instead of writing 0 or 1, the machine will enter into one of these states and halt. This again makes no difference to the computational power, though we prefer to consider the more general model of multi-bit outputs.

Sipser considers also functions with input in Σ^* for an arbitrary alphabet Σ (and hence distiguishes between the *input alphabet* which he denotes as Σ and the *tape alphabet* which he denotes as Γ), while we restrict attention to functions with binary strings as input. Finally (and this is a very minor point) Sipser requires the machine to either move left or right in every step, without the Stay operation, but again staying in place is very easy to emulate by simply moving right and then back left.

The bottom line is that Sipser defines Turing machines as a *seven tuple* consisting of the state space, input alphabet, tape alphabet, transition function, starting state, accpeting state, and rejecting state. Yet, this is simply a different representation of the same concept, just as a graph can be represented in either adjacency list or adjacency matrix form.

6.1.3 Computable functions

We now turn to making one of the most important definitions in this book, that of *computable functions*.

Definition 6.3 — Computable functions. Let $F : \{0,1\}^* \to \{0,1\}^*$ be a (total) function and let M be a Turing machine. We say that M *computes* F if for every $x \in \{0,1\}^*$, $M(x) = F(x)$.

We say that a function F is *computable* if there exists a Turing machine M that computes it.

Defining a function "computable" if and only if it can be computed by a Turing machine might seem "reckless" but, as we'll see in Chapter 7, it turns out that being computable in the sense of Definition 6.3 is equivalent to being computable in essentially any reasonable model of computation. This is known as the *Church Turing Thesis*.[5]

[5] Unlike the *extended* Church Turing Thesis which we discussed in Section 5.5, the Church-Turing thesis itself is widely believed and there are no candidate devices that attack it.

P Definition 6.3 is, as we mentioned above, one of the most important definitions in this book. Please re-read it (and Definition 6.1, which it relies upon).

This is a good point to remind the reader of the distinction between *functions* and *programs*:

$$\text{Functions} \neq \text{Programs} \tag{6.1}$$

A program (or Turing machine) P can *compute* some function F, but it is not the same as F. In particular there can be more than one program to compute the same function. Being computable is a property of *functions*, not of machines.

We will often pay special attention to functions $F : \{0,1\}^* \to \{0,1\}$ that have a single bit of output. Hence we give a special name for the set of functions of this form that are computable.

Definition 6.4 — The class R. We define **R** be the set of all *computable* functions $F : \{0,1\}^* \to \{0,1\}$.

R

Remark 6.5 — Decidable languages. Many texts use the term *decidable languages* instead of *computable functions*. Another term that is often used is *recursive languages* (which is the reason that the letter **R** is often used to denote the set of computable Boolean functions / decidable languages defined in Definition 6.4).

The name "language" has its roots in *formal language theory* as pursued by linguists such as Noam Chomsky. A *formal language* is simply a subset $L \subseteq \{0,1\}^*$ (or more generally $L \subseteq \Sigma^*$ for some finite alphabet Σ). The *membership* or *decision* problem for a language L, is the task of determining, given $x \in \{0,1\}^*$, whether or not $x \in L$. One can see that this task is equivalent to computing the Boolean function $F : \{0,1\}^* \to \{0,1\}$ which is defined as $F(x) = 1$ iff $x \in L$. Thus saying that the function F is computable is equivalent to saying that the corresponding language L is decidable.

6.1.4 Infinite loops and partial functions

One crucial difference between circuits/straight-line programs and Turing machines is the following. Looking at a NAND-CIRC program P, we can always tell how many inputs and how many outputs it has (by simply looking at the `X` and `Y` variables). Furthermore, we are guaranteed that if we invoke P on any input then *some* output will be produced.

In contrast, given any Turing machine M, we cannot determine a priori the length of the output. In fact, we don't even know if an output would be produced at all! For example, it is very easy to come up with a Turing machine whose transition function never outouts `H` and hence never halts.

If a machine M fails to stop and produce an output on some an input x, then it cannot compute any total function F, since clearly on input x, M will fail to output $F(x)$. However, P can still compute a *partial* function.[6]

For example, consider the partial function *DIV* that on input a pair (a,b) of natural numbers, outputs $\lceil a/b \rceil$ if $b > 0$, and is undefined otherwise. We can define a turing machine M that computes *DIV* on input a, b by outputting the first $c = 0, 1, 2, \ldots$ such that $cb \geq a$. If $a > 0$ and $b = 0$ then the machine M will never halt, but this is OK, since *DIV* is undefined on such inputs. If $a = 0$ and $b = 0$, the machine M will output 0, which is also OK, since we don't care about what the program outputs on inputs on which *DIV* is undefined. Formally, we define computability of partial functions as follows:

[6] A *partial function* F from a set A to a set B is a function that is only defined on a *subset* of A, (see Section 1.5.4). We can also think of such a function as mapping A to $B \cup \{\perp\}$ where $\perp$ is a special "failure" symbol such that $F(a) = \perp$ indicates the function F is not defined on a.

Definition 6.6 — Computable (partial or total) functions. Let F be either a total or partial function mapping $\{0,1\}^*$ to $\{0,1\}^*$ and let M be a Turing machine. We say that M *computes* F if for every $x \in \{0,1\}^*$ on which F is defined, $M(x) = F(x)$. [7]

We say that a (partial or total) function F is *computable* if there is a Turing machine that computes it.

[7] Note that if F is a total function, then it is defined on every $x \in \{0,1\}^*$ and hence in this case, this definition is identical to Definition 6.3.

6.2 TURING MACHINES AS PROGRAMMING LANGUAGES

The name "Turing machine", with its "tape" and "head" evokes a physical object, while in contrast we think of a *program* as a piece of text. But we can think of a Turing machine as a program as well. For example, consider the Turing Machine M of Section 6.1.1 that computes the function PAL such that $PAL(x) = 1$ iff x is a palindrome. We can also describe this machine as a *program* using the Python-like pseudocode of the form below

```
# Gets an array Tape that is initialized to [">", x_0 , x_1
 ↪  , .... , x_(n-1), " ", " ", ...]
# At the end of the execution, Tape[1] is equal to 1 if x
 ↪  is a palindrome and is equal to 0 otherwise
def PAL(Tape):
    head = 0
    state = 0 # START
    while (state != 12):
        if (state == 0 && Tape[head]=='0'):
            state = 3 # LOOK_FOR_0
            Tape[head] = 'x'
            head += 1 # move right
        if (state==0 && Tape[head]=='1')
            state = 4 # LOOK_FOR_1
            Tape[head] = 'x'
            head += 1 # move right
        ... # more if statements here
```

The particular details of this program are not important. What is important is that we can describe Turing machines as *programs*. Moreover, note that when translating a Turing machine into a program, the *tape* becomes a *list* or *array* that can hold values from the finite set Σ.[8] The *head position* can be thought of as an integer valued variable that can hold integers of unbounded size. The *state* is a *local register* that can hold one of a fixed number of values in $[k]$.

More generally we can think of every Turing Machine M as equivalent to a program of the following form:

[8] Most programming languages use arrays of fixed size, while a Turing machine's tape is unbounded, though of course there is no need to store an infinite number of ∅ symbols. If you want, you can think of the tape as a list that starts off at some a length that is just long enough to store the input, but is dynamically grown in size as the Turing machine's head explores new positions.

```
# Gets an array Tape that is initialized to [">", x_0 , x_1
 ↪  , .... , x_(n-1), " ", " ", ...]
def M(Tape):
    state = 0
    i     = 0
    while (True):
        if Head[i]=="0" and state==7:
            i += 1
            Head[i]="1"
            state = 19
        elif Head[i]==">" and state == 13:
            Head[i]="0"
            state = 15
        elif ...
        ...
        elif Head[i]==">1" and state == 29:
            break
```

Note that if our programming language has only *Boolean* (i.e., 0/1-valued) variables, then we could replace the variable `state` with $\lceil \log k \rceil$ Boolean variables. Similarly, we can represent each element of the alphabet Σ using $\lceil \log |\Sigma| \rceil$ bits. Hence if our programming language had only Boolean-valued arrays, we could replace the Σ-valued array `Tape[]` with $\lceil \log |\Sigma| \rceil$ Boolean-valued arrays `Tape0[]`,..., `Tape`ℓ`[]` for $\ell = \lceil \log |\Sigma| \rceil$.

6.2.1 The NAND-TM Programming language

We now introduce the *NAND-TM programming language*, which aims to capture the power of a Turing machine in a programming language formalism. The main difference between NAND-TM and NAND-CIRC is that NAND-TM will need to model a *single uniform algorithm* that can compute a function that takes inputs of *arbitrary lengths*. To do so, we extend the NAND-CIRC programming language with two constructs:

- *Loops*: NAND-CIRC is a *straight-line* programming language- a NAND-CIRC program of s lines takes exactly s steps of computation and hence in particular cannot even touch more than $3s$ variables. *Loops* allow us to capture in a short program the instructions for a computation that can take an arbitrary amount of time.

- *Arrays*: A NAND-CIRC program of s lines touches at most $3s$ variables. While we can use variables with names such as `Foo_17` or `Bar[22]`, they are not true arrays, since the number in the identifier is a constant that is "hardwired" into the program.

Thus a good way to remember NAND-TM is using the following informal equation:

$$\text{NAND-}TM = \text{NAND-}CIRC + \text{loops} + \text{arrays} \tag{6.2}$$



Remark 6.7 — NAND-CIRC + loops + arrays = everything.. As we will see, adding loops and arrays to NAND-CIRC is enough to capture the full power of all programming languages! Hence we could replace "NAND-TM" with any of *Python*, *C*, *Javascript*, *OCaml*, etc... in the lefthand side of (6.2). But we're getting ahead of ourselves: this issue will be discussed in Chapter 7.

Concretely, the NAND-TM programming language adds the following features on top of NANC-CIRC:

- We add a special *integer valued* variable `i`. All other variables in NAND-TM will be *Boolean valued* (as in NAND-CIRC).
- We add *arrays* to the language by allowing variable identifiers to have the form `Foo[i]` with `i` being the special integer-valued variable mentioned above. `Foo` is an array of Boolean values, and `Foo[i]` refers to the value of this array at location equal to the current value of the variable `i`.
- We use the convention that *arrays* always start with a capital letter, and *scalar variables* (which are never indexed with `i`) start with lowercase letters. Hence `Foo` is an array and `bar` is a scalar variable.
- The input and output `X` and `Y` are now considered *arrays* with values of zeroes and ones.
- We add a special `MODANDJUMP` instruction that takes two boolean variables a, b as input and does the following:
  - If $a = 1$ and $b = 1$ then `MODANDJUMP`(a, b) increments `i` by one and jumps to the first line of the program.
  - If $a = 0$ and $b = 1$ then `MODANDJUMP`(a, b) decrements `i` by one and jumps to the first line of the program. (If `i` is already equal to 0 then it tays at 0.)
  - If $a = 1$ and $b = 0$ then `MODANDJUMP`(a, b) jumps to the first line of the program without modifying `i`.
  - If $a = b = 0$ then `MODANDJUMP`(a, b) halts execution of the program.

- Note that since MODANDJUMP always either jumps to the first line of the program or halts the computation, the instructions following it will never get executed. Hence in NAND-TM programs the MODANDJUMP instruction appears in the last line of the program and nowhere else.

R

Remark 6.8 — Default values and conventions. We also need one more convention to handle "default values". In a Turing machine the tape could contain in addition to 0 and 1 (and possibly other values) also the special symbol ∅ to indicate that this location is "blank". All our variables, including our arrays, will be Boolean, and so they can contain either the value 0 or 1. However, we can achieve the same effect by adding to every array Foo a "companion array" Foo_nonblank that encodes this information.

Specifically, for every i and array Foo, if the value Foo[i] has not been written to, then it defaults to 0. However, we will establish the convention that for every array Foo there is a "companion array" Foo_nonblank such that Foo_nonblank[i]$=1$ if the value Foo[i] has been written to. In particular we will use this convention for the input and output arrays X and Y. Hence the input corresponds to the values X[0], ..., X[$n-1$] where n is the smallest integer such that X_nonblank[n]$=0$, and the output will be the values Y[0], ..., Y[$m-1$] where m is the smallest integer such that Y_nonblank[m]$=0$. A NAND-TM program gets called with X and X_nonblank initialized to contain the input, and writes to Y and Y_nonblank to produce the output.

6.2.2 Sneak peak: NAND-TM vs Turing machines

As the name implies, NAND-TM programs are a direct implementation of Turing machines in programming language form. we will show the equivalence below but you can already see how the components of Turing machines and NAND-TM programs correspond to one another:

Table 6.2: Turing Machine and NAND-TM analogs

Turing Machine	NAND-TM program
State: single register that takes values in $[k]$	*Scalar variables:* Several variables such as foo, bar etc.. each taking values in $\{0,1\}$.

Turing Machine	NAND-TM program
Tape: One tape containing values in a finite set Σ. Potentially infinite but $T[t]$ defaults to $\emptyset$ for all locations t that have not been accessed.	*Arrays:* Several arrays such as `Foo`, `Bar` etc.. for each such array `Arr` and index j, the value of `Arr` at position j is either 0 or 1. The value defaults to 0 for position that have not been written to.
Head location: A number $i \in \mathbb{N}$ that encodes the position of the head.	*Index variable:* The variable `i` that can be used to access the arrays.
Accessing memory: At every step the Turing machine has access to its local state, but can only access the tape at the position of the current head location.	*Accessing memory:* At every step a NAND-TM program has access to all the scalar variables, but can only access the arrays at the location `i` of the index variable
Control of location: In each step the machine can move the head location by at most one position.	*Control of index variable:* In each iteration of its main loop the program can modify the index `i` by at most one.

6.2.3 Examples

We now present some examples of NAND-TM programs

■ **Example 6.9 — XOR in NAND-TM.** The following is a NAND-TM program to compute the XOR function on inputs of arbitrary length. That is $XOR : \{0,1\}^* \rightarrow \{0,1\}$ such that $XOR(x) = \sum_{i=0}^{|x|-1} x_i \mod 2$ for every $x \in \{0,1\}^*$.

```
temp_0 = NAND(X[0],X[0])
Y_nonblank[0] = NAND(X[0],temp_0)
temp_2 = NAND(X[i],Y[0])
temp_3 = NAND(X[i],temp_2)
temp_4 = NAND(Y[0],temp_2)
Y[0] = NAND(temp_3,temp_4)
MODANDJUMP(X_nonblank[i],X_nonblank[i])
```

■ **Example 6.10 — Increment in Enhanced NAND-TM.** We now present enhanced NAND-TM program to compute the *increment function*. That is, $INC : \{0,1\}^* \rightarrow \{0,1\}^*$ such that for every $x \in \{0,1\}^n$, $INC(x)$ is the $n+1$ bit long string y such that if $X = \sum_{i=0}^{n-1} x_i \cdot 2^i$

is the number represented by x, then y is the (least-significant digit first) binary representation of the number $X + 1$.

We start by showing the program using the "syntactic sugar" we've seen before of using shorthand for some NAND-CIRC programs we have seen before to compute simple functions such as `IF`, `XOR` and `AND` (as well as the constant one function as well as the function `COPY` that just maps a bit to itself).

```
carry = IF(started,carry,one(started))
started = one(started)
Y[i] = XOR(X[i],carry)
carry = AND(X[i],carry)
Y_nonblank[i] = one(started)
MODANDJUMP(X_nonblank[i],X_nonblank[i])
```

The above is not, strictly speaking, a valid enhanced NAND-TM program. If we "open up" all of the syntactic sugar, we get the following valid program to compute this syntactic sugar.

```
temp_0 = NAND(started,started)
temp_1 = NAND(started,temp_0)
temp_2 = NAND(started,started)
temp_3 = NAND(temp_1,temp_2)
temp_4 = NAND(carry,started)
carry = NAND(temp_3,temp_4)
temp_6 = NAND(started,started)
started = NAND(started,temp_6)
temp_8 = NAND(X[i],carry)
temp_9 = NAND(X[i],temp_8)
temp_10 = NAND(carry,temp_8)
Y[i] = NAND(temp_9,temp_10)
temp_12 = NAND(X[i],carry)
carry = NAND(temp_12,temp_12)
temp_14 = NAND(started,started)
Y_nonblank[i] = NAND(started,temp_14)
MODANDJUMP(X_nonblank[i],X_nonblank[i])
```

P

Working out the above two example can go a long way towards understanding the NAND-TM language. See the appendix and our [GitHub repository](https://github.com/boazbk/tcscode] for a full specification of the NAND-TM language.

6.3 EQUIVALENCE OF TURING MACHINES AND NAND-TM PROGRAMS

Given the above discussion, it might not be surprising that Turing machines turn out to be equivalent to NAND-TM programs. Indeed, we designed the NAND-TM language to have this property. Nevertheless, this is an important result, and the first of many other such equivalence results we will see in this book.

Theorem 6.11 — Turing machines and NAND-TM programs are equivalent. For every $F : \{0,1\}^* \to \{0,1\}^*$, F is computable by a NAND-TM program P if and only if there is a Turing Machine M that computes F.

Proof Idea:

To prove such an equivalence theorem, we need to show two directions. We need to be able to **(1)** transform a Turing machine M to a NAND-TM program P that computes the same function as P and **(2)** transform a NAND-TM program P into a Turing machine M that computes the same function as P.

The idea of the proof is illustrated in Fig. 6.7. To show **(1)**, given a Turing machine M, we will create a NAND-TM program P that will have an array `Tape` for the tape of M and scalar (i.e., non array) variable(s) `state` for the state of M. Specifically, since the state of a Turing machine is not in $\{0,1\}$ but rather in a larger set $[k]$, we will use $\lceil \log k \rceil$ variables `state_0` , ..., `state_`$\lceil \log k \rceil - 1$ variables to store the representation of the state. Similarly, to encode the larger alphabet Σ of the tape, we will use $\lceil \log |\Sigma| \rceil$ arrays `Tape_0` , ..., `Tape_`$\lceil \log |\Sigma| \rceil - 1$, such that the ith location of these arrays encodes the ith symbol in the tape for every tape. Using the fact that *every* function can be computed by a NAND-CIRC program, we will be able to compute the transition function of M, replacing moving left and right by decrementing and incrementing `i` respectively.

We show **(2)** using very similar ideas. Given a program P that uses a array variables and b scalar variables, we will create a Turing machine with about 2^b states to encode the values of scalar variables, and an alphabet of about 2^a so we can encode the arrays using our tape. (The reason the sizes are only "about" 2^a and 2^b is that we will need to add some symbols and steps for bookkeeping purposes.) The Turing Machine M will simulate each iteration of the program P by updating its state and tape accordingly.

⋆

Proof of Theorem 6.11. We start by proving the "if" direction of Theorem 6.11. Namely we show that given a Turing machine M, we can

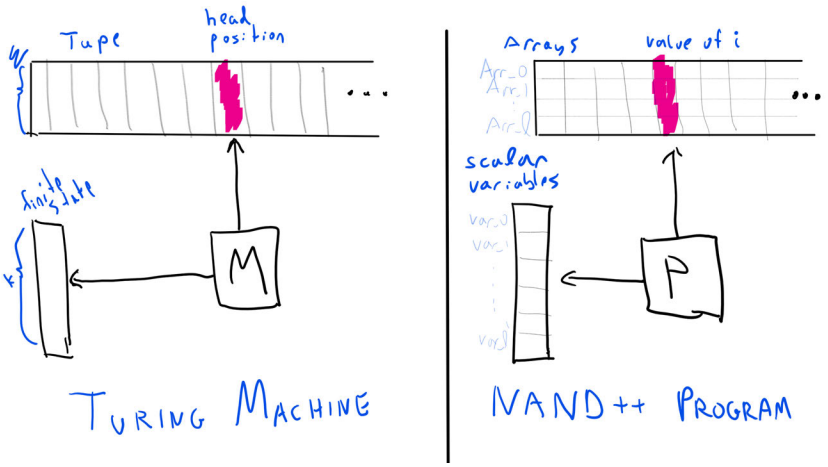


Figure 6.7: Comparing a Turing Machine to a NAND-TM program. Both have an unbounded memory component (the *tape* for a Turing machine, and the *arrays* for a NAND-TM program), as well as a constant local memory (*state* for a Turing machine, and *scalar variables* for a NAND-TM program). Both can only access at each step one location of the unbounded memory, this is the "head" location for a Turing machine, and the value of the index variable `i` for a NAND-TM program.

find a NAND-TM program P_M such that for every input x, if M halts on input x with output y then $P_M(x) = y$. Since our goal is just to show such a program P_M *exists*, we don't need to write out the full code of P_M line by line, and can take advantage of our various "syntactic sugar" in describing it.

The key observation is that by Theorem 4.6 we can compute *every* finite function using a NAND-CIRC program. In particular, consider the transition function $\delta_M : [k] \times \Sigma \to [k] \times \Sigma \times \{\mathsf{L}, \mathsf{R}\}$ of our Turing Machine. We can encode the its components as follows:

- We encode $[k]$ using $\{0,1\}^\ell$ and Σ using $\{0,1\}^{\ell'}$, where $\ell = \lceil \log k \rceil$ and $\ell' = \lceil \log |\Sigma| \rceil$.
- We encode the set $\{\mathsf{L}, \mathsf{R}, \mathsf{S}, \mathsf{H}\}$ using $\{0,1\}^2$. We will choose the encode $\mathsf{L} \mapsto 01$, $\mathsf{R} \mapsto 11$, $\mathsf{S} \mapsto 10$, $\mathsf{H} \mapsto 00$. (This conveniently corresponds to the semantics of the `MODANDJUMP` operation.)

Hence we can identify δ_M with a function $\overline{M} : \{0,1\}^{\ell+\ell'} \to \{0,1\}^{\ell+\ell'+2}$, mapping strings of length $\ell + \ell'$ to strings of length $\ell + \ell' + 2$. By Theorem 4.6 there exists a finite length NAND-CIRC program `ComputeM` that computes this function $\overline{M}$. The NAND-TM program to simulate M will essentially be the following:

```
state, Tape[i], dir0, dir1  = ComputeM(state, Tape[i])
MODANDJUMP(dir0,dir1)
```

where we use `state` as shorthand for the tuple of variables `state_0`, ..., `state_`$\ell - 1$ and `Tape[i]` as shorthand for `Tape_0[i]` ,..., `Tape_`$\ell' - 1$`[i]`. (We need to add a little "book keeping" to translate the output and the input from the tape to the `X` and `Y` variables, but the above is basically it.) Since every step of the main loop of the

above program perfectly mimics the computation of the Turing Machine M as `ComputeM` computes the transition of the Turing Machine, and the program carries out exactly the definition of computation by a Turing Machine as per Definition 6.1.

For the other direction, suppose that P is a NAND-TM program with s lines, ℓ scalar variables, and ℓ' array variables. We will show that there exists a Turing machine M_P with $2^\ell + C$ states and alphabet Σ of size $C' + 2^{\ell'}$ that computes the same functions as P (where C, C' are some constants to be determined later).

Specifically, consider the function $\overline{P} : \{0,1\}^\ell \times \{0,1\}^{\ell'} \rightarrow \{0,1\}^\ell \times \{0,1\}^{\ell'}$ that on input the contents of P's scalar variables and the contents of the array variables at location `i` in the beginning of an iteration, outputs all the new values of these variables at the last line of the iteration, right before the `MODANDJUMP` instruction is executed.

If `foo` and `bar` are the two variables that are used as input to the `MODANDJUMP` instruction, then this means that based on the values of these variables we can compute whether `i` will increase, decrease or stay the same, and whether the program will halt or jump back to the beginning. Hence a Turing machine can simulate an execution of P in one iteration using a finite function applied to its alphabet. The overall operation of the Turing machine will be as follows:

1. The machine M_P encodes the contents of the array variables of P in its tape, and the contents of the scalar variables in (part of) its state.

2. Initially, the machine M_P will scan the input and copy the result to the parts of the tape corresponding to the `X` and `X_nonblank` variables of P. (We use some extra states and alphabet symbols to achieve this.)

3. The machine will M_P then simulates each iterations of P by applying the constant function to update the state and the location of the head, as long as the `loop` variable of P equals 1.

4. When the loop variable equals 0, the machine M_P will scan the output arrays and copy them to the beginning of the tape. (Again we can add some states and alphabet symbols to achieve this.)

5. At the end of this scan the machine M_P will enter its halting state.

The above is not a full formal description of a Turing Machine, but our goal is just to show that such a machine exists. One can see that M_P simulates every step of P, and hence computes the same function as P.

■



Remark 6.12 — Running time equivalence (optional). If we examine the proof of Theorem 6.11 then we can see that the every iteration of the loop of a NAND-TM program corresponds to one step in the execution of the Turing machine. We will come back to this question of measuring number of computation steps later in this course. For now the main take away point is that NAND-TM programs and Turing Machines are essentially equivalent in power even when taking running time into account.

6.3.1 Specification vs implementation (again)

Once you understand the definitions of both NAND-TM programs and Turing Machines, Theorem 6.11 is fairly straightforward. Indeed, NAND-TM programs are not as much a different model from Turing Machines as they are simply a reformulation of the same model using programming language notation. You can think of the difference between a Turing machine and a NAND-TM program as the difference between representing a number using decimal or binary notation. In contrast, the difference between a *function* F and a Turing machine that computes F is much more profound: it is like the difference between an equation E and a number that is a solution for E. For this reason, while we take special care in distinguishing *functions* from *programs* or *machines*, we will often identify the two latter concepts. We will move freely between describing an algorithm as a Turing machine or as a NAND-TM program (as well as some of the other equivalent computational models we will see in Chapter 7 and beyond).

Table 6.3: Specification vs Implementation formalisms

Specification	*Implementation*
Functions mapping $\{0,1\}^*$ to $\{0,1\}$ or to $\{0,1\}^*$.	**Algorithms, Turing Machines, Programs**

6.4 NAND-TM SYNTACTIC SUGAR

Just like we did with NAND-CIRC in Chapter 4, we can use "syntactic sugar" to make NAND-TM programs easier to write. For starters, we can use all of the syntactic sugar of NAND-CIRC, and so have access to macro definitions and conditionals (i.e., if/then). But we can go beyond that and also implement more advanced *looping constructs* than the simple `MODANDJUMP`.

For staters, we can implement GOTO. A GOTO statement corresponds to jumping to a certain line in the execution. For example, if we have code of the form

```
"line1":  do foo
   GOTO("line2")
"line2": do bar
"line3": do blah
```

then the program will only do foo and blah as when it reaches the line GOTO("gohere") it will jump to the line labeled with "gohere". We can achieve the effect of GOTO in NAND-TM using conditionals. In the code below, we assume that we have variables that can encode strings of finite length (which will be our labels). Each such variable can be emulated using a constant number of scalar boolean variables. We also assume that we have access to conditional (i.e., if statements), which we can emulate in the same way as NAND.

To emulate a GOTO statement, we will first modify a program P of the form

```
do foo
do bar
do blah
```

to have the following form (using syntactic sugar for if and variables taking finite length string)

```
pc = "line1"
if (pc=="line1"):
    do foo
    pc = "line2"
if (pc=="line2"):
    do bar
    pc = "line3"
if (pc=="line3"):
    do blah
```

The two programs do the same thing. The variable pc corresponds to the "program counter" and tells the program which line to execute next. We can see that if we wanted to emulate a GOTO("line3") then we could simply modify the instruction pc = "line2" to be pc = "line3".

In NAND-CIRC we could only have GOTOs that go forward in the code, but since in NAND-TM everything is encompassed within a large outer loop, we can use the same ideas to implement GOTO's that can go backwards, as well as conditional loops. Using this, we can emulate all the standard loop constructs such as while, do .. until or for in NAND-TM as well.

R

Remark 6.13 — GOTO's in programming languages. The `GOTO` statement was a staple of most early programming languages, but has largely fallen out of favor and is not included in many modern languages such as *Python*, *Java*, *Javascript*. In 1968, Edsger Dijsktra wrote a famous letter titled "Go to statement considered harmful." (see also Fig. 6.8). The main trouble with `GOTO` is that it makes analysis of programs more difficult by making it harder to argue about *invariants* of the program.

When a program contains a loop of the form:

```
for j in range(100):
    do something

do blah
```

you know that the line of code `do blah` can only be reached if the loop ended, in which case you know that `j` is equal to 100, and might also be able to argue other properties of the state of the program. In contrast, if the program might jump to `do blah` from any other point in the code, then it's very hard for you as the programmer to know what you can rely upon in this code. As Dijkstra said, such invariants are important because *"our intellectual powers are rather geared to master static relations and .. our powers to visualize processes evolving in time are relatively poorly developed"* and so *"we should ... do ...our utmost best to shorten the conceptual gap between the static program and the dynamic process."*

That said, `GOTO` is still a major part of lower level languages where it is used to implement higher level looping constructs such as `while` and `for` loops. For example, even though *Java* doesn't have a `GOTO` statement, the Java Bytecode (which is a lower level representation of Java) does have such a statement. Similarly, Python bytecode has instructions such as `POP_JUMP_IF_TRUE` that implement the `GOTO` functionality, and similar instructions are included in many assembly languages. The way we use `GOTO` to implement a higher level functionality in NAND-TM is reminiscent of the way these various jump instructions are used to implement higher level looping constructs.

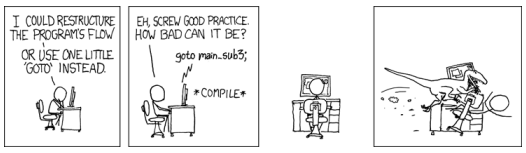


Figure 6.8: XKCD's take on the `GOTO` statement.

6.4.1 Well formed programs: The NAND-TM style manual

The notion of passing between different variants of programs can be extremely useful, as often, given a program P that we want to analyze, it would be simpler for us to first modify it to an equivalent program P' that has some convenient properties. You can think of this as the NAND-TM equivalent of enforcing "coding conventions" that are

often used for programming languages. For example, while this is not part of the Python language, Google's Python style guide stipulates that variables that are initialized to a value and never changed (i.e., constants) are typed with all capital letters. (Similar requirements are used in other style guides.) Of course this does not really restrict the power of Google-conforming Python programs, since every Python program can be transformed to an equivalent one that satisfies this requirement. In fact, many programming languages have automatic programs known as linters that can detect and sometimes modify the program to fit certain standards.

The following solved exercise is an example of that. We will define the notion of a *well-formed program* and show that every NAND-TM program can be transformed into an equivalent one that is well formed.

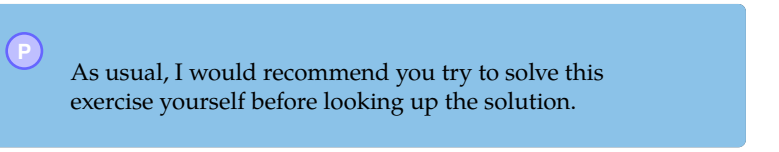
Definition 6.14 — Well-formed programs. We say that a NAND-TM program P is *well formed* if it satisfies the following properties:

- Every reference to a variable in P either has the form `foo` or `foo_123` (a *scalar variable:* alphanumerical string starting with a lowercase letter and no brackets) or the form `Bar[i]` or `Bar_12[i]` (an *array variable* alphanumerical string starting with a capital letter and ending with `[i]`).
- P contains the scalar variables `zero`, `one`, `dir0`, and `dir1` such that `zero` and `one` are always the constants 0 and 1 respectively, and `MODANDJUMP` is always invoked with the variables `dir0` and `dir1`.
- P contains the array variables `Visited` and `Atstart` and code to ensure that `Atstart[` i `]` equals 1 if and only if $i = 0$, and `Visited[`i`]` equals 1 for all the positions i such that the program finished an iteration with the index variable `i` equalling i.

The following exercise shows that we can transform every NAND-TM program P into a well-formed program P' that is equivalent to it. Hence if we are given a NAND-TM program P, we can (and will) often assume without loss of generality that it is well-formed.

Lemma 6.15 For every NAND-TM program P, there exists an NAND-TM program P' equivalent to P that is *well formed* as pre Definition 6.14. That is, for every input $x \in \{0,1\}^*$, either both P and P' do not halt on x, or both P and P' halt on x and produce the same output $y \in \{0,1\}^*$.

Solved Exercise 6.1 — Making an NAND-TM program well formed.. Prove Lemma 6.15



As usual, I would recommend you try to solve this exercise yourself before looking up the solution.

Solution:

Since variable identifiers on their own have no meaning in (enhanced) NAND-TM (other than the special ones `X`, `X_nonblank`, `Y`, `Y_nonblank` that already have the desired properties), we can easily achieve the property that scalars variables start with lowercase and arrays with uppercase using "search and replace". We just have to take care that we don't make two distinct identifiers become the same. For example, we can do so by changing all scalar variable identifiers to lower case, and adding to them the prefix `scalar_`, and adding the prefix `Array_` to all array variable identifiers.

The property that an array variable is never references with a numerical index is more challenging. We need to remove all references to an array variable with an actual numerical index rather than `i`. One thought might be to simply convert a a reference of the form `Arr[17]` to the scalar variable `arr_17`. However, this will not necessarily preserve the functionality of the program. The reason is that we want to ensure that when `i`$=17$ then `Arr[i]` would give us the same value as `arr_17`.

Nevertheless, we can use the approach above with a slight twist. We will demonstrate the solution in a concrete case.(Needless to say, if you needed to solve this question in a problem set or an exam, such a demonstration of a special case would not be sufficient; but this example should be good enough for you to extrapolate a full solution.) Suppose that there are only three references to array variables with numerical indices in the program: `Foo[5]`, `Bar[12]` and `Blah[22]`. We will include three scalar variables `foo_5`, `bar_12` and `blah_22` which will serve as a *cache* for the values of these arrays. We will change all references to `Foo[5]` to `foo_5`, `Bar[12]` to `bar_12` and so on and so forth. But in addition to that, whenever in the code we refer to `Foo[i]` we will check if `i`$=5$ and if so use the value `foo_5` instead, and similarly with `Bar[i]` or `Blah[i]`.

Specifically, we will change our program as follows. We will create an array `Is_5` such that `Is_5[i]`$=1$ if and only `i`$=5$, and similarly create arrays `Is_12`, `Is_22`.

We can then change code of the following form

```
Foo[i] = something
```

to

```
temp = something
foo_5 = IF(Is_5[i],temp,foo_5)
Foo[i] = temp
```

and similarly code of the form

```
blah = NAND(Bar[i],baz)
```

to

```
temp = If(Is_22[i],bar_22,Bar[i])
blah = NAND(temp,baz)
```

To create the arrays we can add code of the following form in the beginning of the program (here we're using enhanced NAND-TM syntax, `GOTO`, and the constant `one` but this syntactic sugar can of course be avoided):

```
# initialization of arrays
GOTO("program body",init_done)
i += one
i += one
i += one
i += one
i += one
Is_5[i] = one
i += one
... # repeat i += one 6 more times
Is_12[i] = one
i += one
... # repeat i += one 9 more times
Is_22[i] = one
i -= one
... # repeat i -= one 21 more times
init_done = one
```

```
LABEL("program body")
original code of program..
```

Using `IF` statements (which can easily be replaced with syntactic sugar) we can handle the conditions that `loop`, `Y`, and `Y_nonblank` are not written to once `loop` is set to 0. We leave completing all the details as an exercise to the reader (see **??**).

■

6.5 UNIFORMITY, AND NAND VS NAND-TM (DISCUSSION)

While NAND-TM adds extra operations over NAND-CIRC, it is not exactly accurate to say that NAND-TM programs or Turing machines are "more powerful" than NAND-CIRC programs. NAND-CIRC programs, having no loops, are simply not applicable for computing functions with an bounded number of inputs. Thus, to compute a function $F : \{0,1\}^* \to \{0,1\}^*$ using NAND-CIRC (or equivalently, Boolean circuits) we need a *collection* of programs/circuits: one for every input length.

The key difference between NAND-CIRC and NAND-TM is that NAND-TM allows us to express the fact that the algorithm for computing parities of length-100 strings is really the same one as the algorithm for computing parities of length-5 strings (or similarly the fact that the algorithm for adding n-bit numbers is the same for every n, etc.). That is, one can think of the NAND-TM program for general parity as the "seed" out of which we can grow NAND-CIRC programs for length 10, length 100, or length 1000 parities as needed.

This notion of a single algorithm that can compute functions of all input lengths is known as *uniformity* of computation and hence we think of Turing machines / NAND-TM as *uniform* model of computation, as opposed to Boolean circuits or NAND-CIRC which is a *nonuniform* model, where we have to specify a different program for every input length.

Looking ahead, we will see that this uniformity leads to another crucial difference between Turing machines and circuits. Turing machines can have inputs and outputs that are longer than the description of the machine as a string and in particular there exists a Turing machine that can "self replicate" in the sense that it can print its own code. This notion of "self replication", and the related notion of "self reference" is crucial to many aspects of computation, as well of course to life itself, whether in the form of digital or biological programs.

For now, what you ought to remember is the following differences between *uniform* and *non uniform* computational models:

- **Non uniform computational models:** Examples are *NAND-CIRC programs* and *Boolean circuits*. These are models where each indi-

vidual program/circuit can compute a *finite* function $f : \{0,1\}^n \rightarrow \{0,1\}^m$. We have seen that *every* finite function can be computed by *some* program/circuit. To discuss computation of an *infinite* function $F : \{0,1\}^* \rightarrow \{0,1\}^*$ we need to allow a *sequence* $\{P_n\}_{n\in\mathbb{N}}$ of programs/circuits (one for every input length), but this does not capture the notion of a *single algorithm* to compute the function F.

- **Uniform computational models:** Examples are *Turing machines* and *NAND-TM programs*. These are model where a single program/-machine can take inputs of *arbitrary length* and hence compute an *infinite* function $F : \{0,1\}^* \rightarrow \{0,1\}^*$. The number of steps that a program/machine takes on some input is not a priori bounded in advance and in particular there is a chance that it will enter into an *infinite loop*. Unlike the nonuniform case, we have *not* shown that every infinite function can be computed by some NAND-TM program/Turing Machine. We will come back to this point in Chapter 8.



Lecture Recap

- *Turing machines* capture the notion of a single algorithm that can evaluate functions of every input length.
- They are equivalent to *NAND-TM programs*, which add loops and arrays to NAND-CIRC.
- Unlike NAND-CIRC or Boolean circuits, the number of steps that a Turing machine takes on a given input is not fixed in advance. In fact, a Turing machine or a NAND-TM program can enter into an *infinite loop* on certain inputs, and not halt at all.

6.6 EXERCISES

Exercise 6.1 — Explicit NAND TM programming. Produce the code of a (syntactic-sugar free) NAND-TM program P that computes the (unbounded input length) *Majority* function $Maj : \{0,1\}^* \rightarrow \{0,1\}$ where for every $x \in \{0,1\}^*$, $Maj(x) = 1$ if and only if $\sum_{i=0}^{|x|} x_i > |x|/2$. We say "produce" rather than "write" because you do not have to write the code of P by hand, but rather can use the programming language of your choice to compute this code.

■

Exercise 6.2 Prove the following closure properties of the set **R** defined in Definition 6.4:

1. If $F \in \mathbf{R}$ then the function $G(x) = 1 - F(x)$ is in **R**.

2. If $F, G \in \mathbf{R}$ then the function $H(x) = F(x) \vee G(x)$ is in $\mathbf{R}$.

3. If $F \in \mathbf{R}$ then the function F^* in in $\mathbf{R}$ where F^* is defined as follows: $F^*(x) = 1$ iff there exist some strings $w_0, \ldots, w_{k-1}$ such that $x = w_0 w_1 \cdots w_{k-1}$ and $F(w_i) = 1$ for every $i \in [k]$.

4. If $F \in \mathbf{R}$ then the function

$$G(x) = \begin{cases} \exists_{y \in \{0,1\}^{|x|}} F(xy) = 1 \\ 0 \qquad\qquad \text{otherwise} \end{cases} \tag{6.3}$$

is in $\mathbf{R}$.

■

Exercise 6.3 — Single vs multiple bit. Prove that for every $F : \{0,1\}^* \to \{0,1\}^*$, the function F is computable if and only if the following function $G : \{0,1\}^* \to \{0,1\}$ is computable, where G is defined as follows:

$$G(x,i,\sigma) = \begin{cases} F(x)_i & i < |F(x)|, \sigma = 0 \\ 1 & i < |F(x)|, \sigma = 1 \\ 0 & i \geq |F(x)| \end{cases}$$

■

Exercise 6.4 — Uncomputability via counting. Recall that $\mathbf{R}$ is the set of all total functions from $\{0,1\}^*$ to $\{0,1\}$ that are computable by a Turing machine (see Definition 6.4). Prove that $\mathbf{R}$ is *countable*. That is, prove that there exists a one-to-one map $DtN : \mathbf{R} \to \mathbb{N}$. You can use the equivalence between Turing machines and NAND-TM programs.

■

Exercise 6.5 — Not every function is computable. Prove that the set of *all* total functions from $\{0,1\}^* \to \{0,1\}$ is *not* countable. You can use the results of Section 2.3.1. (We will see an *explicit* uncomputable function in Chapter 8.)

■

6.7 BIBLIOGRAPHICAL NOTES

Augusta Ada Byron, countess of Lovelace (1815-1852) lived a short but turbulent life, though is today most well known for her collaboration with Charles Babbage (see [Ste87] for a biography). Ada took an immense interest in Babbage's *analytical engine*, which we mentioned in Chapter 3. In 1842-3, she translated from Italian a paper of Menabrea on the engine, adding copious notes (longer than the paper itself) including some examples of programs. Because of these programs, some call Ada "the first computer programmer" though recent evidence shows they were likely written by Babbage himself [Hol01]. Regardless, Ada was clearly one of very few people (perhaps the only

one outside of Babbage himself) to fully appreciate how significant and revolutionary the idea of mechanizing computation truly is.

The books of Shetterly [She16] and Sobel [Sob17] discuss the history of human computers (which were more often than not women) and their important contributions to scientific discoveries.

Alan Turing was one of the intellectual giants of the 20th century. He was not only the first person to define the notion of computation, but also invented and used some of the world's earliest computational devices as part of the effort to break the *Enigma* cipher during World War II, saving millions of lives. Tragically, Turing committed suicide in 1954, following his conviction in 1952 for homosexual acts and a court-mandated hormonal treatment. In 2009, British prime minister Gordon Brown made an official public apology to Turing, and in 2013 Queen Elizabeth II granted Turing a posthumous pardon. Turing's life is the subject of a great book and a mediocre movie.

One of the first programming-language formulations of Turing machines was given by Wang [Wan57]. Our formulation is aimed at making the connection with circuits more direct, with the eventual goal of using it for the Cook-Levin Theorem, as well as results such as $\mathbf{P} \subseteq \mathbf{P}_{/\mathbf{poly}}$ and $\mathbf{BPP} \subseteq \mathbf{P}_{/\mathbf{poly}}$. The website esolangs.org features a large variety of esoteric Turing-complete programming languages. One of the most famous of them is Brainf*ck.

In this book, we use the terminology of *functions* as opposed to *languages*. These are easily related by identifying a language $L \subseteq \{0,1\}^*$ with its characteristic function $1_L : \{0,1\}^* \to \{0,1\}$ defined as $1_L(x) = 1$ iff $x \in L$. The characteristic function of a language is always a total function. The language-like object that corresponds to a *partial function* is known as a promise problem.

Learning Objectives:

- Learn about RAM machines and λ calculus, which are important models of computation.
- See the equivalence between these models and Turing machines.
- See how many other models turn out to be "Turing complete"
- Understand the Church-Turing thesis.

7
Equivalent models of computation

> *"All problems in computer science can be solved by another level of indirection"*, attributed to David Wheeler.

> *"Because we shall later compute with expressions for functions, we need a distinction between functions and forms and a notation for expressing this distinction. This distinction and a notation for describing it, from which we deviate trivially, is given by Church."*, John McCarthy, 1960 (in paper describing the LISP programming language)

So far we have defined the notion of computing a function based on Turing machines, which don't really correspond to the way computation is done in practice. In this chapter we justify this choice by showing that the definition of computable functions will remain the same under a wide variety of computational models. In fact, a widely believed claim known as the *Church-Turing Thesis* holds that *every* "reasonable" definition of computable function is equivalent to ours. We will discuss the Church-Turing Thesis and the potential definitions of "reasonable" in Section 7.8.

7.1 RAM MACHINES AND NAND-RAM

One of the limitations of NAND-TM (and Turing machines) is that we can only access one location of our arrays/tape at a time. If `i`$=22$ and we want to access `Foo`$[957]$ then it will take us at least 923 steps to get there. In contrast, almost every programming language has a formalism for directly accessing memory locations. Hardware implementations also provide so called *Random Access Memory* (*RAM*) which can be thought of as a large array `Memory`, such that given an index p (i.e., memory address, or a *pointer*), we can read from and write to the pth location of `Memory`.[1]

[1] "Random access memory" is quite a misnomer, since it has nothing to do with probability. *Indexed access* would have been more appropriate. However, the term "random access" is standard in both the theoretical and practical literature, and hence we will use it as well.

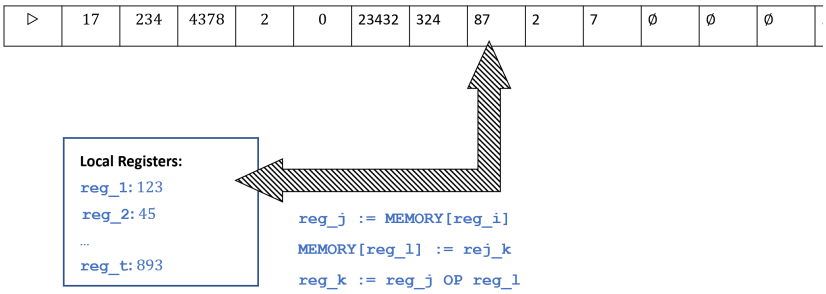


Figure 7.1: A *RAM Machine* contains a finite number of local registers, each of which holds an integer, and an unbounded memory array. It can perform arithmetic operations on its register as well as load to a register r the contents of the memory at the address indexed by the number in register r'.

The computational model that allows access to such a memory is known as a *RAM machine* (sometimes also known as the *Word RAM model*), as depicted in Fig. 7.1. In this model the memory is an array of unbounded size where each cell can store a single *word*, which we think of as a string in $\{0,1\}^w$ and also as a number in $[2^w]$. For example, many modern computing architectures use 64 bit words, in which every memory location holds a string in $\{0,1\}^{64}$ which can also be thought of as a number between 0 and $2^{64}-1 = 9,223,372,036,854,775,807$. The parameter w is known as the *word size* and (when doing theory) is chosen as some function of the input length n. A typical choice is that $w = c \log n$ for some constant c. In addition to the memory array, a RAM machine also contains a constant number of *registers* $r_0, \ldots, r_{k-1}$, each of which can also contain a single word. The operations in this model include loops, arithmetic on registers, and most importantly the ability to read and write to memory at the location specified by one of the register. Hence RAM machines can directly access each location of memory without having to move the "head" to that position as one needs to do in Turing machines.

We will not give a formal definition of RAM Machines, though the bibliographical notes section (Section 7.10) contains sources for such definitions. Rather, we will use an extension of the NAND-TM programming language to capture RAM algorithms. Specifically, we define the *NAND-RAM programming language* to be the following extension of NAND-TM:

- The variables are allowed to be (non negative) *integer valued* rather than only Boolean. That is, a scalar variable `foo` holds an non negative integer in $\mathbb{N}$ (rather than only a bit in $\{0,1\}$), and an array variable `Bar` holds an array of integers. As in the case of RAM machines, we will not allow integers of unbounded size. Concretely, each variable holds a number between 0 and T, where T is the number of steps that have been executed by the program so far.[2]

[2] You can ignore this restriction for now: if we want to hold larger numbers, we can simply execute dummy instructions. This restriction will be useful in later chapters, where we will be interested in a more realistic accounting of running time. Also, while RAM machines have a single memory array, we allow several arrays in NAND-RAM. This does not make any difference. For example, one can simulate five arrays `Array0[]`, ..., `Array4[]` using a single array `Array[]` by replacing calls to `Array`i`[$j$]` with `Array[`$5j+i$`]`.

- We allow *indexed access* to arrays. If `foo` is a scalar and `Bar` is an array, then `Bar[foo]` refers to the location of `Bar` indexed by the value of `foo`. (Note that this means we don't need to have a special index variable `i` any more.)
- As is often the case in programming languages, we will assume that for Boolean operations such as `NAND`, a zero valued integer is considered as *false*, and a nonzero valued integer is considered as *true*.

To make NAND-RAM more realistic and similar to modern computer architecture, we make NAND-RAM "batteries included" and so the following features are built-in into NAND-TM (as opposed to using "syntactic sugar"):[3]

- In addition to `NAND`, NAND-RAM also includes all the basic arithmetic operations of addition, subtraction, multiplication, (integer) division, as well as comparisons (equal, greater than, less than, etc..)
- We will also include as part of the language basic control flow structures such as `if` and `while`.

The full description of the NAND-RAM programing language is in the appendix. However, the most important fact you need to know about NAND-RAM is the following:

[3] The difference between having "built in" vs "syntactic sugar" features is immaterial at this point in the book, but we do so with an eye toward the later parts of this book, when we start counting the number of operations of our algorithms. Even then, the effect of including these features vs implementing them via syntactic sugar will not be very dramatic.

Theorem 7.1 — NAND-TM (TM's) and NAND-RAM (RAM) are equivalent. For every function $F : \{0,1\}^* \to \{0,1\}^*$, F is computable by a NAND-TM program if and only if F is computable by a NAND-RAM program.

Proof Idea:

Clearly NAND-RAM is only more powerful than NAND-TM, and so if a function F is computable by a NAND-TM program then it can be computed by a NAND-RAM program. The challenging direction is of course to transform a NAND-RAM program P to an equivalent NAND-TM program Q. To describe the proof in full we will need to cover the full formal specification of the NAND-RAM language, and show how we can implement every one of its features as syntactic sugar on top of NAND-TM.

This can be done but going over all the operations in detail is rather tedious. Hence we will focus on describing the main ideas behind this transformation. The transformation has two steps:

1. *Indexed access of bit arrays:* NAND-RAM generalizes NAND-TM in two main ways: (**a**) adding *indexed access* to the arrays (ie..,

`Foo[bar]` syntax) and **(b)** moving from *Boolean valued* variables to *integer valued* ones. We will start by showing how to handle **(a)**. Namely, we will show how we can implement in NAND-TM the operation `Setindex(Bar)` such that if `Bar` is an array that encodes some integer j, then after executing `Setindex(Bar)` the value of `i` will equal to j. This will allow us to simulate syntax of the form `Foo[Bar]` by `Setindex(Bar)` followed by `Foo[i]`.

2. *Two dimensional bit arrays:* We will then show how we can use "syntactic sugar" to augment NAND-TM with *two dimensional arrays*. That is, have *two indices* `i` and `j` and *two dimensional arrays*, such that we can use the syntax `Foo[i][j]` to access the `(i,j)`-th location of `Foo`

3. *Arrays of integers:* Finally we will encode a one dimensional array `Arr` of *integers* by a two dimensional `Arrbin` of *bits*. The idea is simple: if $a_{i,0}, \ldots, a_{i,\ell}$ is a binary (prefix-free) representation of `Arr[`i`]`, then `Arrbin[`i`][`j`]` will be equal to $a_{i,j}$.

Once we have arrays of integers, we can use our usual syntactic sugar for functions, `GOTO` etc. to implement the arithmetic and control flow operations of NAND-RAM.

⋆

7.2 THE GORY DETAILS (OPTIONAL)

We do not show the full formal proof of Theorem 7.1 but focus on the most important parts: implementing indexed access, and simulating two dimensional arrays with one dimensional ones. Even these are already quite tedious to describe, as will not be surprising to anyone that has ever written a compiler. Hence you can feel free to merely skim this section. The important point is not for you to know all details by heart but to be convinced that in principle it *is* possible to transform a NAND-RAM program to an equivalent NAND-TM program, and even be convinced that, with sufficient time and effort, *you* could do it if you wanted to.

7.2.1 Indexed access in NAND-TM

Let us choose some prefix-free representation for the natural numbers (see Section 2.4.2). For example, if a natural number k is equal to $\sum_{i=0}^{\ell} k_i \cdot 2^i$ for $\ell = \lfloor \log k \rfloor$, then we can represent it as the string $(k_0, k_0, k_1, k_1, \ldots, k_\ell, k_\ell, 1, 0)$.

To implement indexed access in NAND-TM, we need to be able to do the following. Given an array `Bar`, implement to operation `Setindex(Bar)` that will set `i` to the value encoded by `Bar`. This can be achieved as follows:

1. Set `i` to zero, by decrementing it until we reach the point where `Atzero[i]`$= 1$ (where `Atzero` is an array that has 1 only in position 0).
2. Let `Temp` be an array encoding the number 0.
3. While the number encoded by `Temp` differs from the number encoded by `Bar`:
   a. Increment `Temp`
   b. GOTO the same line (along the performing a `MODANDJUMP` operation that increments `i` by one).

At the end of the loop, `i` is equal to the value at `Bar`, and so we can use this to read or write to arrays at the location corresponding to this value. In code, we can implement the above operations as follows:

```
# assume Atzero is an array such that Atzero[0]=1
# and Atzero[j]=0 for all j>0

# set i to 0.
LABEL("zero_idx")
dir0 = zero
dir1 = one
# corresponds to i <- i-1
GOTO("zero_idx",NOT(Atzero[i]))

...

# zero out temp
#(code below assumes a specific prefix-free encoding in
 ↪  which 10 is the "end marker")
Temp[0] = 1
Temp[1] = 0
# set i to Bar, assume we know how to increment, compare
LABEL("increment_temp")
cond = EQUAL(Temp,Bar)
dir0 = one
dir1 = one
# corresponds to i <- i+1
INC(Temp)
GOTO("increment_temp",cond)
# if we reach this point, i is number encoded by Bar

...
```

```
# final instruction of program
MODANDJUMP(dir0,dir1)
```

7.2.2 Two dimensional arrays in NAND-TM

To implement two dimensional arrays, we want to embed them in a one dimensional array. The idea is that we come up with a *one to one* function $embed : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, and so embed the location (i, j) of the two dimensional array `Two` in the location $embed(i, j)$ of the array `One`.

Since the set $\mathbb{N} \times \mathbb{N}$ seems "much bigger" than the set $\mathbb{N}$, a priori it might not be clear that such a one to one mapping exists. However, once you think about it more, it is not that hard to construct. For example, you could ask a child to use scissors and glue to transform a 10" by 10" piece of paper into a 1" by 100" strip. If you think about it, this is essentially a one to one map from $[10] \times [10]$ to $[100]$. We can generalize this to obtain a one to one map from $[n] \times [n]$ to $[n^2]$ and more generally a one to one map from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$. Specifically, the following map $embed$ would do (see Fig. 7.2):

$$embed(x, y) = \tfrac{1}{2}(x + y)(x + y + 1) + x \quad . \tag{7.1}$$

We ask you to prove that $embed$ is indeed one to one, as well as computable by a NAND-TM program, in Exercise 7.2.

So, we can replace code of the form `Two[Foo][Bar] = something` (i.e., access the two dimensional array `Two` at the integers encoded by the one dimensional arrays `Foo` and `Bar`) by code of the form:

```
Blah = embed(Foo,Bar)
Setindex(Blah)
Two[i] = something
```

Computing `embed` is left for you the reader as Exercise 7.2, but let us hint that this can be done by simply following the grade-school algorithms for multiplication, addition, and division.

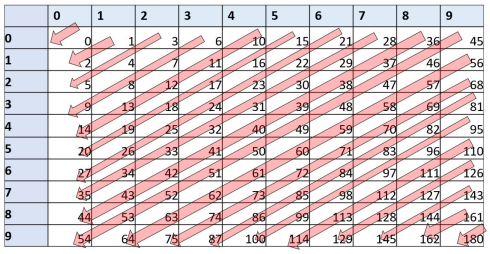


Figure 7.2: Illustration of the map $embed(x, y) = \frac{1}{2}(x + y)(x + y + 1) + x$ for $x, y \in [10]$, one can see that for every distinct pairs (x, y) and (x', y'), $embed(x, y) \neq embed(x', y')$.

7.2.3 All the rest

Once we have two dimensional arrays and indexed access, simulating NAND-RAM with NAND-TM is just a matter of implementing the standard algorithms for arithmetic operations and comparators in NAND-TM. While this is cumbersome, it is not difficult, and the end result is to show that every NAND-RAM program P can be simulated by an equivalent NAND-TM program Q, thus completing the proof of Theorem 7.1.

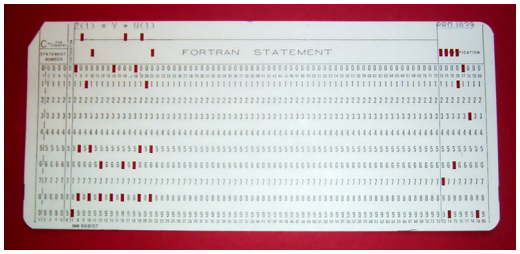


Figure 7.3: A punched card corresponding to a Fortran statement.

7.3 TURING EQUIVALENCE (DISCUSSION)

Any of the standard programming language such as `C`, `Java`, `Python`, `Pascal`, `Fortran` have very similar operations to NAND-RAM. (In-

deed, ultimately they can all be executed by machines which have a fixed number of registers and a large memory array.) Hence using Theorem 7.1, we can simulate any program in such a programming language by a NAND-TM program. In the other direction, it is a fairly easy programming exercise to write an interpreter for NAND-TM in any of the above programming languages. Hence we can also simulate NAND-TM programs (and so by Theorem 6.11, Turing machines) using these programming languages. This property of being equivalent in power to Turing Machines / NAND-TM is called *Turing Equivalent* (or sometimes *Turing Complete*). Thus all programming languages we are familiar with are Turing equivalent.[4]

[4] Some programming language have fixed (even if extremely large) bounds on the amount of memory they can access, which formally prevent them from being applicable to computing infinite functions and hence simulating Turing machines. We ignore such issues in this discussion and assume access to some storage device without a fixed upper bound on its capacity.



Remark 7.2 — Recursion in NAND-RAM (advanced). One concept that appears in many programming languages but we did not include in NAND-RAM programs is *recursion*. However, recursion (and function calls in general) can be implemented in NAND-RAM using the stack data structure. A *stack* is a data structure containing a sequence of elements, where we can "push" elements into it and "pop" them from it in "first in last out" order.

We can implement a stack using an array of integers `Stack` and a scalar variable `stackpointer` that will be the number of items in the stack. We implement `push(foo)` by

```
Stack[stackpointer]=foo
stackpointer += one
```

and implement `bar = pop()` by

```
bar = Stack[stackpointer]
stackpointer -= one
```

We implement a function call to F by pushing the arguments for F into the stack. The code of F will "pop" the arguments from the stack, perform the computation (which might involve making recursive or non recursive calls) and then "push" its return value into the stack. Because of the "first in last out" nature of a stack, we do not return control to the calling procedure until all the recursive calls are done.

The fact that we can implement recursion using a non-recursive language is not surprising. Indeed, *machine languages* typically do not have recursion (or function calls in general), and hence a compiler implements function calls using a stack and `GOTO`. You can find online tutorials on how recursion is implemented via stack in your favorite programming language, whether it's Python , JavaScript, or Lisp/Scheme.

7.3.1 The "Best of both worlds" paradigm

The equivalence between Turing Machines and RAM machines allows us to choose the most convenient language for the task at hand:

- When we want to *prove a theorem* about all programs/algorithms, we can use Turing machines (or NAND-TM) since they are simpler and easier to analyze. In particular, if we want to show that a certain function *can not* be computed, then we will use Turing machines.
- When we want to show that a function *can be computed* we can use RAM machines or NAND-RAM, because they are easier to program in and correspond more closely to high level programming languages we are used to. In fact, we will often describe NAND-RAM programs in an informal manner, trusting that the reader can fill in the details and translate the high level description to the precise program. (This is just like the way people typically use informal or "pseudocode" descriptions of algorithms, trusting that their audience will know to translate these descriptions to code if needed.)

Our usage of Turing Machines / NAND-TM and RAM Machines / NAND-RAM is very similar to the way people use in practice high and low level programming languages. When one wants to produce a device that executes programs, it is convenient to do so for very simple and "low level" programming language. When one wants to describe an algorithm, it is convenient to use as high level a formalism as possible.

Big Idea 7 Using equivalence results such as those between Turing and RAM machines, we can "have our cake and eat it too". We can use a simpler model such as Turing machines when we want to prove something *can't* be done, and use a feature-rich model such as RAM machines when we want to prove something *can* be done.



Figure 7.4: By having the two equivalent languages NAND-TM and NAND-RAM, we can "have our cake and eat it too", using NAND-TM when we want to prove that programs *can't* do something, and using NAND-RAM or other high level languages when we want to prove that programs *can* do something.

7.3.2 Let's talk about abstractions.

> "The programmer is in the unique position that ... he has to be able to think in terms of conceptual hierarchies that are much deeper than a single mind ever needed to face before.", Edsger Dijkstra, "On the cruelty of really teaching computing science", 1988.

At some point in any theory of computation course, the instructor and students need to have *the talk*. That is, we need to discuss the *level*

of abstraction in describing algorithms. In algorithms courses, one typically describes algorithms in English, assuming readers can "fill in the details" and would be able to convert such an algorithm into an implementation if needed. For example, we might describe the breadth first search algorithm to find if two vertices u, v are connected as follows:

Algorithm 7.3 — Breadth First Search.
Input: Graph G, vertices u, v
Operation:

1. Put u in queue Q.
2. While Q is not empty:
   - Remove the top vertex w from Q
   - If $w = v$ then declare "connected" and exit.
   - Mark w and add all unmarked neighbors of w to Q.
3. Declare "unconnected".

We call such a description a *high level description*.

If we wanted to give more details on how to implement breadth first search in a programming language such as Python or C (or NAND-RAM / NAND-TM for that matter), we would describe how we implement the queue data structure using an array, and similarly how we would use arrays to implement the marking. We call such an "intermediate level" description an *implementation level* or *pseudocode* description. Finally, if we want to describe the implementation precisely, we would give the full code of the program (or another fully precise representation, such as in the form of a list of tuples). We call this a *formal* or *low level* description.

While we started off by describing NAND-CIRC, NAND-TM, and NAND-RAM programs at the full formal level, as we progress in this book we will move to implementation and high level description. After all, our goal is not to use these models for actual computation, but rather to analyze the general phenomenon of computation. That said, if you don't understand how the high level description translates to an actual implementation, going "down to the metal" is often an excellent exercise. (One of the most important skills for a computer scientist is the ability to move up and down hierarchies of abstractions.)

A similar distinction applies to the notion of *representation* of objects as strings. Sometimes, to be precise, we give a *low level specification* of exactly how an object maps into a binary string. For example, we might describe an encoding of n vertex graphs as length n^2 binary

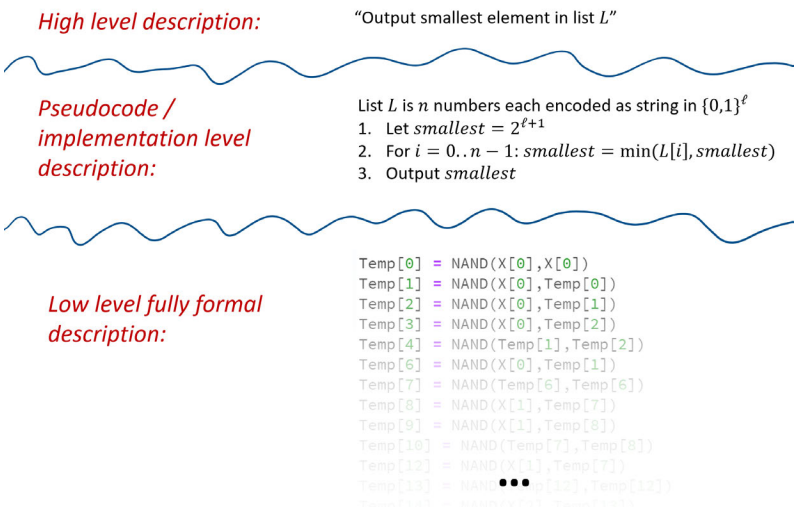


Figure 7.5: We can describe an algorithm at different levels of granularity/detail and precision. At the highest level we just write the idea in words, omitting all details on representation and implementation. In the intermediate level (also known as *implementation* or *pseudocode*) we give enough details of the implementation that would allow someone to derive it, though we still fall short of providing the full code. The lowest level is where the actual code or mathematical description is fully spelled out. These different levels of detail all have their uses, and moving between them is one of the most important skills for a computer scientist.

strings, by saying that we map a graph G over the vertices $[n]$ to a string $x \in \{0,1\}^{n^2}$ such that the $n \cdot i + j$-th coordinate of x is 1 if and only if the edge $\overrightarrow{i\ j}$ is present in G. We can also use an *intermediate* or *implementation level* description, by simply saying that we represent a graph using the adjacency matrix representation.

Finally, because we are translating between the various representations of graphs (and objects in general) can be done via a NAND-RAM (and hence a NAND-TM) program, when talking in a high level we also suppress discussion of representation altogether. For example, the fact that graph connectivity is a computable function is true regardless of whether we represent graphs as adjacency lists, adjacency matrices, list of edge-pairs, and so on and so forth. Hence, in cases where the precise representation doesn't make a difference, we would often talk about our algorithms as taking as input an object O (that can be a graph, a vector, a program, etc.) without specifying how O is encoded as a string.

7.3.3 Definition of "Algorithm"

Up until now we have use the term "algorithm" informally. However, Turing Machines and the range of equivalent models give a way to precisely and formally define algorithms. Hence whenever we refer to an *algorithm* in this book, we will mean that it is an instance of one of the Turing equivalent models, such as Turing machines, NAND-TM, RAM machines, etc. Because of the equivalence of all these models, in many contexts, it will not matter which of these we use.

7.4 LAMBDA CALCULUS AND FUNCTIONAL PROGRAMMING LANGUAGES

The λ calculus is another way to define computable functions. It was proposed by Alonzo Church in the 1930's around the same time as Alan Turing's proposal of the Turing Machine. Interestingly, while Turing Machines are not used for practical computation, the λ calculus has inspired functional programming languages such as LISP, ML and Haskell, and indirectly the development of many other programming languages as well. In this section we will present the λ calculus and show that its power is equivalent to NAND-TM programs (and hence also to Turing machines). Our Github rpository contains a Jupyter notebook with a Python implementation of the λ calculus that you can experiment with to get a better feel for this topic.

The λ operator. At the core of the λ calculus is a way to define "anonymous" functions. For example, instead of defining the squaring function as

$$square(x) = x \times x \tag{7.2}$$

we can write it as

$$\lambda x.x \times x \tag{7.3}$$

and so $(\lambda x.x \times x)(7) = 49$. That is, you can think of $\lambda x.exp(x)$, where exp is some expression as a way of specifying the anonymous function $x \mapsto exp(x)$.[5] Clearly, the name of the argument to a function doesn't matter, and so $\lambda y.y \times y$ is the same as $\lambda x.x \times x$, as both correspond to the squaring function.

[5] Anonymous functions, using either $\lambda x.f(x)$, $x \mapsto f(x)$ or other closely related notation, appear in many programming languages. For example, in *Python* we can define the squaring function using `lambda x: x*x` while in *JavaScript* we can use `x => x*x` or `(x) => x*x`. In *Scheme* we would define it as `(lambda (x) (* x x))`.



Remark 7.4 — Dropping parenthesis. To reduce notational clutter, when writing λ calculus expression we often drop the parenthesis for function evaluation. Hence instead of writing $f(x)$ for the result of applying the function f to the input x, we can also write this as simply $f\ x$. Therefore we can write $(\lambda x.x \times x)7 = 49$. In this chapter, we will use both the $f(x)$ and $f\ \ x$ notations for function application. Function evaluations are associative and bind from left to right, and hence $f\ g\ h$ is the same as $(fg)h$.

7.4.1 Applying functions to functions

A key feature of the λ calculus is that we can use functions as arguments to other functions. For example, can you guess what number is the following expression equal to?

$$(((\lambda f.(\lambda y.(f\ (f\ y))))(\lambda x.x \times x))\ 3) \tag{7.4}$$

P

The expression (7.4) might seem daunting, but before you look at the solution below, try to break it apart to its components, and evaluate each component at a time. Working out this example would go a long way toward understanding the λ calculus.

Let's evaluate (7.4) one step at a time. As nice as it is for the λ calculus to allow anonymous functions, adding names can be very helpful for understanding complicated expressions. So, let us write $F = \lambda f.(\lambda y.(f(fy)))$ and $g = \lambda x.x \times x$.

Therefore (7.4) becomes

$$((F\ g)\ 3)\ . \tag{7.5}$$

On input a function f, F outputs the function $\lambda y.(f(f\ \ y))$, or in other words Ff is the function $y \mapsto f(f(y))$. Our function g is simply $g(x) = x^2$ and so (Fg) is the function that maps y to $(y^2)^2$ or in other words to y^4. Hence $((Fg)3) = 3^4 = 81$.

■ **Example 7.5 — Simplifying a λ expression.** Here is another example of a λ expression:

$$((\lambda x.(\lambda y.x))\ 2)\ 9)\ . \tag{7.6}$$

Let us denote $(\lambda y.x)$ by F. Then (7.6) has the form

$$((\lambda x.F)\ 2)\ 9) \tag{7.7}$$

Now $(\lambda x.F)2$ is equal to $F[x \to 2]$. Since F is $\lambda y.x$ this means that $(\lambda x.F)2$ is the function $\lambda y.2$ that ignores its input and outputs 2 no matter what it is equal to. Hence (7.6) is equivalent to $(\lambda y.2)9$ which is the result of applying the function $y \mapsto 2$ on the input 9, which is simply the number 2.

7.4.2 Obtaining multi-argument functions via Currying

As we've seen, in a λ expression of the form $\lambda x.e$, the expression e can itself involve the λ operator. Thus for example the function

$$\lambda x.(\lambda y.x + y) \tag{7.8}$$

maps x to the function $y \mapsto x + y$.

In particular, if we invoke the function (7.8) on a, and then invoke the result of this invocation on b, we will get the value $a+b$. We can see that the one-argument function (7.8) corresponding to $a \mapsto (b \mapsto a+b)$ can also be thought of as the two-argument function $(a, b) \mapsto a + b$. In general, we will use the λ expression $\lambda x.(\lambda y.f(x, y))$ to simulate the effect of a two argument function $(x, y) \mapsto f(x, y)$. This technique is known as Currying. We will use the shorthand $\lambda x, y.e$ for $\lambda x.(\lambda y.e)$. If $f = \lambda x.(\lambda y.e)$ then $(fa)b)$ corresponds to applying fa and then invoking the resulting function on b, obtaining the result of replacing in e the occurrences of x with a and occurrences of b with y. By our rules of associativity, this is the same as (fab) which we'll sometimes also write as $f(a, b)$.

7.4.3 Formal description of the λ calculus.

We now provide a formal description of the λ calculus. We start with "basic expressions" that contain a single variable such as x or y and build more complex expressions using the following two rules:

- **Application:** If e and e' are λ expressions, then the λ expression $(e\ e')$ corresponds to applying the function described by e to the input e'.

- **Abstraction:** If e is a λ expression and x is a variable, then the λ expression $\lambda x.(e)$ corresponds to the function that on any input z returns the expression $e[x \to z]$ replacing all (free) occurrences of x in e.[6]

Formally λ expressions are defined as follows:

Definition 7.6 — λ expression.. A *λ expression* is either a single variable identifier or an expression that is built from other expressions using the *application* and *abstraction* operations.

Definition 7.6 is a *recursive definition* since we defined the concept of λ expressions in terms of itself. Specifically, a λ expression can either be the "base case" of the form *foo* for a variable identifier *foo*, or it can be of the form (ee') or $\lambda bar.(e)$ where e and e' are other λ expressions and *bar* is a variable identifier. Such a recursive definition might seem confusing at first, but in fact you have known recursive definitions since you were an elementary school student. Consider how we define an *arithmetic expression*: it is an expression that is either just a number, or has one of the forms $(e + e')$, $(e - e')$, $(e \times e')$, or $(e \div e')$, where e and e' are other arithmetic expressions.

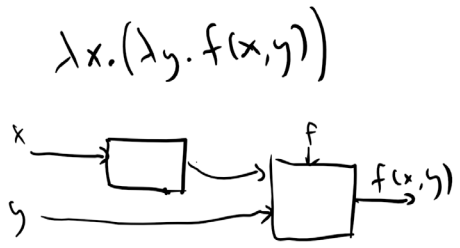


Figure 7.6: In the "currying" transformation, we can create the effect of a two parameter function $f(x, y)$ with the λ expression $\lambda x.(\lambda y.f(x, y))$ which on input x outputs a one-parameter function f_x that has x "hardwired" into it and such that $f_x(y) = f(x, y)$. This can be illustrated by a circuit diagram; see Chelsea Voss's site.

[6] Strictly speaking we should replace only the *free* and not the ones that are *bound* by some other λ operator. For example, if we have the λ expression $\lambda x.(\lambda x.x + 1)(x)$ and invoke it on the number 7 then we get $(\lambda x.x + 1)(7) = 8$ and not the nonsensical expression $(\lambda 7.7 + 1)(7)$. To avoid such annoyances, we can adopt the convention that every instance of $\lambda var.e$ uses a unique variable identifier *var*. See Section 1.5.8 for more discussion on bound and free variables.

R

Remark 7.7 — Precedence and parenthesis.. We will use the following rules to allow us to drop some parenthesis. Function application associates from left to right, and so fgh is the same as $(fg)h$. Function application has a higher precedence than the λ operator, and so $\lambda x.fgx$ is the same as $\lambda x.((fg)x)$.

This is similar to how we use the precedence rules in arithmetic operations to allow us to use fewer parenthesis and so write the expression $(7 \times 3) + 2$ as $7 \times 3 + 2$.

As mentioned in **??**, we also use the shorthand $\lambda x, y.e$ for $\lambda x.(\lambda y.e)$ and the shorthand $f(x, y)$ for $(f\ x)\ y$. This plays nicely with the "Currying" transformation of simulating multi-input functions using λ expressions.

As we have seen in Eq. (7.6), the rule that $(\lambda x.exp)exp'$ is equivalent to $exp[x \rightarrow exp']$ enables us to modify λ expressions and obtain simpler *equivalent form* for them. Another rule that we can use is that the parameter does not matter and hence for example $\lambda y.y$ is the same as $\lambda z.z$. Together these rules define the notion of *equivalence* of λ expressions:

Definition 7.8 — Equivalence of λ expressions. Two λ expressions are *equivalent* if they can be made into the same expression by repeated applications of the following rules: [7]

1. **Evaluation (aka β reduction):** The expression $(\lambda x.exp)exp'$ is equivalent to $exp[x \rightarrow exp']$.
2. **Variable renaming (aka α conversion):** The expression $\lambda x.exp$ is equivalent to $\lambda y.exp[x \rightarrow y]$.

[7] These two rules are commonly known as "β reduction" and "α conversion" in the literature on the λ calculus.

If exp is a λ expression of the form $\lambda x.exp'$ then it naturally corresponds to the function that maps any input z to $exp'[x \rightarrow z]$. Hence the λ calculus naturally implies a computational model. Since in the λ calculus the inputs can themselves be functions, we need to decide how to evaluate an expression such as

$$(\lambda x.f)(\lambda y.gz)\ . \tag{7.9}$$

There are two natural conventions for this:

- *Call by name*: We evaluate (7.9) by first plugging in the righthand expression $(\lambda y.gz)$ as input to the lefthand side function, obtaining $f[x \rightarrow (\lambda y.gz)]$ and then continue from there.

- *Call by value*: We evaluate (7.9) by first evaluating the righthand side and obtaining $h = g[y \rightarrow z]$, and then plugging this into the lefthandside to obtain $f[x \rightarrow h]$.

Because the λ calculus has only *pure* functions, that do not have "side effects", in many cases the order does not matter. In fact, it can be shown that if we obtain an definite irreducible expression (for example, a number) in both strategies, then it will be the same one. However, there could be situations where "call by value" goes into an infinite loop while "call by name" does not. Hence we will use "call by name" henceforth.[8]

[8] "Call by value" is also sometimes known as *eager evaluation*, since it means we always evaluate parameters to functions before they are executed, while "call by name" is also known as *lazy evaluation*, since it means that we hold off on evaluating parameters until we are sure we need them. Most programming languages use eager evaluation, though there are some exceptions (notably Haskell). For programming languages that involve non pure functions, call by value has the advantage that it is much easier to understand when the side effects will take place in the program.

7.4.4 Functions as first class objects

The key property of the λ calculus (and functional languages in general) is that functions are "first-class citizens" in the sense that they can be used as parameters and return values of other functions. Thus, we can invoke one λ expression on another. For example if *DOUBLE* is the λ expression $\lambda f.(\lambda x.f(fx))$, then for every function f, *DOUBLE* f corresponds to the function that invokes f twice on x (i.e., first computes fx and then invokes f on the result). In particular, if $f = \lambda y.(y + 1)$ then *DOUBLE* $f = \lambda x.(x + 2)$.



Remark 7.9 — (Lack of) types. Unlike most programming languages, the pure λ-calculus doesn't have the notion of *types*. Every object in the λ calculus can also be thought of as a λ expression and hence as a function that takes one input and returns one output. All functions take one input and return one output, and if you feed a function an input of a form it didn't expect, it still evaluates the λ expression via "search and replace", replacing all instances of its parameter with copies of the input expression you fed it.

7.5 THE "ENHANCED" Λ CALCULUS

We now discuss the λ calculus as a computational model. We will start by describing an "enhanced" version of the λ calculus that contains some "superfluous features" but is easier to wrap your head around. We will first show how the enhanced λ calculus is equivalent to Turing machines in computational power. Then we will show how all the features of "enhanced λ calculus" can be implemented as "syntactic sugar" and even the "pure" (i.e., non enhanced) λ calculus is equivalent in power to Turing machines (and hence also to RAM machines and all other Turing-equivalent models).

The *enhanced λ calculus* includes the following set of objects and operations:

- **Boolean constants and IF function:** The enhanced λ calculus has the constants 0 and 1 and the *IF* function such that for every $cond \in \{0,1\}$ and λ expressions a, b, $IF cond\ a\ b$ outputs a if $cond = 1$ and outputs b if $cond = 0$.[9] Using *IF* and the constants $0, 1$ we can also compute logical operations such as $AND, OR, NOT, NAND$. For example, $NOT = \lambda a.IFa01$ and $AND = \lambda a, b.IFab0$.

- **Pairs:** We have the function *PAIR* such that $PAIR\ x\ y$ returns the pair (x, y) that holds x and y. We also have the functions *HEAD* and *TAIL* to extract the first and second member of a pair respectively. Hence, $HEAD(PAIRab) = a$ and $TAIL(PAIRab) = b$.

- **Lists and strings:** Using *PAIR* we can also construct *lists*. The idea is that $PAIR\ a\ L$ corresponds to the list obtained by adding the element a to the beginning of a list L. By repeating this operation, we can construct lists of any length. Specifically, we will have a special λ expression *NIL* that corresponds to the *empty list*, which we also denote by $\langle\rangle$. If c is some λ expression, then $PAIR\ c\ NIL$ corresponds to the single-element list $\langle c\rangle$. Now for every λ expressions b, c, the expression $PAIR\ b\ (PAIRCNIL)$ corresponds to the two-element list $\langle b, c\rangle$. Similarly the expression $PAIR\ a(PAIR\ b\ (PAIR\ c\ NIL))$ corresponds to the list $\langle a, b, c\rangle$ and so on and so forth.[10] The function *ISEMPTY* returns 1 on *NIL* and returns 0 on every other list. A *string* is simply a list of bits.

- **List operations:** The enhanced λ calculus also contains the *list-processing functions MAP*, *REDUCE*, and *FILTER*. Given a list $L = \langle x_0, \ldots, x_{n-1}\rangle$ and a function f, $MAP\ L\ f$ applies f on every member of the list to obtain the new list $L' = \langle f(x_0), \ldots, f(x_{n-1})\rangle$. Given a list L as above and a function f whose output is either 0 or 1, $FILTER\ L\ f$ returns the list $\langle x_i\rangle_{fx_i=1}$ containing all the elements of L for which f outputs 1. The function *REDUCE* applies a "combining" operation to a list. For example, $REDUCE\ L\ +\ 0$ will return the sum of all the elements in the list L. The sum of a list is defined recursively as follows: the sum of the empty list is 0, and the sum of a non-empty list L is obtained by recursively summing $TAIL\ L$ (i.e., all elements of L except the first) and adding the result to $HEAD\ L$ (which is the first element of L). More generally, *REDUCE* takes a list L, an operation f (which we think of as taking two arguments) and a λ expression z (which we think of as the "neutral element" for the operation f, such as 0 for addition and 1 for multiplication). The output is defined via

[9] We use *currying* to implement multi-input functions, and so *IF* is the function $cond \mapsto f_{cond}$ where f_1 is the function $x \mapsto (y \mapsto x)$ and f_0 is the function $x \mapsto (y \mapsto y)$. Can you see why? If not, then working this out is a great exercise.

[10] Note that if L is a list, then $HEADL$ is its first element, but $TAILL$ is not the last element but rather all the elements except the first. The second element of a list L can be extracted using $HEAD(TAILL)$. Once again, working out why this is the case is a great exercise.

$$REDUCE\ L\ f\ z = \begin{cases} z & L = NIL \\ f\ (HEADL)\ (REDUCE\ (TAILL)\ f\ z) & \text{otherwise} \end{cases} . \tag{7.10}$$

See Fig. 7.8 for an illustration of the three list-processing operations.

- **Recursion:** Finally, we want to be able to execute *recursive functions*. Since in λ calculus functions are *anonymous*, we can't write a definition of the form $f(x) = blah$ where $blah$ includes calls to f. Instead we use functions f that take an additional input me as a parameter. The operator *RECURSE* will take such a function f as input and return a "recursive version" of f where all the calls to me are replaced by recursive calls to this function. That is, if we have a function F taking two parameters me and x, then *RECURSE* F will be the function f taking one parameter x such that $f(x) = F(f, x)$ for every x.

Solved Exercise 7.1 — Compute NAND using λ calculus. Give a λ expression N such that $N\ x\ y = NAND(x, y)$ for every $x, y \in \{0, 1\}$.

■

Solution:

This can be done in a similar way to how we computed XOR_2. The *NAND* of x, y is equal to 1 unless $x = y = 1$. Hence we can write

$$N = \lambda x, y.IF(x, IF(y, 0, 1), 1) \tag{7.11}$$

■

■ **Example 7.10 — Computing XOR.** Let us see how we can compute the XOR of a list in the enhanced λ calculus. First, we note that we can compute XOR of two bits as follows:

$$NOT = \lambda a.IF(a, 0, 1) \tag{7.12}$$

and

$$XOR_2 = \lambda a, b.IF(b, NOT(a), a) \tag{7.13}$$

(We are using here a bit of syntactic sugar to describe the functions. To obtain the λ expression for XOR we will simply replace the expression (7.12) in (7.13).)

Now recursively we can define the XOR of a list as follows:

$$XOR(L) = \begin{cases} 0 & L \text{ is empty} \\ XOR_2(HEAD(L), XOR(TAIL(L))) & \text{otherwise} \end{cases} \tag{7.14}$$

This means that *XOR* is equal to

$$RECURSE\ (\lambda me, L.IF(ISEMPTY(L), 0, XOR_2(HEAD\ L\ ,\ me(TAIL\ L)))) \ . \tag{7.15}$$

That is, *XOR* is obtained by applying the *RECURSE* operator to the function that on inputs me, L, returns 0 if $ISEMPTY(L)$ and otherwise returns XOR_2 applied to $HEAD(L)$ and to $me(TAIL(L))$.

We could have also computed *XOR* using the *REDUCE* operation, we leave working this out as an exercise to the reader.

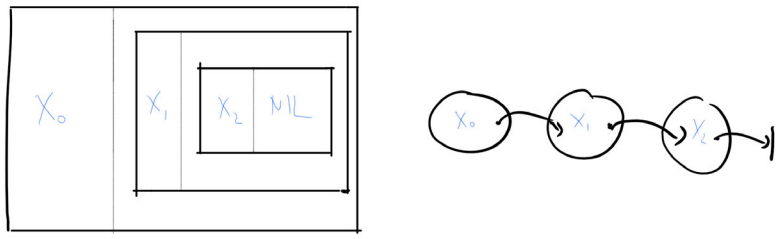


Figure 7.7: A list $\langle x_0, x_1, x_2 \rangle$ in the λ calculus is constructed from the tail up, building the pair $\langle x_2, NIL \rangle$, then the pair $\langle x_1, \langle x_2, NIL \rangle \rangle$ and finally the pair $\langle x_0, \langle x_1, \langle x_2, NIL \rangle \rangle \rangle$. That is, a list is a pair where the first element of the pair is the first element of the list and the second element is the rest of the list. The figure on the left renders this "pairs inside pairs" construction, though it is often easier to think of a list as a "chain", as in the figure on the right, where the second element of each pair is thought of as a *link*, *pointer* or *reference* to the remainder of the list.

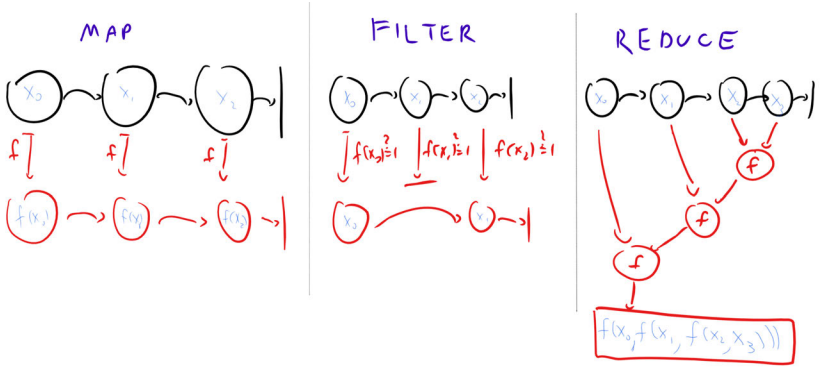


Figure 7.8: Illustration of the *MAP*, *FILTER* and *REDUCE* operations.

7.5.1 Enhanced λ expressions

An *enhanced λ expression* is obtained by composing the objects above with the *application* and *abstraction* rules. We can now define the no-

tion of computing a function using the λ calculus. We will define the *simplification* of a λ expression as the following recursive process:

1. (*Evaluation / β reduction.*) If the expression has the form $(exp_L exp_R)$ then replace the expression with $exp'_L[x \to exp_R]$.

2. (*Renaming / α conversion.*) When we cannot simplify any further, rename the variables so that the first bound variable in the expression is v_0, the second one is v_1, and so on and so forth.



Please make sure you understand why this recursive procedure simply corresponds to the "call by name" evaluation strategy.

The result of simplifying a λ expression is an equivalent expression, and hence if two expressions have the same simplification then they are equivalent.

Definition 7.11 — Computing a function via λ calculus. Let $F : \{0,1\}^* \to \{0,1\}^*$ be a function and exp a λ expression. For every $x \in \{0,1\}^n$, we denote by $LIST(x)$ the λ list $PAIR(x_0, PAIR(x_1, PAIR(\cdots PAIR(x_{n-1} NIL))))$ that corresponds to x.

We say that exp *computes* F if for every $x \in \{0,1\}^*$, the expressions $(expLIST(x))$ and $LIST(F(x))$ are equivalent, and moreover they have the same simplification.

7.5.2 Enhanced λ calculus is Turing-complete

The basic operations of the enhanced λ calculus more or less amount to the Lisp or Scheme programming languages.[11] Given that, it is perhaps not surprising that the enhanced λ-calculus is equivalent to Turing machines:

[11] In Lisp, the *PAIR*, *HEAD* and *TAIL* functions are traditionally called `cons`, `car` and `cdr`.

Theorem 7.12 — Lambda calculus and NAND-TM. For every function $F : \{0,1\}^* \to \{0,1\}^*$, F is computable in the enhanced λ calculus if and only if it is computable by a Turing machine.

Proof Idea:

To prove the theorem, we need to show that **(1)** if F is computable by a λ calculus expression then it is computable by a Turing machine, and **(2)** if F is computable by a Turing machine, then it is computable by an enhanced λ calculus expression.

Showing **(1)** is fairly straightforward. Applying the simplification rules to a λ expression basically amounts to "search and replace"

which we can implement easily in, say, NAND-RAM, or for that matter Python (both of which are equivalent to Turing machines in power). Showing **(2)** essentially amounts to simulating a Turing machine (or writing a NAND-TM interpreter) in a functional programming language such as LISP or Scheme. Showing how this can be done is a good exercise in mastering some functional programming techniques that are useful in their own right.

⋆

Proof of Theorem 7.12. We only sketch the proof. The "if" direction is simple. As mentioned above, evaluating λ expressions basically amounts to "search and replace". It is also a fairly straightforward programming exercise to implement all the above basic operations in an imperative language such as Python or C, and using the same ideas we can do so in NAND-RAM as well, which we can then transform to a NAND-TM program.

For the "only if" direction, we need to simulate a Turing machine, or equivalently a NAND-TM program, using a λ expression. First, by Solved Exercise 7.1 we can compute the *NAND* function, and hence *every* finite function, using the λ calculus. Thus proving the theory boils down to simulating the *arrays* of NAND-TM using the *lists* of the enhanced λ calculus.

We will encode each array `A` of NAND-TM program by a list L of the NAND-CIRC program. We encode the index variable `i` by a special list I that has 1 in the location corresponding to the value of `i` and 0's everywhere else. To simulate moving `i` to the left, we need to remove the first item from the list, while to simulate moving `i` to the right, we add a zero to the head of list.[12]

To extract the `i`-th bit of the array corresponding to L, we need to compute the following function get that on input a pair of lists I and L of bits of the same length n, $get(I, L)$ outputs 1 if and only if there is some $j \in [n]$ such that the j-th element of both I and L is equal to 1. This turns out to be not so hard. The key is to implement the function zip that on input a pair of lists I and L of the same length n, outputs a *list of n pairs* M such that the j-th element of M (which we denote by M_j) is the pair (I_j, L_j). Thus zip "zips together" these two lists of elements into a single list of pairs.[13] It is a good exercise to give a recursive implementation of zip, and so can implement it using the *RECURSE* operator. Once we have zip, we can implement get by applying an appropriate *REDUCE* on the list $zip(I, L)$.

Setting the list L at the i-th location to a certain value requires computing the function $set(I, L, v)$ that outputs a list L' such that $L'_j = L_j$ if $I_j = 0$ and $L'_j = v$ otherwise. The function set can be

[12] In fact, it will be convenient for us to make sure all lists are of the same length, and so at the end of each step we will add a sufficient number of zeroes to the end of each list. This can be done with a simple `REDUCE` operation.

[13] The name zip is a common name for this operation, for example in Python. It should not be be confused with the `zip` compression file format.

implemented by applying *MAP* with an appropriate operator to the list $zip(I, L)$.

We omit the full details of implementing set and get, but the bottom line is that for every NAND-TM program P, we can obtain a λ expression $NEXT_P$ such that, if we let $\sigma = (loop, foo, bar, \ldots, I, X, X_n onblank, Y, Y_n onblank, Baz, Blah, \ldots)$ be the set of Boolean values and lists that encode the current state of P (with a list for each array and for the index variable `i`), then $NEXT_P\sigma$ will encode the state after performing one iteration of P.

Now we can use the following "pseudocode" to simulate the program P. The function SIM_P will obtain an encoding σ_0 of the initial state of P, and output the encoding σ^* of the state of P after it halts. It will be computed as follows:

Algorithm 7.13 — SIM_P: Simulate NAND-TM program in λ calculus.
Input: σ: encoding of the state of a NAND-TM program
Operation:

1. Let $\sigma' = NEXT_P\sigma$.
2. If $halt(\sigma') = 1$ then return σ', where $halt(\sigma')$ is equal to 1 if the configuration σ' is one in which the program will halt when executing the `MODANDJUMP` operation.
3. Otherwise return $SIM_P(\sigma')$.

We can write this algorithm as the λ expression

$$RECURSE\ (\lambda m, \sigma.IF(loop(NEXT_P\sigma)\ ,\ m(NEXT_P\sigma)\ ,\ NEXT_P\sigma)) \tag{7.16}$$

Given SIM_P, we can compute the function computed by P by writing expressions for encoding the input as the initial state, and decoding the output from the final state. We omit the details, though this is fairly straightforward.[14]

■

[14] For example, if `X` is a list representing the input, then we can obtain a list `X_nonblank` of 1's of the same length by simply writing `X_nonblank` = MAP(`X`, $\lambda x.1$).

7.6 THE PURE Λ CALCULUS

While the collection of "basic" functions we allowed for λ calculus is smaller than what's provided by most Lisp dialects, coming from NAND-TM it still seems a little "bloated". Can we make do with less? In other words, can we find a subset of these basic operations that can implement the rest?

P

This is a good point to pause and think how you would implement these operations yourself. For example, start by thinking how you could implement *MAP* using *REDUCE*, and then *REDUCE* using *RECURSE* combined with $0, 1, IF, PAIR, HEAD, TAIL, NIL, ISEMPTY$ together with the λ operations.

Now you can think how you could implement *PAIR*, *HEAD* and *TAIL* based on $0, 1, IF$. The idea is that we can represent a *pair* as *function*.

It turns out that there is in fact a proper subset of these basic operations that can be used to implement the rest. That subset is the empty set. That is, we can implement *all* the operations above using the λ formalism only, even without using 0's and 1's. It's λ's all the way down! The idea is that we encode 0 and 1 themselves as λ expressions, and build things up from there. This is known as Church encoding, as it was originated by Church in his effort to show that the λ calculus can be a basis for all computation.

Theorem 7.14 — Enhanced λ calculus equivalent to pure λ calculus.. There are λ expressions that implement the functions 0,1,*IF*,*PAIR*, *HEAD*, *TAIL*, *NIL*, *ISEMPTY*, *MAP*, *REDUCE*, and *RECURSE*.

We will not write the full formal proof of Theorem 7.14 but outline the ideas involved in it:

- We define 0 to be the function that on two inputs x, y outputs y, and 1 to be the function that on two inputs x, y outputs x. Of course we use Currying to achieve the effect of two-input functions and hence $0 = \lambda x.\lambda y.y$ and $1 = \lambda x.\lambda y.x$.[15]

- The above implementation makes the *IF* function trivial: $IF(cond, a, b)$ is simply $cond\ a\ b$ since $0ab = b$ and $1ab = a$. We can write $IF = \lambda x.x$ to achieve $IF(cond, a, b) = (((IFcond)a)b) = cond\ a\ b$.

- To encode a pair (x, y) we will produce a function $f_{x,y}$ that has x and y "in its belly" and satisfies $f_{x,y}g = gxy$ for every function g. That is, we write $PAIR = \lambda x, y.\lambda g.gxy$. Note that now we can extract the first element of a pair p by writing $p1$ and the second element by writing $p0$, and so $HEAD = \lambda p.p1$ and $TAIL = \lambda p.p0$.

- We define *NIL* to be the function that ignores its input and always outputs 1. That is, $NIL = \lambda x.1$. The *ISEMPTY* function checks, given an input p, whether we get 1 if we apply p to the function

[15] This representation scheme is the common convention for representing `false` and `true` but there are many other alternative representations for 0 and 1 that would have worked just as well.

$z = \lambda x, y.0$ that ignores both its inputs and always outputs 0. For every valid pair of the form $p = PAIRxy$, $pz = pxy = 0$ while $NILz = 1$. Formally, $ISEMPTY = \lambda p.p(\lambda x, y.0)$.

R

Remark 7.15 — Church numerals (optional). There is nothing special about Boolean values. You can use similar tricks to implement *natural numbers* using λ terms. The standard way to do so is to represent the number n by the function $ITER_n$ that on input a function f outputs the function $x \mapsto f(f(\cdots f(x)))$ (n times). That is, we represent the natural number 1 as $\lambda f.f$, the number 2 as $\lambda f.(\lambda x.f(fx))$, the number 3 as $\lambda f.(\lambda x.f(f(fx)))$, and so on and so forth. (Note that this is not the same representation we used for 1 in the Boolean context: this is fine; we already know that the same object can be represented in more than one way.) The number 0 is represented by the function that maps any function f to the identity function $\lambda x.x$. (That is, $0 = \lambda f.(\lambda x.x)$.)

In this representation, we can compute $PLUS(n, m)$ as $\lambda f.\lambda x.(nf)((mf)x)$ and $TIMES(n, m)$ as $\lambda f.n(mf)$. Subtraction and division are trickier, but can be achieved using recursion. (Working this out is a great exercise.)

7.6.1 List processing

Now we come to a bigger hurdle, which is how to implement *MAP*, *FILTER*, and *REDUCE* in the λ calculus. It turns out that we can build *MAP* and *FILTER* from *REDUCE*. For example $MAP(L, f)$ is the same as $REDUCE(L, g)$ where g is the operation that on input x and y, outputs $PAIR(f(x), NIL)$ if y is NIL and otherwise outputs $PAIR(f(x), y)$. (I leave checking this as a (recommended!) exercise for you, the reader.) So, it all boils down to implementing *REDUCE*. We can define $REDUCE(L, g)$ recursively, by setting $REDUCE(NIL, g) = NIL$ and stipulating that given a non-empty list L, which we can think of as a pair $(head, rest)$, $REDUCE(L, g) = g(head, REDUCE(rest, g)))$. Thus, we might try to write a λ expression for *REDUCE* as follows

$$REDUCE = \lambda L, g.IF(ISEMPTY(L), NIL, gHEAD(L)REDUCE(TAIL(L), g)) \; . \tag{7.17}$$

The only fly in this ointment is that the λ calculus does not have the notion of recursion, and so this is an invalid definition. But of course we can use our *RECURSE* operator to solve this problem. We will replace the recursive call to "*REDUCE*" with a call to a function me

that is given as an extra argument, and then apply *RECURSE* to this. Thus $REDUCE = RECURSE\ myREDUCE$ where

$$myREDUCE = \lambda me, L, g.IF(ISEMPTY(L), NIL, gHEAD(L)me(TAIL(L), g)) \ . \tag{7.18}$$

So everything boils down to implementing the *RECURSE* operator, which we now deal with.

7.6.2 The Y combinator, or recursion without recursion

How can we implement recursion without recursion? We will illustrate this using a simple example - the *XOR* function. As shown in Example 7.10, we can write the *XOR* function of a list recursively as follows:

$$XOR(L) = \begin{cases} 0 & L \text{ is empty} \\ XOR_2(HEAD(L), XOR(TAIL(L))) & \text{otherwise} \end{cases} \tag{7.19}$$

where $XOR_2 : \{0,1\}^2 \to \{0,1\}$ is the XOR on two bits. In *Python* we would write this as

```
def xor2(a,b): return 1-b if a else b
def head(L): return L[0]
def tail(L): return L[1:]

def xor(L): return xor2(head(L),xor(tail(L))) if L else 0

print(xor([0,1,1,0,0,1]))
# 1
```

Now, how could we eliminate this recursive call? The main idea is that since functions can take other functions as input, it is perfectly legal in Python (and the λ calculus of course) to give a function *itself* as input. So, our idea is to try to come up with a *non recursive* function `tempxor` that takes *two inputs*: a function and a list, and such that `tempxor(tempxor,L)` will output the XOR of `L`!



At this point you might want to stop and try to implement this on your own in Python or any other programming language of your choice (as long as it allows functions as inputs).

Our first attempt might be to simply use the idea of replacing the recursive call by `me`. Let's define this function as `myxor`

```
def myxor(me,L): return xor2(head(L),me(tail(L))) if L
 ↪  else 0
```

Let's test this out:

```
myxor(myxor,[1,0,1])
```

If you do this, you will get the following complaint from the interpreter: `TypeError: myxor() missing 1 required positional argument`. The problem is that `myxor` expects *two* inputs- a function and a list- while in the call to `me` we only provided a list. To correct this, we modify the call to also provide the function itself:

```
def tempxor(me,L): return xor2(head(L),me(me,tail(L))) if
 ↪  L else 0
```

Note the call `me(me,..)` in the definition of `tempxor`: given a function `me` as input, `tempxor` will actually call the function `me` with itself as the first input. If we test this out now, we see that we actually get the right result!

```
tempxor(tempxor,[1,0,1])
# 0
tempxor(tempxor,[1,0,1,1])
# 1
```

and so we can define `xor(L)` as simply `return tempxor(tempxor,L)`.

The approach above is not specific to XOR. Given a recursive function `f` that takes an input `x`, we can obtain a non recursive version as follows:

1. Create the function `myf` that takes a pair of inputs `me` and `x`, and replaces recursive calls to `f` with calls to `me`.
2. Create the function `tempf` that converts calls in `myf` of the form `me(x)` to calls of the form `me(me,x)`.
3. The function `f(x)` will be defined as `tempf(tempf,x)`

Here is the way we implement the `RECURSE` operator in Python. It will take a function `myf` as above, and replace it with a function g such that `g(x)=myf(g,x)` for every `x`.

```
def RECURSE(myf):
    def tempf(me,x): return myf(lambda y: me(me,y),x)

    return lambda x: tempf(tempf,x)
```

```
xor = RECURSE(myxor)

print(xor([0,1,1,0,0,1]))
# 1

print(xor([1,1,0,0,1,1,1,1]))
# 0
```

From Python to the λ calculus. In the λ calculus, a two input function g that takes a pair of inputs me, y is written as $\lambda me.(\lambda y.g)$. So the function $y \mapsto me(me, y)$ is simply written as $me\ \ me$ and similarly the function $x \mapsto tempf(tempf, x)$ is simply $tempf\ \ tempf$. (Can you see why?) Therefore in the λ calculus, the function `tempf` is `λ me. f(me me)` and similarly and the expression `lambda x: tempf(tempf,x)` is the same as `tempf tempf`. Therefore, if we denote the input of `RECURSE` by f, then the output is equal to FF where $F = \lambda m.f(mm)$, which means that

$$RECURSE = \lambda f.((\lambda m.f(m\ m))\ \ (\lambda m.f(m\ m))) \tag{7.20}$$

The online appendix contains an implementation of the λ calculus using Python. Here is an implementation of the recursive XOR function from that appendix:[16]

```
# XOR of two bits
XOR2 = λ(a,b)(IF(a,IF(b,_0,_1),b))

# Recursive XOR with recursive calls replaced by m
 ↪  parameter
myXOR = λ(m,l)(IF(ISEMPTY(l),_0,XOR2(HEAD(l),m(TAIL(l)))))

# Recurse operator (aka Y combinator)
RECURSE = λf((λm(f(m*m)))(λm(f(m*m))))

# XOR function
XOR = RECURSE(myXOR)

#TESTING:

XOR(PAIR(_1,NIL)) # List [1]
# equals 1

XOR(PAIR(_1,PAIR(_0,PAIR(_1,NIL)))) # List [1,0,1]
# equals 0
```

[16] Because of specific issues of Python syntax, in this implementation we use `f * g` for applying `f` to `g` rather than `fg`, and use `λx(exp)` rather than `λx.exp` for abstraction. We also use `_0` and `_1` for the λ terms for 0 and 1 so as not to confuse with the Python constants.



Remark 7.16 — The Y combinator. The *RECURSE* operator above is better known as the Y combinator.

It is one of a family of a *fixed point operators* that given a lambda expression F, find a *fixed point* f of F such that $f = Ff$. If you think about it, *XOR* is the fixed point of $myXOR$ above. *XOR* is the function such that for every x, if plug in *XOR* as the first argument of $myXOR$ then we get back *XOR*, or in other words $XOR = myXOR\ XOR$. Hence finding a *fixed point* for $myXOR$ is the same as applying *RECURSE* to it.

7.6.3 Infinite loops in the λ calculus

The fact that λ-expressions can simulate NAND-TM programs means that, like Turing machines and NAND-TM programs, the λ calculus can also enter into an infinite loop. For example, consider the λ expression

$$(\lambda x.xxx)(\lambda x.xxx) \tag{7.21}$$

If we try to evaluate it then the first step is to invoke the lefthand function on the righthand one and then obtain

$$(\lambda x.xxx)(\lambda x.xxx)(\lambda x.xxx) \tag{7.22}$$

To evaluate this, the next step would be to apply the second term on the third term,[17] which would result in

$$(\lambda x.xxx)(\lambda x.xxx)(\lambda x.xxx)(\lambda x.xxx) \tag{7.23}$$

We can see that continuing in this way we get longer and longer expressions, and this process never concludes.

[17] This assumes we use the "call by value" evaluation ordering which states that to evaluate a λ expression fg we first evaluate the righthand expression g and then invoke f on it. The "Call by name" or "lazy evaluation" ordering would first evaluate the lefthand expression f and then invoke it on g. In this case both strategies would result in an infinite loop. There are examples though when "call by name" would not enter an infinite loop while "call by value" would. The SML and OCaml programming languages use "call by value" while Haskell uses (a close variant of) "call by name".

7.7 MORE TURING-COMPLETE COMPUTATIONAL MODELS

There is a great variety of models that are computationally equivalent to Turing machines (and hence to NAND-TM/NAND-RAM program). We briefly mention a few examples.

7.7.1 Parallel algorithms and cloud computing

The models of computation we considered so far are inherently sequential, but these days much computation happens in parallel, whether using multi-core processors or in massively parallel distributed computation in data centers or over the Internet. Parallel computing is important in practice, but it does not really make much difference for the question of what can and can't be computed. After all, if a computation can be performed using m machines in t time, then it can be computed by a single machine in time mt.

7.7.2 Game of life, tiling and cellular automata

Many physical systems can be described as consisting of a large number of elementary components that interact with one another. One way to model such systems is using *cellular automata*. This is a system that consists of a large number (or even infinite) cells. Each cell only has a constant number of possible states. At each time step, a cell updates to a new state by applying some simple rule to the state of itself and its neighbors.

A canonical example of a cellular automaton is Conway's Game of Life. In this automata the cells are arranged in an infinite two dimensional grid. Each cell has only two states: "dead" (which we can encode as 0 and identify with $\emptyset$) or "alive" (which we can encode as 1). The next state of a cell depends on its previous state and the states of its 8 vertical, horizontal and diagonal neighbors. A dead cell becomes alive only if exactly three of its neighbors are alive. A live cell continues to live if it has two or three live neighbors. Even though the number of cells is potentially infinite, we can have a finite encoding for the state by only keeping track of the live cells. If we initialize the system in a configuration with a finite number of live cells, then the number of live cells will stay finite in all future steps.

We can think of such a system as encoding a computation by starting it in some initial configuration, and then defining some halting condition (e.g., we halt if the cell at position $(0,0)$ becomes dead) and some way to define an output (e.g., we output the state of the cell at position $(1,1)$). Clearly, given any starting configuration x, we can simulate the game of life starting from x using a NAND-RAM (or NAND-TM) program, and hence every "Game-of-Life computable" function is computable by a NAND-RAM program. Surprisingly, it turns out that the other direction is true as well: as simple as its rules seem, we can simulate a Turing machine using the game of life (see Fig. 7.9). The Wikipedia page for the Game of Life contains some beautiful figures and animations of configurations that produce some very interesting evolutions. See also the book The Nature of Computation.

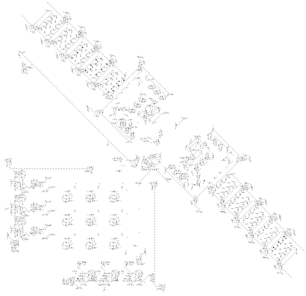


Figure 7.9: A Game-of-Life configuration simulating a Turing Machine. Figure by Paul Rendell.

7.7.3 Configurations of Turing machines and one dimensional cellular automata

It turns out that even one dimensional cellular automata can be Turing complete (see Fig. 7.11). In a *one dimensional automata*, the cells are laid out in one infinitely long line. The next state of each cell is only a function of its past state and the state of both its neighbors.

Definition 7.17 — One dimensional cellular automata. Let Σ be a finite set containing the symbol $\varnothing$. A *one dimensional cellular automation* over alphabet Σ is described by a *transition rule* $r : \Sigma^3 \to \Sigma$, which satisfies $r(\varnothing, \varnothing, \varnothing) = \varnothing$.

An *configuration* of the automaton is specified by a string $\alpha \in \Sigma^*$. We can also think of α as the infinite sequence $(\alpha_0, \alpha_1, \ldots, \alpha_{n-1}, \varnothing, \varnothing, \varnothing, \ldots)$, where $n = |\alpha|$. If α is a configuration and r is a transition rule, then the *next step configuration*, denoted by $\alpha' = NEXT_r(\alpha)$ is defined as follows:

$$alpha'_i = NEXT_r(\alpha_{i-1}, \alpha_i, \alpha_{i+1}) \tag{7.24}$$

for $i = 0, \ldots, n$. If j is smaller than 0 or larger than $n-1$ then we set $\alpha_j = \varnothing$.

In other words, the next state of the automaton r at point i obtained by applying the rule r to the values of α at i and its two neighbors.

Theorem 7.18 — One dimensional automata are Turing complete. For every Turing machine M, there is a one dimension cellular automaton that can simulate M on every input x.

To make this more precise, what Theorem 7.18 says is that for every Turing machine M, there is a one-dimensional cellular automaton $\mathcal{A}$ over some alphabet Σ, such that:

- There are computable maps to encode and decode configurations of M as configurations of $\mathcal{A}$.
- If we initialize $\mathcal{A}$ in the configuration encoding the starting state of M, then at every step t, the configuration of $\mathcal{A}$ at step t encodes the configuration of M at the same step.

Proof Idea:

A *configuration* of M contains its full state at after a particular iteration. That is, the contents of all (non empty) cells of its tape, its current state, as well as the head position. We can encode such a configuration as a string α over some large alphabet Σ. At position j, the symbol α_j will encode the value of the j-th symbol in M's tape. If the head position is i then α_i will encode this fact as well, and also contain an encoding of which state the machine is at.

Given this notion of an encoding, and the fact that the head moves only one position in each step, we can see that after one step of the machine M, the configuration largely stays the same except the locations $i, i-1, i+1$ corresponding to the location of the current variable i and its immediate neighbors. Once we realize this, we can phrase the

progression from one configuration to the next as a one dimensional ceullar automaton! From this observation, Theorem 7.18 follows in a fairly straightforward manner.

⋆

Before proving Theorem 7.18, let us formally define the notion of a *configuration* of a Turing machine (see also Fig. 7.10). We will come back to this notion in later chapters as well.

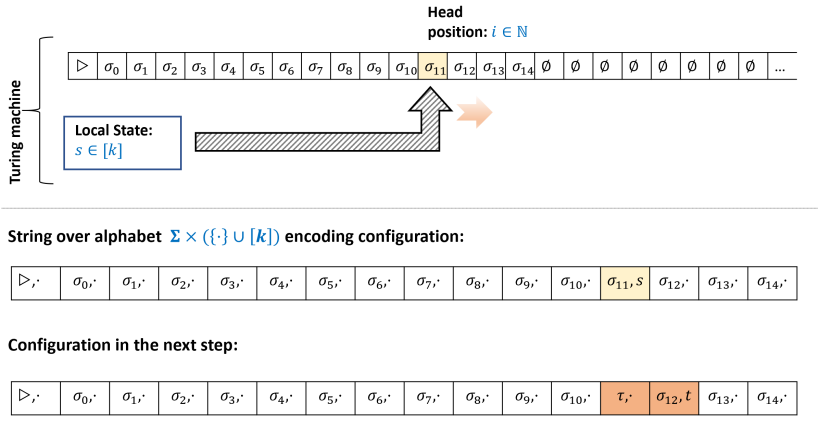


Figure 7.10: A *configuration* of a Turing machine M with alphabet Σ and state space $[k]$ encodes the state of M at a particular step in its execution as a string α over the alphabet $\overline{\Sigma} = \Sigma \times (\{\cdot\} \times [k])$. The string is of length t where t is such that M's tape contains $\varnothing$ in all positions t and larger and M's head is in a position smaller than t. If M's head is in the i-th position, then for $j \neq i$, α_j encodes the value of the j-th cell of M's tape, while α_i encodes both this value as well as the current state of M. If the machine writes the value τ, changes state to t, and moves right, then in the next configuration will contain at position i the value $(\tau, \cdot)$ and at position $i+1$ the value (α_{i+1}, t).

Definition 7.19 below has some technical details, but is not actually that deep or complicated. You would probably understand it better if before starting to read it, you take a moment to stop and think how *you* would encode as a string the state of a Turing machine at a given point in an execution.

Think what are all the components that you need to know in order to be able to continue the execution from this point onwards, and what is a simple way to encode them using a list of strings (which in turn can be encoded as a string). In particular, with an eye towards our future applications, try to think of an encoding which will make it as simple as possible to map a configuration at step t to the configuration at step $t+1$.

Definition 7.19 — Configuration of NAND-TM programs.. Let M be a Turing machine with tape alphabet Σ and state space $[k]$. A *configuration of* M is a string $\alpha \in \overline{\Sigma}^*$ where $\overline{\Sigma} = \Sigma \times (\{\cdot\} \cup [k])$ that satisfies that there is exactly one coordinate i for which $\alpha_i = (\sigma, s)$ for some $\sigma \in \Sigma$ and $s \in [k]$. For all other coordinates j, $\alpha_j = (\sigma', \cdot)$ for some $\sigma' \in \Sigma$.

A configuration $\alpha \in \overline{\Sigma}^*$ of M corresponds to the following state of its execution:

- M's tape contains $\alpha_{j,0}$ for all $j < |\alpha|$ and contains $\varnothing$ for all positions that are at least $|\alpha|$, where we let $\alpha_{j,0}$ be the value σ such that $\alpha_j = (\sigma, t)$ with $\sigma \in \Sigma$ and $t \in \{\cdot\} \cup [k]$. (In other words, since α_j is a pair of an alphabet symbol σ and either a state in $[k]$ or the symbol $\cdot$, $\alpha_{j,0}$ is the first component σ of this pair.)
- M's head is in the unique position i for which α_i has the form (σ, s) for $s \in [k]$, and M's state is equal to s.

Definition 7.19 is a little cumbersome, but ultimately a configuration is simply a string that encodes a *snapshot* of the state of the NAND-TM program at a given point in the execution. (In operating-systems lingo, it would be a "core dump".) Such a snapshot needs to encode the following components:

1. The current head position.
2. The full contents of the large scale memory, that is the tape.
3. The contents of the "local registers", that is the state of the machine.

The precise details of how we encode a configuration are not important, but we do want to record the following simple fact:

Lemma 7.20 Let M be a Turing machine and let $NEXT_M : \overline{\Sigma}^* \to \overline{\Sigma}^*$ be the function that maps a configuration of M to the configuration at the next step of the execution. Then for every $i \in \mathbb{N}$, the value of $NEXT_M(\alpha)_i$ only depends on the coordinates $\alpha_{i-1}, \alpha_i, \alpha_{i+1}$.[18]

We leave proving Lemma 7.20 as Exercise 7.4. It is not a hard exercise, but doing it is a great way to ensure that you are comfortable with the definition of configurations.

Once we have Lemma 7.20 in place, we see that the function $NEXT_M$ that maps a configuration of P into the next one is in fact a valid rule for a one dimensional automata, hence completing the proof of Theorem 7.18. The automaton arising from the proof of Theorem 7.18 has a large alphabet, and furthermore one whose size that depends on the machine M that is being simulated. It turns out that one can obtain an automaton with an alphabet of fixed size that is independent of the program being simulated, and in fact the alphabet of the automaton can be the minimal set $\{0, 1\}$! See Fig. 7.11 for an example of such an Turing-complete automaton.

[18] For simplicity of notation and of phrasing this lemma, we use the convention that if i is "out of bounds", such as $i < 0$ or $i > |\alpha|$, then we assume that $\alpha_i = (\varnothing, \cdot)$.

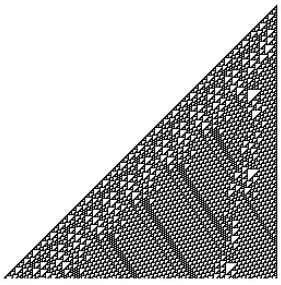


Figure 7.11: Evolution of a one dimensional automata. Each row in the figure corresponds to the configuration. The initial configuration corresponds to the top row and contains only a single "live" cell. This figure corresponds to the "Rule 110" automaton of Stefan Wolfram which is Turing Complete. Figure taken from Wolfram MathWorld.

R

Remark 7.21 — Configurations as binary strings. We can represent a configuration $\alpha \in \overline{\Sigma}^*$ by simply encoding each coordinate of α using $\log |\overline{\Sigma}|$ bits. When we refer to a configuration as a binary string (for example when feeding it as input to other programs) we will assume that this string represents the configuration via the above encoding.

R

Remark 7.22 — Configurations of NAND-TM programs. We can use the same approach to define configurations of a *NAND-TM program*. Such a configuration will need to encode:

1. The current value of the variable `i`.
2. For every scalar variable `foo`, the value of `foo`.
3. For every array variable `Bar`, the value `Bar[`j`]` for every $j \in \{0, \ldots, t-1\}$ where $t-1$ is the largest value that the index variable `i` ever achieved in the computation.

7.7.4 Turing completeness and equivalence, a formal definition (optional)

A *computational model* is some way to define what it means for a *program* (which is represented by a string) to compute a (partial) *function*. A *computational model* $\mathcal{M}$ is *Turing complete*, if we can map every Turing machine (or equivalently NAND-TM program) N into a program P for $\mathcal{M}$ that computes the same function as Q. It is *Turing equivalent* if the other direction holds as well (i.e., we can map every program in $\mathcal{M}$ to a Turing machine that computes the same function). Formally, we can define this notion as follows:[19]

Definition 7.23 — Turing completeness and equivalence (optional). Let $\mathcal{F}$ be the set of all partial functions from $\{0,1\}^*$ to $\{0,1\}^*$. A *computational model* is a map $\mathcal{M} : \{0,1\}^* \to \mathcal{F}$.

We say that a program $P \in \{0,1\}^*$ *$\mathcal{M}$-computes* a function $F \in \mathcal{F}$ if $\mathcal{M}(P) = F$.

A computational model $\mathcal{M}$ is *Turing complete* if there is a computable map $ENCODE_{\mathcal{M}} : \{0,1\}^* \to \{0,1\}^*$ for every Turing machine N (represented as a string), $\mathcal{M}(ENCODE_{\mathcal{M}}(N))$ is equal to the partial function computed by P.

A computational model $\mathcal{M}$ is *Turing equivalent* if it is Turing complete and there exists a computable map $DECODE_{\mathcal{M}} : \{0,1\}^* \to \{0,1\}^*$ such that or every string $P \in \{0,1\}^*$, $N =$

[19] The formal definition is very cumbersome to state, and not crucial for the remainder of this book. Feel free to skip it as long as you understand the general concept of Turing equivalence. This notion is sometimes referred to in the literature as Gödel numbering or admissalbe numbering.

DECODE$_{\mathcal{M}}(P)$ is a string representation of a Turing machine that computes the function $\mathcal{M}(P)$.

Some examples of Turing equivalent models include:

- Turing machines
- NAND-TM programs
- NAND-RAM programs
- Python, JavaScript, C, Lisp, and other programming languages.
- λ calculus
- Game of life (mapping programs and inputs/outputs to starting and ending configurations)
- Programming languages such as Python/C/Javascript/OCaml... (allowing for unbounded storage)

7.8 THE CHURCH-TURING THESIS (DISCUSSION)

> *"[In 1934], Church had been speculating, and finally definitely proposed, that the λ-definable functions are all the effectively calculable functions .... When Church proposed this thesis, I sat down to disprove it ... but, quickly realizing that [my approach failed], I became overnight a supporter of the thesis."*, Stephen Kleene, 1979.

> *"[The thesis is] not so much a definition or to an axiom but ... a natural law."*, Emil Post, 1936.

We have defined functions to be *computable* if they can be computed by a NAND-TM program, and we've seen that the definition would remain the same if we replaced NAND-TM programs by Python programs, Turing machines, λ calculus, cellular automata, and many other computational models. The *Church-Turing thesis* is that this is the only sensible definition of "computable" functions. Unlike the "Physical Extended Church Turing Thesis" (PECTT) which we saw before, the Church Turing thesis does not make a concrete physical prediction that can be experimentally tested, but it certainly motivates predictions such as the PECTT. One can think of the Church-Turing Thesis as either advocating a definitional choice, making some prediction about all potential computing devices, or suggesting some laws of nature that constrain the natural world. In Scott Aaronson's words, "whatever it is, the Church-Turing thesis can only be regarded as extremely successful". No candidate computing device (including quantum computers, and also much less reasonable models such as the hypothetical "closed time curve" computers we mentioned before) has so far mounted a serious challenge to the Church Turing thesis. These devices might potentially make some computations more *effi-*

cient, but they do not change the difference between what is finitely computable and what is not.[20]

[20] The *extended* Church Turing thesis, which we discuss in Section 12.6, stipulates that Turing machines capture also the limit of what can be *efficiently* computable. Just like its physical version, quantum computing presents the main challenge to this thesis.

7.8.1 Different models of computation

We can summarize the models we have seen in the following table:

Table 7.1: Different models for computing finite functions and functions with arbitrary input length.

Computational problems	Type of model	Examples
Finite functions $f : \{0,1\}^n \to \{0,1\}^m$	Non uniform computation (algorithm depends on input length)	Boolean circuits, NAND circuits, straight-line programs (e.g., NAND-CIRC)
Functions with unbounded inputs $F : \{0,1\}^* \to \{0,1\}^*$	Sequential access to memory	Turing machines, NAND-TM programs
–	Indexed access / RAM	RAM machines, NAND-RAM, modern programming languages
–	Other	Lambda calculus, cellular automata

Later on in Chapter 16 we will study *memory bounded* computation. It turns out that NAND-TM programs with a constant amount of memory are equivalent to the model of *finite automata* (the adjectives "deterministic" or "nondeterministic" are sometimes added as well, this model is also known as *finite state machines*) which in turns captures the notion of *regular languages* (those that can be described by regular expressions), which is a concept we will see in Chapter 9.



Lecture Recap

- While we defined computable functions using NAND-TM programs, we could just as well have done so using many other models, including not just NAND-RAM but also Turing machines, RAM machines, the λ-calculus and many other models.
- Very simple models turn out to be "Turing complete" in the sense that they can simulate arbitrarily complex computation.

7.9 EXERCISES

Exercise 7.1 — NAND-TM lookup. This exercise shows part of the proof that NAND-TM can simulate NAND-RAM. Produce the code of a NAND-TM program that computes the function $LOOKUP : \{0,1\}^* \rightarrow \{0,1\}$ that is defined as follows. On input $pf(i)x$, where $pf(i)$ denotes a prefix-free encoding of an integer i, $LOOKUP(pf(i)x) = x_i$ if $i < |x|$ and $LOOKUP(pf(i)x) = 0$ otherwise. (We don't care what $LOOKUP$ outputs on inputs that are not of this form.) You can choose any prefix-free encoding of your choice, and also can use your favorite programming language to produce this code.

■

Exercise 7.2 — Pairing. Let $embed : \mathbb{N}^2 \rightarrow \mathbb{N}$ be the function defined as $embed(x_0, x_1) = \frac{1}{2}(x_0 + x_1)(x_0 + x_1 + 1) + x_1$.

1. Prove that for every $x^0, x^1 \in \mathbb{N}$, $embed(x^0, x^1)$ is indeed a natural number.

2. Prove that $embed$ is one-to-one

3. Construct a NAND-TM program P such that for every $x^0, x^1 \in \mathbb{N}$, $P(pf(x^0)pf(x^1)) = pf(embed(x^0, x^1))$, where pf is the prefix-free encoding map defined above. You can use the syntactic sugar for inner loops, conditionals, and incrementing/decrementing the counter.

4. Construct NAND-TM programs P_0, P_1 such that for for every $x^0, x^1 \in \mathbb{N}$ and $i \in N$, $P_i(pf(embed(x^0, x^1))) = pf(x^i)$. You can use the syntactic sugar for inner loops, conditionals, and incrementing/decrementing the counter.

■

Exercise 7.3 — lambda calculus requires three variables. Prove that for every λ-expression e with no free variables there is an equivalent λ-expression f that only uses the variables x,y, and z.[21]

■

Exercise 7.4 — Next-step function is local. Prove Lemma 7.20 and use it to complete the proof of Theorem 7.18.

■

[21] **Hint:** You can reduce the number of variables a function takes by "pairing them up". That is, define a λ expression $PAIR$ such that for every x, y $PAIRxy$ is some function f such that $f0 = x$ and $f1 = y$. Then use $PAIR$ to iteratively reduce the number of variables used.

7.10 BIBLIOGRAPHICAL NOTES

Chapters 7 in the wonderful book of Moore and Mertens [MM11] contains a great exposition much of this material. Chapter 3 in Savage's book [Sav98] contains a more formal description of RAM machines,

see also the paper [Hag98]. A study of RAM algorithms that are independent of the input size (known as the "transdichotomous RAM model") was initiated by [FW93].

The RAM model can be very useful in studying the concrete complexity of practical algorithms. However, the exact set of operations that are allowed in the RAM model at unit cost can vary between texts and contexts. One needs to be careful in making such definitions, especially if the word size grows, as was already shown by Shamir [Sha79].

The λ-calculus was described by Church in [Chu41]. Pierce's book [Pie02] is a canonical textbook, see also [Bar84]. The "Currying technique" is named after the logician Haskell Curry (the *Haskell* programming language is named after Haskell Curry as well). Curry himself attributed this concept to Moses Schönfinkel, though for some reason the term "Schönfinkeling" never caught on..

Tao has proposed showing the Turing completeness of fluid dynamics (a "water computer") as a way of settling the question of the behavior of the Navier-Stokes equations, see this popular article.

Learning Objectives:

- **The universal machine/program - "one program to rule them all"**
- **A fundamental result in computer science and mathematics: the existence of uncomputable functions.**
- **The *halting problem*: the canonical example for an uncomputable function.**
- **Introduction to the technique of *reductions*.**
- **Rice's Theorem: A "meta tool" for uncomputability results, and a starting point for much of the research on compilers, programming languages, and software verification.**

8
Universality and uncomputability

> *"A function of a variable quantity is an analytic expression composed in any way whatsoever of the variable quantity and numbers or constant quantities."*, Leonhard Euler, 1748.

> *"The importance of the universal machine is clear. We do not need to have an infinity of different machines doing different jobs. ... The engineering problem of producing various machines for various jobs is replaced by the office work of 'programming' the universal machine"*, Alan Turing, 1948

One of the most significant results we showed for Boolean circuits (or equivalently, straight-line programs) is the notion of *universality*: there is a single circuit/program that can evaluate all other circuits/programs. However, this result came with a significant caveat. To evaluate a circuit of s gates, the universal circuit needed to use a larger number of gates. Equivalently, the function that evaluates a given NAND-CIRC program of s lines on a given input requires more than s lines to compute.

It turns out that uniform models such as Turing machines or NAND-TM programs allow us to "break out of this cycle" and obtain a truly *universal Turing machine* U that can evaluate all other machines, including machines that are more complex (e.g., more states) than U itself. (Similarly, there is a *Universal NAND-TM program* U' that can evaluate all NAND-TM programs, including programs that have more lines than U'.)

It is no exaggeration to say that the existence of such a universal program/machine underlies the information technology revolution that began in the latter half of the 20th century (and is still ongoing). Up to that point in history, people have produced various special-purpose calculating devices such as the abacus, the slide ruler, and machines that compute various trigonometric series. But as Turing

(who was perhaps the one to see most clearly the ramifications of universality) observed, a *general purpose computer* is much more powerful. Once build a device that can compute the single universal function we have the ability, *via software*, to extend it to do arbitrary computations. For example, if we want to simulate a new Turing machine M, we do not need to build a new physical machine, but rather can represent M as a string (i.e., using *code*) and then input M to the universal machine U.

Beyond the practical applications, the existence of a universal algorithm also surprising theoretical ramification, and in particular can be used to show the existence of *uncomputable functions*, upending the intuitions of mathematicians over the centuries from Euler to Hilbert. In this chapter we will prove the existence of the universal program, and also show its implications for uncomputability.

8.1 UNIVERSALITY OR A SELF-CIRCULAR EVALUATOR

We now prove the existence of a universal Turing machine:

Theorem 8.1 — Universal Turing Machine. There is a Turing machine U such that on every string M which represents a Turing machine, and $x \in \{0,1\}^*$, $U(M,x) = M(x)$.

That is, if the machine M halts on x and outputs some $y \in \{0,1\}^*$, then $U(M,x) = y$, and if M does not halt on x, then $U(M,x)$ does not halt as well.

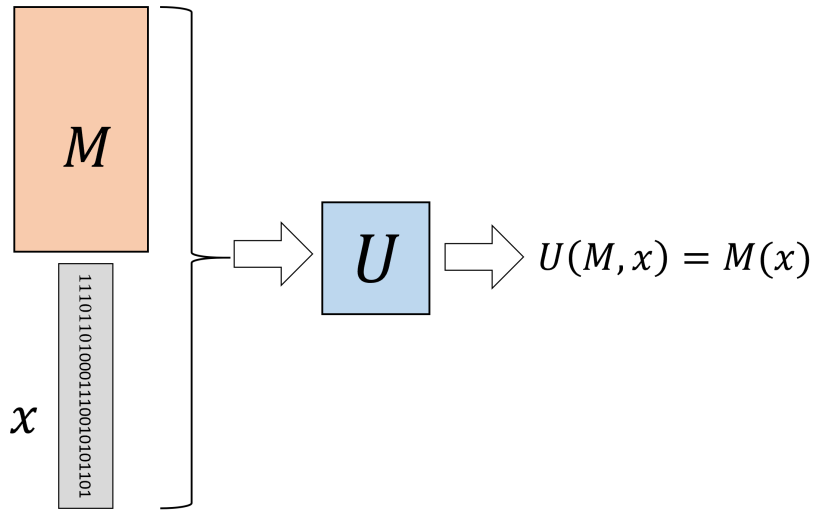


Figure 8.1: A *Universal Turing Machine* is a single Turing Machine U that can evaluate, given input the (description as a string of) arbitrary Turing machine M and input x, the output of M on x. In contrast to the universal circuit depicted in Fig. 5.2, the machine M can be much more complex (e.g., more states or tape alphabet symbols) than U.

A Turing machine U that satisfies the conditions of Theorem 8.1 is known as a *universal Turing machine*. There is more than one universal

Turing machine, but the existence of even a single one is extremely fundamental to both the theory and practice of computer science. By the equivalence results we have seen between different models such as Turing machines, NAND-TM programs, RAM machines, and others, Theorem 8.1 implies (for example) that:

- There exists a universal NAND-TM program that computes the map $P, x \mapsto P(x)$, where P is a NAND-TM program.
- There exists a NAND-RAM program that computes the map $M, x \mapsto M(x)$, where M is a Turing machine.

and more generally,

- For every $\mathcal{X}$ and $\mathcal{Y}$ in the set { Turing Machines, RAM Machines, NAND-TM, NAND-RAM, λ-calculus, JavaScript, Python, ... } of Turing equivalent models, there exists a program/machine in $\mathcal{X}$ that computes the map $P, x \mapsto P(x)$ for every program/machine in $\mathcal{Y}$.

We now present the proof of Theorem 8.1.

Proof Idea:

Once you understand what the theorem says, it is not that hard to prove. The desired program U is an *interpreter* for Turing machines. That is, U gets a representation of the machine M (think of it as source code), and some input x, and needs to simulate the execution of M on x.

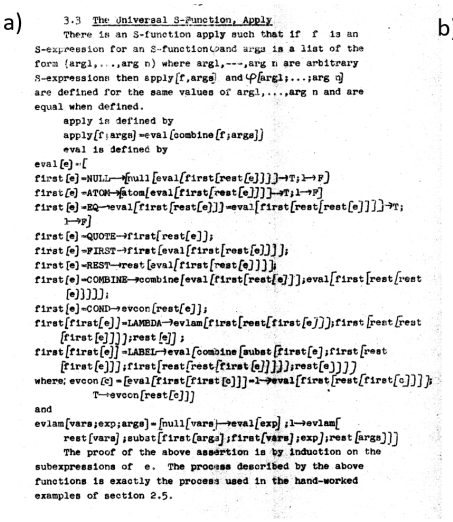
Think of how you would code U in your favorite programming language. First, you would need to decide on some representation scheme for M. For example, you can use an array or a dictionary to encode M's transition function. Then you would use some data structure, such as a list, to store the contents of M's tape. Now you can simulate M step by step, updating the data structure as you go along. The interpreter will continue the simulation until the machine halts.

Once you do that, translating this interpreter from your favorite programming language to a Turing machine can be done just as we have seen in Chapter 7. The end result is what's known as a "meta-circular evaluator": an interpreter for a programming language in the same one. This is a concept that has a long history in computer science starting from the original universal Turing machine. See also Fig. 8.2.

⋆

Theorem 8.1 yields a stronger notion than the universality we proved for Boolean circuits / NAND-CIRC, in the sense that we show a *single* universal Turing machine U that can evaluate *all* Turing machines, including those that have more states than U (see Fig. 8.1).

a)



3.3 The Universal S-function, Apply

There is an S-function apply such that if f is an S-expression for an S-function and args is a list of the form (arg1,...,arg n) where arg1,---,arg n are arbitrary S-expressions then apply[f,args] and φ[arg1;...;arg n] are defined for the same values of arg1,...,arg n and are equal when defined.

apply is defined by

apply[f;args] = eval[combine[f;args]]

eval is defined by

```
eval[e] = [
first[e] = NULL → null[eval[first[rest[e]]]] → T; 1 → F]
first[e] = ATOM → atom[eval[first[rest[e]]]] → T; 1 → F]
first[e] = EQ → eval[first[rest[e]]] = eval[first[rest[rest[e]]]] → T;
    1 → F]
first[e] = QUOTE → first[rest[e]];
first[e] = FIRST → first[eval[first[rest[e]]]];
first[e] = REST → rest[eval[first[rest[e]]]];
first[e] = COMBINE → combine[eval[first[rest[e]]];eval[first[rest[rest
    [e]]]];
first[e] = COND → evcon[rest[e]];
first[first[e]] = LAMBDA → evlam[first[rest[first[e]]];first[rest[rest
    [first[e]]]];rest[e]];
first[first[e]] = LABEL → eval[combine[subst[first[e];first[rest
    [first[e]]];first[rest[rest[first[e]]]];rest[e]]]]
where; evcon[e] = [eval[first[first[e]]] → eval[first[rest[first[e]]]];
        T → evcon[rest[e]]]
and
evlam[vars;exp;args] = [null[vars] → eval[exp]; 1 → evlam[
    rest[vars];subst[first[args];first[vars];exp];rest[args]]]
```

The proof of the above assertion is by induction on the subexpressions of e. The process described by the above functions is exactly the process used in the hand-worked examples of section 2.5.

b)

```
char s[ ] = {
    '\t',
    '0',
    '\n',
    '}',
    ';',
    '\n',
    '\n',
    '/',
    '*',
    '\n',
    (213 lines deleted)
    0
};

/*
 * The string s is a
 * representation of the body
 * of this program from '0'
 * to the end.
 */

main( )
{
        int i;

        printf("char\ts[ ] = {\n");
        for(i=0; s[i]; i++)
                printf("\t%d, \n", s[i]);
        printf("%s", s);
}
```

Figure 8.2: **a**) A particularly elegant example of a "meta-circular evaluator" comes from John McCarthy's 1960 paper, where he defined the Lisp programming language and gave a Lisp function that evaluates an arbitrary Lisp program (see above). Lisp was not initially intended as a practical programming language and this example was merely meant as an illustration that the Lisp universal function is more elegant than the universal Turing machine. It was McCarthy's graduate student Steve Russell who suggested that it can be implemented. As McCarthy later recalled, *"I said to him, ho, ho, you're confusing theory with practice, this eval is intended for reading, not for computing. But he went ahead and did it. That is, he compiled the eval in my paper into IBM 704 machine code, fixing a bug, and then advertised this as a Lisp interpreter, which it certainly was"*. **b**) A self-replicating C program from the classic essay of Thompson [Tho84].

In particular, U can even be used to evaluate itself! This notion of *self reference* will appear time and again in this course, and as we will see, leads to several counter-intuitive phenomena in computing.

R

Remark 8.2 — Universal evaluators in practice. The idea of a "universal program" is of course not limited to theory. For example the C compiler can be and is used to execute programs that are more complicated than itself. (An extreme example of this is Fabrice Bellard's Obfuscated Tiny C Compiler which is a C program of 2048 bytes that can compile a fairly large subset of the C programming language, and in particular can compile itself. It was later extended to the Tiny C compiler that can compile the full ANSI C language as well as some extensions.)

In fact, it is a common practice to use a compiler to compile *itself*: typically version n of a compiler is used to compile the (updated) version $n + 1$. This is also related to the fact that it is possible to write a program that can print its own source code, see Fig. 8.2.

Because we can transform other computational models, including NAND-RAM, λ calculus, C and Python programs, and more, to NAND-TM this means that even the seemingly "weak" NAND-TM programming language is powerful enough to contain an interpreter for all these models.

To prove (and even properly state!) Theorem 8.1, we need to make sure that it is well defined by specifying a representation for Turing machines as strings. For example, one choice for such a representation is represent a Turing machine M using the ASCII encoding of the

source code of the corresponding NAND-TM program P. However, we will use a different choice here. We will start by representing a machine M as a list of *natural numbers*, which we can then easily translate into a binary string using the binary basis. We can encode a machine M by a list of numbers by including the number k of states, the size ℓ of the alphabet, and $k \times \ell$ table of values of its transition function δ_M. Using this representation, we can formally prove Theorem 8.1.

Proof of Theorem 8.1. We will only sketch the proof, giving the major ideas. First, we observe that we can easily write a *Python* program that, on input a representation (k, ℓ, δ_M) of a Turing machine M and an input x, evaluates M on X. Here is the code of this program for concreteness, though you can feel free to skip it if you are not familiar with (or interested in) Python:

```
# constants
Φ = 2 # empty symbol
Δ = 3 # starting symbol
L = 0 # go left
R = 1 # go right
S = 2 # stay
H = 3 # halt

def EVAL(k,l,T,x):
    """Evaluate a Turing machine on input x
       Turing machine is represented by:
       k: number of states
       l: number of symbols in alphabet (containing
 ↪  {0,1,Φ,Δ})
       T: transition function: a dictionary such that
 ↪  T[(s,a)] is equal to (_s,_a,_d)
            where s is old state, a is symbol read
            _s is new state
            _a is symbol to write
            _d in {L,R,S,H} is head movement
    """


    Tape = [ Δ  ] # List/array containing contents of tape

    # Initialize with input
    for i in range(len(x)): Tape.append(int(x[i]))

    i = 0 # current position
    s = 0 # current state
```

```
    while True:
        a = Tape[i]  # read symbol
        _s,_a,_d = T[(s,a)] # lookup transition table
        Tape[i] = _a # write symbol
        s = _s # update state

        # move head:
        if _d == H: break
        if _d == L: i = max(i-1,0)
        if _d == R: i += 1

        # add empty symbols to tape if needed
        if i>= len(Tape): Tape.append(Φ)


    # Scan tape for output
    Y = []
    j = 1
    while j<len(Tape) and Tape[j] != Φ:
        Y.append(Tape[j])
        j += 1

    return Y
```

The above does not prove the theorem as stated, since we need to show a *Turing machine* that computes $EVAL$ rather than a Python program. With enough effort, we can translate this Python code line by line to a Turing machine. However, to prove the theorem we don't need to do this, but can use our "eat the cake and have it too" paradigm. That is, while we need to evaluate a Turing machine, in writing the code for the interpreter we are allowed to use a richer model such as NAND-RAM since it is equivalent in power to Turing machines per Theorem 7.1).

Translating the above Python code to NAND-RAM is truly straightforward. The only issue is that NAND-RAM doesn't have the *dictionary* data structure built in, which we have used above to store the transition function `T`. However, we can represent a dictionary D of the form $\{key_0 : val_0, \ldots, key_{m-1} : val_{m-1}\}$ as simply a list of pairs. To compute $D[k]$ we can scan over all the pairs until we find one of the form (k, v) in which case we return v. Similarly we scan the list to update the dictionary with a new value, either modifying it or appending the pair (key, val) at the end.

The above is a very inefficient way to implement the dictionary data structure in practice, but it suffices for the purpose of proving the theorem.[1]

■

[1] Reading and writing to a dictionary of m values in this implementation takes $\Omega(m)$ steps, but it is in fact possible to do this in $O(\log m)$ steps using a *search tree* data structure or even $O(1)$ (for "typical" instances) using a *hash table*. NAND-RAM and RAM machines correspond to the architecture of modern electronic computers, and so we can implement hash tables and search trees in NAND-RAM just as they are implemented in other programming languages.

8.2 IS EVERY FUNCTION COMPUTABLE?

In Theorem 4.6, we saw that NAND-CIRC programs can compute every finite function $f : \{0,1\}^n \to \{0,1\}$. Therefore a natural guess is that NAND-TM programs (or equivalently, Turing Machines) could compute every infinite function $F : \{0,1\}^* \to \{0,1\}$.

However, this turns out to be *false*. That is, there exists a function $F : \{0,1\}^* \to \{0,1\}$ that is *uncomputable*!

This is actually quite surprising, if you think about it. Our intuitive notion of a "function" (and the notion most scholars had until the 20th century, see the bibliographical notes) is that a function f defines some implicit or explicit way of computing the output $f(x)$ from the input x. The notion of an "uncomputable function" thus seems to be a contradiction in terms, but yet the following theorem shows that such creatures do exist:

Theorem 8.3 — Uncomputable functions. There exists a function $F^* : \{0,1\}^* \to \{0,1\}$ that is not computable by any Turing machine.

Proof Idea:

The idea behind the proof follows quite closely Cantor's proof that the reals are uncountable (Theorem 2.5), and in fact the theorem can also be obtained fairly directly from that result (see Exercise 6.5). However, it is instructive to see the direct proof. The idea is to construct F^* in a way that will ensure that every possible machine M will in fact fail to compute F^*. We can do so by looking at the string representation α_M of the machine M, and define $F^*(\alpha_M)$ to equal 1 if $M(\alpha_M) = 0$ and to equal 0 otherwise. This will guarantee that $F^*(\alpha_M) \neq M(\alpha_M)$ and hence that F^* is not computable by M.

⋆

Proof of Theorem 8.3. The proof is illustrated in Fig. 8.3. We start by defining the following function $G : \{0,1\}^* \to \{0,1\}$:

For every string $x \in \{0,1\}^*$, if x satisfies **(1)** x is a valid representation of Turing machine M_x and **(2)** when the program M_x is executed on the input x it halts and produces an output, then we define $G(x)$ as the first bit of this output. Otherwise (i.e., if x is not a valid representation of a Turing machine, or the machine M_x never halts on x) we define $G(x) = 0$. We define $F^*(x) := 1 - G(x)$.

We claim that there is no Turing machine that computes F^*. Indeed, suppose, towards the sake of contradiction, that there was some machine M that computed F^*, and let x be the binary string that represents the machine M. Then, since M computes F^*, on input x, $F^*(x)$ equals the output of the machine M on x. But by the definition of F^*, $F^*(x)$ must be different than the output of M on the string x that represents it, hence yielding a contradiction.

■



Figure 8.3: We construct an uncomputable function by defining for every two strings x, y the value $1 - M_y(x)$ which equals 0 if the machine described by y outputs 1 on x, and 1 otherwise. We then define $F^*(x)$ to be the "diagonal" of this table, namely $F^*(x) = 1 - M_x(x)$ for every x. The function F^* is uncomputable, because if it was computable by some machine whose string description is x^* then we would get that $M_{x^*}(x^*) = F(x^*) = 1 - M_{x^*}(x^*)$.

P

The proof of Theorem 8.3 is short but subtle. I suggest that you pause here and go back to read it again and think about it - this is a proof that is worth reading at least twice if not three or four times. It is not often the case that a few lines of mathematical reasoning establish a deeply profound fact - that there are problems we simply *cannot* solve.

The type of argument used to prove Theorem 8.3 is known as *diagonalization* since it can be described as defining a function based on the diagonal entries of a table as in Fig. 8.3. The proof can be thought of as an infinite version of the *counting* argument we used for showing lower bound for NAND-CIRC progams in Theorem 5.9. Namely, we show that it's not possible to compute all functions from $\{0,1\}^* \rightarrow \{0,1\}$ by Turing machines simply because there are more functions like that then there are Turing machines.

R

Remark 8.4 — Undecidable languages. We can associate with every function $F : \{0,1\}^* \to \{0,1\}$ the set $L_F = \{x \mid F(x) = 1\}$. As mentioned in Remark 6.5, such sets are often referred to as *languages*. If F is uncomputable then we say that the corresponding language L_F is *undecidable*. (Some texts also use the term *non recursive*.) While we do not use the terminology of languages and decidability in this book, it is very common in the literature. Fortunately it is easy to translate back and forth between uncomputable functions and undecidable languages.

8.3 THE HALTING PROBLEM

Theorem 8.3 shows that there is *some* function that cannot be computed. But is this function the equivalent of the "tree that falls in the forest with no one hearing it"? That is, perhaps it is a function that no one actually *wants* to compute. It turns out that there are natural uncomputable functions:

Theorem 8.5 — Uncomputability of Halting function. Let $HALT : \{0,1\}^* \to \{0,1\}$ be the function such that for every string $M \in \{0,1\}^*$, $HALT(M,x) = 1$ if Turing machine M halts on the input x and $HALT(M,x) = 0$ otherwise. Then $HALT$ is not computable.

Before turning to prove Theorem 8.5, we note that $HALT$ is a very natural function to want to compute. For example, one can think of $HALT$ as a special case of the task of managing an "App store". That is, given the code of some application, the gatekeeper for the store needs to decide if this code is safe enough to allow in the store or not. At a minimum, it seems that we should verify that the code would not go into an infinite loop.

Proof Idea:

One way to think about this proof is as follows:

$$\text{Uncomputability of } F^* + \text{Universality} = \text{Uncomputability of } HALT \tag{8.1}$$

That is, we will use the universal Turing machine that computes $EVAL$ to derive the uncomputability of $HALT$ from the uncomputability of F^* shown in Theorem 8.3. Specifically, the proof will be by contradiction. That is, we will assume towards a contradiction that $HALT$ is computable, and use that assumption, together with the universal Turing machine of Theorem 8.1, to derive that F^* is computable, which will contradict Theorem 8.3.

⋆

Proof of Theorem 8.5. The proof will use the previously established result Theorem 8.3. Recall that Theorem 8.3 shows that the following function $F^* : \{0,1\}^* \to \{0,1\}$ is uncomputable:

$$F^*(x) = \begin{cases} 1 & x(x) = 0 \\ 0 & \text{otherwise} \end{cases} \tag{8.2}$$

where $x(x)$ denotes the output of the Turing machine described by the string x on the input x (with the usual convention that $x(x) = \perp$ if this computation does not halt).

We will show that the uncomputability of F^* implies the uncomputability of *HALT*. Specifically, we will assume, towards a contradiction, that there exists a Turing machine M that can compute the *HALT* function, and use that to obtain a Turing machine M' that computes the function F^*. (This is known as a proof by *reduction*, since we reduce the task of computing F^* to the task of computing *HALT*. By the contrapositive, this means the uncomputability of F^* implies the uncomputability of *HALT*.)

Indeed, suppose that M is a Turing machine that computes *HALT*. Then we can construct a Turing machine M' that computes F^* as follows:[2]

Machine M': (*Goal:* compute F^*)

Assumption: There is a TM M that computes *HALT*.

Input: $x \in \{0,1\}^*$

Operation:

1. Compute $z = M(x,x)$. (We can run a machine M inside a machine M' just like we can call a subroutine in a program. Note that under our assumption that M computes *HALT*, $z = HALT(x,x)$.)
2. If $z = 0$, then output 1.
3. Otherwise, if $z = 1$, then use the universal TM to compute $y = x(x)$ (i.e., the output of the machine described by x on the input x). If $y = 0$ then output 1. Otherwise, output 0.

We claim that M' computes the function F^*. Indeed, suppose that $x(x) = 0$ (and hence $F^*(x) = 1$). In this case, $HALT(x,x) = 1$ and hence, under our assumption that $M(x,x) = HALT(x,x)$, M' will not output anything in Step 2. This means that in Step 3, the machine M' will evaluate $x(x)$, obtain the value $y = 0$, and output the correct value 1.

[2] We are using here a "high level" description of Turing machines, appealing to the "have your cake and eat it too" paradigm, which means that we can always translate this description into a Turing machine, for example by first writing them in NAND-RAM and then transforming the NAND-RAM program to a Turing machine.

Suppose otherwise that $x(x) \neq 0$ (and hence $F^*(x) = 0$). In this case there are two possibilities:

- **Case 1:** The machine described by x does not halt on the input x. In this case, $HALT(x, x) = 0$. Since we assume that M computes *HALT* it means that on input x, x, the machine M must halt and output the value 0. This means that the machine M' will obtain the value $z = 0$ in step 1, and hence stop and output 0 in step 2.
- **Case 2:** The machine described by x halts on the input x and outputs some $y \neq 0$. In this case, since $HALT(x, x) = 1$, under our assumptions, the machine M' will not stop in Step 2. Rather in Step 3 it will run the computation $x(x)$, obtain the value $y \neq 0$, and then output 0.

We see that in all cases, $M'(x) = F^*(x)$, which contradicts the fact that F^* is uncomputable. Hence we reach a contradiction to our original assumption that M computes *HALT*.

■

P

Once again, this is a proof that's worth reading more than once. The uncomputability of the halting problem is one of the fundamental theorems of computer science, and is the starting point for much of the investigations we will see later. An excellent way to get a better understanding of Theorem 8.5 is to go over Section 8.3.2, which presents an alternative proof of the same result.

8.3.1 Is the Halting problem really hard? (discussion)

Many people's first instinct when they see the proof of Theorem 8.5 is to not believe it. That is, most people do believe the mathematical statement, but intuitively it doesn't seem that the Halting problem is really that hard. After all, being uncomputable only means that *HALT* cannot be computed by a Turing machine.

But programmers seem to solve *HALT* all the time by informally or formally arguing that their programs halt. It's true that their programs are written in C or Python, as opposed to Turing machines, but that makes no difference: we can easily translate back and forth between this model and any other programming language.

While every programmer encounters at some point an infinite loop, is there really no way to solve the halting problem? Some people argue that *they* personally can, if they think hard enough, determine whether any concrete program that they are given will halt or not.

Some have even argued that humans in general have the ability to do that, and hence humans have inherently superior intelligence to computers or anything else modeled by Turing machines.[3]

The best answer we have so far is that there truly is no way to solve *HALT*, whether using Macs, PCs, quantum computers, humans, or any other combination of electronic, mechanical, and biological devices. Indeed this assertion is the content of the *Church-Turing Thesis*. This of course does not mean that for *every* possible program P, it is hard to decide if P enters an infinite loop. Some programs don't even have loops at all (and hence trivially halt), and there are many other far less trivial examples of programs that we can certify to never enter an infinite loop (or programs that we know for sure that *will* enter such a loop). However, there is no *general procedure* that would determine for an *arbitrary* program P whether it halts or not. Moreover, there are some very simple programs for which no one knows whether they halt or not. For example, the following Python program will halt if and only if Goldbach's conjecture is false:

```
def isprime(p):
    return all(p % i  for i in range(2,p-1))

def Goldbach(n):
    return any( (isprime(p) and isprime(n-p))
           for p in range(2,n-1))

n = 4
while True:
    if not Goldbach(n): break
    n+= 2
```

Given that Goldbach's Conjecture has been open since 1742, it is unclear that humans have any magical ability to say whether this (or other similar programs) will halt or not.

8.3.2 A direct proof of the uncomputability of *HALT* (optional)

It turns out that we can combine the ideas of the proofs of Theorem 8.3 and Theorem 8.5 to obtain a short proof of the latter theorem, that does not appeal to the uncomputability of F^*. This short proof appeared in print in a 1965 letter to the editor of Christopher Strachey:[4]

> To the Editor, The Computer Journal.
>
> An Impossible Program
>
> Sir,

[3] This argument has also been connected to the issues of consciousness and free will. I am personally skeptical of its relevance to these issues. Perhaps the reasoning is that humans have the ability to solve the halting problem but they exercise their free will and consciousness by choosing not to do so.

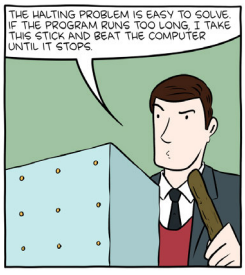


Figure 8.4: SMBC's take on solving the Halting problem.

[4] Christopher Strachey was an English computer scientist and the inventor of the CPL programming language. He was also an early artificial intelligence visionary, programming a computer to play Checkers and even write love letters in the early 1950's, see this New Yorker article and this website.

A well-known piece of folk-lore among programmers holds that it is impossible to write a program which can examine any other program and tell, in every case, if it will terminate or get into a closed loop when it is run. I have never actually seen a proof of this in print, and though Alan Turing once gave me a verbal proof (in a railway carriage on the way to a Conference at the NPL in 1953), I unfortunately and promptly forgot the details. This left me with an uneasy feeling that the proof must be long or complicated, but in fact it is so short and simple that it may be of interest to casual readers. The version below uses CPL, but not in any essential way.

Suppose `T[R]` is a Boolean function taking a routine (or program) `R` with no formal or free variables as its arguments and that for all `R`, `T[R] = True` if `R` terminates if run and that `T[R] = False` if `R` does not terminate.

Consider the routine P defined as follows

```
rec routine P

§L: if T[P] go to L

Return §
```

If `T[P] = True` the routine `P` will loop, and it will only terminate if `T[P] = False`. In each case 'T[P]" has exactly the wrong value, and this contradiction shows that the function T cannot exist.

Yours faithfully,

C. Strachey

Churchill College, Cambridge



Try to stop and extract the argument for proving Theorem 8.5 from the letter above.

Since CPL is not as common today, let us reproduce this proof. The idea is the following: suppose for the sake of contradiction that there exists a program `T` such that `T(f,x)` equals `True` iff `f` halts on input `x`.[5] Then we can construct a program `P` and an input `x` such that `T(P,x)` gives the wrong answer. The idea is that on input `x`, the program `P` will do the following: run `T(x,x)`, and if the answer is `True` then go into an infinite loop, and otherwise halt. Now you can see that `T(P,P)` will give the wrong answer: if `P` halts when it gets its own code as input, then `T(P,P)` is supposed to be `True`, but then `P(P)` will go into

[5] Strachey's letter considers the no-input variant of *HALT*, but as we'll see, this is an immaterial distinction.

an infinite loop. And if `P` does not halt, then `T(P,P)` is supposed to be `False` but then `P(P)` will halt. We can also code this up in Python:

```
def CantSolveMe(T):
    """
    Gets function T that claims to solve HALT.
    Returns a pair (P,x) of code and input on which
    T(P,x) ≠ HALT(x)
    """
    def fool(x):
        if T(x,x):
            while True: pass
        return "I halted"

    return (fool,fool)
```

For example, consider the following Naive Python program `T` that guesses that a given function does not halt if its input contains `while` or `for`

```
def T(f,x):
    """Crude halting tester - decides it doesn't halt if it
     ↪  contains a loop."""
    import inspect
    source = inspect.getsource(f)
    if source.find("while"): return False
    if source.find("for"): return False
    return True
```

If we now set `(f,x) = CantSolveMe(T)`, then `T(f,x)=False` but `f(x)` does in fact halt. This is of course not specific to this particular `T`: for every program `T`, if we run `(f,x) = CantSolveMe(T)` then we'll get an input on which `T` gives the wrong answer to *HALT*.

8.4 REDUCTIONS

The Halting problem turns out to be a linchpin of uncomputability, in the sense that Theorem 8.5 has been used to show the uncomputability of a great many interesting functions. We will see several examples in such results in this chapter and the exercises, but there are many more such results (see Fig. 8.5).

The idea behind such uncomputability results is conceptually simple but can at first be quite confusing. If we know that *HALT* is uncomputable, and we want to show that some other function *BLAH* is uncomputable, then we can do so via a *contrapositive* argument (i.e., proof by contradiction). That is, we show that **if** there exists Turing

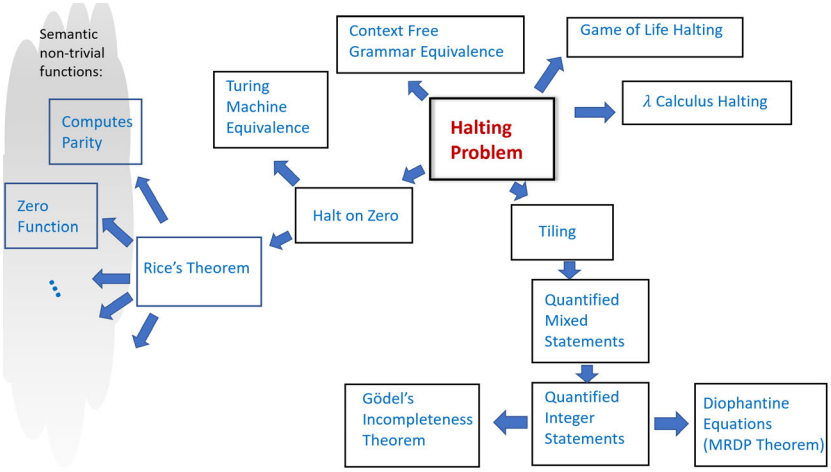


Figure 8.5: Some uncomputability results. An arrow from problem X to problem Y means that we use the uncomputability of X to prove the uncomputability of Y by reducing computing X to computing Y. All of these results except for the MRDP Theorem appear in either the text or exercises. The Halting Problem *HALT* serves as our starting point for all these uncomputability results as well as many others.

machine that computes *BLAH* **then** there exists a Turing machine that computes *HALT*. (Indeed, this is exactly how we showed that *HALT* itself is uncomputable, by reducing this fact to the uncomputability of the function F^* from Theorem 8.3.)

For example, to prove that *BLAH* is uncomputable, we could show that there is a computable function $R : \{0,1\}^* \rightarrow \{0,1\}^*$ such that for every pair M and x, $HALT(M,x) = BLAH(R(M,x))$. The existence of such a function R implies that **if** *BLAH* was computable **then** *HALT* would be computable as well, hence leading to a contradiction! The confusing part about reductions is that we are assuming something we *believe* is false (that *BLAH* has an algorithm) to derive something that we *know* is false (that *HALT* has an algorithm). Michael Sipser describes such results as having the form *"If pigs could whistle then horses could fly"*.

A reduction-based proof has two components. For starters, since we need R to be computable, we should describe the algorithm to compute it. The algorithm to compute R is known as a *reduction* since the transformation R modifies an input to *HALT* to an input to *BLAH*, and hence *reduces* the task of computing *HALT* to the task of computing *BLAH*. The second component of a reduction-based proof is the *analysis* of the algorithm R: namely a proof that R does indeed satisfy the desired properties.

At the end of the day reduction-based proofs are just like other proofs by contradiction, but the fact that they involve hypothetical algorithms that don't really exist tends to make reductions quite confusing. The one silver lining is that at the end of the day the notion of reductions is mathematically quite simple, and so it's not that bad

even if you have to go back to first principles every time you need to remember what is the direction that a reduction should go in.

8.4.1 Example: Halting on the zero problem

Here is a concrete example for a proof by reduction. We define the function $HALTONZERO : \{0,1\}^* \to \{0,1\}$ as follows. Given any string M, $HALTONZERO(M) = 1$ if and only if M describes a Turing machine that halts when it is given the string 0 as input. A priori *HALTONZERO* seems like a potentially easier function to compute than the full-fledged *HALT* function, and so we could perhaps hope that it is not uncomputable. Alas, the following theorem shows that this is not the case:

Theorem 8.6 — Halting without input. *HALTONZERO* is uncomputable.

P

The proof of Theorem 8.6 is below, but before reading it you might want to pause for a couple of minutes and think how you would prove it yourself. In particular, try to think of what a reduction from *HALT* to *HALTONZERO* would look like. Doing so is an excellent way to get some initial comfort with the notion of proofs by reduction, which a technique we will be using time and again in this book.

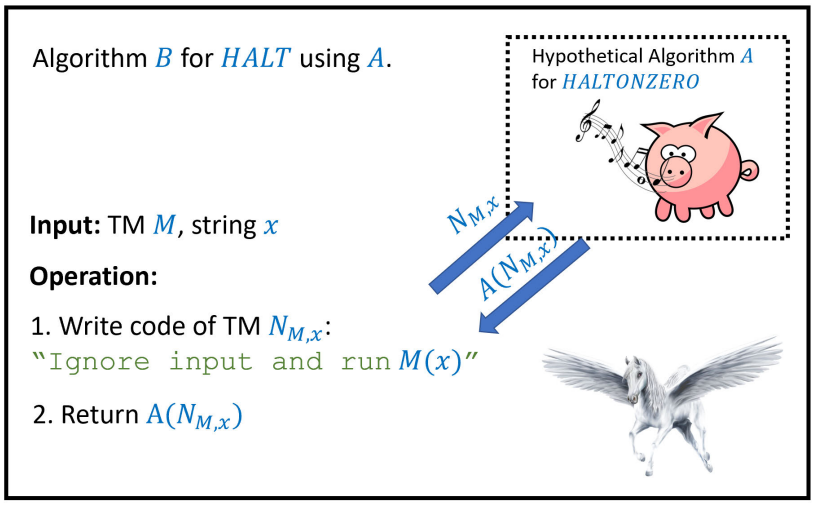


Figure 8.6: To prove Theorem 8.6, we show that *HALTONZERO* is uncomputable by giving a *reduction* from the task of computing *HALT* to the task of computing *HALTONZERO*. This shows that if there was a hypothetical algorithm A computing *HALTONZERO*, then there would be an algorithm B computing *HALT*, contradicting Theorem 8.5. Since neither A nor B actually exists, this is an example of an implication of the form "if pigs could whistle then horses could fly".

Proof of Theorem 8.6. The proof is by reduction from *HALT*, see Fig. 8.6. We will assume, towards the sake of contradiction, that *HALTONZERO* is computable by some algorithm A, and use this

hypothetical algorithm A to construct an algorithm B to compute *HALT*, hence obtaining a contradiction to Theorem 8.5. (As discussed in Section 7.3.3, following our "eat your cake and have it too" paradigm, we just use the generic name "algorithm" rather than worrying whether we model them as Turing machines, NAND-TM programs, NAND-RAM, etc.; this makes no difference since all these models are equivalent to one another.)

Since this is our first proof by reduction from the Halting problem, we will spell it out in more details than usual. Such a proof by reduction consists of two steps:

1. *Description of the reduction:* We will describe the operation of our algorithm B, and how it makes "function calls" to the hypothetical algorithm A.

2. *Analysis of the reduction:* We will then prove that under the hypothesis that Algorithm A computes *HALTONZERO*, Algorithm B will compute *HALT*.

Our Algorithm B works as follows:

Algorithm B: (*Goal:* Compute *HALT*.)

Input: M, x where M represents a Turing machine and x is a string that represents its input.

Assumption: There is an algorithm A such that $A(M) = HALTONZERO(M)$ for every M.

Operation:

1. Let $N_{M,x}$ denote the Turing machine that does the following: *"on input $z \in \{0,1\}^*$, evaluate M on the input x and return the result"*.
2. Return $y = A(N_x)$.

That is, on input a pair (M, x) the algorithm B uses this pair to construct a Turing machine $N_{M,x}$, feeds this machine to A, and outputs the result. The machine $N_{M,x}$ ignores its input z and simply runs M on x.

In pseudocode, the program $N_{M,x}$ will look something like the following:

```
def N(z):
    M = r'.......'
    # a string constant containing desc. of M
    x = r'.......'
    # a string constant containing  x
```

```
    return eval(M,x)
    # note that we ignore the input z
```

That is, if we think of $N_{M,x}$ as a program, then it is a program that contains M and x as "hardwired constants", and given any input z, it simply ignores the input and always returns the result of evaluating M on x.

The algorithm B does *not* actually execute the machine $N_{M,x}$. B merely writes down the description of $N_{M,x}$ as a string (just as we did above) and feeds this string as input to A.

The above completes the *description* of the reduction. The *analysis* is obtained by proving the following claim:

Claim: For every strings M, x, z, the machine $N_{M,x}$ constructed by Algorithm B in Step 1 satisfies that $N_{M,x}$ halts on z if and only if the program described by M halts on the input x.

Proof of Claim: Since $N_{M,x}$ ignores its input and evaluates M on x using the universal Turing machine, it will halt on z if and only if M halts on x.

In particular if we instantiate this claim with the input $z = 0$ to $N_{M,x}$, we see that $HALTONZERO(N_{M,x}) = HALT(M, x)$. Thus if the hypothetical algorithm A satisfies $A(M) = HALTONZERO(M)$ for every M then the algorithm B we construct satisfies $B(M, x) = HALT(M, x)$ for every M, x, contradicting the uncomputability of $HALT$.

■



Remark 8.7 — The hardwiring technique. In the proof of Theorem 8.6 we used the technique of "hardwiring" an input x to a program/machine P. That is, modifying a program P that it uses "hardwired constants" for some of all of its input. This technique is quite common in reductions and elsewhere, and we will often use it again in this course.

8.5 RICE'S THEOREM AND THE IMPOSSIBILITY OF GENERAL SOFTWARE VERIFICATION

The uncomputability of the Halting problem turns out to be a special case of a much more general phenomenon. Namely, that *we cannot certify semantic properties of general purpose programs*. "Semantic properties" mean properties of the *function* that the program computes, as opposed to properties that depend on the particular syntax used by the program.

An example for a *semantic property* of a program P is the property that whenever P is given an input string with an even number of 1's, it outputs 0. Another example is the property that P will always halt whenever the input ends with a 1. In contrast, the property that a C program contains a comment before every function declaration is not a semantic property, since it depends on the actual source code as opposed to the input/output relation.

Checking semantic properties of programs is of great interest, as it corresponds to checking whether a program conforms to a specification. Alas it turns out that such properties are in general *uncomputable*. We have already seen some examples of uncomputable semantic functions, namely *HALT* and *HALTONZERO*, but these are just the "tip of the iceberg". We start by observing one more such example:

Theorem 8.8 — Computing all zero function. Let $ZEROFUNC : \{0,1\}^* \to \{0,1\}$ be the function such that for every $M \in \{0,1\}^*$, $ZEROFUNC(M) = 1$ if and only if M represents a Turing machine such that M outputs 0 on every input $x \in \{0,1\}^*$. Then *ZEROFUNC* is uncomputable.



Despite the similarity in their names, *ZEROFUNC* and *HALTONZERO* are two different functions. For example, if M is a Turing machine that on input $x \in \{0,1\}^*$, halts and outputs the OR of all of x's coordinates, then $HALTONZERO(M) = 1$ (since M does halt on the input 0) but $ZEROFUNC(M) = 0$ (since M does not compute the constant zero function).

Proof of Theorem 8.8. The proof is by reduction to *HALTONZERO*. Suppose, towards the sake of contradiction, that there was an algorithm A such that $A(M) = ZEROFUNC(M)$ for every $M \in \{0,1\}^*$. Then we will construct an algorithm B that solves *HALTONZERO*, contradicting Theorem 8.6.

Given a Turing machine N (which is the input to *HALTONZERO*), our Algorithm B does the following:

1. Construct a Turing Machine M which on input $x \in \{0,1\}^*$, first runs $N(0)$ and then outputs 0.
2. Return $A(M)$.

Now if N halts on the input 0 then the Turing machine M computes the constant zero function, and hence under our assumption that A computes *ZEROFUNC*, $A(M) = 1$. If N does not halt on the input 0, then the Turing machine M will not halt on any input, and

so in particular will *not* compute the constant zero function. Hence under our assumption that A computes *ZEROFUNC*, $A(M) = 0$. We see that in both cases, $ZEROFUNC(M) = HALTONZERO(N)$ and hence the value that Algorithm B returns in step 2 is equal to $HALTONZERO(N)$ which is what we needed to prove.

■

Another result along similar lines is the following:

Theorem 8.9 — Uncomputability of verifying parity. The following function is uncomputable

$$COMPUTES\text{-}PARITY(P) = \begin{cases} 1 & P \text{ computes the parity function} \\ 0 & \text{otherwise} \end{cases} \tag{8.3}$$



We leave the proof of Theorem 8.9 as an exercise (Exercise 8.4). I strongly encourage you to stop here and try to solve this exercise.

8.5.1 Rice's Theorem

Theorem 8.9 can be generalized far beyond the parity function. In fact, this generalization rules out verifying any type of semantic specification on programs. We define a *semantic specification* on programs to be some property that does not depend on the code of the program but just on the function that the program computes.

For example, consider the following two C programs

```
int First(int k) {
    return 2*k;
}

int Second(int n) {
    int i = 0;
    int j = 0
    while (j<n) {
        i = i + 2;
        j=  j + 1;
    }
    return i;
}
```

`First` and `Second` are two distinct C programs, but they compute the same function. A *semantic* property, would be either *true* for both

programs or *false* for both programs, since it depends on the *function* the programs compute and not on their code. An example for a semantic property that both `First` and `Second` satisfy is the following: *"The program P computes a function f mapping integers to integers satisfying that $f(n) \geq n$ for every input n".*

A property is *not semantic* if it depends on the *source code* rather than the input/output behavior. For example, properties such as "the program contains the variable `k`" or "the program uses the `while` operation" are not semantic. Such properties can be true for one of the programs and false for the others.[6] Formally, we define semantic properties as follows:

Definition 8.10 — Semantic properties. A pair of Turing machines M and M' are *functionally equivalent* if for every $x \in \{0,1\}^*$, $M(x) = M'(x)$. [7]

A function $F : \{0,1\}^* \to \{0,1\}$ is *semantic* if for every pair of strings M, M' that represent functionally equivalent Turing machines, $F(M) = F(M')$.

There are two trivial examples of semantic functions: the constant one function and the constant zero function. (For example, if $F(M) = 0$ for every M then clearly $F(M) = F(M')$ for every functionally equivalent M and M'.) Here is a non-trivial example

Solved Exercise 8.1 — $ZEROFUNC$ is semantic. Prove that the function *ZEROFUNC* is semantic.

Solution:

Recall that $ZEROFUNC(M) = 1$ if and only if $M(x) = 0$ for every $x \in \{0,1\}^*$. If M and M' are functionally equivalent, then for every x, $M(x) = M'(x)$. Hence $ZEROFUNC(M) = 1$ if and only if $ZEROFUNC(M') = 1$.

Often the properties of programs that we are most interested in computing are the *semantic* ones, since we want to understand the programs' functionality. Unfortunately, Rice's Theorem tells us that these properties are all uncomputable:

Theorem 8.11 — Rice's Theorem. Let $F : \{0,1\}^* \to \{0,1\}$. If F is semantic and non-trivial then it is uncomputable.

Proof Idea:

The idea behind the proof is to show that every semantic non-trivial function F is at least as hard to compute as *HALTONZERO*.

[6] Properties such as "the program contains the variable `k`" are sometimes known as *syntactic* properties. These terms are used beyond the realm of programming languages: a famous example of a syntactically correct but semantically meaningless sentence in English is Chomsky's "Colorless green ideas sleep furiously." However, formally defining "syntactic properties" is rather subtle and we will not use this terminology in this book, sticking to the terms "semantic" and "non semantic" only.

[7] In particular this means that if M does not halt on x, and hence $M(x) = \perp$, then $M'(x) = \perp$ as well.

This will conclude the proof since by Theorem 8.6, *HALTONZERO* is uncomputable. If a function F is non trivial then there are two machines M_0 and M_1 such that $F(M_0) = 0$ and $F(M_1) = 1$. So, the goal would be to take a machine N and find a way to map it into a machine $M = R(N)$, such that **(i)** if N halts on zero then M is functionally equivalent to M_1 and **(ii)** if N does *not* halt on zero then M is functionally equivalent M_0.

Because F is semantic, if we achieved this, then we would be guaranteed that $HALTONZERO(N) = F(R(N))$, and hence would show that if F was computable, then *HALTONZERO* would be computable as well, contradicting Theorem 8.6.

⋆

Proof of Theorem 8.11. We will not give the proof in full formality, but rather illustrate the proof idea by restricting our attention to a particular semantic function F. However, the same techniques generalize to all possible semantic functoins. Define $MONOTONE : \{0,1\}^* \to \{0,1\}$ as follows: $MONOTONE(M) = 1$ if there does not exist $n \in \mathbb{N}$ and two inputs $x, x' \in \{0,1\}^n$ such that for every $i \in [n]$ $x_i \leq x'_i$ but $M(x)$ outputs 1 and $M(x') = 0$. That is, $MONOTONE(M) = 1$ if it's not possible to find an input x such that flipping some bits of x from 0 to 1 will change M's output in the other direction from 1 to 0. We will prove that *MONOTONE* is uncomputable, but the proof will easily generalize to any semantic function.

We start by noting that *MONOTONE* is neither the constant zero nor the constant one function:

- The machine *INF* that simply goes into an infinite loop on every input satisfies $MONOTONE(INF) = 1$, since *INF* is not defined *anywhere* and so in particular there are no two inputs x, x' where $x_i \leq x'_i$ for every i but $INF(x) = 0$ and $INF(x') = 1$.
- The machine *PAR* that computes the XOR or parity of its input, is not monotone (e.g., $PAR(1,1,0,0,\ldots,0) = 0$ but $PAR(1,0,0,\ldots,0) = 0$) and hence $MONOTONE(PAR) = 0$.

(Note that *INF* and *PAR* are *machines* and not *functions*.)

We will now give a reduction from *HALTONZERO* to *MONOTONE*. That is, we assume towards a contradiction that there exists an algorithm A that computes *MONOTONE* and we will build an algorithm B that computes *HALTONZERO*. Our algorithm B will work as follows:

Algorithm B:

Input: String N describing a Turing machine. (*Goal:* Compute $HALTONZERO(N)$)

Assumption: Access to Algorithm A to compute *MONOTONE*.

Operation:

1. Construct the following machine M: "On input $z \in \{0,1\}^*$ do: **(a)** Run $M(0)$, **(b)** Return $PAR(z)$".
2. Return $1 - A(M)$.

To complete the proof we need to show that B outputs the correct answer, under our assumption that A computes *MONOTONE*. In other words, we need to show that $HALTONZERO(N) = 1 - MONOTONE(M)$. Suppose that N does *not* halt on zero. In this case the program M constructed by Algorithm B enters into an infinite loop in step **(a)** and will never reach step **(b)**. Hence in this case N is functionally equivalent to *INF*.[8] Thus in this case, $MONOTONE(N) = MONOTONE(INF) = 1$.

If N *does* halt on zero, then step **(a)** in M will eventually conclude and M's output will be determined by step **(b)**, where it simply outputs the parity of its input. Hence in this case, M computes the non-monotone parity function (i.e., is functionally equivalent to PAR), and so we get that $MONOTONE(M) = MONOTONE(PAR) = 0$. In both cases, $MONOTONE(M) = 1 - HALTONZERO(N)$, which is what we wanted to prove.

An examination of this proof shows that we did not use anything about *MONOTONE* beyond the fact that it is semantic and non-trivial. For every semantic non-trivial F, we can use the same proof, replacing *PAR* and *INF* with two machines M_0 and M_1 such that $F(M_0) = 0$ and $F(M_1) = 1$. Such machines must exist if F is non trivial.

■

[8] The machine N is not the same machine as *INF*: its description or *code* is different. But it does have the same input/output behavior (in this case) of never halting on any input. Also, while the program M will go into an infinite loop on every input, Algorithm B never actually runs M: it only produces its code and feeds it to A. Hence Algorithm B will *not* enter into an infinite loop even in this case.

R

Remark 8.12 — Semantic is not the same as uncomputable. Rice's Theorem is so powerful and such a popular way of proving uncomputability that people sometimes get confused and think that it is the *only* way to prove uncomputability. In particular, a common misconception is that if a function F is *not* semantic then it is *computable*. This is not at all the case.

For example, consider the following function $HALTNOYALE : \{0,1\}^* \to \{0,1\}$. This is a function that on input a string that represents a NAND-TM program P, outputs 1 if and only if both **(i)** P halts on the input 0, and **(ii)** the program P does not con-

tain a variable with the identifier `Yale`. The function *HALTNOYALE* is clearly not semantic, as it will output two different values when given as input one of the following two functionally equivalent programs:

```
Yale[0] = NAND(X[0],X[0])
Y[0] = NAND(X[0],Yale[0])
```

and

```
Harvard[0] = NAND(X[0],X[0])
Y[0] = NAND(X[0],Harvard[0])
```

However, *HALTNOYALE* is uncomputable since every program P can be transformed into an equivalent (and in fact improved `:))`) program P' that does not contain the variable `Yale`. Hence if we could compute *HALTONYALE* then determine halting on zero for NAND-TM programs (and hence for Turing machines as well).

Moreover, as we will see in Chapter 10, there are uncomputable functions whose inputs are not programs, and hence for which the adjective "semantic" is not applicable.

8.5.2 Halting and Rice's Theorem for other Turing-complete models

As we saw before, many natural computational models turn out to be *equivalent* to one another, in the sense that we can transform a "program" of one model (such as a λ expression, or a game-of-life configurations) into another model (such as a NAND-TM program). This equivalence implies that we can translate the uncomputability of the Halting problem for NAND-TM programs into uncomputability for Halting in other models. For example:

Theorem 8.13 — NAND-TM Machine Halting. Let $NANDTMHALT : \{0,1\}^* \to \{0,1\}$ be the function that on input strings $P \in \{0,1\}^*$ and $x \in \{0,1\}^*$ outputs 1 if the NAND-TM program described by P halts on the input x and outputs 0 otherwise. Then $NANDTMHALT$ is uncomputable.

Once again, this is a good point for you to stop and try to prove the result yourself before reading the proof below.

Proof. We have seen in Theorem 6.11 that for every Turing machine M, there is an equivalent NAND-TM program P_M such that for ev-

ery x, $P_M(x) = M(x)$. In particular this means that $HALT(M) = NANDTMHALT(P_M)$.

The transformation $M \mapsto P_M$ that is obtained from the proof of Theorem 6.11 is *constructive*. That is, the proof yields a way to *compute* the map $M \mapsto P_M$. This means that this proof yields a *reduction* from task of computing *HALT* to the task of computing *NANDTMHALT*, which means that since *HALT* is uncomputable, neither is *NANDTMHALT*.

■

The same proof carries over to other computational models such as the λ *calculus*, *two dimensional* (or even one-dimensional) *automata* etc. Hence for example, there is no algorithm to decide if a λ expression evaluates the identity function, and no algorithm to decide whether an initial configuration of the game of life will result in eventually coloring the cell $(0,0)$ black or not.

Indeed, we can generalize Rice's Theorem to all these models. For example, if $F : \{0,1\}^* \to \{0,1\}$ is a non-trivial function such that $F(P) = F(P')$ for every functionally equivalent NAND-TM programs P, P' then F is uncomputable, and the same holds for NAND-RAM programs, λ-expressions, and all other Turing complete models (as defined in Definition 7.23), see also Exercise 8.8.

8.5.3 Is software verification doomed? (discussion)

Programs are increasingly being used for mission critical purposes, whether it's running our banking system, flying planes, or monitoring nuclear reactors. If we can't even give a certification algorithm that a program correctly computes the parity function, how can we ever be assured that a program does what it is supposed to do? The key insight is that while it is impossible to certify that a *general* program conforms with a specification, it is possible to write a program in the first place in a way that will make it easier to certify. As a trivial example, if you write a program without loops, then you can certify that it halts. Also, while it might not be possible to certify that an *artbirary* program computes the parity function, it is quite possible to write a particular program P for which we can mathematically *prove* that P computes the parity. In fact, writing programs or algorithms and providing proofs for their correctness is what we do all the time in algorithms research.

The field of *software verification* is concerned with verifying that given programs satisfy certain conditions. These conditions can be that the program computes a certain function, that it never writes into a dangeours memory location, that is respects certain invariants, and others. While the general tasks of verifying this may be uncomputable, researchers have managed to do so for many inter-

esting cases, especially if the program is written in the first place in a formalism or programming language that makes verification easier. That said, verification, especially of large and complex programs, remains a highly challenging task in practice as well, and the number of programs that have been formally proven correct is still quite small. Moreover, even phrasing the right theorem to prove (i.e., the specification) if often a highly non-trivial endeavor.

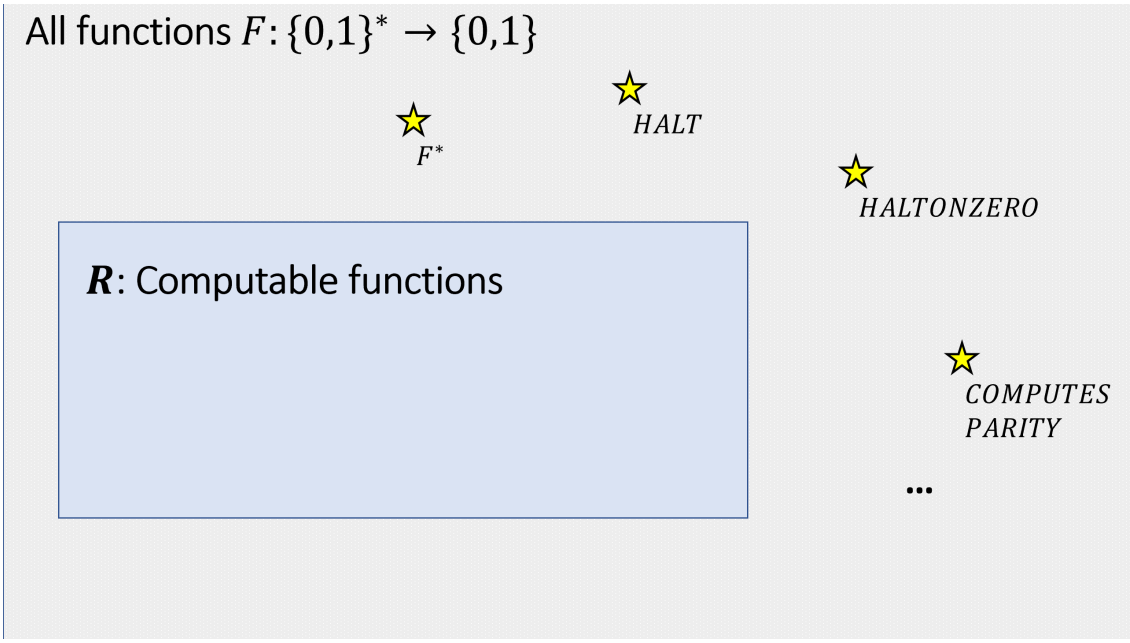


Figure 8.7: The set **R** of computable Boolean functions (Definition 6.4) is a proper subset of the set of all functions mapping $\{0,1\}^*$ to $\{0,1\}$. In this chapter we saw a few examples of elements in the latter set that are not in the former.

Lecture Recap

- There is a *universal* Turing machine (or NAND-TM program) U such that on input a description of a Turing machine M and some input x, $U(M,x)$ halts and outputs $M(x)$ if (and only if) M halts on input x. Unlike in the case of finite computation (i.e., NAND-CIRC programs / circuits), the input to the program U can be a machine M that has more states than U itself.
- Unlike the finite case, there are actually functions that are *inherently uncomputable* in the sense that they cannot be computed by *any* Turing machine.
- These include not only some "degenerate" or "esoteric" functions but also functions that people have deeply care about and conjectured that could be computed.

- If the Church-Turing thesis holds then a function F that is uncomputable according to our definition cannot be computed by any means in our physical world.

8.6 EXERCISES

Exercise 8.1 — NAND-RAM Halt. Let $NANDRAMHALT : \{0,1\}^* \to \{0,1\}$ be the function such that on input (P,x) where P represents a NAND-RAM program, $NANDRAMHALT(P,x) = 1$ iff P halts on the input x. Prove that *NANDRAMHALT* is uncomputable.

Exercise 8.2 — Timed halting. Let $TIMEDHALT : \{0,1\}^* \to \{0,1\}$ be the function that on input (a string representing) a triple (M,x,T), $TIMEDHALT(M,x,T) = 1$ iff the Turing machine M, on input x, halts within at most T steps (where a *step* is defined as one sequence of reading a symbol from the tape, updating the state, writing a new symbol and (potentially) moving the head).

Prove that *TIMEDHALT* is *computable*.

Exercise 8.3 — Space halting (challenging). Let $SPACEHALT : \{0,1\}^* \to \{0,1\}$ be the function that on input (a string representing) a triple (M,x,T), $SPACEHALT(M,x,T) = 1$ iff the Turing machine M, on input x, halts before its head reached the T-th location of its tape. (We don't care how many steps M makes, as long as the head stays inside locations $\{0,\ldots,T-1\}$.)

Prove that *SPACEHALT* is *computable*. See footnote for hint[9]

[9] A machine with alphabet Σ can have at most $|\Sigma|^T$ choices for the contents of the first T locations of its tape. What happens if the machine repeats a previously seen configuration, in the sense that the tape contents, the head location, and the current state, are all identical to what they were in some previous state of the execution?

Exercise 8.4 — Computing parity. Prove Theorem 8.9 without using Rice's Theorem.

Exercise 8.5 — TM Equivalence. Let $EQ : \{0,1\}^* :\to \{0,1\}$ be the function defined as follows, given a string representing a pair (M,M') of Turing machines, $EQ(M,M') = 1$ iff M and M' are functionally equivalent as per Definition 8.10. Prove that *EQ* is uncomputable.

Note that you *cannot* use Rice's Theorem directly, as this theorem only deals with functions that take a single Turing machine as input, and *EQ* takes two machines.

Exercise 8.6 For each of the following two functions, say whether it is computable or not:

1. Given a NAND-TM program P, an input x, and a number k, when we run P on x, does the index variable `i` ever reach k?

2. Given a NAND-TM program P, an input x, and a number k, when we run P on x, does P ever write to an array at index k?

■

Definition 8.14 — Recursively enumerable. Define a function $F : \{0,1\}^* :\to \{0,1\}$ to be *recursively enumerable* if there exists a Turing machine M such that such that for every $x \in \{0,1\}^*$, if $F(x) = 1$ then $M(x) = 1$, and if $F(x) = 0$ then $M(x) = \bot$. (i.e., if $F(x) = 0$ then M does not halt on x.)

1. Prove that every computable F is also recursively enumerable.
2. Prove that there exists F that is not computable but is recursively enumerable. See footnote for hint. [10]
3. Prove that there exists a function $F : \{0,1\}^* \to \{0,1\}$ such that F is not recursively enumerable. See footnote for hint. [11]
4. Prove that there exists a function $F : \{0,1\}^* \to \{0,1\}$ such that F is recursively enumerable but the function $\overline{F}$ defined as $\overline{F}(x) = 1 - F(x)$ is *not* recursively enumerable. See footnote for hint. [12]

[12] *HALT* has this property.

[12] You can either use the diagonalization method to prove this directly or show that the set of all recursively enumerable functions is *countable*.

[12] *HALT* has this property: show that if both *HALT* and $1 - HALT$ were recursively enumerable then *HALT* would be in fact computable.

Exercise 8.7 — Rice's Theorem: standard form. In this exercise we will prove Rice's Theorem in the form that it is typically stated in the literature.

For a Turing machine M, define $L(M) \subseteq \{0,1\}^*$ to be the set of all $x \in \{0,1\}^*$ such that M halts on the input x and outputs 1. (The set $L(M)$ is known in the literature as the *language recognized by* M. Note that M might either output a value other than 1 or not halt at all on inputs $x \notin L(M)$.)

Use Theorem 8.11 to prove that for every $F : \{0,1\}^* \to \{0,1\}$, if **(a)** F is neither the constant zero nor the constant one function, and **(b)** for every M, M' such that $L(M) = L(M')$, $F(M) = F(M')$, then F is uncomputable. See footnote for hint.[13]

[13] Show that any F satisfying **(b)** must be semantic.

■

Exercise 8.8 — Rice's Theorem for general Turing-equivalent models (optional). Let $\mathcal{F}$ be the set of all partial functions from $\{0,1\}^*$ to $\{0,1\}$ and $\mathcal{M} : \{0,1\}^* \to \mathcal{F}$ be a Turing-equivalent model as defined in Definition 7.23. We define a function $F : \{0,1\}^* \to \{0,1\}$ to be *$\mathcal{M}$-semantic* if there exists some $\mathcal{G} : \mathcal{F} \to \{0,1\}$ such that $F(P) = \mathcal{G}(\mathcal{M}(P))$ for every $P \in \{0,1\}^*$.

Prove that for every $\mathcal{M}$-semantic $F : \{0,1\}^* \to \{0,1\}$ that is neither the constant one nor the constant zero function, F is uncomputable.

■

8.7 BIBLIOGRAPHICAL NOTES

Section 7.2 in [MM11] gives a highly recommended overview of uncomputability. Gödel, Escher, Bach [Hof99] is a classic popular science book that touches on uncomputability, and unprovability, and specifically Gödel's Theorem that we will see in Chapter 10. See also the recent book by Holt [Hol18].

The history of the definition of a function is intertwined with the development of mathematics as a field. For many years, a function was identified (as per Euler's quote above) with the means to calculate the output from the input. In the 1800's, with the invention of the Fourier series and with the systematic study of continuity and differentiability, people have starting looking at more general kinds of functions, but the modern definition of a function as an arbitrary mapping was not yet universally accepted. For example, in 1899 Poincare wrote *"we have seen a mass of bizarre functions which appear to be forced to resemble as little as possible honest functions which serve some purpose. ... they are invented on purpose to show that our ancestor's reasoning was at fault, and we shall never get anything more than that out of them"*. Some of this fascinating history is discussed in [**grabiner2005the** ; Gra83; Kle91; Lüt02].

The existence of a universal Turing machine, and the uncomputability of *HALT* was first shown by Turing in his seminal paper [Tur37], though closely related results were shown by Church a year before. These works built on Gödel's 1931 *incompleteness theorem* that we will discuss in Chapter 10.

Adam Yedidia has written software to help in producing universal Turing machines with a small number of states. This is related to the recreational pastime of "Code Golfing" which is about solving a certain computational task using the as short as possible program.

The diagonalization argument used to prove uncomputability of F^* is derived from Cantor's argument for the uncountability of the reals discussed in Chapter 2.

Rice's Theorem was proven in [Ric53]. It is typically stated in a form somewhat different than what we used, see Exercise 8.7.

We do not discuss in the chapter the concept of *recursively enumerable* languages, but it is covered briefly in Definition 8.14. As usual, we use function, as opposto language, notation.

Learning Objectives:

- See that Turing completeness is not always a good thing
- Two important examples of non-Turing-complete, always-halting formalisms: *regular expressions* and *context-free grammars*.
- The pumping lemmas for both these formalisms, and examples of non regular and non context-free functions.
- Examples of computable and uncomputable *semantic properties* of regular expressions and context-free grammars.

9
Restricted computational models

> *"Happy families are all alike; every unhappy family is unhappy in its own way"*, Leo Tolstoy (opening of the book "Anna Karenina").

We have seen that a great many models of computation are *Turing equivalent*, including Turing machines, NAND-TM/NAND-RAM programs, standard programming languages such as C/Python/-Javascript etc.., and other models such as the λ calculus and even the game of life. The flip side of this is that for all these models, Rice's theorem (Theorem 8.11) holds as well, which means that any semantic property of programs in such a model is *uncomputable*.

The uncomputability of halting and other semantic specification problems for Turing equivalent models motivates **restricted computational models** that are (**a**) powerful enough to capture a set of functions useful for certain applications but (**b**) weak enough that we can still solve semantic specification problems on them. In this chapter we discuss several such examples.

9.1 TURING COMPLETENESS AS A BUG

We have seen that seemingly simple computational models or systems can turn out to be Turing complete. The following webpage lists several examples of formalisms that "accidentally" turned out to Turing complete, including supposedly limited languages such as the C preprocessor, CSS, SQL, sendmail configuration, as well as games such as Minecraft, Super Mario, and the card game "Magic: The gathering". Turing completeness is not always a good thing, as it means that such formalisms can give rise to arbitrarily complex behavior. For example, the postscript format (a precursor of PDF) is a Turing-complete programming language meant to describe documents for printing. The expressive power of postscript can allow for short descriptions of very complex images, but it also gave rise to some nasty surprises, such as

the attacks described in this page ranging from using infinite loops as a denial of service attack, to accessing the printer's file system.

■ **Example 9.1 — The DAO Hack.** An interesting recent example of the pitfalls of Turing-completeness arose in the context of the cryptocurrency Ethereum. The distinguishing feature of this currency is the ability to design "smart contracts" using an expressive (and in particular Turing-complete) programming language. In our current "human operated" economy, Alice and Bob might sign a contract to agree that if condition X happens then they will jointly invest in Charlie's company. Ethereum allows Alice and Bob to create a joint venture where Alice and Bob pool their funds together into an account that will be governed by some program P that decides under what conditions it disburses funds from it. For example, one could imagine a piece of code that interacts between Alice, Bob, and some program running on Bob's car that allows Alice to rent out Bob's car without any human intervention or overhead.

Specifically Ethereum uses the Turing-complete programming language solidity which has a syntax similar to JavaScript. The flagship of Ethereum was an experiment known as The "Decentralized Autonomous Organization" or The DAO. The idea was to create a smart contract that would create an autonomously run decentralized venture capital fund, without human managers, where shareholders could decide on investment opportunities. The DAO was at the time the biggest crowdfunding success in history. At its height the DAO was worth 150 million dollars, which was more than ten percent of the total Ethereum market. Investing in the DAO (or entering any other "smart contract") amounts to providing your funds to be run by a computer program. i.e., "code is law", or to use the words the DAO described itself: *"The DAO is borne from immutable, unstoppable, and irrefutable computer code"*. Unfortunately, it turns out that (as we saw in Chapter 8) understanding the behavior of computer programs is quite a hard thing to do. A hacker (or perhaps, some would say, a savvy investor) was able to fashion an input that caused the DAO code to enter into an infinite recursive loop in which it continuously transferred funds into the hacker's account, thereby cleaning out about 60 million dollars out of the DAO. While this transaction was "legal" in the sense that it complied with the code of the smart contract, it was obviously not what the humans who wrote this code had in mind. The Ethereum community struggled with the response to this attack. Some tried to the "Robin Hood" approach of using the

same loophole to drain the DAO funds into a secure account, but it only had limited success. Eventually, the Ethereum community decided that the code can be mutable, stoppable, and refutable. Specifically, the Ethereum maintainers and miners agreed on a "hard fork" (also known as a "bailout") to revert history to before the hacker's transaction occurred. Some community members strongly opposed this decision, and so an alternative currency called Ethereum Classic was created that preserved the original history.

9.2 REGULAR EXPRESSIONS

Searching for a piece of text is a common task in computing. At its heart, the *search problem* is quite simple. We have a collection $X = \{x_0, \ldots, x_k\}$ of strings (e.g., files on a hard-drive, or student records in a database), and the user wants to find out the subset of all the $x \in X$ that are *matched* by some pattern (e.g., all files whose names end with the string `.txt`). In full generality, we can allow the user to specify the pattern by specifying a (computable) *function* $F : \{0,1\}^* \rightarrow \{0,1\}$, where $F(x) = 1$ corresponds to the pattern matching x. That is, the user provides a *program* P in some Turing-complete programming language such as *Python*, and the system will return all the $x \in X$ such that $P(x) = 1$. For example, one could search for all text files that contain the string `important document` or perhaps (letting P correspond to a neural-network based classifier) all images that contain a cat. However, we don't want our system to get into an infinite loop just trying to evaluate the program P!

Because the Halting problem for Turing-complete computational models is uncomputable, we cannot in general verify that a given program P will halt on a given input. For this reason, typical systems for searching files or databases do *not* allow users to specify the patterns using full-fledged programming languages. Rather, such systems use *restricted computational models* that on the one hand are *rich enough* to capture many of the queries needed in practice (e.g., all filenames ending with `.txt`, or all phone numbers of the form `(617) xxx-xxxx`), but on the other hand are *restricted* enough so that they cannot result in an infinite loop.

One of the most popular such computational models is regular expressions. If you ever used an advanced text editor, a command line shell, or have done any kind of manipulations of text files, then you have probably come across regular expressions.

A *regular expression* over some alphabet Σ is obtained by combining elements of Σ with the operation of concatenation, as well as $|$ (corresponding to *or*) and $*$ (corresponding to repetition zero or more

times).[1] For example, the following regular expression over the alphabet $\{0,1\}$ corresponds to the set of all strings $x \in \{0,1\}^*$ where every digit is repeated at least twice:

$$(00(0^*)|11(1^*))^* \; . \tag{9.1}$$

The following regular expression over the alphabet $\{a,\ldots,z,0,\ldots,9\}$ corresponds to the set of all strings that consist of a sequence of one or more of the letters a-d followed by a sequence of one or more digits (without a leading zero):

$$(a|b|c|d)(a|b|c|d)^*(1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)^* \; . \tag{9.2}$$

Formally, regular expressions are defined by the following recursive definition:[2]

Definition 9.2 — Regular expression. A *regular expression* e over an alphabet Σ is a string over $\Sigma \cup \{(,),|,*,\emptyset,\text{""}\}$ that has one of the following forms:

1. $e = \sigma$ where $\sigma \in \Sigma$
2. $e = (e'|e'')$ where e', e'' are regular expressions.
3. $e = (e')(e'')$ where e', e'' are regular expressions. (We often drop the parenthesis when there is no danger of confusion and so write this as $e'\ e''$.)
4. $e = (e')^*$ where e' is a regular expression.

Finally we also allow the following "edge cases": $e = \emptyset$ and $e = \text{""}$.[3]

We will drop parenthesis when they can be inferred from the context. We also use the convention that OR and concatenation are left-associative, and give higher precedence to $*$, then concatenation, and then OR. Thus for example we write $00^*|11$ instead of $((0)(0^*))|((1)(1))$.

Every regular expression e corresponds to a function $\Phi_e : \Sigma^* \to \{0,1\}$ where $\Phi_e(x) = 1$ if x *matches* the regular expression. For example, if $e = (00|11)^*$ then $\Phi_e(110011) = 1$ but $\Phi_e(101) = 0$ (can you see why?).

P

The formal definition of Φ_e is one of those definitions that is more cumbersome to write than to grasp. Thus it might be easier for you to first work it out on your

[1] Common implementations of regular expressions in programming languages and shells typically include some extra operations on top of | and $*$, but these operations can be implemented as "syntactic sugar" using the operators | and $*$.

[2] We have seen recursive definitions before in the setting of λ expressions (Definition 7.6). In a *recursive definition* we start by defining the base case of the simplest regular expressions, and then describe how we can build more complex expressions from simpler ones.

[3] These are the regular expressions corresponding to accepting no strings, and accepting only the empty string respectively.

own and then check that your definition matches what is written below.

Definition 9.3 — Matching a regular expression. Let e be a regular expression over the alphabet Σ. The function $\Phi_e : \Sigma^* \to \{0,1\}$ is defined as follows:

1. If $e = \sigma$ then $\Phi_e(x) = 1$ iff $x = \sigma$.
2. If $e = (e'|e'')$ then $\Phi_e(x) = \Phi_{e'}(x) \vee \Phi_{e''}(x)$ where $\vee$ is the OR operator.
3. If $e = (e')(e'')$ then $\Phi_e(x) = 1$ iff there is some $x', x'' \in \Sigma^*$ such that x is the concatenation of x' and x'' and $\Phi_{e'}(x') = \Phi_{e''}(x'') = 1$.
4. If $e = (e')*$ then $\Phi_e(x) = 1$ iff there are is $k \in \mathbb{N}$ and some $x_0, \ldots, x_{k-1} \in \Sigma^*$ such that x is the concatenation $x_0 \cdots x_{k-1}$ and $\Phi_{e'}(x_i) = 1$ for every $i \in [k]$.
5. Finally, for the edge cases $\Phi_\emptyset$ is the constant zero function, and $\Phi_{\text{""}}$ is the function that only outputs 1 on the empty string "".

We say that a regular expression e over Σ *matches* a string $x \in \Sigma^*$ if $\Phi_e(x) = 1$. We say that a function $F : \Sigma^* \to \{0,1\}$ is *regular* if $F = \Phi_e$ for some regular expression e. [4]

The definitions above are not inherently difficult, but are a bit cumbersome. So you should pause here and go over it again until you understand why it corresponds to our intuitive notion of regular expressions. This is important not just for understanding regular expressions themselves (which are used time and again in a great many applications) but also for getting better at understanding recursive definitions in general.

■ **Example 9.4 — A regular function.** Let $\Sigma = \{a, b, c, d, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ and $F : \Sigma^* \to \{0,1\}$ be the function such that $F(x)$ outputs 1 iff x consists of one or more of the letters a-d followed by a sequence of one or more digits (without a leading zero). Then F is a regular function, since $F = \Phi_e$ where

$$e = (a|b|c|d)(a|b|c|d)^*(0|1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)^* \quad (9.3)$$

[4] We use *function notation* in this book, but other texts often use the notion of *languages*, which are sets of strings. We say that a language $L \subseteq \Sigma^*$ is *regular* if and only if the corresponding function F_L is regular, where $F_L : \Sigma^* \to \{0,1\}$ is the function that outputs 1 on x iff $x \in L$.

is the expression we saw in (9.2).

If we wanted to verify, for example, that $\Phi_e(abc12078) = 1$, we can do so by noticing that the expression $(a|b|c|d)$ matches the string a, $(a|b|c|d)^*$ matches bc, $(0|1|2|3|4|5|6|7|8|9)$ matches the string 1, and the expression $(0|1|2|3|4|5|6|7|8|9)^*$ matches the string 2078. Each one of those boils down to a simpler expression. For example, the expression $(a|b|c|d)^*$ matches the string bc because both of the one-character strings b and c are matched by the expression $a|b|c|d$.

R

Remark 9.5 — Binary alphabet. Regular expression can be defined over any finite alphabet Σ, but as usual, we will focus our attention on the *binary case*, where $\Sigma = \{0,1\}$. Most (if not all) of the theoretical and practical general insights about regular expressions can be gleaned from studying the binary case.

We can think of regular expressions as a type of "programming language". That is, we can think of a regular expression e over the alphabet Σ as a program that computes the function $\Phi_e : \Sigma^* \to \{0,1\}$.[5] This "regular expression programming language" is simpler than general programming languages, in the sense that for every regular expression e, the function Φ_e is computable (and so in particular can be evaluated by an always-halting Turing machine).

[5] Regular expressions (and context free grammars, which we'll see below) are also often thought of as *generative models*, since you can think of them as giving a recipe how to generate strings that match them.

Theorem 9.6 — Regular expression always halt. For every regular expression e over $\{0,1\}$, the function $\Phi_e : \{0,1\}^* \to \{0,1\}$ is computable.[6]

That is, there is a Turing machine M such that for every $x \in \{0,1\}^*$, on input x, M halts with the output $\Phi_e(x)$.

[6] We state this theorem for regular expressions over the binary alphabet $\{0,1\}$, but it generalizes to any finite alphabet Σ.

Proof Idea:

The proof relies on the observation that Definition 9.3 actually specifies a recursive algorithm for *computing* Φ_e. Specifically, each one of our operations -concatenation, OR, and star- can be thought of as reducing the task of testing whether an expression e matches a string x to testing whether some sub-expressions of e match substrings of x. Since these sub-expressions are always shorter than the original expression, this yields a recursive algorithm for checking if e matches x which will eventually terminate at the base cases of the expressions that correspond to a single symbol or the empty string.

⋆

Proof of Theorem 9.6. Definition 9.3 gives a way of recursively computing Φ_e. The key observation is that in our recursive definition of regular expressions, whenever e is made up of one or two expressions e', e'' then these two regular expressions are *smaller* than e, and eventually (when they have size 1) then they must correspond to the non-recursive case of a single alphabet symbol.

Therefore, we can prove the theorem by induction over the length m of e (i.e., the number of symbols in the string e, also denoted as $|e|$). For $m = 1$, e is either a single alphabet symbol, "" or $\emptyset$, and so computing the function Φ_e is straightforward. In the general case, for $m = |e|$ we assume by the induction hypothesis that we have proven the theorem for all expressions of length smaller than m. Now, such an expression of length larger than one can obtained one of three cases using the OR, concatenation, or star operations. We now show that Φ_e will be computable in all these cases:

Case 1: $e = (e'|e'')$ where e', e'' are shorter regular expressions.

In this case by the inductive hypothesis we can compute $\Phi_{e'}$ and $\Phi_{e''}$ and so can compute $\Phi_e(x)$ as $\Phi_{e'}(x) \vee \Phi_{e''}(x)$ (where $\vee$ is the OR operator).

Case 2: $e = (e')(e'')$ where e', e'' are regular expressions.

In this case by the inductive hypothesis we can compute $\Phi_{e'}$ and $\Phi_{e''}$ and so can compute $\Phi_e(x)$ as

$$\bigvee_{i=0}^{|x|-1} (\Phi_{e'}(x_0 \cdots x_{i-1}) \wedge \Phi_{e''}(x_i \cdots x_{|x|-1})) \tag{9.4}$$

where $\wedge$ is the AND operator and for $i < j$, $x_j \cdots x_i$ refers to the empty string.

Case 3: $e = (e')*$ where e' is a regular expression.

In this case by the inductive hypothesis we can compute $\Phi_{e'}$ and so we can compute $\Phi_e(x)$ by enumerating over all k from 1 to $|x|$, and all ways to write x as the concatenation of k strings $x_0 \cdots x_{k-1}$ (we can do so by enumerating over all possible $k-1$ positions in which one string stops and the other begins). If for one of those partitions, $\Phi_{e'}(x_0) = \cdots = \Phi_{e'}(x_{k-1}) = 1$ then we output 1. Otherwise we output 0.

These three cases exhaust all the possibilities for an expression of length larger than one, and hence this completes the proof.

■

9.3 DETERMINISTIC FINITE AUTOMATA, AND EFFICIENT MATCHING OF REGULAR EXPRESSIONS (OPTIONAL)

The proof of Theorem 9.6 gives a recursive algorithm to evaluate whether a given string matches or not a regular expression. But it is not a very efficient algorithm.

However, it turns out that there is a much more efficient algorithm that can match regular expressions in *linear* (i.e., $O(n)$) time. Since we have not yet covered the topics of time and space complexity, we describe this algorithm in high level terms, without making the computational model precise, using the colloquial notion of $O(n)$ running time as is used in introduction to programming courses and whiteboard coding interviews. We will see a formal definition of time complexity in Chapter 12.

Theorem 9.7 — Matching regular expressions in linear time. Let e be a regular expression. Then there is an $O(n)$ time algorithm that computes Φ_e.

The implicit constant in the $O(n)$ term of Theorem 9.7 depends on the expression e. Thus, another way to state Theorem 9.7 is that for every expression e, there is some constant c and an algorithm A that computes Φ_e on n-bit inputs using at most $c \cdot n$ steps. This makes sense, since in practice we often want to compute $\Phi_e(x)$ for a small regular expression e and a large document x. Theorem 9.7 tells us that we can do so with running time that scales linearly with the size of the document, even if it has (potentially) worse dependence on the size of the regular expression.

Proof Idea:

The idea is to define a more efficient recursive algorithm, that determines whether e matches a string $x \in \{0,1\}^n$ by reducing this task to determining whether a related expression e' matches $x_0, \ldots, x_{n-1}$. This will result in an expression for the running time of the form $T(n) = T(n-1) + O(1)$ which solves to $T(n) = O(n)$.

⋆

Proof of Theorem 9.7. The central definition for this proof is the notion of a *restriction* of a regular expression. Given a regular expression e over an alphabet Σ and symbol $\sigma \in \Sigma$, we define $e[\sigma]$ to be a regular expression such that $e[\sigma]$ matches a string x if and only if e matches the string $x\sigma$. For example, if e is the regular expression $01|(01)*(01)$ (i.e., one or more occurrences of 01) then $e[1]$ is equal to $0|(01)*0$ and $e[0]$ will be $\emptyset$. (Can you see now?)

For simplicity, from now on we fix our attention to the case that the alphabet Σ is $\{0,1\}$. Given a regular expression e and $\sigma \in \{0,1\}$, we can compute $e[\sigma]$ recursively as follows:

1. If $e = \tau$ for $\tau \in \{0,1\}$ then $e[\sigma] =$ "" if $\tau = \sigma$ and $e[\sigma] = \emptyset$ otherwise.

2. If $e = e'|e''$ then $e[\sigma] = e'[\sigma]|e''[\sigma]$.

3. If $e = e'\ e''$ then $e[\sigma] = e'\ e''[\sigma]$ if e'' can not match the empty string. Otherwise, $e[\sigma] = e'\ e''[\sigma]|e'[\sigma]$

4. If $e = (e')^*$ then $e[\sigma] = (e')^*(e'[\sigma])$.

5. If $e =$ "" or $e = \emptyset$ then $e[\sigma] = \emptyset$.

We let $C(\ell)$ denote the time to compute $e[\sigma]$ for regular expressions of length at most ℓ.[7]

Using this notion of restriction, we can define the following recursive algorithm for regular expression matching:

Algorithm 9.8 — Regular expression matching in linear time.
Input: Regular expression e over $\{0,1\}$ and $x \in \{0,1\}^n$ for $n \in \mathbb{N}$.
Goal: Compute $\Phi_e(x)$
Operation:

1. If $x =$ "" then return 1 if and only if $\Phi_e(\text{""}) = 1$. (This can be either computed directly or using the algorithm of Theorem 9.6 in time which is a constant depending only on the regular expression e.)

2. Otherwise, compute $\Phi_{e[x_{n-1}]}(x_0 \cdots x_{n-1})$ recursively and output the result.

By the definition of a restriction, for every $\sigma \in \{0,1\}$ and $x' \in \{0,1\}^*$, the expression e matches $x'\sigma$ if and only if $e[\sigma]$ matches x'. Hence for every e and $x \in \{0,1\}^n$, $\Phi_{e[x_{n-1}]}(x_0 \cdots x_{n-2}) = \Phi_e(x)$ and Algorithm 9.8 does return the correct answer. The only remaining task is to analyze its *running time*.

Algorithm 9.8 is a recursive algorithm that on input an expression e and a string $x \in \{0,1\}^n$, does some constant time computation and then calls itself on input some expression e' and a string x of length $n-1$. It will terminate after n steps when it reaches a string of length 0. So, to calculate the running time of Algorithm 9.8 we need to analyze the cost of each step.

[7] The value $C(\ell)$ can be shown to be polynomial in ℓ, though this is not important for this theorem, since we only care about the dependence of the time to compute $\Phi_e(x)$ on the length of x and not about the dependence of this time on the length of e.

Specifically, the running time $T(e,n)$ that it takes for Algorithm 9.8 to compute Φ_e for inputs of length n satisfies the recursive equation:

$$T(e,n) = \max\{T(e[0],n-1), T(e[1],n-1)\} + C(|e|) \tag{9.5}$$

where $C(\ell)$, as before, denotes the time to compute $e[\sigma]$ for expressions e of length at most ℓ. (In the base case $n = 0$, $T(e,0)$ is equal to some constant depending only on e.)

To get some intuition for the expression Eq. (9.5), let us open up the recursion for one level, writing $T(e,n)$ as

$$\begin{aligned} T(e,n) = \max\{&T(e[0][0],n-2) + C(|e[0]|),\\ &T(e[0][1],n-2) + C(|e[0]|),\\ &T(e[1][0],n-2) + C(|e[1]|),\\ &T(e[1][1],n-2) + C(|e[1]|)\} + C(|e|)\ . \end{aligned} \tag{9.6}$$

Continuing this way, we can see that $T(e,n) \leq n \cdot C(\ell) + O(1)$ where ℓ is the largest length of any expression e' that we encounter along the way. Therefore, the following claim suffices to show that Algorithm 9.8 runs in linear time:

Claim: Let e be a regular expression over $\{0,1\}$, then there is some constant c such that for every string $\alpha \in \{0,1\}^*$, if we restrict e to α_0, and then to α_1 and so on and so forth, the resulting expression has length at most c.[8]

Proof of claim: For a regular expression e over $\{0,1\}$ and $\alpha \in \{0,1\}^m$, we denote by $e[\alpha]$ the expression $e[\alpha_0][\alpha_1]\cdots[\alpha_{m-1}]$ obtained by restricting e to α_0 and then to α_1 and so on. We let $S(e) = \{e[\alpha] | \alpha \in \{0,1\}^*\}$. We will prove the claim by showing that for every e, the set $S(e)$ is finite, and hence so is the number $c(e)$ which is the maximum length of e' for $e' \in S(e)$.

We prove this by induction on the structure of e. If e is a symbol, the empty string, or the empty set, then this is straightforward to show as the most expressions $S(e)$ can contain are the expression itself, "", and $\emptyset$. Otherwise we split to the two cases **(i)** $e = e'^*$ and **(ii)** $e = e'e''$, where e', e'' are smaller expressions (and hence by the induction hypothesis $S(e')$ and $S(e'')$ are finite). In the case **(i)**, if $e = (e')^*$ then $e[\alpha]$ is either equal to $(e')^* e'[\alpha]$ or it is simply the empty set if $e'[\alpha] = \emptyset$. Since $e'[\alpha]$ is in the set $S(e')$, the number of distinct expressions in $S(e)$ is at most $|S(e')| + 1$. In the case **(ii)**, if $e = e'e''$ then all the restrictions of e to strings α will either have the form $e'e''[\alpha]$ or the

[8] This claim is strongly related to the Myhill-Nerode Theorem. One direction of this theorem can be thought of as saying that if e is a regular expression then there is at most a finite number of strings $z_0, \ldots, z_{k-1}$ such that $\Phi_{e[z_i]} \neq \Phi_{e[z_j]}$ for every $0 \leq i \neq j < k$.

form $e'e''[\alpha]|e'[\alpha']$ where α' is some string such that $\alpha = \alpha'\alpha''$ and $e[\alpha'']$ matches the empty string. Since $e''[\alpha] \in S(e'')$ and $e'[\alpha'] \in S(e')$, the number of the possible distinct expressions of the form $e[\alpha]$ is at most $|S(e'')| + |S(e'')| \cdot |S(e')|$. This completes the proof of the claim.

The bottom line is that while running Algorithm 9.8 on a regular expression e, all the expressions we ever encounter are in the finite set $S(e)$, no matter how large the input x is, and so the running time of Algorithm 9.8 satisfies the equation $T(n) = T(n-1) + C'$ for some constant C' depending on e. This solves to $O(n)$ where the implicit constant in the Oh notation can (and will) depend on e but crucially, not on the length of the input x.

■

9.3.1 Matching regular expressions using constant memory

Theorem 9.7 is already quite impressive, but we can do even better. Specifically, no matter how long the string x is, we can compute $\Phi_e(x)$ by maintaining only a constant amount of memory and moreover making a *single pass* over x. That is, the algorithm will scan the input x once from start to finish, and then determine whether or not x is matched by the expression e. This is important in the common case of trying to match a short regular expression over a huge file or document that might not even fit in our computer's memory. A single-pass constant-memory algorithm is also known as a deterministic finite automaton (DFA). There is a beautiful theory on the properties of DFA's and their connections with regular expressions. In particular, a function is regular *if and only if* it can be computed by a DFA. We start with showing the "only if" direction:

Theorem 9.9 — DFA for regular expression matching. Let e be a regular expression. Then there is an algorithm that on input $x \in \{0,1\}^*$ computes $\Phi_e(x)$ while making a single pass over x and maintaining a constant amount of memory.

Proof Idea:

The idea is to replace the recursive algorithm of Algorithm 9.8 with a dynamic program, using the technique of memoization. If you haven't taken yet an algorithms course, you might not know these techniques. This is OK; while this more efficient algorithm is crucial for the many practical applications of regular expressions, it is not of great importance for this book.

⋆

Proof of Theorem 9.9. We will replace the recursive Algorithm 9.8 with the following iterative algorithm:

Algorithm 9.10 — Constant memory regular expression matching.
Input: Regular expression e over $\{0,1\}$, string $x \in \{0,1\}^n$.
Goals: Compute $\Phi_e(x)$.
Operation:

1. Let $S = S(e)$ be the set $\{e[\alpha] | \alpha \in \{0,1\}^*\}$ as defined in the proof of Theorem 9.7. Note that S is finite and by definition, for every $e' \in S$ and $\sigma \in \{0,1\}$, $e'[\sigma]$ is in S as well.
2. Define a Boolean variable $v_{e'}$ for every $e' \in S$. Initially we set $v_{e'} = 1$ if and only if e' matches the empty string.
3. For $i = 0, \ldots, n-1$ do the following:
   a. *Copy the variables $\{v_{e'}\}$ to temporary variables:* For every $e' \in S$, we set $temp_{e'} = v_{e'}$.
   b. *Update the variables $\{v_{e'}\}$ based on the i-th bit of x:* Let $\sigma = x_i$ and set $v_{e'} = temp_{e'[\sigma]}$ for every $e' \in S$.
4. Output v_e.

Algorithm 9.10 maintains the invariant that at the end of step i, for every $e' \in S$, the variable $v_{e'}$ is equal if and only if e' matches the string $x_0 \cdots x_{i-1}$. In particular, at the very end, v_e is equal to 1 if and only if e matches the full string $x_0 \cdots x_{n-1}$. Algorithm 9.10 only maintains a constant number of variables (as S is finite), and that it proceeds in one linear scan over the input, and so this proves the theorem.

■

9.3.2 Deterministic Finite Automata

There is another way to think about single-pass constant-memory algorithms, which is as *Deterministic Finite Automata*. If an algorithm A uses c bits of memory, then its memory can be in one of $C = 2^c$ states which we can identify with the set $[C] = \{0, \ldots, 2^c - 1\}$. When the algorithm A is in some state $v \in [C]$ and it reads a bit $\sigma = x_i$ of its input, it moves to a new state $w \in [C]$ which we denote by $A(v, \sigma)$. Thus we can think of such an algorithm A as a function mapping $[C] \times \{0,1\}$ to $[C]$.

If we have such a single-pass constant-memory algorithm A then we can describe A's execution on a string $x \in \{0,1\}^n$ as follows:

- A starts in some initial state $v_0 \in [C]$. (Without loss of generality $v_0 = 0$.)
- For $i = 0, \ldots, n-1$: A updates its state by letting $v_{i+1} = A(v_i, x_i)$.
- The final state v_n determines whether A outputs 0 or 1. We let $\mathcal{A} \subseteq [C]$ denote the set of states on which A outputs 1. This is known as the *accepting states*.

■ **Example 9.11 — DFA for XOR.** Here is a DFA for computing the function $XOR : \{0,1\}^* \to \{0,1\}$ that maps x to $\sum_{i \in [|x|]} x_i \mod 2$.

We will have two states: 0 and 1. The set of accepting states is $\{1\}$, and if we are in a state $v \in \{0,1\}$ and read the bit σ, we will transition to the state v if $\sigma = 0$ and to the state $1 - v$ if $\sigma = 1$. In other words, we transition to the state $v \oplus \sigma$. Hence we can think of this algorithm's execution on input $x \in \{0,1\}^n$ as follows:

- Initially set $v_0 = 0$.
- For every $i \in [n]$, let $v_i = v_{i+1} \oplus x_i$.
- Output v_n.

You can verify that the output of this algorithm is $x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1} = XOR(x)$. We can describe this DFA also graphically, see Fig. 9.1.

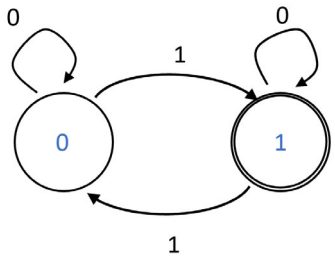


Figure 9.1: A deterministic finite automaton that computes the *XOR* function. It has two states 0 and 1, and when it observes σ it transitions from v to $v \oplus \sigma$.

The formal definition of a DFA is the following:

Definition 9.12 — Deterministic Finite Automaton. A deterministic finite automaton (DFA) with C states over $\{0,1\}$ is a pair $(A, \mathcal{A})$ with $A : [C] \times \{0,1\} \to [C]$ and $\mathcal{A} \subseteq [C]$.

We say that $(A, \mathcal{A})$ *computes* a function $F : \{0,1\}^* \to \{0,1\}$ if for every $n \in \mathbb{N}$ and $x \in \{0,1\}^n$, if we define $v_0 = 0$ and $v_{i+1} = A(v_i, x_i)$ for every $i \in [n]$, then

$$v_n \in \mathcal{A} \Leftrightarrow F(x) = 1 \tag{9.7}$$

P

Our treatment of automata in this book is quite brief. If you find this definition confusing, there are plenty of resources that help you get more comfortable with DFA's. In particular, Chapter 1 of Sipser's book [Sip97] contains an excellent exposition of this material. There are also many websites with online

simulators for automata, as well as translators from regular expressions to automata and vice versa.

A central result in the automata theory is the following:

Theorem 9.13 — DFA and regular expression equivalency. Let $F : \{0,1\}^* \to \{0,1\}$. Then F is regular if and only if there exists a DFA $(A, \mathcal{A})$ that computes F.

Proof Idea:

One direction follows from Theorem 9.9, which shows that for every regular expression e, the function Φ_e can be computed by a DFA (see for example Fig. 9.2). For the other direction, we show that given a DFA $(A, \mathcal{A})$ for every $v, w \in [C]$ we can find a regular expression that would match $x \in \{0,1\}^*$ if an only if the DFA starting in state v, will end up in state w after reading x.

⋆

Proof of Theorem 9.13. Since Theorem 9.9 proves the "only if" direction, we only need to show the "if" direction. Let $(A, \mathcal{A})$ be a DFA with C states that computes the function F. We need to show that F is regular.

For every $v, w \in [C]$ we define $F_{v,w} : \{0,1\}^* \to \{0,1\}$ be the function that maps $x \in \{0,1\}^*$ to 1 if and only if the DFA A, starting at the state v, will reach the state w if it reads x. We will prove that $F_{v,w}$ is regular for every v, w. This will prove the theorem, since by Definition 9.12, $F(x)$ is equal to the OR of $F_{0,w}(x)$ for every $w \in \mathcal{A}$. Hence if we have a regular expression for every function of the form $F_{0,w}$ then (using the | operation of regular expression) we can obtain a regular expression for F as well.

To give a regular expression for functions of the form $F_{v,w}$, it is helpful to think of the DFA $(A, \mathcal{A})$ as an *edge-labeled graph*. That is, consider a directed graph G on the vertices $[C]$, where for every $v \in [C]$ and $\sigma \in \{0,1\}$ we put a directed edge labeled with σ from v to $w = A(v, \sigma)$. (Since it can be the case that $A(v, \sigma) = v$ or that $A(v, 0) = A(v, 1)$, the graph can have self-loops and parallel edges.)

To give regular expressions for the functions $F_{v,w}$, we start by defining the following functions $F^t_{v,w}$: for every $v, w \in [C]$ and $0 \leq t \leq C$, $F^t_{v,w}(x) = 1$ if starting from v and observing x, the automata reaches w *with all intermediate states being in the set* $[t] = \{0, \ldots, t-1\}$*!*. That is, while v, w themselves might be outside $[t]$, $F^t_{v,w}(x) = 1$ only if throughout the execution of the automaton on x (starting from v) it never enters any of these states and still ends up at w. If $t = 0$ then $[t]$ is the empty set, and hence $F^0_{v,w}(x) = 1$ if and only if the automaton

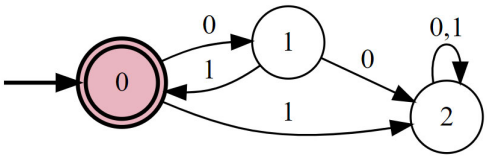


Figure 9.2: A deterministic finite automaton that computes the function $\Phi_{(01)^*}$.

reaches w from v directly, without any intermediate state. This only happens if $|x| = 1$ and there is an edge from v to w that is labeled with x. If $t = C$ then all states are in $[t]$, and hence $F^t_{v,w} = F_{v,w}$.

We will prove the theorem by induction on t, showing that $F^t_{v,w}$ is regular for every v, w and t. For the base case, $F^0_{v,w}$ is regular for every v, w since it can be described one of the expressions $\emptyset$, 0, 1 or $0|1$, depending on whether there are zero, one, or two edges from v to w, and what are their labels.

Asssume, via the induction hypothesis, that for every $v', w' \in [C]$, we have a regular expression $R^t_{v,w}$ that computes $F^t_{v',w'}$. We need to prove that $F^{t+1}_{v,w}$ is regular for every v, w. If the automaton arrives from v to w using the intermediate vertices $[t+1]$, then it visits the t-th vertex zero or more times. If it visits the t-th vertex k times, then each of these k times corresponds to a path from t to t that involves only the vertices $[t]$. Therefore we can compute $F^{t+1}_{v,w}$ using the regular expression

$$R^t_{v,w} \mid R^t_{v,t}(R^t_{t,t})^* R^t_{t,w} \; . \tag{9.8}$$

The first part of the expression corresponds to the strings x that describes paths from v to w that do not involve the state t, and the second part describe paths that involve the state t one or more times (and hence involve travelling from v to t, then potentially returning to t more times, and then going from t to w).

■

9.3.3 Regular functions are closed under complement

Here is an important corollary of Theorem 9.13:

Lemma 9.14 — Regular expressions closed under complement. If $F : \{0,1\}^* \to \{0,1\}$ is regular then so is the function $\overline{F}$, where $\overline{F}(x) = 1 - F(x)$ for every $x \in \{0,1\}^*$.

Proof. If F is regular then by Theorem 9.7 it can be computed by a constant-space one-pass algorithm A. But then the algorithm $\overline{A}$ which does the same computation and outputs the negation of the output of A also utilizes constant space and one pass and computes $\overline{F}$. By Theorem 9.13 this implies that $\overline{F}$ is regular as well.

■

9.4 LIMITATIONS OF REGULAR EXPRESSIONS

The fact that functions computed by regular expressions always halt is one of the reasons why they are so useful. When you make a regular expression search, you are guaranteed that that it will terminate with a result. This is why operating systems and text editors, for example,

often restrict their search interface to regular expressions and don't allow searching by specifying an arbitrary function. But this always-halting property comes at a cost. Regular expressions cannot compute every function that is computable by Turing machines. In fact there are some very simple (and useful!) functions that they cannot compute, such as the following:

Lemma 9.15 — Matching parenthesis. Let $\Sigma = \{\langle, \rangle\}$ and $MATCHPAREN : \Sigma^* \to \{0,1\}$ be the function that given a string of parenthesis, outputs 1 if and only if every opening parenthesis is matched by a corresponding closed one. Then there is no regular expression over Σ that computes *MATCHPAREN*.

Lemma 9.15 is a consequence of the following result known as the *pumping lemma*:

Theorem 9.16 — Pumping Lemma. Let e be a regular expression. Then there is some number n_0 such that for every $w \in \{0,1\}^*$ with $|w| > n_0$ and $\Phi_e(w) = 1$, it holds that we can write $w = xyz$ where $|y| \geq 1$, $|xy| \leq n_0$ and such that $\Phi_e(xy^k z) = 1$ for every $k \in \mathbb{N}$.

Proof Idea:

The idea behind the proof is very simple (see Fig. 9.3). Let n_0 be twice the number of symbols that are used in the expression e, then the only way that there is some w with $|w| > n_0$ and $\Phi_e(w) = 1$ is that e contains the $*$ (i.e. star) operator and that there is a nonempty substring y of w that was matched by $(e')^*$ for some sub-expression e' of e. We can now repeat y any number of times and still get a matching string.

⋆

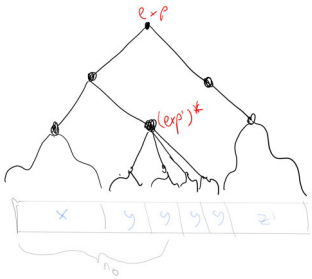


Figure 9.3: To prove the "pumping lemma" we look at a word w that is much larger than the regular expression e that matches it. In such a case, part of w must be matched by some sub-expression of the form $(e')^*$, since this is the only operator that allows matching words longer than the expression. If we look at the "leftmost" such sub-expression and define y^k to be the string that is matched by it, we obtain the partition needed for the pumping lemma.

P

The pumping lemma is a bit cumbersome to state, but one way to remember it is that it simply says the following: *"if a string matching a regular expression is long enough, one of its substrings must be matched using the $*$ operator"*.

Proof of Theorem 9.16. To prove the lemma formally, we use induction on the length of the expression. Like all induction proofs, this is going to be somewhat lengthy, but at the end of the day it directly follows the intuition above that *somewhere* we must have used the star operation. Reading this proof, and in particular understanding how the formal proof below corresponds to the intuitive idea above, is a very good way to get more comfort with inductive proofs of this form.

Our inductive hypothesis is that for an n length expression, $n_0 = 2n$ satisfies the conditions of the lemma. The base case is when the expression is a single symbol or that it is $\emptyset$ or "" in which case the condition is satisfied just because there is no matching string of length more than one. Otherwise, e is of the form **(a)** $e'|e''$, **(b)**, $(e')(e'')$, **(c)** or $(e')^*$ where in all these cases the subexpressions have fewer symbols than e and hence satisfy the induction hypothesis.

In case **(a)**, every string w matching e must match either e' or e''. In the former case, since e' satisfies the induction hypothesis, if $|w| > n_0$ then we can write $w = xyz$ such that xy^kz matches e' for every k, and hence this is matched by e as well.

In case **(b)**, if w matches $(e')(e'')$. then we can write $w = w'w''$ where w' matches e' and w'' matches e''. Again we split to subcases. If $|w'| > 2|e'|$, then by the induction hypothesis we can write $w' = xyz$ of the form above such that xy^kz matches e' for every k and then xy^kzw'' matches $(e')(e'')$. This completes the proof since $|xy| \leq 2|e'|$ and so in particular $|xy| \leq 2(|e'|+|e''|) \leq 2|e$, and hence zw'' can be play the role of z in the proof. Otherwise, if $|w'| \leq 2|e'|$ then since $|w|$ is larger than $2|e|$ and $w = w'w''$ and $e = e'e''$, we get that $|w'| + |w''| > 2(|e'| + |e''|)$. Thus, if $|w'| \leq 2|e'|$ it must be that $|w''| > 2|e''|$ and hence by the induction hypothesis we can write $w'' = xyz$ such that xy^kz matches e'' for every k and $|xy| \leq 2|e''|$. Therefore we get that $w'xy^kz$ matches $(e')(e'')$ for every k and since $|w'| \leq 2|e'|$, $|w'xy| \leq 2(|e'|+|e'|)$ and this completes the proof since $w'x$ can play the role of x in the statement.

Now in the case **(c)**, if w matches $(e')^*$ then $w = w_0 \cdots w_t$ where w_i is a nonempty string that matches e' for every i. If $|w_0| > 2|e'|$ then we can use the same approach as in the concatenation case above. Otherwise, we simply note that if x is the empty string, $y = w_0$, and $z = w_1 \cdots w_t$ then xy^kz will match $(e')^*$ for every k.

■

R

Remark 9.17 — Recursive definitions and inductive proofs. When an object is *recursively defined* (as in the case of regular expressions) then it is natural to prove properties of such objects by *induction*. That is, if we want to prove that all objects of this type have property P, then it is natural to use an inductive steps that says that if o', o'', o''' etc have property P then so is an object o that is obtained by composing them.

Using the pumping lemma, we can easily prove Lemma 9.15:

Proof of Lemma 9.15. Suppose, towards the sake of contradiction, that there is an expression e such that $\Phi_e = \mathit{MATCHPAREN}$. Let

n_0 be the number from Lemma 9.15 and let $w = \langle^{n_0}\rangle^{n_0}$ (i.e., n_0 left parenthesis followed by n_0 right parenthesis). Then we see that if we write $w = xyz$ as in Lemma 9.15, the condition $|xy| \leq n_0$ implies that y consists solely of left parenthesis. Hence the string xy^2z will contain more left parenthesis than right parenthesis. Hence $MATCHPAREN(xy^2z) = 0$ but by the pumping lemma $\Phi_e(xy^2z) = 1$, contradicting our assumption that $\Phi_e = MATCHPAREN$. ■

The pumping lemma is a very useful tool to show that certain functions are *not* computable by a regular language. However, it is *not* an "if and only if" condition for regularity. There are non regular functions which still satisfy the conditions of the pumping lemma. To understand the pumping lemma, it is important to follow the order of quantifiers in Theorem 9.16. In particular, the number n_0 in the statement of Theorem 9.16 depends on the regular expression (in particular we can choose n_0 to be twice the number of symbols in the expression). So, if we want to use the pumping lemma to rule out the existence of a regular expression e computing some function F, we need to be able to choose an appropriate w that can be arbitrarily large and satisfies $F(w) = 1$. This makes sense if you think about the intuition behind the pumping lemma: we need w to be large enough as to force the use of the star operator.

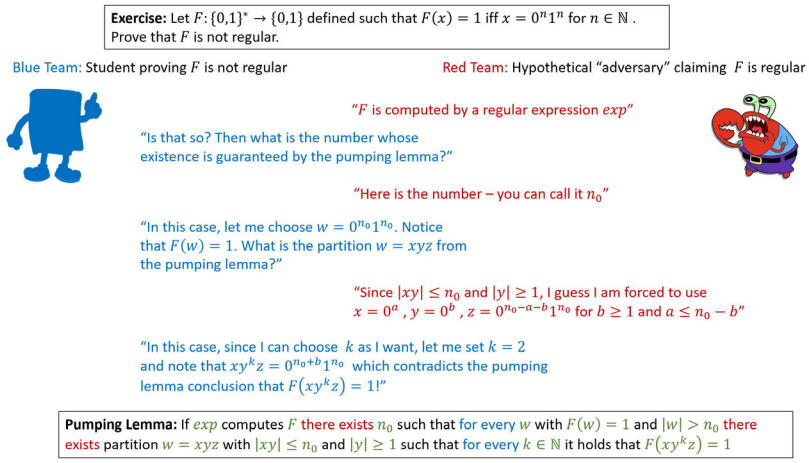


Figure 9.4: A cartoon of a proof using the pumping lemma that a function F is not regular. The pumping lemma states that if F is regular then *there exists* a number n_0 such that *for every* large enough w with $F(w) = 1$, *there exists* a partition of w to $w = xyz$ satisfying certain conditions such that *for every* $k \in \mathbb{N}$, $F(xy^kz) = 1$. You can imagine a pumping-lemma based proof as a game between you and the adversary. Every *there exists* quantifier corresponds to an object you are free to choose on your own (and base your choice on previously chosen objects). Every *for every* quantifier corresponds to an object the adversary can choose arbitrarily (and again based on prior choices) as long as it satisfies the conditions. A valid proof corresponds to a strategy by which no matter what the adversary does, you can win the game by obtaining a contradiction which would be a choice of k that would result in $F(xy^ky) = 0$, hence violating the conclusion of the pumping lemma.

Solved Exercise 9.1 — Palindromes is not regular. Prove that the following function over the alphabet $\{0, 1, ;\}$ is not regular: $PAL(w) = 1$ if and only if $w = u;u^R$ where $u \in \{0,1\}^*$ and u^R denotes u "reversed": the string $u_{|u|-1} \cdots u_0$.[9]

■

[9] The *Palindrome* function is most often defined without an explicit separator character ;, but the version with such a separator is a bit cleaner and so we use it here. This does not make much difference, as one can easily encode the separator as a special binary string instead.

Solution:

We use the pumping lemma. Suppose towards the sake of contradiction that there is a regular expression e computing PAL, and let n_0 be the number obtained by the pumping lemma (Theorem 9.16). Consider the string $w = 0^{n_0};0^{n_0}$. Since the reverse of the all zero string is the all zero string, $PAL(w) = 1$. Now, by the pumping lemma, if PAL is computed by e, then we can write $w = xyz$ such that $|xy| \leq n_0$, $|y| \geq 1$ and $PAL(xy^k z) = 1$ for every $k \in \mathbb{N}$. In particular, it must hold that $PAL(xz) = 1$, but this is a contradiction, since $xz = 0^{n_0-|y|};0^{n_0}$ and so its two parts are not of the same length and in particular are not the reverse of one another.

■

For yet another example of a pumping-lemma based proof, see Fig. 9.4 which illustrates a cartoon of the proof of the non-regularity of the function $F : \{0,1\}^* \to \{0,1\}$ which is defined as $F(x) = 1$ iff $x = 0^n 1^n$ for some $n \in \mathbb{N}$ (i.e., x consists of a string of consecutive zeroes, followed by a string of consecutive ones of the same length).

9.5 OTHER SEMANTIC PROPERTIES OF REGULAR EXPRESSIONS

Regular expressions are widely used beyond just searching. For example, regular expressions are often used to define *tokens* (such as what is a valid variable identifier, or keyword) in programming languages. But they also have other uses. One nice example is the recent work on the NetKAT network programming language. In recent years, the world of networking moved from fixed topologies to "software defined networks", that are run by programmable switches that can implement policies such as "if packet is secured by SSL then forward it to A, otherwise forward it to B". By its nature, one would want to use a formalism for such policies that is guaranteed to always halt (and quickly!) and that where it is possible to answer semantic questions such as "does C see the packets moved from A to B" etc. The NetKAT language uses a variant of regular expressions to achieve precisely that.

Such applications use the fact that because regular expressions are so restricted, we can not just solve the halting problem for them, but also answer other *semantic questions* about regular languages. Such semantic questions would not be solvable for Turing-complete models due to Rice's Theorem (Theorem 8.11). For example, we can tell whether two regular expressions are *equivalent*, as well as whether a regular expression computes the constant zero function.

Theorem 9.18 — Emptiness of regular languages is computable. There is an algorithm that given a regular expression e, outputs 1 if and only if Φ_e is the constant zero function.

Proof Idea:

The idea is that we can directly observe this from the structure of the expression. The only way it will output the constant zero function is if it has the form $\emptyset$ or is obtained by concatenating $\emptyset$ with other expressions.

⋆

Proof of Theorem 9.18. Define a regular expression to be "empty" if it computes the constant zero function. The algorithm simply follows the following rules:

- If an expression has the form σ or "" then it is not empty.
- If e is not empty then $e|e'$ is not empty for every e'.
- If e is not empty then e^* is not empty.
- If e and e' are both not empty then $e\ e'$ is not empty.
- $\emptyset$ is empty.

Using these rules it is straightforward to come up with a recursive algorithm to determine emptiness. We leave verifying the details to the reader.

■

Theorem 9.19 — Equivalence of regular expressions is computable. Let $REGEQ : \{0,1\}^* \to \{0,1\}$ be the function that on input (a string representing) a pair of regular expressions e, e', $REGEQ(e,e') = 1$ if and only if $\Phi_e = \Phi_{e'}$. Then $REGEQ$ is computable.

Proof Idea:

The idea is to show that given a pair of regular expression e and e' we can find an expression e'' such that $\Phi_{e''}(x) = 1$ if and only if $\Phi_e(x) \neq \Phi_(e'')(x)$. Therefore $\Phi_{e''}$ is the constant zero function if and only if e and e' are equivalent, and thus we can test for emptiness of e'' to determine equivalence of e and e'.

⋆

Proof of Theorem 9.19. Theorem 9.18 above is actually a special case of Theorem 9.19, since emptiness is the same as checking equivalence

with the expression $\emptyset$. However we will prove **??** from Theorem 9.18. The idea is that given e and e', we will compute an expression e'' such that $\Phi_{e''}(x) = 1$ if and only if $\Phi_e(x) \neq \Phi_{e'}(x)$. One can see that e is equivalent to e' if and only if e'' is empty.

To show that we can construct such a regular expression, not that for every bits $a, b \in \{0,1\}$, $a \neq b$ if and only if

$$(a \wedge \overline{b}) \vee (\overline{a} \wedge b) \,. \tag{9.9}$$

Hence we need to construct e'' such that for every x,

$$\Phi_{e''}(x) = (\Phi_e(x) \wedge \overline{\Phi_{e'}(x)}) \vee (\overline{\Phi_e(x)} \wedge \Phi_{e'}(x)) \,. \tag{9.10}$$

To construct this expression, we need to show how given any pair of expressions e and e', we can construct expressions $e \wedge e'$ and $\overline{e}$ that compute the functions $\Phi_e \wedge \Phi_{e'}$ and $\overline{\Phi_e}$ respectively. (Computing the expression for $e \vee e'$ is straightforward using the $|$ operation of regular expressions.)

Specifically, by Lemma 9.14, regular functions are closed under negation, which means that for every regular expression e, there is an expression $\overline{e}$ such that $\Phi_{\overline{e}}(x) = 1 - \Phi_e(x)$ for every $x \in \{0,1\}^*$. Now, for every two expression e and e', the expression

$$e \wedge e' = \overline{(\overline{e}|\overline{e'})} \tag{9.11}$$

computes the AND of the two expressions. Given these two transformations, we see that for every regular expressions e and e' we can find a regular expression e'' satisfying (9.10) such that e'' is empty if and only if e and e' are equivalent.

■

9.6 CONTEXT FREE GRAMMARS

If you have ever written a program, you've experienced a *syntax error*. You might also have had the experience of your program entering into an *infinite loop*. What is less likely is that the compiler or interpreter entered an infinite loop when trying to figure out if your program has a syntax error.

When a person designs a programming language, they need to determines its *syntax*. That is, they need to determine which strings corresponds to valid programs, and which ones do not. A compiler or interpreter is given a string x as an input and needs to determine whether x corresponds to a valid program or it contains a syntax error. To ensure that the compiler will always halt in this computation, language designers typically *don't* use a general Turing-complete mechanism to express their syntax but rather a *restricted* computational model. One of the most popular choices for such models is *context free grammars*.

To explain context free grammars, let's begin with a canonical example. Let us try to define a function $ARITH : \Sigma^* \to \{0,1\}$ that takes as input a string x over the alphabet $\Sigma = \{(,),+,-,\times,\div,0,1,2,3,4,5,6,7,8,9\}$ and returns 1 if and only if the string x represents a valid arithmetic expression. Intuitively, we build expressions by applying an operation to smaller expressions, or enclosing them in parenthesis, where the "base case" corresponds to expressions that are simply numbers. A bit more precisely, we can make the following definitions:

- A *digit* is one of the symbols $0,1,2,3,4,5,6,7,8,9$.
- A *number* is a sequence of digits.[10]
- An *operation* is one of $+,-,\times,\div$
- An *expression* has either the form "*number*", the form "*subexpression1 operation subexpression2*", or the form "(*subexpression*)".

[10] For simplicity we drop the condition that the sequence does not have a leading zero, though it is not hard to encode it in a context-free grammar as well.

A context free grammar (CFG) is a formal way of specifying such conditions. We can think of a CFG as a set of rules to *generate* valid expressions. In the example above, there is a rule $expression \Rightarrow expression \times expression$ which tells us that if we have built two valid expressions $exp1$ and $exp2$, then the expression $exp1 \times exp2$ is valid too.

Note that the rules of a context-free grammar are often *recursive*: the rule $expression \Rightarrow expression \times expression$ defines valid expressions in terms of itself. For such a grammar to make sense, it must have also some non-recursive rules, such as the rule $number \Rightarrow 0$.

We now make the formal definition of context-free grammars:

Definition 9.20 — Context Free Grammar. Let Σ be some finite set. A *context free grammar* (*CFG*) *over* Σ is a triple (V,R,s) where:

- V is a set disjoint from Σ of *variables*
- $v \in V$ is the *initial variable*.
- R is a set of *rules*, which are pairs (v,z) with $v \in V$ and $z \in (\Sigma \cup V)^*$. We often write the rule (v,z) as $v \Rightarrow z$ and say that z *can be derived* from v.

■ **Example 9.21 — Context free grammar for arithmetic expressions.** The example above of well-formed arithmetic expressions can be captured formally by the following context free grammar:

- The alphabet Σ is $\{(,),+,-,\times,\div,0,1,2,3,4,5,6,7,8,9\}$
- The variables are $V = \{expression\ ,\ number\ ,\ digit\ ,\ operation\}$.
- The rules correspond the set R containing the following pairs:
  - $operation \Rightarrow +$, $operation \Rightarrow -$, $operation \Rightarrow \times$, $operation \Rightarrow \div$
  - $digit \Rightarrow 0$,..., $digit \Rightarrow 9$
  - $number \Rightarrow digit$
  - $number \Rightarrow digit\ number$
  - $expression \Rightarrow number$
  - $expression \Rightarrow expression\ operation\ expression$
  - $expression \Rightarrow (expression)$
- The starting variable is $expression$

People use many different notations to write context free grammars. One of the most common notations is the Backus–Naur form. In this notation we write a rule of the form $v \Rightarrow a$ (where v is a variable and a is a string) in the form `<v> := a`. If we have several rules of the form $v \mapsto a$, $v \mapsto b$, and $v \mapsto c$ then we can combine them as `<v> := a|b|c`. For example, the Backus-Naur description for the context free grammar of Example 9.21 is the following (using ASCII equivalents for operations):

```
operation  := +|-|*|/
digit      := 0|1|2|3|4|5|6|7|8|9
number     := digit|digit number
expression := number|expression operation
 ↪  expression|(expression)
```

Another example of a context free grammar is the "matching parenthesis" grammar, which can be represented in Backus-Naur as follows:

```
match  := ""|match match|(match)
```

You can verify that a string over the alphabet { `(,)` } can be generated from this grammar (where `match` is the starting expression and `""` corresponds to the empty string) if and only if it consists of a matching set of parenthesis.

9.6.1 Context-free grammars as a computational model

We can think of a CFG over the alphabet Σ as defining a function that maps every string x in Σ^* to 1 or 0 depending on whether x can be

generated by the rules of the grammars. We now make this definition formally.

Definition 9.22 — Deriving a string from a grammar. If $G = (V, R, s)$ is a context-free grammar over Σ, then for two strings $\alpha, \beta \in (\Sigma \cup V)^*$ we say that β *can be derived in one step* from α, denoted by $\alpha \Rightarrow_G \beta$, if we can obtain β from α by applying one of the rules of G. That is, we obtain β by replacing in α one occurrence of the variable v with the string z, where $v \Rightarrow z$ is a rule of G.

We say that β *can be derived* from α, denoted by $\alpha \Rightarrow^*_G \beta$, if it can be derived by some finite number k of steps. That is, if there are $\alpha_1, \ldots, \alpha_{k-1} \in (\Sigma \cup V)^*$, so that $\alpha \Rightarrow_G \alpha_1 \Rightarrow_G \alpha_2 \Rightarrow_G \cdots \Rightarrow_G \alpha_{k-1} \Rightarrow_G \beta$.

We say that $x \in \Sigma^*$ is *matched* by $G = (V, R, s)$ if x can be derived from the starting variable s (i.e., if $s \Rightarrow^*_G x$). We define the *function computed by* (V, R, s) to be the map $\Phi_{V,R,s} : \Sigma^* \to \{0,1\}$ such that $\Phi_{V,R,s}(x) = 1$ iff x is matched by (V, R, s). A function $F : \Sigma^* \to \{0,1\}$ is *context free* if $F = \Phi_{V,R,s}$ for some CFG (V, R, s). [11]

[11] As in the case of Definition 9.3 we can also use *language* rather than *function* notation and say that a language $L \subseteq \Sigma^*$ is *context free* if the function F such that $F(x) = 1$ iff $x \in L$ is context free.

A priori it might not be clear that the map $\Phi_{V,R,s}$ is computable, but it turns out that this is the case.

Theorem 9.23 — Context-free grammars always halt. For every CFG (V, R, s) over $\{0,1\}$, the function $\Phi_{V,R,s} : \{0,1\}^* \to \{0,1\}$ is computable. [12]

[12] As usual we restrict attention to grammars over $\{0,1\}$ although the proof extends to any finite alphabet Σ.

Proof. We only sketch the proof. We start with the observation we can convert every CFG to an equivalent version of *Chomsky normal form*, where all rules either have the form $u \to vw$ for variables u, v, w or the form $u \to \sigma$ for a variable u and symbol $\sigma \in \Sigma$, plus potentially the rule $s \to$ "" where s is the starting variable.

The idea behind such a transformation is to simply add new variables as needed, and so for example we can translate a rule such as $v \to u\sigma w$ into the three rules $v \to ur$, $r \to tw$ and $t \to \sigma$.

Using the Chomsky Normal form we get a natural recursive algorithm for computing whether $s \Rightarrow^*_G x$ for a given grammar G and string x. We simply try all possible guesses for the first rule $s \to uv$ that is used in such a derivation, and then all possible ways to partition x as a concatenation $x = x'x''$. If we guessed the rule and the partition correctly, then this reduces our task to checking whether $u \Rightarrow^*_G x'$ and $v \Rightarrow^*_G x''$, which (as it involves shorter strings) can be done recursively. The base cases are when x is empty or a single symbol, and can be easily handled.

■

R

Remark 9.24 — Parse trees. While we focus on the task of *deciding* whether a CFG matches a string, the algorithm to compute $\Phi_{V,R,s}$ actually gives more information than that. That is, on input a string x, if $\Phi_{V,R,s}(x) = 1$ then the algorithm yields the sequence of rules that one can apply from the starting vertex s to obtain the final string x. We can think of these rules as determining a *tree* with s being the *root* vertex and the sinks (or *leaves*) corresponding to the substrings of x that are obtained by the rules that do not have a variable in their second element. This tree is known as the *parse tree* of x, and often yields very useful information about the structure of x.

Often the first step in a compiler or interpreter for a programming language is a *parser* that transforms the source into the parse tree (also known as the abstract syntax tree). There are also tools that can automatically convert a description of a context-free grammars into a parser algorithm that computes the parse tree of a given string. (Indeed, the above recursive algorithm can be used to achieve this, but there are much more efficient versions, especially for grammars that have particular forms, and programming language designers often try to ensure their languages have these more efficient grammars.)

9.6.2 The power of context free grammars

Context free grammars can capture every regular expression:

Theorem 9.25 — Context free grammars and regular expressions. Let e be a regular expression over $\{0,1\}$, then there is a CFG (V,R,s) over $\{0,1\}$ such that $\Phi_{V,R,s} = \Phi_e$.

Proof. We will prove the theorem by induction on the length of e. If e is an expression of one bit length, then $e = 0$ or $e = 1$, in which case we leave it to the reader to verify that there is a (trivial) CFG that computes it. Otherwise, we fall into one of the following case: **case 1:** $e = e'e''$, **case 2:** $e = e'|e''$ or **case 3:** $e = (e')^*$ where in all cases e', e'' are shorter regular expressions. By the induction hypothesis have grammars (V', R', s') and (V'', R'', s'') that compute $\Phi_{e'}$ and $\Phi_{e''}$ respectively. By renaming of variables, we can also assume without loss of generality that V' and V'' are disjoint.

In case 1, we can define the new grammar as follows: we add a new starting variable $s \notin V \cup V'$ and the rule $s \mapsto s's''$. In case 2, we can define the new grammar as follows: we add a new starting variable

$s \notin V \cup V'$ and the rules $s \mapsto s'$ and $s \mapsto s''$. Case 3 will be the only one that uses *recursion*. As before we add a new starting variable $s \notin V \cup V'$, but now add the rules $s \mapsto$ "" (i.e., the empty string) and also add, for every rule of the form $(s', \alpha) \in R'$, the rule $s \mapsto s\alpha$ to R.

We leave it to the reader as (a very good!) exercise to verify that in all three cases the grammars we produce capture the same function as the original expression. ■

It turns out that CFG's are strictly more powerful than regular expressions. In particular, as we've seen, the "matching parenthesis" function *MATCHPAREN* can be computed by a context free grammar, whereas, as shown in Lemma 9.15, it cannot be computed by regular expressions. Here is another example:

Solved Exercise 9.2 — Context free grammar for palindromes. Let $PAL : \{0,1,;\}^* \to \{0,1\}$ be the function defined in Solved Exercise 9.1 where $PAL(w) = 1$ iff w has the form $u;u^R$. Then PAL can be computed by a context-free grammar ■

Solution:

A simple grammar computing PAL can be described using Backus–Naur notation:

```
start      := ; | 0 start 0 | 1 start 1
```

One can prove by induction that this grammar generates exactly the strings w such that $PAL(w) = 1$. ■

A more interesting example is computing the strings of the form $u;v$ that are *not* palindromes:

Solved Exercise 9.3 — Non palindromes. Prove that there is a context free grammar that computes $NPAL : \{0,1,;\}^* \to \{0,1\}$ where $NPAL(w) = 1$ if $w = u;v$ but $v \neq u^R$. ■

Solution:

Using Backus–Naur notation we can describe such a grammar as follows

```
palindrome      := ; | 0 palindrome 0 | 1 palindrome 1
different       := 0 palindrome 1 | 1 palindrome 0
start           := different | 0 start | 1 start | start
 ↪  0 | start 1
```

In words, this means that we can characterize a string w such that $NPAL(w) = 1$ as having the following form

$$w = \alpha b u; u^R b' \beta \tag{9.12}$$

where α, β, u are arbitrary strings and $b \neq b'$. Hence we can generate such a string by first generating a palindrome $u; u^R$ (`palindrome` variable), then adding either 0 on the right and 1 on the left to get something that is *not* a palindrome (`different` variable), and then we can add arbitrary number of 0's and 1's on either end (the `start` variable).

■

9.6.3 Limitations of context-free grammars (optional)

Even though context-free grammars are more powerful than regular expressions, there are some simple languages that are *not* captured by context free grammars. One tool to show this is the context-free grammar analog of the "pumping lemma" (Theorem 9.16):

Theorem 9.26 — Context-free pumping lemma. Let (V, R, s) be a CFG over Σ, then there is some $n_0 \in \mathbb{N}$ such that for every $x \in \Sigma^*$ with $|x| > n_0$, if $\Phi_{V,R,s}(x) = 1$ then $x = abcde$ such that $|b|+|c|+|d| \leq n_1$, $|b| + |d| \geq 1$, and $\Phi_{V,R,s}(ab^k cd^k e) = 1$ for every $k \in \mathbb{N}$.



The context-free pumping lemma is even more cumbersome to state than its regular analog, but you can remember it as saying the following: *"If a long enough string is matched by a grammar, there must be a variable that is repeated in the derivation."*

Proof of Theorem 9.26. We only sketch the proof. The idea is that if the total number of symbols in the rules R is k_0, then the only way to get $|x| > k_0$ with $\Phi_{V,R,s}(x) = 1$ is to use *recursion*. That is, there must be some variable $v \in V$ such that we are able to derive from v the value bvd for some strings $b, d \in \Sigma^*$, and then further on derive from v some string $c \in \Sigma^*$ such that bcd is a substring of x. If we try to take the minimal such v, then we can ensure that $|bcd|$ is at most some constant depending on k_0 and we can set n_0 to be that constant ($n_0 = 10 \cdot |R| \cdot k_0$ will do, since we will not need more than $|R|$ applications of rules, and each such application can grow the string by at most k_0 symbols).

Thus by the definition of the grammar, we can repeat the derivation to replace the substring bcd in x with $b^k cd^k$ for every $k \in \mathbb{N}$ while retaining the property that the output of $\Phi_{V,R,s}$ is still one.

■

Using Theorem 9.26 one can show that even the simple function $F : \{0,1\}^* \to \{0,1\}$ defined as follows:

$$F(x) = \begin{cases} 1 & x = ww \text{ for some } w \in \{0,1\}^* \\ 0 & \text{otherwise} \end{cases} \tag{9.13}$$

is not contaxt free. (In contrast, the function $G : \{0,1\}^* \to \{0,1\}$ defined as $G(x) = 1$ iff $x = w_0 w_1 \cdots w_{n-1} w_{n-1} w_{n-2} \cdots w_0$ for some $w \in \{0,1\}^*$ and $n = |w|$ is context free, can you see why?.)

Solved Exercise 9.4 — Equality is not context-free. Let $EQ : \{0,1,;\}^* \to \{0,1\}$ be the function such that $EQ(x) = 1$ if and only if $x = u;u$ for some $u \in \{0,1\}^*$. Then EQ is not context free.

■

Solution:

We use the context-free pumping lemma. Suppose towards the sake of contradiction that there is a grammar G that computes EQ, and let n_0 be the constant obtained from Theorem 9.26.

Consider the string $x = 1^{n_0}0^{n_0};1^{n_0}0^{n_0}$, and write it as $x = abcde$ as per Theorem 9.26, with $|bcd| \leq n_0$ and with $|b| + |d| \geq 1$. By Theorem 9.26, it should hold that $EQ(ace) = 1$. However, by case analysis this can be shown to be a contradiction.

First of all, unless b is on the left side of the ; separator and d is on the right side, dropping b and d will definitely make the two parts different. But if it is the case that b is on the left side and d is on the right side, then by the condition that $|bcd| \leq n_0$ we know that b is a string of only zeros and d is a string of only ones. If we drop b and d then since one of them is non empty, we get that there are either less zeroes on the left side than on the right side, or there are less ones on the right side than on the left side. In either case, we get that $EQ(ace) = 0$, obtaining the desired contradiction.

■

9.7 SEMANTIC PROPERTIES OF CONTEXT FREE LANGUAGES

As in the case of regular expressions, the limitations of context free grammars do provide some advantages. For example, emptiness of context free grammars is decidable:

Theorem 9.27 — Emptiness for CFG's is decidable. There is an algorithm that on input a context-free grammar G, outputs 1 if and only if Φ_G is the constant zero function.

Proof Idea:

The proof is easier to see if we transform the grammar to Chomsky Normal Form as in Theorem 9.23. Given a grammar G, we can recursively define a non-terminal variable v to be *non empty* if there is either a rule of the form $v \Rightarrow \sigma$, or there is a rule of the form $v \Rightarrow uw$ where both u and w are non empty. Then the grammar is non empty if and only if the starting variable s is non-empty.

⋆

Proof of Theorem 9.27. We assume that the grammar G in Chomsky Normal Form as in Theorem 9.23. We consider the following procedure for marking variables as "non empty":

1. We start by marking all variables v that are involved in a rule of the form $v \Rightarrow \sigma$ as non empty.
2. We then continue to mark v as non empty if it is involved in a rule of the form $v \Rightarrow uw$ where u, w have been marked before.

We continue this way until we cannot mark any more variables. We then declare that the grammar is empty if and only if s has not been marked. To see why this is a valid algorithm, note that if a variable v has been marked as "non empty" then there is some string $\alpha \in \Sigma^*$ that can be derived from v. On the other hand, if v has not been marked, then every sequence of derivations from v will always have a variable that has not been replaced by alphabet symbols. Hence in particular Φ_G is the all zero function if and only if the starting variable s is not marked "non empty".

■

9.7.1 Uncomputability of context-free grammar equivalence (optional)

By analogy to regular expressions, one might have hoped to get an algorithm for deciding whether two given context free grammars are equivalent. Alas, no such luck. It turns out that the equivalence problem for context free grammars is *uncomputable*. This is a direct corollary of the following theorem:

Theorem 9.28 — Fullness of CFG's is uncomputable. For every set Σ, let $CFGFULL_\Sigma$ be the function that on input a context-free grammar G over Σ, outputs 1 if and only if G computes the constant 1 function. Then there is some finite Σ such that $CFGFULL_\Sigma$ is uncomputable.

Theorem 9.28 immediately implies that equivalence for context-free grammars is uncomputable, since computing "fullness" of a grammar G over some alphabet $\Sigma = \{\sigma_0, \ldots, \sigma_{k-1}\}$ corresponds to checking whether G is equivalent to the grammar $s \Rightarrow$ ""$|s\sigma_0| \cdots |s\sigma_{k-1}$. Note that Theorem 9.28 and Theorem 9.27 together imply that context-free

grammars, unlike regular expressions, are *not* closed under complement. (Can you see why?) Since we can encode every element of Σ using $\lceil \log |\Sigma| \rceil$ bits (and this finite encoding can be easily carried out within a grammar) Theorem 9.28 implies that fullness is also uncomputable for grammars over the binary alphabet.

Proof Idea:

We prove the theorem by reducing from the Halting problem. To do that we use the notion of *configurations* of NAND-TM programs, as defined in Definition 7.19. Recall that a *configuration* of a program P is a binary string s that encodes all the information about the program in the current iteration.

We define Σ to be $\{0,1\}$ plus some separator characters and define $INVALID_P : \Sigma^* \rightarrow \{0,1\}$ to be the function that maps every string $L \in \Sigma^*$ to 1 if and only L does *not* encode a sequence of configurations that correspond to a valid halting history of the computation of P on the empty input.

The heart of the proof is to show that $INVALID_P$ is context-free. Once we do that, we see that P halts on the empty input if and only if $INVALID_P(L) = 1$ for *every* L. To show that, we will encode the list in a special way that makes it amenable to deciding via a context-free grammar. Specifically we will reverse all the odd-numbered strings.

⋆

Proof of Theorem 9.28. We only sketch the proof. We will show that if we can compute *CFGFULL* then we can solve *HALTONZERO*, which has been proven uncomputable in Theorem 8.6. Let M be an input Turing machine for *HALTONZERO*. We will use the notion of *configurations* of a Turing machine, as defined in Definition 7.19.

Recall that a *configuration* of Turing machine M and input x captures the full state of M at some point of the computation. The particular details of configurations are not so important, but what you need to remember is that:

- A configuration can be encoded by a binary string $\sigma \in \{0,1\}^*$.
- The *initial* configuration of M on the input 0 is some fixed string.
- A *halting configuration* will have the value a certain state (which can be easily "read off" from it) set to 1.
- If σ is a configuration at some step i of the computation, we denote by $NEXT_M(\sigma)$ as the configuration at the next step. $NEXT_M(\sigma)$ is a string that agrees with σ on all but a constant number of coordinates (those encoding the position corresponding to the head position and the two adjacent ones). On those coordinates, the value of $NEXT_M(\sigma)$ can be computed by some finite function.

We will let the alphabet $\Sigma = \{0,1\} \cup \{\|, \#\}$. A *computation history* of M on the input 0 is a string $L \in \Sigma$ that corresponds to a list $\|\sigma_0\#\sigma_1\|\sigma_2\#\sigma_3 \cdots \sigma_{t-2}\|\sigma_{t-1}\#$ (i.e., $\|$ comes before an even numbered block, and $\|$ comes before an odd numbered one) such that if i is even then σ_i is the string encoding the configuration of P on input 0 at the beginning of its i-th iteration, and if i is odd then it is the same except the string is *reversed*. (That is, for odd i, $rev(\sigma_i)$ encodes the configuration of P on input 0 at the beginning of its i-th iteration.)[13]

[13] Reversing the odd-numbered block is a technical trick to help with making the function $INVALID_M$ we'll define below context free.

We now define $INVALID_M : \Sigma^* \to \{0,1\}$ as follows:

$$INVALID_M(L) = \begin{cases} 0 & L \text{ is a valid computation history of } M \text{ on } 0 \\ 1 & \text{otherwise} \end{cases} \tag{9.14}$$

We will show the following claim:

CLAIM: $INVALID_M$ is context-free.

The claim implies the theorem. Since M halts on 0 if and only if there exists a valid computation history, $INVALID_M$ is the constant one function if and only if M does *not* halt on 0. In particular, this allows us to reduce determining whether M halts on 0 to determining whether the grammar G_M corresponding to $INVALID_M$ is full.

We now turn to the proof of the claim. We will not show all the details, but the main point $INVALID_M(L) = 1$ if *at least one* of the following three conditions hold:

1. L is not of the right format, i.e. not of the form $\langle$binary-string$\rangle\#\langle$binary-string$\rangle\|\langle$binary-string$\rangle\#\cdots$.
2. L contains a substring of the form $\|\sigma\#\sigma'\|$ such that $\sigma' \neq rev(NEXT_P(\sigma))$
3. L contains a substring of the form $\#\sigma\|\sigma'\#$ such that $\sigma' \neq NEXT_P(rev(\sigma))$

Since context-free functions are closed under the OR operation, the claim will follow if we show that we can verify conditions 1, 2 and 3 via a context-free grammar.

For condition 1 this is very simple: checking that L *is* of the correct format can be done using a regular expression. Since regular expressions are closed under negation, this means that checking that L is *not* of this format can also be done by a regular expression and hence by a context-free grammar.

For conditions 2 and 3, this follows via very similar reasoning to that showing that the function F such that $F(u\#v) = 1$ iff $u \neq rev(v)$ is context-free, see Solved Exercise 9.3. After all, the $NEXT_M$ function only modifies its input in a constant number of places. We leave filling out the details as an exercise to the reader. Since $INVALID_M(L) = 1$

if and only if L satisfies one of the conditions 1., 2. or 3., and all three conditions can be tested for via a context-free grammar, this completes the proof of the claim and hence the theorem.

■

9.8 SUMMARY OF SEMANTIC PROPERTIES FOR REGULAR EXPRESSIONS AND CONTEXT-FREE GRAMMARS

To summarize, we can often trade *expressiveness* of the model for *amenability to analysis*. If we consider computational models that are *not* Turing complete, then we are sometimes able to bypass Rice's Theorem and answer certain semantic questions about programs in such models. Here is a summary of some of what is known about semantic questions for the different models we have seen.

Table 9.1: Computability of semantic properties

Model	**Halting**	**Emptiness**	**Equivalence**
Regular expressions	Computable	Computable	Computable
Context free grammars	Computable	Computable	Uncomputable
Turing-complete models	Uncomputable	Uncomputable	Uncomputable



Remark 9.29 — Unrestricted Grammars (optional). The reason we call context free grammars "context free" is because if we have a rule of the form $v \mapsto a$ it means that we can always replace v with the string a, no matter the *context* in which v appears. More generally, we might want to consider cases where our replacement rules depend on the context.

This gives rise to the notion of *general grammars* that allow rules of the form $a \Rightarrow b$ where both a and b are strings over $(V \cup \Sigma)^*$. The idea is that if, for example, we wanted to enforce the condition that we only apply some rule such as $v \mapsto 0w1$ when v is surrounded by three zeroes on both sides, then we could do so by adding a rule of the form $000v000 \mapsto 0000w1000$ (and of course we can add much more general conditions). Alas, this generality comes at a cost - these general grammars are Turing complete and hence their halting problem is undecidable.



Lecture Recap

- The uncomputability of the Halting problem for general models motivates the definition of restricted computational models.
- In some restricted models we can answer *semantic* questions such as: does a given program terminate, or do two programs compute the same function?
- *Regular expressions* are a restricted model of computation that is often useful to capture tasks of string matching. We can test efficiently whether an expression matches a string, as well as answer questions such as Halting and Equivalence.
- *Context free grammars* is a stronger, yet still not Turing complete, model of computation. The halting problem for context free grammars is computable, but equivalence is not computable.

9.9 EXERCISES

Exercise 9.1 — Closure properties of regular functions. Suppose that $F, G : \{0,1\}^* \rightarrow \{0,1\}$ are regular. For each one of the following definitions of the function H, either prove that H is always regular or give a counterexample for regular F, G that would make H not regular.

1. $H(x) = F(x) \vee G(x)$.
2. $H(x) = F(x) \wedge G(x)$
3. $H(x) = NAND(F(x), G(x))$.
4. $H(x) = F(x^R)$ where x^R is the reverse of x: $x^R = x_{n-1}x_{n-2}\cdots x_o$ for $n = |x|$.
5. $H(x) = \begin{cases} 1 & x = uv \text{ s.t. } F(u) = G(v) = 1 \\ 0 & \text{otherwise} \end{cases}$
6. $H(x) = \begin{cases} 1 & x = uu \text{ s.t. } F(u) = G(u) = 1 \\ 0 & \text{otherwise} \end{cases}$
7. $H(x) = \begin{cases} 1 & x = uu^R \text{ s.t. } F(u) = G(u) = 1 \\ 0 & \text{otherwise} \end{cases}$

■

Exercise 9.2 — Closure properties of context-free functions. Suppose that $F, G : \{0,1\}^* \rightarrow \{0,1\}$ are context free. For each one of the following definitions of the function H, either prove that H is always context free or give a counterexample for regular F, G that would make H not context free.

1. $H(x) = F(x) \vee G(x)$.

2. $H(x) = F(x) \wedge G(x)$
3. $H(x) = NAND(F(x), G(x))$.
4. $H(x) = F(x^R)$ where x^R is the reverse of x: $x^R = x_{n-1}x_{n-2}\cdots x_o$ for $n = |x|$.
5. $H(x) = \begin{cases} 1 & x = uv \text{ s.t. } F(u) = G(v) = 1 \\ 0 & \text{otherwise} \end{cases}$
6. $H(x) = \begin{cases} 1 & x = uu \text{ s.t. } F(u) = G(u) = 1 \\ 0 & \text{otherwise} \end{cases}$
7. $H(x) = \begin{cases} 1 & x = uu^R \text{ s.t. } F(u) = G(u) = 1 \\ 0 & \text{otherwise} \end{cases}$

■

Exercise 9.3 Prove that the function $F : \{0,1\}^* \rightarrow \{0,1\}$ such that $F(x) = 1$ if and only if $|x|$ is a power of two is not context free.

■

9.10 BIBLIOGRAPHICAL NOTES

The relation of regular expressions with finite automata is a beautiful topic, on which we only touch upon in this texts. It is covered more extensively in [**hopcroft** ; Sip97; Koz97]. These texts also discuss topics such as *non deterministic finite automata* (NFA) and the relation between context-free grammars and pushdown automata. The Chomsky Hierarchy is a hierarchy of grammars from the least restrictive (most powerful) Type 0 grammars, which correspond to *recursively enumerable* languages (see Definition 8.14) to the most restrictive Type 3 grammars, which correspond to regular languages. Context-free languages correspond to Type 2 grammars. Type 1 grammars are *context sensitive grammars*. These are more powerful than context-free grammars but still less powerful than Turing machines. In particular functions/languages corresponding to context-sensitive grammars are always computable, and in fact can be computed by a linear bounded automatons which are non-deterministic algorithms that take $O(n)$ space. For this reason, the class of functions/languages corresponding to context-sensitive grammars is also known as the complexity class **NSPACE**$O(n)$; we discuss space-bounded complexity in Chapter 16). While Rice's Theorem tells us that we cannot compute any non-trivial semantic property of Type 0 grammars, the situation is more complex for other types of grammars: some semantic properties can be determined and some cannot, depending on the grammar's place in the hierarchy.

Learning Objectives:

- See more examples of uncomputable functions that are not as tied to computation.
- See Gödel's incompleteness theorem - a result that shook the world of mathematics in the early 20th century.

10
Is every theorem provable?

> *"Take any definite unsolved problem, such as ... the existence of an infinite number of prime numbers of the form $2^n + 1$. However unapproachable these problems may seem to us and however helpless we stand before them, we have, nevertheless, the firm conviction that their solution must follow by a finite number of purely logical processes..."*
>
> *"...This conviction of the solvability of every mathematical problem is a powerful incentive to the worker. We hear within us the perpetual call: There is the problem. Seek its solution. You can find it by pure reason, for in mathematics there is no ignorabimus."*, David Hilbert, 1900.

> *"The meaning of a statement is its method of verification."*, Moritz Schlick, 1938 (aka "The verification principle" of logical positivism)

The problems shown uncomputable in Chapter 8, while natural and important, still intimately involved NAND-TM programs or other computing mechanisms in their definitions. One could perhaps hope that as long as we steer clear of functions whose inputs are themselves programs, we can avoid the "curse of uncomputability". Alas, we have no such luck.

In this chapter we will see an example of a natural and seemingly "computation free" problem that nevertheless turns out to be uncomputable: solving Diophantine equations. As a corollary, we will see one of the most striking results of 20th century mathematics: *Gödel's Incompleteness Theorem*, which showed that there are some mathematical statements (in fact, in number theory) that are *inherently unprovable*. We will actually start with the latter result, and then show the former.

10.1 HILBERT'S PROGRAM AND GÖDEL'S INCOMPLETENESS THEOREM

> *"And what are these ...vanishing increments? They are neither finite quantities, nor quantities infinitely small, nor yet nothing. May we not call them the ghosts of departed quantities?"*, George Berkeley, Bishop of Cloyne, 1734.

The 1700's and 1800's were a time of great discoveries in mathematics but also of several crises. The discovery of calculus by Newton and Leibnitz in the late 1600's ushered a golden age of problem solving. Many longstanding challenges succumbed to the new tools that were discovered, and mathematicians got ever better at doing some truly impressive calculations. However, the rigorous foundations behind these calculations left much to be desired. Mathematicians manipulated infinitesimal quantities and infinite series cavalierly, and while most of the time they ended up with the correct results, there were a few strange examples (such as trying to calculate the value of the infinite series $1 - 1 + 1 - 1 + 1 + ...$) which seemed to give out different answers depending on the method of calculation. This led to a growing sense of unease in the foundations of the subject which was addressed in works of mathematicians such as Cauchy, Weierstrass, and Riemann, who eventually placed analysis on firmer foundations, giving rise to the ϵ's and δ's that students taking honors calculus grapple with to this day.

In the beginning of the 20th century, there was an effort to replicate this effort, in greater rigor, to all parts of mathematics. The hope was to show that all the true results of mathematics can be obtained by starting with a number of axioms, and deriving theorems from them using logical rules of inference. This effort was known as the *Hilbert program*, named after the influential mathematician David Hilbert.

Alas, it turns out the results we've seen dealt a devastating blow to this program, as was shown by Kurt Gödel in 1931:

Theorem 10.1 — Gödel's Incompleteness Theorem: informal version. For every sound proof system for sufficiently rich mathematical statements, there is a mathematical statement that is *true* but is not *provable*.

Before proving Theorem 10.2, we need to specify what does it mean to be "provable" (and even formally define the notion of a "mathematical statement"). Thus we need to define the notion of a *proof system*. In geometry and other areas of mathematics, proof systems are often defined by starting with some basic assumptions or *axioms*

and then deriving more statements by using *inference rules* such as the famous Modus Ponens, but what axioms shall we use? What rules?

Our idea will be to use an extremely general notion of proof, not even restricting ourselves to ones that have the form of axioms and inference. A *proof* will be simply a piece of text- a finite string- that satisfies:

1. (*effectiveness*) Given a statement x and a proof w (both of which can be encoded as strings) we can verify that w is a valid proof for x. (For example, by going line by line and checking that each line does indeed follow from the preceding ones using one of the allowed inference rules.)

2. (*soundness*) If there is a valid proof w for x then x is true.

Those seem like rather minimal requirements that one would want from every proof system. Requirement 2 (soundness) is the very definition of a proof system: you shouldn't be able to prove things that are not true. Requirement 1 is also essential. If it there is no set of rules (i.e., an algorithm) to check that a proof is valid then in what sense is it a proof system? We could replace it with the system where the "proof" for a statement x would simply be "trust me: it's true".

A mathematical statement will also simply be a string. Mathematical statements states a fact about some mathematical object. For example, the following is a mathematical statement:

> *"The number* 2,696,635,869,504,783,333,238,805,675,613,588,278,597,832,162,617,892,474,670,798,113 *is prime".*

(This happens to be a *false* statement; can you see why?)

Mathematical statements don't have to be about numbers. They can talk about any other mathematical object including sets, strings, functions, graphs and yes, even *programs*. Thus, another example of a mathematical statement is the following:

> The following Python function halts on every positive integer n
>
> ```
> def f(n):
>     if n==1: return 1
>     return f(3*n+1) if n % 2 else f(n//2)
> ```

(We actually don't know if this statement is true or false.)

We start by considering statements of the second type. Our first formalization of Theorem 10.2 will be the following

Theorem 10.2 — Gödel's Incompleteness Theorem: computational variant. Let $V : \{0,1\}^* \to \{0,1\}$ a computable purported verification procedure for mathematical statements of the form "Program P halts on the zero input" and "Program P does not halt on the zero input". Then either:

- *V is not sound:* There exists a false statement x and a string $w \in \{0,1\}^*$ such that $V(x,w) = 1$.

  or

- *V is not complete:* There exists a true statement x such that for every $w \in \{0,1\}^*$, $V(x,w) = 0$.

Proof Idea:

If we had such a complete and sound proof system then we could solve the *HALTONZERO* problem. On input a program P, we would search all purported proofs w and halt as soon as we find a proof of either "P halts on zero" or "P does not halt on zero". If the system is sound and complete then we will eventually find such a proof, and it will provide us with the correct output.

⋆

Proof of Theorem 10.2. Assume for the sake of contradiction that there was such a proof system V. We will use V to build an algorithm A that computes *HALTONZERO*, hence contradicting Theorem 8.6. Our algorithm A will will work as follows:

Algorithm A:

- **Input:** NAND-TM program P
- **Goal:** Determine if P halts on the input 0.
- **Assumption:** We have access to a proof system V such that for every statement x of the form "Program Q halts on 0" or "Program Q does not halt on 0", there exists some string $w \in \{0,1\}^*$ such that $V(x,w) = 1$ if and only if x is true.

Operation:

- For $n = 0, 1, 2, \ldots$:
  - For $w \in \{0,1\}^n$:
    - If $V("P \text{ halts on } 0", w) = 1$ output 1
    - If $V("P \text{ does not halt on } 0", w) = 1$ output 0

If P halts on 0 then under our assumption there exists w that proves this fact, and so when Algorithm A reaches $n = |w|$ we will eventually find this w and output 1, unless we already halted before. But we cannot halt before and output a wrong answer because it would contradict the soundness of the proof system. Similarly, this shows that if P does *not* halt on 0 then (since we assume there is a proof of this fact too) our algorithm A will eventually halt and output 0.

■

R

Remark 10.3 — The Gödel statement (optional). One can extract from the proof of Theorem 10.2 a procedure that for every proof system V, yields a true statement x^* that cannot be proven in V. But Gödel's proof gave a very explicit description of such a statement x^* which is closely related to the "Liar's paradox". That is, Gödel's statement x^* was designed to be true if and only if $\forall_{w\in\{0,1\}^*} V(x,w) = 0$. In other words, it satisfied the following property

$$x^* \text{ is true} \Leftrightarrow x^* \text{ does not have a proof in } V \qquad (10.1)$$

One can see that if x^* is true, then it does not have a proof, but it is false then (assuming the proof system is sound) then it cannot have a proof, and hence x^* must be both true and unprovable. One might wonder how is it possible to come up with an x^* that satisfies a condition such as (10.1) where the same string x^* appears on both the righthand side and the lefthand side of the equation. The idea is that the proof of **??** yields a way to transform every statement x into a statement $F(x)$ that is true if and only if x does not have a proof in V. Thus x^* needs to be a *fixed point* of F: a sentence such that $x^* = F(x^*)$. It turns out that we can always find such a fixed point of F. We've already seen this phenomenon in the λ calculus, where the Y combinator maps every F into a fixed point YF of F. This is very related to the idea of programs that can print their own code. Indeed, Scott Aaronson likes to describe Gödel's statement as follows:

> The following sentence repeated twice, the second time in quotes, is not provable in the formal system V. "The following sentence repeated twice, the second time in quotes, is not provable in the formal system V."

In the argument above we actually showed that x^* is *true*, under the assumption that V is sound. Since x^* is true and does not have a proof in V, this means that we cannot carry the above argument in the system V, which means that V cannot prove its own soundness (or even consistency: that there is no proof of both a statement and its negation). Using this idea, it's not hard to get Gödel's second incompleteness theorem, which says that every sufficiently rich V cannot prove its own consistency. That is, if we formalize the statement c^* that is true if and only if V is consistent (i.e., V cannot prove both a statement and the statement's negation), then c^* cannot be proven in V.

10.2 QUANTIFIED INTEGER STATEMENTS

There is something "unsatisfying" about Theorem 10.2. Sure, it shows there are statements that are unprovable, but they don't feel like "real" statements about math. After all, they talk about *programs* rather than numbers, matrices, or derivatives, or whatever it is they teach in math courses. It turns out that we can get an analogous result for statements such as "there are no integers x and y such that $x^2 - 2 = y^7$", or "there are integers x, y, z such that $x^2 + y^6 = z^{11}$" that only talk about *natural numbers*.[1] It doesn't get much more "real math" than this. Indeed, the 19th century mathematician Leopold Kronecker famously said that "God made the integers, all else is the work of man."

[1] I do not know if these statements are actually true or false, see here.

To make this more precise, let us define the notion of *quantified integer statements*:

Definition 10.4 — Quantified integer statements. A *quantified integer statement* is a well-formed statement with no unbound variables involving integers, variables, the operators $>, <, \times, +, -, =$, the logical operations $\neg$ (NOT), $\wedge$ (AND), and $\vee$ (OR), as well as quantifiers of the form $\exists_{x \in \mathbb{N}}$ and $\forall_{y \in \mathbb{N}}$ where x, y are variable names.

We often care deeply about determining the truth of quantified integer statements. For example, the statement that Fermat's Last Theorem is true for $n = 3$ can be phrased as the quantified integer statement

$$\neg \exists_{a \in \mathbb{N}} \exists_{b \in \mathbb{N}} \exists_{c \in \mathbb{N}} (a > 0) \wedge (b > 0) \wedge (c > 0) \wedge (a \times a \times a + b \times b \times b = c \times c \times c) \ . \tag{10.2}$$

The twin prime conjecture, that states that there is an infinite number of numbers p such that both p and $p + 2$ are primes can be phrased

as the quantified integer statement

$$\forall_{n\in\mathbb{N}}\exists_{p\in\mathbb{N}}(p > n) \wedge PRIME(p) \wedge PRIME(p+2) \tag{10.3}$$

where we replace an instance of $PRIME(q)$ with the statement $(q > 1) \wedge \forall_{a\in\mathbb{N}}\forall_{b\in\mathbb{N}}(a = 1) \vee (a = q) \vee \neg(a \times b = q)$.

The claim (mentioned in Hilbert's quote above) that are infinitely many primes of the form $p = 2^n + 1$ can be phrased as follows:

$$\begin{gathered}\forall_{n\in\mathbb{N}}\exists_{p\in\mathbb{N}}(p > n) \wedge PRIME(p) \wedge \\ (\forall_{k\in\mathbb{N}}(k \neq 2 \ \wedge \ PRIME(k)) \Rightarrow \neg DIVIDES(k, p-1))\end{gathered} \tag{10.4}$$

where $DIVIDES(a, b)$ is the statement $\exists_{c\in\mathbb{N}} b \times c = a$. In English, this corresponds to the claim that for every n there is some $p > n$ such that all of $p - 1$'s prime factors are equal to 2.



Remark 10.5 — Syntactic sugar for quantified integer statements. To make our statements more readable, we often use syntactic sugar and so write $x \neq y$ as shorthand for $\neg(x = y)$, and so on. Similarly, the "implication operator" $a \Rightarrow b$ is "syntactic sugar" or shorthand for $\neg a \vee b$, and the "if and only if operator" $a \Leftrightarrow$ is shorthand for $(a \Rightarrow b) \wedge (b \Rightarrow a)$. We will also allow ourselves the use of "macros": plugging in one quantified integer statement in another, as we did with *DIVIDES* and *PRIME* above.

Much of number theory is concerned with determining the truth of quantified integer statements. Since our experience has been that, given enough time (which could sometimes be several centuries) humanity has managed to do so for the statements that it cared enough about, one could (as Hilbert did) hope that eventually we would be able to prove or disprove all such statements. Alas, this turns out to be impossible:

Theorem 10.6 — Gödel's Incompleteness Theorem for quantified integer statements. Let $V : \{0,1\}^* \to \{0,1\}$ a computable purported verification procedure for quantified integer statements. Then either:

- *V is not sound:* There exists a false statement x and a string $w \in \{0,1\}^*$ such that $V(x, w) = 1$.

  or

- *V is not complete:* There exists a true statement x such that for every $w \in \{0,1\}^*$, $V(x, w) = 0$.

Theorem 10.6 is a direct corollary of the following result, just as Theorem 10.2 was a direct corollary of the uncomputability of *HALTONZERO*:

Theorem 10.7 — Uncomputability of quantified integer statements. Let $QIS : \{0,1\}^* \to \{0,1\}$ be the function that given a (string representation of) a quantified integer statement outputs 1 if it is true and 0 if it is false. [2] Then *QIS* is uncomputable.

[2] Since a quantified integer statement is simply a sequence of symbols, we can easily represent it as a string. We will assume that *every* string represents some quantified integer statement, by mapping strings that do not correspond to such a statement to an arbitrary statement such as $\exists_{x \in \mathbb{N}} x = 1$.

P

Please stop here and make sure you understand why the uncomputability of *QIS* (i.e., Theorem 10.7) means that there is no sound and complete proof system for proving quantified integer statements (i.e., Theorem 10.6). This follows in the same way that Theorem 10.2 followed from the uncomputability of *HALTONZERO*, but working out the details is a great exercise (see Exercise 10.1)

In the rest of this chapter, we will show the proof of **??**.

10.3 DIOPHANTINE EQUATIONS AND THE MRDP THEOREM

Many of the functions people wanted to compute over the years involved solving equations. These have a much longer history than mechanical computers. The Babylonians already knew how to solve some quadratic equations in 2000BC, and the formula for all quadratics appears in the Bakhshali Manuscript that was composed in India around the 3rd century. During the Renaissance, Italian mathematicians discovered generalization of these formulas for cubic and quartic (degrees 3 and 4) equations. Many of the greatest minds of the 17th and 18th century, including Euler, Lagrange, Leibniz and Gauss worked on the problem of finding such a formula for *quintic* equations to no avail, until in the 19th century Ruffini, Abel and Galois showed that no such formula exists, along the way giving birth to *group theory*.

However, the fact that there is no closed-form formula does not mean we can not solve such equations. People have been solving higher degree equations numerically for ages. The Chinese manuscript Jiuzhang Suanshu from the first century mentions such approaches. Solving polynomial equations is by no means restricted only to ancient history or to students' homeworks. The gradient descent method is the workhorse powering many of the machine learning tools that have revolutionized Computer Science over the last several years.

But there are some equations that we simply do not know how to solve *by any means*. For example, it took more than 200 years until people succeeded in proving that the equation $a^{11} + b^{11} = c^{11}$ has no solution in integers.[3] The notorious difficulty of so called *Diophantine equations* (i.e., finding *integer* roots of a polynomial) motivated the mathematician David Hilbert in 1900 to include the question of finding a general procedure for solving such equations in his famous list of twenty-three open problems for mathematics of the 20th century. I don't think Hilbert doubted that such a procedure exists. After all, the whole history of mathematics up to this point involved the discovery of ever more powerful methods, and even impossibility results such as the inability to trisect an angle with a straightedge and compass, or the non-existence of an algebraic formula for quintic equations, merely pointed out to the need to use more general methods.

Alas, this turned out not to be the case for Diophantine equations. In 1970, Yuri Matiyasevich, building on a decades long line of work by Martin Davis, Hilary Putnam and Julia Robinson, showed that there is simply *no method* to solve such equations in general:

Theorem 10.8 — MRDP Theorem. Let $DIO : \{0,1\}^* \to \{0,1\}$ be the function that takes as input a string describing a 100-variable polynomial with integer coefficients $P(x_0, \ldots, x_{99})$ and outputs 1 if and only if there exists $z_0, \ldots, z_{99} \in \mathbb{N}$ s.t. $P(z_0, \ldots, z_{99}) = 0$.

Then DIO is uncomputable. [4]



Remark 10.9 — Active code vs static data. The difficulty in finding a way to distinguish between "code" such as NAND-TM programs, and "static content" such as polynomials is just another manifestation of the phenomenon that *code* is the same as *data*. While a fool-proof solution for distinguishing between the two is inherently impossible, finding heuristics that do a reasonable job keeps many firewall and anti-virus manufacturers very busy (and finding ways to bypass these tools keeps many hackers busy as well).

10.4 HARDNESS OF QUANTIFIED INTEGER STATEMENTS

We will not prove the MRDP Theorem (Theorem 10.8). However, as we mentioned, we will prove the uncomputability of *QIS* (i.e., Theorem 10.7), which is a special case of the MRDP Theorem. The reason is that a Diophantine equation is a special case of a quantified integer statement where the only quantifier is $\exists$. This means that deciding the truth of quantified integer statements is a potentially harder problem

[3] This is a special case of what's known as "Fermat's Last Theorem" which states that $a^n + b^n = c^n$ has no solution in integers for $n > 2$. This was conjectured in 1637 by Pierre de Fermat but only proven by Andrew Wiles in 1991. The case $n = 11$ (along with all other so called "regular prime exponents") was established by Kummer in 1850.

[4] As usual, we assume some standard way to express numbers and text as binary strings. The constant 100 is of course arbitrary; the problem is known to be uncomputable even for polynomials of degree four and at most 58 variables. In fact the number of variables can be reduced to nine, at the expense of the polynomial having a larger (but still constant) degree. See Jones's paper for more about this issue.

than solving Diophantine equations, and so it is potentially *easier* to prove that *QIS* is uncomputable.

P

If you find the last sentence confusing, it is worthwhile to reread it until you are sure you follow its logic. We are so accustomed to trying to find *solutions* for problems that it can sometimes be hard to follow the arguments for showing that problems are *uncomputable*.

Our proof of the uncomputability of *QIS* (i.e. Theorem 10.7) will, as usual, go by reduction from the Halting problem, but we will do so in two steps:

1. We will first use a reduction from the Halting problem to show that deciding the truth of *quantified mixed statements* is uncomputable. Quantified mixed statements involve both strings and integers. Since quantified mixed statements are a more general concept than quantified integer statements, it is *easier* to prove the uncomputability of deciding their truth.

2. We will then reduce the problem of quantified mixed statements to quantifier integer statements.

10.4.1 Step 1: Quantified mixed statements and computation histories

We define *quantified mixed statements* as statements involving not just integers and the usual arithmetic operators, but also *string variables* as well.

Definition 10.10 — Quantified mixed statements. A *quantified mixed statement* is a well-formed statement with no unbound variables involving integers, variables, the operators $>, <, \times, +, -, =$, the logical operations $\neg$ (NOT), $\wedge$ (AND), and $\vee$ (OR), as well as quantifiers of the form $\exists_{x\in\mathbb{N}}$, $\exists_{a\in\{0,1\}^*}$, $\forall_{y\in\mathbb{N}}$, $\forall_{b\in\{0,1\}^*}$ where x, y, a, b are variable names. These also include the operator $|a|$ which returns the length of a string valued variable a, as well as the operator a_i where a is a string-valued variable and i is an integer valued expression which is true if i is smaller than the length of a and the ith coordinate of a is 1, and is false otherwise.

For example, the true statement that for every string a there is a string b that corresponds to a in reverse order can be phrased as the following quantified mixed statement

$$\forall_{a\in\{0,1\}^*}\exists_{b\in\{0,1\}^*}(|a| = |b|) \wedge (\forall_{i\in\mathbb{N}} i < |a| \Rightarrow (a_i \Leftrightarrow b_{|a|-i}) \; . \tag{10.5}$$

Quantified mixed statements are more general than quantified integer statements, and so the following theorem is potentially easier to prove than Theorem 10.7:

Theorem 10.11 — Uncomputability of quantified mixed statements. Let $QMS : \{0,1\}^* \to \{0,1\}$ be the function that given a (string representation of) a quantified mixed statement outputs 1 if it is true and 0 if it is false. Then QMS is uncomputable.

Proof Idea:

The idea behind the proof is similar to that used in showing that one-dimensional cellular automata are Turing complete (Theorem 7.18) as well as showing that equivalence (or even "fullness") of context free grammars is uncomputable (Theorem 9.28). We use the notion of a *configuration* of a NAND-TM program as in Definition 7.19. Such a configuration can be thought of as a string α over some large-but-finite alphabet Σ describing its current state, including the values of all arrays, scalars, and the index variable `i`. It can be shown that if α is the configuration at a certain step of the execution and β is the configuration at the next step, then $\beta_j = \alpha_j$ for all j outside of $\{i-1, i, i+1\}$ where i is the value of `i`. In particular, every value β_j is simply a function of $\alpha_{j-1,j,j+1}$. Using these observations we can write a *quantified mixed statement* $NEXT(\alpha, \beta)$ that will be true if and only if β is the configuration encoding the next step after α. Since a program P halts on input x if and only if there is a sequence of configurations $\alpha^0, \ldots, \alpha^{t-1}$ (known as a *computation history*) starting with the initial configuration with input x and ending in a halting configuration, we can define a quantified mixed statement to determine if there is such a statement by taking a universal quantifier over all strings H (for *history*) that encode a tuple $(\alpha^0, \alpha^1, \ldots, \alpha^{t-1})$ and then checking that α^0 and α^{t-1} are valid starting and halting configurations, and that $NEXT(\alpha^j, \alpha^{j+1})$ is true for every $j \in \{0, \ldots, t-2\}$.

⋆

Proof of Theorem 10.11. The proof will be obtained by a reduction from the Halting problem. Specifically, we will use the notion of a *configuration* of a NAND-TM program (Definition 7.19) that we have seen in the context of proving that one dimensional cellular automata are Turing complete. We need the following facts about configurations:

- For every (well formed[5]) NAND-TM program P, there is a finite alphabet Σ, and a *configuration* of P is a string $\alpha \in \Sigma^*$.
- A configuration α encodes all the state of the program at a particular iteration, including the array, scalar, and index variables.

[5] We can always transform a NAND-TM program into an equivalent one that is well formed (see Lemma 6.15), and hence can assume this property without loss of generality.

- If α is a configuration, then $\beta = NEXT_P(\alpha)$ denotes the configuration of the computation after one more iteration. β is a string over Σ of length either $|\alpha|$ or $|\alpha|+1$, and every coordinate of β is a function of just three coordinates in α. That is, for every $j \in \{0,\ldots,|\beta|-1\}$, $\beta_j = MAP_P(\alpha_{j-1}, \alpha_j, \alpha_{j+1})$ where $MAP_P : \Sigma^3 \to \Sigma$ is some function depending on P.[6]

- There are simple conditions to check whether a string α is a valid starting configuration corresponding to an input x, as well as to check whether a string α is an halting configuration. In particular these conditions can be phrased as quantified mixed statements.

- A program P halts on input x if and only if there exists a sequence of configurations $H = (\alpha^0, \alpha^1, \ldots, \alpha^{T-1})$ such that **(i)** α^0 is a valid starting configuration of P with input x, **(ii)** α^{T-1} is a valid halting configuration of P, and **(iii)** $\alpha^{i+1} = NEXT_P(\alpha^i)$ for every $i \in \{0,\ldots,T-2\}$.

[6] The alphabet Σ contains a special "default" element, which we can denote by $\varnothing$, such that if $j-1 < 0$ or j or $j+1$ are at least $|\alpha|$, we use $\varnothing$ as input instead of α_{j-1} or α_{j+1} respectively. We extend the length of β to be one longer than α if and only if $N_P(\alpha_{|\alpha|-1}, \varnothing, \varnothing) \neq \varnothing$.

Let U be a universal NAND-TM program. Such a program exists by Theorem 8.1. We define $HALT_U$ as the function such that $HALT_U(w) = 1$ if and only if U halts on the input w. We claim that the function $HALT_U$ is uncomputable. Indeed, for every NAND-TM program P (which we identify with its representation as a string) and input $x \in \{0,1\}^*$ to P, $HALT(P,x) = HALT_U(\langle P, x\rangle)$ where $\langle P, x\rangle$ is some encoding of the pair (P,x) as a string. Hence if we could compute $HALT_U$ then we could compute $HALT$, contradicting Theorem 8.5.

Let Σ be the alphabet needed to encode configurations of U, and let $\ell = \lceil \log(|\Sigma|+1) \rceil$. Then we can encode any symbol in $\Sigma \cup \{;\}$ (where ";" is some separator symbol we'll use) as a string in $\{0,1\}^\ell$, and so in particular can encode a sequence $\alpha^0; \alpha^1; \cdots; \alpha^T$ of configurations of U as a single binary string which we'll also name as H. Given any input $w \in \{0,1\}^*$, we will create a mixed integer statement φ_w that will have the following form:

$$\varphi_w = \exists_{H \in \{0,1\}^*} H \text{ encodes a valid sequence of configurations of a halting computation of } U \text{ on } w \tag{10.6}$$

The reasons we can encode this condition as an MIS are the following:

1. The conditions for checking that the initial configuration is valid are simple, and we can extract the first configuration from H by first looking at an index i which is a multiple of ℓ such that $H_i \cdots H_{i+\ell-1}$ encodes the separator symbol ";" and such that i is the first such index. Another way to say it is that i is the position of

the first separator if **there exists** k such that $i = k \times \ell$ and $H_{i,\ldots,i+\ell-1}$ and **for every** $j \in \mathbb{N}$, if $j < i$ then $H_{j,\ldots,j+\ell-1}$ does *not* encode ";". This can be captured using the operators allowed in a quantified mixed statetment and the $\forall$ and $\exists$ quantifiers.

2. We can similarly check that the last configuration is halting. Extracting the position i that encodes the last separator can be done in a way analogous to that of extracting the first one.

3. We can define a quantified mixed predicate $NEXT(\alpha, \beta)$ that is true if and only if $\beta = NEXT_U(\beta)$ (i.e., β encodes the configuration obtained by proceeding from α in one computational step). Indeed $NEXT(\alpha, \beta)$ is true if **for every** $i \in \{0, \ldots, |\beta|\}$ which is a multiple of ℓ, $\beta_{i,\ldots,i+\ell-1} = MAP_U(\alpha_{i-\ell,\cdots,i+2\ell-1})$ where $MAP_U : \{0,1\}^{3\ell} \to \{0,1\}^{\ell}$ is the finite function above (identifying elements of Σ with their encoding in $\{0,1\}^{\ell}$). Since MAP_U is a finite function, we can express it using the logical operations *AND*,*OR*, *NOT* (for example by computing MAP_U with *NAND*'s).

4. We can then write the condition that **for every** substring of H that has the form $\alpha ENC(;)\beta$ with $\alpha, \beta \in \{0,1\}^{\ell}$ and $ENC(;)$ being the encoding of the separator ";", it holds that $NEXT(\alpha, \beta)$ is true.

Together the above yields a computable procedure that maps every input $w \in \{0,1\}^*$ to $HALT_U$ into a quantified mixed statement φ_w such that $HALT_U(w) = 1$ if and only if $QMS(\varphi_w) = 1$. This reduces computing $HALT_U$ to computing *QMS*, and hence the uncomputability of $HALT_U$ implies the uncomputability of *QMS*.

■

10.4.2 Step 2: Reducing mixed statements to integer statements

We now show how to prove Theorem 10.7 using Theorem 10.11. The idea is again a proof by reduction. We will show a transformation of every quantifier mixed statement φ into a quantified *integer* statement ξ that does not use string-valued variables such that φ is true if and only if ξ is true.

To remove string-valued variables from a statement, we encode them by integers. We will show that we can encode a string $x \in \{0,1\}^*$ by a pair of numbers $(X, n) \in \mathbb{N}$ s.t.

- $n = |x|$
- There is a quantified integer statement $COORD(X, i)$ that for every $i < n$, will be true if $x_i = 1$ and will be false otherwise.

This will mean that we can replace a "for all" quantifier over strings such as $\forall_{x \in \{0,1\}^*}$ with a pair of quantifiers over *integers* of the form

$\forall_{X\in\mathbb{N}}\forall_{n\in\mathbb{N}}$ (and similarly replace an existential quantifier of the form $\exists_{x\in\{0,1\}^*}$ with a pair of quantifiers $\exists_{X\in\mathbb{N}}\exists_{n\in\mathbb{N}}$) . We can later replace all calls to $|x|$ by n and all calls to x_i by $COORD(X,i)$. This means that if we are able to define *COORD* via a quantified integer statement, then we obtain a proof of Theorem 10.7, since we can use it to map every mixed quantified statement φ to an equivalent quantified integer statement ξ such that ξ is true if and only if φ is true, and hence $QMS(\varphi) = QIS(\xi)$. Such a procedure implies that the task of computing *QMS* reduces to the task of computing *QIS*, which means that the uncomputability of *QMS* implies the uncomputability of *QIS*.

The above shows that proof of the theorem all boils down to finding the right encoding of strings as integers, and the right way to implement *COORD* as a quantified integer statement. To achieve this we use the following technical result :

Lemma 10.12 — Constructible prime sequence. There is a sequence of prime numbers $p_0 < p_1 < p_2 < \cdots$ such that there is a quantified integer statement $PCOORD(p,i)$ that is true if and only if $p = p_i$.

Using Lemma 10.12 we can encode a $x \in \{0,1\}^*$ by the numbers (X,n) where $X = \prod_{x_i=1} p_i$ and $n = |x|$. We can then define the statement $COORD(X,i)$ as

$$\forall_{p\in\mathbb{N}} PCOORD(p,i) \Rightarrow DIVIDES(p,X) \tag{10.7}$$

where $DIVIDES(a,b)$, as before, is defined as $\exists_{c\in\mathbb{N}} a\times c = b$. Note that indeed if X,n encodes the string $x \in \{0,1\}^*$, then for every $i < n$, $COORD(X,i) = x_i$, since p_i divides X if and only if $x_i = 1$.

Thus all that is left to conclude the proof of Theorem 10.7 is to prove Lemma 10.12, which we now proceed to do.

Proof. The sequence of prime numbers we consider is the following: We fix C to be a suficiently large constant ($C = 2^{2^{34}}$ will do) and define p_i to be the smallest prime number that is in the interval $[(i+C)^3+1,(i+C+1)^3-1]$. It is known that there exists such a prime number for every $i \in \mathbb{N}$. Given this, the definition of $PCOORD(p,i)$ is simple:

$$(p > (i+C)\times(i+C)\times(i+C)) \wedge (p < (i+C+1)\times(i+C+1)\times(i+C+1)) \wedge \left(\forall_{p'} \neg PRIME(p') \vee (p' \leq i) \vee (p' \geq p)\right) , \tag{10.8}$$

We leave it to the reader to verify that $PCOORD(p,i)$ is true iff $p = p_i$. ■

To sum up we have shown that for every quantified mixed statement φ, we can compute a quantified integer statement ξ such that $QMS(\varphi) = 1$ if and only if $QIS(\xi) = 1$. Hence the uncomputability of *QMS* (Theorem 10.11) implies the uncomputability of *QIS*, completing the proof of Theorem 10.7, and so also the proof of Gödel's

Incompleteness Theorem for quantified integer statements (Theorem 10.6).

Lecture Recap

- Uncomputable functions include also functions that seem to have nothing to do with NAND-TM programs or other computational models such as determining the satisfiability of diophantine equations.
- This also implies that for any sound proof system (and in particular every finite axiomatic system) S, there are interesting statements X (namely of the form "$F(x) = 0$" for an uncomputable function F) such that S is not able to prove either X or its negation.

10.5 EXERCISES

Remark 10.13 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 10.1 — Gödel's Theorem from uncomputability of QIS. Prove Theorem 10.6 using Theorem 10.7

■

Exercise 10.2 — Expression for floor. Let $FSQRT(n,m) = \forall_{j\in\mathbb{N}}((j \times j) > m) \vee (j \leq n)$. Prove that $FSQRT(n,m)$ is true if and only if $n = \lfloor\sqrt{m}\rfloor$.

■

Exercise 10.3 — Expression for computing the index. Recall that in **??** asked you to prove that at iteration t of a NAND-TM program the the variable `i` is equal to $t - r(r+1)$ if $t \leq (r+1)^2$ and equals $(r+2)(r+1)t$ otherwise, where $r = \lfloor\sqrt{t+1/4} - 1/2\rfloor$. Prove that there is a quantified integer statement $INDEX$ with parameters t, i such that $INDEX(t,i)$ is true if and i is the value of `i` after t iterations.

■

Exercise 10.4 — Expression for computing the previous line. Give the following quantified integer expressions:

1. $MOD(a,b,c)$ which is true if and only if $b = a \mod c$. Note if a program has s lines then the line executed at step t is equal to $t \mod s$.

2. Suppose that P is the three line NAND-CIRC program listed below. Give a quantified integer statement $LAST(n,t,t')$ such that $LAST(t,t')$ is true if and only if $t' - n$ is the largest step smaller than $t - n$ in which the variable on the righthand side of the line executed at step $t - n$ is written to. If this variable is an input variable `x_i` then let $LAST(n,t,t')$ to be true if the current index location equals t' and $t' < n$.

■

```
y_0    := foo_i  NAND foo_i
foo_i  := x_i NAND x_i
loop := validx_i NAND validx_i
```

Exercise 10.5 — axiomatic proof systems. For every representation of logical statements as strings, we can define an axiomatic proof system to consist of a finite set of strings A and a finite set of rules $I_0, \ldots, I_{m-1}$ with $I_j : (\{0,1\}^*)^{k_j} \rightarrow \{0,1\}^*$ such that a proof $(s_1, \ldots, s_n)$ that s_n is true is valid if for every i, either $s_i \in A$ or is some $j \in [m]$ and are $i_1, \ldots, i_{k_j} < i$ such that $s_i = I_j(s_{i_1}, \ldots, i_{k_j})$. A system is *sound* if whenever there is no false s such that there is a proof that s is true Prove that for every uncomputable function $F : \{0,1\}^* \rightarrow \{0,1\}$ and every sound axiomatic proof system S (that is characterized by a finite number of axioms and inference rules), there is some input x for which the proof system S is not able to prove neither that $F(x) = 0$ nor that $F(x) \neq 0$.

■

10.6 BIBLIOGRAPHICAL NOTES

As mentioned before, Gödel, Escher, Bach [Hof99] is a highly recommended book covering Gödel's Theorem. A classic popular science book about Fermat's Last Theorem is [Sin97].

Cantor's are used for both Turing's and Gödel's theorems. In a twist of fate, using techniques originating from the works Gödel and Turing, Paul Cohen showed in 1963 that Cantor's *Continuum Hypothesis* is independent of the axioms of set theory, which means that neither it nor its negation is provable from these axioms and hence in some sense can be considered as "neither true nor false" (see [Coh08]). The Continuum Hypothesis is the conjecture that for every subset S of $\mathbb{R}$, either there is a one-to-one and onto map between S and $\mathbb{N}$ or there is a one-to-one and onto map between S and $\mathbb{R}$. It was conjectured by Cantor and listed by Hilbert in 1900 as one of the most important

problems in mathematics. See also the non-conventional survey of Shelah [She03]. See here for recent progress on a related question.

Thanks to Alex Lombardi for pointing out an embarrassing mistake in the description of Fermat's Last Theorem. (I said that it was open for exponent 11 before Wiles' work.)

III

EFFICIENT ALGORITHMS

Learning Objectives:

- **Describe at a high level some interesting computational problems.**
- **The difference between polynomial and exponential time.**
- **Examples of techniques for obtaining efficient algorithms**
- **Examples of how seemingly small differences in problems can make (at least apparent) huge differences in their computational complexity.**

11
Efficient computation

> *"The problem of distinguishing prime numbers from composite and of resolving the latter into their prime factors is ... one of the most important and useful in arithmetic ... Nevertheless we must confess that all methods ... are either restricted to very special cases or are so laborious ... they try the patience of even the practiced calculator ... and do not apply at all to larger numbers."*, Carl Friedrich Gauss, 1798

> *"For practical purposes, the difference between algebraic and exponential order is often more crucial than the difference between finite and non-finite."*, Jack Edmunds, "Paths, Trees, and Flowers", 1963

> *"What is the most efficient way to sort a million 32-bit integers?"*, Eric Schmidt to Barack Obama, 2008
>
> *"I think the bubble sort would be the wrong way to go."*, Barack Obama.

So far we have been concerned with which functions are computable and which ones are not. But now we return to *quantitative considerations* and study the time that it takes to compute functions mapping strings to strings, as a function of the input length. This is of course extremely important in the practice of computing, and the reason why we often care so much about the difference between $O(n \log n)$ time algorithm and $O(n^2)$ time one. In contexts such as introduction to programming courses, coding interviews, and actual algorithm design, terms such as "$O(n)$ runnning time" are often used in an informal way. That is, people don't have a precise definition of what a linear-time algorithm is, but rather assume that "they'll know it when they see it". However, in this course we will make precise

definitions, using our mathematical models of computation. This will allow us to ask (and sometimes answer) questions such as:

- "Is there a function that can be computed in $O(n^2)$ time but not in $O(n)$ time?"
- "Are there natural problems for which the *best* algorithm (and not just the *best known*) requires $2^{\Omega(n)}$ time?"

In this chapter we will survey some examples of computational problems, for some of which we know efficient (e.g., n^c-time for a small constant c) algorithms, and for others the best known algorithms are exponential. We want to get a feel as to the kinds of problems that lie on each side of this divide and also see how some seemingly minor changes in formulation can make the (known) complexity of a problem "jump" from polynomial to exponential. We will not formally define the notion of running time in this chapter, and so will use the same "I know it when I see it" notion of an $O(n)$ or $O(n^2)$ time algorithms as one you've seen in introduction to computer science courses. In Chapter 12, we will define this notion precisely, using our NAND-TM and NAND-RAM programming languages.

One of the nice things about the theory of computation is that it turns out that, like in the context of computability, the details of th precise computational model or programming language don't matter that much. Specifically, in this course, we will often not be as concerned with the difference between $O(n)$ and $O(n^2)$, as much as the difference between *polynomial* and *exponential* running time. One of the interesting phenomenona of computing is that there is often a kind of a "threshold phenomenon" or "zero-one law" for running time, where many natural problems can either be solved in polynomial running time with a not-too-large exponent (e.g., something like $O(n^2)$ or $O(n^3)$), or require exponential (e.g., at least $2^{\Omega(n)}$ or $2^{\Omega(\sqrt{n})}$) time to solve. The reasons for this phenomenon are still not fully understood, but some light on this is shed by the concept of *NP completeness*, which we will encounter later. As we will see, questions about polynomial versus exponential time are often *insensitive* to the choice of the particular computational model, just like we saw that the question of whether a function F is computable is insensitive to whether you use NAND-TM, λ-calculus, Turing machines, or Javascript as your model of computation.

11.1 PROBLEMS ON GRAPHS

We now present a few examples of computational problems that people are interested in solving. Many of the problems will involve *graphs*.

We have already encountered graphs in the context of Boolean circuits, but let us now quickly recall the basic notation. A graph G consists of a set of *vertices* V and *edges* E where each edge is a pair of vertices. In a *directed* graph, an edge is an ordered pair (u, v), which we sometimes denote as $\overrightarrow{u\,v}$. In an *undirected* graph, an edge is an unordered pair (or simply a set) $\{u, v\}$ which we sometimes denote as $\overline{u\,v}$ or $u \sim v$.[1] We will assume graphs are undirected and *simple* (i.e., containing no parallel edges or self-loops) unless stated otherwise.

We typically will think of the vertices in a graph as simply the set $[n]$ of the numbers from 0 till $n-1$. Graphs can be represented either in the *adjacency list* representation, which is a list of n lists, with the ith list corresponding to the neighbors of the ith vertex, or the *adjacency matrix* representation, which is an $n \times n$ matrix A with $A_{i,j}$ equalling 1 if the edge $\overrightarrow{u\,v}$ is present and equalling 0 otherwise.[2] We can transform between these two representations using $O(n^2)$ operations, and hence for our purposes we will mostly consider them as equivalent. We will sometimes consider *labeled* or *weighted* graphs, where we assign a label or a number to the edges or vertices of the graph, but mostly we will try to keep things simple and stick to the basic notion of an unlabeled, unweighted, simple undirected graph.

There is a reason that graphs are so ubiquitous in computer science and other sciences. They can be used to model a great many of the data that we encounter. These are not just the "obvious" networks such as the road network (which can be thought of as a graph of whose vertices are locations with edges corresponding to road segments), or the web (which can be thought of as a graph whose vertices are web pages with edges corresponding to links), or social networks (which can be thought of as a graph whose vertices are people and the edges correspond to friend relation). Graphs can also denote correlations in data (e.g., graph of observations of features with edges corresponding to features that tend to appear together), causal relations (e.g., gene regulatory networks, where a gene is connected to gene products it derives), or the state space of a system (e.g., graph of configurations of a physical system, with edges corresponding to states that can be reached from one another in one step).

We now give some examples of computational problems on graphs. As mentioned above, to keep things simple, we will restrict our attention to undirected simple graphs. In all cases the input graph $G = (V, E)$ will have n vertices and m edges.

[1] An equivalent viewpoint is that an undirected graph is like a directed graph with the property that whenever the edge $\overrightarrow{u\,v}$ is present then so is the edge $\overrightarrow{v\,u}$.

[2] In an undirected graph, the adjacency matrix A is *symmetric*, in the sense that it satisfies $A_{i,j} = A_{j,i}$.

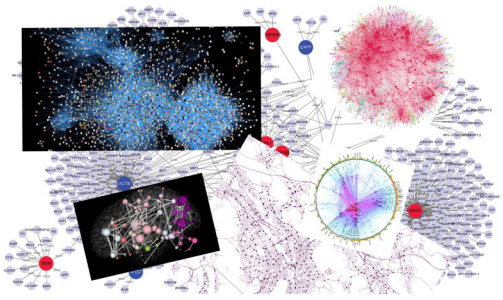


Figure 11.1: Some examples of graphs found on the Internet.

11.1.1 Finding the shortest path in a graph

The *shortest path problem* is the task of, given a graph $G = (V, E)$ and two vertices $s, t \in V$, to find the length of the shortest path between s and t (if such a path exists). That is, we want to find the smallest

number k such that there are vertices $v_0, v_1, \ldots, v_k$ with $v_0 = s$, $v_k = t$ and for every $i \in \{0, \ldots, k-1\}$ an edge between v_i and v_{i+1}. Formally, we define $MINPATH : \{0,1\}^* \to \{0,1\}^*$ to be the function that on input a triple (G, s, t) (represented as a string) outputs the number k which is the length of the shortest path in G between s and t or a string representing `no path` if no such path exists. (In practice people often want to also find the actual path and not just its length; it turns out that the algorithms to compute the length of the path often yield the actual path itself as a byproduct, and so everything we say about the task of computing the length also applies to the task of finding the path.)

If each vertex has at least two neighbors then there can be an *exponential* number of paths from s to t, but fortunately we do not have to enumerate them all to find the shortest path. We can do so by performing a breadth first search (BFS), enumerating s's neighbors, and then neighbors' neighbors, etc.. in order. If we maintain the neighbors in a list we can perform a BFS in $O(n^2)$ time, while using a queue we can do this in $O(m)$ time.[3]

More formally, the algorithm for computing the function *MINPATH* can be described as follows:

Algorithm BFSPATH:

- **Input:** Graph $G = (V, E)$, vertices s, t
- **Goal:** Find the length k of the shortest path $v_0, v_1, \ldots, v_k$ such that $v_0 = s$, $v_k = t$ and $\{v_i, v_{i+1}\} \in E$ for every $i \in [k]$, if such a path exists.
- **Operation:**
  1. We maintain a *queue* Q of vertices, initially Q contains only the pair s.
  2. We maintain a *dictionary*[4] D keyed by the vertices, for every vertex v, $D[v]$ is either equal to a natural number or to ∞. Initially we set set $D[s] = 0$ and $D[v] = \infty$ for every $v \in V \setminus \{s\}$.
  3. While Q is not empty do the following:
     a. Pop a vertex v from the top of the queue.
     b. If $v = t$ then halt and output $D[v]$.
     c. Otherwise, for every neighbor w of v such that $D[w] = \infty$, set $D[w] = D[v] + 1$ and add w to the queue.
  4. Output "no path"

[3] A *queue* stores a list of elements in "First In First Out (FIFO)" order and so each "pop" operation removes an element from the queue in the order that they were "pushed" into it; see the Wikipedia page. Since we assume $m \geq n - 1$, $O(m)$ is the same as $O(n + m)$. Dijkstra's algorithm is a well-known generalization of BFS to *weighted* graphs.

[4] A *dictionary* or associative array data structure D allows to associate with every key v (which can be thought of as a string) a value $D[v]$.

Since we only add to the queue vertices w with $D[w] = \infty$ (and then immediately set $D[w]$ to an actual number), we never push to the queue a vertex more than once, and hence the algorithm takes n "push" and "pop" operations. It returns the correct answer since add the vertices to the queue in the order of their distance from s, and hence we will reach t after we have explored all the vertices that are closer to s than t. Hence algorithm **BFSPATH** computes *MINPATH*.



Remark 11.1 — On data structures. If you've ever taken an algorithms course, you have probably encountered many *data structures* such as **lists**, **arrays**, **queues**, **stacks**, **heaps**, **search trees**, **hash tables** and many mores. Data structures are extremely important in computer science, and each one of those offers different tradeoffs between overhead in storage, operations supported, cost in time for each operation, and more. For example, if we store n items in a list, we will need a linear (i.e., $O(n)$ time) scan to retreive one of them, while we achieve the same operation in $O(1)$ time if we used a hash table. However, when we only care about polynomial-time algorithms, such factors of $O(n)$ in the running time will not make much difference. Similarly, if we don't care about the difference between $O(n)$ and $O(n^2)$, then it doesn't matter if we represent graphs as adjacency lists or adjacency matrices. Hence we will often describe our algorithms at a very high level, without specifying the particular data structures that are used to implement them. It should however be always clear that there exists *some* data structure that will be sufficient for our purposes.

11.1.2 Finding the longest path in a graph

The *longest path problem* is the task of, given a graph $G = (V, E)$ and two vertices $s, t \in V$, to find the length of the *longest* simple (i.e., non intersecting) path between s and t. If the graph is a road network, then the longest path might seem less motivated than the shortest path, but of course graphs can be and are used to model a variety of phenomena, and in many such cases the longest path (and some of its variants) are highly motivated. In particular, finding the longest path is a generalization of the famous Hamiltonian path problem which asks for a *maximally long* simple path (i.e., path that visits all n vertices once) between s and t, as well as the notorious traveling salesman problem (TSP) of finding (in a weighted graph) a path visiting all vertices of cost at most w. TSP is a classical optimization problem, with applications ranging from planning and logistics to DNA sequencing and astronomy.

A priori it is not clear that finding the longest path should be harder than finding the shortest path, but this turns out to be the case. While we know how to find the shortest path in $O(n)$ time, for the longest path problem we have not been able to significantly improve upon the trivial brute force algorithm that tries all paths.

Specifically, in a graph of degree at most d, we can enumerate over all paths of length k by going over the (at most d) neighbors of each vertex. This would take about $O(d^k)$ steps, and since the longest simple path can't have length more than the number of vertices, this means that the brute force algorithms runs in $O(d^n)$ time (which we can bound by $O(n^n)$ since the maximum degree is n). The best algorithm for the longest path improves on this, but not by much: it takes $\Omega(c^n)$ time for some constant $c > 1$.[5]

11.1.3 Finding the minimum cut in a graph

Given a graph $G = (V, E)$, a *cut* is a subset S of V such that S is neither empty nor is it all of V. The edges cut by S are those edges where one of their endpoints is in S and the other is in $\overline{S} = V \setminus S$. We denote this set of edges by $E(S, \overline{S})$. If $s, t \in V$ then an s, t *cut* is a cut such that $s \in S$ and $t \in \overline{S}$. (See Fig. 11.3.) The *minimum* s, t *cut problem* is the task of finding, given s and t, the minimum number k such that there is an s, t cut cutting k edges (once again, the problem is also sometimes phrased as finding the set that achieves this minimum; it turns out that algorithms to compute the number often yield the set as well).[6] Formally, we define $MINCUT : \{0,1\}^* \to \{0,1\}^*$ to be the function that on input a triple (G, s, t) of a graph and two vertices (represented as a string), outputs the minimum number k such that there exists a set S containing s and not t with exactly k edges that touch S and its complement.

The minimum s, t cut problem appears in many applications. Minimum cuts often correspond to *bottlenecks*. For example, in a communication network the minimum cut between s and t corresponds to the smallest number of edges that, if dropped, will disconnect s from t. Similar applications arise in scheduling and planning. In the setting of image segmentation, one can define a graph whose vertices are pixels and whose edges correspond to neighboring pixels of distinct colors. If we want to separate the foreground from the background then we can pick (or guess) a foreground pixel s and background pixel t and ask for a minimum cut between them.

Here is an algorithm to compute *MINCUT*:

Algorithm MINCUTNAIVE:

- **Input:** Graph $G = (V, E)$ and two distinct vertices $s, t \in V$

[5] At the moment the best record is $c \sim 1.65$ or so. Even obtaining an $O(2^n)$ time bound is not that simple, see Exercise 11.1.

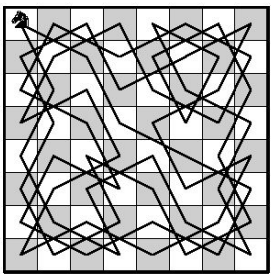


Figure 11.2: A *knight's tour* can be thought of as a maximally long path on the graph corresponding to a chessboard where we put an edge between any two squares that can be reached by one step via a legal knight move.

[6] One can also define the problem of finding the *global minimum cut* (i.e., the non-empty and non-everything set S that minimizes the number of edges cut). A polynomial time algorithm for the minimum s, t cut can be used to solve the global minimum cut in polynomial time as well (can you see why?).

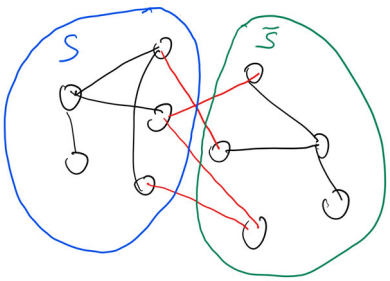


Figure 11.3: A *cut* in a graph $G = (V, E)$ is simply a subset S of its vertices. The edges that are *cut* by S are all those whose one endpoint is in S and the other one is in $\overline{S} = V \setminus S$. The cut edges are colored red in this figure.

- **Goal:** Return $k = \min_{S \subseteq V, s \in S, t \notin S} |E(S, \overline{S})|$
- **Operation:**
  1. Let $k_0 \leftarrow |E| + 1$
  2. For every set $S \subseteq V$ such that $s \in S$ and $t \notin T$ do:
     a. Set $k = 0$.
     b. For every edge $\{u, v\} \in E$, if $u \in S$ and $v \notin S$ then set $k \leftarrow k + 1$.
     c. If $k < k_0$ then let $k_0 \leftarrow k$
  3. Return k_0

P

It is an excellent exercise for you to pause at this point and verify: **(1)** that you understand what this algorithm does, **(2)** that you understand why this algorithm will in fact return the value of the minimum cut in the graph, and **(3)** that you can analyze the running time of this algorithm.

The precise running time of algorithm **MINCUTNAIVE** will depend on the data structures we use to store the graph and the sets, but even if we had the best data structures, the running time of **MINCUTNAIVE** will be terrible. Indeed, if a graph has n vertices, then for every pair s, t of distinct vertices, there are 2^{n-2} sets S that contain s but don't contain t. (Can you see why?) Since we are enumerating over all of those in Step 2, even if we could compute for each such set S the value $|E(S, \overline{S})|$ in constant time, our running time would still be exponential.

Since minimum cut is a problem we want to solve, this seems like bad news. After all, **MINCUTNAIVE** is the most natural algorithm to solve the problem, and if it takes exponential time, then perhaps the problem can't be solved efficiently at all. However, this turns out not to be case. As we've seen in this course time and again, there is a difference between the *function MINCUT* and the *algorithm* **MINCUTNAIVE** to solve it. There can be more than one algorithm to compute the same function, and some of those algorithms might be more efficient than others. Luckily this is one of those cases. There do exist much faster algorithms that compute *MINCUT* in *polynomial time* (which, as mentioned in the mathematical background lecture, we denote by $poly(n)$).

There are several algorithms to do so, but many of them rely on the Max-Flow Min-Cut Theorem that says that the minimum cut between s and t equals the maximum amount of *flow* we can send

from s to t, if every edge has unit capacity. Specifically, imagine that every edge of the graph corresponded to a pipe that could carry one unit of water per one unit of time (say 1 liter of water per second). Now suppose we want to send a maximum amount of water per time unit from our *source* s to the *sink* t. If there is an s,t-cut of at most k edges, then this maximum will be at most k. Indeed, such a cut S will be a "bottleneck" since at most k units can flow from S to its complement $\overline{S}$. The above reasoning can be used to show that the maximum flow from s to t is *at most* the value of the minimum s,t-cut. The surprising and non-trivial content of the Max-Flow Min-Cut Theorem is that the maximum flow is also *at leat* the value of the minimum cut, and hence computing the cut is the same as computing the flow.

A *flow* on a graph G of m edges can be thought of as a vector $x \in \mathbb{R}^m$ where for every edge e, x_e corresponds to the amount of water per time-unit that flows on e. We think of an edge e an an ordered pair (u,v) (we can choose the order arbitrarily) and let x_e be the amount of flow that goes from u to v. (If the flow is in the other directoin then we make x_e negative.) Since every edge has capacity one, we know that $-1 \leq x_e \leq 1$ for every edge e. A valid flow has the property that the amount of water leaving the source s is the same as the amount entering the sink t, and that for every other vertex v, the amount of water entering and leaving v is the same.

Mathematically, we can write these conditions as follows:

$$\begin{aligned} \sum_{e \ni s} x_e + \sum_{e \ni t} x_e &= 0 \\ \sum_{e \ni v} x_e &= 0 \quad \forall_{v \in V \setminus \{s,t\}} \\ -1 \leq x_e &\leq 1 \quad \forall_{e \in E} \end{aligned} \tag{11.1}$$

where for every vertex v, summing over $e \ni v$ means summing over all the edges that touch v.

The maximum flow problem can be thought of as the task of maximizing $\sum_{e \ni s} x_e$ over all the vectors $x \in \mathbb{R}^m$ that satisfy the above conditions (11.1). This is a special case of a very general task known as linear programming, where one wants to find the maximum of $f(x)$ over $x \in \mathbb{R}^m$ that satisfies certain linear inequalities where $f : \mathbb{R}^m \to \mathbb{R}$ is a linear function. Luckily, there are polynomial-time algorithms for solving linear programming, and hence we can solve the maximum flow (and so, equivalently, minimum cut) problem in polynomial time. In fact, there are much better algorithms for maximum-flow/minimum-cut, even for weighted directed graphs, with currently the record standing at $O(\min\{m^{10/7}, m\sqrt{n}\})$.[7]

[7] TODO: add references in biliographical notes: Madry, Lee-Sidford

11.1.4 Finding the maximum cut in a graph

We can also define the *maximum cut* problem of finding, given a graph $G = (V, E)$ the subset $S \subseteq V$ that *maximizes* the number of edges cut by S.[8] Like its cousin the minimum cut problem, the maximum cut problem is also very well motivated. For example, it arises in VLSI design, and also has some surprising relation to analyzing the Ising model in statistical physics.

Once again, a priori it might not be clear that the maximum cut problem should be harder than minimum cut but this turns out to be the case. We do not know of an algorithm that solves this problem much faster than the trivial "brute force" algorithm that tries all 2^n possibilities for the set S.

[8] We can also consider the variant where one is given s, t and looks for the s, t-cut that maximizes the number of edges cut. The two variants are equivalent up to $O(n^2)$ factors in the running time, but we use the global max cut forumlation since it is more common in the literature.

11.1.5 A note on convexity

There is an underlying reason for the sometimes radical difference between the difficulty of maximizing and minimizing a function over a domain. If $D \subseteq \mathbb{R}^n$, then a function $f : D \to R$ is *convex* if for every $x, y \in D$ and $p \in [0, 1]$ $f(px + (1 - p)y) \leq pf(x) + (1 - p)f(y)$. That is, f applied to the p-weighted midpoint between x and y is smaller than the p-weighted average value of f. If D itself is convex (which means that if x, y are in D then so is the line segment between them), then this means that if x is a *local minimum* of f then it is also a *global minimum*. The reason is that if $f(y) < f(x)$ then every point $z = px + (1 - p)y$ on the line segment between x and y will satisfy $f(z) \leq pf(x) + (1 - p)f(y) < f(x)$ and hence in particular x cannot be a local minimum. Intuitively, local minima of functions are much easier to find than global ones: after all, any "local search" algorithm that keeps finding a nearby point on which the value is lower, will eventually arrive at a local minima.[9] Indeed, under certain technical conditions, we can often efficiently find the minimum of convex functions, and this underlies the reason problems such as minimum cut and shortest path are easy to solve. On the other hand, *maximizing* a convex function (or equivalently, minimizing a *concave* function) can often be a hard computational task. A *linear* function is both convex and concave, which is the reason both the maximization and minimization problems for linear functions can be done efficiently.

The minimum cut problem is not a priori a convex minimization task, because the set of potential cuts is *discrete*. However, it turns out that we can embed it in a continuous and convex set via the (linear) maximum flow problem. The "max flow min cut" theorem ensuring that this embedding is "tight" in the sense that the minimum "fractional cut" that we obtain through the maximum-flow linear program will be the same as the true minimum cut. Unfortunately, we don't

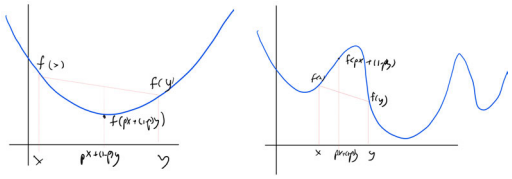


Figure 11.4: In a *convex* function f (left figure), for every x and y and $p \in [0, 1]$ it holds that $f(px + (1 - p)y) \leq p \cdot f(x) + (1-p) \cdot f(y)$. In particular this means that every *local minimum* of f is also a *global minimum*. In contrast in a *non convex* function there can be many local minima.

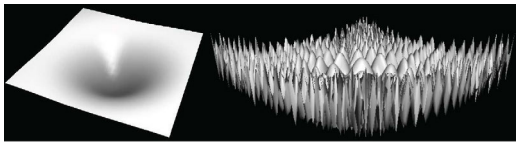


Figure 11.5: In the high dimensional case, if f is a *convex* function (left figure) the global minimum is the only local minimum, and we can find it by a local-search algorithm which can be thought of as dropping a marble and letting it "slide down" until it reaches the global minimum. In contrast, a non-convex function (right figure) might have an exponential number of local minima in which any local-search algorithm could get stuck.

[9] One example of such a local search algorithm is gradient descent which takes a small step in the direction that would reduce the value by the most amount based on the current derivative. There are also algorithms that take advantage of the *second derivative* (hence are known as *second order methods*) to potentially converge faster.

know of such a tight embedding in the setting of the *maximum* cut problem.

The issue of convexity arises time and again in the context of computation. For example, one of the basic tasks in machine learning is *empirical risk minimization*. That is, given a set of labeled examples $(x_1, y_1), \ldots, (x_m, y_m)$, where each $x_i \in \{0,1\}^n$ and $y_i \in \{0,1\}$, we want to find the function $h : \{0,1\}^n \to \{0,1\}$ from some class H that minimizes the *error* in the sense of minimizing the number of i's such that $h(x_i) \neq y_i$. Like in the minimum cut problem, to make this a better behaved computational problem, we often embed it in a continuous domain, including functions that could output a real number and replacing the condition $h(x_i) \neq y_i$ with minimizing some continuous *loss function* $\ell(h(x_i), y_i)$.[10] When this embedding is *convex* then we are guaranteed that the global minimizer is unique and can be found in polynomial time. When the embedding is *non convex*, we have no such guarantee and in general there can be many global or local minima. That said, even if we don't find the global (or even a local) minima, this continuous embedding can still help us. In particular, when running a local improvement algorithm such as Gradient Descent, we might still find a function h that is "useful" in the sense of having a small error on future examples from the same distribution.[11]

[10] We also sometimes replace or enhance the condition that h is in the class H by adding a *regularizing term* of the form $R(h)$ to the minimization problem, where $R : H \to \mathbb{R}$ is some measure of the "complexity" of h. As a general rule, the larger or more "complex" functions h we allow, the easier it is to fit the data, but the more danger we have of "overfitting".

[11] In machine learning parlance, this task is known as *supervised learning*. The set of examples $(x_1, y_1), \ldots, (x_m, y_m)$ is known as the *training set*, and the error on additional samples from the same distribution is known as the *generalization error*, and can be measured by checking h against a *test set* that was not used in training it.

11.2 BEYOND GRAPHS

Not all computational problems arise from graphs. We now list some other examples of computational problems that are of great interest.

11.2.1 The 2SAT problem

A *propositional formula* φ involves n variables $x_1, \ldots, x_n$ and the logical operators AND ($\wedge$), OR ($\vee$), and NOT ($\neg$, also denoted as $\bar{\cdot}$). We say that such a formula is in *conjunctive normal form* (CNF for short) if it is an AND of ORs of variables or their negations (we call a term of the form x_i or $\overline{x}_i$ a *literal*). For example, this is a CNF formula

$$(x_7 \vee \overline{x}_{22} \vee x_{15}) \wedge (x_{37} \vee x_{22}) \wedge (x_{55} \vee \overline{x}_7) \tag{11.2}$$

We say that a formula is a k-CNF it is an AND of ORs where each OR involves exactly k literals. The 2SAT problem is to find out, given a 2-CNF formula φ, whether there is an assignment $x \in \{0,1\}^n$ that *satisfies* φ, in the sense that it makes it evaluate to 1 or "True".

Determining the satisfiability of Boolean formulas arises in many applications and in particular in software and hardware verification, as well as scheduling problems. The trivial, brute-force, algorithm for 2SAT will enumerate all the 2^n assignments $x \in \{0,1\}^n$ but fortunately we can do much better.

The key is that we can think of every constraint of the form $\ell_i \vee \ell_j$ (where ℓ_i, ℓ_j are *literals*, corresponding to variables or their negations) as an *implication* $\overline{\ell}_i \Rightarrow \ell_j$, since it corresponds to the constraints that if the literal $\ell'_i = \overline{\ell}_i$ is true then it must be the case that ℓ_j is true as well. Hence we can think of φ as a directed graph between the $2n$ literals, with an edge from ℓ_i to ℓ_j corresponding to an implication from the former to the latter. It can be shown that φ is unsatisfiable if and only if there is a variable x_i such that there is a directed path from x_i to $\overline{x}_i$ as well as a directed path from $\overline{x}_i$ to x_i (see Exercise 11.2). This reduces 2SAT to the (efficiently solvable) problem of determining connectivity in directed graphs.

11.2.2 The 3SAT problem

The 3SAT problem is the task of determining satisfiability for 3CNFs. One might think that changing from two to three would not make that much of a difference for complexity. One would be wrong. Despite much effort, we do not know of a significantly better than brute force algorithm for 3SAT (the best known algorithms take roughy 1.3^n steps).

Interestingly, a similar issue arises time and again in computation, where the difference between two and three often corresponds to the difference between tractable and intractable. We do not fully understand the reasons for this phenomenon, though the notions of **NP** completeness we will see later does offer a partial explanation. It may be related to the fact that optimzing a polynomial often amounts to equations on its derivative. The derivative of a a quadratic polynomial is linear, while the derivative of a cubic is quadratic, and, as we will see, the difference between solving linear and quadratic equations can be quite profound.

11.2.3 Solving linear equations

One of the most useful problems that people have been solving time and again is solving n linear equations in n variables. That is, solve equations of the form

$$\begin{array}{ccccc} a_{0,0}x_0 + a_{0,1}x_1 & + \cdots & + a_{0,n-1}x_{n-1} & = b_0 \\ a_{1,0}x_0 + a_{1,1}x_1 & + \cdots & + a_{1,n-1}x_{n-1} & = b_1 \\ \vdots + \vdots & + \vdots & + \vdots & = \vdots \\ a_{n-1,0}x_0 + a_{n-1,1}x_1 & + \cdots & + a_{n-1,n-1}x_{n-1} & = b_{n-1} \end{array} \tag{11.3}$$

where $\{a_{i,j}\}_{i,j\in[n]}$ and $\{b_i\}_{i\in[n]}$ are real (or rational) numbers. More compactly, we can write this as the equations $Ax = b$ where A is an $n \times n$ matrix, and we think of x, b are column vectors in $\mathbb{R}^n$.

The standard Gaussian elimination algorithm can be used to solve such equations in polynomial time (i.e., determine if they have a solution, and if so, to find it).[12] As we discussed above, if we are willing to allow some loss in precision, we even have algorithms that handle linear *inequalities*, also known as linear programming. In contrast, if we insist on *integer* solutions, the task of solving for linear equalities or inequalities is known as integer programming, and the best known algorithms are exponential time in the worst case.

[12] To analyze this fully we need to ensure that the bit complexity of the numbers involved does not grow too much, but fortunately we can indeed ensure this using Cramer's rule. Also, as is usually the case when talking about real numbers, we do not care much for the distinction between solving equations exactly and solving them to arbitrarily good precision.

R

Remark 11.2 — Bit complexity of numbers. Whenever we discuss problems whose inputs correspond to numbers, the input length corresponds to how many bits are needed to describe the number (or, as is equivalent up to a constant factor, the number of digits in base 10, 16 or any other constant). The difference between the length of the input and the magnitude of the number itself can be of course quite profound. For example, most people would agree that there is a huge difference between having a billion (i.e. 10^9) dollars and having nine dollars. Similarly there is a huge difference between an algorithm that takes n steps on an n-bit number and an algorithm that takes 2^n steps.

One example, is the problem (discussed below) of finding the prime factors of a given integer N. The natural algorithm is to search for such a factor by trying all numbers from 1 to N, but that would take N steps which is *exponential* in the input length, which is number of bits needed to describe N. [13] It is an important and long open question whether there is such an algorithm that runs in time polynomial in the input length (i.e., polynomial in $\log N$).

[13] The running time of this algorithm can be easily improved to roughly $\sqrt{N}$, but this is still exponential (i.e., $2^{n/2}$) in the number n of bits to describe N.

11.2.4 Solving quadratic equations

Suppose that we want to solve not just *linear* but also equations involving *quadratic* terms of the form $a_{i,j,k}x_j x_k$. That is, suppose that we are given a set of quadratic polynomials $p_1, \ldots, p_m$ and consider the equations $\{p_i(x) = 0\}$. To avoid issues with bit representations, we will always assume that the equations contain the constraints $\{x_i^2 - x_i = 0\}_{i\in[n]}$. Since only 0 and 1 satisfy the equation $a^2 - a$, this assumption means that we can restrict attention to solutions in $\{0,1\}^n$. Solving quadratic equations in several variable is a classical and extremely well motivated problem. This is the generalization of the classical case of single-variable quadratic equations that generations of high school students grapple with. It also generalizes the quadratic assignment problem, introduced in the 1950's as a way to optimize as-

signment of economic activities. Once again, we do not know a much better algorithm for this problem than the one that enumerates over all the 2^n possiblities.

11.3 MORE ADVANCED EXAMPLES

We now list a few more examples of interesting problems that are a little more advanced but are of significant interest in areas such as physics, economics, number theory, and cryptography.

11.3.1 Determinant of a matrix

The determinant of a $n \times n$ matrix A, denoted by $\det(A)$, is an extremely important quantity in linear algebra. For example, it is known that $\det(A) \neq 0$ if and only if A is *nonsingular*, which means that it has an inverse A^{-1}, and hence we can always uniquely solve equations of the form $Ax = b$ where x and b are n-dimensional vectors. More generally, the determinant can be thought of as a quantiative measure as to what extent A is far from being singular. If the rows of A are "almost" linearly dependent (for example, if the third row is very close to being a linear combination of the first two rows) then the determinant will be small, while if they are far from it (for example, if they are are *orthogonal* to one another, then the determinant will be large). In particular, for every matrix A, the absolute value of the determinant of A is at most the product of the norms (i.e., square root of sum of squares of entries) of the rows, with equality if and only if the rows are orthogonal to one another.

The determinant can be defined in several ways. For example, it is known that det is the only function that satisfies the following conditions:

1. $\det(AB) = \det(A)\det(B)$ for every square matrices A, B.
2. For every $n \times n$ *triangular* matrix T with diagonal entries $d_0, \ldots, d_{n-1}$, $\det(T) = \prod_{i=0}^{n} d_i$. In particular $\det(I) = 1$ where I is the identity matrix.[14]
3. $\det(S) = -1$ where S is a "swap matrix" that corresponds to swapping two rows or two columns of I. That is, there are two coordinates a, b such that for every i, j, $S_{i,j} = \begin{cases} 1 & i = j \text{ , } i \notin \{a,b\} \\ 1 & \{i,j\} = \{a,b\} \\ 0 & \text{otherwise} \end{cases}$.

[14] A *triangular* matrix is one in which either all entries below the diagonal, or all entries above the diagonal, are zero.

Note that conditions 1. and 2. together imply that $\det(A^{-1}) = \det(A)^{-1}$ for every invertible matrix A. Using these rules and the Gaussian elimination algorithm, it is possible to tell whether A is singular or not, and in the latter case, decompose A as a product of a

polynomial number of swap matrices and triangular matrices. (Indeed one can verify that the row operations in Gaussian elimination corresponds to either multiplying by a swap matrix or by a triangular matrix.) Hence we can compute the determinant for an $n \times n$ matrix using a polynomial time of arithmetic operations.[15]

[15] The cost for performing each arithmetic operation depends on the number of bits needed to represent each entry, and accounting for this can sometimes be subtle, though ultimately doable.

11.3.2 The permanent (mod 2) problem

Given an $n \times n$ matrix A, the *permanent* of A is the sum over all permutations π (i.e., π is a member of the set S_n of one-to-one and onto functions from $[n]$ to $[n]$) of the product $\prod_{i=0}^{n-1} A_{i,\pi(i)}$. The permanent of a matrix is a natural quantity, and has been studied in several contexts including combinatorics and graph theory. It also arises in physics where it can be used to describe the quantum state of multiple boson particles (see here and here).

If the entries of A are integers, then we can also define a *Boolean* function $perm_2(A)$ which will output the result of the permanent modulo 2. A priori computing this would seem to require enumerating over all $n!$ possiblities. However, it turns out we can compute $perm_2(A)$ in polynomial time! The key is that modulo 2, $-x$ and $+x$ are the same quantity and hence the permanent modulo 2 is the same as taking the following quantity modulo 2:

$$\sum_{\pi \in S_n} sign(\pi) \prod_{i=0}^{n-1} A_{i,\pi(i)} \tag{11.4}$$

where the *sign* of a permutation π is a number in $\{+1, -1\}$ which can be defined in several ways, one of which is that $sign(\pi)$ equals $+1$ if the number of swaps that "Bubble" sort performs starting an array sorted according to π is even, and it equals -1 if this number is odd.[16]

[16] It turns out that this definition is independent of the sorting algorithm, and for example if $sign(\pi) = -1$ then one cannot sort an array ordered according to π using an even number of swaps.

From a first look, (11.4) does not seem like it makes much progress. After all, all we did is replace one formula involving a sum over $n!$ terms with an even more complicated formula involving a sum over $n!$ terms. But fortunately (11.4) also has an alternative description: it is yet another way to describe the determinant of the matrix A, which as mentioned can be computed using a process similar to Gaussian elimination.

11.3.3 The permanent (mod 3) problem

Emboldened by our good fortune above, we might hope to be able to compute the permanent modulo any prime p and perhaps in full generality. Alas, we have no such luck. In a similar "two to three" type of a phenomenon, we do not know of a much better than brute force algorithm to even compute the permanent modulo 3.

11.3.4 Finding a zero-sum equilibrium

A *zero sum game* is a game between two players where the payoff for one is the same as the penalty for the other. That is, whatever the first player gains, the second player loses. As much as we want to avoid them, zero sum games do arise in life, and the one good thing about them is that at least we can compute the optimal strategy.

A zero sum game can be specified by an $n \times n$ matrix A, where if player 1 chooses action i and player 2 chooses action j then player one gets $A_{i,j}$ and player 2 loses the same amount. The famous Min Max Theorem by John von Neumann states that if we allow probabilistic or "mixed" strategies (where a player does not choose a single action but rather a *distribution* over actions) then it does not matter who plays first and the end result will be the same. Mathematically the min max theorem is that if we let Δ_n be the set of probability distributions over $[n]$ (i.e., non-negative columns vectors in $\mathbb{R}^n$ whose entries sum to 1) then

$$\max_{p \in \Delta_n} \min_{q \in \Delta_n} p^\top A q = \min_{q \in \Delta_n} \max_{p \in \Delta_n} p^\top A q \tag{11.5}$$

The min-max theorem turns out to be a corollary of linear programming duality, and indeed the value of (11.5) can be computed efficiently by a linear program.

11.3.5 Finding a Nash equilibrium

Fortunately, not all real-world games are zero sum, and we do have more general games, where the payoff of one player does not necessarily equal the loss of the other. John Nash won the Nobel prize for showing that there is a notion of *equilibrium* for such games as well. In many economic texts it is taken as an article of faith that when actual agents are involved in such a game then they reach a Nash equilibrium. However, unlike zero sum games, we do not know of an efficient algorithm for finding a Nash equilibrium given the description of a general (non zero sum) game. In particular this means that, despite economists' intuitions, there are games for which natural stategies will take exponential number of steps to converge to an equilibrium.

11.3.6 Primality testing

Another classical computational problem, that has been of interest since the ancient greeks, is to determine whether a given number N is prime or composite. Clearly we can do so by trying to divide it with all the numbers in $2, \ldots, N-1$, but this would take at least N steps which is *exponential* in its bit complexity $n = \log N$. We can reduce these N steps to $\sqrt{N}$ by observing that if N is a composite of the form $N = PQ$ then either P or Q is smaller than $\sqrt{N}$. But this is

still quite terrible. If N is a 1024 bit integer, $\sqrt{N}$ is about 2^{512}, and so running this algorithm on such an input would take much more than the lifetime of the universe.

Luckily, it turns out we can do radically better. In the 1970's, Rabin and Miller gave *probabilistic* algorithms to determine whether a given number N is prime or composite in time $poly(n)$ for $n = \log N$. We will discuss the probabilistic model of computation later in this course. In 2002, Agrawal, Kayal, and Saxena found a deterministic $poly(n)$ time algorithm for this problem. This is surely a development that mathematicians from Archimedes till Gauss would have found exciting.

11.3.7 Integer factoring

Given that we can efficiently determine whether a number N is prime or composite, we could expect that in the latter case we could also efficiently *find* the factorization of N. Alas, no such algorithm is known. In a surprising and exciting turn of events, the *non existence* of such an algorithm has been used as a basis for encryptions, and indeed it underlies much of the security of the world wide web. We will return to the factoring problem later in this course. We remark that we do know much better than brute force algorithms for this problem. While the brute force algorithms would require $2^{\Omega(n)}$ time to factor an n-bit integer, there are known algorithms running in time roughly $2^{O(\sqrt{n})}$ and also algorithms that are widely believed (though not fully rigorously analyzed) to run in time roughly $2^{O(n^{1/3})}$.[17]

[17] The "roughly" adjective above refers to neglecting factors that are polylogarithmic in n.

11.4 OUR CURRENT KNOWLEDGE

The difference between an exponential and polynomial time algorithm might seem merely "quantiative" but it is in fact extremely significant. As we've already seen, the brute force exponential time algorithm runs out of steam very very fast, and as Edmonds says, in practice there might not be much difference between a problem where the best algorithm is exponential and a problem that is not solvable at all. Thus the efficient algorithms we mention above are widely used and power many computer science applications. Moreover, a polynomial-time algorithm often arises out of significant insight to the problem at hand, whether it is the "max-flow min-cut" result, the solvability of the determinant, or the group theoretic structure that enables primality testing. Such insight can be useful regardless of its computational implications.

At the moment we do not know whether the "hard" problems are truly hard, or whether it is merely because we haven't yet found the right algorithms for them. However, we will now see that there are

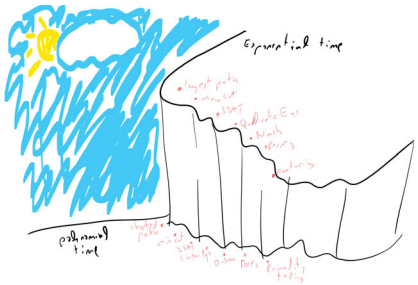


Figure 11.6: The current computational status of several interesting problems. For all of them we either know a polynomial-time algorithm or the known algorithms require at least 2^{n^c} for some $c > 0$. In fact for all except the *factoring* problem, we either know an $O(n^3)$ time algorithm or the best known algorithm require at least $2^{\Omega(n)}$ time where n is a natural parameter such that there is a brute force algorithm taking roughly 2^n or $n!$ time. Whether this "cliff" between the easy and hard problem is a real phenomenon or a reflection of our ignorane is still an open question.

problems that do *inherently require* exponential time. We just don't know if any of the examples above fall into that category.

11.5 LECTURE SUMMARY

- There are many natural problems that have polynomial-time algorithms, and other natural problems that we'd love to solve, but for which the best known algorithms are exponential.
- Often a polynomial time algorithm relies on discovering some hidden structure in the problem, or finding a surprising equivalent formulation for it.
- There are many interesting problems where there is an *exponential gap* between the best known algorithm and the best algorithm that we can rule out. Closing this gap is one of the main open questions of theoretical computer science.

11.6 EXERCISES

R

Remark 11.3 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 11.1 — exponential time algorithm for longest path. The naive algorithm for computing the longest path in a given graph could take more than $n!$ steps. Give a $poly(n)2^n$ time algorithm for the longest path problem in n vertex graphs.[18]

[18] **Hint:** Use dynamic programming to compute for every $s, t \in [n]$ and $S \subseteq [n]$ the value $P(s,t,S)$ which equals 1 if there is a simple path from s to t that uses exactly the vertices in S. Do this iteratively for S's of growing sizes.

Exercise 11.2 — 2SAT algorithm. For every 2CNF φ, define the graph G_φ on $2n$ vertices corresponding to the literals $x_1, \ldots, x_n, \overline{x}_1, \ldots, \overline{x}_n$, such that there is an edge $\overrightarrow{\ell_i\ \ell_j}$ iff the constraint $\overline{\ell}_i \vee \ell_j$ is in φ. Prove that φ is unsatisfiable if and only if there is some i such that there is a path from x_i to $\overline{x}_i$ and from $\overline{x}_i$ to x_i in G_φ. Show how to use this to solve 2SAT in polynomial time.

11.7 BIBLIOGRAPHICAL NOTES

19

[19] TODO: add reference to best algorithm for longest path - probably the Bjorklund algorithm

11.8 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

11.9 ACKNOWLEDGEMENTS

Learning Objectives:

- **Formally modeling running time, and in particular notions such as $O(n)$ or $O(n^3)$ time algorithms.**
- **The classes P and EXP modelling polynomial and exponential time respectively.**
- **The *time hierarchy theorem*, that in particular says that for every $k \geq 1$ there are functions we *can* compute in $O(n^{k+1})$ time but *can not* compute in $O(n^k)$ time.**
- **The class $\mathbf{P}_{/\mathbf{poly}}$ of *non uniform* computation and the result that $\mathbf{P} \subseteq \mathbf{P}_{/\mathbf{poly}}$**

12
Modeling running time

> "When the measure of the problem-size is reasonable and when the sizes assume values arbitrarily large, an asymptotic estimate of ... the order of difficulty of [an] algorithm .. is theoretically important. It cannot be rigged by making the algorithm artificially difficult for smaller sizes", Jack Edmonds, "Paths, Trees, and Flowers", 1963

> "The computational complexity of a sequence is to be measured by how fast a multitape Turing machine can print out the terms of the sequence. This particular abstract model of a computing device is chosen because much of the work in this area is stimulated by the rapidly growing importance of computation through the use of digital computers, and all digital computers in a slightly idealized form belong to the class of multitape Turing machines.", Juris Hartmanis and Richard Stearns, "On the computational complexity of algorithms", 1963.

In Chapter 11 we saw examples of efficient algorithms, and made some claims about their running time, but did not give a mathematically precise definition for this concept. We do so in this chapter, using the NAND-TM and NAND-RAM models we have seen before.[1] Since we think of programs that can take as input a string of arbitrary length, their running time is not a fixed number but rather what we are interested in is measuring the *dependence* of the number of steps the program takes on the length of the input. That is, for any program P, we will be interested in the maximum number of steps that P takes on inputs of length n (which we often denote as $T(n)$).[2] For example, if a function F can be computed by a NAND-RAM (or NAND-TM program/Turing machine) program that on inputs of length n takes $O(n)$ steps then we will think of F as "efficiently computable", while if

[1] NAND-TM programs are a variant of Turing machines, while NAND-RAM programs are a way to model RAM machines, and hence all of the discussion in this chapter applies to those and many other models as well.

[2] Because we are interested in the *maximum* number of steps for inputs of a given length, this concept is often known as *worst case complexity*. The *minimum* number of steps (or "best case" complexity) to compute a function on length n inputs is typically not a meaningful quantity since essentially every natural problem will have some trivially easy instances. However, the *average case complexity* (i.e., complexity on a "typical" or "random" input) is an interesting concept which we'll return to when we discuss *cryptography*. That said, worst-case complexity is the most standard and basic of the complexity measures, and will be our focus in most of this course.

any such program requires $2^{\Omega(n)}$ steps to compute F then we consider F "intractable".

12.1 FORMALLY DEFINING RUNNING TIME

We start by defining running time separately for both NAND-RAM and NAND-TM programs. We will later see that the two measures are closely related. Roughly speaking, we will say that a function F is computable in time $T(n)$ there exists a NAND-RAM program that when given an input x, halts and outputs $F(x)$ within at most $T(|x|)$ steps. The formal definition is as follow:

Definition 12.1 — Running time. Let $T : \mathbb{N} \to \mathbb{N}$ be some function mapping natural numbers to natural numbers. We say that a function $F : \{0,1\}^* \to \{0,1\}$ is *computable in $T(n)$ NAND-RAM time* if there exists a NAND-RAM program P such that for every every sufficiently large n and every $x \in \{0,1\}^n$, when given input x, the program P halts after executing at most $T(n)$ lines and outputs $F(x)$. [3]

Similarly, we say that F is *computable in $T(n)$ NAND-TM time* if there is a NAND-TM program P computing F such that on every sufficiently large n and $x \in \{0,1\}^n$, on input x, P executes at most $T(n)$ lines before it halts with the output $F(x)$.

We let $TIME_{<<}(T(n))$ denote the set of Boolean functions that are computable in $T(n)$ NAND-RAM time, and define $TIME_{++}(T(n))$ analogously. The set $TIME(T(n))$ (without any subscript) corresponds to $TIME_{<<}(T(n))$.

[3] The relaxation of considering only "sufficiently large" n's is not very important but it is convenient since it allows us to avoid dealing explicitly with un-interesting "edge cases". In most cases we will anyway be interested in determining running time only up to constant and even polynomial factors. Note that we can always compute a function on a finite number of inputs using a lookup table.



Definition 12.1 is not very complicated but is one of the most important definitions of this book. Please make sure to stop, re-read it, and make sure you understand it. Note that although they are defined using computational models, $TIME_{<<}(T(n))$ and $TIME_{++}(T(n))$ are classes of *functions*, not of *programs*. If P is a NAND-TM program then a statement such as "P is a member of $TIME_{++}(n^2)$" does not make sense.

In the definition of time complexity, we count the number of times a line is *executed*, not the number of lines in the program. For example, if a NAND-TM program P has 20 lines, and on some input x it takes a 1,000 iterations of its loop before it halts, then the number of lines executed on this input is 20,000.

To make this count meaningful, we use the "vanilla" flavors of NAND-TM and NAND-RAM, "unpacking" any syntactic sugar. For example, if a NAND-TM program P contains a line with the syntactic sugar `foo =`

`MACRO(bar)` where `MACRO` is some macro/function that is defined elsewhere using 100 lines of vanilla NAND-TM, then executing this line counts as executing 100 steps rather than a single one.

Unlike the notion of computability, the exact running time can be a function of the model we use. However, it turns out that if we only care about "coarse enough" resolution (as will most often be the case) then the choice of the model, whether it is NAND-RAM, NAND-TM, or Turing or RAM machines of various flavors, does not matter. (This is known as the *extended* Church-Turing Thesis). Nevertheless, to be concrete, we will use NAND-RAM programs as our "default" computational model for measuring time, which is why we say that F is computable in $T(n)$ time without any qualifications, or write $TIME(T(n))$ without any subscript, we mean that this holds with respect to NAND-RAM machines.



Remark 12.2 — Why NAND-RAM?. Given that so far we have emphasized NAND-TM as our "main" model of computation, the reader might wonder why we use NAND-RAM as the default yardstick where running time is involved. As we will see, this choice does not make much difference, but NAND-RAM or *RAM Machines* correspond more closely to the notion of running time as discussed in algorithms text or the practice of computer science.

12.1.1 Nice time bounds

The class $TIME(T(n))$ is defined in Definition 12.1 with respect to a *time* measure function $T(n)$. When considering time bounds, we will want to restrict our attention to "nice" bounds such as $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^{\sqrt{n}})$, $O(2^n)$, and so on. We do so to avoid pathological examples such as non-monotone functions (where the time to compute a function on inputs of size n could be smaller than the time to compute it on shorter inputs) or other degenerate cases such as running time that is not sufficient to read the input or cases where the running time bound itself is hard to compute. Thus we make the following definition:

Definition 12.3 — Nice functions. A function $T : \mathbb{N} \to \mathbb{N}$ is a *nice time bound function* (or nice function for short) if:

1. $T(n) \geq n$
2. $T(n) \geq T(n')$ whenever $n \geq n'$

3. There is a NAND-RAM program that on input numbers n, i, given in binary, can compute in $O(T(n))$ steps the i-th bit of a prefix-free representation of $T(n)$ (represented as a string in some prefix-free way).

All the "normal" time complexity bounds we encounter in applications such as $T(n) = 100n$, $T(n) = n^2 \log n$,$T(n) = 2^{\sqrt{n}}$, etc. are "nice". Hence from now on we will only care about the class $TIME(T(n))$ when T is a "nice" function. Condition 3. of Definition 12.3 means that we can compute the binary representation of $T(n)$ in time which itself is roughly $T(n)$. This condition is typically easily satisfied. For example, for arithmetic functions such as $T(n) = n^3$ or $T(n) = \lfloor n \rfloor^{1.2} \log n \rfloor$ we can typically compute the binary representation of $T(n)$ much faster than that: we can do so in time which is polynomial in the *number of bits* in this representation. Since the number of bits is $O(\log T(n))$, any quantity that is polynomial in this number will be much smaller than $T(n)$ for large enough n.

The two main time complexity classes we will be interested in are the following:

- **Polynomial time:** A function $F : \{0,1\}^* \to \{0,1\}$ is *computable in polynomial time* if it is in the class $\mathbf{P} = \cup_{c \in \{1,2,3,\ldots\}} TIME(n^c)$. That is, $F \in \mathbf{P}$ if there is an algorithm to compute F that runs in time at most *polynomial* (i.e., at most n^c for some constant c) in the length of the input.

- **Exponential time:** A function $F : \{0,1\}^* \to \{0,1\}$ is *computable in exponential time* if it is in the class $\mathbf{EXP} = \cup_{c \in \{1,2,3,\ldots\}} TIME(2^{n^c})$. That is, $F \in \mathbf{EXP}$ if there is an algorithm to compute F that runs in time at most *exponential* (i.e., at most 2^{n^c} for some constant c) in the length of the input.

In other words, these are defined as follows:

Definition 12.4 — P and EXP. Let $F : \{0,1\}^* \to \{0,1\}$. We say that $F \in \mathbf{P}$ if there is a polynomial $p : \mathbb{N} \to \mathbb{R}$ and a NAND-RAM program P such that for every $x \in \{0,1\}^*$, $P(x)$ runs in at most $p(|x|)$ steps and outputs $F(x)$.

We say that $F \in \mathbf{EXP}$ if there is a polynomial $p : \mathbb{N} \to \mathbb{R}$ and a NAND-RAM program P such that for every $x \in \{0,1\}^*$, $P(x)$ runs in at most $2^{p(|x|)}$ steps and outputs $F(x)$.



Please make sure you understand why Definition 12.4 and the bullets above define the same classes.

Since exponential time is much larger than polynomial time, $\mathbf{P} \subseteq \mathbf{EXP}$. All of the problems we listed in Chapter 11 are in **EXP**,[4] but as we've seen, for some of them there are much better algorithms that demonstrate that they are in fact in the smaller class **P**.

P	**EXP** (but not known to be in **P**)
Shortest path	Longest Path
Min cut	Max cut
2SAT	3SAT
Linear eqs	Quad. eqs
Zerosum	Nash
Determinant	Permanent
Primality	Factoring

A table of the examples from Chapter 11. All these problems are in **EXP** but the only the ones on the left column are currently known to be in **P** as well (i.e., they have a polynomial-time algorithm).

[4] Strictly speaking, many of these problems correspond to *non Boolean* functions, but we will sometimes "abuse notation" and refer to non Boolean functions as belonging to **P** or **EXP**. We can easily extend the definitions of these classes to non Boolean and partial functions. Also, for every non-Boolean function $F : \{0,1\}^* \to \{0,1\}^*$, we can define a Boolean variant $Bool(F)$ such that F can be computed in polynomial time if and only if $Bool(F)$ is. See Exercise 12.2

12.1.2 Non-boolean and partial functions (optional)

Most of the time we will restrict attention to computing functions that are *total* (i.e., defined on every input) and *Boolean* (i.e., have a single bit of output). However, Definition 12.1 naturally extends to non Boolean and to partial functions as well. We now describe this extension, although we will try to stick to Boolean total functions to the extent possible, so as to minimize the "cognitive overload" for the reader and amount of notation they need to keep in their head.

We will define $\overline{TIME}_{<<}(T(n))$ and $\overline{TIME}_{++}(T(n))$ to be the generalization of $TIME_{<<}(T(n))$ and $TIME_{++}(T(n))$ to functions that may be partial or have more than one bit of output, and define $\overline{\mathbf{P}}$ and $\overline{\mathbf{EXP}}$ similarly. Specifically the formal definition is as follows:

Definition 12.5 — Time complexity for partial and non-Boolean functions. Let $T : \mathbb{N} \to \mathbb{N}$ be a nice function, and let F be a (possibly partial) function mapping $\{0,1\}^*$ to $\{0,1\}^*$. We say that F is in $\overline{TIME}_{<<}(T(n))$ (respectively $\overline{TIME}_{++}(T(n))$) if there exists a NAND-RAM (respectively NAND-TM) program P such that for every sufficiently large n and $x \in \{0,1\}^n$ on which F is defined, on input x the program P halts after executing at most $T(n)$ steps and outputs $F(x)$.

We let $\overline{TIME}(T(n))$ (without a subscript) denote the set $\overline{TIME}_{<<}(T(n))$ and let $\overline{\mathbf{P}} = \cup_{c\in\{1,2,3,\ldots\}}\overline{TIME}(n^c)$ and $\overline{\mathbf{EXP}} = \cup_{c\in\{1,2,3,\ldots\}}\overline{TIME}(2^{n^c})$.

12.2 EFFICIENT SIMULATION OF RAM MACHINES: NAND-RAM VS NAND-TM

We have seen in Theorem 7.1 that for every NAND-RAM program P there is a NAND-TM program P' that computes the same function as P. It turns out that the P' is not much slower than P. That is, we can prove the following theorem:

Theorem 12.6 — Efficient simulation of NAND-RAM with NAND-TM. There are absolute constants a, b such that for every function F and nice function $T : \mathbb{N} \to \mathbb{N}$, if $F \in TIME_{<<}(T(n))$ then there is a NAND-TM program P' that computes F in $T'(n) = a \cdot T(n)^b$. That is, $TIME_{<<}(T(n)) \subseteq TIME_{++}(aT(n)^b)$

The constant b can be easily shown to be at most five, and with more effort can be optimized further. Theorem 12.6 means that the definition of the classes **P** and **EXP** are robust to the choice of model, and will not make a difference whether we use NAND-TM or NAND-RAM. The same proof also shows that *Turing Machines* can simulate NAND-RAM programs (and hence RAM machines). In fact, similar results are known for many models including cellular automata, C/Python/Javascript programs, parallel computers, and a great many other models, which justifies the choice of **P** as capturing a technology-independent notion of tractability. As we discussed before, this equivalence between NAND-TM and NAND-RAM (as well as other models) allows us to pick our favorite model depending on the task at hand (i.e., "have our cake and eat it too"). When we want to *design* an algorithm, we can use the extra power and convenience afforded by NAND-RAM. When we want to *analyze* a program or prove a *negative result*, we can restrict attention to NAND-TM programs.

Proof Idea:

The idea behind the proof is simple. It follows closely the proof of Theorem 7.1, where we have shown that every function F that is computable by a NAND-RAM program P is computable by a NAND-TM program P'. To prove Theorem 12.6, we follow the exact same proof but just check that the overhead of the simulation of P by P' is polynomial. The proof has many details, but is not deep. It is therefore much more important that you understand the *statement* of this theorem than its proof.

⋆

Proof of Theorem 12.6. As mentioned above, we follow the proof of Theorem 7.1 (simulation of NAND-RAM programs using NAND-TM programs) and use the exact same simulation, but with a more careful accounting of the number of steps that the simulation costs. Recall, that the simulation of NAND-RAM works by "peeling off" features of NAND-RAM one by one, until we are left with NAND-TM.

We will not provide the full details but will present the main ideas used in showing that every feature of NAND-RAM can be simulated by NAND-TM with at most a polynomial overhead:

1. Recall that every NAND-RAM variable or array element can contain an integer between 0 and T where T is the number of lines that have been executed so far. Therefore if P is a NAND-RAM program that computes F in $T(n)$ time, then on inputs of length n, all integers used by P are of magnitude at most $T(n)$. This means that the largest value `i` can ever reach is at most $T(n)$ and so each one of P's variables can be thought of as an array of at most $T(n)$ indices, each of which holds a natural number of magnitude at most $T(n)$, which can be represented using $O(\log T(n))$ bits.

2. As in the proof of Theorem 7.1, we can think of the integer array `Foo` as a two dimensional *bit* array `Foo_2D` (where `Foo_2D[`i`][`j`]` is the j-th bit of `Foo[`i`]`) and then encode the latter as a *one dimensional* bit array `Foo_1D` by mapping `Foo_2D[`i`][`j`]` to `Foo_1D[`$embed(i,j)$`]` where $embed : \mathbb{N} \times N \rightarrow \mathbb{N}$ is some one-to-one function that embeds the two dimensional space $\mathbb{N} \times \mathbb{N}$ into the one dimensional $\mathbb{N}$. Specifically, if we use $embed(i,j) = \frac{1}{2}(i+j)(i+j+1)+j$ as in Exercise 7.2, then we can see that if $i, j \leq O(T(n))$, then $embed(i,j) \leq O(T(n)^2)$. We also use the fact that $embed$ is itself computable in polynomial time in the length of its input.

3. All the arithmetic operations on integers use the grade-school algorithms, that take time that is polynomial in the number of bits of the integers, which is $poly(\log T(n))$ in our case.

4. In NAND-TM one cannot access an array at an arbitrary location but rather only at the location of the index variable `i`. Nevertheless, if `Foo` is an array that encodes some integer $k \in \mathbb{N}$ (where $k \leq T(n)$ in our case), then, as we've seen in the proof of Theorem 7.1, we can write NAND-TM code that will set the index variable `i` to k. Specifically, using enhanced NAND-TM we can write a loop that will run k times (for example, by decrementing the integer represented by `Foo` until it reaches 0), to ensure that an array `Marker` that will equal to 0 in all coordinates except the k-th one. We can then

decrement `i` until it reaches 0 and scan `Marker` until we reach the point that `Marker[i]`$= 1$.

5. To transform the above from *enhanced* to *standard* (i.e., "vanilla") NAND-TM, all that is left is to follow our proof of **??** and show we can simulate `i+= foo` and `i-= bar` in vanilla NAND-TM using our "breadcrumbs" and "wait for the bus" techniques. To simulate T steps of enhanced NAND-TM we will need at most $O(T^2)$ steps of vanilla NAND-TM (see Fig. 12.1). Indeed, suppose that the largest point that the index `i` reached in the computation so far is R, and we are in the worst case where we are trying, for example, to increment `i` while it is in a "decreasing" phase. Within at most $2R$ steps we will be back in the same position at an "increasing" phase. Using this argument we can see that in the worst case, if we need to simulate T steps of enhanced NAND-TM we will use $O(1+2+\cdots+T) = O(T^2)$ steps of vanilla NAND-TM.

Together these observations imply that the simulation of T steps of NAND-RAM can be done in $O(T^a)$ steps of vanilla NAND-TM for some constant a, i.e., time polynomial in T. (A rough accounting can show that this constant a is at most five; a more careful analysis can improve it further though this does not matter much.) ■

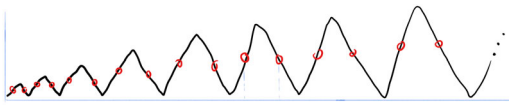


Figure 12.1: The path an index variable takes in a NAND-TM program. If we need to simulate T steps of an enhanced NAND-TM program using vanilla NAND-TM then in the worst case we will need to wait at every time for the next time `i` arrives at the same location, which will yield a cost of $O(1+2+\cdots+T) = O(T^2)$ steps.

Remark 12.7 — Turing machines and other models. If we follow the equivalence results between NAND-TM/NAND-RAM and other models, including Turing machines, RAM machines, Game of life, λ calculus, and many others, then we can see that these results also have at most a polynomial overhead in the simulation in each way. [5] It is a good exercise to go through, for example, the proof of Theorem 6.11 and verify that it establishes that Turing machines and NAND-TM programs are equivalent up to polynomial overhead.

Theorem 12.6 shows that the classes **P** and **EXP** are *robust* with respect to variation in the choice of the computational model. They are also robust with respect to our choice of the representation of the input. For example, whether we decide to represent graphs as adjacency matrices or adjacency lists will not make a difference as to whether a function on graphs is in **P** or **EXP**. The reason is that changing from one representation to another at most squares the size of the input, and a quantity is polynomial in n if and only if it is polynomial in n^2.

[5] For the λ calculus, one needs to be careful about the order of application of the reduction steps, which can matter for computational efficiency, see for example this paper.

More generally, for every function $F : \{0,1\}^* \to \{0,1\}$, the answer to the question of whether $F \in \mathbf{P}$ (or whether $F \in \mathbf{EXP}$) is unchanged by switching representations, as long as transforming one representation to the other can be done in polynomial time (which essentially holds for all reasonable representations).

12.3 EFFICIENT UNIVERSAL MACHINE: A NAND-RAM INTERPRETER IN NAND-RAM

We have seen in Theorem 8.1 the "universal program" or "interpreter" U for NAND-TM. Examining that proof, and combining it with Theorem 12.6 , we can see that the program U has a *polynomial* overhead, in the sense that it can simulate T steps of a given NAND-TM (or NAND-RAM) program P on an input x in $O(T^a)$ steps for some constant a. But in fact, by directly simulating NAND-RAM programs, we can do better with only a *constant* multiplicative overhead:

Theorem 12.8 — Efficient universality of NAND-RAM. There is a NAND-RAM program U that computes the partial function $TIMEDEVAL : \{0,1\}^* \to \{0,1\}^*$ defined as follows:

$$TIMEDEVAL(P, x, 1^T) = P(x) \tag{12.1}$$

if P is a valid representation of a NAND-RAM program which produces an output on x within at most T steps. If P does not produce an output within this time then *TIMEDEVAL* outputs an encoding of a special `fail` symbol. Moreover, for every program P, the running time of U on input $P, x, 1^T$ is $O(T)$. (The hidden constant in the O-notation can depend on the program P but is at most polynomial in the length of P's description as a string.).

R

Remark 12.9 — What does 1^T mean?. The function *TIMEDEVAL* has a curious feature - its third input has the form 1^T. We use this notation to indicate a string of T ones. (For example, if we write 1^5 we mean the string 11111 rather using this to mean the integer one to the fifth power, which is a cumbersome way to write one; it will be always clear from context whether a particular input is an integer or such a string.)

Why don't we simply assume that the function *TIMEDEVAL* gets the integer T in the binary representation? The reason has to do with how we define time as a function of the input length. If we represent T as a string using the binary basis, then its length will be $O(\log T)$, which means that we could not say that a program that takes $O(T)$ steps to compute

TIMEDEVAL would actually be considered as running in time *exponential* in its input. Thus, when we want to allow programs to run in time that is polynomial (as opposed to logarithmic) in some parameter m, we will often provide them with input of the form 1^m (i.e., a string of ones of length m).

There is nothing deep about representing inputs this way: this is merely a convention. However, it is a widely used one, especially in cryptography, and so is worth getting familiar with.

P

Before reading the proof of Theorem 12.8, try to think how you would compute *TIMEDEVAL* using your favorite programming language. That is, how you would write a program `Timed_Eval(P,x,T)` that gets a NAND-RAM program `P` (represented in some convenient form), a string `x`, and an integer `T`, and simulates `P` for `T` steps. You will likely find that your program requires $O(T)$ steps to perform this simulation. As in the case of Theorem 12.6, the proof of Theorem 12.8 is not very deep and it more important to understand its *statement*. If you understand how you would go about writing an interpreter for NAND-RAM using a modern programming language such as Python, then you know everything you need to know about this theorem.

Proof of Theorem 12.8. To present a universal NAND-RAM program in full we would need to describe a precise representation scheme, as well as the full NAND-RAM instructions for the program. While this can be done, it is more important to focus on the main ideas, and so we just sketch the proof here. A specification of NAND-RAM is given in the Appendix, and for the purposes of this simulation, we can simply use the representation of the code NAND-RAM as an ASCII string.

The program U gets as input a NAND-RAM program P, an input x, and a time bound T (given in the form 1^T) and needs to simulate the execution of P for T steps. To do so, U will do the following:

1. U will maintain variables `current_line`, and `number_steps` for the current line to be executed and the number of steps executed so far.

2. U will scan the code of P to find the number t of unique variable names that P uses. If we denote these names by $var_0, \ldots, var_{t-1}$ then U maintains an array `Var_numbers` that contains a list of pairs of the form (var_s, s) for $s \in [t]$. Using `Var_numbers` we can translate

the name of a variable to a number in $[t]$ that corresponds to its index.

3. U will maintain an array `Var_values` that will contain the current values of all P's variables. If the s-th variable of P is a scalar variable, then its value will be stored in location `Var_values`$[s]$. If it is an array variable then the value of its i-th element will be stored in location `Var_values`$[t \cdot i + s]$.

4. To simulate a single step of P, the program U will recover the line corresponding to `line_counter` and execute it. Since NAND-RAM has a constant number of arithmetic operations, we can simulate choosing which operation to execute with a sequence of a constantly many if-then-else's. When executing these operations, U will use the variable `step_counter` that keeps track of the iteration counter of P.

Simulating a single step of P will take $O(|P|)$ steps for the program U where $|P|$ is the length of the description of P as a string (which in particular is an upper bound on the number t of variable P uses). Hence the total running time will be $O(|P|T)$ which is $O(T)$ when suppressing constants that depend on the program P.

To be a little more concrete, here is some "pseudocode" description of the program U:[6]

[6] We use Python-like syntax in this pseudocode, but it is not valid Python code.

```
def U(P,x,1^T):
    t = number_variable_identifiers(P) # number of
     ↪ distinct identifiers used in P

    L = number_lines(P)

    # denote names of P's variables as var_0,...,
     ↪ var_(t-1)
    Var_numbers = array encoding list [
     ↪ (var_0,0),...,(var_(t-1),t-1)]
    # Var_numbers: encode variable identifiers as number
     ↪ 0...t-1

    Var_values = unbounded array initialized to 0
    # if s in [t] corresponds to scalar then Var_values[s]
     ↪ is value of variable corresponding to s.
    # if s corresponds to array then Var_values[t*i+s] is
     ↪ value of variable corresponding to s at position i

    def varid(name):
        # scan the array Var_numbers and
```

```
    # return the number between 0 and t-1
    ...

def get_scalar_value(name):
    return Var_values[varid(name)]

def get_array_value(name,i):
    return Var_values[t*i+varid(name)]

def set_scalar_value(name,val):
    Var_values[varid(name)] = val

def set_array_value(name,i,val):
    Var_values[t*i+varid(name)] = val

for i=0..|x|-1:
    set_array_value("X",i,x[i])
    set_array_value("X_nonblank",i,1)

current_line = 0
number_steps = 0

do {
    line = P[current_line] # extract current line of P

    # code to execute line
    # We use get/set procedures above to update vars
    ...
    # Update counters
    current_line = current_line + 1 (mod L)
    number_steps = number_steps + 1

} until get_scalar_value("loop")==0 or (number_steps >=
 ↪  T)

# Produce output:
if get_scalar_value("loop")==1: return "FAIL"
m = smallest m s.t. get_array_value("Y_nonblank",m)=0
return [get_array_value("Y",i) for i=0..m-1]
```

■

12.4 TIME HIERARCHY THEOREM

We have seen that there are uncomputable functions, but are there functions that can be computed, but only at an exorbitant cost? For example, is there a function that *can* be computed in time 2^n, but *can not* be computed in time $2^{0.9n}$? It turns out that the answer is **Yes**:

Theorem 12.10 — Time Hierarchy Theorem. For every nice function T, there is a function $F : \{0,1\}^* \to \{0,1\}$ in $TIME(T(n)\log n) \setminus TIME(T(n))$. [7]

Note that in particular this means that **P** is *strictly contained* in **EXP**.

[7] There is nothing special about $\log n$, and we could have used any other efficiently computable function that tends to infinity with n.

Proof Idea:

In the proof of Theorem 8.5 (the uncomputability of the Halting problem), we have shown that the function $HALT$ cannot be computed in any finite time. An examination of the proof shows that it gives something stronger. Namely, the proof shows that if we fix our computational budget to be T steps, then not only we can't distinguish between programs that halt and those that do not, but cannot even distinguish between programs that halt within at most T' steps and those that take more than that (where T' is some number depending on T). Therefore, the proof of Theorem 12.10 follows the ideas of the uncomputability of the halting problem, but again with a more careful accounting of the running time.

⋆

If you fully understand the proof of Theorem 8.5, then reading the following proof should not be hard. If you don't, then this is an excellent opportunity to review this reasoning.

Proof of Theorem 12.10. Recall the Halting function $HALT : \{0,1\}^* \to \{0,1\}$ that was defined as follows: $HALT(P,x)$ equals 1 for every program P and input x s.t. P halts on input x, and is equal to 0 otherwise. We cannot use the Halting function of course, as it is uncomputable and hence not in $TIME(T'(n))$ for any function T'. However, we will use the following variant of it:

We define the *Bounded Halting* function $HALT_T(P,x)$ to equal 1 for every NAND-RAM program P such that $|P| \leq \log\log|x|$, and such that P halts on the input x within $100T(|x|)$ steps. $HALT_T$ equals 0 on all other inputs.[8]

[8] The constant 100 and the function $\log\log n$ are rather arbitrary, and are chosen for convenience in this proof.

Theorem 12.10 is an immediate consequence of the following two claims:

Claim 1: $HALT_T \in TIME(T(n)\, logn)$

and

Claim 2: $HALT_T \notin TIME(T(n))$.

Please make sure you understand why indeed the theorem follows directly from the combination of these two claims. We now turn to proving them.

Proof of claim 1: We can easily check in linear time whether an input has the form P, x where $|P| \leq \log\log|x|$. Since $T(\cdot)$ is a nice function, we can evaluate it in $O(T(n))$ time. Thus, we can perform the check above, compute $T(|P|+|x|)$ and use the universal NAND-RAM program of Theorem 12.8 to evaluate $HALT_T$ in at most $poly(|P|)T(n)$ steps.[9] Since $(\log\log n)^a = o(\log n)$ for every a, this will be smaller than $T(n)\log n$ for every sufficiently large n, hence completing the proof.

[9] Recall that we use $poly(m)$ to denote a quantity that is bounded by am^b for some constants a, b and every sufficiently large m.

Proof of claim 2: This proof is the heart of Theorem 12.10, and is very reminiscent of the proof that *HALT* is not computable. Assume, toward the sake of contradiction, that there is some NAND-RAM program P^* that computes $HALT_T(P,x)$ within $T(|P|+|x|)$ steps. We are going to show a contradiction by creating a program Q and showing that under our assumptions, if Q runs for less than $T(n)$ steps when given (a padded version of) its own code as input then it actually runs for more than $T(n)$ steps and vice versa. (It is worth rereading the last sentence twice or thrice to make sure you understand this logic. It is very similar to the direct proof of the uncomputability of the halting problem where we obtained a contradiction by using an assumed "halting solver" to construct a program that, given its own code as input, halts if and only if it does not halt.)

We will define Q to be the program that on input a string z does the following:

1. If z does not have the form $z = P1^m$ where P represents a NAND-RAM program and $|P| < 0.1\log\log m$ then return 0.
2. Compute $b = P^*(P,z)$ (at a cost of at most $T(|P|+|z|)$ steps, under our assumptions).
3. If $b = 1$ then Q goes into an infinite loop, otherwise it halts.

We chose m sufficiently large so that $|Q| < 0.001\log\log m$ where $|Q|$ denotes the length of the description of Q as a string. We will reach a contradiction by splitting into cases according to whether or not $HALT_T(Q, Q1^m)$ equals 0 or 1.

On the one hand, if $HALT_T(Q,Q1^m) = 1$, then under our assumption that P^* computes $HALT_T$, Q will go into an infinite loop on input $z = Q1^m$, and hence in particular Q does *not* halt within $100T(|Q|+m)$ steps on the input z. But this contradicts our assumption that $HALT_T(Q,Q1^m) = 1$.

This means that it must hold that $HALT_T(Q,Q1^m) = 0$. But in this case, since we assume P^* computes $HALT_T$, Q does not do anything in

phase 3 of its computation, and so the only computation costs come in phases 1 and 2 of the computation. It is not hard to verify that Phase 1 can be done in linear and in fact less than $5|z|$ steps. Phase 2 involves executing P^*, which under our assumption requires $T(|Q|+m)$ steps. In total we can perform both phases in less than $10T(|Q|+m)$ in steps, which by definition means that $HALT_T(Q, Q1^m) = 1$, but this is of course a contradiction. This completes the proof of Claim 2 and hence of Theorem 12.10.

■

The time hierarchy theorem tells us that there are functions we can compute in $O(n^2)$ time but not $O(n)$, in 2^n time, but not $2^{\sqrt{n}}$, etc.. In particular there are most definitely functions that we can compute in time 2^n but not $O(n)$. We have seen that we have no shortage of natural functions for which the best *known* algorithm requires roughly 2^n time, and that many people have invested significant effort in trying to improve that. However, unlike in the finite vs. infinite case, for all of the examples above at the moment we do not know how to rule out even an $O(n)$ time algorithm. We will however see that there is a single unproven conjecture that would imply such a result for most of these problems.

R

Remark 12.11 — Time hierarchy for NAND-TM and Turing machines. The time hierarchy theorem relies on the existence of an efficient universal NAND-RAM program, as proven in Theorem 12.8. We have mentioned that other models, such as NAND-TM programs and Turing machines, are polynomially

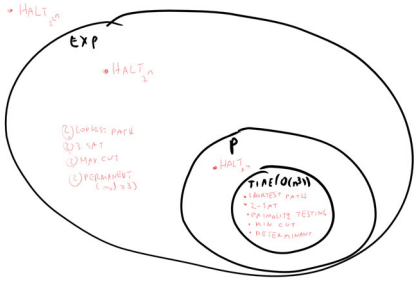


Figure 12.2: Some complexity classes and some of the functions we know (or conjecture) to be contained in them.

12.5 UNROLLING THE LOOP: UNIFORM VS NON UNIFORM COMPUTATION

We have now seen two measures of "computation cost" for functions. For a finite function $G : \{0,1\}^n \to \{0,1\}^m$, we said that $G \in SIZE(T)$ if there is a T-line NAND-CIRC program that computes G. We saw that *every* function mapping $\{0,1\}^n$ to $\{0,1\}^m$ can be computed using at most $O(m2^n)$ lines. For infinite functions $F : \{0,1\}^* \to \{0,1\}^*$, we can define the "complexity" by the smallest T such that $F \in TIME(T(n))$. Is there a relation between the two?

For simplicity, let us restrict attention to Boolean (i.e., single-bit output) functions $F : \{0,1\}^* \to \{0,1\}$. For every such function, define $F_{\restriction n} : \{0,1\}^n \to \{0,1\}$ to be the restriction of F to inputs of size n. We have seen two ways to define that F is computable within a roughly $T(n)$ amount of resources:

1. There is a *single algorithm* P that computes F within $T(n)$ steps on all inputs of length n. In such a case we say that F is *uniformly* computable (or more often, simply "computable") within $T(n)$ steps.

2. For every n, there is a $T(n)$ NAND-CIRC program Q_n that computes $F_{\restriction n}$. In such a case we say that F can be computed via a *non uniform* $T(n)$ bounded sequence of algorithms.

Unlike the first condition, where there is a single algorithm or "recipe" to compute F on all possible inputs, in the second condition we allow the restriction $F_{\restriction n}$ to be computed by a completely different program Q_n for every n. One can see that the second condition is much more relaxed, and hence we might expect that every function satisfying the first condition satisfies the second one as well (up to a small overhead in the bound $T(n)$). This indeed turns out to be the case:

Theorem 12.12 — Nonuniform computation contains uniform computation. There is some $c \in \mathbb{N}$ s.t. for every nice $T : \mathbb{N} \to \mathbb{N}$ and $F : \{0,1\}^* \to \{0,1\}$ in $TIME_{++}(T(n))$ and every sufficiently large $n \in N$, $F_{\restriction n}$ is in $SIZE(cT(n))$.

Proof Idea:

To prove Theorem 12.12 we use the technique of "unraveling the loop". That is, we can use "copy paste" to replace a program P that uses a loop that iterates for at most T times with a "loop free" program that has about T times as many lines as P.

Let us give an example using C-like syntax. Suppose we had a program of the form:

```
do {
    // some code
} while (loop==1)
```

and we had the guarantee that the program would iterate the loop for at most 4 times before it breaks.

Then we could change it to an equivalent loop-free program of the following form:

```
// some code
if (loop) {
    // some code
    }
if (loop) {
    // some code
```

```
}
if (loop) {
    // some code
}
```

That is all there is to the proof of Theorem 12.12 ⋆

Proof of Theorem 12.12. The proof follows by the argument of "unraveling the loop". If P is a NAND-TM program of L lines and $T : \mathbb{N} \to \mathbb{N}$ is a function such that for every input $x \in \{0,1\}^n$, P halts after executing at most $T(n)$ lines (and hence iterating at most $\lfloor T(n)/L \rfloor$ times) then we can obtain a NAND-CIRC program Q on n inputs as follows:

```
P{i<-0}
IF (loop) P⟨i<-1⟩
IF (loop) P⟨i<-0⟩
IF (loop) P⟨i<-1⟩
IF (loop) P⟨i<-2⟩
IF (loop) P⟨i<-1⟩
IF (loop) P⟨i<-0⟩
IF (loop) P⟨i<-1⟩
...
IF (loop) P⟨i<-R⟩
```

where for every number j, we denote by `P⟨i<-`j`⟩` the NAND-CIRC program that is obtained by replacing all references of the form `Foo[i]` (which are allowed in NAND-TM, but illegal in NAND that has no index variable `i`) with references of the form `Foo[`j`]` (which are allowed in NAND, since j is simply a number). Whenever we see a reference to the variable `X_nonblank[`i`]` in the program we will replace it with `one` or `zero` depending on whether $i < n$. Similarly, we will replace all references to `X[`i`]` for $i \geq n$ with `zero`. (We can use our standard syntactic sugar to create the constant `zero` and `one` variables.)

We simply repeat the lines of the form `IF (loop) P⟨i<-`j`⟩` for $\lfloor T(n)/L \rfloor - 1$ times, replacing each time j by $0, 1, 0, 1, 2, \ldots$ as in the definition of (standard or "vanilla") NAND-TM in **??**. We replace `IF` with the appropriate syntactic sugar, which will incur a multiplicative overhead of at most 4 in the number of lines. After this replacement, each line of the form `IF (loop) P⟨i<-`j`⟩` corresponds to at most $4L$ lines of standard sugar-free NAND. Thus the total cost is at most $4L \cdot (\frac{T(n)}{L}) \leq 4 \cdot T(n)$ lines.[10]

[10] The constant 4 can be improved, but this does not really make much difference.

■

By combining Theorem 12.12 with Theorem 12.6, we get that if $F \in \mathit{TIME}(T(n))$ then there are some constants a, b such that for every large enough n, $F_{\restriction n} \in \mathit{SIZE}(aT(n)^b)$. (In fact, by direct inspection of the proofs we can see that $a = b = 5$ would work.)

12.5.1 Algorithmic transformation of NAND-TM to NAND and "Proof by Python" (optional)

The proof of Theorem 12.12 is *algorithmic*, in the sense that the proof yields a polynomial-time algorithm that given a NAND-TM program P and parameters T and n, produces a NAND-CIRC program Q of $O(T)$ lines that agrees with P on all inputs $x \in \{0,1\}^n$ (as long as P runs for less than T steps these inputs.) Thus the same proof gives the following theorem:

Theorem 12.13 — NAND-TM to NAND compiler. There is an $O(n)$-time NAND-RAM program *COMPILE* such that on input a NAND-TM program P, and strings of the form $1^n, 1^m, 1^T$ outputs a NAND-CIRC program Q_P of at most $O(T)$ lines with n bits of inputs and m bits of output satisfying the following property.

For every $x \in \{0,1\}^n$, if P halts on input x within fewer than T steps and outputs some string $y \in \{0,1\}^m$, then $Q_P(x) = y$.

We omit the proof of the Theorem 12.13 since it follows in a fairly straightforward way from the proof of Theorem 12.12. However, for the sake of concreteness, here is a *Python* implementation of the function *COMPILE*. (The reader can feel free to skip it.)

For starters, let us consider an imperfect but very simple program that unrolls the loop. The following program will work correctly for the case that $m = 1$ and that the underlying NAND-TM program had the property that it only modifies the value of the `Y[0]` variable once. (A property that can be ensured by adding appropriate flag variables and some `IF` syntactic sugar.)

```
def COMPILE(P,T,n):
    '''
    Gets P = NAND-TM program
    T - time bound, n - number of inputs, single output
    Produces NAND-CIRC program of T lines that computes
    the restriction of P to inputs of length n and T
↪   steps.

    assumes program contains "one" and "zero" variables and
↪   that Y[0] is never modified after the correct value is
    written, so it is safe to run for an additional number
↪   of loops.
```

```
    '''
    numlines = P.count("\n")

    result = ""
    for t in range(T // numlines):
        i = index(t) # value of i in T-th iteration
        X_nonblank_i = ('one' if i < n else 'zero' )
        X_i = ('X[i]' if i< n else 'zero')
        Q =
         ↪  P.replace('Validx[i]',X_nonblank_i).replace('X[i]',X_i)
        result += Q.replace('[i]',f'[{i}]')
    return result
```

The `index` function takes a number t and returns the value of the index variable `i` in the t-th iteration. Recall that this value in NAND-TM follows the sequence $0, 1, 0, 1, 2, 1, 0, 1, 2, \ldots$ and it can be computed in Python as follows:

```
from math import sqrt
def index(k):
    return min([abs(k-int(r)*(int(r)+1)) for r in
     ↪  [sqrt(k)-0.5,sqrt(k)+0.5]])
```

Below is a more "robust" implementation of `COMPILE`, that works for an arbitrarily large number of outputs, and makes no assumptions on the structure of the input program.

```
def COMPILE(P,T,n,m):
    '''
    Gets P = NAND PP program
    T - time bound, n - number of inputs, m - number of
 ↪  outputs
    Produces NAND-CIRC program of O(T) lines that computes
    the restriction of P to inputs of length n and T steps
    '''
    lines = [l for l in P.split('\n') if l] # lines of P

    # initialization
    result = r'''
temp = NAND(X[0],X[0])
one = NAND(X[0],temp)
zero = NAND(one,one)
nothalted = NAND(X[0],temp)
halted = NAND(one,one)
'''[1:]
```

```
    # assign_to = IF(halted,assign_to,new_value)
    IFCODE = r'''
iftemp_0 = NAND(new_value,nothalted)
iftemp_1 = NAND(assign_to,halted)
assign_to = NAND(iftemp_0,iftemp_1)
'''[1:]

    UPDATEHALTED = r'''
halted = NAND(nothalted,loop)
nothalted = NAND(halted,halted)
    '''[1:]

    for t in range(T // len(lines)):
        j = index(t)
        for line in lines:
            if j>= m:
                line = line.replace('Y[i]','temp')
            if j< n:
                line = line.replace('X_nonblank[i]','one')
            else:
                line =
                 ↪  line.replace('X_nonblank[i]','zero')
                line = line.replace('X[i]','zero')

            line = line.replace('[i]',f'[{j}]')
            idx = line.find("=")
            lefthand = line[:idx].strip()
            righthand = line[idx+1:].strip()
            result += "new_value = " + righthand + "\n"
            result += IFCODE.replace("assign_to",lefthand)
        result += UPDATEHALTED

    return result
```

Since NAND-RAM programs can be simulated by NAND-TM programs with polynomial overhead, we see that we can simulate a $T(n)$ time NAND-RAM program on length n inputs with a $poly(T(n))$ size NAND-CIRC program.

P

To make sure you understand this transformation, it is an excellent exercise to verify the following equivalent characterization of the class **P** (see Exercise 12.6). Prove that for every $F : \{0,1\}^* \to \{0,1\}$, $F \in$ **P** if and only if there is a polynomial-time NAND-TM

(or NAND-RAM, it doesn't matter) program P such that for every $n \in \mathbb{N}$, $P(1^n)$ outputs a description of an n input NAND-CIRC program Q_n that computes the restriction $F_{\restriction n}$ of F to inputs in $\{0,1\}^n$. (Note that since P runs in polynomial time and hence has an output of at most polynomial length, Q_n has at most a polynomial number of lines.)

12.5.2 The class $\mathbf{P}_{/\mathbf{poly}}$

We can define the "non uniform" analog of the class $\mathbf{P}$ as follows:

Definition 12.14 — $\mathbf{P}_{/\mathbf{poly}}$. For every $F : \{0,1\}^* \to \{0,1\}$, we say that $F \in \mathbf{P}_{/\mathbf{poly}}$ if there is some polynomial $p : \mathbb{N} \to \mathbb{R}$ such that for every $n \in \mathbb{N}$, $F_{\restriction n} \in SIZE(p(n))$ where $F_{\restriction n}$ is the restriction of F to inputs in $\{0,1\}^n$.

Theorem 12.12 implies that $\mathbf{P} \subseteq \mathbf{P}_{/\mathbf{poly}}$.

P Please make sure you understand why this is the case.

Using the equivalence of NAND-CIRC programs and Boolean circuits, we can also define $\mathbf{P}_{/\mathbf{poly}}$ as the class of functions $F : \{0,1\}^* \to \{0,1\}$ such that the restriction of F to $\{0,1\}^n$ is computable by a Boolean circuit of $poly(n)$ size (say with gates in the set $\wedge, \vee, \neg$ though any universal gateset will do); see Fig. 12.3.

The notation $\mathbf{P}_{/\mathbf{poly}}$ is used for historical reasons. It was introduced by Karp and Lipton, who considered this class as corresponding to functions that can be computed by polynomial-time Turing Machines (or equivalently, NAND-TM programs) that are given for any input length n a polynomial in n long *advice string*. That this is an equivalent characterization is shown in the following theorem:

Theorem 12.15 — $\mathbf{P}_{/\mathbf{poly}}$ characterization by advice. Let $F : \{0,1\}^* \to \{0,1\}$. Then $F \in \mathbf{P}_{/\mathbf{poly}}$ if and only if there exists a polynomial $p : \mathbb{N} \to \mathbb{N}$, a polynomial-time NAND-TM program P and a sequence $\{a_n\}_{n \in \mathbb{N}}$ of strings, such that for every $n \in \mathbb{N}$:

- $|a_n| \leq p(n)$
- For every $x \in \{0,1\}^n$, $P(a_n, x) = F(x)$.

Proof. We only sketch the proof. For the "only if" direction, if $F \in \mathbf{P}_{/\mathbf{poly}}$ then we can use for a_n simply the description of the corresponding NAND-CIRC program Q_n, and for P the program that computes

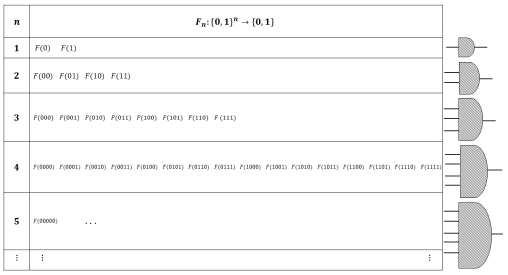


Figure 12.3: We can think of an infinite function $F : \{0,1\}^* \to \{0,1\}$ as a collection of finite functions $F_0, F_1, F_2, \ldots$ where $F_{\restriction n} : \{0,1\}^n \to \{0,1\}$ is the restriction of F to inputs of length n. We say F is in $\mathbf{P}_{/\mathbf{poly}}$ if for every n, the function $F_{\restriction n}$ is computable by a polynomial size NAND-CIRC program, or equivalently, a polynomial sized Boolean circuit. (We drop in this figure the "edge case" of F_0 though as a constant function, it can always be computed by a constant sized NAND-CIRC program.)

in polynomial time the *NANDEVAL* function that on input an n-input NAND-CIRC program Q and a string $x \in \{0,1\}^n$, outputs $Q(n)$.

For the "if" direction, we can use the same "unrolling the loop" technique of Theorem 12.12 to show that if P is a polynomial-time NAND-TM program, then for every $n \in \mathbb{N}$, the map $x \mapsto P(a_n, x)$ can be computed by a polynomial size NAND-CIRC program Q_n. ■



To make sure you understand the definition of $\mathbf{P}_{/\mathbf{poly}}$, I highly encourage you to work out fully the details of the proof of Theorem 12.15.

12.5.3 Simulating NAND with NAND-TM?

Theorem 12.12 shows that every function in $TIME(T(n))$ is in $SIZE(poly(T(n)))$. One can ask if there is an inverse relation. Suppose that F is such that $F_{\restriction n}$ has a "short" NAND-CIRC program for every n. Can we say that it must be in $TIME(T(n))$ for some "small" T? The answer is an emphatic **no**. Not only is $\mathbf{P}_{/\mathbf{poly}}$ not contained in $\mathbf{P}$, in fact $\mathbf{P}_{/\mathbf{poly}}$ contains functions that are *uncomputable*!

Theorem 12.16 — $\mathbf{P}_{/\mathbf{poly}}$ contains uncomputable functions. There exists an *uncomputable* function $F : \{0,1\}^* \to \{0,1\}$ such that $F \in \mathbf{P}_{/\mathbf{poly}}$.

Proof Idea:

Since $\mathbf{P}_{/\mathbf{poly}}$ corresponds to non uniform computation, a function F is in $\mathbf{P}_{/\mathbf{poly}}$ if for every $n \in \mathbb{N}$, the restriction $F_{\restriction n}$ to inputs of length n has a small circuit/program, even if the circuits for different values of n are completely different from one another. In particular, if F has the property that for every equal-length inputs x and x', $F(x) = F(x')$ then this means that $F_{\restriction n}$ is either the constant function zero or the constant function one for every $n \in \mathbb{N}$. Since the constant function has a (very!) small circuit, such a function F will always be in $\mathbf{P}_{/\mathbf{poly}}$ (indeed even in smaller classes). Yet by a reduction from the Halting problem, we can obtain a function with this property that is uncomputable.

⋆

Proof of **??**. Consider the following "unary halting function" $UH : \{0,1\}^* \to \{0,1\}$ defined as follows. We let $S : \mathbb{N} \to \{0,1\}^*$ be the function that on input $n \in \mathbb{N}$, outputs the string that corresponds to the binary representation of the number n without the most significant 1 digit. Note that S is *onto*. For every $x \in \{0,1\}$, we define

$UH(x) = HALTONZERO(S(|x|))$. That is, if n is the length of x, then $UH(x) = 1$ if and only if the string $S(n)$ encodes a NAND-TM program that halts on the input 0.

UH is uncomputable, since otherwise we could compute *HALTONZERO* by transforming the input program P into the integer n such that $P = S(n)$ and then running $UH(1^n)$ (i.e., *UH* on the string of n ones). On the other hand, for every n, $UH_n(x)$ is either equal to 0 for all inputs x or equal to 1 on all inputs x, and hence can be computed by a NAND-CIRC program of a *constant* number of lines. ■

The issue here is of course *uniformity*. For a function $F : \{0,1\}^* \to \{0,1\}$, if F is in $TIME(T(n))$ then we have a *single* algorithm that can compute $F_{\restriction n}$ for every n. On the other hand, $F_{\restriction n}$ might be in $SIZE(T(n))$ for every n using a completely different algorithm for every input length. For this reason we typically use $\mathbf{P}_{/\mathbf{poly}}$ not as a model of *efficient* computation but rather as a way to model *inefficient computation*. For example, in cryptography people often define an encryption scheme to be secure if breaking it for a key of length n requires more than a polynomial number of NAND lines. Since $\mathbf{P} \subseteq \mathbf{P}_{/\mathbf{poly}}$, this in particular precludes a polynomial time algorithm for doing so, but there are technical reasons why working in a non uniform model makes more sense in cryptography. It also allows to talk about security in non asymptotic terms such as a scheme having "128 bits of security".

R

Remark 12.17 — Non uniformity in practice. While it can sometimes be a real issue, in many natural settings the difference between uniform and non-uniform computation does not seem to so important. In particular, in all the examples of problems not known to be in $\mathbf{P}$ we discussed before: longest path, 3SAT, factoring, etc., these problems are also not known to be in $\mathbf{P}_{/\mathbf{poly}}$ either. Thus, for "natural" functions, if you pretend that $TIME(T(n))$ is roughly the same as $SIZE(T(n))$, you will be right more often than wrong.

12.5.4 Uniform vs. Nonuniform computation: A recap

To summarize, the two models of computation we have described so far are:

- NAND-CIRC programs, which have no loops, can only compute finite functions, and the time to execute them is exactly the number of lines they contain. These are also known as *straight-line programs* or *Boolean circuits*.

- NAND-TM programs, which include loops, and hence a single program can compute a function with unbounded input length. These are equivalent (up to polynomial factors) to *Turing Machines* or (up to polylogarithmic factors) to *RAM machines*.

For a function $F : \{0,1\}^* \rightarrow \{0,1\}$ and some nice time bound $T : \mathbb{N} \rightarrow \mathbb{N}$, we know that:

- If F is computable in time $T(n)$ then there is a sequence $\{P_n\}$ of NAND-CIRC programs with $|P_n| = poly(T(n))$ such that P_n computes $F_{\restriction n}$ (i.e., restriction of F to $\{0,1\}^n$) for every n.

- The reverse direction is not necessarily true - there are examples of functions $F : \{0,1\}^n \rightarrow \{0,1\}$ such that $F_{\restriction n}$ can be computed by even a constant size NAND-CIRC program but F is uncomputable.

This means that non uniform complexity is more useful to establish *hardness* of a function than its *easiness*.

12.6 EXTENDED CHURCH-TURING THESIS

We have mentioned the Church-Turing thesis, that posits that the definition of computable functions using NAND-TM programs captures the definition that would be obtained by all physically realizable computing devices. The *extended* Church Turing is the statement that the same holds for *efficiently computable* functions, which is typically interpreted as saying that NAND-TM programs can simulate every physically realizable computing device with polynomial overhead.

In other words, the extended Church Turing thesis says that for every *scalable computing device* C (which has a finite description but can be in principle used to run computation on arbitrarily large inputs), there are some constants a, b such that for every function $F : \{0,1\}^* \rightarrow \{0,1\}$ that C can compute on n length inputs using an $S(n)$ amount of physical resources, F is in $TIME(aS(n)^b)$.

All the current constructions of scalable computational models and programming language conform to the Extended Church-Turing Thesis, in the sense that they can be with polynomial overhead by Turing Machines (and hence also by NAND-TM or NAND-RAM programs). consequently, the classes **P** and **EXP** are robust to the choice of model, and we can use the programming language of our choice, or high level descriptions of an algorithm, to determine whether or not a problem is in **P**.

Like the Church-Turing thesis itself, the extended Church-Turing thesis is in the asymptotic setting and does not directly yield an experimentally testable prediction. However, it can be instantiated with

more concrete bounds on the overhead, which would yield predictions such as the *Physical Extended Church-Turing Thesis* we mentioned before, which would be experimentally testable.

In the last hundred+ years of studying and mechanizing computation, no one has yet constructed a scalable computing device (or even gave a convincing blueprint) that violates the extended Church Turing Thesis. However, *quantum computing*, if realized, will pose a serious challenge to this thesis.[11] However, even if the promises of quantum computing are fully realized, the extended Church-Turing thesis is "morally" correct, in the sense that, while we do need to adapt the thesis to account for the possibility of quantum computing, its broad outline remains unchanged. We are still able to model computation mathematically, we can still treat programs as strings and have a universal program, and we still have hierarchy and uncomputability results.[12] Moreover, for most (though not all!) concrete problems we care about, the prospect of quantum computing does not seem to change their time complexity. In particular, out of all the example problems mentioned in Chapter 11, as far as we know, the complexity of only one— integer factoring— is affected by modifying our model to include quantum computers as well.

[11] Large scale quantum computers have not yet been built, and even if they are constructed, we have no *proof* that they would offer super polynomial advantage over "classical" computing devices. However, there seems to be no fundamental physical obstacle to constructing them, and there are strong reasons to conjecture that they do in fact offer such an advantage.

[12] Quantum computing is *not* a challenge to the (non extended) Church Turing thesis, as a function is computable by a quantum computer if and only if it is computable by a "classical" computer or a NAND-TM program. It is only the running time of computing the function that can be affected by moving to the quantum model.



Lecture Recap

- We can define the time complexity of a function using NAND-TM programs, and similarly to the notion of computability, this appears to capture the inherent complexity of the function.
- There are many natural problems that have polynomial-time algorithms, and other natural problems that we'd love to solve, but for which the best known algorithms are exponential.
- The definition of polynomial time, and hence the class **P**, is robust to the choice of model, whether it is Turing machines, NAND-TM, NAND-RAM, modern programming languages, and many other models.
- The time hierarchy theorem shows that there are *some* problems that can be solved in exponential, but not in polynomial time. However, we do not know if that is the case for the natural examples that we described in this lecture.
- By "unrolling the loop" we can show that every function computable in time $T(n)$ can be computed by a sequence of NAND-CIRC programs (one for every input length) each of size at most $poly(T(n))$

12.7 EXERCISES

R

Remark 12.18 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 12.1 — Equivalence of different definitions of P and EXP.. Prove that the classes **P** and **EXP** defined in Definition 12.4 are equal to $\cup_{c\in\{1,2,3,\ldots\}} TIME(n^c)$ and $\cup_{c\in\{1,2,3,\ldots\}} TIME(2^{n^c})$ respectively. (If $S_1, S_2, S_3, \ldots$ is a collection of sets then the set $S = \cup_{c\in\{1,2,3,\ldots\}} S_c$ is the set of all elements e such that there exists some $c \in \{1,2,3,\ldots\}$ where $e \in S_c$.)

■

Exercise 12.2 — Boolean functions. For every function $F : \{0,1\}^* \to \{0,1\}^*$, define $Bool(F)$ to be the function mapping $\{0,1\}^*$ to $\{0,1\}$ such that on input a (string representation of a) triple (x, i, σ) with $x \in \{0,1\}^*$, $i \in \mathbb{N}$ and $\sigma \in \{0,1\}$,

$$Bool(F)(x,i,\sigma) = \begin{cases} F(x)_i & \sigma = 0, i < |F(x)| \\ 1 & \sigma = 1, i < |F(x)| \\ 0 & \text{otherwise} \end{cases} \tag{12.2}$$

where $F(x)_i$ is the i-th bit of the string $F(x)$.

Prove that $F \in \overline{\mathbf{P}}$ if and only if $Bool(F) \in \mathbf{P}$.

■

Exercise 12.3 — Composition of polynomial time. Prove that if $F, G : \{0,1\}^* \to \{0,1\}^*$ are in $\overline{\mathbf{P}}$ then their *composition* $F \circ G$, which is the function H s.t. $H(x) = F(G(x))$, is also in $\overline{\mathbf{P}}$.

■

Exercise 12.4 — Non composition of exponential time. Prove that there is some $F, G : \{0,1\}^* \to \{0,1\}^*$ s.t. $F, G \in \overline{\mathbf{EXP}}$ but $F \circ G$ is not in **EXP**.[13]

[13] TODO: check that this works, idea is that we can do bounded halting.

■

Exercise 12.5 — Oblivious program. We say that a NAND-TM program P is oblivious if there is some functions $T : \mathbb{N} \to \mathbb{N}$ and $i : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ such that for every input x of length n, it holds that:

* P halts when given input x after exactly $T(n)$ steps.

* For $t \in \{1, \ldots, T(n)\}$, after P executes the tth step of the execution the value of the index `i` is equal to $t(n, i)$. In particular this value does

not depend on x but only on its length.[14] Let $F : \{0,1\}^* \to \{0,1\}^*$ be such that there is some function $m : \mathbb{N} \to \mathbb{N}$ satisfying $|F(x)| = m(|x|)$ for every x, and let P be a NAND-TM program that computes F in $T(n)$ time for some nice T. Then there is an *oblivious* NAND-TM program P' that computes F in time $O(T^2(n) \log T(n))$.

[14] An oblivious program P cannot compute functions whose output length is not a function of the input length, though this is not a real restriction, as we can always embed variable output functions in fixed length ones using some special "end of output" marker.

■

Exercise 12.6 — Alternative characterization of P. Prove that for every $F : \{0,1\}^* \to \{0,1\}$, $F \in \mathbf{P}$ if and only if there exists a polynomial time NAND-TM program P such that $P(1^n)$ outputs a NAND-CIRC program Q_n that computes the restriction of F to $\{0,1\}^n$.

■

12.8 BIBLIOGRAPHICAL NOTES

15

[15] TODO: add reference to best algorithm for longest path - probably the Bjorklund algorithm

12.9 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

12.10 ACKNOWLEDGEMENTS

Learning Objectives:

- **Introduce the notion of *polynomial-time reductions* as a way to relate the complexity of problems to one another.**
- **See several examples of such reductions.**
- **3SAT as a basic starting point for reductions.**

13
Polynomial-time reductions

Let us consider several of the problems we have encountered before:

- Finding the longest path in a graph
- Finding the maximum cut in a graph
- The 3SAT problem: deciding whether a given 3CNF formula has a satisfying assignment.
- Solving quadratic equations over n variables $x_0, \ldots, x_{n-1} \in \mathbb{R}$.

All of these have the following properties:

- These are important problems, and people have spent significant effort on trying to find better algorithms for them.
- Each one of these problems is a *search* problem, whereby we search for a solution that is "good" in some easy to define sense (e.g., a long path, a satisfying assignment, etc..).
- Each of these problems has trivial exponential time algorithms that involve enumerating all possible solutions.
- At the moment, for all these problems the best known algorithms are not much better than the trivial one in the worst case.

In this chapter and in Chapter 14 we will see that, despite their apparent differences, we can relate these problems by their complexity. In fact, it turns out that all these problems are *computationally equivalent*, in the sense that solving one of them immediately implies solving the others. This phenomenon, known as **NP** *completeness*, is one of the surprising discoveries of theoretical computer science, and we will see that it has far-reaching ramifications.

13.0.1 Decision problems

For reasons of technical conditions rather than anything substantial, we will concern ourselves with *decision problems* (i.e., Yes/No questions) or in other words *Boolean* (i.e., one-bit output) functions. Thus, we will model all the problems as functions mapping $\{0,1\}^*$ to $\{0,1\}$:

- The *3SAT* problem can be phrased as the function $3SAT : \{0,1\}^* \to \{0,1\}$ that maps a 3CNF formula φ to 1 if there exists some assignment x that satisfies it, and to 0 otherwise.[1]
- The *quadratic equations* problem corresponds to the function $QUADEQ : \{0,1\}^* \to \{0,1\}$ that maps a set of quadratic equations E to 1 if there is an assignment x that satisfies all equations, and to 0 otherwise.
- The *longest path* problem corresponds to the function $LONGPATH : \{0,1\}^* \to \{0,1\}$ that maps a graph G and a number k to 1 if there is a simple[2] path in G of length at least k, and maps (G,k) to 0 otherwise. The longest path problem is a generalization of the well-known Hamiltonian Path Problem of determining whether a path of length n exists in a given n vertex graph.
- The *maximum cut* problem corresponds to the function $MAXCUT : \{0,1\}^* \to \{0,1\}$ that maps a graph G and a number k to 1 if there is a cut in G that cuts at least k edges, and maps (G,k) to 0 otherwise.

[1] We assume some representation of formulas as strings, and define the function to output 0 if its input is not a valid representation. We will use the same convention for all the other functions below.

[2] Recall that a *simple* path in a graph is one that does not visit any vertex more than once. For the *shortest path problem* we can assume that a path is simple without loss of generality since removing a loop (a portion of the path that starts from the same vertex and returns to it) only makes the path shorter. For the *longest path problem* we need to make this restriction to avoid "degenerate" paths such as paths that repeat endlessly the same loop.

13.1 REDUCTIONS

Suppose that $F, G : \{0,1\}^* \to \{0,1\}$ are two functions. How can we show that they are "computationally equivalent"? The idea is that we show that an efficient algorithm for F would imply an efficient algorithm for G and vice versa. The key to this is the notion of a *reduction*. Roughly speaking, we will say that F *reduces to* G (denoted as $F \leq_p G$) if F is "no harder" than G, in the sense that a polynomial-time algorithm for G implies a polynomial-time algorithm for F. The formal definition is as follows:[3]

Definition 13.1 — Reductions. Let $F, G : \{0,1\}^* \to \{0,1\}^*$. We say that F *reduces to* G, denoted by $F \leq_p G$ if there is a polynomial-time computable $R : \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*$,

$$F(x) = G(R(x)) \ . \tag{13.1}$$

We say that F and G have *equivalent complexity* if $F \leq_p G$ and $G \leq_p F$.

Solved Exercise 13.1 — Reductions and P. Prove that if $F \leq_p G$ and $G \in \mathbf{P}$ then $F \in \mathbf{P}$.

[3] Several notions of reductions are defined in the literature. The notion defined in Definition 13.1 is often known as a *mapping reduction*, *many to one reduction* or a *Karp reduction*.

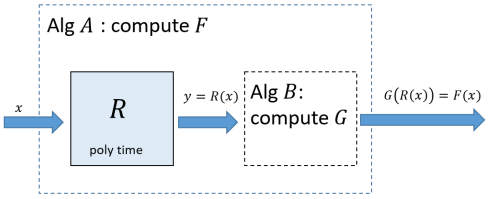


Figure 13.1: If $F \leq_p G$ then we can transform a polynomial-time algorithm B that computes G into a polynomial-time algorithm A that computes F. To compute $F(x)$ we can run the reduction R guaranteed by the fact that $F \leq_p G$ to obtain $y = F(x)$ and then run our algorithm B for G to compute $G(y)$.

P

As usual, solving this exercise on your own is an excellent way to make sure you understand Definition 13.1. This exercise justifies the informal description of $F \leq_p G$ as saying that "F is no harder than G."

Solution:

Suppose there was an algorithm B that compute F in time $p(n)$ where p is its input size. Then, (13.1) directly gives an algorithm A to compute F (see Fig. 13.1). Indeed, on input $x \in \{0,1\}^*$, Algorithm A will run the polynomial-time reduction R to obtain $y = R(x)$ and then return $B(y)$. By (13.1), $G(R(x)) = F(x)$ and hence Algorithm A will indeed compute F.

We now show that A runs in polynomial time. By assumption, R can be computed in time $q(n)$ for some polynomial q. In particular, this means that $|y| \leq q(|x|)$ (as just writing down y takes $|y|$ steps). This, computing $B(y)$ will take at most $p(|y|) \leq p(q(|x|))$ steps. Thus the total running time of A on inputs of length n is at most the time to compute y, which is bounded by $q(n)$, and the time to compute $B(y)$, which is bounded by $p(q(n))$, and since the composition of two polynomials is a polynomial, A runs in polynomial time.

Since we think of $F \leq_p G$ as saying that (as far as polynomial-time computation is concerned) F is "easier or equal in difficulty to" G, we would expect that if $F \leq_p G$ and $G \leq_p H$, then it would hold that $F \leq_p H$. Indeed this is the case:

Lemma 13.2 For every $F, G, H : \{0,1\}^* \to \{0,1\}$, if $F \leq_p G$ and $G \leq_p H$ then $F \leq_p H$.

P

We leave the proof of Lemma 13.2 as Exercise 13.2. Pausing now and doing this exercise is an excellent way to verify that you understood the definition of reductions.

R

Remark 13.3 — Polynomial reductions, completeness and soundness. We have seen reductions before in the context of proving the uncomputability of problems such as *HALTONZERO* and others. The most crucial

difference between the notion in Definition 13.1 and previously occuring notions is that in the context of relating the time complexity of problems, we need the reduction to be computable in *polynomial time*, as opposed to merely computable. Definition 13.1 also restricts reductions to have a very specific format. That is, to show that $F \leq_p G$, rather than allowing a general algorithm for F that uses a "magic box" that computes G, we only allow an algorithm that computes $F(x)$ by outputting $G(R(x))$. This restricted form is convenient for us, but people have defined and used more general reductions as well.

Since both F and G are Boolean functions, the condition $F(x) = G(R(x))$ in (13.1) is equivalent to the following two implications: **(i)** if $F(x) = 1$ then $G(R(x)) = 1$, and **(ii)** if $G(R(x)) = 1$ then $F(x) = 1$. Traditionally, condition **(i)** is known as *completness* and condition **(ii)** is known as *soundness*. We can think of this as saying that the reduction R is *complete* if every 1-input of F (i.e. x such that $F(x) = 1$) is mapped by R to a 1-input of G, and that it is *sound* if no 0-input of F will ever be mapped to a 1-input of G. As we will see below, it is often the case that establishing **(ii)** is the more challenging part.

13.2 SOME EXAMPLE REDUCTIONS

We will now use reductions to relate the computational complexity of the problems mentioned above — 3SAT, Quadratic Equations, Maximum Cut, and Longest Path. We start by reducing 3SAT to the latter three problems, demonstrating that solving any one of them will solve 3SAT. Along the way we will introduce one more problem: the *independent set* problem. Like the others, it shares the characteristics that it is an important and well-motivated computational problem, and that the best known algorithm for it takes exponential time. In Chapter 14 we will show the other direction: reducing each one of these problems to 3SAT in one fell swoop.

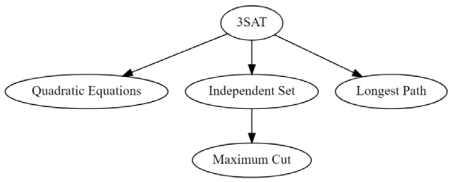


Figure 13.2: Our first stage in showing equivalence is to reduce 3SAT to the three other problems

13.2.1 Reducing 3SAT to quadratic equations

Let us now see our first example of a reduction. Recall that in the *quadratic equation* problem, the input is a list of n-variate polynomials $p_0, \ldots, p_{m-1} : \mathbb{R}^n \to \mathbb{R}$ that are all of degree at most two (i.e., they are *quadratic*) and with integer coefficients.[4] The task is to find out whether there is a solution $x \in \mathbb{R}^n$ to the equations $p_0(x) = 0$, $p_1(x) = 0$, ..., $p_{m-1}(x) = 0$.

[4] The latter condition is for convenience and can be achieved by scaling.

For example, the following is a set of quadratic equations over the variables x_0, x_1, x_2:

$$\begin{aligned} x_0^2 - x_0 &= 0 \\ x_1^2 - x_1 &= 0 \\ x_2^2 - x_2 &= 0 \\ 1 - x_0 - x_1 + x_0 x_1 &= 0 \end{aligned} \tag{13.2}$$

You can verify that $x \in \mathbb{R}^3$ satisfies this set of equations if and only if $x \in \{0,1\}^3$ and $x_0 \vee x_1 = 1$.

We will show how to reduce 3SAT to the problem of Quadratic Equations.

Theorem 13.4 — Hardness of quadratic equations.

$$3SAT \leq_p QUADEQ \tag{13.3}$$

where $3SAT$ is the function that maps a 3SAT formula φ to 1 if it is satisfiable and to 0 otherwise, and $QUADEQ$ is the function that maps a set E of quadratic equations over $\{0,1\}^n$ to 1 it has a solution and to 0 otherwise.

Proof Idea:

At the end of the day, a 3SAT formula can be thought of as a list of equations on some variables $x_0, \ldots, x_{n-1}$. Namely, the equations are that each of the x_i's should be equal to either 0 or 1, and that the variables should satisfy some set of constraints which corresponds to the OR of three variables or their negation. To show that $3SAT \leq_p QUADEQ$ we need to give a polynomial-time reduction that maps a 3SAT formula φ into a set of quadratic equations E such that E has a solution if and only if φ is satisfiable. The idea is that we can transform a 3SAT formula φ first to a set of *cubic* equations by mapping every constraint of the form $(x_{12} \vee \overline{x}_{15} \vee x_{24})$ into an equation of the form $(1-x_{12})x_{15}(1-x_{24}) = 0$. We can then turn this into a *quadratic equation* by mapping any cubic equation of the form $x_i x_j x_k = 0$ into the two quadratic equations $y_{i,j} = x_i x_j$ and $y_{i,j} x_k = 0$.

⋆

Proof of Theorem 13.4. To prove Theorem 13.4 we need to give a polynomial-time transformation of every 3SAT formula φ into a set of quadratic equations E, and prove that $3SAT(\varphi) = QUADEQ(E)$.

We now describe the transformation of a formula φ to equations E and show the completeness and soundness conditions. Recall that a *3SAT formula* φ is a formula such as $(x_{17} \vee \overline{x}_{101} \vee x_{57}) \wedge (x_{18} \vee \overline{x}_{19} \vee \overline{x}_{101}) \wedge \cdots$. That is, φ is composed of the AND of m *3SAT clauses*

where a 3SAT clause is the OR of three variables or their negation. A *quadratic equations* instance E is composed of a list of equations, each of involving a sum of variables or their products, such as $x_{19}x_{52} - x_{12} + 2x_{33} = 2$, etc.. We will include the constraints $x_i^2 - x_i = 0$ for every $i \in [n]$ in our equations, which means that we can restrict attention to assignments where $x_i \in \{0,1\}$ for every i.

There is a natural way to map a 3SAT instance into a set of *cubic* equations E', and that is to map a clause such as $(x_{17} \vee \overline{x}_{101} \vee x_{57})$ (which is equivalent to the negation of $\overline{x}_{17} \wedge x_{101} \wedge \overline{x}_{57}$) to the equation $(1 - x_{17})x_{101}(1 - x_{57}) = 0$. Therefore, we can map a formula φ with n variables m clauses into a set E' of $m + n$ cubic equations on n variables (that is, one equation per each clause, plus one equation of the form $x_i^2 - x_i = 0$ for each variable to ensure that its value is in $\{0,1\}$) such that every assignment $a \in \{0,1\}^n$ to the n variables satisfies the original formula if and only if it satisfies the equations of E'.

To make the equations *quadratic* we introduce for every two distinct $i, j \in [n]$ a variable $y_{i,j}$ and include the constraint $y_{i,j} - x_i x_j = 0$ in the equations. This is a quadratic equation that ensures that $y_{i,j} = x_i x_j$ for every such $i, j \in [n]$. Now we can turn any cubic equation in the x's into a quadratic equation in the x and y variables. For example, we can "open up the parentheses" of an equation such as $(1 - x_{17})x_{101}(1 - x_{57}) = 0$ to $x_{101} - x_{17}x_{101} - x_{101}x_{57} + x_{17}x_{101}x_{57} = 0$. We can now replace the cubic term $x_{17}x_{101}x_{57}$ with the quadratic term $y_{17,101}x_{57}$. This can be done for every cubic equation in the same way, replacing any cubic term $x_i x_j x_k$ with the term $y_{i,j}x_k$. The end result will be a set of $m + n + \binom{n}{2}$ equations (one equation per clause, one equation per variable to ensure $x_i^2 - x_i = 0$, and one equation per pair i, j to ensure $y_{i,j} = x_i x_j = 0$) on the $n + \binom{n}{2}$ variables $x_0, \ldots, x_{n-1}$ and $y_{i,j}$ for all pairs of distinct variables i, j.[5]

To complete the proof we need to show that if we transform φ to E in this way then the 3SAT formula φ is satisfiable if and only if the equations E have a solution. This is essentially immediate from the construction, but as this is our first reduction, we spell this out fully:

- **Completeness:** We claim that if φ is satisfiable then the equations E have a solution. To prove this we need to show how to transform a satisfying assignment $a \in \{0,1\}^n$ to the variables of φ (that is, a_i is the value assigned to x_i) to a solution to the variables of E. Specifically, if $a \in \{0,1\}^n$ is such an assignment then by design a satisfies all the *cubic* equations E' that we constructed above. But then, if we assign to the $n + \binom{n}{2}$ variables the values $a_0, \ldots, a_{n-1}$ and $\{a_i a_j\}$ for all $\{i, j\} \subseteq [n]$ then by construction this will satisfy all the quadratic equations of E as well.

[5] $\binom{n}{2} = \frac{1}{2}n(n-1)$ is the number of all size two subsets of $[n]$. We consider $\{i, j\}$ to be the same as $\{j, i\}$ since $x_i x_j = x_j x_i$ for every i, j.

- **Soundness:** We claim that if the equations E have a solution then φ is satisfiable. Indeed, suppose that $z \in \mathbb{R}^{n+\binom{n}{2}}$ is a solution to the equations E. A priori z could be any vector of $n + \binom{n}{2}$ numbers, but the fact that E contains the equations $x_i^2 - x_i = 0$ and $y_{i,j} - x_i x_j = 0$ means that if z satisfies these equations then the values it assigns to x_i must be in $\{0,1\}$ for every i, and the value it assigns to $y_{i,j}$ must be $x_i x_j$ for every $\{i,j\} \subseteq [n]$. Therefore by the way we constructed our equations, the value assigned x must be a solution of the original cubic equations E' and hence also of the original formula φ, which in particular implies φ is satisfiable.

This reduction can be easily implemented in about a dozen lines of Python or any other programming language, see Fig. 13.3. ■



Figure 13.3: Reducing 3SAT to satisfiability of quadratic equations. On the righthand side is Python code implementing the reduction of Theorem 13.4 and on the lefthand side is the output of this reduction on an example 3SAT instance.

13.3 THE INDEPENDENT SET PROBLEM

For a graph $G = (V, E)$, an independent set (also known as a *stable set*) is a subset $S \subseteq V$ such that there are no edges with both endpoints in S (in other words, $E(S, S) = \emptyset$). Every "singleton" (set consisting of a single vertex) is trivially an independent set, but finding larger independent sets can be challenging. The *maximum independent set* problem (henceforth simply "independent set") is the task of finding the largest independent set in the graph.[6] The independent set problem is naturally related to *scheduling problems*: if we put an edge between two conflicting tasks, then an independent set corresponds to a set of tasks that can all be scheduled together without conflicts. But it also arises in very different settings, including trying to find structure in protein-protein interaction graphs.[7]

To phrase independent set as a decision problem, we think of it as a function $ISET : \{0,1\}^* \to \{0,1\}$ that on input a graph G and a number k outputs 1 if and only if the graph G contains an independent set of size at least k. We will now reduce 3SAT to Independent set.

Theorem 13.5 — Hardness of Independent Set. $3SAT \leq_p ISET$.

Proof Idea:

The idea is that finding a satisfying assignment to a 3SAT formula corresponds to satisfying many local constraints without creating any conflicts. One can think of "$x_{17} = 0$" and "$x_{17} = 1$" as two conflicting events, and of the constraints $x_{17} \vee \overline{x}_5 \vee x_9$ as creating a conflict between the events "$x_{17} = 0$", "$x_5 = 1$" and "$x_9 = 0$", saying that these three cannot simultaneosly co-occur. Using these ideas, we can we can think of solving a 3SAT problem as trying to schedule non conflicting events, though the devil is, as usual, in the details.

[6] While we will not consider it here, people have also looked at the *maximal* (as opposed to *maximum*) independent set, which is the task of finding a "local maximum" of an independent set: an independent set S such that one cannot add a vertex to it without losing the independence property (such a set is known as a *vertex cover*). Finding a maximal independent set can be done efficiently by a greedy algorithm, but this local maximum can be much smaller than the global maximum.

[7] In the molecular biology literature, people often refer to the computationally equivalent clique problem.

⋆

Proof of Theorem 13.5. Given a 3SAT formula φ on n variables and with m clauses, we will create a graph G with $3m$ vertices as follows: (see Fig. 13.4 for an example)

- A clause C in φ has the form $C = y \vee y' \vee y''$ where y, y', y'' are *literals* (variables or their negation). For each such clause C, we will add three vertices to G, and label them (C, y), (C, y'), and (C, y'') respectively. We will also add the three edges between all pairs of these vertices, so they form a *triangle*. Since there are m clauses in φ, the graph G will have $3m$ vertices.

- In addition to the above edges, we also add an edge between every pair vertices of the form (C, y) and (C', y') where y and y' are *conflicting* literals. That is, we add an edge between (C, y) and (C, y') if there is an i such that $y = x_i$ and $y' = \overline{x}_i$ or vice versa.

The above construction of G based on φ can clearly be carried out in polynomial time. Hence to prove the theorem we need to show that φ is satisfiable if and only if G contains an independent set of m vertices. We now show both directions of this equivalence:

Part 1: Completeness. The "completeness" direction is to show that if φ has a satisfying assignment x^*, then G has an independent set S^* of m vertices. Let us now show this.

Indeed, suppose that φ has a satisfying assignment $x^* \in \{0,1\}^n$. Then for every clause $C = y \vee y' \vee y''$ of φ, one of the literals y, y', y'' must evaluate to *true* under the assignment x^* (as otherwise it would not satisfy φ). We let S be a set of m vertices that is obtained by choosing for every clause C one vertex of the form (C, y) such that y evaluates to true under x^*. (If there is more than one such vertex for the same C, we arbitrarily choose one of them.)

We claim that S is an independent set. Indeed, suppose otherwise that there was a pair of vertices (C, y) and (C', y') in S that have an edge between them. Since we picked one vertex out of each triangle corresponding to a clause, it must be that $C \neq C'$. Hence the only way that there is an edge between (C, y) and (C, y') is if y and y' are conflicting literals (i.e. $y = x_i$ and $y' = \overline{x}_i$ for some i). But that would that they can't both evaluate to *true* under the assignment x^*, which contradicts the way we constructed the set S. This completes the proof of the completeness condition.

Part 2: Soundness. The "soundness" direction is to show that if G has an independent set S^* of m vertices, then φ has a satisfying assignment $x^* \in \{0,1\}^n$. Let us now show this.

Indeed, suppose that G has an independent set $S*$ with m vertices. We will define an assignment $x^* \in \{0,1\}^n$ for the variables of φ as follows. For every $i \in [n]$, we set x^*_i according to the following rules:

- If S^* contains a vertex of the form (C, x_i) then we set $x^*_i = 1$.
- If S^* contains a vertex of the form $(C, \overline{x_i})$ then we set $x^*_i = 0$.
- If S^* does not contain a vertex of either of these forms, then it does not matter which value we give to x^*_i, but for concreteness we'll set $x^*_i = 0$.

The first observation is that x^* is indeed well defined, in the sense that the rules above do not conflict with one another, and ask to set x^*_i to be both 0 and 1. This follows from the fact that S^* is an *independent set* and hence if it contains a vertex of the form (C, x_i) then it cannot contain a vertex of the form $(C', \overline{x_i})$.

We now claim that x^* is a satisfying assignment for φ. Indeed, since S^* is an independent set, it cannot have more than one vertex inside each one of the m triangles $(C, y), (C, y'), (C, y'')$ corresponding to a clause of φ. Hence since $|S^*| = m$, it must have exactly one vertex in each such triangle. For every clause C of φ, if (C, y) is the vertex in S^* in the triangle corresponding to C, then by the way we defined x^*, the literal y must evaluate to *true*, which means that x^* satisfies this clause. Therefore x^* satisfies all clauses of φ, which is the definition of a satisfying assignment.

This completes the proof of Theorem 13.5 ■

13.4 REDUCING INDEPENDENT SET TO MAXIMUM CUT

Theorem 13.6 — Hardness of Max Cut. $ISET \leq_p MAXCUT$

Proof Idea:

We will map a graph G into a graph H such that a large independent set in G becomes a partition cutting many edges in H. We can think of a cut in H as coloring each vertex either "blue" or "red". We will add a special "source" vertex s^*, connect it to all other vertices, and assume without loss of generality that it is colored blue. Hence the more vertices we color red, the more edges from s^* we cut. Now, for every edge u, v in the original graph G we will add a special "gadget" which will be a small subgraph that involves u,v, the source s^*, and two other additional vertices. We design the gadget in a way so that if the red vertices are not an independent set in G then the corresponding cut in H will be "penalized" in the sense that it would not



Figure 13.4: The reduction of 3SAT to Independent Set. On the righthand side is *Python* code that implements this reduction. On the lefthand side is a sample output of the reduction. We use black for the "triangle edges" and red for the "conflict edges". Note that the satisfying assignment $x^* = 0110$ corresponds to the independent set $(0, \neg x_3), (1, \neg x_0), (2, x_2)$.

cut as many edges. Once we set for ourselves this objective, it is not hard to find a gadget that achieves it— see the proof below.

⋆

Proof of Theorem 13.6. We will transform a graph G of n vertices and m edges into a graph H of $n + 1 + 2m$ vertices and $n + 5m$ edges in the following way: the graph H will contain all vertices of G (though not the edges between them!) and in addition to that will contain:

* A special vertex s^* that is connected to all the vertices of G

* For every edge $e = \{u, v\} \in E(G)$, two vertices e_0, e_1 such that e_0 is connected to u and e_1 is connected to v, and moreover we add the edges $\{e_0, e_1\}, \{e_0, s^*\}, \{e_1, s^*\}$ to H.

Theorem 13.6 will follow by showing that G contains an independent set of size at least k if and only if H has a cut cutting at least $k + 4m$ edges. We now prove both directions of this equivalence:

Part 1: Completeness. If I is an independent k-sized set in G, then we can define S to be a cut in H of the following form: we let S contain all the vertices of I and for every edge $e = \{u, v\} \in E(G)$, if $u \in I$ and $v \notin I$ then we add e_1 to S; if $u \notin I$ and $v \in I$ then we add e_0 to S; and if $u \notin I$ and $v \notin I$ then we add both e_0 and e_1 to S. (We don't need to worry about the case that both u and v are in I since it is an independent set.) We can verify that in all cases the number of edges from S to its complement in the gadget corresponding to e will be four (see Fig. 13.5). Since s^* is not in S, we also have k edges from s^* to I, for a total of $k + 4m$ edges.

Part 2: Soundness. Suppose that S is a cut in H that cuts at least $C = k + 4m$ edges. We can assume that s^* is not in S (otherwise we can "flip" S to its complement $\overline{S}$, since this does not change the size of the cut). Now let I be the set of vertices in S that correspond to the original vertices of G. If I was an independent set of size k then would be done. This might not always be the case but we will see that if I is not an independent set then its also larger than k. Specifically, we define $m_{in} = |E(I, I)|$ be the set of edges in G that are contained in I and let $m_{out} = m - m_{in}$ (i.e., if I is an independent set then $m_{in} = 0$ and $m_{out} = m$). By the properties of our gadget we know that for every edge $\{u, v\}$ of G, we can cut at most three edges when both u and v are in S, and at most four edges otherwise. Hence the number C of edges cut by S satisfies $C \leq |I| + 3m_{in} + 4m_{out} = |I| + 3m_{in} + 4(m - m_{in}) = |I| + 4m - m_{in}$. Since $C = k + 4m$ we get that $|I| - m_{in} \geq k$. Now we can transform I into an independent set I' by going over every one of the m_{in} edges that are inside I and removing one of the endpoints of the edge from it. The resulting set I' is an independent set in the graph G of size $|I| - m_{in} \geq k$ and so this concludes the proof of the soundness condition.

■

13.5 REDUCING 3SAT TO LONGEST PATH

[8]

One of the most basic algorithms in Computer Science is Dijkstra's algorithm to find the *shortest path* between two vertices. We now show that in contrast, an efficient algorithm for the *longest path* problem would imply a polynomial-time algorithm for 3SAT.

Theorem 13.7 — Hardness of longest path.

$$3SAT \leq_p LONGPATH \tag{13.4}$$

Proof Idea:

To prove Theorem 13.7 need to show how to transform a 3CNF formula φ into a graph G and two vertices s, t such that G has a path of length at least k if and only if φ is satisfiable. The idea of the reduction is sketched in Fig. 13.7 and Fig. 13.8. We will construct a graph that contains a potentially long "snaking path" that corresponds to all variables in the formula. We will add a "gadget" corresponding to each clause of φ in a way that we would only be able to use the gadgets if we have a satisfying assignment.

⋆

Proof of Theorem 13.7. We build a graph G that "snakes" from s to t as follows. After s we add a sequence of n long loops. Each loop has an "upper path" and a "lower path". A simple path cannot take both the upper path and the lower path, and so it will need to take exactly one of them to reach s from t.

Our intention is that a path in the graph will correspond to an assignment $x \in \{0,1\}^n$ in the sense that taking the upper path in the ith loop corresponds to assigning $x_i = 1$ and taking the lower path corresponds to assigning $x_i = 0$. When we are done snaking through all the n loops corresponding to the variables to reach t we need to pass through m "obstacles": for each clause j we will have a small gadget consisting of a pair of vertices s_j, t_j that have three paths between them. For example, if the jth clause had the form $x_{17} \vee \overline{x}_{55} \vee x_{72}$ then one path would go through a vertex in the lower loop corresponding to x_{17}, one path would go through a vertex in the upper loop corresponding to x_{55} and the third would go through the lower loop corresponding to x_{72}. We see that if we went in the first stage according to a satisfying assignment then we will be able to find a free vertex to travel from s_j to t_j. We link t_1 to s_2, t_2 to s_3, etc and link t_m to t. Thus

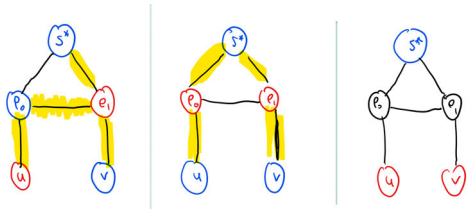


Figure 13.5: In the reduction of independent set to max cut, we have a "gadget" corresponding to every edge $e = \{u, v\}$ in the original graph. If we think of the side of the cut containing the special source vertex s^* as "blue" and the other side as "red", then the leftmost and center figures show that if u and v are not both red then we can cut four edges from the gadget. In contrast, by enumerating all possibilities one can verify that if both u and v are red, then no matter how we color the intermediate vertices e_0, e_1, we will cut at most three edges from the gadget.

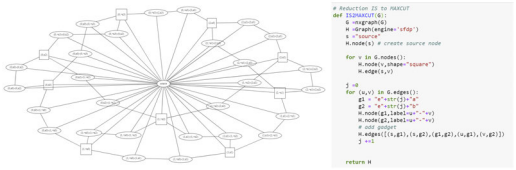


Figure 13.6: The reduction of independent set to max cut. On the righthand side is Python code implementing the reduction. On the lefthand side is an example output of the reduction where we apply it to the independent set instance that is obtained by running the reduction of Theorem 13.5 on the 3CNF formula $(x_0 \vee \overline{x}_3 \vee x_2) \wedge (\overline{x}_0 \vee x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee x_2 \vee x_3)$.

[8] This section is still a little messy, feel free to skip it or just read it without going into the proof details

a satisfying assignment would correspond to a path from s to t that goes through one path in each loop corresponding to the variables, and one path in each loop corresponding to the clauses. We can make the loop corresponding to the variables long enough so that we must take the entire path in each loop in order to have a fighting chance of getting a path as long as the one corresponds to a satisfying assignment. But if we do that, then the only way if we are able to reach t is if the paths we took corresponded to a satisfying assignment, since otherwise we will have one clause j where we cannot reach t_j from s_j without using a vertex we already used before.

■

Lecture Recap

- The computational complexity of many seemingly unrelated computational problems can be related to one another through the use of *reductions*.
- If $F \leq_p G$ then a polynomial-time algorithm for G can be transformed into a polynomial-time algorithm for F.
- Equivalently, if $F \leq_p G$ and F does *not* have a polynomial-time algorithm then neither does G.
- We've developed many techniques to show that $3SAT \leq_p F$ for interesting functions F. Sometimes we can do so by using *transitivity* of reductions: if $3SAT \leq_p G$ and $G \leq_p F$ then $3SAT \leq_p F$.

13.6 EXERCISES

Remark 13.8 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

[9]

Exercise 13.1 Prove **??**

■

Exercise 13.2 — Transitivity of reductions. Prove that if $F \leq_p G$ and $G \leq_p H$ then $F \leq_p H$.

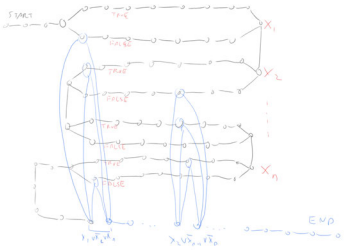


Figure 13.7: We can transform a 3SAT formula φ into a graph G such that the longest path in the graph G would correspond to a satisfying assignment in φ. In this graph, the black colored part corresponds to the variables of φ and the blue colored part corresponds to the vertices. A sufficiently long path would have to first "snake" through the black part, for each variable choosing either the "upper path" (corresponding to assigning it the value `True`) or the "lower path" (corresponding to assigning it the value `False`). Then to achieve maximum length the path would traverse through the blue part, where to go between two vertices corresponding to a clause such as $x_{17} \vee \overline{x}_{32} \vee x_{57}$, the corresponding vertices would have to have been not traversed before.

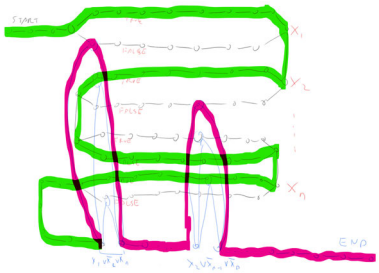


Figure 13.8: The graph above with the longest path marked on it, the part of the path corresponding to variables is in green and part corresponding to the clauses is in pink.

[9] TODO: Maybe mention either in exercise or in body of the lecture some NP hard results motivated by science. For example, shortest superstring that is motivated by genome sequencing, protein folding, maybe others.

13.7 BIBLIOGRAPHICAL NOTES

Reduction of independent set to max cut taken from these notes.

13.8 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

13.9 ACKNOWLEDGEMENTS

Learning Objectives:

- Introduce the class NP capturing a great many important computational problems
- NP-completeness: evidence that a problem might be intractable.
- The P vs NP problem.

14
NP, NP completeness, and the Cook-Levin Theorem

> *"In this paper we give theorems that suggest, but do not imply, that these problems, as well as many others, will remain intractable perpetually"*, Richard Karp, 1972

> *"Sad to say, but it will be many more years, if ever before we really understand the Mystical Power of Twoness... 2-SAT is easy, 3-SAT is hard, 2-dimensional matching is easy, 3-dimensional matching is hard. Why? oh, Why?"* Eugene Lawler

14.1 THE CLASS NP

So far we have shown that 3SAT is no harder than Quadratic Equations, Independent Set, Maximum Cut, and Longest Path. But to show that these problems are *computationally equivalent* we need to give reductions in the other direction, reducing each one of these problems to 3SAT as well. It turns out we can reduce all three problems to 3SAT in one fell swoop.

In fact, this result extends far beyond these particular problems. All of the problems we discussed in Chapter 13, and a great many other problems, share the same commonality: they are all *search* problems, where the goal is to decide, given an instance x, whether there exists a *solution* y that satisfies some condition that can be verified in polynomial time. For example, in 3SAT, the instance is a formula and the solution is an assignment to the variable; in Max-Cut the instance is a graph and the solution is a cut in the graph; and so on and so forth. It turns out that *every* such search problem can be reduced to 3SAT.

To make this precise, we make the following mathematical definition: we define the class **NP** to contain all Boolean functions that correspond to a *search problem* of the form above— that is, functions that output 1 on x if and only if there exists a solution w such that

the pair (x, w) satisfies some polynomial-time checkable condition. Formally, **NP** is defined as follows:

Definition 14.1 — NP. We say that $F : \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some constants $a, b \in \mathbb{N}$ and $V : \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and for every $x \in \{0,1\}^n$,

$$F(x) = 1 \Leftrightarrow \exists_{w \in \{0,1\}^{an^b}} \text{ s.t. } V(xw) = 1 \ . \tag{14.1}$$

In other words, for F to be in **NP**, there needs to exist some polynomial-time computable verification function V, such that if $F(x) = 1$ then there must exist w (of length polynomial in $|x|$) such that $V(xw) = 1$, and if $F(x) = 0$ then for *every* such w, $V(xw) = 0$. Since the existence of this string w certifies that $F(x) = 1$, w is often referred to as a *certificate*, *witness*, or *proof* that $F(x) = 1$.

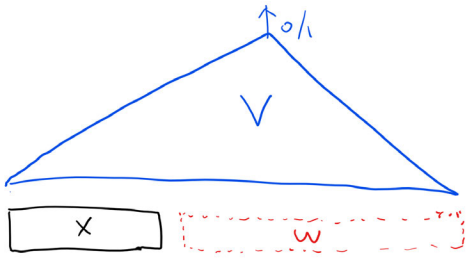


Figure 14.1: The class **NP** corresponds to problems where solutions can be *efficiently verified*. That is, this is the class of functions F such that $F(x) = 1$ if there is a "solution" w of length polynomial in $|x|$ that can be verified by a polynomial-time algorithm V.

See also Fig. 14.1 for an illustration of Definition 14.1. The name **NP** stands for "nondeterministic polynomial time" and is used for historical reasons; see the bibiographical notes. The string w in (**??**) is sometimes known as a *solution*, *certificate*, or *witness* for the instance x.

R

Remark 14.2 — NP and proof systems. The definition of **NP** means that for every $F \in \mathbf{NP}$ and string $x \in \{0,1\}^*$, $F(x) = 1$ if and only if there is a *short and efficiently verifiable proof* of this fact. That is, we can think of the function V in Definition 14.1 as a *verifier* algorithm, similar to what we've seen in Section 10.1. The verifier checks whether a given string $w \in \{0,1\}^*$ is a valid proof for the statement "$F(x) = 1$". Essentially all proof systems considered in mathematics involve line-by-line checks that can be carried out in polynomial time. Thus the heart of **NP** is asking for statements that have *short* (i.e., polynomial in the size of the statements) proof. Indeed, as we will see in Chapter 15, Kurt Gödel phrased the question of whether **NP** $=$ **P** as asking whether "the mental work of a mathematician [in proving theorems] could be completely replaced by a machine".

P

Definition 14.1 is *assymetric* in the sense that there is a difference between an output of 1 and an output of 0. You should make sure you understand why this definition does *not* guarantee that if $F \in \mathbf{NP}$ then the function $1 - F$ (i.e., the map $x \mapsto 1 - F(x)$) is in **NP** as well. In fact, it is believed that there do exist functions F satisfying $F \in \mathbf{NP}$ but $1 - F \notin \mathbf{NP}$. [1] This is in

contrast to the class **P** which (as you should verify) *does* satisfy that if $F \in \mathbf{P}$ then $1 - F$ is in **P** as well.

[1] For example, as shown below, $3SAT \in \mathbf{NP}$, but the function $\overline{3SAT}$ that on input a 3CNF formula φ outputs 1 if and only if φ is *not* satisfiable is not known (nor believed) to be in **NP**.

14.1.1 Examples of NP functions

■ **Example 14.3 —** $3SAT \in \mathbf{NP}$. $3SAT$ is in **NP** since for every ℓ-variable formula φ, $3SAT(\varphi) = 1$ if and only if there exists a satisfying assignment $x \in \{0,1\}^\ell$ such that $\varphi(x) = 1$, and we can check this condition in polynomial time.

The above reasoning explains why $3SAT$ is in **NP**, but since this is our first example, we will now belabor the point and expand out in full formality what is the precise representation of the witness w and the algorithm V that demonstrate that $3SAT$ is in **NP**.

Specifically, we can represent a 3CNF formula φ with k variables and m clauses as a string of length $n = O(m \log k)$, since every one of the m clauses involves three variables and their negation, and the identity of each variable can be represented using $\lceil \log_2 k \rceil$. We assume that every variable participates in some clause (as otherwise it can be ignored) and hence that $m \geq k$, which in particular means that n is larger than both m and k.

We can represent an assignment to the k variables using a k-length string, which, since $n > k$, can be "padded" to a string $w \in \{0,1\}^n$ in some standard way. (For example, if $y \in \{0,1\}^k$ is the assignment, we can let $w = y10^{n-k-1}$; given the string w we can "read off" y, by chopping off all the zeroes at the end of w until we encounter the first 1, which we remove as well.)

Now checking whether a given assignment $y \in \{0,1\}^k$ satisfies a given k-variable 3CNF φ can be done in polynomial time through the following algorithm V:

Algorithm V:

Input:

1. 3CNF formula φ with k variables and m clauses (encoded as a string of length $n = O(m \log k)$)
2. Assignment $y \in \{0,1\}^k$ to the variables of φ (encoded using padding as a string $w \in \{0,1\}^n$)

Output: 1 if and only if y satisfies φ.

Operation:

1. For every clause $C = (\ell_1 \vee \ell_2 \vee \ell_3)$ of φ (where ℓ_1, ℓ_2, ℓ_3 are literals), if all three literals evaluate to *false* under the assignment y then halt and output 0.
2. Output 1.

Algorithm V runs in time polynomial in the length n of φ's description as a string. Indeed there are m clauses, and checking the evaluation of a literal of the form y_i or $\neg y_j$ can be done by scanning the k-length string y, and hence the running time of Algorithm V is at most $O(mk) = O(n^2)$, as both k and m are smaller than n.

By its definition the algorithm outputs 1 if and only if the assignment y satisfies all the clauses of the 3CNF formula φ, which means that $3SAT(\varphi) = 1$ if and only if there exists some $w \in \{0,1\}^n$ such that $V(\varphi w) = 1$ which is precisely the condition needed to show that $3SAT \in$ **NP** per Definition 14.1.



Remark 14.4 — Padding a witness. The "padding trick" we used in Example 14.3 can always be used to expand a witness of length smaller than an^b to a witness of exactly that length. Therefore one can think of the condition (**??**) in Definition 14.1as simply stipulating that the "solution" w to the problem x is of length *at most* polynomial in $|x|$.

Here are some more examples for problems in **NP**. For each one of these problems we merely sketch how the witness is represented and why it is efficiently checkable, but working out the details can be a good way to get more comfortable with Definition 14.1:

- *QUADEQ* is in **NP** since for every ℓ-variable instance of quadratic equations E, $QUADEQ(E) = 1$ if and only if there exists an assignment $x \in \{0,1\}^\ell$ that satisfies E. We can check the condition that x satisfies E in polynomial time by enumerating over all the equations in E, and for each such equation e, plug in the values of x and verify that e is satisfied.

- *ISET* is in **NP** since for every graph G and integer k, $ISET(G,k) = 1$ if and only if there exists a set S of k vertices that contains no pair of neighbors in G. We can check the condition that S is an independent set of size $\geq k$ in polynomial time by first checking that $|S| \geq k$ and then enumerating over all edges $\{u,v\}$ in G, and for each such edge verify that either $u \notin S$ or $v \notin S$.

- *LONGPATH* is in **NP** since for every graph G and integer k, $LONGPATH(G,k) = 1$ if and only if there exists a simple path P in G that is of length at least k. We can check the condition that P is a simple path of length k in polynomial time by checking that it has the form $(v_0, v_1, \ldots, v_k)$ where each v_i is a vertex in G, no v_i is repeated, and for every $i \in [k]$, the edge $\{v_i, v_{i+1}\}$ is present in the graph.

- *MAXCUT* is in **NP** since for every graph G and integer k, $MAXCUT(G,k) = 1$ if and only if there exists a cut $(S, \overline{S})$ in G that cuts at least k edges. We can check that condition that $(S, \overline{S})$ is a cut of value at least k in polynomial time by checking that S is a subset of G's vertices and enumerating over all the edges $\{u,v\}$ of G, counting those edges such that $u \in S$ and $v \notin S$ or vice versa.

14.1.2 Basic facts about NP

The definition of **NP** is one of the most important definitions of this book, and is worth while taking the time to digest and internalize. The following solved exercises establish some basic properties of this class. As usual, I highly recommend that you try to work out the solutions yourself.

Solved Exercise 14.1 — Verifying is no harder than solving. Prove that $\mathbf{P} \subseteq \mathbf{NP}$. ■

Solution:

Suppose that $F \in \mathbf{P}$. Define the following function V: $V(x0^n) = 1$ iff $n = |x|$ and $F(x) = 1$. (V outputs 0 on all other inputs.) Since $F \in \mathbf{P}$ we can clearly compute V in polynomial time as well.

Let $x \in \{0,1\}^n$ be some string. If $F(x) = 1$ then $V(x0^n) = 1$. On the other hand, if $F(x) = 0$ then for every $w \in \{0,1\}^n$, $V(xw) = 0$.

Therefore, setting $a = b = 1$, we see that V satisfies (**??**), and establishes that $F \in$ **NP**.

■



Remark 14.5 — NP does not mean non-polynomial!. People sometimes think that **NP** stands for "non polynomial time". As Solved Exercise 14.1 shows, this is far from the truth, and in fact every polynomial-time computable function is in **NP** as well.

If F is in **NP** it certainly does *not* mean that F is hard to compute (though it does not, as far as we know, necessarily mean that it's easy to compute either). Rather, it means that F is *easy to verify*, in the technical sense of Definition 14.1.

Solved Exercise 14.2 — NP is in exponential time. Prove that **NP** $\subseteq$ **EXP**.

■

Solution:

Suppose that $F \in$ **NP** and let V be the polynomial-time computable function that satisfies (**??**) and a, b the corresponding constants. Then the following is an exponential-time algorithm A to compute F:

> **Algorithm A:**
>
> **Input:** $x \in \{0,1\}^*$, let $n = |x|$
>
> **Operation:**
>
> 1. For every $w \in \{0,1\}^{an^b}$, if $V(xw) = 1$ then halt and output 1.
> 2. Output 0.

Since $V \in$ **P**, for every $x \in \{0,1\}^n$, Algorithm A runs in time $poly(n)2^{an^b}$. Moreover by (**??**), A will output 1 on x if and only if $F(x) = 1$.

■

Solved Exercise 14.1 and Solved Exercise 14.2 together imply that

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP} \,. \tag{14.2}$$

The time hierarchy theorem (Theorem 12.10) implies that **P** $\subsetneq$ **EXP** and hence at least one of the two inclusions **P** $\subseteq$ **NP** or **NP** $\subseteq$ **EXP** is *strict*. It is believed that both of them are in fact strict inclusions. That is, it is believed that there are functions in **NP** that cannot be

computed in polynomial time (this is the **P** $\neq$ **NP** conjecture) and that there are functions F in **EXP** for which we cannot even efficiently *certify* that $F(x) = 1$ for a given input x.[2]

We have previously informally equated the notion of $F \leq_p G$ with F being "no harder than G" and in particular have seen in Solved Exercise 13.1 that if $G \in$ **P** and $F \leq_p G$, then $F \in$ **P** as well. The following exercise shows that if $F \leq_p G$ then it is also "no harder to verify" than G. That is, regardless of whether or not it is in **P**, if G has the property that solutions to it can be efficiently verified, then so does F.

[2] One function F that is believed to lie outside **NP** is the function $\overline{3SAT}$ defined as $\overline{3SAT}(\varphi) = 1 - 3SAT(\varphi)$ for every 3CNF formula φ. The conjecture that $\overline{3SAT} \notin$ **NP** is known as the "**NP** $\neq$ **coNP**" conjecture. It implies the **P** $\neq$ **NP** conjecture (can you see why?).

Solved Exercise 14.3 — Reductions and NP. Let $F, G : \{0,1\}^* \to \{0,1\}$. Show that if $F \leq_p G$ and $G \in$ **NP** then $F \in$ **NP**.

■

Solution:

Suppose that G is in **NP** and in particular there exists a, b and $V \in$ **P** such that for every $y \in \{0,1\}^*$, $G(y) = 1 \Leftrightarrow \exists_{w \in \{0,1\}^{a|y|^b}} V(yw) = 1$. Define $V'(x,w) = 1$ iff $V(R(x)w) = 1$ where R is the polynomial-time reduction demonstrating that $F \leq_p G$. Then for every $x \in \{0,1\}^*$,

$$F(x) = 1 \Leftrightarrow G(R(x)) = 1 \Leftrightarrow \exists_{w \in \{0,1\}^{a|R(x)|^b}} V(R(x)w) = 1 \Leftrightarrow \exists_{w \in \{0,1\}^{a|R(x)|^b}} V'(x,w) = 1 \tag{14.3}$$

Since there are some constants a', b' such that $|R(x)| \leq a'|x|^{b'}$ for every $x \in \{0,1\}^*$, by simple padding we can modify V' to an algorithm that certifies that $F \in$ **NP**.

■

14.2 FROM NP TO 3SAT: THE COOK-LEVIN THEOREM

We have seen everal example of problems for which we do not know if their best algorithm is polynomial or exponential, but we can show that they are in **NP**. That is, we don't know if they are easy to *solve*, but we do know that it is easy to *verify* a given solution. There are many, many, *many*, more examples of interesting functions we would like to compute that are easily shown to be in **NP**. What is quite amazing is that if we can solve 3SAT then we can solve all of them!

The following is one of the most fundamental theorems in Computer Science:

Theorem 14.6 — Cook-Levin Theorem. For every $F \in$ **NP**, $F \leq_p 3SAT$.

We will soon show the proof of Theorem 14.6, but note that it immediately implies that *QUADEQ*, *LONGPATH*, and *MAXCUT* all

reduce to 3*SAT*. Combining it with the reductions we've seen in Chapter 13, it implies that all these problems are *equivalent!* For example, to reduce *QUADEQ* to *LONGPATH*, we can first reduce *QUADEQ* to 3*SAT* using Theorem 14.6 and use the reduction we've seen in Theorem 13.7 from 3*SAT* to *LONGPATH*. That is, since $QUADEQ \in \mathbf{NP}$, Theorem 14.6 implies that $QUADEQ \leq_p 3SAT$, and Theorem 13.7 implies that $3SAT \leq_p LONGPATH$, which by the transitivity of reductions (Lemma 13.2) means that $QUADEQ \leq_p LONGPATH$. Similarly, since $LONGPATH \in \mathbf{NP}$, we can use Theorem 14.6 and Theorem 13.4 to show that $LONGPATH \leq_p 3SAT \leq_p QUADEQ$, concluding that *LONGPATH* and *QUADEQ* are computationally equivalent.

There is of course nothing special about *QUADEQ* and *LONGPATH* here: by combining (14.6) with the reductions we saw, we see that just like 3*SAT*, *every* $F \in \mathbf{NP}$ reduces to *LONGPATH*, and the same is true for *QUADEQ* and *MAXCUT*. All these problems are in some sense "the hardest in **NP**" since an efficient algorithm for any one of them would imply an efficient algorithm for *all* the problems in **NP**. This motivates the following definition:

Definition 14.7 — NP-hardness and NP-completeness. We say that $G : \{0,1\}^* \to \{0,1\}$ is **NP** *hard* if for every $F \in \mathbf{NP}$, $F \leq_p G$.

We say that $G : \{0,1\}^* \to \{0,1\}$ is **NP** *complete* if G is **NP** hard and G is in **NP**.

The Cook-Levin Theorem (Theorem 14.6) can be rephrased as saying that 3*SAT* is **NP** hard, and since it is also in **NP**, this means that 3*SAT* is **NP** complete. Together with the reductions of Chapter 13, Theorem 14.6 shows that despite their superficial differences, 3SAT, quadratic equations, longest path, independent set, and maximum cut, are all **NP**-complete. Many thousands of additional problems have been shown to be **NP**-complete, arising from all the sciences, mathematics, economics, engineering and many other fields.[3]

[3] For some partial lists, see this Wikipedia page and this website.

14.2.1 What does this mean?

As we've seen in Solved Exercise 14.1, $\mathbf{P} \subseteq \mathbf{NP}$. *The* most famous conjecture in Computer Science is that this containment is *strict*. That is, it is widely conjectured that $\mathbf{P} \neq \mathbf{NP}$. One way to refute the conjecture that $\mathbf{P} \neq \mathbf{NP}$ is to give a polynomial-time algorithm for even a single one of the **NP**-complete problems such as 3SAT, Max Cut, or the thousands of others that have been studied in all fields of human endeavors. The fact that these problems have been studied by so many people, and yet not a single polynomial-time algorithm for any of them has been found, supports that conjecture that indeed $\mathbf{P} \neq \mathbf{NP}$. In fact, for many of these problems (including all the ones we mentioned above), we don't even know of a $2^{o(n)}$-time algorithm! However, to the

frustration of computer scientists, we have not yet been able to prove that **P** $\neq$ **NP** or even rule out the existence of an $O(n)$-time algorithm for 3SAT. Resolving whether or not **P** $=$ **NP** is known as the P vs NP problem. A million-dollar prize has been offered for the solution of this problem, a popular book has been written, and every year a new paper comes out claiming a proof of **P** $=$ **NP** or **P** $\neq$ **NP**, only to wither under scrutiny.[4] The following 120 page survey of Aaronson, as well as chapter 3 in Wigderson's upcoming book are excellent sources for summarizing what is known about this problem.

One of the mysteries of computation is that people have observed a certain empirical "zero-one law" or "dichotomy" in the computational complexity of natural problems, in the sense that many natural problems are either in **P** (often in $TIME(O(n))$ or $TIME(O(n^2))$), or they are are **NP** hard. This is related to the fact that for most natural problems, the best known algorithm is either exponential or polynomial, with not too many examples where the best running time is some strange intermediate complexity such as $2^{2^{\sqrt{\log n}}}$. However, it is believed that there exist problems in **NP** that are neither in **P** nor are **NP**-complete, and in fact a result known as "Ladner's Theorem" shows that if **P** $\neq$ **NP** then this is indeed the case (see also Exercise 14.1 and Fig. 14.2).

5

6

14.2.2 The Cook-Levin Theorem: Proof outline

We will now prove the Cook-Levin Theorem, which is the underpinning to a great web of reductions from 3SAT to thousands of problems across great many fields. Some problems that have been shown to be **NP**-complete include: minimum-energy protein folding, minimum surface-area foam configuration, map coloring, optimal Nash equilibrium, quantum state entanglement, minimum supersequence of a genome, minimum codeword problem, shortest vector in a lattice, minimum genus knots, positive Diophantine equations, integer programming, and many many more. The worst-case complexity of all these problems is (up to polynomial factors) equivalent to that of 3SAT, and through the Cook-Levin Theorem, to all problems in **NP**.

To prove Theorem 14.6 we need to show that $F \leq_p 3SAT$ for every $F \in$ **NP**. We will do so in three stages. We define two intermediate problems: *NANDSAT* and *3NAND*. We will shortly show the definitions of these two problems, but Theorem 14.6 will follow from combining the following three results:

1. *NANDSAT* is **NP** hard (Lemma 14.8).

2. $NANDSAT \leq_p 3NAND$ (Lemma 14.10).

[4] The following web page keeps a catalog of these failed attempts. At the time of this writing, it lists about 110 papers claiming to resolve the question, of which about 60 claim to prove that **P** $=$ **NP** and about 50 claim to prove that **P** $\neq$ **NP**.

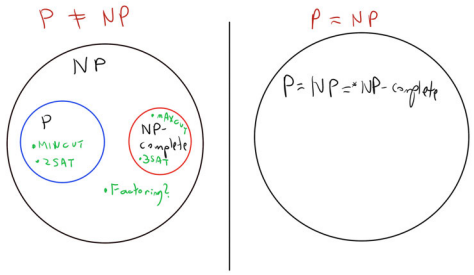


Figure 14.2: The world if **P** $\neq$ **NP** (left) and **P** $=$ **NP** (right). In the former case the set of **NP**-complete problems is disjoint from **P** and Ladner's theorem shows that there exist problems that are neither in **P** nor are **NP**-complete. (There are remarkably few natural candidates for such problems, with some prominent examples being decision variants of problems such as integer factoring, lattice shortest vector, and finding Nash equilibria.) In the latter case that **P** $=$ **NP** the notion of **NP**-completeness loses its meaning, as essentially all functions in **P** (save for the trivial constant zero and constant one functions) are **NP**-complete.

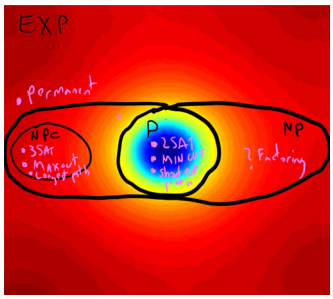


Figure 14.3: A rough illustration of the (conjectured) status of problems in exponential time. Darker colors correspond to higher running time, and the circle in the middle is the problems in **P**. **NP** is a (conjectured to be proper) superclass of **P** and the **NP**-complete problems (or **NPC** for short) are the "hardest" problems in **NP**, in the sense that a solution for one of them implies a solution for all other problems in **NP**. It is conjectured that all the **NP**-complete problems require at least $\exp(n^\epsilon)$ time to solve for a constant $\epsilon > 0$, and many require $\exp(\Omega(n))$ time. The *permanent* is not believed to be contained in **NP** though it is **NP**-hard, which means that a polynomial-time algorithm for it implies that **P** $=$ **NP**.

[5] TODO: maybe add examples of NP hard problems as a barrier to understanding - problems from economics, physics, etc.. that prevent having a closed-form solutions

[6] TODO: maybe include knots

3. $3NAND \leq_p 3SAT$ (Lemma 14.11).

By the transitivity of reductions, it will follow that for every $F \in$ **NP**,

$$F \leq_p NANDSAT \leq_p 3NAND \leq_p 3SAT \tag{14.4}$$

hence establishing Theorem 14.6.

We will prove these three results Lemma 14.8, Lemma 14.10 and Lemma 14.11 one by one, providing the requisite definitions as we go along.

14.3 THE *NANDSAT* PROBLEM, AND WHY IT IS NP HARD.

We define the *NANDSAT* problem as follows. On input a string $Q \in \{0,1\}^*$, we define $NANDSAT(Q) = 1$ if and only if Q is a valid representation of an n-input and single-output NAND program and there exists some $w \in \{0,1\}^n$ such that $Q(w) = 1$. While we don't need this to prove Lemma 14.8, note that *NANDSAT* is in **NP** since we can verify that $Q(w) = 1$ using the polyonmial-time algorithm for evaluating NAND-CIRC programs.[7] We now prove that *NANDSAT* is **NP** hard.

[7] Q is a NAND-CIRC program and not a NAND-TM program, and hence it is only defined on inputs of some particular size n. Evaluating Q on any input $w \in \{0,1\}^n$ can be done in time polynomial in the number of lines of Q.

Lemma 14.8 *NANDSAT* is **NP** hard.

Proof Idea:

To prove Lemma 14.8 we need to show that for every $F \in$ **NP**, $F \leq_p NANDSAT$. The high-level idea is that by the definition of **NP**, there is some NAND-TM program P^* and some polynomial $T(\cdot)$ such that $F(x) = 1$ if and only if there exists some $w \in \{0,1\}^{a|x|^b}$ such that $P^*(xw)$ outputs 1 within $T(|x|)$ steps. Now by "unrolling the loop" of the NAND-TM program P^* we can convert it into an $O(T(n))$ NAND-CIRC program Q' with $n + an^b$ inputs and a single output such that for every $x \in \{0,1\}^n$ and $w \in \{0,1\}^{an^b}$, $Q'(xw) = P^*(xw)$. on input $x \in \{0,1\}$ that on input w will simulate $P^*(xw)$ for $T(|x|)$ steps. The next step is to *hardwire* the input x to Q' to obtain an $O(T(n))$ line NAND-CIRC program Q with $m = an^b$ inputs such that for every $w \in \{0,1\}^m$, $Q'(w) = Q(xw)$. By construction it will be the case that for every $x \in \{0,1\}^n$, $F(x) = 1$ if and only if there exists $w \in \{0,1\}^{an^b}$ such that $Q(w) = 1$, and hence this shows that $F \leq_p NANDSAT$.

⋆

The proof is a little bit technical but ultimately follows quite directly from the definition of **NP**, as well as of NAND and NAND-TM programs. If you find it confusing, try to pause here and work out the proof

yourself from these definitions, using the idea of "unrolling the loop" of a NAND-TM program. It might also be useful for you to think how you would implement in your favorite programming language the function `unroll` which on input a NAND-TM program P and numbers T, n would output an n-input NAND-CIRC program Q of $O(|T|)$ lines such that for every input $z \in \{0,1\}^n$, if P halts on z within at most T steps and outputs y, then $Q(z) = y$.

Proof of Lemma 14.8. We now present the details. Let $F \in \mathbf{NP}$. To prove Lemma 14.8 we need to give a polynomial-time computable function that will map every $x^* \in \{0,1\}^*$ to a NAND-CIRC program Q such that $F(x) = NANDSAT(Q)$.

Let $x^* \in \{0,1\}^*$ be such a string and let $n = |x^*|$ be its length. By Definition 14.1 there exists $V \in \mathbf{P}$ and $a, b \in \mathbb{N}$ such that $F(x^*) = 1$ if and only if there exists $w \in \{0,1\}^{an^b}$ such that $V(x^*w) = 1$.

Let $m = an^b$. Since $V \in \mathbf{P}$ there is some NAND-TM program P^* that computes V on inputs of the form xw with $x \in \{0,1\}^n$ and $w \in \{0,1\}^m$ in at most $(n+m)^c$ time for some constant c. Using our "unrolling the loop NAND-TM to NAND compiler" of Theorem 12.13, we can obtain a NAND-CIRC program Q' that has $n+m$ inputs and at most $O((n+m)^c)$ lines such that $Q'(xw) = P^*(xw)$ for every $x \in \{0,1\}^n$ and $w \in \{0,1\}^m$.

Now we can use the following simple but useful "hardwiring" technique to obtain a program:

Lemma 14.9 — Hardwiring Lemma. Given a T-line NAND-CIRC program Q' of $n+m$ inputs and $x^* \in \{0,1\}^n$, we can obtain in polynomial a program Q with m inputs of $T+3$ lines such that for ever $w \in \{0,1\}^m$, $Q(w) = Q'(x^*w)$.

Proof. To compute Q, we simply do a "search and replace" for all references in Q' to `X[`i`]` for $i \in [n]$, and transform them to either the variable `zero` or `one` depending on whether x^*_i is equal to 0 or 1 respectively. By adding three lines to the beginning of Q', we can ensure that the `zero` and `one` variables will have the correct value. The only thing that then remains to do another search and replace to transform all references to the variables `X[`n`]`,..., `X[`$n+m-1$`]` to the variables `X[0]`, ..., `X[`$m-1$`]` so that the m inputs to the new program Q will correspond to last m inputs of the original program Q'. See Fig. 14.4 for an implementation of this reduction in Python. ■

Using Lemma 14.9, we obtain a program Q of m inputs such that $Q(w) = Q'(x^*w) = P^*(x^*w)$ for every $w \in \{0,1\}^m$. Since we know that

$F(x^*) = 1$ if and only if there exists $w \in \{0,1\}^m$ such that $P^*(x^*w) = 1$, this means that $F(x^*) = 1$ if and only if $NANDSAT(Q) = 1$, which is what we wanted to prove.

■

```python
# "hardwire" an input x to a NAND program Q
# return Q' s.t. Q'(y)=Q(xy)
def hardwire(Q,x):
    n = len(x)

    for i in range(n):
        Q = Q.replace(f'X[{i}]',('one' if x[i] else 'zero'))

    # move x_n,x_n+1,.... to x_0,x_1,...
    for i in range(n,2*len(Q)):
        Q = Q.replace(f'X[{i}]',f'X[{i-n}]')

    return CONSTPREFIX+Q

CONSTPREFIX = r'''
temp = NAND(X[0],X[0])
one = NAND(X[0],temp)
zero = NAND(one,one)
'''[1:]
```

```python
print(xor5)
```

```
u   =  NAND(X[0],X[1])
v   = NAND( X[0] , u)
w   = NAND( X[1] , u)
s   = NAND( v   , w)
u   = NAND( s   , X[2])
v   = NAND( s   , u)
w   = NAND( X[2] , u)
s   = NAND( v   , w)
u   = NAND( s   , X[3])
v   = NAND( s   , u)
w   = NAND( X[3] , u)
s   = NAND( v   , w)
u   = NAND( s   , X[4])
v   = NAND( s   , u)
w   = NAND( X[4] , u)
Y[0] = NAND( v   , w)
```

```python
print(hardwire(xor5,[1,0,1]))
```

```
temp = NAND(X[0],X[0])
one = NAND(X[0],temp)
zero = NAND(one,one)
u   =  NAND(one,zero)
v   = NAND( one , u)
w   = NAND( zero , u)
s   = NAND( v   , w)
u   = NAND( s   , one)
v   = NAND( s   , u)
w   = NAND( one , u)
s   = NAND( v   , w)
u   = NAND( s   , X[0])
v   = NAND( s   , u)
w   = NAND( X[0] , u)
s   = NAND( v   , w)
u   = NAND( s   , X[1])
v   = NAND( s   , u)
w   = NAND( X[1] , u)
Y[0] = NAND( v   , w)
```

Figure 14.4: Given an T-line NAND-CIRC program Q that has $n + m$ inputs and some $x^* \in \{0,1\}^n$, we can transform Q into a $T + 3$ line NAND-CIRC program Q' that computes the map $w \mapsto Q(x^*w)$ for $w \in \{0,1\}^m$ by simply adding code to compute the `zero` and `one` constants, replacing all references to `X[`i`]` with either `zero` or `one` depending on the value of x^*_i, and then replacing the remaining references to `X[`j`]` with `X[`$j-n$`]`. Above is Python code that implements this transformation, as well as an example of its execution on a simple program.

14.4 THE $3NAND$ PROBLEM

The $3NAND$ problem is defined as follows: the input is a logical formula φ on a set of variables $z_0, \ldots, z_{r-1}$ which is an AND of constraints of the form $z_i = NAND(z_j, z_k)$. For example, the following is a $3NAND$ formula with 5 variables and 3 constraints:

$$(z_3 = NAND(z_0, z_2)) \wedge (z_1 = NAND(z_0, z_2)) \wedge (z_4 = NAND(z_3, z_1)) \tag{14.5}$$

The output of $3NAND$ on input φ is 1 if and only if there is an assignment to the variables of φ that makes it evaluate to "true" (that is, there is some assignment $z \in \{0,1\}^r$ satisfying all of the constraints of φ). As usual, we can represent φ as a string, and so think of $3NAND$ as a function mapping $\{0,1\}^*$ to $\{0,1\}$. We now prove that $3NAND$ is **NP** hard:

Lemma 14.10 $NANDSAT \leq_p 3NAND$.

Proof Idea:

To prove Lemma 14.10 we need to give a polynomial-time map from every NAND-CIRC program Q to a 3NAND formula Ψ such that there exists w such that $Q(w) = 1$ if and only if there exists z satisfying Ψ. For every line i of Q, we define a corresponding variable z_i of Ψ. If the line i has the form `foo = NAND(bar,blah)` then we will add the clause $z_i = NAND(z_j, z_k)$ where j and k are the last lines in which `bar` and `blah` were written to. We will also set variables corresponding to the input variables, as well as add a clause to ensure that the final

output is 1. The resulting reduction can be implemented in about a dozen lines of Python, see **??**.

⋆

```
# Reduce NANDSAT to 3NAND
# Input: NAND prog Q
# Output: 3NAND formula Ψ
#         s.t. Ψ satisfiable iff Q is
def NANDSAT23NAND_(Q):
    Q = CONSTPREFIX + Q
    n, _ = numinout(Q)
    Ψ = ''

    #varidx[u] is n+line where u is last written to
    varidx = defaultdict(lambda : n+ 2) # line 2 corresponds to zero

    for i in range(n): varidx[f'X[{i}]'] = i # setup x_0...x_n-1

    j = n
    for line in Q.split('\n'):
        if not line.strip(): continue
        foo,bar,blah = splitline(line) # split "foo = NAND(bar,blah)"
        Ψ +=  f"(z{j} = NAND(z{varidx[bar]},z{varidx[blah]}) ) ∧ "
        varidx[foo] = j
        j += 1
    Ψ += f"(z{varidx['Y[0]']} = NAND(z{varidx['zero']},z{varidx['zero']}) )"
    return Ψ
```

```
print(xor5)
```

```
u   =  NAND(X[0],X[1])
v   = NAND( X[0] , u)
w   = NAND( X[1] , u)
s   = NAND( v    , w)
u   = NAND( s    , X[2])
v   = NAND( s    , u)
w   = NAND( X[2] , u)
s   = NAND( v    , w)
u   = NAND( s    , X[3])
v   = NAND( s    , u)
w   = NAND( X[3] , u)
s   = NAND( v    , w)
u   = NAND( s    , X[4])
v   = NAND( s    , u)
w   = NAND( X[4] , u)
Y[0] = NAND( v    , w)
```

```
print(NANDSAT23NAND_(xor5))
```

```
(z5 = NAND(z0,z0) ) ∧ (z6 = NAND(z0,z5) ) ∧ (z7 = NAND(z6,z6) ) ∧ (z8 = NAND(z0,z1) ) ∧ (z9 = NAND(z
0,z8) ) ∧ (z10 = NAND(z1,z8) ) ∧ (z11 = NAND(z9,z10) ) ∧ (z12 = NAND(z11,z2) ) ∧ (z13 = NAND(z11,z1
2) ) ∧ (z14 = NAND(z2,z12) ) ∧ (z15 = NAND(z13,z14) ) ∧ (z16 = NAND(z15,z3) ) ∧ (z17 = NAND(z15,z16)
) ∧ (z18 = NAND(z3,z16) ) ∧ (z19 = NAND(z17,z18) ) ∧ (z20 = NAND(z19,z4) ) ∧ (z21 = NAND(z19,z20) )
∧ (z22 = NAND(z4,z20) ) ∧ (z23 = NAND(z21,z22) ) ∧ (z23 = NAND(z7,z7) )
```

```
eval3NAND(NANDSAT23NAND_(xor5),[1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1])
```

```
True
```

Figure 14.5: Python code to reduce an instance Q of *NANDSAT* to an instance Ψ of *3NAND*. In the example above we transform the NAND-CIRC program `xor5` which has 5 input variables and 16 lines, into a *3NAND* formula Ψ that has 24 variables and 20 clauses. Since `xor5` outputs 1 on the input $1,0,0,1,1$, there exists an assignment $z \in \{0,1\}^{24}$ to Ψ such that $(z_0, z_1, z_2, z_3, z_4) = (1,0,0,1,1)$ and Ψ evaluates to *true* on z.

Proof of Lemma 14.10. To prove Lemma 14.10 we need to give a reduction from *NANDSAT* to *3NAND*. Let Q be a NAND-CIRC program with n inputs, one output, and m lines. We can assume without loss of generality that Q contains the variables `one` and `zero` as usual.

We map Q to a *3NAND* formula Ψ as follows:

- Ψ has $m+n$ variables $z_0, \ldots, z_{m+n-1}$.
- The first n variables $z_0, \ldots, z_{n-1}$ will corresponds to the inputs of Q. The next m variables $z_n, \ldots, z_{n+m-1}$ will correspond to the m lines of Q.
- For every $\ell \in \{n, n+1, \ldots, n+m\}$, if the $\ell - n$-th line of the program Q is `foo = NAND(bar,blah)` then we add to Ψ the constraint $z_\ell = NAND(z_j, z_k)$ where $j-n$ and $k-n$ correspond to the last lines in which the variables `bar` and `blah` (respectively) were written to. If one or both of `bar` and `blah` was not written to before then we use z_{ℓ_0} instead of the corresponding value z_j or z_k in the constraint, where $\ell_0 - n$ is the line in which `zero` is assigned a value. If one or both of `bar` and `blah` is an input variable `X[i]` then we use z_i in the constraint.
- Let ℓ^* be the last line in which the output `y_0` is assigned a value. Then we add the constraint $z_{\ell^*} = NAND(z_{\ell_0}, z_{\ell_0})$ where $\ell_0 - n$ is as above the last line in which `zero` is assigned a value. Note that this is effectively the constraint $z_{\ell^*} = NAND(0,0) = 1$.

To complete the proof we need to show that there exists $w \in \{0,1\}^n$ s.t. $Q(w) = 1$ if and only if there exists $z \in \{0,1\}^{n+m}$ that satisfies all constraints in Ψ. We now show both sides of this equivalence.

Part I: Completeness. Suppose that there is $w \in \{0,1\}^n$ s.t. $Q(w) = 1$. Let $z \in \{0,1\}^{n+m}$ be defined as follows: for $i \in [n]$, $z_i = w_i$ and for $i \in \{n, n+1, \ldots, n+m\}$ z_i equals the value that is assigned in the $(i-n)$-th line of Q when executed on w. Then by construction z satisfies all of the constraints of Ψ (including the constraint that $z_{\ell^*} = NAND(0,0) = 1$ since $Q(w) = 1$.)

Part II: Soundness. Suppose that there exists $z \in \{0,1\}^{n+m}$ satisfying Ψ. Soundness will follow by showing that $Q(z_0, \ldots, z_{n-1}) = 1$ (and hence in particular there exists $w \in \{0,1\}^n$, namely $w = z_0 \cdots z_{n-1}$, such that $Q(w) = 1$). To do this we will prove the following claim $(*)$: for every $\ell \in [m]$, $z_{\ell+n}$ equals the value assigned in the ℓ-th step of the execution of the program Q on $z_0, \ldots, z_{n-1}$. Note that because z satisfies the constraints of Ψ, $(*)$ is sufficient to prove the soundness condition since these constraints imply that the last value assigned to the variable `y_0` in the execution of Q on $z_0 \cdots w_{n-1}$ is equal to 1. To prove $(*)$ suppose, towards a contradiction, that it is false, and let ℓ be the smallest number such that $z_{\ell+n}$ is *not* equal to the value assigned in the ℓ-th step of the exeuction of Q on $z_0, \ldots, z_{n-1}$. But since z satisfies the constraints of Ψ, we get that $z_{\ell+n} = NAND(z_i, z_j)$ where (by the assumption above that ℓ is *smallest* with this property) these values *do* correspond to the values last assigned to the variables on the righthand side of the assignment operator in the ℓ-th line of the program. But this means that the value assigned in the ℓ-th step is indeed simply the NAND of z_i and z_j, contradicting our assumption on the choice of ℓ.

■

14.5 FROM 3NAND TO 3SAT

To conclude the proof of Theorem 14.6, we need to show Lemma 14.11 and show that $3NAND \leq_p 3SAT$:

Lemma 14.11 $3NAND \leq_p 3SAT$.

Proof Idea:

To prove Lemma 14.11 we need to map a 3NAND formula φ into a 3SAT formula ψ such that φ is satisfiable if and only if ψ is. The idea is that we can transform every NAND constraint of the form $a = NAND(b,c)$ into the AND of ORs involving the variables a, b, c and their negations, where each of the ORs contains at most three terms. The construction is fairly straightforward, and the details are given below.

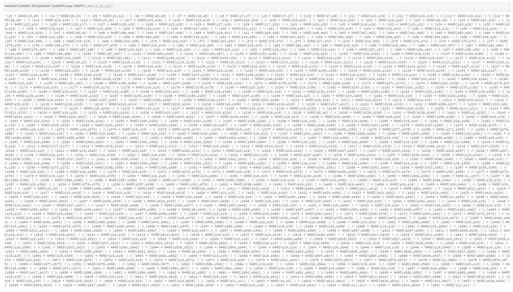


Figure 14.6: A $3NAND$ instance that is obtained by taking a NAND-TM program for computing the *AND* function, unrolling it to obtain a *NANDSAT* instance, and then composing it with the reduction of Lemma 14.10.

⋆

P It is a good exercise for you to try to find a 3CNF formula ξ on three variables a, b, c such that $\xi(a, b, c)$ is true if and only if $a = NAND(b, c)$. Once you do so, try to see why this implies a reduction from 3*NAND* to 3*SAT*, and hence completes the proof of Lemma 14.11

```
# Reduce 3NAND to 3SAT
# Input: 3NAND formula Ψ
# Output: 3CNF formula φ
#         s.t.  φ satisfiable iff Ψ is
def NAND23SAT_(Ψ):
    ϕ = ""
    for (a,b,c) in getnandclauses(Ψ):
        ϕ += f'(¬{a} ∨ ¬{b} ∨ ¬{c}) ∧ ({a} ∨ {b}  ∨ {b}) ∧ ({a}  ∨  {c} ∨ {c}) ∧ '
    return ϕ[:-3] # chop off redundant ∧
```

```
Ψ = "(x0 = NAND(x2,x3) ) ∧ (x3 = NAND(x2,x1) ) ∧ (x1 = NAND(x2,x3) ) "
NAND23SAT_(Ψ)
```

```
'(¬x0 ∨ ¬x2 ∨ ¬x3) ∧ (x0 ∨ x2  ∨ x2) ∧ (x0  ∨  x3 ∨ x3) ∧ (¬x3 ∨ ¬x2 ∨ ¬x1) ∧ (x3 ∨ x2  ∨ x2) ∧ (x3
∨  x1 ∨ x1) ∧ (¬x1 ∨ ¬x2 ∨ ¬x3) ∧ (x1 ∨ x2  ∨ x2) ∧ (x1  ∨  x3 ∨ x3)'
```

Figure 14.7: Code and example output for the reduction given in Lemma 14.11 of 3*NAND* to 3*SAT*.

Proof of Lemma 14.11. The constraint

$$z_i = NAND(z_j, z_k) \tag{14.6}$$

is satisfied if $z_i = 1$ whenever $(z_j, z_k) \neq (1, 1)$. By going through all cases, we can verify that (14.6) is equivalent to the constraint

$$(\overline{z_i} \vee \overline{z_j} \vee \overline{z_k}) \wedge (z_i \vee z_j) \wedge (z_i \vee z_k) \quad . \tag{14.7}$$

Indeed if $z_j = z_k = 1$ then the first constraint of Eq. (14.7) is only true if $z_i = 0$. On the other hand, if either of z_j or z_k equals 0 then unless $z_i = 1$ either the second or third constraints will fail. This means that, given any 3NAND formula φ over n variables $z_0, \ldots, z_{n-1}$, we can obtain a 3SAT formula ψ over the same variables by replacing every 3*NAND* constraint of φ with three 3*OR* constraints as in Eq. (14.7).[8] Because of the equivalence of (14.6) and (14.7), the formula ψ satisfies that $\psi(z_0, \ldots, z_{n-1}) = \varphi(z_0, \ldots, z_{n-1})$ for every assignment $z_0, \ldots, z_{n-1} \in \{0, 1\}^n$ to the variables. In particular ψ is satisfiable if and only if φ is, thus completing the proof.

■

[8] The resulting forumula will have some of the OR's involving only two variables. If we wanted to insist on each formula involving three distinct variables we can always add a "dummy variable" z_{n+m} and include it in all the OR's involving only two variables, and add a constraint requiring this dummy variable to be zero.

14.6 WRAPPING UP

We have shown that for every function F in **NP**, $F \leq_p NANDSAT \leq_p 3NAND \leq_p 3SAT$, and so 3*SAT* is **NP**-hard. Since in Chapter 13 we saw that $3SAT \leq_p QUADEQ$, $3SAT \leq_p ISET$, $3SAT \leq_p MAXCUT$

and $3SAT \leq_p LONGPATH$, all these problems are **NP**-hard as well. Finally, since all the aforementioned problems are in **NP**, they are all in fact **NP**-complete and have equivalent complexity. There are thousands of other natural problems that are **NP**-complete as well. Finding a polynomial-time algorithm for any one of them will imply a polynomial-time algorithm for all of them.

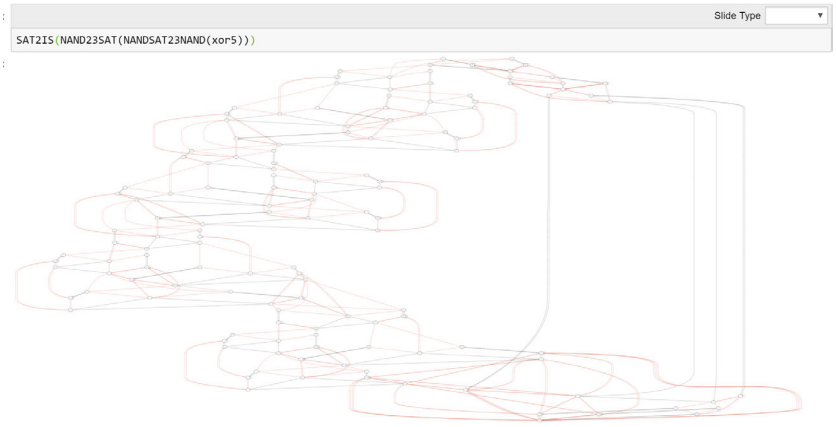


Figure 14.8: An instance of the *independent set* problem obtained by applying the reductions $NANDSAT \leq_p 3NAND \leq_p 3SAT \leq_p ISAT$ starting with the `xor5` NAND-CIRC program.



Lecture Recap

- Many of the problems for which we don't know polynomial-time algorithms are **NP**-complete, which means that finding a polynomial-time algorithm for one of them would imply a polynomial-time algorithm for *all* of them.
- It is conjectured that **NP** $\neq$ **P** which means that we believe that polynomial-time algorithms for these problems are not merely *unknown* but are *nonexistent*.
- While an **NP**-hardness result means for example that a full-fledged "textbook" solution to a problem such as MAX-CUT that is as clean and general as the algorithm for MIN-CUT probably does not exist, it does not mean that we need to give up whenever we see a MAX-CUT instance. Later in this course we will discuss several strategies to deal with **NP**-hardness, including *average-case complexity* and *approximation algorithms*.

14.7 EXERCISES

R

Remark 14.12 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 14.1 — Poor man's Ladner's Theorem. Prove that if there is no $n^{O(\log^2 n)}$ time algorithm for $3SAT$ then there is some $F \in \mathbf{NP}$ such that $F \notin \mathbf{P}$ and F is not **NP** complete.[9]

[9] **Hint:** Use the function F that on input a formula φ and a string of the form 1^t, outputs 1 if and only if φ is satisfiable and $t = |\varphi|^{\log|\varphi|}$.

■

14.8 BIBLIOGRAPHICAL NOTES

10

Eugene Lawler's quote on the "mystical power of twoness" was taken from the wonderful book "The Nature of Computation" by Moore and Mertens. See also this memorial essay on Lawler by Lenstra.

[10] TODO: credit surveys of Avi, Madhu

14.9 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

14.10 ACKNOWLEDGEMENTS

Learning Objectives:

- Explore the consequences of P = NP
- *Search-to-decision* reduction: transform algorithms that solve decision version to search version for NP-complete problems.
- Optimization and learning problems
- Quantifier elimination and solving problems in the polynomial hierarchy.
- What is the evidence for P = NP vs P ≠ NP?

15
What if P equals NP?

> *"You don't have to believe in God, but you should believe in The Book."*, Paul Erdős, 1985. [1]

> *"No more half measures, Walter"*, Mike Ehrmantraut in "Breaking Bad", 2010.

> *"The evidence in favor of* [**P** ≠ **NP**] *and* [*its algebraic counterpart*] *is so overwhelming, and the consequences of their failure are so grotesque, that their status may perhaps be compared to that of physical laws rather than that of ordinary mathematical conjectures."*, Volker Strassen, laudation for Leslie Valiant, 1986.

> *"Suppose aliens invade the earth and threaten to obliterate it in a year's time unless human beings can find the* [*fifth Ramsey number*]*. We could marshal the world's best minds and fastest computers, and within a year we could probably calculate the value. If the aliens demanded the* [*sixth Ramsey number*]*, however, we would have no choice but to launch a preemptive attack."*, Paul Erdős, as quoted by Graham and Spencer, 1990. [2]

We have mentioned that the question of whether **P** = **NP**, which is equivalent to whether there is a polynomial-time algorithm for 3*SAT*, is the great open question of Computer Science. But why is it so important? In this chapter, we will try to figure out the implications of such an algorithm.

First, let us get one qualm out of the way. Sometimes people say, *"What if* **P** = **NP** *but the best algorithm for 3SAT takes* n^{100} *time?"* Well, n^{100} is much larger than, say, $2^{\sqrt{n}}$ for any input shorter than 10^{60} bits, which is way, way larger than the world's total storage capacity (estimated at a "mere" 10^{21} bits or about 200 exabytes at the time of this writing). So another way to phrase this question is to say, "what if the complexity of 3SAT is exponential for all inputs that we will ever en-

[1] Paul Erdős (1913-1996) was one of the most prolific mathematicians of all times. Though he was an atheist, Erdős often referred to "The Book" in which God keeps the most elegant proof of each mathematical theorem.

[2] The k-th Ramsey number, denoted as $R(k, k)$, is the smallest number n such that for every graph G on n vertices, both G and its complement contain a k-sized independent set. If **P** = **NP** then we can compute $R(k, k)$ in time polynomial in 2^k, while otherwise it can potentially take closer to $2^{2^{2k}}$ steps.

counter, but then grows much smaller than that?" To me this sounds like the computer science equivalent of asking, "what if the laws of physics change completely once they are out of the range of our telescopes?". Sure, this is a valid possibility, but wondering about it does not sound like the most productive use of our time.

So, as the saying goes, we'll keep an open mind, but not so open that our brains fall out, and assume from now on that:

- There is a mathematical god,

  and

- She does not "pussyfoot around" or take "half measures". If God decided to make $3SAT$ *easy*, then $3SAT$ will have a $10^6 \cdot n$ (or at worst $10^6 n^2$) -time algorithm (i.e., $3SAT$ will be in $TIME(cn)$ or $TIME(cn^2)$ for a not-too-large constant c). If she decided to make $3SAT$ *hard*, then for every $n \in \mathbb{N}$, $3SAT$ on n variables cannot be solved by a NAND-CIRC program of fewer than $2^{10^{-6}n}$ lines.[3]

So far, most of our evidence points to the latter possibility of 3SAT being exponentially hard, but we have not ruled out the former possibility either. In this chapter we will explore some of its consequences.

[3] Using the relations we've seen between $SIZE(T(n))$ and $TIME(T(n))$ (i.e., Theorem 12.12), $3SAT \notin SIZE(T(n))$ then it is also in $TIME(T(n)^\epsilon)$ for some constant ϵ that can be shown to be at least $1/5$.

15.1 SEARCH-TO-DECISION REDUCTION

A priori, having a fast algorithm for 3SAT might not seem so impressive. Sure, it will allow us to decide the satisfiability of not just 3CNF formulas but also of quadratic equations, as well as find out whether there is a long path in a graph, and solve many other decision problems. But this is not typically what we want to do. It's not enough to know *if* a formula is satisfiable— we want to discover the actual satisfying assignment. Similarly, it's not enough to find out if a graph has a long path— we want to actually *find* the path.

It turns out that if we can solve these decision problems, we can solve the corresponding search problems as well:

Theorem 15.1 — Search vs Decision. Suppose that $\mathbf{P} = \mathbf{NP}$. Then for every polynomial-time algorithm V and $a, b \in \mathbb{N}$,there is a polynomial-time algorithm $FIND_V$ such that for every $x \in \{0,1\}^n$, if there exists $y \in \{0,1\}^{an^b}$ satisfying $V(xy) = 1$, then $FIND_V(x)$ finds some string y' satisfying this condition.

To understand what the statement of Theorem 15.1 means, let us look at the special case of the *MAXCUT* problem. It is not hard to see that there

is a polyomial-time algorithm *VERIFYCUT* such that $VERIFYCUT(G,k,S) = 1$ if and only if S is a subset of G's vertices that cuts at least k edges. Theorem 15.1 implies that if **P** $=$ **NP** then there is a polynomial-time algorithm *FINDCUT* that on input G,k outputs a set S such that $VERIFYCUT(G,k,S) = 1$ if such a set exists. This means that if **P** $=$ **NP**, by trying all values of k we can find in polynomial time a maximum cut in any given graph. We can use a similar argument to show that if **P** $=$ **NP** then we can find a satisfying assignment for every satisfiable 3CNF formula, find the longest path in a graph, solve integer programming, and so and so forth.

Proof Idea:

The idea behind the proof of Theorem 15.1 is simple; let us demonstrate it for the special case of $3SAT$. (In fact, this case is not so "special"— since $3SAT$ is **NP**-complete, we can reduce the task of solving the search problem for *MAXCUT* or any other problem in **NP** to the task of solving it for $3SAT$.) Suppose that **P** $=$ **NP** and we are given a satisfiable 3CNF formula φ, and we now want to find a satisfying assignment y for φ. Define $3SAT_0(\varphi)$ to output 1 if there is a satisfying assignment y for φ such that its first bit is 0, and similarly define $3SAT_1(\varphi) = 1$ if there is a satisfying assignment y with $y_0 = 1$. The key observation is that both $3SAT_0$ and $3SAT_1$ are in **NP**, and so if **P** $=$ **NP** then we can compute them in polynomial time as well. Thus we can use this to find the first bit of the satisfying assignment. We can continue in this way to recover all the bits.

⋆

Proof of Theorem 15.1. If **P** $=$ **NP** then for every polynomial-time algorithm V and $a,b \in \mathbb{N}$, there is a polynomial-time algorithm $STARTSWITH_V$ that on input $x \in \{0,1\}^*$ and $z \in \{0,1\}^\ell$, outputs 1 if and only if there exists some $y \in \{0,1\}^{an^b}$ such that the first ℓ bits of y are equal to z and $V(xy) = 1$. Indeed, we leave it as an exercise to verify that the $STARTSWITH_V$ function is in **NP** and hence can be solved in polynomial time if **P** $=$ **NP**.

Now for any such polynomial-time V and $a,b \in \mathbb{N}$, we can implement $FIND_V(x)$ as follows:

Algorithm $FIND_V$**:**

Input: $x \in \{0,1\}^n$

Goal: Find $z \in \{0,1\}^{an^b}$ such that $V(xz) = 1$, if such z exists.

Operation:

1. For $\ell = 0, \ldots, an^b - 1$ do the following:

a. Let $b_0 = STARTSWITH_V(xz_0 \cdots z_{\ell-1}0)$ and $b_1 = STARTSWITH_V(xz_0 \cdots z_{\ell-1}1)$.
b. If $b_0 = b_1 = 0$ then halt and output "no z exists". (*In this there is no extension of $xz_0 \ldots z_{\ell-1}$ that would cause V to output 1.*)
c. If $b = 1$ (*i.e., can extend $xz_0 \ldots z_{\ell-1}$ with 0*) then $z_\ell = 0$, otherwise (*i.e., can extend $xz_0 \ldots z_{\ell-1}$ with 1*) $z_\ell = 1$.

2. Output $z_0, \ldots, z_{an^b-1}$.

To analyze the *FIND* algorithm, note that it makes $2an^b$ invocations to $STARTSWITH_V$ and hence if the latter is polynomial-time, then so is $FIND_V$. Now suppose that x is such that there exists *some* y satisfying $V(xy) = 1$. We claim that at every step $\ell = 0, \ldots, an^b - 1$, we maintain the invariant that there exists $y \in \{0,1\}^{an^b}$ whose first ℓ bits are z s.t. $V(xy) = 1$. Note that this claim implies the theorem, since in particular it means that for $\ell = an^b - 1$, z satisfies $V(xz) = 1$.

We prove the claim by induction. For $\ell = 0$, this holds vacuously. Now for every $\ell > 0$, if the call $STARTSWITH_V(xz_0 \cdots z_{\ell-1}0)$ returns 1, then we are guaranteed the invariant by definition of $STARTSWITH_V$. Now under our inductive hypothesis, there is $y_\ell, \ldots, y_{an^b-1}$ such that $P(xz_0, \ldots, z_{\ell-1}y_\ell, \ldots, y_{an^b-1}) = 1$. If the call to $STARTSWITH_V(xz_0 \cdots z_{\ell-1}0)$ returns 0 then it must be the case that $y_\ell = 1$, and hence when we set $z_\ell = 1$ we maintain the invariant.

■

15.2 OPTIMIZATION

Theorem 15.1 allows us to find solutions for **NP** problems if **P** = **NP**, but it is not immediately clear that we can find the *optimal* solution. For example, suppose that **P** = **NP**, and you are given a graph G. Can you find the *longest* simple path in G in polynomial time?



This is actually an excellent question for you to attempt on your own. That is, assuming **P** = **NP**, give a polynomial-time algorithm that on input a graph G, outputs a maximally long simple path in the graph G.

It turns out the answer is *Yes*. The idea is simple: if **P** = **NP** then we can find out in polynomial time if an n-vertex graph G contains a simple path of length n, and moreover, by Theorem 15.1, if G does contain such a path, then we can find it. (Can you see why?) If G does not contain a simple path of length n, then we will check if it contains

a simple path of length $n-1$, and continue in this way to find the largest k such that G contains a simple path of length k.

The above reasoning was not specifically tailored to finding paths in graphs. In fact, it can be vastly generalized to proving the following result:

Theorem 15.2 — Optimization from P = NP. Suppose that $\mathbf{P} = \mathbf{NP}$. Then for every polynomial-time computable function $f : \{0,1\}^* \to \{0,1\}^*$ there is a polynomial-time algorithm OPT such that on input $x \in \{0,1\}^*$, $OPT(x, 1^m) = \max_{y \in \{0,1\}^m} f(x,y)$ (where we identify the output of $f(x)$ with a natural number via the binary representation).

Moreover under the same assumption, there is a polynomial-time algorithm $FINDOPT$ such that for every $x \in \{0,1\}^*$, $FINDOPT(x, 1^m)$ outputs $y^* \in \{0,1\}^*$ such that $f(x,y^*) = OPT(x,y^*)$.



The statement of Theorem 15.2 is a bit cumbersome. To understand it, think how it would subsume the example above of a polynomial time algorithm for finding the maximum length path in a graph. In this case the function f would be the map that on input a pair x, y outputs 0 if the pair (x,y) does not represent some graph and a simple path inside the graph respectively; otherwise $f(x,y)$ would equal the length of the path y in the graph x. Since a path in an n vertex graph can be represented by at most $n \log n$ bits, for every x representing a graph of n vertices, finding $\max_{y \in \{0,1\}^{n \log n}} f(x,y)$ corresponds to finding the length of the maximum simple path in the graph corresponding to x, and finding the string y^* that achieves this maximum corresponds to actually finding the path.

Proof Idea:

The proof follows by generalizing our ideas from the longest path example above. Let f be as in the theorem statement. If $\mathbf{P} = \mathbf{NP}$ then for every for every string $x \in \{0,1\}^*$ and number k, we can test in in $poly(|x|, m)$ time whether there exists y such that $f(x,y) \geq k$, or in other words test whether $\max_{y \in \{0,1\}^m} f(x,y) \geq k$. If $f(x,y)$ is an integer between 0 and $poly(|x| + |y|)$ (as is the case in the example of longest path) then we can just try out all possiblities for k to find the maximum number k for which $\max_y f(x,y) \geq k$. Otherwise, we can use *binary search* to hone down on the right value. Once we do so, we can use search-to-decision to actually find the string y^* that achieves the maximum.

⋆

Proof of Theorem 15.2. For every f as in the theorem statement, we can define the Boolean function $F : \{0,1\}^* \to \{0,1\}$ as follows.

$$F(x, 1^m, k) = \begin{cases} 1 & \exists_{y \in \{0,1\}^m} f(x,y) \geq k \\ 0 & \text{otherwise} \end{cases} \tag{15.1}$$

Since f is computable in polynomial time, F is in **NP**, and so under our assumption that **P** $=$ **NP**, F itself can be computed in polynomial time. Now, for every x and m, we can compute the largest k such that $F(x, 1^m, k) = 1$ by a binary search. Specifically, we will do this as follows:

1. We maintain two numbers a, b such that we are guaranteed that $a \leq \max_{y \in \{0,1\}^m} f(x,y) < b$.
2. Initially we set $a = 0$ and $b = 2^{T(n)}$ where $T(n)$ is the running time of f. (A function with $T(n)$ running time can't output more than $T(n)$ bits and so can't output a number larger than $2^{T(n)}$.)
3. At each point in time, we compute the midpoint $c = \lfloor (a+b)/2 \rfloor$) and let $y = F(1^n, c)$.
   a. If $y = 1$ then we set $a = c$ and leave b as it is.
   b. If $y = 0$ then we set $b = c$ and leave a as it is.
4. We then go back to step 3, until $b \leq a + 1$.

Since $|b - a|$ shrinks by a factor of 2, within $\log_2 2^{T(n)} = T(n)$ steps, we will get to the point at which $b \leq a + 1$, and then we can simply output a. Once we find the maximum value of k such that $F(x, 1^m, k) = 1$, we can use the search to decision reduction of Theorem 15.1 to obtain the actual value $y^* \in \{0,1\}^m$ such that $f(x, y^*) = k$. ■

■ **Example 15.3 — Integer programming.** One application for Theorem 15.2 is in solving *optimization problems*. For example, the task of *linear programming* is to find $y \in \mathbb{R}^n$ that maximizes some linear objective $\sum_{i=0}^{n-1} c_i y_i$ subject to the constraint that y satisfies linear inequalities of the form $\sum_{i=0}^{n-1} a_i y_i \leq c$. As we discussed in Section 11.1.3, there is a known polynomial-time algorithm for linear programming. However, if we want to place additional constraints on y, such as requiring the coordinates of y to be *integer* or $0/1$ *valued* then the best-known algorithms run in exponential time in the worst case. However, if **P** $=$ **NP** then Theorem 15.2 tells us

that we would be able to solve all problems of this form in polynomial time. For every string x that describes a set of constraints and objective, we will define a function f such that if y satisfies the constraints of x then $f(x,y)$ is the value of the objective, and otherwise we set $f(x,y) = -M$ where M is some large number. We can then use Theorem 15.2 to compute the y that maximizes $f(x,y)$ and that will give us the assignment for the variables that satisfies our constraints and maximizes the objective. (If the computation results in y such that $f(x,y) = -M$ then we can double M and try again; if the true maximum objective is achieved by some string y^*, then eventually M will be large enough so that $-M$ would be smaller than the objective achieved by y^*, and hence when we run procedure of Theorem 15.2 we would get a value larger than $-M$.)

R

Remark 15.4 — Need for binary search.. In many examples, such as the case of finding longest path, we don't need to use the binary search step in Theorem 15.2, and can simply enumerate over all possible values for k until we find the correct one. One example where we do need to use this binary search step is in the case of the problem of finding a maximum length path in a *weighted* graph. This is the problem where G is a weighted graph, and every edge of G is given a weight which is a number between 0 and 2^k. Theorem 15.2 shows that we can find the maximum-weight simple path in G (i.e., simple path maximizing the sum of the weights of its edges) in time polynomial in the number of vertices and in k.

Beyond just this example there is a vast field of mathematical optimization that studies problems of the same form as in Theorem 15.2. In the context of optimization, x typically denotes a set of constraints over some variables (that can be Boolean, integer, or real valued), y encodes an assignment to these variables, and $f(x,y)$ is the value of some *objective function* that we want to maximize. Given that we don't know efficient algorithms for **NP** complete problems, researchers in optimization research study special cases of functions f (such as linear programming and semidefinite programming) where it *is* possible to optimize the value efficiently. Optimization is widely used in a great many scientific agreas including machine learning, engineering, economics and operations research.

15.2.1 Example: Supervised learning

One classical optimization task is *supervised learning*. In supervised learning we are given a list of *examples* $x_0, x_1, \ldots, x_{m-1}$ (where we can think of each x_i as a string in $\{0,1\}^n$ for some n) and the *labels* for them $y_0, \ldots, y_{n-1}$ (which we will think of simply bits, i.e., $y_i \in \{0,1\}$). For example, we can think of the x_i's as images of either dogs or cats, for which $y_i = 1$ in the former case and $y_i = 0$ in the latter case. Our goal is to come up with a *hypothesis* or *predictor* $h : \{0,1\}^n \to \{0,1\}$ such that if we are given a new example x that has an (unknown to us) label y, then with high probability h will *predict* the label. That is, with high probability it will hold that $h(x) = y$. The idea in supervised learning is to use the *Occam's Razor principle*: the simplest hypothesis that explains the data is likely to be correct. There are several ways to model this, but one popular approach is to pick some fairly simple function $H : \{0,1\}^{k+n} \to \{0,1\}$. We think of the first k inputs as the *parameters* and the last n inputs as the example data. (For example, we can think of the first k inputs of H as specifying the weights and connections for some neural network that will then be applied on the latter n inputs.) We can then phrase the supervised learning problem as finding, given a set of labeled examples $S = \{(x_0, y_0), \ldots, (x_{m-1}, y_{m-1})\}$, the set of parameters $\theta_0, \ldots, \theta_{k-1} \in \{0,1\}$ that minimizes the number of errors made by the predictor $x \mapsto H(\theta, x)$.[4]

In other words, we can define for every set S as above the function $F_S : \{0,1\}^k \to [m]$ such that $F_S(\theta) = \sum_{(x,y)\in S} |H(\theta, x) - y|$. Now, finding the value θ that minimizes $F_S(\theta)$ is equivalent to solving the supervised learning problem with respect to H. For every polynomial-time computable $H : \{0,1\}^{k+n} \to \{0,1\}$, the task of minimizing $F_S(\theta)$ can be "massaged" to fit the form of Theorem 15.2 and hence if $\mathbf{P} = \mathbf{NP}$, then we can solve the supervised learning problem in great generality. In fact, this observation extends to essentially any learning model, and allows for finding the optimal predictors given the minimum number of examples. (This is in contrast to many current learning algorithms, which often rely on having access to an extremely large number of examples— far beyond the minimum needed, and in particular far beyond the number of examples humans use for the same tasks.)

[4] This is often known as Empirical Risk Minimization.

15.2.2 Example: Breaking cryptosystems

We will discuss *cryptography* later in this course, but it turns out that if $\mathbf{P} = \mathbf{NP}$ then almost every cryptosystem can be efficiently broken. One approach is to treat finding an encryption key as an instance of a supervised learning problem. If there is an encryption scheme that maps a "plaintext" message p and a key θ to a "cipher-

text" c, then given examples of ciphertext/plaintext pairs of the form $(c_0, p_0), \ldots, (c_{m-1}, p_{m-1})$, our goal is to find the key θ such that $E(\theta, p_i) = c_i$ where E is the encryption algorithm. While you might think getting such "labeled examples" is unrealistic, it turns out (as many amateur homebrew crypto designers learn the hard way) that this is actually quite common in real-life scenarios, and that it is also possible to relax the assumption to having more minimal prior information about the plaintext (e.g., that it is English text). We defer a more formal treatment to Chapter 20.

15.3 FINDING MATHEMATICAL PROOFS

In the context of Gödel's Theorem, we discussed the notion of a *proof system* (see Section 10.1). Generally speaking, a *proof system* can be thought of as an algorithm $V : \{0,1\}^* \to \{0,1\}$ (known as the *verifier*) such that given a *statement* $x \in \{0,1\}^*$ and a *candidate proof* $w \in \{0,1\}^*$, $V(x,w) = 1$ if and only if w encodes a valid proof for the statement x. Any type of proof system that is used in mathematics for geometry, number theory, analysis, etc., is an instance of this form. In fact, standard mathematical proof systems have an even simpler form where the proof w encodes a *sequence* of lines $w^0, \ldots, w^m$ (each of which is itself a binary string) such that each line w^i is either an *axiom* or follows from some prior lines through an application of some *inference rule*. For example, Peano's axioms encode a set of axioms and rules for the natural numbers, and one can use them to formalize proofs in number theory. Also, there are some even stronger axiomatic systems, the most popular one being Zermelo–Fraenkel with the Axiom of Choice or ZFC for short. Thus, although mathematicians typically write their papers in natural language, proofs of number theorists can typically be translated to ZFC or similar systems, and so in particular the existence of an n-page proof for a statement x implies that there exists a string w of length $poly(n)$ (in fact often $O(n)$ or $O(n^2)$) that encodes the proof in such a system. Moreover, because verifying a proof simply involves going over each line and checking that it does indeed follow from the prior lines, it is fairly easy to do that in $O(|w|)$ or $O(|w|^2)$ (where as usual $|w|$ denotes the length of the proof w). This means that for every reasonable proof system V, the following function $SHORTPROOF_V : \{0,1\}^* \to \{0,1\}$ is in **NP**, where for every input of the form $x1^m$, $SHORTPROOF_V(x, 1^m) = 1$ if and only if there exists $w \in \{0,1\}^*$ with $|w| \leq m$ s.t. $V(xw) = 1$. That is, $SHORTPROOF_V(x, 1^m) = 1$ if there is a proof (in the system V) of length at most m bits that x is true. Thus, if **P** $=$ **NP**, then despite Gödel's Incompleteness Theorems, we can still automate mathematics in the sense of finding proofs that are not too long for every statement that has one. (Frankly speaking, if the shortest proof for some state-

ment requires a terabyte, then human mathematicians won't ever find this proof either.) For this reason, Gödel himself felt that the question of whether $SHORTPROOF_V$ has a polynomial time algorithm is of great interest. As Gödel wrote in a letter to John von Neumann in 1956 (before the concept of **NP** or even "polynomial time" was formally defined):

> One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n, allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\psi(F,n)$ be the number of steps the machine requires for this and let $\varphi(n) = \max_F \psi(F,n)$. The question is how fast $\varphi(n)$ grows for an optimal machine. One can show that $\varphi \geq k \cdot n$ [for some constant $k > 0$]. If there really were a machine with $\varphi(n) \sim k \cdot n$ (or even $\sim k \cdot n^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, [5] the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem.

For many reasonable proof systems (including the one that Gödel referred to), $SHORTPROOF_V$ is in fact **NP**-complete, and so Gödel can be thought of as the first person to formulate the **P** vs **NP** question. Unfortunately, the letter was only discovered in 1988.
[6]

[5] The undecidability of Entscheidungsproblem refers to the uncomputability of the function that maps a statement in first order logic to 1 if and only if that statement has a proof.

[6] TODO: Maybe add example on finding Nash equilibrium

15.4 QUANTIFIER ELIMINATION (ADVANCED)

If **P** = **NP** then we can solve all **NP** *search* and *optimization* problems in polynomial time. But can we do more? It turns out that the answer is that *Yes we can!*

An **NP** decision problem can be thought of as the task of deciding, given some string $x \in \{0,1\}^*$ the truth of a statement of the form

$$\exists_{y \in \{0,1\}^{p(|x|)}} V(xy) = 1 \tag{15.2}$$

for some polynomial-time algorithm V and polynomial $p : \mathbb{N} \to \mathbb{N}$. That is, we are trying to determine, given some string x, whether *there exists* a string y such that x and y satisfy some polynomial-time checkable condition V. For example, in the *independent set* problem, the string x represents a graph G and a number k, the string y represents some subset S of G's vertices, and the condition that we check is

whether $|S| \geq k$ and there is no edge $\{u, v\}$ in G such that both $u \in S$ and $v \in S$.

We can consider more general statements such as checking, given a string $x \in \{0,1\}^*$, the truth of a statement of the form

$$\exists_{y \in \{0,1\}^{p_0(|x|)}} \forall_{z \in \{0,1\}^{p_1(|x|)}} V(xyz) = 1 \;, \tag{15.3}$$

which in words corresponds to checking, given some string x, whether *there exists* a string y such that *for every* string z, the triple (x, y, z) satisfy some polynomial-time checkable condition. We can also consider more levels of quantifiers such as checking the truth of the statement

$$\exists_{y \in \{0,1\}^{p_0(|x|)}} \forall_{z \in \{0,1\}^{p_1(|x|)}} \exists_{w \in \{0,1\}^{p_2(|x|)}} V(xyzw) = 1 \tag{15.4}$$

and so on and so forth.

For example, given an n-input NAND-CIRC program P, we might want to find the *smallest* NAND-CIRC program P' that computes the same function as P. The question of whether there is such a P' that can be described by a string of at most s bits can be phrased as

$$\exists_{P' \in \{0,1\}^s} \forall_{x \in \{0,1\}^n} P(x) = P'(x) \tag{15.5}$$

which has the form (15.3).[7] Another example of a statement involving a levels of quantifiers would be to check, given a chess position x, whether there is a strategy that guarantees that White wins within a steps. For example is $a = 3$ we woud want to check if given the borad position x, *there exists* a move y for White such that *for every* move z for Black *there exists* a move w for White that ends in a a checkmate.

It turns out that if $\mathbf{P} = \mathbf{NP}$ then we can solve these kinds of problems as well:

Theorem 15.5 — Polynomial hierarchy collapse. If $\mathbf{P} = \mathbf{NP}$ then for every $a \in \mathbb{N}$, polynomial $p : \mathbb{N} \to \mathbb{N}$ and polynomial-time algorithm V, there is a polynomial-time algorithm $SOLVE_{V,a}$ that on input $x \in \{0,1\}^n$ returns 1 if and only if

$$\exists_{y_0 \in \{0,1\}^m} \forall_{y_1 \in \{0,1\}^m} \cdots \mathcal{Q}_{y_{a-1} \in \{0,1\}^m} V(xy_0y_1 \cdots y_{a-1}) = 1 \tag{15.6}$$

where $m = p(n)$ and $\mathcal{Q}$ is either $\exists$ or $\forall$ depending on whether a is odd or even, respectively. [8]

Proof Idea:

To understand the idea behind the proof, consider the special case where we want to decide, given $x \in \{0,1\}^n$, whether for every $y \in \{0,1\}^n$ there exists $z \in \{0,1\}^n$ such that $V(xyz) = 1$. Consider the function F such that $F(xy) = 1$ if there exists $z \in \{0,1\}^n$ such that $V(xyz) = 1$. Since V runs in polynomial-time $F \in \mathbf{NP}$ and hence if

[7] Since NAND-CIRC programs are equivalent to Boolean circuits, the search problem corresponding to (15.5) known as the circuit minimization problem and is widely studied in Engineering. You can skip ahead to Section 15.4.1 to see a particularly compelling application of this.

[8] For the ease of notation, we assume that all the strings we quantify over have the same length $m = p(n)$, but using simple padding one can show that this captures the general case of strings of different polynomial lengths.

P $=$ **NP**, then there is an algorithm V' that on input x, y outputs 1 if and only if there exists $z \in \{0,1\}^n$ such that $V(xyz) = 1$. Now we can see that the original statement we consider is true if and only if for every $y \in \{0,1\}^n$, $V'(xy) = 1$, which means it is false if and only if the following condition $(*)$ holds: there exists some $y \in \{0,1\}^n$ such that $V'(xy) = 0$. But for every $x \in \{0,1\}^n$, the question of whether the condition $(*)$ is itself in **NP** (as we assumed V' can be computed in polynomial time) and hence under the assumption that **P** $=$ **NP** we can determine in polynomial time whether the condition $(*)$, and hence our original statement, is true.

⋆

Proof of Theorem 15.5. We prove the theorem by induction. We assume that there is a polynomial-time algorithm $SOLVE_{V,a-1}$ that can solve the problem (15.6) for $a-1$ and use that to solve the problem for a. For $a = 1$, $SOLVE_{V,a-1}(x) = 1$ iff $V(x) = 1$ which is a polynomial-time computation since V runs in polynomial time. For every x, y_0, define the statement φ_{x,y_0} to be the following:

$$\varphi_{x,y_0} = \forall_{y_1 \in \{0,1\}^m} \exists_{y_2 \in \{0,1\}^m} \cdots Q_{y_{a-1} \in \{0,1\}^m} V(xy_0y_1 \cdots y_{a-1}) = 1 \quad (15.7)$$

By the definition of $SOLVE_{V,a}$, for every $x \in \{0,1\}^n$, our goal is that $SOLVE_{V,a}(x) = 1$ if and only if there exists $y_0 \in \{0,1\}^m$ such that φ_{x,y_0} is true.

The *negation* of φ_{x,y_0} is the statement

$$\overline{\varphi}_{x,y_0} = \exists_{y_1 \in \{0,1\}^m} \forall_{y_2 \in \{0,1\}^m} \cdots \overline{Q}_{y_{a-1} \in \{0,1\}^m} V(xy_0y_1 \cdots y_{a-1}) = 0 \quad (15.8)$$

where $\overline{Q}$ is $\exists$ if Q was $\forall$ and $\overline{Q}$ is $\forall$ otherwise. (Please stop and verify that you understand why this is true, this is a generalization of the fact that if Ψ is some logical condition then the negation of $\exists_y \forall_z \Psi(y,z)$ is $\forall_y \exists_z \neg\Psi(y,z)$.)

The crucial observation is that $\overline{\varphi}_{x,y_0}$ is exactly a statement of the form we consider with $a-1$ quantifiers instead of a, and hence by our inductive hypothesis there is some polynomial time algorithm $\overline{S}$ that on input xy_0 outputs 1 if and only if $\overline{\varphi}_{x,y_0}$ is true. If we let S be the algorithm that on input x, y_0 outputs $1 - \overline{S}(xy_0)$ then we see that S outputs 1 if and only if φ_{x,y_0} is true. Hence we can rephrase the original statement (15.6) as follows:

$$\exists_{y_0 \in \{0,1\}^m} S(xy_0) = 1 \quad (15.9)$$

but since S is a polynomial-time algorithm, Eq. (15.9) is clearly a statement in **NP** and hence under our assumption that **P** $=$ **NP** there is

a polynomial time algorithm that on input $x \in \{0,1\}^n$, will determine if (15.9) is true and so also if the original statement (15.6) is true.

■

The algorithm of Theorem 15.5 can also solve the search problem as well: find the value y_0 that certifies the truth of (15.6). We note that while this algorithm is in polynomial time, the exponent of this polynomial blows up quite fast. If the original NANDSAT algorithm required $\Omega(n^2)$ time, solving a levels of quantifiers would require time $\Omega(n^{2^a})$.[9]

[9] We do not know whether such loss is inherent. As far as we can tell, it's possible that the *quantified boolean formula* problem has a linear-time algorithm. We will, however, see later in this course that it satisfies a notion known as **PSPACE**-hardness that is even stronger than **NP**-hardness.

15.4.1 Application: self improving algorithm for $3SAT$

Suppose that we found a polynomial-time algorithm A for $3SAT$ that is "good but not great". For example, maybe our algorithm runs in time cn^2 for some not too small constant c. However, it's possible that the *best possible* SAT algorithm is actually much more efficient than that. Perhaps, as we guessed before, there is a circuit C^* of at most $10^6 n$ gates that computes 3SAT on n variables, and we simply haven't discovered it yet. We can use Theorem 15.5 to "bootstrap" our original "good but not great" 3SAT algorithm to discover the optimal one. The idea is that we can phrase the question of whether there exists a size s circuit that computes 3SAT for all length n inputs as follows: *there exists* a size $\leq s$ circuit C such that *for every* formula φ described by a string of length at most n, if $C(\varphi) = 1$ then *there exists* an assignment x to the variables of φ that satisfies it. One can see that this is a statement of the form (15.4) and hence if $\mathbf{P} = \mathbf{NP}$ we can solve it in polynomial time as well. We can therefore imagine investing huge computational resources in running A one time to discover the circuit C^* and then using C^* for all further computation.

15.5 APPROXIMATING COUNTING PROBLEMS (ADVANCED, OPTIONAL)

Given a NAND-CIRC program P, if $\mathbf{P} = \mathbf{NP}$ then we can find an input x (if one exists) such that $P(x) = 1$. But what if there is more than one x like that? Clearly we can't efficiently output all such x's; there might be exponentially many. But we can get an arbitrarily good multiplicative approximation (i.e., a $1 \pm \epsilon$ factor for arbitrarily small $\epsilon > 0$) for the number of such x's, as well as output a (nearly) uniform member of this set. We defer the details to later in this course, when we learn about *randomized computation*. However, we state (without proof) the following theorem for now:

Theorem 15.6 — Approximate counting if P = NP. Let $V : \{0,1\}^* \to \{0,1\}$ be some polynomial-time algorithm, and suppose that **P** = **NP**.

Then there exists an algorithm $COUNT_V$ that on input $x, 1^m, \epsilon$, runs in time polynomial in $|x|, m, 1/\epsilon$ and outputs a number $K \in \{0, \ldots, 2^m\}$ such that

$$(1-\epsilon)K \leq \left|\{y \in \{0,1\}^m \ : \ V(xy) = 1\}\right| \leq (1+\epsilon)K \qquad (15.10)$$

That is, K gives an approximation up to a factor of $1 \pm \epsilon$ for the number of *witnesses* for x with respect to the verifying algorithm V.

P

Once again, to understand this theorem it can be useful to see how it implies that if **P** $=$ **NP** then there is a polynomial-time algorithm that given a graph G and a number k, can compute a number K that is within a 1 ± 0.01 factor equal to the number of simple paths in G of length k. (That is, K is between 0.99 to 1.01 times the number of such paths.)

15.6 WHAT DOES ALL OF THIS IMPLY?

So, what will happen if we have a $10^6 n$ algorithm for $3SAT$? We have mentioned that **NP**-hard problems arise in many contexts, and indeed scientists, engineers, programmers and others routinely encounter such problems in their daily work. A better $3SAT$ algorithm will probably make their lives easier, but that is the wrong place to look for the most foundational consequences. Indeed, while the invention of electronic computers did of course make it easier to do calculations that people were already doing with mechanical devices and pen and paper, the main applications computers are used for today were not even imagined before their invention.

An exponentially faster algorithm for all **NP** problems would be no less radical an improvement (and indeed, in some sense would be more) than the computer itself, and it is as hard for us to imagine what it would imply as it was for Babbage to envision today's world. For starters, such an algorithm would completely change the way we program computers. Since we could automatically find the "best" (in any measure we chose) program that achieves a certain task, we would not need to define *how* to achieve a task, but only specify tests as to what would be a good solution, and could also ensure that a program satisfies an exponential number of tests without actually running them.

The possibility that **P** $=$ **NP** is often described as "automating creativity". There is something to that analogy, as we often think of a creative solution as one that is hard to discover but that, once the

"spark" hits, is easy to verify. But there is also an element of hubris to that statement, implying that the most impressive consequence of such an algorithmic breakthrough will be that computers would succeed in doing something that humans already do today. In fact, creativity already is to a large extent automated or minimized (e.g., just see how much popular media content is mass-produced), and as in most professions we should expect to see the need for humans diminish with time even if $\mathbf{P} \neq \mathbf{NP}$.

Nevertheless, artificial intelligence, like many other fields, will clearly be greatly impacted by an efficient 3SAT algorithm. For example, it is clearly much easier to find a better Chess-playing algorithm when, given any algorithm P, you can find the smallest algorithm P' that plays Chess better than P. Moreover, as we mentioned above, much of machine learning (and statistical reasoning in general) is about finding "simple" concepts that explain the observed data, and if $\mathbf{NP} = \mathbf{P}$, we could search for such concepts automatically for any notion of "simplicity" we see fit. In fact, we could even "skip the middle man" and do an automatic search for the learning algorithm with smallest generalization error. Ultimately the field of Artificial Intelligence is about trying to "shortcut" billions of years of evolution to obtain artificial programs that match (or beat) the performance of natural ones, and a fast algorithm for **NP** would provide the ultimate shortcut.[10]

More generally, a faster algorithm for **NP** problems would be immensely useful in any field where one is faced with computational or quantitative problems— which is basically all fields of science, math, and engineering. This will not only help with concrete problems such as designing a better bridge, or finding a better drug, but also with addressing basic mysteries such as trying to find scientific theories or "laws of nature". In a fascinating talk, physicist Nima Arkani-Hamed discusses the effort of finding scientific theories in much the same language as one would describe solving an **NP** problem, for which the solution is easy to verify or seems "inevitable", once found, but that requires searching through a huge landscape of possibilities to reach, and that often can get "stuck" at local optima:

> *"the laws of nature have this amazing feeling of inevitability... which is associated with local perfection."*

> *"The classical picture of the world is the top of a local mountain in the space of ideas. And you go up to the top and it looks amazing up there and absolutely incredible. And you learn that there is a taller mountain out there. Find it, Mount Quantum.... they're not smoothly connected ... you've got to make a jump to go from classical*

[10] One interesting theory is that $\mathbf{P} = \mathbf{NP}$ and evolution has already discovered this algorithm, which we are already using without realizing it. At the moment, there seems to be very little evidence for such a scenario. In fact, we have some partial results in the other direction showing that, regardless of whether $\mathbf{P} = \mathbf{NP}$, many types of "local search" or "evolutionary" algorithms require exponential time to solve 3SAT and other **NP**-hard problems.

> *to quantum ... This also tells you why we have such major challenges in trying to extend our understanding of physics. We don't have these knobs, and little wheels, and twiddles that we can turn. We have to learn how to make these jumps. And it is a tall order. And that's why things are difficult."*

Finding an efficient algorithm for **NP** amounts to always being able to search through an exponential space and find not just the "local" mountain, but the tallest peak.

But perhaps more than any computational speedups, a fast algorithm for **NP** problems would bring about a *new type of understanding*. In many of the areas where **NP**-completeness arises, it is not as much a barrier for solving computational problems as it is a barrier for obtaining "closed-form formulas" or other types of more constructive descriptions of the behavior of natural, biological, social and other systems. A better algorithm for **NP**, even if it is "merely" $2^{\sqrt{n}}$-time, seems to require obtaining a new way to understand these types of systems, whether it is characterizing Nash equilibria, spin-glass configurations, entangled quantum states, or any of the other questions where **NP** is currently a barrier for analytical understanding. Such new insights would be very fruitful regardless of their computational utility.

15.7 CAN P $\neq$ NP BE NEITHER TRUE NOR FALSE?

The Continuum Hypothesis is a conjecture made by Georg Cantor in 1878, positing the non-existence of a certain type of infinite cardinality.[11] This was considered one of the most important open problems in set theory, and settling its truth or falseness was the first problem put forward by Hilbert in the 1900 address we mentioned before. However, using the theories developed by Gödel and Turing, in 1963 Paul Cohen proved that both the Continuum Hypothesis and its negation are consistent with the standard axioms of set theory (i.e., the Zermelo-Fraenkel axioms + the Axiom of choice, or "ZFC" for short).[12]

Today, many (though not all) mathematicians interpret this result as saying that the Continuum Hypothesis is neither true nor false, but rather is an axiomatic choice that we are free to make one way or the other. Could the same hold for **P** $\neq$ **NP**?

In short, the answer is *No*. For example, suppose that we are trying to decide between the "3SAT is easy" conjecture (there is an $10^6 n$ time algorithm for 3SAT) and the "3SAT is hard" conjecture (for every n, any NAND-CIRC program that solves n variable 3SAT takes $2^{10^{-6}n}$ lines). Then, since for $n = 10^8$, $2^{10^{-6}n} > 10^6 n$, this boils down to the finite question of deciding whether or not there is a 10^{13}-line

[11] One way to phrase it is that for every infinite subset S of the real numbers $\mathbb{R}$, either there is a one-to-one and onto function $f : S \to \mathbb{R}$ or there is a one-to-one and onto function $f : S \to \mathbb{N}$.

[12] Formally, what he proved is that if ZFC is consistent, then so is ZFC when we assume either the continuum hypothesis or its negation.

NAND-CIRC program deciding 3SAT on formulas with 10^8 variables. If there is such a program then there is a finite proof of its existence, namely the approximately 1TB file describing the program, and for which the verification is the (finite in principle though infeasible in practice) process of checking that it succeeds on all inputs.[13] If there isn't such a program, then there is also a finite proof of that, though any such proof would take longer since we would need to enumerate over all *programs* as well. Ultimately, since it boils down to a finite statement about bits and numbers; either the statement or its negation must follow from the standard axioms of arithmetic in a finite number of arithmetic steps. Thus, we cannot justify our ignorance in distinguishing between the "3SAT easy" and "3SAT hard" cases by claiming that this might be an inherently ill-defined question. Similar reasoning (with different numbers) applies to other variants of the **P** vs **NP** question. We note that in the case that 3SAT is hard, it may well be that there is no *short* proof of this fact using the standard axioms, and this is a question that people have been studying in various restricted forms of proof systems.

[13] This inefficiency is not necessarily inherent. Later in this course we may discuss results in program-checking, interactive proofs, and average-case complexity, that can be used for efficient verification of proofs of related statements. In contrast, the inefficiency of verifying *failure* of all programs could well be inherent.

15.8 IS $\mathrm{P} = \mathrm{NP}$ "IN PRACTICE"?

The fact that a problem is **NP**-hard means that we believe there is no efficient algorithm that solve it in the *worst case*. It does not, however, mean that every single instance of the problem is hard. For example, if all the clauses in a 3SAT instance φ contain the same variable x_i (possibly in negated form), then by guessing a value to x_i we can reduce φ to a 2SAT instance which can then be efficiently solved. Generalizations of this simple idea are used in "SAT solvers", which are algorithms that have solved certain specific interesting SAT formulas with thousands of variables, despite the fact that we believe SAT to be exponentially hard in the worst case. Similarly, a lot of problems arising in economics and machine learning are **NP**-hard.[14] And yet vendors and customers manage to figure out market-clearing prices (as economists like to point out, there is milk on the shelves) and mice succeed in distinguishing cats from dogs. Hence people (and machines) seem to regularly succeed in solving interesting instances of **NP**-hard problems, typically by using some combination of guessing while making local improvements.

[14] Actually, the computational difficulty of problems in economics such as finding optimal (or any) equilibria is quite subtle. Some variants of such problems are **NP**-hard, while others have a certain "intermediate" complexity.

It is also true that there are many interesting instances of **NP**-hard problems that we do *not* currently know how to solve. Across all application areas, whether it is scientific computing, optimization, control or more, people often encounter hard instances of **NP** problems on which our current algorithms fail. In fact, as we will see, all of our digital security infrastructure relies on the fact that some concrete and

easy-to-generate instances of, say, 3SAT (or, equivalently, any other **NP**-hard problem) are exponentially hard to solve.

Thus it would be wrong to say that **NP** is easy "in practice", nor would it be correct to take **NP**-hardness as the "final word" on the complexity of a problem, particularly when we have more information about how any given instance is generated. Understanding both the "typical complexity" of **NP** problems, as well as the power and limitations of certain heuristics (such as various local-search based algorithms) is a very active area of research. We will see more on these topics later in this course.
15

[15] Talk more about coping with NP hardness. Main two approaches are *heuristics* such as SAT solvers that succeed on *some* instances, and *proxy measures* such as mathematical relaxations that instead of solving problem X (e.g., an integer program) solve program X' (e.g., a linear program) that is related to that. Maybe give compressed sensing as an example, and least square minimization as a proxy for maximum apostoriori probability.

15.9 WHAT IF $\mathbf{P \neq NP}$?

So, $\mathbf{P = NP}$ would give us all kinds of fantastical outcomes. But we strongly suspect that $\mathbf{P \neq NP}$, and moreover that there is no much-better-than-brute-force algorithm for 3SAT. If indeed that is the case, is it all bad news?

One might think that impossibility results, telling you that you *cannot* do something, is the kind of cloud that does not have a silver lining. But in fact, as we already alluded to before, it does. A hard (in a sufficiently strong sense) problem in **NP** can be used to create a code that *cannot be broken*, a task that for thousands of years has been the dream of not just spies but of many scientists and mathematicians over the generations. But the complexity viewpoint turned out to yield much more than simple codes, achieving tasks that people had previously not even dared to dream of. These include the notion of *public key cryptography*, allowing two people to communicate securely without ever having exchanged a secret key; *electronic cash*, allowing private and secure transaction without a central authority; and *secure multiparty computation*, enabling parties to compute a joint function on private inputs without revealing any extra information about it. Also, as we will see, computational hardness can be used to replace the role of *randomness* in many settings.

Furthermore, while it is often convenient to pretend that computational problems are simply handed to us, and that our job as computer scientists is to find the most efficient algorithm for them, this is not how things work in most computing applications. Typically even formulating the problem to solve is a highly non-trivial task. When we discover that the problem we want to solve is **NP**-hard, this might be a useful sign that we used the wrong formulation for it.

Beyond all these, the quest to understand computational hardness — including the discoveries of lower bounds for restricted computational models, as well as new types of reductions (such as those arising from "probabilistically checkable proofs") — has already had

surprising *positive* applications to problems in algorithm design, as well as in coding for both communication and storage. This is not surprising since, as we mentioned before, from group theory to the theory of relativity, the pursuit of impossibility results has often been one of the most fruitful enterprises of mankind.

✓ **Lecture Recap**

- The question of whether **P** $=$ **NP** is one of the most important and fascinating questions of computer science and science at large, touching on all fields of the natural and social sciences, as well as mathematics and engineering.
- Our current evidence and understanding supports the "SAT hard" scenario that there is no much-better-than-brute-force algorithm for 3SAT or many other **NP**-hard problems.
- We are very far from *proving* this, however. Researchers have studied proving lower bounds on the number of gates to compute explicit functions in *restricted forms* of circuits, and have made some advances in this effort, along the way generating mathematical tools that have found other uses. However, we have made essentially no headway in proving lower bounds for *general* models of computation such as NAND and NAND-TM programs. Indeed, we currently do not even know how to rule out the possibility that for every $n \in \mathbb{N}$, *SAT* restricted to n-length inputs has a NAND-CIRC program of $10n$ lines (even though there *exist* n-input functions that require $2^n/(10n)$ lines to compute).
- Understanding how to cope with this computational intractability, and even benefit from it, comprises much of the research in theoretical computer science.

15.10 EXERCISES

R

Remark 15.7 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

15.11 BIBLIOGRAPHICAL NOTES

16

[16] TODO: Scott's two surveys

15.12 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

- Polynomial hiearchy hardness for circuit minimization and related problems, see for example this paper.

15.13 ACKNOWLEDGEMENTS

16
Space bounded computation

PLAN: Example of space bounded algorithms, importance of preserving space. The classes L and PSPACE, space hierarchy theorem, PSPACE=NPSPACE, constant space = regular languages.

16.1 LECTURE SUMMARY

16.2 EXERCISES



Remark 16.1 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

16.3 BIBLIOGRAPHICAL NOTES

16.4 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

16.5 ACKNOWLEDGEMENTS

IV

RANDOMIZED COMPUTATION

Learning Objectives:

- Review the basic notion of probability theory that we will use.
- Sample spaces, and in particular the space $\{0,1\}^n$
- Events, probabilities of unions and intersections.
- Random variables and their expectation, variance, and standard deviation.
- Independence and correlation for both events and random variables.
- Markov, Chebyshev and Chernoff tail bounds (bounding the probability that a random variable will deviate from its expectation).

17
Probability Theory 101

> *"God doesn't play dice with the universe"*, Albert Einstein

> *"Einstein was doubly wrong ... not only does God definitely play dice, but He sometimes confuses us by throwing them where they can't be seen."*, Stephen Hawking

> *" 'The probability of winning a battle' has no place in our theory because it does not belong to any [random experiment]. Probability cannot be applied to this problem any more than the physical concept of work can be applied to the 'work' done by an actor reciting his part."*, Richard Von Mises, 1928 (paraphrased)

> *"I am unable to see why 'objectivity' requires us to interpret every probability as a frequency in some random experiment; particularly when in most problems probabilities are frequencies only in an imaginary universe invented just for the purpose of allowing a frequency interpretation."*, E.T. Jaynes, 1976

Before we show how to use randomness in algorithms, let us do a quick review of some basic notions in probability theory. This is not meant to replace a course on probability theory, and if you have not seen this material before, I highly recommend you look at additional resources to get up to speed.[1] Fortunately, we will not need many of the advanced notions of probability theory, but, as we will see, even the so-called "simple" setting of tossing n coins can lead to very subtle and interesting issues.

[1] Harvard's STAT 110 class (whose lectures are available on youtube) is a highly recommended introduction to probability. See also these lecture notes from MIT's "Mathematics for Computer Science" course.

17.1 RANDOM COINS

The nature of randomness and probability is a topic of great philosophical, scientific and mathematical depth. Is there actual random-

ness in the world, or does it proceed in a deterministic clockwork fashion from some initial conditions set at the beginning of time? Does probability refer to our uncertainty of beliefs, or to the frequency of occurrences in repeated experiments? How can we define probability over infinite sets?

These are all important questions that have been studied and debated by scientists, mathematicians, statisticians and philosophers. Fortunately, we will not need to deal directly with these questions here. We will be mostly interested in the setting of tossing n random, unbiased and independent coins. Below we define the basic probabilistic objects of *events* and *random variables* when restricted to this setting. These can be defined for much more general probabilistic experiments or *sample spaces*, and later on we will briefly discuss how this can be done. However, the n-coin case is sufficient for almost everything we'll need in this course.

If instead of "heads" and "tails" we encode the sides of each coin by "zero" and "one", we can encode the result of tossing n coins as a string in $\{0,1\}^n$. Each particular outcome $x \in \{0,1\}^n$ is obtained with probability 2^{-n}. For example, if we toss three coins, then we obtain each of the 8 outcomes $000, 001, 010, 011, 100, 101, 110, 111$ with probability $2^{-3} = 1/8$ (see also Fig. 17.1). We can describe the experiment of tossing n coins as choosing a string x uniformly at random from $\{0,1\}^n$, and hence we'll use the shorthand $x \sim \{0,1\}^n$ for x that is chosen according to this experiment.

An *event* is simply a subset A of $\{0,1\}^n$. The *probability of* A, denoted by $\Pr_{x\sim\{0,1\}^n}[A]$ (or $\Pr[A]$ for short, when the sample space is understood from the context), is the probability that an x chosen uniformly at random will be contained in A. Note that this is the same as $|A|/2^n$ (where $|A|$ as usual denotes the number of elements in the set A). For example, the probability that x has an even number of ones is $\Pr[A]$ where $A = \{x : \sum_{i=0}^{n-1} x_i = 0 \mod 2\}$. In the case $n = 3$, $A = \{000, 011, 101, 110\}$, and hence $\Pr[A] = \frac{4}{8} = \frac{1}{2}$. It turns out this is true for every n:

Lemma 17.1

$$\Pr_{x\sim\{0,1\}^n}[\sum_{i=0}^{n-1} x_i \text{ is even }] = 1/2 \tag{17.1}$$

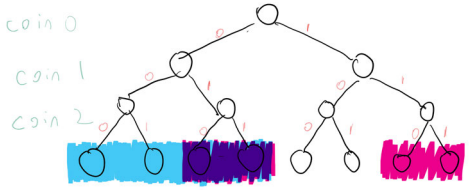


Figure 17.1: The probabilistic experiment of tossing three coins corresponds to making $2 \times 2 \times 2 = 8$ choices, each with equal probability. In this example, the blue set corresponds to the event $A = \{x \in \{0,1\}^3 \mid x_0 = 0\}$ where the first coin toss is equal to 0, and the pink set corresponds to the event $B = \{x \in \{0,1\}^3 \mid x_1 = 1\}$ where the second coin toss is equal to 1 (with their intersection having a purplish color). As we can see, each of these events contains 4 elements (out of 8 total) and so has probability $1/2$. The intersection of A and B contains two elements, and so the probability that both of these events occur is $2/8 = 1/4$.

P To test your intuition on probability, try to stop here and prove the lemma on your own.

Proof of Lemma 17.1. Let $A = \{x \in \{0,1\}^n : \sum_{i=0}^{n-1} x_i = 0 \mod 2\}$. Since every x is obtained with probability 2^{-n}, to show this we need to

show that $|A| = 2^n/2 = 2^{n-1}$. For every $x_0, \ldots, x_{n-2}$, if $\sum_{i=0}^{n-2} x_i$ is even then $(x_0, \ldots, x_{n-1}, 0) \in A$ and $(x_0, \ldots, x_{n-1}, 1) \notin A$. Similarly, if $\sum_{i=0}^{n-2} x_i$ is odd then $(x_0, \ldots, x_{n-1}, 1) \in A$ and $(x_0, \ldots, x_{n-1}, 0) \notin A$. Hence, for every one of the 2^{n-1} prefixes $(x_0, \ldots, x_{n-2})$, there is exactly a single continuation of $(x_0, \ldots, x_{n-2})$ that places it in A.

■

We can also use the *intersection* ($\cap$) and *union* ($\cup$) operators to talk about the probability of both event A *and* event B happening, or the probability of event A *or* event B happening. For example, the probability p that x has an *even* number of ones *and* $x_0 = 1$ is the same as $\Pr[A \cap B]$ where $A = \{x \in \{0,1\}^n : \sum_{i=0}^{n-1} x_i = 0 \mod 2\}$ and $B = \{x \in \{0,1\}^n : x_0 = 1\}$. This probability is equal to $1/4$ for $n > 1$. (It is a great exercise for you to pause here and verify that you understand why this is the case.)

Because intersection corresponds to considering the logical AND of the conditions that two events happen, while union corresponds to considering the logical OR, we will sometimes use the $\wedge$ and $\vee$ operators instead of $\cap$ and $\cup$, and so write this probability $p = \Pr[A \cap B]$ defined above also as

$$\Pr_{x \sim \{0,1\}^n} \left[\sum_i x_i = 0 \mod 2 \wedge x_0 = 1 \right] . \tag{17.2}$$

If $A \subseteq \{0,1\}^n$ is an event, then $\overline{A} = \{0,1\}^n \setminus A$ corresponds to the event that A does *not* happen. Since $|\overline{A}| = 2^n - |A|$, we get that

$$\Pr[\overline{A}] = \frac{|\overline{A}|}{2^n} = \frac{2^n - |A|}{2^n} = 1 - \frac{|A|}{2^n} = 1 - \Pr[A] \tag{17.3}$$

This makes sense: since A happens if and only if $\overline{A}$ does *not* happen, the probability of $\overline{A}$ should be one minus the probability of A.

R

Remark 17.2 — Remember the sample space. While the above definition might seem very simple and almost trivial, the human mind seems not to have evolved for probabilistic reasoning, and it is surprising how often people can get even the simplest settings of probability wrong. One way to make sure you don't get confused when trying to calculate probability statements is to always ask yourself the following two questions: **(1)** Do I understand what is the **sample space** that this probability is taken over?, and **(2)** Do I understand what is the definition of the **event** that we are analyzing?.

For example, suppose that I were to randomize seating in my course, and then it turned out that students sitting in row 7 performed better on the final: how surprising should we find this? If we started out with

the hypothesis that there is something special about the number 7 and chose it ahead of time, then the event that we are discussing is the event A that students sitting in number 7 had better performance on the final, and we might find it surprising. However, if we first looked at the results and then chose the row whose average performance is best, then the event we are discussing is the event B that there exists *some* row where the performance is higher than the overall average. B is a superset of A, and its probability (even if there is no correlation between sitting and performance) can be quite significant.

17.1.1 Random variables

Events correspond to Yes/No questions, but often we want to analyze finer questions. For example, if we make a bet at the roulette wheel, we don't want to just analyze whether we won or lost, but also *how much* we've gained. A (real valued) *random variable* is simply a way to associate a number with the result of a probabilistic experiment. Formally, a random variable is simply a function $X : \{0,1\}^n \to \mathbb{R}$ that maps every outcome $x \in \{0,1\}^n$ to a real number $X(x)$.[2] For example, the function $sum : \{0,1\}^n \to \mathbb{R}$ that maps x to the sum of its coordinates (i.e., to $\sum_{i=0}^{n-1} x_i$) is a random variable.

[2] In many probability texts a random variable is always defined to have values in the set $\mathbb{R}$ of real numbers, and this will be our default option as well. However, in some contexts in theoretical computer science we can consider random variables mapping to other sets such as $\{0,1\}^*$.

The *expectation* of a random variable X, denoted by $\mathbb{E}[X]$, is the average value that that this number takes, taken over all draws from the probabilistic experiment. In other words, the expectation of X is defined as follows:

$$\mathbb{E}[X] = \sum_{x \in \{0,1\}^n} 2^{-n} X(x) \; . \tag{17.4}$$

If X and Y are random variables, then we can define $X + Y$ as simply the random variable that maps a point $x \in \{0,1\}^n$ to $X(x) + Y(x)$. One basic and very useful property of the expectation is that it is *linear*:

Lemma 17.3 — Linearity of expectation.

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y] \tag{17.5}$$

Proof.

$$\begin{aligned} \mathbb{E}[X+Y] = \sum_{x \in \{0,1\}^n} 2^{-n} \left(X(x) + Y(x)\right) = \\ \sum_{x \in \{0,1\}^b} 2^{-n} X(x) + \sum_{x \in \{0,1\}^b} 2^{-n} Y(x) = \\ \mathbb{E}[X] + \mathbb{E}[Y] \end{aligned} \tag{17.6}$$

■

Similarly, $\mathbb{E}[kX] = k\,\mathbb{E}[X]$ for every $k \in \mathbb{R}$. For example, using the linearity of expectation, it is very easy to show that the expectation of the sum of the x_i's for $x \sim \{0,1\}^n$ is equal to $n/2$. Indeed, if we write $X = \sum_{i=0}^{n-1} x_i$ then $X = X_0 + \cdots + X_{n-1}$ where X_i is the random variable x_i. Since for every i, $\Pr[X_i = 0] = 1/2$ and $\Pr[X_i = 1] = 1/2$, we get that $\mathbb{E}[X_i] = (1/2)\cdot 0 + (1/2)\cdot 1 = 1/2$ and hence $\mathbb{E}[X] = \sum_{i=0}^{n-1} \mathbb{E}[X_i] = n \cdot (1/2) = n/2$.

P

If you have not seen discrete probability before, please go over this argument again until you are sure you follow it; it is a prototypical simple example of the type of reasoning we will employ again and again in this course.

If A is an event, then 1_A is the random variable such that $1_A(x)$ equals 1 if $x \in A$, and $1_A(x) = 0$ otherwise. Note that $\Pr[A] = \mathbb{E}[1_A]$ (can you see why?). Using this and the linearity of expectation, we can show one of the most useful bounds in probability theory:

Lemma 17.4 — Union bound. For every two events A, B, $\Pr[A \cup B] \leq \Pr[A] + \Pr[B]$

P

Before looking at the proof, try to see why the union bound makes intuitive sense. We can also prove it directly from the definition of probabilities and the cardinality of sets, together with the equation $|A \cup B| \leq |A| + |B|$. Can you see why the latter equation is true? (See also Fig. 17.2.)

Proof of Lemma 17.4. For every x, the variable $1_{A\cup B}(x) \leq 1_A(x) + 1_B(x)$. Hence, $\Pr[A \cup B] = \mathbb{E}[1_{A\cup B}] \leq \mathbb{E}[1_A + 1_B] = \mathbb{E}[1_A] + \mathbb{E}[1_B] = \Pr[A] + \Pr[B]$. ■

The way we often use this in theoretical computer science is to argue that, for example, if there is a list of 100 bad events that can happen, and each one of them happens with probability at most $1/10000$, then with probability at least $1 - 100/10000 = 0.99$, no bad event happens.

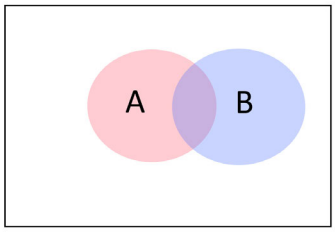


Figure 17.2: The *union bound* tells us that the probability of A or B happening is at most the sum of the individual probabilities. We can see it by noting that for every two sets $|A \cup B| \leq |A| + |B|$ (with equality only if A and B have no intersection).

17.1.2 Distributions over strings

While most of the time we think of random variables as having as output a *real number*, we sometimes consider random variables whose output is a *string*. That is, we can think of a map $Y : \{0,1\}^n \to \{0,1\}^*$ and consider the "random variable" Y such

that for every $y \in \{0,1\}^*$, the probability that Y outputs y is equal to $\frac{1}{2^n}|\{x \in \{0,1\}^n \mid Y(x) = y\}|$. To avoid confusion, we will typically refer to such string-valued random variables as *distributions* over strings. So, a *distribution* Y over strings $\{0,1\}^*$ can be thought of as a finite collection of strings $y_0, \ldots, y_{M-1} \in \{0,1\}^*$ and probabilities $p_0, \ldots, p_{M-1}$ (which are non-negative numbers summing up to one), so that $\Pr[Y = y_i] = p_i$.

Two distributions Y and Y' are *identical* if they assign the same probability to every string. For example, consider the following two functions $Y, Y' : \{0,1\}^2 \rightarrow \{0,1\}^2$. For every $x \in \{0,1\}^2$, we define $Y(x) = x$ and $Y'(x) = x_0(x_0 \oplus x_1)$ where $\oplus$ is the XOR operations. Although these are two different functions, they induce the same distribution over $\{0,1\}^2$ when invoked on a uniform input. The distribution $Y(x)$ for $x \sim \{0,1\}^2$ is of course the uniform distribution over $\{0,1\}^2$. On the other hand Y' is simply the map $00 \mapsto 00$, $01 \mapsto 01$, $10 \mapsto 11$, $11 \mapsto 10$ which is a permutation over the map $F : \{0,1\}^2 \rightarrow \{0,1\}^2$ defined as $F(x_0x_1) = x_0x_1$ and the map $G : \{0,1\}^2 \rightarrow \{0,1\}^2$ defined as $G(x_0x_1) = x_0(x_0 \oplus x_1)$

17.1.3 More general sample spaces.

While in this chapter we assume that the underlying probabilistic experiment corresponds to tossing n independent coins, everything we say easily generalizes to sampling x from a more general finite or countable set S (and not-so-easily generalizes to uncountable sets S as well). A *probability distribution* over a finite set S is simply a function $\mu : S \rightarrow [0,1]$ such that $\sum_{x \in S} \mu(s) = 1$. We think of this as the experiment where we obtain every $x \in S$ with probability $\mu(s)$, and sometimes denote this as $x \sim \mu$. An *event* A is a subset of S, and the probability of A, which we denote by $\Pr_\mu[A]$, is $\sum_{x \in A} \mu(x)$. A *random variable* is a function $X : S \rightarrow \mathbb{R}$, where the probability that $X = y$ is equal to $\sum_{x \in S \text{ s.t. } X(x)=y} \mu(x)$.

[3] TODO: add exercise on simulating die tosses and choosing a random number in $[m]$ by coin tosses

17.2 CORRELATIONS AND INDEPENDENCE

One of the most delicate but important concepts in probability is the notion of *independence* (and the opposing notion of *correlations*). Subtle correlations are often behind surprises and errors in probability and statistical analysis, and several mistaken predictions have been blamed on miscalculating the correlations between, say, housing prices in Florida and Arizona, or voter preferences in Ohio and Michigan. See also Joe Blitzstein's aptly named talk "Conditioning is the Soul of Statistics".[4]

Two events A and B are *independent* if the fact that A happens makes B neither more nor less likely to happen. For example, if we

[4] Another thorny issue is of course the difference between *correlation* and *causation*. Luckily, this is another point we don't need to worry about in our clean setting of tossing n coins.

think of the experiment of tossing 3 random coins $x \in \{0,1\}^3$, and we let A be the event that $x_0 = 1$ and B the event that $x_0 + x_1 + x_2 \geq 2$, then if A happens it is more likely that B happens, and hence these events are *not* independent. On the other hand, if we let C be the event that $x_1 = 1$, then because the second coin toss is not affected by the result of the first one, the events A and C are independent.

The formal definition is that events A and B are *independent* if $\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$. If $\Pr[A \cap B] > \Pr[A] \cdot \Pr[B]$ then we say that A and B are *positively correlated*, while if $\Pr[A \cap B] < \Pr[A] \cdot \Pr[B]$ then we say that A and B are *negatively correlated* (see Fig. 17.1).

If we consider the above examples on the experiment of choosing $x \in \{0,1\}^3$ then we can see that

$$\begin{aligned} \Pr[x_0 = 1] &= \tfrac{1}{2} \\ \Pr[x_0 + x_1 + x_2 \geq 2] = \Pr[\{011, 101, 110, 111\}] &= \tfrac{4}{8} = \tfrac{1}{2} \end{aligned} \tag{17.7}$$

but

$$\Pr[x_0 = 1 \ \wedge \ x_0 + x_1 + x_2 \geq 2] = \Pr[\{101, 110, 111\}] = \tfrac{3}{8} > \tfrac{1}{2} \cdot \tfrac{1}{2} \tag{17.8}$$

and hence, as we already observed, the events $\{x_0 = 1\}$ and $\{x_0 + x_1 + x_2 \geq 2\}$ are not independent and in fact are positively correlated. On the other hand, $\Pr[x_0 = 1 \wedge x_1 = 1] = \Pr[\{110, 111\}] = \frac{2}{8} = \frac{1}{2} \cdot \frac{1}{2}$ and hence the events $\{x_0 = 1\}$ and $\{x_1 = 1\}$ are indeed independent.

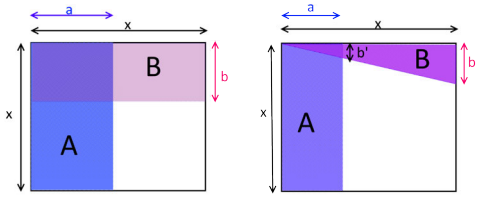


Figure 17.3: Two events A and B are *independent* if $\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$. In the two figures above, the empty $x \times x$ square is the sample space, and A and B are two events in this sample space. In the left figure, A and B are independent, while in the right figure they are negatively correlated, since B is less likely to occur if we condition on A (and vice versa). Mathematically, one can see this by noticing that in the left figure the areas of A and B respectively are $a \cdot x$ and $b \cdot x$, and so their probabilities are $\frac{a \cdot x}{x^2} = \frac{a}{x}$ and $\frac{b \cdot x}{x^2} = \frac{b}{x}$ respectively, while the area of $A \cap B$ is $a \cdot b$ which corresponds to the probability $\frac{a \cdot b}{x^2}$. In the right figure, the area of the triangle B is $\frac{b \cdot x}{2}$ which corresponds to a probability of $\frac{b}{2x}$, but the area of $A \cap B$ is $\frac{b' \cdot a}{2}$ for some $b' < b$. This means that the probability of $A \cap B$ is $\frac{b' \cdot a}{2x^2} < \frac{b}{2x} \cdot \frac{a}{x}$, or in other words $\Pr[A \cap B] < \Pr[A] \cdot \Pr[B]$.

R

Remark 17.5 — Disjointness vs independence. People sometimes confuse the notion of *disjointness* and *independence*, but these are actually quite different. Two events A and B are *disjoint* if $A \cap B = \emptyset$, which means that if A happens then B definitely does not happen. They are *independent* if $\Pr[A \cap B] = \Pr[A]\Pr[B]$ which means that knowing that A happens gives us no information about whether B happened or not. If A and B have nonzero probability, then being disjoint implies that they are *not* independent, since in particular it means that they are negatively correlated.

Conditional probability: If A and B are events, and A happens with nonzero probability then we define the probability that B happens *conditioned on* A to be $\Pr[B|A] = \Pr[A \cap B]/\Pr[A]$. This corresponds to calculating the probability that B happens if we already know that A happened. Note that A and B are independent if and only if $\Pr[B|A] = \Pr[B]$.

More than two events: We can generalize this definition to more than two events. We say that events $A_1, \ldots, A_k$ are *mutually independent* if knowing that any set of them occurred or didn't occur does not change the probability that an event outside the set occurs. Formally, the condition is that for every subset $I \subseteq [k]$,

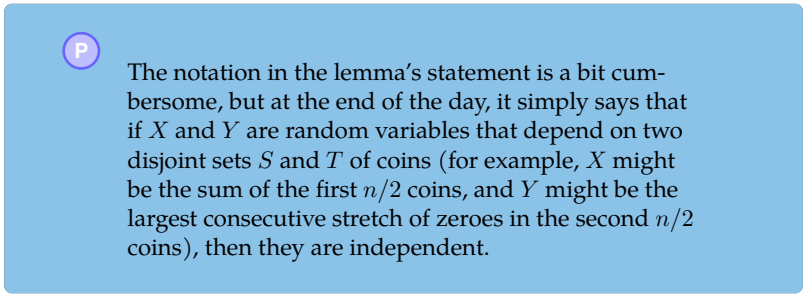
$$\Pr[\wedge_{i\in I} A_i] = \prod_{i\in I} \Pr[A_i]. \tag{17.9}$$

For example, if $x \sim \{0,1\}^3$, then the events $\{x_0 = 1\}$, $\{x_1 = 1\}$ and $\{x_2 = 1\}$ are mutually independent. On the other hand, the events $\{x_0 = 1\}$, $\{x_1 = 1\}$ and $\{x_0 + x_1 = 0 \mod 2\}$ are *not* mutually independent, even though every pair of these events is independent (can you see why? see also Fig. 17.4).

17.2.1 Independent random variables

We say that two random variables $X : \{0,1\}^n \to \mathbb{R}$ and $Y : \{0,1\}^n \to \mathbb{R}$ are independent if for every $u, v \in \mathbb{R}$, the events $\{X = u\}$ and $\{Y = v\}$ are independent.[5] In other words, X and Y are independent if $\Pr[X = u \wedge Y = v] = \Pr[X = u]\Pr[Y = v]$ for every $u, v \in \mathbb{R}$. For example, if two random variables depend on the result of tossing different coins then they are independent:

Lemma 17.6 Suppose that $S = \{s_0, \ldots, s_{k-1}\}$ and $T = \{t_0, \ldots, t_{m-1}\}$ are disjoint subsets of $\{0, \ldots, n-1\}$ and let $X, Y : \{0,1\}^n \to \mathbb{R}$ be random variables such that $X = F(x_{s_0}, \ldots, x_{s_{k-1}})$ and $Y = G(x_{t_0}, \ldots, x_{t_{m-1}})$ for some functions $F : \{0,1\}^k \to \mathbb{R}$ and $G : \{0,1\}^m \to \mathbb{R}$. Then X and Y are independent.



The notation in the lemma's statement is a bit cumbersome, but at the end of the day, it simply says that if X and Y are random variables that depend on two disjoint sets S and T of coins (for example, X might be the sum of the first $n/2$ coins, and Y might be the largest consecutive stretch of zeroes in the second $n/2$ coins), then they are independent.

Proof of Lemma 17.6. Let $a, b \in \mathbb{R}$, and let $A = \{x \in \{0,1\}^k : F(x) = a\}$ and $B = \{x \in \{0,1\}^m : F(x) = b\}$. Since S and T are disjoint, we can reorder the indices so that $S = \{0, \ldots, k-1\}$ and $T = \{k, \ldots, k+m-1\}$ without affecting any of the probabilities. Hence we can write $\Pr[X = a \wedge X = b] = |C|/2^n$ where $C = \{x_0, \ldots, x_{n-1} : (x_0, \ldots, x_{k-1}) \in A \wedge (x_k, \ldots, x_{k+m-1}) \in B\}$. Another way to write this using string concatenation is that $C = \{xyz : x \in A, y \in B, z \in \{0,1\}^{n-k-m}\}$, and hence $|C| = |A||B|2^{n-k-m}$, which means that

$$\frac{|C|}{2^n} = \frac{|A|}{2^k}\frac{|B|}{2^m}\frac{2^{n-k-m}}{2^{n-k-m}} = \Pr[X = a]\Pr[Y = b]. \tag{17.10}$$

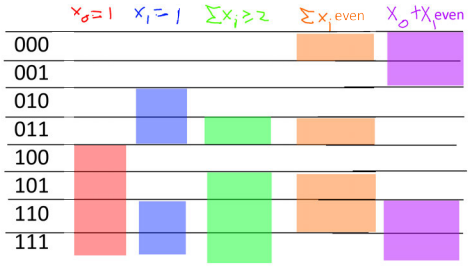


Figure 17.4: Consider the sample space $\{0,1\}^n$ and the events A, B, C, D, E corresponding to A: $x_0 = 1$, B: $x_1 = 1$, C: $x_0 + x_1 + x_2 \geq 2$, D: $x_0 + x_1 + x_2 = 0 mod 2$ and D: $x_0 + x_1 = 0 mod 2$. We can see that A and B are independent, C is positively correlated with A and positively correlated with B, the three events A, B, D are mutually independent, and while every pair out of A, B, E is independent, the three events A, B, E are not mutually independent since their intersection has probability $\frac{2}{8} = \frac{1}{4}$ instead of $\frac{1}{2}\cdot\frac{1}{2}\cdot\frac{1}{2} = \frac{1}{8}$.

[5] We use $\{X = u\}$ as shorthand for $\{x \mid X(x) = u\}$.

■

Note that if X and Y are independent random variables then (if we let S_X, S_Y denote all the numbers that have positive probability of being the output of X and Y, respectively) it holds that:

$$\mathbb{E}[XY] = \sum_{a\in S_X, b\in S_Y} \Pr[X=a \wedge Y=b]\cdot ab \stackrel{(1)}{=} \sum_{a\in S_X, b\in S_Y} \Pr[X=a]\Pr[Y=b]\cdot ab \stackrel{(2)}{=} \left(\sum_{a\in S_X} \Pr[X=a]\cdot a\right)\left(\sum_{b\in S_Y}\Pr[Y=b]b\right) \stackrel{(3)}{=} \mathbb{E}[X]\,\mathbb{E}[Y] \tag{17.11}$$

where the first equality ($=^{(1)}$) follows from the independence of X and Y, the second equality ($=^{(2)}$) follows by "opening the parentheses" of the righthand side, and the third inequality ($=^{(3)}$) follows from the definition of expectation. (This is not an "if and only if"; see Exercise 17.3.)

Another useful fact is that if X and Y are independent random variables, then so are $F(X)$ and $G(Y)$ for all functions $F, G : \mathbb{R} \to R$. This is intuitively true since learning $F(X)$ can only provide us with less information than does learning X itself. Hence, if learning X does not teach us anything about Y (and so also about $F(Y)$) then neither will learning $F(X)$. Indeed, to prove this we can write for every $a, b \in \mathbb{R}$:

$$\begin{aligned}\Pr[F(X)=a \wedge G(Y)=b] &= \sum_{x \text{ s.t.} F(x)=a, y \text{ s.t. } G(y)=b} \Pr[X=x\wedge Y=y] = \\ &\sum_{x \text{ s.t.} F(x)=a, y \text{ s.t. } G(y)=b} \Pr[X=x]\Pr[Y=y] = \\ &\left(\sum_{x \text{ s.t.} F(x)=a} \Pr[X=x]\right)\cdot\left(\sum_{y \text{ s.t.} G(y)=b}\Pr[Y=y]\right) = \\ &\Pr[F(X)=a]\Pr[G(Y)=b].\end{aligned} \tag{17.12}$$

17.2.2 Collections of independent random variables.

We can extend the notions of independence to more than two random variables: we say that the random variables $X_0, \ldots, X_{n-1}$ are *mutually independent* if for every $a_0, \ldots, a_{n-1} \in \mathbb{R}$,

$$\Pr\left[X_0 = a_0 \wedge \cdots \wedge X_{n-1} = a_{n-1}\right] = \Pr[X_0=a_0]\cdots\Pr[X_{n-1}=a_{n-1}]. \tag{17.13}$$

And similarly, we have that

Lemma 17.7 — Expectation of product of independent random variables. If $X_0, \ldots, X_{n-1}$ are mutually independent then

$$\mathbb{E}[\prod_{i=0}^{n-1} X_i] = \prod_{i=0}^{n-1} \mathbb{E}[X_i]. \tag{17.14}$$

Lemma 17.8 — Functions preserve independence. If $X_0, \ldots, X_{n-1}$ are mutually independent, and $Y_0, \ldots, Y_{n-1}$ are defined as $Y_i = F_i(X_i)$ for some functions $F_0, \ldots, F_{n-1} : \mathbb{R} \to \mathbb{R}$, then $Y_0, \ldots, Y_{n-1}$ are mutually independent as well.



We leave proving Lemma 17.7 and Lemma 17.8 as Exercise 17.4 Exercise 17.5. It is good idea for you stop now and do these exercises to make sure you are comfortable with the notion of independence, as we will use it heavily later on in this course.

17.3 CONCENTRATION

The name "expectation" is somewhat misleading. For example, suppose that you and I place a bet on the outcome of 10 coin tosses, where if they all come out to be 1's then I pay you 100,000 dollars and otherwise you pay me 10 dollars. If we let $X : \{0,1\}^{10} \to \mathbb{R}$ be the random variable denoting your gain, then we see that

$$\mathbb{E}[X] = 2^{-10} \cdot 100000 - (1 - 2^{-10})10 \sim 90. \tag{17.15}$$

But we don't really "expect" the result of this experiment to be for you to gain 90 dollars. Rather, 99.9% of the time you will pay me 10 dollars, and you will hit the jackpot 0.01% of the times.

However, if we repeat this experiment again and again (with fresh and hence *independent* coins), then in the long run we do expect your average earning to be 90 dollars, which is the reason why casinos can make money in a predictable way even though every individual bet is random. For example, if we toss n coins, then as n grows, the number of coins that come up ones will be more and more *concentrated* around $n/2$ according to the famous "bell curve" (see Fig. 17.5).

Much of probability theory is concerned with so called *concentration* or *tail* bounds, which are upper bounds on the probability that a random variable X deviates too much from its expectation. The first and simplest one of them is Markov's inequality:

Theorem 17.9 — Markov's inequality. If X is a non-negative random variable then $\Pr[X \geq k\,\mathbb{E}[X]] \leq 1/k$.

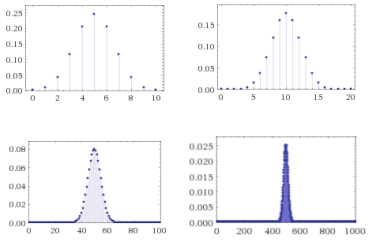


Figure 17.5: The probabilities that we obtain a particular sum when we toss $n = 10, 20, 100, 1000$ coins converge quickly to the Gaussian/normal distribution.

P

Markov's Inequality is actually a very natural statement (see also Fig. 17.6). For example, if you know that the average (not the median!) household income in the US is 70,000 dollars, then in particular you can deduce that at most 25 percent of households make more than 280,000 dollars, since otherwise, even if the remaining 75 percent had zero income, the top 25 percent alone would cause the average income to be larger than 70,000. From this example you can already see that in many situations, Markov's inequality will not be *tight* and the probability of deviating from expectation will be much smaller: see the Chebyshev and Chernoff inequalities below.

Proof of Theorem 17.9. Let $\mu = \mathbb{E}[X]$ and define $Y = 1_{X \geq k\mu}$. That is, $Y(x) = 1$ if $X(x) \geq k\mu$ and $Y(x) = 0$ otherwise. Note that by definition, for every x, $Y(x) \leq X/(k\mu)$. We need to show $\mathbb{E}[Y] \leq 1/k$. But this follows since $\mathbb{E}[Y] \leq \mathbb{E}[X/k(\mu)] = \mathbb{E}[X]/(k\mu) = \mu/(k\mu) = 1/k$. ■

Going beyond Markov's Inequality: Markov's inequality says that a (non-negative) random variable X can't go too crazy and be, say, a million times its expectation, with significant probability. But ideally we would like to say that with high probability, X should be very close to its expectation, e.g., in the range $[0.99\mu, 1.01\mu]$ where $\mu = \mathbb{E}[X]$. This is not generally true, but does turn out to hold when X is obtained by combining (e.g., adding) many independent random variables. This phenomenon, variants of which are known as "law of large numbers", "central limit theorem", "invariance principles" and "Chernoff bounds", is one of the most fundamental in probability and statistics, and is one that we heavily use in computer science as well.

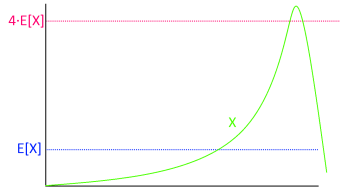


Figure 17.6: Markov's Inequality tells us that a non-negative random variable X cannot be much larger than its expectation, with high probability. For example, if the expectation of X is μ, then the probability that $X > 4\mu$ must be at most $1/4$, as otherwise just the contribution from this part of the sample space will be too large.

17.3.1 Chebyshev's Inequality

A standard way to measure the deviation of a random variable from its expectation is by using its *standard deviation*. For a random variable X, we define the *variance* of X as $\mathrm{Var}[X] = \mathbb{E}[(X - \mu)^2]$ where $\mu = \mathbb{E}[X]$; i.e., the variance is the average squared distance of X from its expectation. The *standard deviation* of X is defined as $\sigma[X] = \sqrt{\mathrm{Var}[X]}$. (This is well-defined since the variance, being an average of a square, is always a non-negative number.)

Using Chebyshev's inequality, we can control the probability that a random variable is too many standard deviations away from its expectation.

Theorem 17.10 — Chebyshev's inequality. Suppose that $\mu = \mathbb{E}[X]$ and $\sigma^2 = \mathrm{Var}[X]$. Then for every $k > 0$, $\Pr[|X - \mu| \geq k\sigma] \leq 1/k^2$.

Proof. The proof follows from Markov's inequality. We define the random variable $Y = (X - \mu)^2$. Then $\mathbb{E}[Y] = \mathrm{Var}[X] = \sigma^2$, and hence by Markov the probability that $Y > k^2\sigma^2$ is at most $1/k^2$. But clearly $(X - \mu)^2 \geq k^2\sigma^2$ if and only if $|X - \mu| \geq k\sigma$. ■

One example of how to use Chebyshev's inequality is the setting when $X = X_1 + \cdots + X_n$ where X_i's are *independent and identically distributed* (i.i.d for short) variables with values in $[0, 1]$ where each has expectation $1/2$. Since $\mathbb{E}[X] = \sum_i \mathbb{E}[X_i] = n/2$, we would like to say that X is very likely to be in, say, the interval $[0.499n, 0.501n]$. Using Markov's inequality directly will not help us, since it will only tell us that X is very likely to be at most $100n$ (which we already knew, since it always lies between 0 and n). However, since $X_1, \ldots, X_n$ are independent,

$$\mathrm{Var}[X_1 + \cdots + X_n] = \mathrm{Var}[X_1] + \cdots + \mathrm{Var}[X_n] \,. \tag{17.16}$$

(We leave showing this to the reader as Exercise 17.6.)

For every random variable X_i in $[0, 1]$, $\mathrm{Var}[X_i] \leq 1$ (if the variable is always in $[0, 1]$, it can't be more than 1 away from its expectation), and hence (17.16) implies that $\mathrm{Var}[X] \leq n$ and hence $\sigma[X] \leq \sqrt{n}$. For large n, $\sqrt{n} \ll 0.001n$, and in particular if $\sqrt{n} \leq 0.001n/k$, we can use Chebyshev's inequality to bound the probability that X is not in $[0.499n, 0.501n]$ by $1/k^2$.

17.3.2 The Chernoff bound

Chebyshev's inequality already shows a connection between independence and concentration, but in many cases we can hope for a quantitatively much stronger result. If, as in the example above, $X = X_1 + \ldots + X_n$ where the X_i's are bounded i.i.d random variables of mean $1/2$, then as n grows, the distribution of X would be roughly the *normal* or *Gaussian* distribution— that is, distributed according to the *bell curve* (see Fig. 17.5 and Fig. 17.7). This distribution has the property of being *very* concentrated in the sense that the probability of deviating k standard deviations from the mean is not merely $1/k^2$ as is guaranteed by Chebyshev, but rather is roughly e^{-k^2}.[6] That is, we have an *exponential decay* of the probability of deviation.

The following extremely useful theorem shows that such exponential decay occurs every time we have a sum of independent and bounded variables. This theorem is known under many names in different communities, though it is mostly called the Chernoff bound in the computer science literature:

[6] Specifically, for a normal random variable X of expectation μ and standard deviation σ, the probability that $|X - \mu| \geq k\sigma$ is at most $2e^{-k^2/2}$.

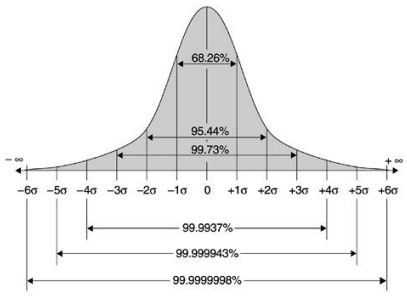


Figure 17.7: In the *normal distribution* or the Bell curve, the probability of deviating k standard deviations from the expectation shrinks *exponentially* in k^2, and specifically with probability at least $1 - 2e^{-k^2/2}$, a random variable X of expectation μ and standard deviation σ satisfies $\mu - k\sigma \leq X \leq \mu + k\sigma$. This figure gives more precise bounds for $k = 1, 2, 3, 4, 5, 6$. (Image credit:Imran Baghirov)

Theorem 17.11 — Chernoff/Hoeffding bound. If $X_1, \ldots, X_n$ are i.i.d random variables such that $X_i \in [0,1]$ and $\mathbb{E}[X_i] = p$ for every i, then for every $\epsilon > 0$

$$\Pr[\left|\sum_{i=0}^{n-1} X_i - pn\right| > \epsilon n] \leq 2 \cdot e^{-2\epsilon^2 n}. \tag{17.17}$$

We omit the proof, which appears in many texts, and uses Markov's inequality on i.i.d random variables $Y_0, \ldots, Y_n$ that are of the form $Y_i = e^{\lambda X_i}$ for some carefully chosen parameter λ. See Exercise 17.9 for a proof of the simple (but highly useful and representative) case where each X_i is $\{0,1\}$ valued and $p = 1/2$. (See also Exercise 17.10 for a generalization.)

[7]

[7] TODO: maybe add an example application of Chernoff. Perhaps a probabilistic method proof using Chernoff+Union bound.

17.4 LECTURE SUMMARY

- A basic probabilistic experiment corresponds to tossing n coins or choosing x uniformly at random from $\{0,1\}^n$.
- *Random variables* assign a real number to every result of a coin toss. The *expectation* of a random variable X is its average value, and there are several *concentration* results showing that under certain conditions, random variables deviate significantly from their expectation only with small probability.

17.5 EXERCISES

R

Remark 17.12 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 17.1 Suppose that we toss three independent fair coins $a, b, c \in \{0,1\}$. What is the probability that the XOR of a,b, and c is equal to 1? What is the probability that the AND of these three values is equal to 1? Are these two events independent?

■

Exercise 17.2 Give an example of random variables $X, Y : \{0,1\}^3 \to \mathbb{R}$ such that $\mathbb{E}[XY] \neq \mathbb{E}[X]\,\mathbb{E}[Y]$.

■

Exercise 17.3 Give an example of random variables $X, Y : \{0,1\}^3 \to \mathbb{R}$ such that X and Y are *not* independent but $\mathbb{E}[XY] = \mathbb{E}[X]\,\mathbb{E}[Y]$.

■

Exercise 17.4 — Product of expectations. Prove Lemma 17.7

■

Exercise 17.5 — Transformations preserve independence. Prove Lemma 17.8

■

Exercise 17.6 — Variance of independent random variables. Prove that if $X_0, \ldots, X_{n-1}$ are independent random variables then $\mathrm{Var}[X_0 + \cdots + X_{n-1}] = \sum_{i=0}^{n-1} \mathrm{Var}[X_i]$.

■

Exercise 17.7 — Entropy (challenge). Recall the definition of a distribution μ over some finite set S. Shannon defined the *entropy* of a distribution μ, denoted by $H(\mu)$, to be $\sum_{x \in S} \mu(x) \log(1/\mu(x))$. The idea is that if μ is a distribution of entropy k, then encoding members of μ will require k bits, in an amortized sense. In this exercise we justify this definition. Let μ be such that $H(\mu) = k$.

1. Prove that for every one to one function $F : S \to \{0,1\}^*$, $\mathbb{E}_{x \sim \mu} |F(x)| \geq k$.
2. Prove that for every ϵ, there is some n and a one-to-one function $F : S^n \to \{0,1\}^*$, such that $\mathbb{E}_{x \sim \mu^n} |F(x)| \leq n(k+\epsilon)$, where $x \sim \mu$ denotes the experiments of choosing $x_0, \ldots, x_{n-1}$ each independently from S using the distribution μ.

■

Exercise 17.8 — Entropy approximation to binomial. Let $H(p) = p \log(1/p) + (1-p)\log(1/(1-p))$.[8] Prove that for every $p \in (0,1)$ and $\epsilon > 0$, if n is large enough then[9]

$$2^{(H(p)-\epsilon)n} \binom{n}{pn} \leq 2^{(H(p)+\epsilon)n} \tag{17.18}$$

where $\binom{n}{k}$ is the binomial coefficient $\frac{n!}{k!(n-k)!}$ which is equal to the number of k-size subsets of $\{0, \ldots, n-1\}$.

■

Exercise 17.9 — Chernoff using Stirling.

1. Prove that $\Pr_{x \sim \{0,1\}^n}[\sum x_i = k] = \binom{n}{k} 2^{-n}$.
2. Use this and Exercise 17.8 to prove the Chernoff bound for the case that $X_0, \ldots, X_n$ are i.i.d. random variables over $\{0,1\}$ each equaling 0 and 1 with probability $1/2$.

■

Exercise 17.10 — Poor man's Chernoff. Let $X_0, \ldots, X_n$ be i.i.d random variables with $\mathbb{E}\, X_i = p$ and $\Pr[0 \leq X_i \leq 1] = 1$. Define $Y_i = X_i - p$.

[8] While you don't need this to solve this exercise, this is the function that maps p to the entropy (as defined in Exercise 17.7) of the p-biased coin distribution over $\{0,1\}$, which is the function $\mu : \{0,1\} \to [0,1]$ s.y. $\mu(0) = 1-p$ and $\mu(1) = p$.

[9] **Hint:** Use Stirling's formula for approximating the factorial function.

1. Prove that for every $j_1, \ldots, j_n \in \mathbb{N}$, if there exists one i such that j_i is odd then $\mathbb{E}[\prod_{i=0}^{n-1} Y_i^{j_i}] = 0$.
2. Prove that for every k, $\mathbb{E}[(\sum_{i=0}^{n-1} Y_i)^k] \leq (10kn)^{k/2}$.[10]
3. Prove that for every $\epsilon > 0$, $\Pr[|\sum_i Y_i| \geq \epsilon n] \geq 2^{-\epsilon^2 n/(10000 \log 1/\epsilon)}$.[11]

■

Exercise 17.11 — Simulating distributions using coins. Our model for probability involves tossing n coins, but sometimes algorithms require sampling from other distributions, such as selecting a uniform number in $\{0, \ldots, M-1\}$ for some M. Fortunately, we can simulate this with an exponentially small probability of error: prove that for every M, if $n > k\lceil \log M \rceil$, then there is a function $F : \{0,1\}^n \to \{0, \ldots, M-1\} \cup \{\perp\}$ such that **(1)** The probability that $F(x) = \perp$ is at most 2^{-k} and **(2)** the distribution of $F(x)$ conditioned on $F(x) \neq \perp$ is equal to the uniform distribution over $\{0, \ldots, M-1\}$.[12]

■

Exercise 17.12 — Sampling. Suppose that a country has 300,000,000 citizens, 52 percent of which prefer the color "green" and 48 percent of which prefer the color "orange". Suppose we sample n random citizens and ask them their favorite color (assume they will answer truthfully). What is the smallest value n among the following choices so that the probability that the majority of the sample answers "green" is at most 0.05?

a. 1,000

b. 10,000

c. 100,000

d. 1,000,000

■

Exercise 17.13 Would the answer to Exercise 17.12 change if the country had 300,000,000,000 citizens?

■

Exercise 17.14 — Sampling (2). Under the same assumptions as Exercise 17.12, what is the smallest value n among the following choices so that the probability that the majority of the sample answers "green" is at most 2^{-100}?

a. 1,000

b. 10,000

c. 100,000

d. 1,000,000

[10] **Hint:** Bound the number of tuples $j_0, \ldots, j_{n-1}$ such that every j_i is even and $\sum j_i = k$.

[11] **Hint:** Set $k = 2\lceil \epsilon^2 n/1000 \rceil$ and then show that if the event $|\sum Y_i| \geq \epsilon n$ happens then the random variable $(\sum Y_i)^k$ is a factor of ϵ^{-k} larger than its expectation.

[12] **Hint:** Think of $x \in \{0,1\}^n$ as choosing k numbers $y_1, \ldots, y_k \in \{0, \ldots, 2^{\lceil \log M \rceil} - 1\}$. Output the first such number that is in $\{0, \ldots, M-1\}$.

e. It is impossible to get such low probability since there are fewer than 2^{100} citizens.

■

13

[13] TODO: add some exercise about the probabilistic method

17.6 BIBLIOGRAPHICAL NOTES

Learning Objectives:

- See examples of randomized algorithms
- Get more comfort with analyzing probabilistic processes and tail bounds
- Success amplification using tail bounds

18
Probabilistic computation

> *"in 1946 .. (I asked myself) what are the chances that a Canfield solitaire laid out with 52 cards will come out successfully? After spending a lot of time trying to estimate them by pure combinatorial calculations, I wondered whether a more practical method ... might not be to lay it our say one hundred times and simple observe and count"*, Stanislaw Ulam, 1983

> *"The salient features of our method are that it is probabilistic ... and with a controllable miniscule probability of error."*, Michael Rabin, 1977

In early computer systems, much effort was taken to drive *out* randomness and noise. Hardware components were prone to non-deterministic behavior from a number of causes, whether it was vacuum tubes overheating or actual physical bugs causing short circuits (see Fig. 18.1). This motivated John von Neumann, one of the early computing pioneers, to write a paper on how to *error correct* computation, introducing the notion of *redundancy*.

So it is quite surprising that randomness turned out not just a hindrance but also a *resource* for computation, enabling us to achieve tasks much more efficiently than previously known. One of the first applications involved the very same John von Neumann. While he was sick in bed and playing cards, Stan Ulam came up with the observation that calculating statistics of a system could be done much faster by running several randomized simulations. He mentioned this idea to von Neumann, who became very excited about it; indeed, it turned out to be crucial for the neutron transport calculations that were needed for development of the Atom bomb and later on the hydrogen bomb. Because this project was highly classified, Ulam, von Neumann and their collaborators came up with the codeword "Monte Carlo" for this approach (based on the famous casinos where Ulam's

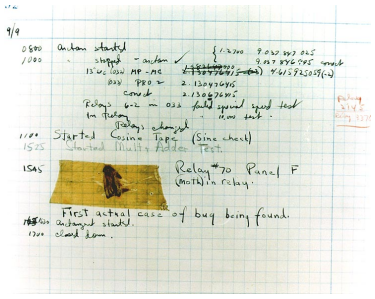


Figure 18.1: A 1947 entry in the log book of the Harvard MARK II computer containing an actual bug that caused a hardware malfunction. By Courtesy of the Naval Surface Warfare Center.

uncle gambled). The name stuck, and randomized algorithms are known as Monte Carlo algorithms to this day.[1]

In this chapter, we will see some examples of randomized algorithms that use randomness to compute a quantity in a faster or simpler way than was known otherwise. We will describe the algorithms in an informal / "pseudo-code" way, rather than as NAND or NAND-TM programs. In Chapter 19 we will discuss how to augment the NAND and NAND-TM models to incorporate the ability to "toss coins".

[1] Some texts also talk about "Las Vegas algorithms" that always return the right answer but whose running time is only polynomial on the average. Since this Monte Carlo vs Las Vegas terminology is confusing, we will not use these terms anymore, and simply talk about randomized algorithms.

18.1 FINDING APPROXIMATELY GOOD MAXIMUM CUTS.

We start with the following example. Recall the *maximum cut problem* of finding, given a graph $G = (V, E)$, the cut that maximizes the number of edges. This problem is **NP**-hard, which means that we do not know of any efficient algorithm that can solve it, but randomization enables a simple algorithm that can cut at least half of the edges:

Theorem 18.1 — Approximating max cut. There is an efficient probabilistic algorithm that on input an n-vertex m-edge graph G, outputs a cut $(S, \overline{S})$ that cuts at least $m/2$ of the edges of G in expectation.

Proof Idea:

We simply choose a *random cut*: we choose a subset S of vertices by choosing every vertex v to be a member of S with probability $1/2$ independently. It's not hard to see that each edge is cut with probability $1/2$ and so the expected number of cut edges is $m/2$.

⋆

Proof of Theorem 18.1. The algorithm is extremely simple:

Algorithm Random Cut:

Input: Graph $G = (V, E)$ with n vertices and m edges. Denote $V = \{v_0, v_1, \ldots, v_{n-1}\}$.

Operation:

1. Pick x uniformly at random in $\{0,1\}^n$.
2. Let $S \subseteq V$ be the set $\{v_i : x_i = 1 , i \in [n]\}$ that includes all vertices corresponding to coordinates of x where $x_i = 1$.
3. Output the cut $(S, \overline{S})$.

We claim that the expected number of edges cut by the algorithm is $m/2$. Indeed, for every edge $e \in E$, let X_e be the random variable such

that $X_e(x) = 1$ if the edge e is cut by x, and $X_e(x) = 0$ otherwise. For every such edge $e = \{i, j\}$, $X_e(x) = 1$ if and only if $x_i \neq x_j$. Since the pair (x_i, x_j) obtains each of the values $00, 01, 10, 11$ with probability $1/4$, the probability that $x_i \neq x_j$ is $1/2$ and hence $\mathbb{E}[X_e] = 1/2$. If we let X be the random variable corresponding to the total number of edges cut by S, then $X = \sum_{e \in E} X_e$ and hence by linearity of expectation

$$\mathbb{E}[X] = \sum_{e \in E} \mathbb{E}[X_e] = m(1/2) = m/2 \; . \tag{18.1}$$

■

18.1.1 Amplification

Theorem 18.1 gives us an algorithm that cuts $m/2$ edges in *expectation*. But, as we saw before, expectation does not immediately imply concentration, and so a priori, it may be the case that when we run the algorithm, most of the time we don't get a cut matching the expectation. Luckily, we can *amplify* the probability of success by repeating the process several times and outputting the best cut we find. We start by arguing that the probability the algorithm above succeeds in cutting at least $m/2$ edges is not *too* tiny.

Lemma 18.2 The probability that a random cut in an m edge graph cuts at least $m/2$ edges is at least $1/(2m)$.

Proof Idea:

To see the idea behind the proof, think of the case that $m = 1000$. In this case one can show that we will cut at least 500 edges with probability at least 0.001 (and so in particular larger than $1/(2m) = 1/2000$). Specifically, if we assume otherwise, then this means that with probability more than 0.999 the algorithm cuts 499 or fewer edges. But since we can never cut more than the total of 1000 edges, given this assumption, the highest value the expected number of edges cut is if we cut exactly 499 edges with probability 0.999 and cut 1000 edges with probability 0.001. Yet even in this case the expected number of edges will be $0.999 \cdot 499 + 0.001 \cdot 1000 < 500$, which contradicts the fact that we've calculated the expectation to be at least 500 in Theorem 18.1.

⋆

Proof of Lemma 18.2. Let p be the probability that we cut at least $m/2$ edges and suppose, towards a contradiction, that $p < 1/(2m)$. Since the number of edges cut is an integer, and $m/2$ is a multiple of 0.5, by definition of p, with probability $1 - p$ we cut at most $m/2 - 0.5$ edges. Moreover, since we can never cut more than m edges, under

our assumption that $p < m/2$, we can bound the expected number of edges cut by

$$pm + (1-p)(m/2 - 0.5) \leq pm + m/2 - 0.5 \tag{18.2}$$

But if $p < 1/(2m)$ then $pm < 0.5$ and so the righthand side is smaller than $m/2$, which contradicts the fact that (as proven in Theorem 18.1) the expected number of edges cut is at least $m/2$.

■

Success amplification. Lemma 18.2 shows that our algorithm succeeds at least *some* of the time, but we'd like to succeed almost *all* of the time. The approach to do that is to simply *repeat* our algorithm many times, with fresh randomness each time, and output the best cut we get in one of these repetitions. It turns out that with extremely high probability we will get a cut of size at least $m/2$. For example, if we repeat this experiment $2000m$ times, then (using the inequality $(1 - 1/k)^k \leq 1/e \leq 1/2$) we can show that the probability that we will never cut at least $m/2$ edges is at most

$$(1 - 1/(2m))^{2000m} \leq 2^{-1000} . \tag{18.3}$$

More generally, the same calculations can be used to show the following lemma:

Lemma 18.3 There is a algorithm that on input a graph $G = (V, E)$ and a number k, runs in time polynomial in $|V|$ and k and outputs a cut $(S, \overline{S})$ such that

$$\Pr[\text{number of edges cut by } (S, \overline{S}) \geq |E|/2] \geq 1 - 2^{-k} . \tag{18.4}$$

Proof of Lemma 18.3. The algorithm will work as follows:

Algorithm AMPLIFY RANDOM CUT:

Input: Graph $G = (V, E)$ with n vertices and m edges. Denote $V = \{v_0, v_1, \ldots, v_{n-1}\}$. Number $k > 0$.

Operation:

1. Repeat the following $200km$ times:
   a. Pick x uniformly at random in $\{0,1\}^n$.
   b. Let $S \subseteq V$ be the set $\{v_i : x_i = 1 , i \in [n]\}$ that includes all vertices corresponding to coordinates of x where $x_i = 1$.
   c. If $(S, \overline{S})$ cuts at least $m/2$ then halt and output $(S, \overline{S})$.
2. Output "failed"

We leave completing the analysis as an exercise to the reader (see Exercise 18.1).

■

18.1.2 Two-sided amplification

The analysis above relied on the fact that the maximum cut has *one sided error*. By this we mean that if we get a cut of size at least $m/2$ then we know we have succeeded. This is common for randomized algorithms, but is not the only case. In particular, consider the task of computing some Boolean function $F : \{0,1\}^* \to \{0,1\}$. A randomized algorithm A for computing F, given input x, might toss coins and succeed in outputting $F(x)$ with probability, say, 0.9. We say that A has *two sided errors* if there is positive probability that $A(x)$ outputs 1 when $F(x) = 0$, and positive probability that $A(x)$ outputs 0 when $F(x) = 1$. In such a case, to simplify A's success, we cannot simply repeat it k times and output 1 if a single one of those repetitions resulted in 1, nor can we output 0 if a single one of the repetitions resulted in 0. But we can output the *majority value* of these repetitions. By the Chernoff bound (Theorem 17.11), with probability *exponentially close* to 1 (i.e., $1 - 2^{\Omega(k)}$), the fraction of the repetitions where A will output $F(x)$ will be at least, say 0.89, and in such cases we will of course output the correct answer.

The above translates into the following theorem

Theorem 18.4 — Two-sided amplification. If $F : \{0,1\}^* \to \{0,1\}$ is a function such that there is a polynomial-time algorithm A satisfying

$$\Pr[A(x) = F(x)] \geq 0.51 \tag{18.5}$$

for every $x \in \{0,1\}^*$, then there is a polynomial time algorithm B satisfying

$$\Pr[B(x) = F(x)] \geq 1 - 2^{-|x|} \tag{18.6}$$

for every $x \in \{0,1\}^*$.

We omit the proof of Theorem 18.4, since we will prove a more general result later on in Theorem 19.4.

18.1.3 What does this mean?

We have shown a probabilistic algorithm that on any m edge graph G, will output a cut of at least $m/2$ edges with probability at least $1 - 2^{-1000}$. Does it mean that we can consider this problem as "easy"? Should we be somewhat wary of using a probabilistic algorithm, since it can sometimes fail?

First of all, it is important to emphasize that this is still a *worst case* guarantee. That is, we are not assuming anything about the *input*

graph: the probability is only due to the *internal randomness of the algorithm*. While a probabilistic algorithm might not seem as nice as a deterministic algorithm that is *guaranteed* to give an output, to get a sense of what a failure probability of 2^{-1000} means, note that:

- The chance of winning the Massachusetts Mega Million lottery is one over $(75)^5 \cdot 15$ which is roughly 2^{-35}. So 2^{-1000} corresponds to winning the lottery about 300 times in a row, at which point you might not care so much about your algorithm failing.
- The chance for a U.S. resident to be struck by lightning is about $1/700000$, which corresponds to about 2^{-45} chance that you'll be struck by lightning the very second that you're reading this sentence (after which again you might not care so much about the algorithm's performance).
- Since the earth is about 5 billion years old, we can estimate the chance that an asteroid of the magnitude that caused the dinosaurs' extinction will hit us this very second to be about 2^{-60}. It is quite likely that even a deterministic algorithm will fail if this happens.

So, in practical terms, a probabilistic algorithm is just as good as a deterministic one. But it is still a theoretically fascinating question whether randomized algorithms actually yield more power, or whether is it the case that for any computational problem that can be solved by probabilistic algorithm, there is a deterministic algorithm with nearly the same performance.[2] For example, we will see in Exercise 18.2 that there is in fact a deterministic algorithm that can cut at least $m/2$ edges in an m-edge graph. We will discuss this question in generality in Chapter 19. For now, let us see a couple of examples where randomization leads to algorithms that are better in some sense than the known deterministic algorithms.

[2] This question does have some significance to practice, since hardware that generates high quality randomness at speed is nontrivial to construct.

18.1.4 Solving SAT through randomization

The 3SAT problem is **NP** hard, and so it is unlikely that it has a polynomial (or even subexponential) time algorithm. But this does not mean that we can't do at least somewhat better than the trivial 2^n algorithm for n-variable 3SAT. The best known worst-case algorithms for 3SAT are randomized, and are related to the following simple algorithm, variants of which are also used in practice:

Algorithm WalkSAT:

Input: An n variable 3CNF formula φ.

Parameters: $T, S \in \mathbb{N}$

Operation:

1. Repeat the following T steps:

a. Choose a random assignment $x \in \{0,1\}^n$ and repeat the following for S steps:
   1. If x satisfies φ then output x.
   2. Otherwise, choose a random clause $(\ell_i \vee \ell_j \vee \ell_k)$ that x does not satisfy, choose a random literal in ℓ_i, ℓ_j, ℓ_k and modify x to satisfy this literal.

2. If all the $T \cdot S$ repetitions above did not result in a satisfying assignment then output `Unsatisfiable`

The running time of this algorithm is $S \cdot T \cdot poly(n)$, and so the key question is how small we can make S and T so that the probability that WalkSAT outputs `Unsatisfiable` on a satisfiable formula φ is small. It is known that we can do so with $ST = \tilde{O}((4/3)^n) = \tilde{O}(1.333\ldots^n)$ (see Exercise 18.4 for a weaker result), but we'll show below a simpler analysis yielding $ST = \tilde{O}(\sqrt{3}^n) = \tilde{O}(1.74^n)$, which is still much better than the trivial 2^n bound.[3]

Theorem 18.5 — WalkSAT simple analysis. If we set $T = 100 \cdot \sqrt{3}^n$ and $S = n/2$, then the probability we output `Unsatisfiable` for a satisfiable φ is at most $1/2$.

[3] At the time of this writing, the best known randomized algorithms for 3SAT run in time roughly $O(1.308^n)$, and the best known deterministic algorithms run in time $O(1.3303^n)$ in the worst case.

Proof. Suppose that φ is a satisfiable formula and let x^* be a satisfying assignment for it. For every $x \in \{0,1\}^n$, denote by $\Delta(x, x^*)$ the number of coordinates that differ between x and x^*. The heart of the proof is the following claim:

Claim I: For every x, x^* as above, in every local improvement step, the value $\Delta(x, x^*)$ is decreased by one with probability at least $1/3$.

Proof of Claim I: Since x^* is a *satisfying* assignment, if C is a clause that x does *not* satisfy, then at least one of the variables involve in C must get different values in x and x^*. Thus when we change x by one of the three literals in the clause, we have probability at least $1/3$ of decreasing the distance.

The second claim is that our starting point is not that bad:

Claim 2: With probability at least $1/2$ over a random $x \in \{0,1\}^n$, $\Delta(x, x^*) \leq n/2$.

Proof of Claim II: Consider the map $FLIP : \{0,1\}^n \to \{0,1\}^n$ that simply "flips" all the bits of its input from 0 to 1 and vice versa. That is, $FLIP(x_0, \ldots, x_{n-1}) = (1-x_0, \ldots, 1-x_{n-1})$. Clearly $FLIP$ is one to one. Moreover, if x is of distance k to x^*, then $FLIP(x)$ is distance $n-k$ to x^*. Now let B be the "bad event" in which x is of distance $> n/2$ from x^*. Then the set $A = FLIP(B) = \{FLIP(x) : x \in \{0,1\}^n\}$ satisfies

$|A| = |B|$ and that if $x \in A$ then x is of distance $< n/2$ from x^*. Since A and B are disjoint events, $\Pr[A] + \Pr[B] \leq 1$. Since they have the same cardinality, they have the same probability and so we get that $2\Pr[B] \leq 1$ or $\Pr[B] \leq 1/2$. (See also Fig. 18.2).

Claims I and II imply that each of the T iterations of the outer loop succeeds with probability at least $0.5 \cdot \sqrt{3}^{-n/2}$. Indeed, by Claim II, the original guess x will satisfy $\Delta(x, x^*) \leq n/2$, and by Claim I, even conditioned on all the history so far, for each of the $S = n/2$ steps we have probability $\geq 1/3$ of being "lucky" and decreasing the distance at one. The chance we will be lucky in all $n/2$ steps is hence at least $(1/3)^{n/2} = \sqrt{3}^{n/2}$.

Since any single iteration of the outer loop succeeds with probability at least $\frac{1}{2} \cdot \sqrt{3}^{-n}$, the probability that we never do so in $T = 100\sqrt{3}^{n}$ repetitions is at most $(1 - \frac{1}{2\sqrt{3}^{n}})^{100 \cdot \sqrt{3}^{n}} \leq (1/e)^{50}$.

■

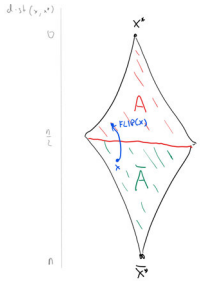


Figure 18.2: For every $x^* \in \{0,1\}^n$, we can sort all strings in $\{0,1\}^n$ according to their distance from x^* (top to bottom in the above figure), where we let $A = \{x \in \{0,1\}^n \mid dist(x, x^* \leq n/2\}$ be the "top half" of strings. If we define $FLIP : \{0,1\}^n \to \{0,1\}$ to be the map that "flips" the bits of a given string x then it maps every $x \in \overline{A}$ to an output $FLIP(x) \in A$ in a one-to-one way, and so it demonstrates that $|\overline{A}| \leq |A|$ which implies that $\Pr[A] \geq \Pr[\overline{A}]$ and hence $\Pr[A] \geq 1/2$.

18.1.5 Bipartite matching.

The *matching* problem is one of the canonical optimization problems, arising in all kinds of applications: matching residents with hospitals, kidney donors with patients, flights with crews, and many others. One prototypical variant is *bipartite perfect matching*. In this problem, we are given a bipartite graph $G = (L \cup R, E)$ which has $2n$ vertices partitioned into n-sized sets L and R, where all edges have one endpoint in L and the other in R. The goal is to determine whether there is a *perfect matching*, a subset $M \subseteq E$ of n disjoint edges. That is, M matches every vertex in L to a unique vertex in R.

The bipartite matching problem turns out to have a polynomial-time algorithm, since we can reduce finding a matching in G to finding a maximum flow (or equivalently, minimum cut) in a related graph G' (see Fig. 18.3). However, we will see a different probabilistic algorithm to determine whether a graph contains such a matching.

Let us label G's vertices as $L = \{\ell_0, \ldots, \ell_{n-1}\}$ and $R = \{r_0, \ldots, r_{n-1}\}$. A matching M corresponds to a *permutation* $\pi \in S_n$ (i.e., one-to-one and onto function $\pi : [n] \to [n]$) where for every $i \in [n]$, we define $\pi(i)$ to be the unique j such that M contains the edge $\{\ell_i, r_j\}$. Define an $n \times n$ matrix $A = A(G)$ where $A_{i,j} = 1$ if and only if the edge $\{\ell_i, r_j\}$ is present and $A_{i,j} = 0$ otherwise. The correspondence between matchings and permutations implies the following claim:

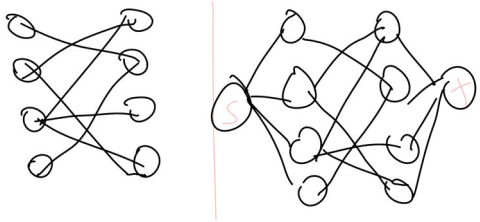


Figure 18.3: The bipartite matching problem in the graph $G = (L \cup R, E)$ can be reduced to the minimum s, t cut problem in the graph G' obtained by adding vertices s, t to G, connecting s with L and connecting t with R.

Lemma 18.6 — Matching polynomial. Define $P = P(G)$ to be the polynomial mapping $\mathbb{R}^{n^2}$ to $\mathbb{R}$ where

$$P(x_{0,0}, \ldots, x_{n-1,n-1}) = \sum_{\pi \in S_n} \left(\prod_{i=0}^{n-1} sign(\pi) A_{i,\pi(i)} \right) \prod_{i=0}^{n-1} x_{i,\pi(i)} \qquad (18.7)$$

Then G has a perfect matching if and only if P is not identically zero. That is, G has a perfect matching if and only if there exists some assignment $x = (x_{i,j})_{i,j\in[n]} \in \mathbb{R}^{n^2}$ such that $P(x) \neq 0$.[4]

Proof. If G has a perfect matching M^*, then let π^* be the permutation corresponding to M and let $x^* \in \mathbb{Z}^{n^2}$ defined as follows: $x_{i,j} = 1$ if $j = \pi(i)$ and $x_{i,j} = 0$. Note that for every $\pi \neq \pi^*$, $\prod_{i=0}^{n-1} x_{i,\pi(i)} = 0$ but $\prod_{i=0}^{n-1} x^*_{i,\pi^*(i)} = 1$. Hence $P(x^*)$ will equal $\prod_{i=0}^{n-1} A_{i,\pi^*(i)}$. But since M^* is a perfect matching in G, $\prod_{i=0}^{n-1} A_{i,\pi^*(i)} = 1$.

On the other hand, suppose that P is not identically zero. By (18.7), this means that at least one of the terms $\prod_{i=0}^{n-1} A_{i,\pi(i)}$ is not equal to zero. But then this permutation π must be a perfect matching in G. ■

As we've seen before, for every $x \in \mathbb{R}^{n^2}$, we can compute $P(x)$ by simply computing the *determinant* of the matrix $A(x)$, which is obtained by replacing $A_{i,j}$ with $A_{i,j}x_{i,j}$. This reduces testing perfect matching to the *zero testing* problem for polynomials: given some polynomial $P(\cdot)$, test whether P is identically zero or not. The intuition behind our randomized algorithm for zero testing is the following:

> *If a polynomial is not identically zero, then it can't have "too many" roots.*

This intuition sort of makes sense. For one variable polynomials, we know that a nonzero linear function has at most one root, a quadratic function (e.g., a parabola) has at most two roots, and generally a degree d equation has at most d roots. While in more than one variable there can be an infinite number of roots (e.g., the polynomial $x_0 + y_0$ vanishes on the line $y = -x$) it is still the case that the set of roots is very "small" compared to the set of all inputs. For example, the root of a bivariate polynomial form a curve, the roots of a three-variable polynomial form a surface, and more generally the roots of an n-variable polynomial are a space of dimension $n - 1$.

This intuition leads to the following simple randomized algorithm:

> *To decide if P is identically zero, choose a "random" input x and check if $P(x) \neq 0$.*

This makes sense: if there are only "few" roots, then we expect that with high probability the random input x is not going to be one of those roots. However, to transform this into an actual algorithm, we need to make both the intuition and the notion of a "random" input precise. Choosing a random real number is quite problematic, especially when you have only a finite number of coins at your disposal, and so we start by reducing the task to a finite setting. We will use the following result

[4] The sign of a permutation $\pi : [n] \to [n]$, denoted by $sign(\pi)$, can be defined in several equivalent ways, one of which is that $sign(\pi) = (-1)^{INV(\pi)}$ where $INV(pi) = |\{(x, y) \in [n] \mid x < y \wedge \pi(x) > \pi(y)\}$ (i.e., $INV(\pi)$ is the number of pairs of elements that are *inverted* by π). The importance of the term $sign(\pi)$ is that it makes P equal to the *determinant* of the matrix $(x_{i,j})$ and hence efficiently computable.

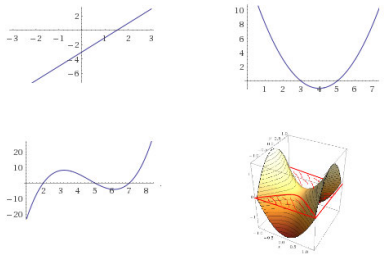


Figure 18.4: A degree d curve in one variable can have at most d roots. In higher dimensions, a n-variate degree-d polynomial can have an infinite number roots though the set of roots will be an $n - 1$ dimensional surface. Over a finite field $\mathbb{F}$, an n-variate degree d polynomial has at most $d|\mathbb{F}|^{n-1}$ roots.

Theorem 18.7 — Schwartz–Zippel lemma. For every integer q, and polynomial $P : \mathbb{R}^n \to \mathbb{R}$ with integer coefficients. If P has degree at most d and is not identically zero, then it has at most dq^{n-1} roots in the set $[q]^n = \{(x_0, \ldots, x_{n-1}) : x_i \in \{0, \ldots, q-1\}\}$.

We omit the (not too complicated) proof of Theorem 18.7. We remark that it holds not just over the real numbers but over any field as well. Since the matching polynomial P of Lemma 18.6 has degree at most n, Theorem 18.7 leads directly to a simple algorithm for testing if it is nonzero:

Algorithm Perfect-Matching:

Input: Bipartite graph G on $2n$ vertices $\{\ell_0, \ldots, \ell_{n-1}, r_0, \ldots, r_{n-1}\}$.

Operation:

1. For every $i, j \in [n]$, choose $x_{i,j}$ independently at random from $[2n] = \{0, \ldots 2n-1\}$.
2. Compute the determinant of the matrix $A(x)$ whose $(i,j)^{th}$ entry equals $x_{i,j}$ if the edge $\{\ell_i, r_j\}$ is present and 0 otherwise.
3. Output `no perfect matching` if this determinant is zero, and output `perfect matching` otherwise.

This algorithm can be improved further (e.g., see Exercise 18.5). While it is not necessarily faster than the cut-based algorithms for perfect matching, it does have some advantages. In particular, it is more amenable for parallelization. (However, it also has the significant disadvantage that it does not produce a matching but only states that one exists.) The Schwartz–Zippel Lemma, and the associated zero testing algorithm for polynomials, is widely used across computer science, including in several settings where we have no known deterministic algorithm matching their performance.



Lecture Recap

- Using concentration results, we can *amplify* in polynomial time the success probability of a probabilistic algorithm from a mere $1/p(n)$ to $1 - 2^{-q(n)}$ for every polynomials p and q.
- There are several randomized algorithms that are better in various senses (e.g., simpler, faster, or other advantages) than the best known deterministic algorithm for the same problem.

18.2 EXERCISES



Remark 18.8 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 18.1 — Amplification for max cut. Prove Lemma 18.3

Exercise 18.2 — Deterministic max cut algorithm. [5]

[5] TODO: add exercise to give a deterministic max cut algorithm that gives $m/2$ edges. Talk about greedy approach.

Exercise 18.3 — Simulating distributions using coins. Our model for probability involves tossing n coins, but sometimes algorithm require sampling from other distributions, such as selecting a uniform number in $\{0, \ldots, M-1\}$ for some M. Fortunately, we can simulate this with an exponentially small probability of error: prove that for every M, if $n > k\lceil \log M \rceil$, then there is a function $F : \{0,1\}^n \to \{0, \ldots, M-1\} \cup \{\perp\}$ such that **(1)** The probability that $F(x) = \perp$ is at most 2^{-k} and **(2)** the distribution of $F(x)$ conditioned on $F(x) \neq \perp$ is equal to the uniform distribution over $\{0, \ldots, M-1\}$.[6]

[6] **Hint:** Think of $x \in \{0,1\}^n$ as choosing k numbers $y_1, \ldots, y_k \in \{0, \ldots, 2^{\lceil \log M \rceil} - 1\}$. Output the first such number that is in $\{0, \ldots, M-1\}$.

Exercise 18.4 — Better walksat analysis.

1. Prove that for every $\epsilon > 0$, if n is large enough then for every $x^* \in \{0,1\}^n$ $\Pr_{x \sim \{0,1\}^n}[\Delta(x, x^*) \leq n/3] \leq 2^{-(1-H(1/3)-\epsilon)n}$ where $H(p) = p\log(1/p) + (1-p)\log(1/(1-p))$ is the same function as in Exercise 17.8.
2. Prove that $2^{1-H(1/4)+(1/4)\log 3} = (3/2)$.
3. Use the above to prove that for every $\delta > 0$ and large enough n, if we set $T = 1000 \cdot (3/2+\delta)^n$ and $S = n/4$ in the WalkSAT algorithm then for every satisfiable 3CNF φ, the probability that we output `unsatisfiable` is at most $1/2$.

Exercise 18.5 — Faster bipartite matching (challenge). [7]

[7] TODO: add exercise to improve the matching algorithm by working modulo a prime

18.3 BIBLIOGRAPHICAL NOTES

monte carlo history: `http://permalink.lanl.gov/object/tr?what=info:lanl-repo/lareport/LA-UR-88-9068`

18.4 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

18.5 ACKNOWLEDGEMENTS

Learning Objectives:

- **Formal definition of probabilistic polynomial time: the class BPP.**
- **Proof that that every function in BPP can be computed by $poly(n)$-sized NAND-CIRC programs/circuits.**
- **Relations between BPP and NP.**
- **Pseudorandom generators**

19
Modeling randomized computation

> *"Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin."* John von Neumann, 1951.

So far we have described randomized algorithms in an informal way, assuming that an operation such as "pick a string $x \in \{0,1\}^n$" can be done efficiently. We have neglected to address two questions:

1. How do we actually efficiently obtain random strings in the physical world?

2. What is the mathematical model for randomized computations, and is it more powerful than deterministic computation?

The first question is of both practical and theoretical importance, but for now let's just say that there are various physical sources of "random" or "unpredictable" data. A user's mouse movements and typing pattern, (non solid state) hard drive and network latency, thermal noise, and radioactive decay have all been used as sources for randomness. For example, many Intel chips come with a random number generator built in. One can even build mechanical coin tossing machines (see Fig. 19.1).[1]

In this chapter we focus on the second question: formally modeling probabilistic computation and studying its power. Modeling randomized computation is actually quite easy. We can add the following operations to our NAND, NAND-TM and NAND-RAM programming languages:

```
foo = RAND()
```

where `foo` is a variable. The result of applying this operation is that `foo` is assigned a random bit in $\{0,1\}$. (Every time the `RAND` operation is invoked it returns a fresh independent random bit.) We call

[1] The output of processes such as above can be thought of as a binary string sampled from some distribution μ that might have significant unpredictablity (or *entropy*) but is not necessarily the *uniform* distribution over $\{0,1\}^n$. Indeed, as this paper shows, even (real-world) coin tosses do not have exactly the distribution of a uniformly random string. Therefore, to use the resulting measurements for randomized algorithms, one typically needs to apply a "distillation" or *randomness extraction* process to the raw measurements to transform them to the uniform distribution.

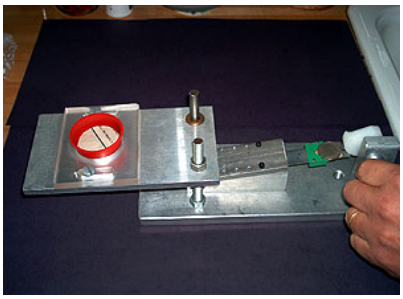


Figure 19.1: A mechanical coin tosser built for Percy Diaconis by Harvard technicians Steve Sansone and Rick Haggerty

the resulting languages RNAND, RNAND-TM, and RNAND-RAM respectively.

We can use this to define the notion of a function being computed by a randomized $T(n)$ time algorithm for every nice time bound $T : \mathbb{N} \to \mathbb{N}$, as well as the notion of a finite function being computed by a size S randomized NAND-CIRC program (or, equivalently, a randomized circuit with S gates that correspond to either the NAND or coin-tossing operations). However, for simplicity we will not define randomized computation in full generality, but simply focus on the class of functions that are computable by randomized algorithms *running in polynomial time*, which by historical convention is known as **BPP**:

Definition 19.1 — BPP. Let $F : \{0,1\}^* \to \{0,1\}$. We say that $F \in$ **BPP** if there exist constants $a, b \in \mathbb{N}$ and an RNAND-RAM program P such that for every $x \in \{0,1\}^*$, on input x, the program P halts within at most $a|x|^b$ steps and

$$\Pr[P(x) = F(x)] \geq \frac{2}{3} \tag{19.1}$$

where this probability is taken over the result of the RAND operations of P. [2]

[2] **BPP** stands for "bounded probability polynomial time", and is used for historical reasons.

The same polynomial-overhead simulation of NAND-RAM programs by NAND-TM programs we saw in Theorem 12.6 extends to *randomized* programs as well. Hence the class **BPP** is the same regardless of whether it is defined via RNAND-TM or RNAND-RAM programs.

19.0.1 An alternative view: random coins as an "extra input"

While we presented randomized computation as adding an extra "coin tossing" operation to our programs, we can also model this as being given an additional extra input. That is, we can think of a randomized algorithm A as a *deterministic* algorithm A' that takes *two inputs* x and r where the second input r is chosen at random from $\{0,1\}^m$ for some $m \in \mathbb{N}$ (see Fig. 19.2). The equivalence to the Definition 19.1 is shown in the following theorem:

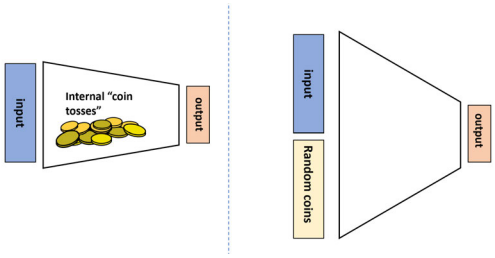


Figure 19.2: The two equivalent views of randomized algorithms. We can think of such an algorithm as having access to an internal `RAND()` operation that outputs a random independent value in $\{0,1\}$ whenever it is invoked, or we can think of it as a deterministic algorithm that in addition to the standard input $x \in \{0,1\}^n$ obtains an additional auxiliary input $r \in \{0,1\}^m$ that is chosen uniformly at random.

Theorem 19.2 — Alternative characterization of BPP. Let $F : \{0,1\}^* \to \{0,1\}$. Then $F \in$ **BPP** if and only if there exists $a, b \in \mathbb{N}$ and $G : \{0,1\}^* \to \{0,1\}$ such that G is in **P** and for every $x \in \{0,1\}^*$,

$$\Pr_{r \sim \{0,1\}^{a|x|^b}}[G(xr) = F(x)] \geq \frac{2}{3} . \tag{19.2}$$

Proof Idea:

The idea behind the proof is that, as illustrated in Fig. 19.2, we can simply replace sampling a random coin with reading a bit from the extra "random input" r and vice versa. To prove this rigorously we need to work through some slightly cumbersome formal notation. This might be one of those proofs that is easier to work out on your own than to read.

⋆

Proof of Theorem 19.2. We start by showing the "only if" direction. Let $F \in$ **BPP** and let P be an RNAND-RAM program that computes F as per Definition 19.1, and let $a, b \in \mathbb{N}$ be such that on every input of length n, the program P halts within at most an^b steps. We will construct a polynomial-time NAND-RAM program P' that computes a function G satisfying the conditions of (19.2).

The program P' is very simple:

Program P': (Deterministic NAND-RAM program)

Inputs:

- $x \in \{0,1\}^n$
- $r \in \{0,1\}^{an^b}$

Goal: Output y that has the same distribution as the output of the RNAND-RAM program P on input x.

Operation:

1. Copy the string r to an array `Coins`. That is `Coins[`i`]` $= r_i$ for all $i \in [an^b]$.
2. Let `coincounter` be a variable and set it to 0.
3. Simulate an execution the RNAND-RAM program P, replacing any line of the form `foo = RAND()` with the two lines: `foo = Coins[coincounter]` and `coincounter = coincounter + 1`.

The program P' is a deterministic polynomial time NAND-RAM program, and so computes some function $G \in$ **P**. By its construction, the distribution of $P'(xr)$ for random $r \in \{0,1\}^{2n^b}$ is identical to the distribution of $P(x)$ (where in the latter case the sample space is the results of the `RAND()` calls), and hence in particular it will hold that $\Pr_{r\in\{0,1\}^{an^b}}[P'(xr) = F(x)] \geq 2/3$.

For the other direction, given a function $G \in$ **P** satisfying the condition (19.2) and a NAND-RAM program P' that computes G in polynomial time, we can construct an RNAND-RAM program P that computes F in polynomial time. On input $x \in \{0,1\}^n$, the program P

will simply use the `RNAND()` instruction an^b times to fill an array `R[0]` , ..., `R[`$an^b - 1$`]` and then execute the original program P' on input xr where r_i is the i-th element of the array `R`. Once again, it is clear that if P' runs in polynomial time then so will P, and for every input x and $r \in \{0,1\}^{an^b}$, the output of P on input x and where the coin tosses outcome is r is equal to $P'(xr)$.

■

R

Remark 19.3 — Definitions of BPP and NP. The characterization of **BPP** Theorem 19.2 is reminiscent of the characterization of **NP** in Definition 14.1, with the randomness in the case of **BPP** playing the role of the solution in the case of **NP**. However, there are important differences between the two:

- The definition of **NP** is "one sided": $F(x) = 1$ if *there exists* a solution w such that $G(xw) = 1$ and $F(x) = 0$ if *for every* string w of the appropriate length, $G(xw) = 0$. In contrast, the characterization of **BPP** is symmetric with respect to the cases $F(x) = 0$ and $F(x) = 1$.
- The relation between **NP** and **BPP** is not immediately clear. It is not known whether **BPP** $\subseteq$ **NP**, **NP** $\subseteq$ **BPP**, or these two classes are incomparable. It is however known (with a non-trivial proof) that if **P** $=$ **NP** then **BPP** $=$ **P** (see Theorem 19.11).
- Most importantly, the definition of **NP** is "ineffective," since it does not yield a way of actually finding whether there exists a solution among the exponentially many possibilities. By contrast, the definition of **BPP** gives us a way to compute the function in practice by simply choosing the second input at random.

Random tapes Theorem 19.2 motivates sometimes considering the randomness of an RNAND-TM (or RNAND-RAM) program as an extra input. As such, if A is a randomized algorithm that on inputs of length n makes at most $p(n)$ coin tosses, we will often use the notation $A(x;r)$ (where $x \in \{0,1\}^n$ and $r \in \{0,1\}^{p(n)}$) to refer to the result of executing x when the coin tosses of A correspond to the coordinates of r. This second, or "auxiliary," input is sometimes referred to as a "random tape." This terminology originates from the model of randomized Turing machines.

19.0.2 Amplification

The number $2/3$ might seem arbitrary, but as we've seen in Chapter 18 it can be amplified to our liking:

Theorem 19.4 — Amplification. Let P be an RNAND-RAM program, $F \in \{0,1\}^* \to \{0,1\}$, and $T : \mathbb{N} \to \mathbb{N}$ be a nice time bound such that for every $x \in \{0,1\}^*$, on input x the program P runs in at most $T(|x|)$ steps and moreover $\Pr[P(x) = F(x)] \geq \frac{1}{2} + \epsilon$ for some $\epsilon > 0$. Then for every k, there is a program P' taking at most $O(k \cdot T(n)/\epsilon^2)$ steps such that on input $x \in \{0,1\}^*$, $\Pr[P'(x) = F(x)] > 1 - 2^{-k}$.

Proof Idea:

The proof is the same as we've seen before in the maximum cut and other examples. We use the Chernoff bound to argue that if we run the program $O(k/\epsilon^2)$ times, each time using fresh and independent random coins, then the probability that the majority of the answers will not be correct will be less than 2^{-k}. Amplification can be thought of as a "polling" of the choices for randomness for the algorithm (see Fig. 19.3).

⋆

Proof of Theorem 19.4. We can run P on input x for $t = 10k/\epsilon^2$ times, using fresh randomness in each execution, and compute the outputs $y_0, \ldots, y_{t-1}$. We output the value y that appeared the largest number of times. Let X_i be the random variable that is equal to 1 if $y_i = F(x)$ and equal to 0 otherwise. The random variables $X_0, \ldots, X_{t-1}$ are i.i.d. and satisfy $\mathbb{E}[X_i] = \Pr[X_i = 1] \geq 1/2 + \epsilon$, and hence by linearity of expectation $\mathbb{E}[\sum_{i=0}^{t-1} X_i] \geq t(1/2 + \epsilon)$. For the plurality value to be *incorrect*, it must hold that $\sum_{i=0}^{t-1} X_i \leq t/2$, which means that $\sum_{i=0}^{t-1} X_i$ is at least ϵt far from its expectation. Hence by the Chernoff bound (Theorem 17.11), the probability that the plurality value is not correct is at most $2e^{-\epsilon^2 t}$, which is smaller than 2^{-k} for our choice of t. ■

There is nothing special about NAND-RAM in Theorem 19.4. The same proof can be used to amplify randomized NAND or NAND-TM programs as well.

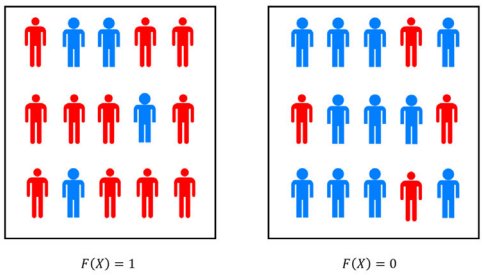


Figure 19.3: If $F \in$ **BPP** then there is randomized polynomial-time algorithm P with the following property: In the case $F(x) = 0$ two thirds of the "population" of random choices satisfy $P(x;r) = 0$ and in the case $F(x) = 1$ two thirds of the population satisfy $P(x;r) = 1$. We can think of amplification as a form of "polling" of the choices of randomness. By the Chernoff bound, if we poll a sample of $O(\frac{\log(1/\delta)}{\epsilon^2})$ random choices r, then with probability at least $1 - \delta$, the fraction of r's in the sample satisfying $P(x;r) = 1$ will give us an estimate of the fraction of the population within an ϵ margin of error. This is the same calculation used by pollsters to determine the needed sample size in their polls.

19.1 BPP AND NP COMPLETENESS

Since "noisy processes" abound in nature, randomized algorithms can be realized physically, and so it is reasonable to propose **BPP** rather than **P** as our mathematical model for "feasible" or "tractable" computation. One might wonder if this makes all the previous chapters irrelevant, and in particular if the theory of **NP** completeness still applies to probabilistic algorithms. Fortunately, the answer is *Yes*:

Theorem 19.5 — NP hardness and BPP. Suppose that F is **NP**-hard and $F \in$ **BPP**. Then **NP** $\subseteq$ **BPP**.

Before seeing the proof, note that Theorem 19.5 implies that if there was a randomized polynomial time algorithm for any **NP**-complete problem such as $3SAT$, *ISET* etc., then there would be such an algorithm for *every* problem in **NP**. Thus, regardless of whether our model of computation is deterministic or randomized algorithms, **NP** complete problems retain their status as the "hardest problems in **NP**."

Proof Idea:

The idea is to simply run the reduction as usual, and plug it into the randomized algorithm instead of a deterministic one. It would be an excellent exercise, and a way to reinforce the definitions of **NP**-hardness and randomized algorithms, for you to work out the proof for yourself. However for the sake of completeness, we include this proof below.

⋆

Proof of Theorem 19.5. Suppose that F is **NP**-hard and $F \in$ **BPP**. We will now show that this implies that **NP** $\subseteq$ **BPP**. Let $G \in$ **NP**. By the definition of **NP**-hardness, it follows that $G \leq_p F$, or that in other words there exists a polynomial-time computable function $R : \{0,1\}^* \to \{0,1\}^*$ such that $G(x) = F(R(x))$ for every $x \in \{0,1\}^*$. Now if F is in **BPP** then there is a polynomial-time RNAND-TM program P such that

$$\Pr[P(y) = F(y)] \geq 2/3 \tag{19.3}$$

for *every* $y \in \{0,1\}^*$ (where the probability is taken over the random coin tosses of P). Hence we can get a polynomial-time RNAND-TM program P' to compute G by setting $P'(x) = P(R(x))$. By (19.3) $\Pr[P'(x) = F(R(x))] \geq 2/3$ and since $F(R(x)) = G(x)$ this implies that $\Pr[P'(x) = G(x)] \geq 2/3$, which proves that $G \in$ **BPP**.

■

Most of the results we've seen about **NP** hardness, including the search to decision reduction of Theorem 15.1, the decision to optimization reduction of Theorem 15.2, and the quantifier elimination result of Theorem 15.5, all carry over in the same way if we replace **P** with **BPP** as our model of efficient computation. Thus if **NP** $\subseteq$ **BPP** then we get essentially all of the strange and wonderful consequences of **P** $=$ **NP**. Unsurprisingly, we cannot rule out this possiblity. In fact, unlike **P** $=$ **EXP**, which is ruled out by the time hierarchy theorem, we don't even know how to rule out the possiblity that **BPP** $=$ **EXP**! Thus a priori it's possible (though seems highly unlikely) that randomness is a magical tool that allows us to speed up arbitrary exponential time

computation.[3] Nevertheless, as we discuss below, it is believed that randomization's power is much weaker and **BPP** lies in much more "pedestrian" territory.

[3] At the time of this writing, the largest "natural" complexity class which we can't rule out being contained in **BPP** is the class **NEXP**, which we did not define in this course, but corresponds to non deterministic exponential time. See this paper for a discussion of this question.

19.2 THE POWER OF RANDOMIZATION

A major question is whether randomization can add power to computation. Mathematically, we can phrase this as the following question: does **BPP** $=$ **P**? Given what we've seen so far about the relations of other complexity classes such as **P** and **NP**, or **NP** and **EXP**, one might guess that:

1. We do not know the answer to this question.
2. But we suspect that **BPP** is different than **P**.

One would be correct about the former, but wrong about the latter. As we will see, we do in fact have reasons to believe that **BPP** $=$ **P**. This can be thought of as supporting the *extended Church Turing hypothesis* that deterministic polynomial-time NAND-TM program (or, equivalently, polynomial-time Turing machines) capture what can be feasibly computed in the physical world.

We now survey some of the relations that are known between **BPP** and other complexity classes we have encountered. (See also Fig. 19.4.)

19.2.1 Solving BPP in exponential time

It is not hard to see that if F is in **BPP** then it can be computed in *exponential* time.

Theorem 19.6 — Simulating randomized algorithms in exponential time. **BPP** $\subseteq$ **EXP**

The proof of Theorem 19.6 readily follows by enumerating over all the (exponentially many) choices for the random coins. We omit the formal proof, as doing it by yourself is an excellent way to get comfortable with Definition 19.1.

19.2.2 Simulating randomized algorithms by circuits or straight-line programs.

We have seen in Theorem 12.12 that if F is in **P**, then there is a polynomial $p : \mathbb{N} \to \mathbb{N}$ such that for every n, the restriction $F_{\restriction n}$ of F to inputs $\{0,1\}^n$ is in $SIZE(p(n))$. (In other words, that **P** $\subseteq$ $\mathbf{P}_{/\mathbf{poly}}$.) A priori it is

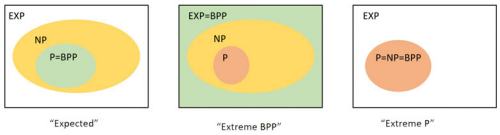


Figure 19.4: Some possibilities for the relations between **BPP** and other complexity classes. Most researchers believe that **BPP** $=$ **P** and that these classes are *not* powerful enough to solve **NP**-complete problems, let alone all problems in **EXP**. However, we have not even been able yet to rule out the possiblity that randomness is a "silver bullet" that allows exponential speedup on all problems, and hence **BPP** $=$ **EXP**. As we've already seen, we also can't rule out that **P** $=$ **NP**. Interestingly, in the latter case, **P** $=$ **BPP**.

not at all clear that the same holds for a function in **BPP**, but this does turn out to be the case.

Theorem 19.7 — Randomness does not help for non uniform computation. **BPP** $\subseteq \mathbf{P}_{/\mathbf{poly}}$. That is, for every $F \in$ **BPP**, there exist some $a, b \in \mathbb{N}$ such that for every $n > 0$, $F_{\restriction n} \in SIZE(an^b)$ where $F_{\restriction n}$ is the restriction of F to inputs in $\{0,1\}^n$.

Proof Idea:

The idea behind the proof is that we can first amplify by repetition the probability of success from $2/3$ to $1 - 0.1 \cdot 2^{-n}$. This will allow us to show that there exists a single fixed choice of "favorable coins" that would cause the algorithm to output the right answer on *all* of the possible 2^n inputs. We can then use the standard "unravelling the loop" technique to transform an RNAND-TM program to an RNAND-CIRC program, and "hardwire" the favorable choice of random coins to transform the RNAND-CIRC program into a plain old deterministic NAND-CIRC program.

⋆

Proof of Theorem 19.7. Suppose that $F \in$ **BPP**. Let P be a polynomial-time RNAND-TM program that computes F as per Definition 19.1. Using Theorem 19.4, we can *amplify* the success probability of P to obtain an RNAND-TM program P' that is at most a factor of $O(n)$ slower (and hence still polynomial time) such that for every $x \in \{0,1\}^n$

$$\Pr_{r \sim \{0,1\}^m}[P'(x;r) = F(x)] \geq 1 - 0.1 \cdot 2^{-n} \;, \tag{19.4}$$

where m is the number of coin tosses that P' uses on inputs of length n. We use the notation $P'(x;r)$ to denote the execution of P' on input x and when the result of the coin tosses corresponds to the string r.

For every $x \in \{0,1\}^n$, define the "bad" event B_x to hold if $P'(x) \neq F(x)$, where the sample space for this event consists of the coins of P'. Then by (19.4), $\Pr[B_x] \leq 0.1 \cdot 2^{-n}$ for every $x \in \{0,1\}^n$. Since there are 2^n many such x's, by the union bound we see that the probability that the *union* of the events $\{B_x\}_{x \in \{0,1\}^n}$ is at most 0.1. This means that if we choose $r \sim \{0,1\}^m$, then with probability at least 0.9 it will be the case that for *every* $x \in \{0,1\}^n$, $F(x) = P'(x;r)$. (Indeed, otherwise the event B_x would hold for some x.) In particular, because of the mere fact that the the probability of $\cup_{x \in \{0,1\}^n} B_x$ is smaller than 1, this means that *there exists* a particular $r^* \in \{0,1\}^m$ such that

$$P'(x;r^*) = F(x) \tag{19.5}$$

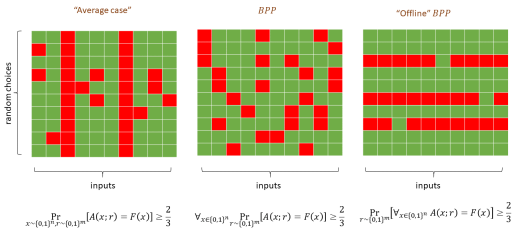


Figure 19.5: The possible guarantees for a randomized algorithm A computing some function F. In the tables above, the columns correspond to different inputs and the rows to different choices of the random tape. A cell at position r, x is colored green if $A(x;r) = F(x)$ (i.e., the algorithm outputs the correct answer) and red otherwise. The standard **BPP** guarantee corresponds to the middle figure, where for every input x, at least two thirds of the choices r for a random tape will result in A computing the correct value. That is, every column is colored green in at least two thirds of its coordinates. In the left figure we have an "average case" guarantee where the algorithm is only guaranteed to output the correct answer with probabilty two thirds over a *random* input (i.e., at most one third of the total entries of the table are colored red, but there could be an all red column). The right figure corresponds to the "offline **BPP**" case, with probability at least two thirds over the random choice r, r will be good for *every* input. That is, at least two thirds of the rows are all green. Theorem 19.7 (**BPP** $\subseteq \mathbf{P}_{/\mathbf{poly}}$) is proven by amplifying the success of a **BPP** algorithm until we have the "offline **BPP**" guarantee, and then hardwiring the choice of the randomness r to obtain a nonuniform deterministic algorithm.

for every $x \in \{0,1\}^n$.

Now let us use the standard "unravelling the loop" the technique and transform P' into a NAND-CIRC program Q of polynomial in n size, such that $Q(xr) = P'(x;r)$ for every $x \in \{0,1\}^n$ and $r \in \{0,1\}^m$. Then by "hardwiring" the values $r^*_0, \ldots, r^*_{m-1}$ in place of the last m inputs of Q, we obtain a new NAND-CIRC program Q_{r^*} that satisfies by (19.5) that $Q_{r^*}(x) = F(x)$ for every $x \in \{0,1\}^n$. This demonstrates that $F_{\restriction n}$ has a polynomial sized NAND-CIRC program, hence completing the proof of Theorem 19.7.

■

R

Remark 19.8 — Randomness and non uniformity. The proof of Theorem 19.7 actually yields more than its statement. We can use the same "unrolling the loop" arguments we've used before to show that the restriction to $\{0,1\}^n$ of every function in **BPP** is also computable by a polynomial-size RNAND-CIRC program (i.e., NAND-CIRC program with the `RAND` operation). Like in the **P** vs $SIZE(poly(n))$ case, there are also functions outside **BPP** whose restrictions can be computed by polynomial-size RNAND-CIRC programs. Nevertheless the proof of Theorem 19.7 shows that even such functions can be computed by polynomial sized NAND-CIRC programs without using the `rand` operations. This can be phrased as saying that $BPSIZE(T(n)) \subseteq SIZE(O(nT(n)))$ (where $BPSIZE$ is defined in the natural way using RNAND progams). The stronger version of Theorem 19.7 we mentioned can be phrased as saying that $\mathbf{BPP}_{/\mathbf{poly}} = \mathbf{P}_{/\mathbf{poly}}$.

19.3 DERANDOMIZATION

The proof of Theorem 19.7 can be summarized as follows: we can replace a $poly(n)$-time algorithm that tosses coins as it runs with an algorithm that uses a single set of coin tosses $r^* \in \{0,1\}^{poly(n)}$ which will be good enough for all inputs of size n. Another way to say it is that for the purposes of computing functions, we do not need "online" access to random coins and can generate a set of coins "offline" ahead of time, before we see the actual input.

But this does not really help us with answering the question of whether **BPP** equals **P**, since we still need to find a way to generate these "offline" coins in the first place. To derandomize an RNAND-TM program we will need to come up with a *single* deterministic algorithm that will work for *all input lengths*. That is, unlike in the case of RNAND-CIRC programs, we cannot choose for every input length n some string $r^* \in \{0,1\}^{poly(n)}$ to use as our random coins.

Can we derandomize randomized algorithms, or does randomness add an inherent extra power for computation? This is a fundamentally interesting question but is also of practical significance. Ever since people started to use randomized algorithms during the Manhattan project, they have been trying to remove the need for randomness and replace it with numbers that are selected through some deterministic process. Throughout the years this approach has often been used successfully, though there have been a number of failures as well.[4]

A common approach people used over the years was to replace the random coins of the algorithm by a "randomish looking" string that they generated through some arithmetic progress. For example, one can use the digits of π for the random tape. Using these type of methods corresponds to what von Neumann referred to as a "state of sin". (Though this is a sin that he himself frequently committed, as generating true randomness in sufficient quantity was and still is often too expensive.) The reason that this is considered a "sin" is that such a procedure will not work in general. For example, it is easy to modify any probabilistic algorithm A such as the ones we have seen in Chapter 18, to an algorithm A' that is *guaranteed to fail* if the random tape happens to equal the digits of π. This means that the procedure "replace the random tape by the digits of π" does not yield a *general* way to transform a probabilistic algorithm to a deterministic one that will solve the same problem. Of course, this procedure does not *always* fail, but we have no good way to determine when it fails and when it succeeds. This reasoning is not specific to π and holds for every deterministically produced string, whether it obtained by π, e, the Fibonacci series, or anything else.

An algorithm that checks if its random tape is equal to π and then fails seems to be quite silly, but this is but the "tip of the iceberg" for a very serious issue. Time and again people have learned the hard way that one needs to be very careful about producing random bits using deterministic means. As we will see when we discuss cryptography, many spectacular security failures and break-ins were the result of using "insufficiently random" coins.

[4] One amusing anecdote is a recent case where scammers managed to predict the imperfect "pseudo-random generator" used by slot machines to cheat casinos. Unfortunately we don't know the details of how they did it, since the case was sealed.

19.3.1 Pseudorandom generators

So, we can't use any *single* string to "derandomize" a probabilistic algorithm. It turns out however, that we can use a *collection* of strings to do so. Another way to think about it is that rather than trying to *eliminate* the need for randomness, we start by focusing on *reducing* the amount of randomness needed. (Though we will see that if we reduce the randomness sufficiently, we can eventually get rid of it altogether.)

We make the following definition:

Definition 19.9 — Pseudorandom generator. A function $G : \{0,1\}^\ell \to \{0,1\}^m$ is a (T,ϵ)-*pseudorandom generator* if for every NAND-CIRC program P with m inputs and one output of at most T lines,

$$\left| \Pr_{s\sim\{0,1\}^\ell}[P(G(s))=1] - \Pr_{r\sim\{0,1\}^m}[P(r)=1] \right| < \epsilon \tag{19.6}$$

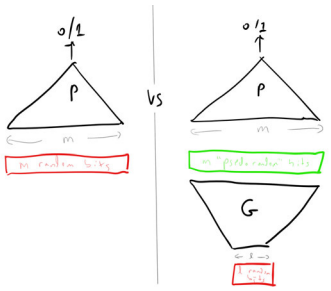


Figure 19.6: A pseudorandom generator G maps a short string $s \in \{0,1\}^\ell$ into a long string $r \in \{0,1\}^m$ such that an small program P cannot distinguish between the case that it is provided a random input $r \sim \{0,1\}^m$ and the case that it is provided a "pseudorandom" input of the form $r = G(s)$ where $s \sim \{0,1\}^\ell$. The short string s is sometimes called the *seed* of the pseudorandom generator, as it is a small object that can be thought as yielding a large "tree of randomness".

P

This is a definition that's worth reading more than once, and spending some time to digest it. Note that it takes several parameters:

- T is the limit on the number of lines of the program P that the generator needs to "fool". The larger T is, the stronger the generator.
- ϵ is how close is the output of the pseudorandom generator to the true uniform distribution over $\{0,1\}^m$. The smaller ϵ is, the stronger the generator.
- ℓ is the input length and m is the output length. If $\ell \geq m$ then it is trivial to come up with such a generator: on input $s \in \{0,1\}^\ell$, we can output $s_0, \ldots, s_{m-1}$. In this case $\Pr_{s\sim\{0,1\}^\ell}[P(G(s)) = 1]$ will simply equal $\Pr_{r\in\{0,1\}^m}[P(r) = 1]$, no matter how many lines P has. So, the smaller ℓ is and the larger m is, the stronger the generator, and to get anything non-trivial, we need $m > \ell$.

Furthermore note that although our eventual goal is to fool probabilistic randomized algorithms that take an unbounded number of inputs, Definition 19.9 refers to *finite* and *deterministic* NAND-CIRC programs.

We can think of a pseudorandom generator as a "randomness amplifier." It takes an input s of ℓ bits chosen at random and expands these ℓ bits into an output r of $m > \ell$ *pseudorandom* bits. If ϵ is small enough then the pseudorandom bits will "look random" to any NAND-CIRC program that is not too big. Still, there are two questions we haven't answered:

- *What reason do we have to believe that pseudorandom generators with non-trivial parameters exist?*
- *Even if they do exist, why would such generators be useful to derandomize randomized algorithms?* After all, Definition 19.9 does not involve RNAND-TM or RNAND-RAM programs, but rather deterministic NAND-CIRC programs with no randomness and no loops.

We will now (partially) answer both questions. For the first question, let us come clean and confess we do not know how to *prove* that interesting pseudorandom generators exist. By *interesting* we mean pseudorandom generators that satisfy that ϵ is some small constant (say $\epsilon < 1/3$), $m > \ell$, and the function G itself can be computed in $poly(m)$ time. Nevertheless, Lemma 19.12 (whose statement and proof is deferred to the end of this chapter) shows that if we only drop the last condition (polynomial-time computability), then there do in fact exist pseudorandom generators where m is *exponentially larger* than ℓ.



At this point you might want to skip ahead and look at the *statement* of Lemma 19.12. However, since its *proof* is somewhat subtle, I recommend you defer reading it until you've finished reading the rest of this chapter.

19.3.2 From existence to constructivity

The fact that there *exists* a pseudorandom generator does not mean that there is one that can be efficiently computed. However, it turns out that we can turn complexity "on its head" and use the assumed *non existence* of fast algorithms for problems such as 3SAT to obtain pseudorandom generators that can then be used to transform randomized algorithms into deterministic ones. This is known as the *Hardness vs Randomness* paradigm. A number of results along those lines, most of which are outside the scope of this course, have led researchers to believe the following conjecture:

Optimal PRG conjecture: There is a polynomial-time computable function $PRG : \{0,1\}^* \rightarrow \{0,1\}$ that yields an *exponentially secure pseudorandom generator*.

Specifically, there exists a constant $\delta > 0$ such that for every ℓ and $m < 2^{\delta\ell}$, if we define $G : \{0,1\}^\ell \rightarrow \{0,1\}^m$ as $G(s)_i = PRG(s,i)$ for every $s \in \{0,1\}^\ell$ and $i \in [m]$, then G is a $(2^{\delta\ell}, 2^{-\delta\ell})$ pseudorandom generator.



The "optimal PRG conjecture" is worth while reading more than once. What it posits is that we can obtain (T,ϵ) pseudorandom generator G such that every output bit of G can be computed in time polynomial in the length ℓ of the input, where T is exponentially large in ℓ and ϵ is exponentially small in ℓ. (Note that

we could not hope for the entire output to be computable in ℓ, as just writing the output down will take too long.)

To understand why we call such a pseudorandom generator "optimal," it is a great exercise to convince yourself that, for example, there does not exist a $(2^{1.1\ell}, 2^{-1.1\ell})$ pseudorandom generator (in fact, the number δ in the conjecture must be smaller than 1). To see that we can't have $T \gg 2^\ell$, note that if we allow a NAND-CIRC program with much more than 2^ℓ lines then this NAND-CIRC program could "hardwire" inside it all the outputs of G on all its 2^ℓ inputs, and use that to distinguish between a string of the form $G(s)$ and a uniformly chosen string in $\{0,1\}^m$. To see that we can't have $\epsilon \ll 2^{-\ell}$, note that by guessing the input s (which will be successful with probability $2^{-2\ell}$), we can obtain a small (i.e., $O(\ell)$ line) NAND-CIRC program that achieves a $2^{-\ell}$ advantage in distinguishing a pseudorandom and uniform input. Working out these details is a highly recommended exercise.

We emphasize again that the optimal PRG conjecture is, as its name implies, a *conjecture*, and we still do not know how to *prove* it. In particular, it is stronger than the conjecture that $\mathbf{P} \neq \mathbf{NP}$. But we do have some evidence for its truth. There is a spectrum of different types of pseudorandom generators, and there are weaker assumptions than the optimal PRG conjecture that suffice to prove that $\mathbf{BPP} = \mathbf{P}$. In particular this is known to hold under the assumption that there exists a function $F \in \mathbf{TIME}(2^{O(n)})$ and $\epsilon > 0$ such that for every sufficiently large n, $F_{\restriction n}$ is not in $SIZE(2^{\epsilon n})$. The name "Optimal PRG conjecture" is non standard. This conjecture is sometimes known in the literature as the existence of *exponentially strong pseudorandom functions*.[5]

[5] A pseudorandom generator of the form we posit, where each output bit can be computed individually in time polynomial in the seed length, is commonly known as a *pseudorandom function generator*. For more on the many interesting results and connections in the study of *pseudorandomness*, see this monograph of Salil Vadhan.

19.3.3 Usefulness of pseudorandom generators

We now show that optimal pseudorandom generators are indeed very useful, by proving the following theorem:

Theorem 19.10 — Derandomization of BPP. Suppose that the optimal PRG conjecture is true. Then $\mathbf{BPP} = \mathbf{P}$.

Proof Idea:

The optimal PRG conjecture tells us that we can achieve *exponential expansion* of ℓ truly random coins into as many as $2^{\delta\ell}$ "pseudorandom coins." Looked at from the other direction, it allows us to reduce the need for randomness by taking an algorithm that uses m coins and converting it into an algorithm that only uses $O(\log m)$ coins. Now an algorithm of the latter type by can be made fully deterministic by enu-

merating over all the $2^{O(\log m)}$ (which is polynomial in m) possibilities for its random choices.

⋆

We now proceed with the proof details.

Proof of Theorem 19.10. Let $F \in$ **BPP** and let P be a NAND-TM program and a, b, c, d constants such that for every $x \in \{0,1\}^n$, $P(x)$ runs in at most $c \cdot n^d$ steps and $\Pr_{r\sim\{0,1\}^m}[P(x;r) = F(x)] \geq 2/3$. By "unrolling the loop" and hardwiring the input x, we can obtain for every input $x \in \{0,1\}^n$ a NAND-CIRC program Q_x of at most, say, $T = 10c \cdot n^d$ lines, that takes m bits of input and such that $Q(r) = P(x;r)$.

Now suppose that $G : \{0,1\}^\ell \to \{0,1\}$ is a $(T, 0.1)$ pseudorandom generator. Then we could deterministically estimate the probability $p(x) = \Pr_{r\sim\{0,1\}^m}[Q_x(r) = 1]$ up to 0.1 accuracy in time $O(T \cdot 2^\ell \cdot m \cdot cost(G))$ where $cost(G)$ is the time that it takes to compute a single output bit of G.

The reason is that we know that $\tilde{p}(x) = \Pr_{s\sim\{0,1\}^\ell}[Q_x(G(s)) = 1]$ will give us such an estimate for $p(x)$, and we can compute the probability $\tilde{p}(x)$ by simply trying all 2^ℓ possibillites for s. Now, under the optimal PRG conjecture we can set $T = 2^{\delta\ell}$ or equivalently $\ell = \frac{1}{\delta}\log T$, and our total computation time is polynomial in $2^\ell = T^{1/\delta}$. Since $T \leq 10c \cdot n^d$, this running time will be polynomial in n.

This completes the proof, since we are guaranteed that $\Pr_{r\sim\{0,1\}^m}[Q_x(r) = F(x)] \geq 2/3$, and hence estimating the probability $p(x)$ to within 0.1 accuracy is sufficient to compute $F(x)$. ■

19.4 P = NP AND BPP VS P

Two computational complexity questions that we cannot settle are:

- Is **P** = **NP**? Where we believe the answer is *negative*.
- Is **BPP** = **P**? Where we believe the answer is *positive*.

However we can say that the "conventional wisdom" is correct on at least one of these questions. Namely, if we're wrong on the first count, then we'll be right on the second one:

Theorem 19.11 — Sipser–Gács Theorem. If **P** = **NP** then **BPP** = **P**.



Before reading the proof, it is instructive to think why this result is not "obvious." If **P** = **NP** then given any randomized algorithm A and input x,

we will be able to figure out in polynomial time if there is a string $r \in \{0,1\}^m$ of random coins for A such that $A(xr) = 1$. The problem is that even if $\Pr_{r\in\{0,1\}^m}[A(xr) = F(x)] \geq 0.9999$, it can still be the case that even when $F(x) = 0$ there exists a string r such that $A(xr) = 1$.

The proof is rather subtle. It is much more important that you understand the *statement* of the theorem than that you follow all the details of the proof.

Proof Idea:

The construction follows the "quantifier elimination" idea which we have seen in Theorem 15.5. We will show that for every $F \in$ **BPP**, we can reduce the question of some input x satisfies $F(x) = 1$ to the question of whether a formula of the form $\exists_{u\in\{0,1\}^m} \forall_{v\in\{0,1\}^k} P(u, v)$ is true, where m, k are polynomial in the length of x and P is polynomial-time computable. By Theorem 15.5, if **P** $=$ **NP** then we can decide in polynomial time whether such a formula is true or false.

The idea behind this construction is that using amplification we can obtain a randomized algorithm A for computing F using m coins such that for every $x \in \{0,1\}^n$, if $F(x) = 0$ then the set $S \subseteq \{0,1\}^m$ of coins that make A output 1 is extremely tiny, and if $F(x) = 1$ then it is very large. Now in the case $F(x) = 1$, one can show that this means that there exists a small number k of "shifts" $s_0, \ldots, s_{k-1}$ such that the union of the sets $S \oplus s_i$ covers $\{0,1\}^m$, while in the case $F(x) = 0$ this union will always be of size at most $k|S|$ which is much smaller than 2^m. We can express the condition that there exists $s_0, \ldots, s_{k-1}$ such that $\cup_{i\in[k]}(S \oplus s_i) = \{0,1\}^m$ as a statement with a constant number of quantifiers.

⋆

Proof of Theorem 19.11. Let $F \in$ **BPP**. Using Theorem 19.4, there exists a polynomial-time algorithm A such that for every $x \in \{0,1\}^n$, $\Pr_{x\in\{0,1\}^m}[A(xr) = F(x)] \geq 1 - 2^{-n}$ where m is polynomial in n. In particular (since an exponential dominates a polynomial, and we can always assume n is sufficiently large), it holds that

$$\Pr_{x\in\{0,1\}^m}[A(xr) = F(x)] \geq 1 - \tfrac{1}{10m^2} \,. \tag{19.7}$$

Let $x \in \{0,1\}^n$, and let $S_x \subseteq \{0,1\}^m$ be the set $\{r \in \{0,1\}^m : A(xr) = 1\}$. By our assumption, if $F(x) = 0$ then $|S_x| \leq \frac{1}{10m^2}2^m$ and if $F(x) = 1$ then $|S_x| \geq (1 - \frac{1}{10m^2})2^m$.

For a set $S \subseteq \{0,1\}^m$ and a string $s \in \{0,1\}^m$, we define the set $S \oplus s$ to be $\{r \oplus s : r \in S\}$ where $\oplus$ denotes the XOR operation. That is, $S \oplus s$ is the set S "shifted" by s. Note that $|S \oplus s| = |S|$. (Please make sure that you see why this is true.)

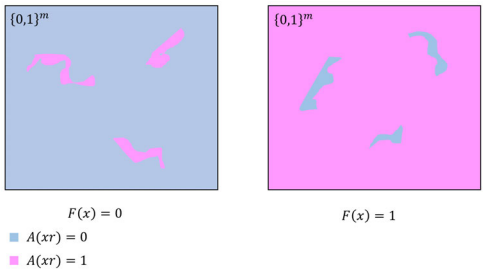


Figure 19.7: If $F \in$ **BPP** then through amplification we can ensure that there is an algorithm A to compute F on n-length inputs and using m coins such that $\Pr_{r\in\{0,1\}^m}[A(xr) \neq F(x)] \ll 1/poly(m)$. Hence if $F(x) = 1$ then almost all of the 2^m choices for r will cause $A(xr)$ to output 1, while if $F(x) = 0$ then $A(xr) = 0$ for almost all r's. To prove the Sipser–Gács Theorem we consider several "shifts" of the set $S \subseteq \{0,1\}^m$ of the coins r such that $A(xr) = 1$. If $F(x) = 1$ then we can find a set of k shifts $s_0, \ldots, s_{k-1}$ for which $\cup_{i\in[k]}(S \oplus s_i) = \{0,1\}^m$. If $F(x) = 0$ then for every such set $|cup_{i\in[k]}S_i| \leq k|S| \ll 2^m$. We can phrase the question of whether there is such a set of shift using a constant number of quantifiers, and so can solve it in polynomial time if **P** $=$ **NP**.

The heart of the proof is the following two claims:

CLAIM I: For every subset $S \subseteq \{0,1\}^m$, if $|S| \leq \frac{1}{1000m}2^m$, then for every $s_0, \ldots, s_{100m-1} \in \{0,1\}^m$, $\cup_{i\in[100m]}(S \oplus s_i) \subsetneq \{0,1\}^m$.

CLAIM II: For every subset $S \subseteq \{0,1\}^m$, if $|S| \geq \frac{1}{2}2^m$ then there exist a set of string $s_0, \ldots, s_{100m-1}$ such that $\cup_{i\in[100m]}(S \oplus s_i) \subsetneq \{0,1\}^m$.

CLAIM I and CLAIM II together imply the theorem. Indeed, they mean that under our assumptions, for every $x \in \{0,1\}^n$, $F(x) = 1$ if and only if

$$\exists_{s_0,\ldots,s_{100m-1}\in\{0,1\}^m} \cup_{i\in[100m]} (S_x \oplus s_i) = \{0,1\}^m \tag{19.8}$$

which we can re-write as

$$\exists_{s_0,\ldots,s_{100m-1}\in\{0,1\}^m} \forall_{w\in\{0,1\}^m} \Big(w \in (S_x \oplus s_0) \vee w \in (S_x \oplus s_1) \vee \cdots w \in (S_x \oplus s_{100m-1}) \Big) \tag{19.9}$$

or equivalently

$$\exists_{s_0,\ldots,s_{100m-1}\in\{0,1\}^m} \forall_{w\in\{0,1\}^m} \Big(A(x(w \oplus s_0)) = 1 \vee A(x(w \oplus s_1)) = 1 \vee \cdots \vee A(x(w \oplus s_{100m-1})) = 1 \Big) \tag{19.10}$$

which (since A is computable in polynomial time) is exactly the type of statement shown in Theorem 15.5 to be decidable in polynomial time if **P** = **NP**.

We see that all that is left is to prove **CLAIM I** and **CLAIM II**. **CLAIM I** follows immediately from the fact that

$$\cup_{i\in[100m-1]}|S_x \oplus s_i| \leq \sum_{i=0}^{100m-1} |S_x \oplus s_i| = \sum_{i=0}^{100m-1} |S_x| = 100m|S_x| \,. \tag{19.11}$$

To prove **CLAIM II**, we will use a technique known as the *probabilistic method* (see the proof of Lemma 19.12 for a more extensive discussion). Note that this is a completely different use of probability than in the theorem statement, we just use the methods of probability to prove an *existential* statement.

Proof of CLAIM II: Let $S \subseteq \{0,1\}^m$ with $|S| \geq 0.5 \cdot 2^m$ be as in the claim's statement. Consider the following probabilistic experiment: we choose $100m$ random shifts $s_0, \ldots, s_{100m-1}$ independently at random in $\{0,1\}^m$, and consider the event *GOOD* that $\cup_{i\in[100m]}(S \oplus s_i) = \{0,1\}^m$. To prove CLAIM II it is enough to show that Pr[*GOOD*] > 0, since that means that in particular there must *exist* shifts $s_0, \ldots, s_{100m-1}$ that satisfy this condition.

For every $z \in \{0,1\}^m$, define the event BAD_z to hold if $z \notin \cup_{i\in[100m-1]}(S \oplus s_i)$. The event *GOOD* holds if BAD_z fails for every $z \in \{0,1\}^m$, and so our goal is to prove that Pr$[\cup_{z\in\{0,1\}^m} BAD_z] < 1$. By

the union bound, to show this, it is enough to show that $\Pr[BAD_z] < 2^{-m}$ for every $z \in \{0,1\}^m$. Define the event BAD_z^i to hold if $z \notin S \oplus s_i$. Since every shift s_i is chosen independently, for every fixed z the events $BAD_z^0, \ldots, BAD_z^{100m-1}$ are mutually independent,[6] and hence

$$\Pr[BAD_z] = \Pr[\cap_{i\in[100m-1]} BAD_z^i] = \prod_{i=0}^{100m-1} \Pr[BAD_z^i] \; . \tag{19.12}$$

So this means that the result will follow by showing that $\Pr[BAD_z^i] \leq \frac{1}{2}$ for every $z \in \{0,1\}^m$ and $i \in [100m]$ (as that would allow to bound the righthand side of (19.12) by 2^{-100m}). In other words, we need to show that for every $z \in \{0,1\}^m$ and set $S \subseteq \{0,1\}^m$ with $|S| \geq \frac{1}{2}2^m$,

$$\Pr_{s\in\{0,1\}^m}[z \in S \oplus s] \geq \tfrac{1}{2} \; . \tag{19.13}$$

To show this, we observe that $z \in S \oplus s$ if and only if $s \in S \oplus z$ (can you see why). Hence we can rewrite the probability on the lefthand side of (19.13) as $\Pr_{s\in\{0,1\}^m}[s \in S \oplus z]$ which simply equals $|S \oplus z|/2^m = |S|/2^m \geq 1/2$! This concludes the proof of **CLAIM I** and hence of Theorem 19.11.

■

[6] The condition of independence here is subtle. It is *not* the case that all of the $2^m \times 100m$ events $\{BAD_z^i\}_{z\in\{0,1\}^m, i\in[100m]}$ are mutually independent. Only for a fixed $z \in \{0,1\}^m$, the $100m$ events of the form BAD_z^i are mutually independent.

19.5 NON-CONSTRUCTIVE EXISTENCE OF PSEUDORANDOM GENERATORS (ADVANCED, OPTIONAL)

We now show that, if we don't insist on *constructivity* of pseudorandom generators, then we can show that there exists pseudorandom generators with output that *exponentially larger* in the input length.

Lemma 19.12 — Existence of inefficient pseudorandom generators. There is some absolute constant C such that for every ϵ, T, if $\ell > C(\log T + \log(1/\epsilon))$ and $m \leq T$, then there is an (T,ϵ) pseudorandom generator $G : \{0,1\}^\ell \to \{0,1\}^m$.

Proof Idea:

The proof uses an extremely useful technique known as the "probabilistic method" which is not too hard mathematically but can be confusing at first.[7] The idea is to give a "non constructive" proof of existence of the pseudorandom generator G by showing that if G was chosen at random, then the probability that it would be a valid (T,ϵ) pseudorandom generator is positive. In particular this means that there *exists* a single G that is a valid (T,ϵ) pseudorandom generator. The probabilistic method is just a *proof technique* to demonstrate the existence of such a function. Ultimately, our goal is to show the existence of a *deterministic* function G that satisfies the condition.

[7] There is a whole (highly recommended) book by Alon and Spencer devoted to this method.

⋆

The above discussion might be rather abstract at this point, but would become clearer after seeing the proof.

Proof of Lemma 19.12. Let ϵ, T, ℓ, m be as in the lemma's statement. We need to show that there exists a function $G : \{0,1\}^\ell \to \{0,1\}^m$ that "fools" every T line program P in the sense of (19.6). We will show that this follows from the following claim:

Claim I: For every fixed NAND-CIRC program P, if we pick $G : \{0,1\}^\ell \to \{0,1\}^m$ *at random* then the probability that (19.6) is violated is at most 2^{-T^2}.

Before proving Claim I, let us see why it implies Lemma 19.12. We can identify a function $G : \{0,1\}^\ell \to \{0,1\}^m$ with its "truth table" or simply the list of evaluations on all its possible 2^ℓ inputs. Since each output is an m bit string, we can also think of G as a string in $\{0,1\}^{m\cdot 2^\ell}$. We define $\mathcal{F}_\ell^m$ to be the set of all functions from $\{0,1\}^\ell$ to $\{0,1\}^m$. As discussed above we can identify $\mathcal{F}_\ell^m$ with $\{0,1\}^{m\cdot 2^\ell}$ and choosing a random function $G \sim \mathcal{F}_\ell^m$ corresponds to choosing a random $m \cdot 2^\ell$-long bit string.

For every NAND-CIRC program P let B_P be the event that, if we choose G at random from $\mathcal{F}_\ell^m$ then (19.6) is violated with respect to the program P. It is important to understand what is the sample space that the event B_P is defined over, namely this event depends on the choice of G and so B_P is a subset of $\mathcal{F}_\ell^m$. An equivalent way to define the event B_P is that it is the subset of all functions mapping $\{0,1\}^\ell$ to $\{0,1\}^m$ that violate (19.6), or in other words:

$$B_P = \left\{ G \in \mathcal{F}_\ell^m \;\middle|\; \left| \tfrac{1}{2^\ell} \sum_{s\in\{0,1\}^\ell} P(G(s)) - \tfrac{1}{2^m} \sum_{r\in\{0,1\}^m} P(r) \right| > \epsilon \right\} \tag{19.14}$$

(We've replaced here the probability statements in (19.6) with the equivalent sums so as to reduce confusion as to what is the sample space that B_P is defined over.)

To understand this proof it is crucial that you pause here and see how the definition of B_P above corresponds to (19.14). This may well take re-reading the above text once or twice, but it is a good exercise at parsing probabilistic statements and learning how to identify the *sample space* that these statements correspond to.

Now, we've shown in Theorem 5.7 that up to renaming variables (which makes no difference to program's functionality) there are $2^{O(T \log T)}$ NAND-CIRC programs of at most T lines. Since $T \log T < T^2$ for sufficiently large T, this means that if Claim I is true, then by the union bound it holds that the probability of the union of

B_P over *all* NAND-CIRC programs of at most T lines is at most $2^{O(T \log T)} 2^{-T^2} < 0.1$ for sufficiently large T. What is important for us about the number 0.1 is that it is smaller than 1. In particular this means that there *exists* a single $G^* \in \mathcal{F}_\ell^m$ such that G^* *does not* violate (19.6) with respect to any NAND-CIRC program of at most T lines, but that precisely means that G^* is a (T, ϵ) pseudorandom generator.

Hence to conclude the proof of Lemma 19.12, it suffices to prove Claim I. Choosing a random $G : \{0,1\}^\ell \to \{0,1\}^m$ amounts to choosing $L = 2^\ell$ random strings $y_0, \ldots, y_{L-1} \in \{0,1\}^m$ and letting $G(x) = y_x$ (identifying $\{0,1\}^\ell$ and $[L]$ via the binary representation). This means that proving the claim amounts to showing that for every fixed function $P : \{0,1\}^m \to \{0,1\}$, if $L > 2^{C(\log T + \log \epsilon)}$ (which by setting $C > 4$, we can ensure is larger than $10T^2/\epsilon^2$) then the probability that

$$\left| \frac{1}{L} \sum_{i=0}^{L-1} P(y_i) - \Pr_{s \sim \{0,1\}^m}[P(s) = 1] \right| > \epsilon \tag{19.15}$$

is at most 2^{-T^2}.

(**??**) follows directly from the Chernoff bound. Indeed, if we let for every $i \in [L]$ the random variable X_i denote $P(y_i)$, then since $y_0, \ldots, y_{L-1}$ is chosen independently at random, these are independently and identically distributed random variables with mean $\mathbb{E}_{y \sim \{0,1\}^m}[P(y)] = \Pr_{y \sim \{0,1\}^m}[P(y) = 1]$ and hence the probability that they deviate from their expectation by ϵ is at most $2 \cdot 2^{-\epsilon^2 L/2}$.

■



Lecture Recap

- We can model randomized algorithms by either adding a special "coin toss" operation or assuming an extra randomly chosen input.
- The class **BPP** contains the set of Boolean functions that can be computed by polynomial time randomized algorithms.
- We know that $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{EXP}$.
- We also know that $\mathbf{BPP} \subseteq \mathbf{P}_{/\mathbf{poly}}$.
- The relation between **BPP** and **NP** is not known, but we do know that if $\mathbf{P} = \mathbf{NP}$ then $\mathbf{BPP} = \mathbf{P}$.
- Pseudorandom generators are objects that take a short random "seed" and expand it to a much longer output that "appears random" for efficient algorithms. We conjecture that exponentially strong pseudorandom generators exist. Under this conjecture, $\mathbf{BPP} = \mathbf{P}$.

19.6 EXERCISES



Remark 19.13 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

19.7 BIBLIOGRAPHICAL NOTES

19.8 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

19.9 ACKNOWLEDGEMENTS

V

ADVANCED TOPICS

Learning Objectives:

- Definition of perfect secrecy
- The one-time pad encryption scheme
- Necessity of long keys for perfect secrecy
- Computational secrecy and the derandomized one-time pad.
- Public key encryption
- A taste of advanced topics

20 Cryptography

> *"Human ingenuity cannot concoct a cipher which human ingenuity cannot resolve."*, Edgar Allen Poe, 1841

> *"I hope my handwriting, etc. do not give the impression I am just a crank or circle-squarer…. The significance of this conjecture [that certain encryption schemes are exponentially secure against key recovery attacks] .. is that it is quite feasible to design ciphers that are effectively unbreakable."*, John Nash, letter to the NSA, 1955.

> *""Perfect Secrecy" is defined by requiring of a system that after a cryptogram is intercepted by the enemy the a posteriori probabilities of this cryptogram representing various messages be identically the same as the a priori probabilities of the same messages before the interception. It is shown that perfect secrecy is possible but requires, if the number of messages is finite, the same number of possible keys."*, Claude Shannon, 1945

> *"We stand today on the brink of a revolution in cryptography."*, Whitfeld Diffie and Martin Hellman, 1976

Cryptography - the art or science of "secret writing" - has been around for several millenia, and for almost all of that time Edgar Allan Poe's quote above held true. Indeed, the history of cryptography is littered with the figurative corpses of cryptosystems believed secure and then broken, and sometimes with the actual corpses of those who have mistakenly placed their faith in these cryptosystems.

Yet, something changed in the last few decades, which is the "revolution" alluded to (and to a large extent initiated by) Diffie and Hellman's 1976 paper quoted above. New cryptosystems have been found that have not been broken despite being subjected to immense efforts involving both human ingenuity and computational power on a scale

that completely dwarves the "code breakers" of Poe's time. Even more amazingly, these cryptosystem are not only seemingly unbreakable, but they also achieve this under much harsher conditions. Not only do today's attackers have more computational power but they also have more data to work with. In Poe's age, an attacker would be lucky if they got access to more than a few encryptions of known messages. These days attackers might have massive amounts of data- terabytes or more - at their disposal. In fact, with *public key* encryption, an attacker can generate as many ciphertexts as they wish.

The key to this success has been a clearer understanding of both how to *define* security for cryptographic tools and how to relate this security to *concrete computational problems*. Cryptography is a vast and continuously changing topic, but we will touch on some of these issues in this chapter.

20.1 CLASSICAL CRYPTOSYSTEMS

A great many cryptosystems have been devised and broken throughout the ages. Let us recount just some of these stories. In 1587, Mary the queen of Scots, and the heir to the throne of England, wanted to arrange the assassination of her cousin, queen Elisabeth I of England, so that she could ascend to the throne and finally escape the house arrest under which she had been for the last 18 years. As part of this complicated plot, she sent a coded letter to Sir Anthony Babington.

Mary used what's known as a *substitution cipher* where each letter is transformed into a different obscure symbol (see Fig. 20.1). At a first look, such a letter might seem rather inscrutable- a meaningless sequence of strange symbols. However, after some thought, one might recognize that these symbols *repeat* several times and moreover that different symbols repeat with different frequencies. Now it doesn't take a large leap of faith to assume that perhaps each symbol corresponds to a different letter and the more frequent symbols correspond to letters that occur in the alphabet with higher frequency. From this observation, there is a short gap to completely breaking the cipher, which was in fact done by queen Elisabeth's spies who used the decoded letters to learn of all the co-conspirators and to convict queen Mary of treason, a crime for which she was executed. Trusting in superficial security measures (such as using "inscrutable" symbols) is a trap that users of cryptography have been falling into again and again over the years. (As in many things, this is the subject of a great XKCD cartoon, see Fig. 20.2.)

The Vigenère cipher is named after Blaise de Vigenère who described it in a book in 1586 (though it was invented earlier by Bellaso). The idea is to use a collection of substitution cyphers - if there are n different ciphers then the first letter of the plaintext is encoded with

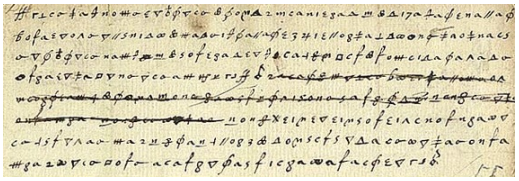


Figure 20.1: Snippet from encrypted communication between queen Mary and Sir Babington

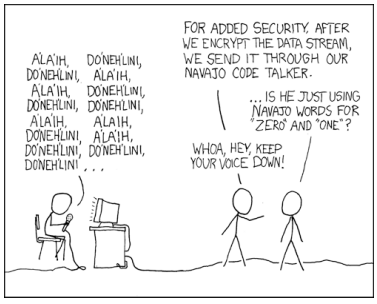


Figure 20.2: XKCD's take on the added security of using uncommon symbols

the first cipher, the second with the second cipher, the nth with the nth cipher, and then the $n + 1^{st}$ letter is again encoded with the first cipher. The key is usually a word or a phrase of n letters, and the ith substitution cipher is obtained by shifting each letter k_i positions in the alphabet. This "flattens" the frequencies and makes it much harder to do frequency analysis, which is why this cipher was considered "unbreakable" for 300+ years and got the nickname "le chiffre indéchiffrable" ("the unbreakable cipher"). Nevertheless, Charles Babbage cracked the Vigenère cipher in 1854 (though he did not publish it). In 1863 Friedrich Kasiski broke the cipher and published the result. The idea is that once you guess the length of the cipher, you can reduce the task to breaking a simple substitution cipher which can be done via frequency analysis (can you see why?). Confederate generals used Vigenère regularly during the civil war, and their messages were routinely cryptanalzed by Union officers.



Figure 20.3: Confederate Cipher Disk for implementing the Vigenère cipher

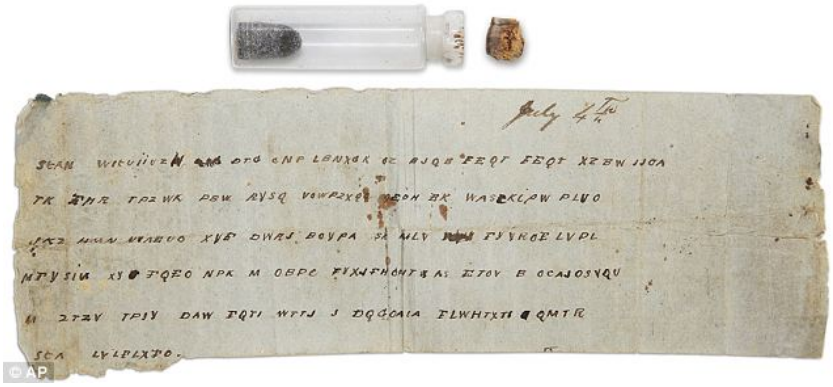


Figure 20.4: Confederate encryption of the message "Gen'l Pemberton: You can expect no help from this side of the river. Let Gen'l Johnston know, if possible, when you can attack the same point on the enemy's lines. Inform me also and I will endeavor to make a diversion. I have sent some caps. I subjoin a despatch from General Johnston."

The *Enigma* cipher was a mechanical cipher (looking like a typewriter, see Fig. 20.5) where each letter typed would get mapped into a different letter depending on the (rather complicated) key and current state of the machine which had several rotors that rotated at different paces. An identically wired machine at the other end could be used to decrypt. Just as many ciphers in history, this has also been believed by the Germans to be "impossible to break" and even quite late in the war they refused to believe it was broken despite mounting evidence to that effect. (In fact, some German generals refused to believe it was broken even *after* the war.) Breaking Enigma was an heroic effort which was initiated by the Poles and then completed by the British at Bletchley Park, with Alan Turing (of the Turing machines) playing a key role. As part of this effort the Brits built arguably the world's first large scale mechanical computation devices (though they looked more similar to washing machines than to iPhones). They were also helped along the way by some quirks and errors of the German operators. For example, the fact that their messages ended with "Heil Hitler" turned out to be quite useful.

Here is one entertaining anecdote: the Enigma machine would never map a letter to itself. In March 1941, Mavis Batey, a cryptanalyst at Bletchley Park received a very long message that she tried to decrypt. She then noticed a curious property— the message did *not* contain the letter "L".[1] She realized that the probability that no "L"'s appeared in the message is too small for this to happen by chance. Hence she surmised that the original message must have been composed *only* of L's. That is, it must have been the case that the operator, perhaps to test the machine, have simply sent out a message where he repeatedly pressed the letter "L". This observation helped her decode the next message, which helped inform of a planned Italian attack and secure a resounding British victory in what became known as "the Battle of Cape Matapan". Mavis also helped break another Enigma machine. Using the information she provided, the Brits were able to feed the Germans with the false information that the main allied invasion would take place in Pas de Calais rather than on Normandy.

In the words of General Eisenhower, the intelligence from Bletchley park was of "priceless value". It made a huge difference for the Allied war effort, thereby shortening World War II and saving millions of lives. See also this inteview with Sir Harry Hinsley.



Figure 20.5: In the *Enigma* mechanical cipher the secret key would be the settings of the rotors and internal wires. As the operator types up their message, the encrypted version appeared in the display area above, and the internal state of the cipher was updated (and so typing the same letter twice would generally result in two different letters output). Decrypting follows the same process: if the sender and receiver are using the same key then typing the ciphertext would result in the plaintext appearing in the display.

[1] Here is a nice exercise: compute (up to an order of magnitude) the probability that a 50-letter long message composed of random letters will end up not containing the letter "L".

20.2 DEFINING ENCRYPTION

Many of the troubles that cryptosystem designers faced over history (and still face!) can be attributed to not properly defining or understanding what are the goals they want to achieve in the first place. Let us focus on the setting of *private key encryption*.[2] A *sender* (traditionally

[2] If you don't know what "private key" means, you can ignore this adjective for now. For thousands of years, "private key encryption" was synonymous with encryption. Only in the 1970's was the concept of *public key encryption* invented, see Definition 20.10.

called "Alice") wants to send a message (known also as a *plaintext*) $x \in \{0,1\}^*$ to a *receiver* (traditionally called "Bob"). They would like their message to be kept secret from an *adversary* who listens in or "eavesdrops" on the communication channel (and is traditionally called "Eve").

Alice and Bob share a *secret key* $k \in \{0,1\}^*$. Alice uses the key k to "scramble" or *encrypt* the plaintext x into a *ciphertext* y, and Bob uses the key k to "unscramble" or *decrypt* the ciphertext y back into the plaintext x. This motivates the following definition which attempts to capture what it means for an encryption scheme to be *valid* or "make sense", regardless of whether or not it is *secure*:

Definition 20.1 — Valid encryption scheme. Let $L : \mathbb{N} \to \mathbb{N}$ be some function. A pair of polynomial-time computable functions (E, D) mapping strings to strings is a *valid private key encryption scheme* (or *encryption scheme* for short) with plaintext length function $L(\cdot)$ if for every $k \in \{0,1\}^n$ and $x \in \{0,1\}^{L(n)}$,

$$D(k, E(k, x)) = x \ . \tag{20.1}$$

We also require that our encryption schemes are *ciphertext length regular* in the sense that all ciphertexts corresponding to keys of the same length are of the same length: there is some function $C : \mathbb{N} \to \mathbb{N}$ such that for every $k \in \{0,1\}^n$ and $x \in \{0,1\}^{L(n)}$, $|E(k,x)| = C(n)$. [3]

We will often write the first input (i.e., the key) to the encryption and decryption as a subscript and so can write (20.1) also as $D_k(E_k(x)) = x$.

[3] We call the function $L : \mathbb{N} \to \mathbb{N}$ the *length function* of (E, D). The "ciphertext length regularity" condition is added for technical convenience and is not at all important. You can ignore it in a first reading.

20.3 DEFINING SECURITY OF ENCRYPTION

Definition 20.1 says nothing about the *security* of E and D, and even allows the trivial encryption scheme that ignores the key altogether and sets $E_k(x) = x$ for every x. Defining security is not a trivial matter.

P

You would appreciate the subtleties of defining security of encryption more if at this point you take a five minute break from reading, and try (possibly with a partner) to brainstorm on how you would mathematically define the notion that an encryption scheme is *secure*, in the sense that it protects the secrecy of the plaintext x.

Throughout history, many attacks on cryptosystems were rooted in the cryptosystem designers' reliance on "security through

obscurity"— trusting that the fact their *methods* are not known to their enemy will protect them from being broken. This is a faulty assumption - if you reuse a method again and again (even with a different key each time) then eventually your adversaries will figure out what you are doing. And if Alice and Bob meet frequently in a secure location to decide on a new method, they might as well take the opportunity to exchange their secrets. These considerations led Auguste Kerckhoffs in 1883 to state the following principle:

> *A cryptosystem should be secure even if everything about the system, except the key, is public knowledge.* [4]

[4] The actual quote is "Il faut qu'il n'exige pas le secret, et qu'il puisse sans inconvénient tomber entre les mains de l'ennemi" loosely translated as "The system must not require secrecy and can be stolen by the enemy without causing trouble". According to Steve Bellovin the NSA version is "assume that the first copy of any device we make is shipped to the Kremlin".

Why is it OK to assume the key is secret and not the algorithm? Because we can always choose a fresh key. But of course that won't help us much if our key is "1234" or "passw0rd!". In fact, if you use *any* deterministic algorithm to choose the key then eventually your adversary will figure this out. Therefore for security we must choose the key at *random* and can restate Kerckhoffs's principle as follows:

> *There is no secrecy without randomness*

This is such a crucial point that is worth repeating:

> *There is no secrecy without randomness*

At the heart of every cryptographic scheme there is a secret key, and the secret key is always chosen at random. A corollary of that is that to understand cryptography, you need to know probability theory.

R

Remark 20.2 — Randomness in the real world. Choosing the secrets for cryptography requires generating randomness, which is often done by measuring some "unpredictable" or "high entropy" data, and then applying hash functions to the result to "extract" a uniformly random string. Great care must be taken in doing this, and randomness generators often turn out to be the Achilles heel of secure systems.

In 2006 a programmer removed a line of code from the procedure to generate entropy in OpenSSL package distributed by Debian since it caused a warning in some automatic verification code. As a result for two years (until this was discovered) all the randomness generated by this procedure used only the process ID as an "unpredictable" source. This means that all communication done by users in that period is fairly easily breakable (and in particular, if some entities recorded that communication they could break it also retroactively). See XKCD's take on that incident.

In 2012 two separate teams of researchers scanned a large number of RSA keys on the web and found out that about 4 percent of them are easy to break. The main issue were devices such as routers, internet-connected printers and such. These devices sometimes run variants of Linux- a desktop operating system- but without a hard drive, mouse or keyboard, they don't have access to many of the entropy sources that desktop have. Coupled with some good old fashioned ignorance of cryptography and software bugs, this led to many keys that are downright trivial to break, see this blog post and this web page for more details.

Since randomness is so crucial to security, breaking the procedure to generate randomness can lead to a complete break of the system that uses this randomness. Indeed, the Snowden documents, combined with observations of Shumow and Ferguson, strongly suggest that the NSA has deliberately inserted a *trapdoor* in one of the pseudorandom generators published by the National Institute of Standards and Technologies (NIST). Fortunately, this generator wasn't widely adapted but apparently the NSA did pay 10 million dollars to RSA security so the latter would make this generator their default option in their products.

20.4 PERFECT SECRECY

If you think about encryption scheme security for a while, you might come up with the following principle for defining security: *"An encryption scheme is secure if it is not possible to recover the key k from $E_k(x)$"*. However, a moment's thought shows that the key is not really what we're trying to protect. After all, the whole point of an encryption is to protect the confidentiality of the *plaintext* x. So, we can try to define that *"an encryption scheme is secure if it is not possible to recover the plaintext x from $E_k(x)$"*. Yet it is not clear what this means either. Suppose that an encryption scheme reveals the first 10 bits of the plaintext x. It might still not be possible to recover x completely, but on an intuitive level, this seems like it would be extremely unwise to use such an encryption scheme in practice. Indeed, often even *partial information* about the plaintext is enough for the adversary to achieve its goals.

The above thinking led Shannon in 1945 to formalize the notion of *perfect secrecy*, which is that an encryption reveals absolutely nothing about the message. There are several equivalent ways to define it, but perhaps the cleanest one is the following:

Definition 20.3 — Perfect secrecy. A valid encryption scheme (E, D) with length $L(\cdot)$ is *perfectly secret* if for every $n \in \mathbb{N}$ and plaintexts

$x, x' \in \{0,1\}^{L(n)}$, the following two distributions Y and Y' over $\{0,1\}^*$ are identical:

- Y is obtained by sampling $k \sim \{0,1\}^n$ and outputting $E_k(x)$.
- Y' is obtained by sampling $k \sim \{0,1\}^n$ and outputting $E_k(x')$.

P

This definition might take more than one reading to parse. Try to think of how this condition would correspond to your intuitive notion of "learning no information" about x from observing $E_k(x)$, and to Shannon's quote in the beginning of this chapter.

In particular, suppose that you knew ahead of time that Alice sent either an encryption of x or an encryption of x'. Would you learn anything new from observing the encryption of the message that Alice actually sent? It may help you to look at Fig. 20.6.

20.4.1 Example: Perfect secrecy in the battlefield

To understand Definition 20.3, suppose that Alice sends only one of two possible messages: "attack" or "retreat", which we denote by x_0 and x_1 respectively, and that she sends each one of those messages with probability $1/2$. Let us put ourselves in the shoes of *Eve*, the eavesdropping adversary. A priori we would have guessed that Alice sent either x_0 or x_1 with probability $1/2$. Now we observe $y = E_k(x_i)$ where k is a uniformly chosen key in $\{0,1\}^n$. How does this new information cause us to update our beliefs on whether Alice sent the plaintext x_0 or the plaintext x_1?

P

Before reading the next paragraph, you might want to try the analysis yourself. You may find it useful to look at the Wikipedia entry on Bayesian Inference or these MIT lecture notes.

Let us define $p_0(y)$ to be the probability (taken over $k \sim \{0,1\}^n$) that $y = E_k(x_0)$ and similarly $p_1(y)$ to be $\Pr_{k\sim\{0,1\}^n}[y = E_k(x_1)]$. Note that, since Alice chooses the message to send at random, our a priori probability for observing y is $\frac{1}{2}p_0(y) + \frac{1}{2}p_1(y)$. However, as per Definition 20.3, the perfect secrecy condition guarantees that $p_0(y) = p_1(y)$! Let us denote the number $p_0(y) = p_1(y)$ by p. By the formula for conditional probability, the probability that Alice sent the message x_0 conditioned on our observation y is simply[5]

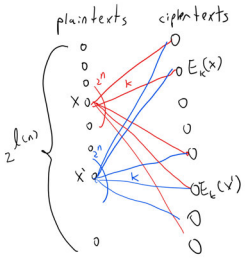


Figure 20.6: For any key length n, we can visualize an encryption scheme (E, D) as a graph with a vertex for every one of the $2^{L(n)}$ possible plaintexts and for every one of the ciphertexts in $\{0,1\}^*$ of the form $E_k(x)$ for $k \in \{0,1\}^n$ and $x \in \{0,1\}^{L(n)}$. For every plaintext x and key k, we add an edge labeled k between x and $E_k(x)$. By the validity condition, if we pick any fixed key k, the map $x \mapsto E_k(x)$ must be one-to-one. The condition of perfect secrecy simply corresponds to requiring that every two plaintexts x and x' have exactly the same set of neighbors (or multi-set, if there are parallel edges).

[5] The equation (20.2) is a special case of *Bayes' rule* which, although a simple restatement of the formula for conditional probability, is an extremely important and widely used tool in statistics and data analysis.

$$\Pr[i = 0 | y = E_k(x_i)] = \frac{\Pr[i = 0 \wedge y = E_k(x_i)]}{\Pr[y = E_k(x)]} \,. \tag{20.2}$$

Since the probability that $i = 0$ and y is the ciphertext $E_k(0)$ is equal to $\frac{1}{2} \cdot p_0(y)$, and the a priori probability of observing y is $\frac{1}{2}p_0(y) + \frac{1}{2}p_1(y)$, we can rewrite (20.2) as

$$\Pr[i = 0 | y = E_k(x_i)] = \frac{\frac{1}{2}p_0(y)}{\frac{1}{2}p_0(y) + \frac{1}{2}p_1(y)} = \frac{p}{p+p} = \frac{1}{2} \tag{20.3}$$

using the fact that $p_0(y) = p_1(y) = p$. This means that observing the ciphertext y did not help us at all! We still would not be able to guess whether Alice sent "attack" or "retreat" with better than 50/50 odds!

This example can be vastly generalized to show that perfect secrecy is indeed "perfect" in the sense that observing a ciphertext gives Eve *no additional information* about the plaintext beyond her a priori knowledge.

20.4.2 Constructing perfectly secret encryption

Perfect secrecy is an extremely strong condition, and implies that an eavesdropper does not learn *any* information from observing the ciphertext. You might think that an encryption scheme satisfying such a strong condition will be impossible, or at least extremely complicated, to achieve. However it turns out we can in fact obtain perfectly secret encryption scheme fairly easily. Such a scheme for two-bit messages is illustrated in Fig. 20.7

In fact, this can be generalized to any number of bits:

Theorem 20.4 — One Time Pad (Vernam 1917, Shannon 1949). There is a perfectly secret valid encryption scheme (E, D) with $L(n) = n$.

Proof Idea:

Our scheme is the one-time pad also known as the "Vernam Cipher", see Fig. 20.8. The encryption is exceedingly simple: to encrypt a message $x \in \{0,1\}^n$ with a key $k \in \{0,1\}^n$ we simply output $x \oplus k$ where $\oplus$ is the bitwise XOR operation that outputs the string corresponding to XORing each coordinate of x and k.

⋆

Proof of Theorem 20.4. For two binary strings a and b of the same length n, we define $a \oplus b$ to be the string $c \in \{0,1\}^n$ such that $c_i = a_i + b_i \mod 2$ for every $i \in [n]$. The encryption scheme (E, D) is defined as follows: $E_k(x) = x \oplus k$ and $D_k(y) = y \oplus k$. By the associative law of addition (which works also modulo two), $D_k(E_k(x)) = (x \oplus k) \oplus k = x \oplus (k \oplus k) = x \oplus 0^n = x$, using the fact

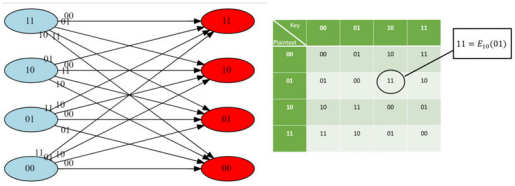


Figure 20.7: A perfectly secret encryption scheme for two-bit keys and messages. The blue vertices represent plaintexts and the red vertices represent ciphertexts, each edge mapping a plaintext x to a ciphertext $y = E_k(x)$ is labeled with the corresponding key k. Since there are four possible keys, the degree of the graph is four and it is in fact a complete bipartite graph. The encryption scheme is valid in the sense that for every $k \in \{0,1\}^2$, the map $x \mapsto E_k(x)$ is one-to-one, which in other words means that the set of edges labeled with k is a *matching*.

that for every bit $\sigma \in \{0,1\}$, $\sigma + \sigma \mod 2 = 0$ and $\sigma + 0 = \sigma \mod 2$. Hence (E, D) form a valid encryption.

To analyze the perfect secrecy property, we claim that for every $x \in \{0,1\}^n$, the distribution $Y_x = E_k(x)$ where $k \sim \{0,1\}^n$ is simply the uniform distribution over $\{0,1\}^n$, and hence in particular the distributions Y_x and $Y_{x'}$ are identical for every $x, x' \in \{0,1\}^n$. Indeed, for every particular $y \in \{0,1\}^n$, the value y is output by Y_x if and only if $y = x \oplus k$ which holds if and only if $k = x \oplus y$. Since k is chosen uniformly at random in $\{0,1\}^n$, the probability that k happens to equal $x \oplus y$ is exactly 2^{-n}, which means that every string y is output by Y_x with probability 2^{-n}.

■

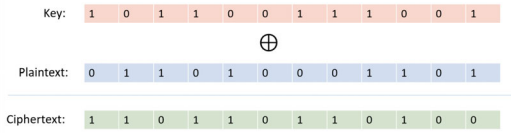


Figure 20.8: In the *one time pad* encryption scheme we encrypt a plaintext $x \in \{0,1\}^n$ with a key $k \in \{0,1\}^n$ by the ciphertext $x \oplus k$ where $\oplus$ denotes the bitwise XOR operation.

P

The argument above is quite simple but is worth reading again. To understand why the one-time pad is perfectly secret, it is useful to envision it as a bipartite graph as we've done in Fig. 20.7. (In fact the encryption scheme of Fig. 20.7 is precisely the one-time pad for $n = 2$.) For every n, the one-time pad encryption scheme corresponds to a bipartite graph with 2^n vertices on the "left side" corresponding to the plaintexts in $\{0,1\}^n$ and 2^n vertices on the "right side" corresponding to the ciphertexts $\{0,1\}^n$. For every $x \in \{0,1\}^n$ and $k \in \{0,1\}^n$, we connect x to the vertex $y = E_k(x)$ with an edge that we label with k. One can see that this is the complete bipartite graph, where every vertex on the left is connected to *all* vertices on the right. In particular this means that for every left vertex x, the distribution on the ciphertexts obtained by taking a random $k \in \{0,1\}^n$ and going to the neighbor of x on the edge labeled k is the uniform distribution over $\{0,1\}^n$. This ensures the perfect secrecy condition.

20.5 NECESSITY OF LONG KEYS

So, does Theorem 20.4 give the final word on cryptography, and means that we can all communicate with perfect secrecy and live happily ever after? No it doesn't. While the one-time pad is efficient, and gives perfect secrecy, it has one glaring disadvantage: to communicate n bits you need to store a key of length n. In contrast, practically used cryptosystems such as AES-128 have a short key of 128 bits (i.e., 16 bytes) that can be used to protect terabytes or more of communication! Imagine that we all needed to use the one time pad. If that was the case, then if you had to communicate with m people, you would have to maintain (securely!) m huge files that are each as long as the

length of the maximum total communication you expect with that person. Imagine that every time you opened an account with Amazon, Google, or any other service, they would need to send you in the mail (ideally with a secure courier) a DVD full of random numbers, and every time you suspected a virus, you'd need to ask all these services for a fresh DVD. This doesn't sound so appealing.

This is not just a theoretical issue. The Soviets have used the one-time pad for their confidential communication since before the 1940's. In fact, even before Shannon's work, the U.S. intelligence already knew in 1941 that the one-time pad is in principle "unbreakable" (see page 32 in the Venona document). However, it turned out that the hassle of manufacturing so many keys for all the communication took its toll on the Soviets and they ended up reusing the same keys for more than one message. They did try to use them for completely different receivers in the (false) hope that this wouldn't be detected. The Venona Project of the U.S. Army was founded in February 1943 by Gene Grabeel (see Fig. 20.9), a former home economics teacher from Madison Heights, Virgnia and Lt. Leonard Zubko. In October 1943, they had their breakthrough when it was discovered that the Russians were reusing their keys.[6] In the 37 years of its existence, the project has resulted in a treasure chest of intelligence, exposing hundreds of KGB agents and Russian spies in the U.S. and other countries, including Julius Rosenberg, Harry Gold, Klaus Fuchs, Alger Hiss, Harry Dexter White and many others.

Unfortunately it turns out that that such long keys are *necessary* for perfect secrecy:

Theorem 20.5 — Perfect secrecy requires long keys. For every perfectly secret encryption scheme (E, D) the length function L satisfies $L(n) \leq n$.

Proof Idea:

The idea behind the proof is illustrated in Fig. 20.10. If the number of keys is smaller than the number of messages then the neighborhoods of all vertices in the corresponding graphs cannot be identical.

⋆

Proof of Theorem 20.5. Let E, D be a valid encryption scheme with messages of length L and key of length $n < L$. We will show that (E, D) is not perfectly secret by providing two plaintexts $x_0, x_1 \in \{0,1\}^L$ such that the distributions Y_{x_0} and Y_{x_1} are not identical, where Y_x is the distribution obtained by picking $k \sim \{0,1\}^n$ and outputting $E_k(x)$.

[6] Credit to this discovery is shared by Lt. Richard Hallock, Carrie Berry, Frank Lewis, and Lt. Karl Elmquist, and there are others that have made important contribution to this project. See pages 27 and 28 in the document.



Figure 20.9: Gene Grabeel, who founded the U.S. Russian SigInt program on 1 Feb 1943. Photo taken in 1942, see Page 7 in the Venona historical study.

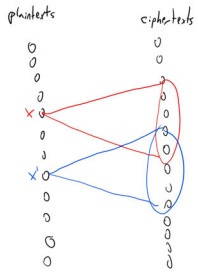


Figure 20.10: An encryption scheme where the number of keys is smaller than the number of plaintexts corresponds to a bipartite graph where the degree is smaller than the number of vertices on the left side. Together with the validity condition this implies that there will be two left vertices x, x' with non-identical neighborhoods, and hence the scheme does *not* satisfy perfect secrecy.

We choose $x_0 = 0^L$. Let $S_0 \subseteq \{0,1\}^*$ be the set of all ciphertexts that have nonzero probability of being output in Y_{x_0}. That is, $S = \{y \mid \exists_{k \in \{0,1\}^n} y = E_k(x_0)\}$. Since there are only 2^n keys, we know that $|S_0| \leq 2^n$.

We will show the following claim:

Claim I: There exists some $x_1 \in \{0,1\}^L$ and $k \in \{0,1\}^n$ such that $E_k(x_1) \notin S_0$.

Claim I implies that the string $E_k(x_1)$ has positive probability of being output by Y_{x_1} and zero probability of being output by Y_{x_0} and hence in particular Y_{x_0} and Y_{x_1} are not identical. To prove Claim I, just choose a fixed $k \in \{0,1\}^n$. By the validity condition, the map $x \mapsto E_k(x)$ is a one to one map of $\{0,1\}^L$ to $\{0,1\}^*$ and hence in particular the *image* of this map: the set $I = \{y \mid \exists_{x \in \{0,1\}^L} y = E_k(x)\}$ has size at least (in fact exactly) 2^L. Since $|S_0| = 2^n < 2^L$, this means that $|I| > |S_0|$ and so in particular there exists some string y in $I \setminus S_0$. But by the definition of I this means that there is some $x \in \{0,1\}^L$ such that $E_k(x) \notin S_0$ which concludes the proof of Claim I and hence of Theorem 20.5.

■

20.6 COMPUTATIONAL SECRECY

To sum up the previous episodes, we now know that:

- It is possible to obtain a perfectly secret encryption scheme with key length the same as the plaintext.

  and

- It is not possible to obtain such a scheme with key that is even a single bit shorter than the plaintext.

How does this mesh with the fact that, as we've already seen, people routinely use cryptosystems with a 16 byte (i.e., 128 bit) key but many terabytes of plaintext? The proof of Theorem 20.5 does give in fact a way to break all these cryptosystems, but an examination of this proof shows that it only yields an algorithm with time *exponential in the length of the key*. This motivates the following relaxation of perfect secrecy to a condition known as *"computational secrecy"*. Intuitively, an encryption scheme is computationally secret if no polynomial time algorithm can break it. The formal definition is below:

Definition 20.6 — Computational secrecy. Let (E, D) be a valid encryption scheme where for keys of length n, the plaintexts are of length $L(n)$ and the ciphertexts are of length $m(n)$. We say that (E, D) is *computationally secret* if for every polynomial $p : \mathbb{N} \to \mathbb{N}$, and large

enough n, if P is an $m(n)$-input and single output NAND-CIRC program of at most $p(n)$ lines, and $x_0, x_1 \in \{0,1\}^{L(n)}$ then

$$\left| \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [P(E_k(x_0))] - \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [P(E_k(x_1))] \right| < \frac{1}{p(n)} \tag{20.4}$$



Definition 20.6 requires a second or third read and some practice to truly understand. One excellent exercise to make sure you follow it is to see that if we allow P to be an *arbitrary* function mapping $\{0,1\}^{m(n)}$ to $\{0,1\}$, and we replace the condition in (20.4) that the lefhand side is smaller than $\frac{1}{p(L(n))}$ with the condition that it is equal to 0 then we get the perfect secrecy condition of Definition 20.3. Indeed if the distributions $E_k(x_0)$ and $E_k(x_1)$ are identical then applying any function P to them we get the same expectation. On the other hand, if the two distributions above give a different probability for some element $y^* \in \{0,1\}^{m(n)}$, then the function $P(y)$ that outputs 1 iff $y = y^*$ will have a different expectation under the former distribution than under the latter.

Definition 20.6 raises two natural questions:

- Is it strong enough to ensure that a computationally secret encryption scheme protects the secrecy of messages that are encrypted with it?

- It is weak enough that, unlike perfect secrecy, it is possible to obtain a computationally secret encryption scheme where the key is much smaller than the message?

To the best of our knowledge, the answer to both questions is *Yes*. Regarding the first question, it is not hard to show that if, for example, Alice uses a computationally secret encryption algorithm to encrypt either "attack" or "retreat" (each chosen with probability $1/2$), then as long as she's restricted to polynomial-time algorithms, an adversary Eve will not be able to guess the message with probability better than, say, 0.51, even after observing its encrypted form. (We omit the proof, but it is an excellent exercise for you to work it out on your own.)

To answer the second question we will show that under the same assumption we used for derandomizing **BPP**, we can obtain a computationally secret cryptosystem where the key is almost *exponentially* smaller than the plaintext.

20.6.1 Stream ciphers or the "derandomized one-time pad"

It turns out that if pseudorandom generators exist as in the optimal PRG conjecture, then there exists a computationally secret encryption scheme with keys that are much shorter than the plaintext. The construction below is known as a stream cipher, though perhaps a better name is the "derandomized one-time pad". It is widely used in practice with keys on the order of a few tens or hundreds of bits protecting many terabytes or even petabytes of communication.

Theorem 20.7 — Derandomized one-time pad. Suppose that the optimal PRG conjecture is true. Then for every constant $a \in \mathbb{N}$ there is a computationally secret encryption scheme (E, D) with plaintext length $L(n)$ at least n^a.

Proof Idea:

The proof is illustrated in Fig. 20.11. We simply take the one-time pad on L bit plaintexts, but replace the key with $G(k)$ where k is a string in $\{0,1\}^n$ and $G : \{0,1\}^n \to \{0,1\}^L$ is a pseudorandom generator.

⋆

Proof of Theorem 20.7. Since an exponential function of the form $2^{\delta n}$ grows faster than any polynomial of the form n^a, under the optimal PRG conjecture we can obtain a polynomial-time computable $(2^{\delta n}, 2^{-\delta n})$ pseudorandom generator $G : \{0,1\}^n \to \{0,1\}^L$ for $L = n^a$. We now define our encryption scheme as follows: given key $k \in \{0,1\}^n$ and plaintext $x \in \{0,1\}^L$, the encryption $E_k(x)$ is simply $x \oplus G(k)$. To decrypt a string $y \in \{0,1\}^m$ we output $y \oplus G(k)$. This is a valid encryption since G is computable in polynomial time and $(x \oplus G(k)) \oplus G(k) = x \oplus (G(k) \oplus G(k)) = x$ for every $x \in \{0,1\}^L$.

Computational secrecy follows from the condition of a pseudorandom generator. Suppose, towards a contradiction, that there is a polynomial p, NAND-CIRC program Q of at most $p(L)$ lines and $x, x' \in \{0,1\}^{L(n)}$ such that

$$\left| \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [Q(E_k(x))] - \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [Q(E_k(x'))] \right| > \tfrac{1}{p(L)} \tag{20.5}$$

which by the definition of our encryption scheme means that

$$\left| \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [Q(G(k) \oplus x)] - \mathop{\mathbb{E}}_{k \sim \{0,1\}^n} [Q(G(k) \oplus x')] \right| > \tfrac{1}{p(L)} \,. \tag{20.6}$$

Now since (as we saw in the security analysis of the one-time pad), the distribution $r \oplus x$ and $r \oplus x'$ are identical, where $r \sim \{0,1\}^L$, it follows that

$$\mathop{\mathbb{E}}_{r \sim \{0,1\}^L} [Q(r \oplus x)] - \mathop{\mathbb{E}}_{r \sim \{0,1\}^L} [Q(r \oplus x')] = 0 \,. \tag{20.7}$$



Figure 20.11: In a *stream cipher* or "derandomized one-time pad" we use a pseudorandom generator $G : \{0,1\}^n \to \{0,1\}^L$ to obtain an encryption scheme with a key length of n and plaintexts of length L. We encrypt the plaintext $x \in \{0,1\}^L$ with key $k \in \{0,1\}^n$ by the ciphertext $x \oplus G(k)$.

By plugging (20.7) into (20.6) we can derive that

$$\left|\mathop{\mathbb{E}}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x)] - \mathop{\mathbb{E}}_{r\sim\{0,1\}^L}[Q(r\oplus x)] + \mathop{\mathbb{E}}_{r\sim\{0,1\}^L}[Q(r\oplus x')] - \mathop{\mathbb{E}}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x')]\right| > \tfrac{1}{p(L)} \ . \tag{20.8}$$

(Please make sure that you can see why this is true.)

Now we can use the *triangle inequality* that $|A+B| \leq |A| + |B|$ for every two numbers A, B, applying it for $A = \mathbb{E}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x)] - \mathbb{E}_{r\sim\{0,1\}^L}[Q(r\oplus x)]$ and $B = \mathbb{E}_{r\sim\{0,1\}^L}[Q(r\oplus x')] - \mathbb{E}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x')]$ to derive

$$\left|\mathop{\mathbb{E}}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x)] - \mathop{\mathbb{E}}_{r\sim\{0,1\}^L}[Q(r\oplus x)]\right| + \left|\mathop{\mathbb{E}}_{r\sim\{0,1\}^L}[Q(r\oplus x')] - \mathop{\mathbb{E}}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x')]\right| > \tfrac{1}{p(L)} \ . \tag{20.9}$$

In particular, either the first term or the second term of the lefthand-side of (20.9) must be at least $\frac{1}{2p(L)}$. Let us assume the first case holds (the second case is analyzed in exactly the same way). Then we get that

$$\left|\mathop{\mathbb{E}}_{k\sim\{0,1\}^n}[Q(G(k)\oplus x)] - \mathop{\mathbb{E}}_{r\sim\{0,1\}^L}[Q(r\oplus x)]\right| > \tfrac{1}{2p(L)} \ . \tag{20.10}$$

But if we now define the NAND-CIRC program P_x that on input $r \in \{0,1\}^L$ outputs $Q(r\oplus x)$ then (since XOR of L bits can be computed in $O(L)$ lines), we get that P_x has $p(L) + O(L)$ lines and by (20.10) it can distinguish between an input of the form $G(k)$ and an input of the form $r \sim \{0,1\}^k$ with advantage better than $\frac{1}{2p(L)}$. Since a polynomial is dominated by an exponential, if we make L large enough, this will contradict the $(2^{\delta n}, 2^{-\delta n})$ security of the pseudorandom generator G. ■

R

Remark 20.8 — Stream ciphers in practice. The two most widely used forms of (private key) encryption schemes in practice are *stream ciphers* and *block ciphers*. (To make things more confusing, a block cipher is always used in some mode of operation and some of these modes effectively turn a block cipher into a stream cipher.) A block cipher can be thought as a sort of a "random invertible map" from $\{0,1\}^n$ to $\{0,1\}^n$, and can be used to construct a pseudorandom generator and from it a stream cipher, or to encrypt data directly using other modes of operations. There are a great many other security notions and considerations for encryption schemes beyond computational secrecy. Many of those involve handling scenarios such as *chosen plaintext*, *man in the middle*, and *chosen ciphertext* attacks, where the adversary is not just merely a passive eavesdropper but can influence the communication in some way. While this chapter is

meant to give you some taste of the ideas behind cryptography, there is much more to know before applying it correctly to obtain secure applications, and a great many people have managed to get it wrong.

20.7 COMPUTATIONAL SECRECY AND NP

We've also mentioned before that an efficient algorithm for **NP** could be used to break all cryptography. We now give an example of how this can be done:

Theorem 20.9 — Breaking encryption using NP algorithm. If $\mathbf{P} = \mathbf{NP}$ then there is no computationally secret encryption scheme with $L(n) > n$.

Furthermore, for every valid encryption scheme (E, D) with $L(n) > n + 100$ there is a polynomial p such that for every large enough n there exist $x_0, x_1 \in \{0,1\}^{L(n)}$ and a $p(n)$-line NAND-CIRC program EVE s.t.

$$\Pr_{i \sim \{0,1\}, k \sim \{0,1\}^n}[EVE(E_k(x_i)) = i] \geq 0.99 \,. \tag{20.11}$$

Note that the "furthermore" part is extremely strong. It means that if the plaintext is even a little bit larger than the key, then we can already break the scheme in a very strong way. That is, there will be a pair of messages x_0, x_1 (think of x_0 as "sell" and x_1 as "buy") and an efficient strategy for Eve such that if Eve gets a ciphertext y then she will be able to tell whether y is an encryption of x_0 or x_1 with probability very close to 1.[7] The condition $\mathbf{P} = \mathbf{NP}$ can be relaxed to $\mathbf{NP} \subseteq \mathbf{BPP}$ and even the weaker condition $\mathbf{NP} \subseteq \mathbf{P}_{/\mathbf{poly}}$ with essentially the same proof.

[7] We model breaking the scheme as Eve outputting 0 or 1 corresponding to whether the message sent was x_0 or x_1. Note that we could have just as well modified Eve to output x_0 instead of 0 and x_1 instead of 1. The key point is that a priori Eve only had a 50/50 chance of gussing whether Alice sent x_0 or x_1 but after seeing the ciphertext this chance increases to better than 99/1.

Proof Idea:

The proof follows along the lines of Theorem 20.5 but this time paying attention to the computational aspects. If $\mathbf{P} = \mathbf{NP}$ then for every plaintext x and ciphertext y, we can efficiently tell whether there exists $k \in \{0,1\}^n$ such that $E_k(x) = y$. So, to prove this result we need to show that if the plaintexts are long enough, there would exist a pair x_0, x_1 such that the probability that a random encryption of x_1 also is a valid encryption of x_0 will be very small. The details of how to show this are below.

⋆

Proof of Theorem 20.9. We focus on showing only the "furthermore" part since it is the more interesting and the other part follows by essentially the same proof.

Suppose that (E, D) is such an encryption, let n be large enough, and let $x_0 = 0^{L(n)}$. For every $x \in \{0,1\}^{L(n)}$ we define S_x to be the set of all valid encryption of x. That is $S_x = \{y \mid \exists_{k \in \{0,1\}^n} y = E_k(x)\}$. As in the proof of Theorem 20.5, since there are 2^n keys k, $|S_x| \leq 2^n$ for every $x \in \{0,1\}^{L(n)}$.

We denote by S_0 the set S_{x_0}. We define our algorithm *EVE* to output 0 on input $y \in \{0,1\}^*$ if $y \in S_0$ and to output 1 otherwise. This can be implemented in polynomial time if $\mathbf{P} = \mathbf{NP}$, since the key k can serve the role of an efficiently verifiable solution. (Can you see why?) Clearly $\Pr[EVE(E_k(x_0)) = 0] = 1$ and so in the case that *EVE* gets an encryption of x_0 then she guesses correctly with probability 1. The remainder of the proof is devoted to showing that there exists $x_1 \in \{0,1\}^{L(n)}$ such that $\Pr[EVE(E_k(x_1)) = 0] \leq 0.01$, which will conclude the proof by showing that *EVE* guesses wrongly with probability at most $\frac{1}{2}0 + \frac{1}{2}0.01 < 0.01$.

Consider now the following probabilistic experiment (which we define solely for the sake of analysis). We consider the sample space of choosing x uniformly in $\{0,1\}^{L(n)}$ and define the random variable $Z_k(x)$ to equal 1 if and only if $E_k(x) \in S_0$. For every k, the map $x \mapsto E_k(x)$ is one-to-one, which means that the probability that $Z_k = 1$ is equal to the probability that $x \in E_k^{-1}(S_0)$ which is $\frac{|S_0|}{2^{L(n)}}$. So by the linearity of expectation $\mathbb{E}[\sum_{k \in \{0,1\}^n} Z_k] \leq \frac{2^n |S_0|}{2^{L(n)}} \leq \frac{2^{2n}}{2^{L(n)}}$.

We will now use the following extremely simple but useful fact known as the *averaging principle* (see also **??**): for every random variable Z, if $\mathbb{E}[Z] = \mu$, then with positive probability $Z \leq \mu$. (Indeed, if $Z > \mu$ with probability one, then the expected value of Z will have to be larger than μ, just like you can't have a class in which all students got A or A- and yet the overall average is B+.) In our case it means that with positive probability $\sum_{k \in \{0,1\}^n} Z_k \leq \frac{2^{2n}}{2^{L(n)}}$. In other words, there exists some $x_1 \in \{0,1\}^{L(n)}$ such that $\sum_{k \in \{0,1\}^n} Z_k(x_1) \leq \frac{2^{2n}}{2^{L(n)}}$. Yet this means that if we choose a random $k \sim \{0,1\}^n$, then the probability that $E_k(x_1) \in S_0$ is at most $\frac{1}{2^n} \cdot \frac{2^{2n}}{2^{L(n)}} = 2^{n-L(n)}$. So, in particular if we have an algorithm *EVE* that outputs 0 if $x \in S_0$ and outputs 1 otherwise, then $\Pr[EVE(E_k(x_0)) = 0] = 1$ and $\Pr[EVE(E_k(x_1)) = 0] \leq 2^{n-L(n)}$ which will be smaller than $2^{-10} < 0.01$ if $L(n) \geq n + 10$.

■

In retrospect Theorem 20.9 is perhaps not surprising. After all, as we've mentioned before it is known that the Optimal PRG conjecture (which is the basis for the derandomized one-time pad encryption) is *false* if $\mathbf{P} = \mathbf{NP}$ (and in fact even if $\mathbf{NP} \subseteq \mathbf{BPP}$ or even $\mathbf{NP} \subseteq \mathbf{P}_{/\mathbf{poly}}$).

20.8 PUBLIC KEY CRYPTOGRAPHY

People have been dreaming about heavier-than-air flight since at least the days of Leonardo Da Vinci (not to mention Icarus from the greek mythology). Jules Verne wrote with rather insightful details about going to the moon in 1865. But, as far as I know, in all the thousands of years people have been using secret writing, until about 50 years ago no one has considered the possibility of communicating securely without first exchanging a shared secret key.

Yet in the late 1960's and early 1970's, several people started to question this "common wisdom". Perhaps the most surprising of these visionaries was an undergraduate student at Berkeley named Ralph Merkle. In the fall of 1974 Merkle wrote in a project proposal for his computer security course that while "it might seem intuitively obvious that if two people have never had the opportunity to prearrange an encryption method, then they will be unable to communicate securely over an insecure channel... I believe it is false". The project proposal was rejected by his professor as "not good enough". Merkle later submitted a paper to the communication of the ACM where he apologized for the lack of references since he was unable to find any mention of the problem in the scientific literature, and the only source where he saw the problem even *raised* was in a science fiction story. The paper was rejected with the comment that "Experience shows that it is extremely dangerous to transmit key information in the clear." Merkle showed that one can design a protocol where Alice and Bob can use T invocations of a hash function to exchange a key, but an adversary (in the random oracle model, though he of course didn't use this name) would need roughly T^2 invocations to break it. He conjectured that it may be possible to obtain such protocols where breaking is *exponentially harder* than using them, but could not think of any concrete way to doing so.

We only found out much later that in the late 1960's, a few years before Merkle, James Ellis of the British Intelligence agency GCHQ was having similar thoughts. His curiosity was spurred by an old World-War II manuscript from Bell labs that suggested the following way that two people could communicate securely over a phone line. Alice would inject noise to the line, Bob would relay his messages, and then Alice would subtract the noise to get the signal. The idea is that an adversary over the line sees only the sum of Alice's and Bob's signals, and doesn't know what came from what. This got James Ellis thinking whether it would be possible to achieve something like that digitally. As Ellis later recollected, in 1970 he realized that in principle this should be possible, since he could think of an hypothetical black box B that on input a "handle" α and plaintext x would give a

"ciphertext" y and that there would be a secret key β corresponding to α, such that feeding β and y to the box would recover x. However, Ellis had no idea how to actually instantiate this box. He and others kept giving this question as a puzzle to bright new recruits until one of them, Clifford Cocks, came up in 1973 with a candidate solution loosely based on the factoring problem; in 1974 another GCHQ recruit, Malcolm Williamson, came up with a solution using modular exponentiation.

But among all those thinking of public key cryptography, probably the people who saw the furthest were two researchers at Stanford, Whit Diffie and Martin Hellman. They realized that with the advent of electronic communication, cryptography would find new applications beyond the military domain of spies and submarines, and they understood that in this new world of many users and point to point communication, cryptography will need to scale up. Diffie and Hellman envisioned an object which we now call "trapdoor permutation" though they called "one way trapdoor function" or sometimes simply "public key encryption". Though they didn't have full formal definitions, their idea was that this is an injective function that is easy (e.g., polynomial-time) to *compute* but hard (e.g., exponential-time) to *invert*. However, there is a certain *trapdoor*, knowledge of which would allow polynomial time inversion. Diffie and Hellman argued that using such a trapdoor function, it would be possible for Alice and Bob to communicate securely *without ever having exchanged a secret key*. But they didn't stop there. They realized that protecting the *integrity* of communication is no less important than protecting its *secrecy*. Thus they imagined that Alice could "run encryption in reverse" in order to certify or *sign* messages.

At the point, Diffie and Hellman were in a position not unlike physicists who predicted that a certain particle should exist but without any experimental verification. Luckily they met Ralph Merkle, and his ideas about a probabilistic *key exchange protocol*, together with a suggestion from their Stanford colleague John Gill, inspired them to come up with what today is known as the Diffie Hellman Key Exchange (which unbeknownst to them was found two years earlier at GCHQ by Malcolm Williamson). They published their paper "New Directions in Cryptography" in 1976, and it is considered to have brought about the birth of modern cryptography.

The Diffie-Hellman Key Exchange is still widely used today for secure communication. However, it still felt short of providing Diffie and Hellman's elusive trapdoor function. This was done the next year by Rivest, Shamir and Adleman who came up with the RSA trapdoor function, which through the framework of Diffie and Hellman yielded not just encryption but also signatures.[8] From this point on began a

[8] A close variant of the RSA function was discovered earlier by Clifford Cocks at GCHQ, though as far as I can tell Cocks, Ellis and Williamson did not realize the application to digital signatures.

flurry of advances in cryptography which hasn't died down till this day.

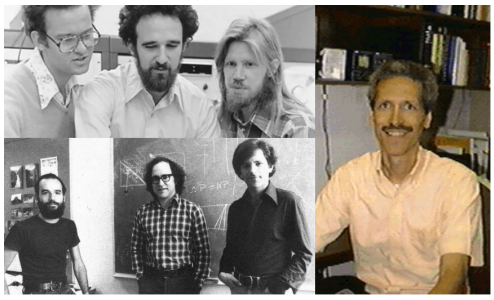


Figure 20.12: Top left: Ralph Merkle, Martin Hellman and Whit Diffie, who together came up in 1976 with the concept of *public key encryption* and a *key exchange protocol*. Bottom left: Adi Shamir, Ron Rivest, and Leonard Adleman who, following Diffie and Hellman's paper, discovered the RSA function that can be used for public key encryption and digital signatures. Interestingly, one can see the equation **P** = **NP** on the blackboard behind them. Right: John Gill, who was the first person to suggest to Diffie and Hellman that they use modular exponentiation as an easy-to-compute but hard-to-invert function.

20.8.1 Defining public key encryption

A *public key encryption* consists of a triple of algorithms:

- The *key generation algorithm*, which we denote by $KeyGen$ or KG for short, is a randomized algorithm that outputs a pair of strings (e,d) where e is known as the *public* (or *encryption*) key, and d is known as the *private* (or *decryption*) key. The key generation algorithm gets as input 1^n (i.e., a string of ones of length n). We refer to n as the *security parameter* of the scheme. The bigger we make n, the more secure the encryption will be, but also the less efficient it will be.
- The *encryption algorithm*, which we denote by E, takes the encryption key e and a plaintext x, and outputs the ciphertext $y = E_e(x)$.
- The *decryption algorithm*, which we denote by D, takes the decryption key d and a ciphertext y, and outputs the plaintext $x = D_d(y)$.

We now make this a formal definition:

Definition 20.10 — Public Key Encryption. A *computationally secret public key encryption* with plaintext length $L : \mathbb{N} \rightarrow \mathbb{N}$ is a triple of randomized polynomial-time algorithms (KG, E, D) that satisfy the following conditions:

- For every n, if (e,d) is output by $KG(1^n)$ with positive probability, and $x \in \{0,1\}^{L(n)}$, then $D_d(E_e(x)) = x$ with probability one.
- For every polynomial p, and sufficiently large n, if P is a NAND-CIRC program of at most $p(n)$ lines then for every $x, x' \in \{0,1\}^{L(n)}$, $|\mathbb{E}[P(e,E_e(x))] - \mathbb{E}[P(e,E_e(x'))]| < 1/p(n)$, where this probability is taken over the coins of KG and E.

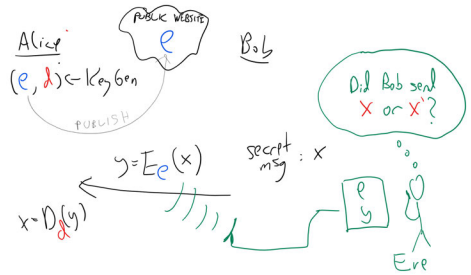


Figure 20.13: In a *public key encryption*, Alice generates a private/public keypair (e,d), publishes e and keeps d secret. To encrypt a message for Alice, one only needs to know e. To decrypt it we need to know d.

Definition 20.10 allows E and D to be *randomized* algorithms. In fact, it turns out that it is *necessary* for E to be randomized to obtain computational secrecy. It also turns out that, unlike the private key case, we can transform a public-key encryption that works for messages that are *only one bit long* into a public-key encryption scheme that can encrypt arbitrarily long messages, and in particular messages that are *longer than the key*. In particular this means that we cannot obtain a perfectly secret public-key encryption scheme even for one-bit long messages (since it would imply a perfectly secret public-key, and hence in particular private-key, encryption with messages longer than the key).

We will not give full constructions for public key encryption schemes in this chapter, but will mention some of the ideas that underlie the most widely used schemes today. These generally belong to one of two families:

- *Group theoretic constructions* based on problems such as *integer factoring* and the *discrete logarithm* over finite fields or elliptic curves.
- *Lattice/coding based constructions* based on problems such as the *closest vector in a lattice* or *bounded distance decoding*.

Group-theory based encryptions such as the RSA cryptosystem, the Diffie-Hellman protocol, and Elliptic-Curve Cryptography, are currently more widely implemented. But the lattice/coding schemes are recently on the rise, particularly because the known group theoretic encryption schemes can be broken by *quantum computers*, which we'll discuss later in this course.[9]

[9] If you want to learn more about the different types of public key assumptions, you can take a look at my own survey on this topic.

20.8.2 Diffie-Hellman key exchange

As just one example of how public key encryption schemes are constructed, let us now describe the Diffie-Hellman key exchange. We describe the Diffie-Hellman protocol in a somewhat of an informal level, without presenting a full security analysis.

The computational problem underlying the Diffie Hellman protocol is the *discrete logarithm problem*. Let's suppose that g is some integer. We can compute the map $x \mapsto g^x$ and also its *inverse* $y \mapsto \log_g y$.[10] However, suppose now that we use *modular arithmetic* and work modulo some prime number p. If p has n binary digits and g is in $[p]$ then we can compute the map $x \mapsto g^x \mod p$ in time polynomial in n.[11] On the other hand, because of the "wraparound" property of modular arithmetic, we cannot run binary search to find the inverse of this map (known as the *discrete logarithm*). In fact, there is no known polynomial-time algorithm for computing this discrete logarithm map map $(g, x, p) \mapsto \log_g x \mod p$, where we define $\log_g x \mod p$ as the number $a \in [p]$ such that $g^a = x \mod p$.

[10] One way to compute a logarithm is by *binary search*: start with some interval $[x_{min}, x_{max}]$ that is guaranteed to contain $\log_g y$. We can then test whether the interval's midpoint x_{mid} satisfies $g^{x_{mid}} > y$, and based on that halve the size of the interval.

[11] This is not trivial, and is a great exercise for you to work this out. As a hint, start by showing that one can compute the map $k \mapsto g^{2^k} \mod p$ using k modular multiplications modulo p. If you're stumped, you can look up this Wikipedia entry.

The Diffie-Hellman protocol for Bob to send a message to Alice is as follows:

- **Alice:** Chooses p to be a random n bit long prime (which can be done by choosing random numbers and running a primality testing algorithm on them), and g and a at random in $[p]$. She sends to Bob the triple $(p, g, g^a \mod p)$.
- **Bob:** Given the triple (p, g, h), Bob sends a message $x \in \{0,1\}^L$ to Alice by choosing b at random in $[p]$, and sending to Alice the pair $(g^b \mod p, rep(h^b \mod p) \oplus x)$ where $rep : [p] \to \{0,1\}^*$ is some "representation function" that maps $[p]$ to $\{0,1\}^L$.[12]

- **Alice:** Given g', z, Alice recovers x by outputting $rep(g'^a \mod p) \oplus z$.

The correctness of the protocol follows from the simple fact that $(g^a)^b = (g^b)^a$ for every g, a, b and this still holds if we work modulo p. Its security relies on the computational assumption that computing this map is hard, even in a certain "average case" sense (this computational assumption is known as the Decisional Diffie Hellman assumption). The Diffie-Hellman key exchange protocol can be thought of as a public key encryption where the Alice's first message is the public key, and Bob's message is the encryption.

One can think of the Diffie-Hellman protocol as being based on a "trapdoor pseudorandom generator" whereas the triple g^a, g^b, g^{ab} looks "random" to someone that doesn't know a, but someone that does know a can see that raising the second element to the a-th power yields the third element. The Diffie-Hellman protocol can be described abstractly in the context of any finite Abelian group for which we can efficiently compute the group operation. It has been implemented on other groups than numbers modulo p, and in particular Elliptic Curve Cryptography (ECC) is obtained by basing the Diffie Hellman on elliptic curve groups which gives some practical advantages.[13] Another common group theoretic basis for key-exchange/public key encryption protocol is the RSA function. A big disadvantage of Diffie-Hellman (both the modular arithmetic and elliptic curve variants) and RSA is that both schemes can be broken in polynomial time by a *quantum computer*. We will discuss quantum computing later in this course.

[13] The main advantage in ECC is that the best known algorithms for computing discrete logarithms over elliptic curve groups take time $2^{\epsilon n}$ for some $\epsilon > 0$ where n is the number of bits to describe a group element. In contrast, for the multiplicative group modulo a prime p the best algorithm take time $2^{O(n^{1/3} polylog(n))}$ which means that (assuming the known algorithms are optimal) we need to set the prime to be bigger (and so have larger key sizes with corresponding overhead in communication and computation) to get the same level of security.

20.9 OTHER SECURITY NOTIONS

There is a great deal to cryptography beyond just encryption schemes, and beyond the notion of a passive adversary. A central objective is *integrity* or *authentication*: protecting communications from being modified by an adversary. Integerity is often more fundamental than secrecy: whether it is a software update or viewing the news, you might often not care about the communication being secret as much as that it indeed came from its claimed source. *Digital signature schemes* are the analog of public key encryption for authentication, and are widely used (in particular as the basis for public key certificates) to provide a foundation of trust in the digital world.

Similarly, even for encryption, we often need to ensure security against *active attacks*, and so notions such as non-malleability and adaptive chosen ciphertext security have been proposed. An encryption scheme is only as secure as the secret key, and mechanisms to make sure the key is generated properly, and is protected against re-

fresh or even compromise (i.e., forward secrecy) have been studied as well. Hopefully this chapter provides you with some appreciation for cryptography as an intellectual field, but does not imbue you with a false self of confidence in implementing it.

Cryptographic hash functions is another widely used tool with a variety of uses, including extracting randomness from high entropy sources, achieving hard-to-forge short "digests" of files, protecting passwords, and much more.

20.10 MAGIC

Beyond encryption and signature schemes, cryptographers have managed to obtain objects that truly seem paradoxical and "magical". We briefly discuss some of these objects. We do not give any details, but hopefully this will spark your curiosity to find out more.

20.10.1 Zero knowledge proofs

On October 31, 1903, the mathematician Frank Nelson Cole, gave an hourlong lecture to a meeting of the American Mathematical Society where he did not speak a single word. Rather, he calculated on the board the value $2^{67} - 1$ which is equal to $147,573,952,589,676,412,927$, and then showed that this number is equal to $193,707,721 \times 761,838,257,287$. Cole's proof showed that $2^{67} - 1$ is not a prime, but it also revealed additional information, namely its actual factors. This is often the case with proofs: they teach us more than just the validity of the statements.

In *Zero Knowledge Proofs* we try to achieve the opposite effect. We want a proof for a statement X where we can *rigorously show* that the proofs reveals *absolutely no additional information about* X beyond the fact that it is true. This turns out to be an extremely useful object for a variety of tasks including authentication, secure protocols, voting, anonymity in cryptocurrencies, and more. Constructing these objects relies on the theory of **NP** completeness. Thus this theory that originally was designed to give a *negative result* (show that some problems are hard) ended up yielding *positive applications*, enabling us to achieve tasks that were not possible otherwise.

20.10.2 Fully homomorphic encryption

Suppose that we are given a bit-by-bit encryption of a string $E_k(x_0), \ldots, E_k(x_{n-1})$. By design, these ciphertexts are supposed to be "completely unscrutable" and we should not be able to extract any information about x_i's from it. However, already in 1978, Rivest, Adleman and Dertouzos observed that this does not imply that we could not *manipulate* these encryptions. For example, it turns out the security of an encryption scheme does not immediately rule out the

ability to take a pair of encryptions $E_k(a)$ and $E_k(b)$ and compute from them $E_k(a NAND b)$ *without knowing the secret key* k. But do there exist encryption schemes that allow such manipulations? And if so, is this a bug or a feature?

Rivest et al already showed that such encryption schemes could be *immensely* useful, and their utility has only grown in the age of cloud computing. After all, if we can compute NAND then we can use this to run any algorithm P on the encrypted data, and map $E_k(x_0), \ldots, E_k(x_{n-1})$ to $E_k(P(x_0, \ldots, x_{n-1}))$. For example, a client could store their secret data x in encrypted form on the cloud, and have the cloud provider perform all sorts of computation on these data without ever revealing to the provider the private key, and so without the provider *ever learning any information* about the secret data.

The question of *existence* of such a scheme took much longer time to resolve. Only in 2009 Craig Gentry gave the first construction of an encryption scheme that allows to compute a universal basis of gates on the data (known as a *Fully Homomorphic Encryption scheme* in crypto parlance). Gentry's scheme left much to be desired in terms of efficiency, and improving upon it has been the focus of an intensive research program that has already seen significant improvements.

20.10.3 Multiparty secure computation

Cryptography is about enabling mutually distrusting parties to achieve a common goal. Perhaps the most general primitive achieving this objective is secure multiparty computation. The idea in secure multiparty computation is that n parties interact together to compute some function $F(x_0, \ldots, x_{n-1})$ where x_i is the private input of the i-th party. The crucial point is that there is *no commonly trusted party or authority* and that nothing is revealed about the secret data beyond the function's output. One example is an *electronic voting protocol* where only the total vote count is revealed, with the privacy of the individual voters protected, but without having to trust any authority to either count the votes correctly or to keep information confidential. Another example is implementing a second price (aka Vickrey) auction where $n-1$ parties submit bids to an item owned by the n-th party, and the item goes to the highest bidder but at the price of the *second highest bid*. Using secure multiparty computation we can implement second price auction in a way that will ensure the secrecy of the numerical values of all bids (including even the top one) except the second highest one, and the secrecy of the identity of all bidders (including even the second highest bidder) except the top one. We emphasize that such a protocol requires no trust even in the auctioneer itself, that will also not learn any additional information. Secure multiparty computation

can be used even for computing *randomized* processes, with one example being playing Poker over the net without having to trust any server for correct shuffling of cards or not revealing the information.

Lecture Recap

- We can formally define the notion of security of an encryption scheme.
- *Perfect secrecy* ensures that an adversary does not learn *anything* about the plaintext from the ciphertext, regardless of their computational powers.
- The one-time pad is a perfectly secret encryption with the length of the key equaling the length of the message. No perfectly secret encryption can have key shorter than the message.
- *Computational secrecy* can be as good as perfect secrecy since it ensures that the advantage that computationally bounded adversaries gain from observing the ciphertext is exponentially small. If the optimal PRG conjecture is true then there exists a computationally secret encryption scheme with messages that can be (almost) *exponentially bigger* than the key.
- There are many cryptographic tools that go well beyond private key encryption. These include *public key encryption*, *digital signatures* and *hash functions*, as well as more "magical" tools such as *multiparty secure computation*, *fully homomorphic encryption*, *zero knowledge proofs*, and many others.

20.11 EXERCISES

Remark 20.11 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

20.12 BIBLIOGRAPHICAL NOTES

Much of this text is taken from my lecture notes on cryptography.

Shannon's manuscript was written in 1945 but was classified, and a partial version was only published in 1949. Still it has revolutionized cryptography, and is the forerunner to much of what followed.

John Nash made seminal contributions in mathematics and game theory, and was awarded both the Abel Prize in mathematics and the Nobel Memorial Prize in Economic Sciences. However, he has struggled with mental illness throughout his life. His biography, A Beautiful Mind was made into a popular movie. It is natural to compare Nash's 1955 letter to the NSA to Gödel's letter to von Neumann we mentioned before. From the theoretical computer science point of view, the crucial difference is that while Nash informally talks about exponential vs polynomial computation time, he does not mention the word "Turing Machine" or other models of computation, and it is not clear if he is aware or not that his conjecture can be made mathematically precise (assuming a formalization of "sufficiently complex types of enciphering").

The definition of computational secrecy we use is the notion of *computational indistinguishability* (known to be equivalent to *semantic security*) that was given by Goldwasser and Micali in 1982.

Although they used a different terminology, Diffie and Hellman already made clear in their paper that their protocol can be used as a public key encryption, with the first message being put in a "public file". In 1985, ElGamal showed how to obtain a *signature scheme* based on the Diffie Hellman ideas, and since he described the Diffie-Hellman encryption scheme in the same paper, it is sometimes also known as ElGamal encryption.

Zero-knowledge proofs were constructed by Goldwasser, Micali, and Rackoff in 1982, and their wide applicability was shown (using the theory of **NP** completeness) by Goldreich, Micali, and Wigderson in 1986.

Two party and multiparty secure computation protocols were constructed (respectively) by Yao in 1982 and Goldreich, Micali, and Wigderson in 1987. The latter work gave a general transformation from security against passive adversaries to security against active adversaries using zero knowledge proofs.

20.13 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

20.14 ACKNOWLEDGEMENTS

21
Proofs and algorithms

> *"Let's not try to define knowledge, but try to define zero-knowledge."*, Shafi Goldwasser.

Proofs have captured human imagination for thousands of years, ever since the publication of Euclid's *Elements*, a book second only to the bible in the number of editions printed.

Plan:

- Proofs and algorithms
- Interactive proofs
- Zero knowledge proofs
- Propositions as types, Coq and other proof assistants.

21.1 LECTURE SUMMARY

21.2 EXERCISES

R

Remark 21.1 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

21.3 BIBLIOGRAPHICAL NOTES

21.4 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

21.5 ACKNOWLEDGEMENTS

Learning Objectives:

- **See main aspects in which quantum mechanics differs from local deterministic theories.**
- **Model of quantum circuits, or equivalently QNAND-CIRC programs**
- **The complexity class BQP and what we know about its relation to other classes**
- **Ideas behind Shor's Algorithm and the Quantum Fourier Transform**

22
Quantum computing

> *"We always have had (secret, secret, close the doors!) ... a great deal of difficulty in understanding the world view that quantum mechanics represents ... It has not yet become obvious to me that there's no real problem. ... Can I learn anything from asking this question about computers–about this may or may not be mystery as to what the world view of quantum mechanics is?"* , Richard Feynman, 1981

> *"The only difference between a probabilistic classical world and the equations of the quantum world is that somehow or other it appears as if the probabilities would have to go negative"*, Richard Feynman, 1981

There were two schools of natural philosophy in ancient Greece. *Aristotle* believed that objects have an *essence* that explains their behavior, and a theory of the natural world has to refer to the *reasons* (or "final cause" to use Aristotle's language) as to why they exhibit certain phenomena. *Democritus* believed in a purely mechanistic explanation of the world. In his view, the universe was ultimately composed of elementary particles (or *Atoms*) and our observed phenomena arise from the interactions between these particles according to some local rules. Modern science (arguably starting with Newton) has embraced Democritus' point of view, of a mechanistic or "clockwork" universe of particles and forces acting upon them.

While the classification of particles and forces evolved with time, to a large extent the "big picture" has not changed from Newton till Einstein. In particular it was held as an axiom that if we knew fully the current *state* of the universe (i.e., the particles and their properties such as location and velocity) then we could predict its future state at any point in time. In computational language, in all these theories the state of a system with n particles could be stored in an array of $O(n)$ numbers, and predicting the evolution of the system can be done by

running some efficient (e.g., $poly(n)$ time) deterministic computation on this array.

22.1 THE DOUBLE SLIT EXPERIMENT

Alas, in the beginning of the 20th century, several experimental results were calling into question this "clockwork" or "billiard ball" theory of the world. One such experiment is the famous double slit experiment. Here is one way to describe it. Suppose that we buy one of those baseball pitching machines, and aim it at a soft plastic wall, but put a *metal barrier with a single slit* between the machine and the plastic wall (see Fig. 22.1). If we shoot baseballs at the plastic wall, then some of the baseballs would bounce off the metal barrier, while some would make it through the slit and dent the wall. If we now carve out an additional slit in the metal barrier then more balls would get through, and so the plastic wall would be *even more dented*.

So far this is pure common sense, and it is indeed (to my knowledge) an accurate description of what happens when we shoot baseballs at a plastic wall. However, this is not the same when we shoot *photons*. Amazingly, if we shoot with a "photon gun" (i.e., a laser) at a wall equipped with photon detectors through some barrier, then (as shown in Fig. 22.2) in some positions of the wall we will see *fewer* hits when the two slits are open than one only ones of them is!.[1] In particular there are positions in the wall that are hit when the first slit is open, hit when the second gun is open, but are *not hit at all when both slits are open!*.

It seems as if each photon coming out of the gun is aware of the global setup of the experiment, and behaves differently if two slits are open than if only one is. If we try to "catch the photon in the act" and place a detector right next to each slit so we can see exactly the path each photon takes then something even more bizarre happens. The mere fact that we *measure* the path changes the photon's behavior, and now this "destructive interference" pattern is gone and the number of times a position is hit when two slits are open is the sum of the number of times it is hit when each slit is open.

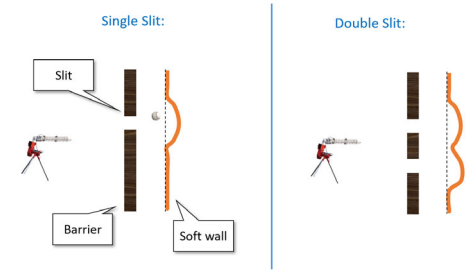


Figure 22.1: In the "double baseball experiment" we shoot baseballs from a gun at a soft wall through a hard barrier that has one or two slits open in it. There is only "constructive interference" in the sense that the dent in each position in the wall when both slits are open is the sum of the dents when each slit is open on its own.

[1] A nice illustrated description of the double slit experiment appears in this video.

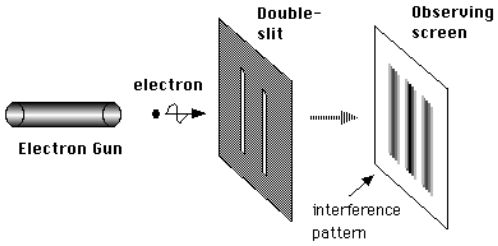


Figure 22.2: The setup of the double slit experiment in the case of photon or electron guns. We see also *destructive* interference in the sense that there are some positions on the wall that get *fewer* hits when both slits are open than they get when only one of the slits is open. Image credit: Wikipedia.

P You should read the paragraphs above more than once and make sure you appreciate how truly mind boggling these results are.

22.2 QUANTUM AMPLITUDES

The double slit and other experiments ultimately forced scientists to accept a very counterintuitive picture of the world. It is not merely

about nature being randomized, but rather it is about the probabilities in some sense "going negative" and cancelling each other!

To see what we mean by this, let us go back to the baseball experiment. Suppose that the probability a ball passes through the left slit is p_L and the probability that it passes through the right slit is p_R. Then, if we shoot N balls out of each gun, we expect the wall will be hit $(p_L + p_R)N$ times. In contrast, in the quantum world of photons instead of baseballs, it can sometimes be the case that in both the first and second case the wall is hit with positive probabilities p_L and p_R respectively but somehow when both slits are open the wall (or a particular position in it) is not hit at all. It's almost as if the probabilities can "cancel each other out".

To understand the way we model this in quantum mechanics, it is helpful to think of a "lazy evaluation" approach to probability. We can think of a probabilistic experiment such as shooting a baseball through two slits in two different ways:

- When a ball is shot, "nature" tosses a coin and decides if it will go through the left slit (which happens with probability p_L), right slit (which happens with probability p_R), or bounce back. If it passes through one of the slits then it will hit the wall. Later we can look at the wall and find out whether or not this event happened, but the fact that the event happened or not is determined independently of whether or not we look at the wall.

- The other viewpoint is that when a ball is shot, "nature" computes the probabilities p_L and p_R as before, but does *not* yet "toss the coin" and determines what happened. Only when we actually look at the wall, nature tosses a coin and with probability $p_L + p_R$ ensures we see a dent. That is, nature uses "lazy evaluation", and only determines the result of a probabilistic experiment when we decide to *measure* it.

While the first scenario seems much more natural, the end result in both is the same (the wall is hit with probability $p_L + p_R$) and so the question of whether we should model nature as following the first scenario or second one seems like asking about the proverbial tree that falls in the forest with no one hearing about it.

However, when we want to describe the double slit experiment with photons rather than baseballs, it is the second scenario that lends itself better to a quantum generalization. Quantum mechanics associates a number α known as an *amplitude* with each probabilistic experiment. This number α can be *negative*, and in fact even *complex*. We never observe the amplitudes directly, since whenever we *measure* an event with amplitude α, nature tosses a coin and determines

that the event happens with probability $|\alpha|^2$. However, the sign (or in the complex case, phase) of the amplitudes can affect whether two different events have *constructive* or *destructive* interference.

Specifically, consider an event that can either occur or not (e.g. "detector number 17 was hit by a photon"). In classical probability, we model this by a probability distribution over the two outcomes: a pair of non-negative numbers p and q such that $p + q = 1$, where p corresponds to the probability that the event occurs and q corresponds to the probability that the event does not occur. In quantum mechanics, we model this also by pair of numbers, which we call *amplitudes*. This is a pair of (potentially negative or even complex) numbers α and β such that $|\alpha|^2 + |\beta|^2 = 1$. The probability that the event occurs is $|\alpha|^2$ and the probability that it does not occur is $|\beta|^2$. In isolation, these negative or complex numbers don't matter much, since we anyway square them to obtain probabilities. But the interaction of positive and negative amplitudes can result in surprising *cancellations* where somehow combining two scenarios where an event happens with positive probability results in a scenario where it never does.



If you don't find the above description confusing and unintuitive, you probably didn't get it. Please make sure to re-read the above paragraphs until you are thoroughly confused.

Quantum mechanics is a mathematical theory that allows us to calculate and predict the results of the double-slit and many other experiments. If you think of quantum mechanics as an explanation as to what "really" goes on in the world, it can be rather confusing. However, if you simply "shut up and calculate" then it works amazingly well at predicting experimental results. In particular, in the double slit experiment, for any position in the wall, we can compute numbers α and β such that photons from the first and second slit hit that position with probabilities $|\alpha|^2$ and $|\beta|^2$ respectively. When we open both slits, the probability that the position will be hit is proportional to $|\alpha + \beta|^2$, and so in particular, if $\alpha = -\beta$ then it will be the case that, despite being hit when *either* slit one or slit two are open, the position is *not hit at all* when they both are. If you are confused by quantum mechanics, you are not alone: for decades people have been trying to come up with explanations for "the underlying reality" behind quantum mechanics, including Bohmian Mechanics, Many Worlds and others. However, none of these interpretations have gained universal acceptance and all of those (by design) yield the same experimental predictions. Thus at this point many scientists prefer to just ignore the

question of what is the "true reality" and go back to simply "shutting up and calculating".

R

Remark 22.1 — Complex vs real, other simplifications. If (like the author) you are a bit intimidated by complex numbers, don't worry: you can think of all amplitudes as *real* (though potentially *negative*) numbers without loss of understanding. All the "magic" of quantum computing already arises in this case, and so we will often restrict attention to real amplitudes in this chapter.

We will also only discuss so-called *pure* quantum states, and not the more general notion of *mixed* states. Pure states turn out to be sufficient for understanding the algorithmic aspects of quantum computing.

More generally, this chapter is not meant to be a complete description of quantum mechanics, quantum information theory, or quantum computing, but rather illustrate the main points where these differ from classical computing.

22.3 BELL'S INEQUALITY

There is something weird about quantum mechanics. In 1935 Einstein, Podolsky and Rosen (EPR) tried to pinpoint this issue by highlighting a previously unrealized corollary of this theory. They showed that the idea that nature does not determine the results of an experiment until it is measured results in so called "spooky action at a distance". Namely, making a measurement of one object may instantaneously effect the state (i.e., the vector of amplitudes) of another object in the other end of the universe.

Since the vector of amplitudes is just a mathematical abstraction, the EPR paper was considered to be merely a thought experiment for philosophers to be concerned about, without bearing on experiments. This changed when in 1965 John Bell showed an actual experiment to test the predictions of EPR and hence pit intuitive common sense against the quantum mechanics. Quantum mechanics won: it turns out that it *is* in fact possible to use measurements to create correlations between the states of objects far removed from one another that cannot be explained by any prior theory. Nonetheless, since the results of these experiments are so obviously wrong to anyone that has ever sat in an armchair, that there are still a number of Bell denialists arguing that this can't be true and quantum mechanics is wrong.

So, what is this Bell's Inequality? Suppose that Alice and Bob try to convince you they have telepathic ability, and they aim to prove it via the following experiment. Alice and Bob will be in separate closed

rooms.[2] You will interrogate Alice and your associate will interrogate Bob. You choose a random bit $x \in \{0,1\}$ and your associate chooses a random $y \in \{0,1\}$. We let a be Alice's response and b be Bob's response. We say that Alice and Bob win this experiment if $a \oplus b = x \wedge y$. In other words, Alice and Bob need to output two bits that *disagree* if $x = y = 1$ and *agree* otherwise.[3]

Now if Alice and Bob are not telepathic, then they need to agree in advance on some strategy. It's not hard for Alice and Bob to succeed with probability $3/4$: just always output the same bit. Moreover, by doing some case analysis, we can show that no matter what strategy they use, Alice and Bob cannot succeed with higher probability than that:[4]

Theorem 22.2 — Bell's Inequality. For every two functions $f, g : \{0,1\} \to \{0,1\}$, $\Pr_{x,y \in \{0,1\}}[f(x) \oplus g(y) = x \wedge y] \leq 3/4$.

Proof. Since the probability is taken over all four choices of $x, y \in \{0,1\}$, the only way the theorem can be violated if if there exist two functions f, g that satisfy

$$f(x) \oplus g(y) = x \wedge y \tag{22.1}$$

for all the four choices of $x, y \in \{0,1\}^2$. Let's plug in all these four choices and see what we get (below we use the equalities $z \oplus 0 = z$, $z \wedge 0 = 0$ and $z \wedge 1 = z$):

$$\begin{aligned} f(0) \oplus g(0) &= 0 \quad \text{(plugging in } x = 0, y = 0) \\ f(0) \oplus g(1) &= 0 \quad \text{(plugging in } x = 0, y = 1) \\ f(1) \oplus g(0) &= 0 \quad \text{(plugging in } x = 1, y = 0) \\ f(1) \oplus g(1) &= 1 \quad \text{(plugging in } x = 1, y = 1) \end{aligned} \tag{22.2}$$

If we XOR together the first and second equalities we get $g(0) \oplus g(1) = 0$ while if we XOR together the third and fourth equalities we get $g(0) \oplus g(1) = 1$, thus obtaining a contradiction. ■

An amazing experimentally verified fact is that quantum mechanics allows for "telepathy".[5] Specifically, it has been shown that using the weirdness of quantum mechanics, there is in fact a strategy for Alice and Bob to succeed in this game with probability larger than $3/4$ (in fact, they can succeed with probability about 0.85, see Lemma 22.5).

22.4 QUANTUM WEIRDNESS

Some of the counterintuitive properties that arise from quantum mechanics include:

[2] If you are extremely paranoid about Alice and Bob communicating with one another, you can coordinate with your assistant to perform the experiment exactly at the same time, and make sure that the rooms are sufficiently far apart (e.g., are on two different continents, or maybe even one is on the moon and another is on earth) so that Alice and Bob couldn't communicate to each other in time the results of their respective coins even if they do so at the speed of light.

[3] This form of Bell's game was shown by Clauser, Horne, Shimony, and Holt.

[4] Theorem 22.2 below assumes that Alice and Bob use *deterministic* strategies f and g respectively. More generally, Alice and Bob could use a *randomized* strategy, or equivalently, each could choose f and g from some *distributions* $\mathcal{F}$ and $\mathcal{G}$ respectively. However the *averaging principle* (**??**) implies that if all possible deterministic strategies succeed with probability at most $3/4$, then the same is true for all randomized strategies.

[5] More accurately, one either has to give up on a "billiard ball type" theory of the universe or believe in telepathy (believe it or not, some scientists went for the latter option).

- **Interference** - As we've seen, quantum amplitudes can "cancel each other out".

- **Measurement** - The idea that amplitudes are negative as long as "no one is looking" and "collapse" (by squaring them) to positive probabilities when they are *measured* is deeply disturbing. Indeed, as shown by EPR and Bell, this leads to various strange outcomes such as "spooky actions at a distance", where we can create correlations between the results of measurements in places far removed. Unfortunately (or fortunately?) these strange outcomes have been confirmed experimentally.

- **Entanglement** - The notion that two parts of the system could be connected in this weird way where measuring one will affect the other is known as *quantum entanglement*.

As counter-intuitive as these concepts are, they have been experimentally confirmed, so we just have to live with them.



Remark 22.3 — More on quantum. The discussion in this lecture is quite brief and somewhat superficial. The chapter on quantum computation in my book with Arora (see draft here) is one relatively short resource that contains essentially everything we discuss here and more. See also this blog post of Aaronson for a high level explanation of Shor's algorithm which ends with links to several more detailed expositions. This lecture of Aaronson contains a great discussion of the feasibility of quantum computing (Aaronson's course lecture notes and the book that they spawned are fantastic reads as well). The videos of Umesh Vazirani'z EdX course are an accessible and recommended introduction to quantum computing. See the "bibliographical notes" section at the end of this chapter for more resources.

22.5 QUANTUM COMPUTING AND COMPUTATION - AN EXECUTIVE SUMMARY.

One of the strange aspects of the quantum-mechanical picture of the world is that unlike in the billiard ball example, there is no obvious algorithm to simulate the evolution of n particles over t time periods in $poly(n,t)$ steps. In fact, the natural way to simulate n quantum particles will require a number of steps that is *exponential* in n. This is a huge headache for scientists that actually need to do these calculations in practice.

In the 1981, physicist Richard Feynman proposed to "turn this lemon to lemonade" by making the following almost tautological observation:

> If a physical system cannot be simulated by a computer in T steps, the system can be considered as performing a computation that would take more than T steps.

So, he asked whether one could design a quantum system such that its outcome y based on the initial condition x would be some function $y = f(x)$ such that **(a)** we don't know how to efficiently compute in any other way, and **(b)** is actually useful for something.[6] In 1985, David Deutsch formally suggested the notion of a quantum Turing machine, and the model has been since refined in works of Deutsch and Josza and Bernstein and Vazirani. Such a system is now known as a *quantum computer*.

For a while these hypothetical quantum computers seemed useful for one of two things. First, to provide a general-purpose mechanism to simulate a variety of the real quantum systems that people care about, such as various interactions inside molecules in quantum chemistry. Second, as a challenge to the *Extended Church Turing hypothesis* which says that every physically realizable computation device can be modeled (up to polynomial overhead) by Turing machines (or equivalently, NAND-TM / NAND-RAM programs).

Quantum chemistry is important (and in particular understanding it can be a bottleneck for designing new materials, drugs, and more), but it is still a rather niche area within the broader context of computing (and even scientific computing) applications. Hence for a while most researchers (to the extent they were aware of it), thought of quantum computers as a theoretical curiosity that has little bearing to practice, given that this theoretical "extra power" of quantum computer seemed to offer little advantage in the majority of the problems people want to solve in areas such as combinatorial optimization, machine learning, data structures, etc..

To some extent this is still true today. As far as we know, quantum computers, if built, will *not* provide exponential speed ups for 95% of the applications of computing.[7] In particular, as far as we know, quantum computers will *not* help us solve **NP** complete problems in polynomial or even sub-exponential time, though *Grover's algorithm* (Remark 22.4) does yield a quadratic advantage in many cases.

However, there is one cryptography-sized exception: In 1994 Peter Shor showed that quantum computers can solve the integer factoring and discrete logarithm in polynomial time. This result has captured the imagination of a great many people, and completely energized re-

[6] As its title suggests, Feynman's lecture was actually focused on the other side of simulating physics with a computer. However, he mentioned that as a "side remark" one could wonder if it's possible to simulate physics with a new kind of computer - a "quantum computer" which would "not [be] a Turing machine, but a machine of a different kind". As far as I know, Feynman did not suggest that such a computer could be useful for computations completely outside the domain of quantum simulation. Indeed, he was more interested in the question of whether quantum mechanics could be simulated by a classical computer.

[7] This "95 percent" is a figure of speech, but not completely so. At the time of this writing, cryptocurrency mining electricity consumption is estimated to use up at least 70Twh or 0.3 percent of the world's production, which is about 2 to 5 percent of the total energy usage for the computing industry. All the current cryptocurrencies will be broken by quantum computers. Also, for many web servers the TLS protocol (which is based on the current non-lattice based systems would be completely broken by quantum computing) is responsible for about 1 percent of the CPU usage.

search into quantum computing. This is both because the hardness of these particular problems provides the foundations for securing such a huge part of our communications (and these days, our economy), as well as it was a powerful demonstration that quantum computers could turn out to be useful for problems that a-priori seemed to have nothing to do with quantum physics.

As we'll discuss later, at the moment there are several intensive efforts to construct large scale quantum computers. It seems safe to say that, as far as we know, in the next five years or so there will not be a quantum computer large enough to factor, say, a 1024 bit number. On the other hand, it does seem quite likely that in the very near future there will be quantum computers which achieve *some* task exponentially faster than the best-known way to achieve the same task with a classical computer. When and if a quantum computer is built that is strong enough to break reasonable parameters of Diffie Hellman, RSA and elliptic curve cryptography is anybody's guess. It could also be a "self destroying prophecy" whereby the existence of a small-scale quantum computer would cause everyone to shift away to lattice-based crypto which in turn will diminish the motivation to invest the huge resources needed to build a large scale quantum computer.[8]

[8] Of course, given that we're still hearing of attacks exploiting "export grade" cryptography that was supposed to disappear in 1990's, I imagine that we'll still have products running 1024 bit RSA when everyone has a quantum laptop.

R

Remark 22.4 — Quantum computing and NP. Despite popular accounts of quantum computers as having variables that can take "zero and one at the same time" and therefore can "explore an exponential number of possibilities simultaneously", their true power is much more subtle and nuanced. In particular, as far as we know, quantum computers do *not* enable us to solve **NP** complete problems such as 3SAT in polynomial or even sub-exponential time. However, Grover's search algorithm does give a more modest advantage (namely, quadratic) for quantum computers over classical ones for problems in **NP**. In particular, due to Grover's search algorithm, we know that the k-SAT problem for n variables can be solved in time $O(2^{n/2}poly(n))$ on a quantum computer for every k. In contrast, the best known algorithms for k-SAT on a classical computer take roughly $2^{(1-\frac{1}{k})n}$ steps.

22.6 QUANTUM SYSTEMS

Before we talk about *quantum* computing, let us recall how we physically realize "vanilla" or *classical* computing. We model a *logical bit* that can equal 0 or a 1 by some physical system that can be in one of two states. For example, it might be a wire with high or low voltage,

charged or uncharged capacitor, or even (as we saw) a pipe with or without a flow of water, or the presence or absence of a soldier crab. A *classical* system of n bits is composed of n such "basic systems", each of which can be in either a "zero" or "one" state. We can model the state of such a system by a string $s \in \{0,1\}^n$. If we perform an operation such as writing to the 17-th bit the NAND of the 3rd and 5th bits, this corresponds to applying a *local* function to s such as setting $s_{17} = 1 - s_3 \cdot s_5$.

In the *probabilistic* setting, we would model the state of the system by a *distribution*. For an individual bit, we could model it by a pair of non-negative numbers α, β such that $\alpha + \beta = 1$, where α is the probability that the bit is zero and β is the probability that the bit is one. For example, applying the *negation* (i.e., NOT) operation to this bit corresponds to mapping the pair (α, β) to (β, α) since the probability that $NOT(\sigma)$ is equal to 1 is the same as the probability that σ is equal to 0. This means that we can think of the NOT function as the linear map $N : \mathbb{R}^2 \to \mathbb{R}^2$ such that $N \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \beta \\ \alpha \end{pmatrix}$ or equivalently as the matrix $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.

If we think of the n-bit system as a whole, then since the n bits can take one of 2^n possible values, we model the state of the system as a vector p of 2^n probabilities. For every $s \in \{0,1\}^n$, we denote by e_s the 2^n dimensional vector that has 1 in the coordinate corresponding to s (identifying it with a number in $[2^n]$), and so can write p as $\sum_{s \in \{0,1\}^n} p_s e_s$ where p_s is the probability that the system is in the state s.

Applying the operation above of setting the 17-th bit to the NAND of the 3rd and 5th bits, corresponds to transforming the vector p to the vector Fp where $F : \mathbb{R}^{2^n} \to \mathbb{R}^{2^n}$ is the linear map that maps e_s to $e_{s_0 \cdots s_{16}(1-s_3 \cdot s_5)s_{18} \cdots s_{n-1}}$.[9]

[9] Since $\{e_s\}_{s \in \{0,1\}^n}$ is a *basis* for R^{2^n}, it suffices to define the map F on vectors of this form.



Please make sure you understand why performing the operation will take a system in state p to a system in the state Fp. Understanding the evolution of probabilistic systems is a prerequisite to understanding the evolution of quantum systems.

If your linear algebra is a bit rusty, now would be a good time to review it, and in particular make sure you are comfortable with the notions of *matrices*, *vectors*, (orthogonal and orthonormal) *bases*, and *norms*.

22.6.1 Quantum amplitudes

In the quantum setting, the state of an individual bit (or "qubit", to use quantum parlance) is modeled by a pair of numbers (α, β) such that $|\alpha|^2 + |\beta|^2 = 1$. While in general these numbers can be *complex*, for the rest of this chapter, we will often assume they are *real* (though potentially negative), and hence often drop the absolute value operator. (This turns out not to make much of a difference in explanatory power.) As before, we think of α^2 as the probability that the bit equals 0 and β^2 as the probability that the bit equals 1. As we did before, we can model the NOT operation by the map $N : \mathbb{R}^2 \to \mathbb{R}^2$ where $N(\alpha, \beta) = (\beta, \alpha)$.

Following quantum tradition, instead of using e_0 and e_1 as we did above, from now on we will denote the vector $(1, 0)$ by $|0\rangle$ and the vector $(0, 1)$ by $|1\rangle$ (and moreover, think of these as column vectors). This is known as the Dirac "ket" notation. This means that NOT is the unique linear map $N : \mathbb{R}^2 \to \mathbb{R}^2$ that satisfies $N|0\rangle = |1\rangle$ and $N|1\rangle = |0\rangle$. In other words, in the quantum case, as in the probabilistic case, NOT corresponds to the matrix

$$N = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} . \tag{22.3}$$

In classical computation, we typically think that there are only two operations that we can do on a single bit: keep it the same or negate it. In the quantum setting, a single bit operation corresponds to any linear map $OP : \mathbb{R}^2 \to \mathbb{R}^2$ that is *norm preserving* in the sense that for every α, β, if we apply OP to the vector $\begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ then we obtain a vector $\begin{pmatrix} \alpha' \\ \beta' \end{pmatrix}$ such that $\alpha'^2 + \beta'^2 = \alpha^2 + \beta^2$. Such a linear map OP corresponds to a unitary two by two matrix.[10] Keeping the bit the same corresponds to the matrix $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and (as we've seen) the NOT operations corresponds to the matrix $N = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. But there are other operations we can use as well. One such useful operation is the *Hadamard* operation, which corresponds to the matrix

$$H = \tfrac{1}{\sqrt{2}} \begin{pmatrix} +1 & +1 \\ +1 & -1 \end{pmatrix} . \tag{22.4}$$

In fact it turns out that Hadamard is all that we need to add to a classical universal basis to achieve the full power of quantum computing.

[10] As we mentioned, quantum mechanics actually models states as vectors with *complex* coordinates. However, this does not make any qualitative difference to our discussion.

22.6.2 Recap

The *state* of a *quantum system* of n qubits is modeled by an 2^n dimensional vector ψ of unit norm (i.e., squares of all coordinates sums up to 1), which we write as $\psi = \sum_{x\in\{0,1\}^n} \psi_x |x\rangle$ where $|x\rangle$ is the column vector that has 0 in all coordinates except the one corresponding to x (identifying $\{0,1\}^n$ with the numbers $\{0,\ldots,2^n-1\}$). We use the convention that if a, b are strings of lengths k and ℓ respectively then we can write the $2^{k+\ell}$ dimensional vector with 1 in the ab-th coordinate and zero elsewhere not just as $|ab\rangle$ but also as $|a\rangle|b\rangle$. In particular, for every $x \in \{0,1\}^n$, we can write the vector $|x\rangle$ also as $|x_0\rangle|x_1\rangle\cdots|x_{n-1}\rangle$. This notation satisfies certain nice distributive laws such as $|a\rangle(|b\rangle + |b'\rangle)|c\rangle = |abc\rangle + |ab'c\rangle$.

A *quantum operation* on such a system is modeled by a $2^n \times 2^n$ *unitary matrix* U (one that satisfies $UU^\top = I$ where $U^\top$ is the *transpose* operation, or conjugate transpose for complex matrices). If the system is in state ψ and we apply to it the operation U, then the new state of the system is $U\psi$.

When we *measure* an n-qubit system in a state $\psi = \sum_{x\in\{0,1\}^n} \psi_x|x\rangle$, then we observe the value $x \in \{0,1\}^n$ with probability $|\psi_x|^2$. In this case, the system *collapses* to the state $|x\rangle$.

22.7 ANALYSIS OF BELL'S INEQUALITY (OPTIONAL)

Now that we have the notation in place, we can show a strategy for Alice and Bob to display "quantum telepathy" in Bell's Game. Recall that in the classical case, Alice and Bob can succeed in the "Bell Game" with probability at most $3/4 = 0.75$. We now show that quantum mechanics allows them to succeed with probability at least 0.8.[11]

[11] The strategy we show is not the best one. Alice and Bob can in fact succeed with probability $\cos^2(\pi/8) \sim 0.854$.

Lemma 22.5 There is a 2-qubit quantum state $\psi \in \mathbb{C}^4$ so that if Alice has access to the first qubit of ψ, can manipulate and measure it and output $a \in \{0,1\}$ and Bob has access to the second qubit of ψ and can manipulate and measure it and output $b \in \{0,1\}$ then $\Pr[a \oplus b = x \wedge y] \geq 0.8$.

Proof. Alice and Bob will start by preparing a 2-qubit quantum system in the state

$$\psi = \tfrac{1}{\sqrt{2}}|00\rangle + \tfrac{1}{\sqrt{2}}|11\rangle \tag{22.5}$$

(this state is known as an EPR pair). Alice takes the first qubit of the system to her room, and Bob takes the second qubit to his room. Now, when Alice receives x if $x = 0$ she does nothing and if $x = 1$ she applies the unitary map $R_{-\pi/8}$ to her qubit where $R_\theta = \begin{pmatrix} cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$ is the unitary operation corresponding to

rotation in the plane with angle θ. When Bob receives y, if $y = 0$ he does nothing and if $y = 1$ he applies the unitary map $R_{\pi/8}$ to his qubit. Then each one of them measures their qubit and sends this as their response.

Recall that to win the game Bob and Alice want their outputs to be more likely to differ if $x = y = 1$ and to be more likely to agree otherwise. We will split the analysis in one case for each of the four possible values of x and y.

Case 1: $x = 0$ and $y = 0$. If $x = y = 0$ then the state does not change. * Because the state ψ is proportional to $|00\rangle + |11\rangle$, the measurements of Bob and Alice will always agree (if Alice measures 0 then the state collapses to $|00\rangle$ and so Bob measures 0 as well, and similarly for 1). Hence in the case $x = y = 1$, Alice and Bob always win.

Case 2: $x = 0$ and $y = 1$. If $x = 0$ and $y = 1$ then after Alice measures her bit, if she gets 0 then the system collapses to the state $|00\rangle$, in which case after Bob performs his rotation, his qubit is in the state $\cos(\pi/8)|0\rangle + \sin(\pi/8)|1\rangle$. Thus, when Bob measures his qubit, he will get 0 (and hence agree with Alice) with probability $\cos^2(\pi/8) \geq 0.85$. Similarly, if Alice gets 1 then the system collapses to $|11\rangle$, in which case after rotation Bob's qubit will be in the state $-\sin(\pi/8)|0\rangle + \cos(\pi/8)|1\rangle$ and so once again he will agree with Alice with probability $\cos^2(\pi/8)$.

The analysis for **Case 3**, where $x = 1$ and $y = 0$, is completely analogous to Case 2. Hence Alice and Bob will agree with probability $\cos^2(\pi/8)$ in this case as well.[12]

[12] We are using the (not too hard) observation that the result of this experiment is the same regardless of the order in which Alice and Bob apply their rotations and measurements.

Case 4: $x = 1$ and $y = 1$. For the case that $x = 1$ and $y = 1$, after both Alice and Bob perform their rotations, the state will be proportional to

$$R_{-\pi/8}|0\rangle R_{\pi/8}|0\rangle + R_{-\pi/8}|1\rangle R_{\pi/8}|1\rangle \ . \tag{22.6}$$

Intuitively, since we rotate one state by 45 degrees and the other state by -45 degrees, they will become orthogonal to each other, and the measurements will behave like independent coin tosses that agree with probability 1/2. However, for the sake of completeness, we now show the full calculation.

Opening up the coefficients and using $\cos(-x) = \cos(x)$ and $\sin(-x) = -\sin(x)$, we can see that (22.6) is proportional to

$$\begin{aligned} &\cos^2(\pi/8)|00\rangle + \cos(\pi/8)\sin(\pi/8)|01\rangle \\ -&\sin(\pi/8)\cos(\pi/8)|10\rangle + \sin^2(\pi/8)|11\rangle \\ &-\sin^2(\pi/8)|00\rangle + \sin(\pi/8)\cos(\pi/8)|01\rangle \\ -&\cos(\pi/8)\sin(\pi/8)|10\rangle + \cos^2(\pi/8)|11\rangle \ . \end{aligned} \tag{22.7}$$

Using the trigonometric identities $2\sin(\alpha)\cos(\alpha) = \sin(2\alpha)$ and $\cos^(\alpha) - \sin^2(\alpha) = \cos(2\alpha)$, we see that the probability of getting any one of $|00\rangle, |10\rangle, |01\rangle, |11\rangle$ is proportional to $\cos(\pi/4) = \sin(\pi/4) = \frac{1}{\sqrt{2}}$. Hence all four options for (a, b) are equally likely, which mean that in this case $a = b$ with probability 0.5.

Taking all the four cases together, the overall probability of winning the game is at least $\frac{1}{4} \cdot 1 + \frac{1}{2} \cdot 0.85 + \frac{1}{4} \cdot 0.5 = 0.8$.

■

R

Remark 22.6 — Quantum vs probabilistic strategies. It is instructive to understand what is it about quantum mechanics that enabled this gain in Bell's Inequality. For this, consider the following analogous probabilistic strategy for Alice and Bob. They agree that each one of them output 0 if he or she get 0 as input and outputs 1 with probability p if they get 1 as input. In this case one can see that their success probability would be $\frac{1}{4} \cdot 1 + \frac{1}{2}(1-p) + \frac{1}{4}[2p(1-p)] = 0.75 - 0.5p^2 \leq 0.75$. The quantum strategy we described above can be thought of as a variant of the probabilistic strategy for parameter p set to $\sin^2(\pi/8) = 0.15$. But in the case $x = y = 1$, instead of disagreeing only with probability $2p(1-p) = 1/4$, because we can use these negative probabilities in the quantum world and rotate the state in opposite directions, and hence the probability of disagreement ends up being $\sin^2(\pi/4) = 0.5$.

22.8 QUANTUM COMPUTATION

Recall that in the classical setting, we modeled computation as obtained by a sequence of *basic operations*. We had two types of computational models:

- *Non uniform models of computation* such as Boolean circuits and NAND-CIRC programs, where a finite function $f : \{0,1\}^n \to \{0,1\}$ is computable in size T if it can be expressed as a combination of T basic operations (gates in a circuit or lines in a NAND-CIRC program)
- *Uniform models of computation* such as Turing machines and NAND-TM programs, where an infinite function $F : \{0,1\}^* \to \{0,1\}$ is computable in time $T(n)$ if there is a single algorithm that on input $x \in \{0,1\}^n$ evaluates $F(x)$ using at most $T(n)$ basic steps.

When considering *efficient computation*, we defined the class **P** to consist of all infinite functions $F : \{0,1\}^* \to \{0,1\}$ that can be computed by a Turing machine or NAND-TM program in time $p(n)$ for

some polynomial $p(\cdot)$. We defined the class $\mathbf{P}_{/\mathbf{poly}}$ to consists of all infinite functions $F : \{0,1\}^* \to \{0,1\}$ such that for every n, the restriction $F_{\restriction n}$ of F to $\{0,1\}^n$ can be computed by a Boolean circuit or NAND-CIRC program of size at most $p(n)$ for some polynomial $p(\cdot)$.

We will do the same for *quantum computation*, focusing mostly on the *non uniform* setting of quantum circuits, since that is simpler, and already illustrates the important differences with classical computing.

22.8.1 Quantum circuits

A *quantum circuit* is analogous to a Boolean circuit, and can be described as a directed acyclic graph. One crucial difference that the *out degree* of every vertex in a quantum circuit is at most one. This is because we cannot "reuse" quantum states without *measuring* them (which collapses their "probabilities"). Therefore, we cannot use the same qubit as input for two different gates.[13] Another more technical difference is that to express our operations as unitary matrices, we will need to make sure all our gates are *reversible*. This is not hard to ensure. For example, in the quantum context, instead of thinking of *NAND* as a (non reversible) map from $\{0,1\}^2$ to $\{0,1\}$, we will think of it as the reversible map on *three* qubits that maps a, b, c to $a, b, c \oplus NAND(a,b)$ (i.e., flip the last bit if *NAND* of the first two bits is 1). Equivalently, the NAND operation corresponds to the 8×8 unitary matrix U_{NAND} such that (identifying $\{0,1\}^3$ with $[8]$) for every $a, b, c \in \{0,1\}$, if $|abc\rangle$ is the basis element with 1 in the abc-th coordinate and zero elsewhere, then $U_{NAND}|abc\rangle = |ab(c \oplus NAND(a,b))\rangle$.[14] If we order the rows and columns as $000, 001, 010, \ldots, 111$, then U_{NAND} can be written as the following matrix:

$$U_{NAND} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \tag{22.8}$$

If we have an n qubit system, then for $i, j, k \in [n]$, we will denote by $U^{i,j,k}_{NAND}$ as the $2^n \times 2^n$ unitary matrix that corresponds to applying U_{NAND} to the i-th, j-th, and k-th bits, leaving the others intact. That is, for every $v = \sum_{x \in \{0,1\}^n} v_x |x\rangle$, $U^{i,j,k}_{NAND} v = \sum_{x \in \{0,1\}^n} v_x |x_0 \cdots x_{k-1}(x_k \oplus NAND(x_i, x_j))x_{k+1} \cdots x_{n-1}\rangle$.

As mentioned above, we will also use the *Hadamard* or *HAD* operation, A *quantum circuit* is obtained by applying a sequence of U_{NAND}

[13] This is known as the No Cloning Theorem.

[14] Readers familiar with quantum computing should note that U_{NAND} is a close variant of the so called Toffoli gate and so QNAND-CIRC programs correspond to quantum circuits with the Hadamard and Toffoli gates.

and *HAD* gates, which correspond to the matrix

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} +1 & +1 \\ +1 & -1 \end{pmatrix} . \tag{22.9}$$

Another way to write define H is that for $b \in \{0,1\}$, $H|b\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}(-1)^b|1\rangle$. We define HAD^i to be the $2^n \times 2^n$ unitary matrix that applies *HAD* to the i-th qubit and leaves the others intact. Using the ket notation, we can write this as

$$HAD^i \sum_{x\in\{0,1\}^n} v_x|x\rangle = \frac{1}{\sqrt{2}} \sum_{x\in\{0,1\}^n} |x_0 \cdots x_{i-1}\rangle \left(|0\rangle + (-1)^{x_i}|1\rangle\right) |x_i \cdots x_{n-1}\rangle . \tag{22.10}$$

A *quantum circuit* is obtained by composing these basic operations on some m qubits. If $m \geq n$, we use a circuit to compute a function $f : \{0,1\}^n \to \{0,1\}$:

- On input x, we initialize the system to hold $x_0, \ldots, x_{n-1}$ in the first n qubits, and initialize all remaining $m - n$ qubits to zero.
- We execute each elementary operation one by one.
- At the end of the computation, we *measure* the system, and output the result of the last qubit (i.e. the qubit in location $m - 1$).[15]
- We say that the circuit *computes* f, if the probability that this output equals $f(x)$ is at least $2/3$. Note that this probability is obtained by summing up the squares of the amplitudes of all coordinates in the final state of the system corresponding to vectors $|y\rangle$ where $y_{m-1} = f(x)$.

[15] For simplicity we restrict attention to functions with a single bit of output, though the definition of quantum circuits naturally extends to circuits with multiple outputs.

Formally this is defined as follows:

Definition 22.7 — Quantum circuit. A *quantum circuit of m inputs and s gates* over the $\{U_{NAND}, HAD\}$ basis is a sequence of s unitary $2^n \times 2^n$ matrices $U_0, \ldots, U_{s-1}$ such that each matrix U_ℓ is either of the form $NAND^{i,j,k}$ for $i, j, k \in [n]$ or HAD^i for $i \in [n]$.

A quantum circuit *computes* a function $f : \{0,1\}^n \to \{0,1\}$ if the following is true for every $x \in \{0,1\}^n$:

Let v be the vector

$$v = U_{s-1}U_{s-2}\cdots U_1U_0|x0^{m-n}\rangle \tag{22.11}$$

and write v as $\sum_{y\in\{0,1\}^m} v_y|y\rangle$. Then

$$\sum_{y\in\{0,1\}^m \text{ s.t. } y_{m-1}=f(x)} |v_y|^2 \geq \frac{2}{3} . \tag{22.12}$$

P

Please stop here and see that this definition makes sense to you.

Once we have the notion of quantum circuits, we can define the quantum analog of $\mathbf{P}_{/\mathbf{poly}}$ (i.e., define the class of functions computable by *polynomial size quantum circuits*) as follows:

Definition 22.8 — $\mathbf{BQP}_{/\mathbf{poly}}$. Let $F : \{0,1\}^* \to \{0,1\}$. We say that $F \in \mathbf{BQP}_{/\mathbf{poly}}$ if there exists some polynomial $p : \mathbb{N} \to \mathbb{N}$ such that for every $n \in \mathbb{N}$, if $F_{\restriction n}$ is the restriction of F to inputs of length n, then there is a quantum circuit of size at most $p(n)$ that computes $F_{\restriction n}$.

R

Remark 22.9 — The obviously exponential fallacy. A priori it might seem "obvious" that quantum computing is exponentially powerful, since to perform a quantum computation on n bits we need to maintain the 2^n dimensional state vector and apply $2^n \times 2^n$ matrices to it. Indeed popular descriptions of quantum computing (too) often say something along the lines that the difference between quantum and classical computer is that a classical bit can either be zero or one while a qubit can be in both states at once, and so in many qubits a quantum computer can perform exponentially many computations at once.

Depending on how you interpret it, this description is either false or would apply equally well to *probabilistic computation*, even though we've already seen that every randomized algorithm can be simulated by a similar-sized circuit, and in fact we conjecture that $\mathbf{BPP} = \mathbf{P}$.

Moreover, this "obvious" approach for simulating a quantum computation will take not just exponential time but *exponential space* as well, while can be shown that using a simple recursive formula one can calculate the final quantum state using *polynomial space* (in physics this is known as "Feynman path integrals"). So, the exponentially long vector description by itself does not imply that quantum computers are exponentially powerful. Indeed, we cannot *prove* that they are (i.e., as far as we know, every QNAND-CIRC program could be simulated by a NAND-CIRC program with polynomial overhead), but we do have some problems (integer factoring most prominently) for which they

do provide exponential speedup over the currently best *known* classical (deterministic or probabilistic) algorithms.

22.8.2 QNAND-CIRC programs (optional)

Just like in the classical case, there is an equivalence between circuits and straight-line programs, and so we can define the programming language QNAND that is the quantum analog of our NAND-CIRC programming language. To do so, we only add a single operation: `HAD(foo)` which applies the single-bit operation H to the variable `foo`. We also use the following interpretation to make `NAND` reversible: `foo = NAND(bar,blah)` means that we modify `foo` to be the XOR of its original value and the NAND of `bar` and `blah`. (In other words, apply the 8 by 8 unitary transformation U_{NAND} defined above to the three qubits corresponding to `foo`, `bar` and `blah`.) If `foo` is initialized to zero then this makes no difference.

If P is a QNAND-CIRC program with n input variables, ℓ workspace variables, and m output variables, then running it on the input $x \in \{0,1\}^n$ corresponds to setting up a system with $n+m+\ell$ qubits and performing the following process:

1. We initialize the input variables `X[0]` ... `X[`$n-1$`]` to $x_0, \ldots, x_{n-1}$ and all other variables to 0.
2. We execute the program line by line, applying the corresponding physical operation H or U_{NAND} to the qubits that are referred to by the line.
3. We *measure* the output variables `Y[0]`, ..., `Y[`$m-1$`]` and output the result (if there is more than one output then we measure more variables).

22.8.3 Uniform computation

Just as in the classical case, we can define *uniform* computational models. For example, we can define the *QNAND-TM programming language* to be QNAND augmented with loops and arrays just like NAND-TM is obtained from NAND. Using this we can define the class **BQP** which is the uniform analog of $\mathbf{BQP}_{/\mathbf{poly}}$. Just as in the classical setting it holds that $\mathbf{BPP} \subseteq \mathbf{P}_{/\mathbf{poly}}$, in the quantum setting it can be shown that $\mathbf{BQP} \subseteq \mathbf{BQP}_{/\mathbf{poly}}$. Just like the classical case, we can also use Quantum Turing Machines instead of QNAND-TM to define **BQP**.

Yet another way to define **BQP** is the following: a function $F : \{0,1\}^* \rightarrow \{0,1\}$ is in **BQP** if **(1)** $F \in \mathbf{BQP}_{/\mathbf{poly}}$ and **(2)** moreover for every n, the quantum circuit that verifies this can be generated by a *classical polynomial time NAND-TM program* (or, equivalently,

a polynomial-time Turing machine).[16] We use this definition here, though an equivalent one can be made using QNAND-TM or quantum Turing machines:

Definition 22.10 — The class BQP. Let $F : \{0,1\}^* \to \{0,1\}$. We say that $F \in$ **BQP** if there exists a polynomial time NAND-TM program P such that for every n, $P(1^n)$ is the description of a quantum circuit C_n that computes the restriction of F to $\{0,1\}^n$.

P

One way to verify that you've understood these definitions it to see that you can prove (1) **P** $\subseteq$ **BQP** and in fact the stronger statement **BPP** $\subseteq$ **BQP**, (2) **BQP** $\subseteq$ **EXP**, and (3) For every **NP**-complete function F, if $F \in$ **BQP** then **NP** $\subseteq$ **BQP**. Exercise 22.1 asks you to work these out.

The relation between **NP** and **BQP** is not known (see also Remark 22.4). It is widely believed that **NP** $\not\subseteq$ **BQP**, but there is no consensus whether or not **BQP** $\subseteq$ **NP**. It is quite possible that these two classes are *incomparable*, in the sense that **NP** $\not\subseteq$ **BQP** (and in particular no **NP**-complete function belongs to **BQP**) but also **BQP** $\not\subseteq$ **NP** (and there are some interesting candidates for such problems).

It can be shown that *QNANDEVAL* (evaluating a quantum circuit on an input) is computable by a polynomial size QNAND-CIRC program, and moreover this program can even be generated *uniformly* and hence *QNANDEVAL* is in **BQP**. This allows us to "port" many of the results of classical computational complexity into the quantum realm as well.

R

Remark 22.11 — Restricting attention to circuits. Because the non uniform model is a little cleaner to work with, in the rest of this chapter we mostly restrict attention to this model, though all the algorithms we discuss can be implemented in uniform computation as well.

22.9 PHYSICALLY REALIZING QUANTUM COMPUTATION

To realize quantum computation one needs to create a system with n independent binary states (i.e., "qubits"), and be able to manipulate small subsets of two or three of these qubits to change their state. While by the way we defined operations above it might seem that one needs to be able to perform arbitrary unitary operations on these two or three qubits, it turns out that there several choices for *universal*

[16] This is analogous to the alternative characterization of **P** that appears in Exercise 12.6.

sets - a small constant number of gates that generate all others. The biggest challenge is how to keep the system from being measured and *collapsing* to a single classical combination of states. This is sometimes known as the *coherence time* of the system. The threshold theorem says that there is some absolute constant level of errors τ so that if errors are created at every gate at rate smaller than τ then we can recover from those and perform arbitrary long computations. (Of course there are different ways to model the errors and so there are actually several threshold *theorems* corresponding to various noise models).

There have been several proposals to build quantum computers:

- Superconducting quantum computers use superconducting electric circuits to do quantum computation. This is the direction where there has been most recent progress towards "beating" classical computers.
- Trapped ion quantum computers Use the states of an ion to simulate a qubit. People have made some recent advances on these computers too. While it's not at all clear that's the right measuring yard, the current best implementation of Shor's algorithm (for factoring 15) is done using an ion-trap computer.
- Topological quantum computers use a different technology. Topological qubits are more stable by design and hence error correction is less of an issue, but constructing them is extremely challenging.

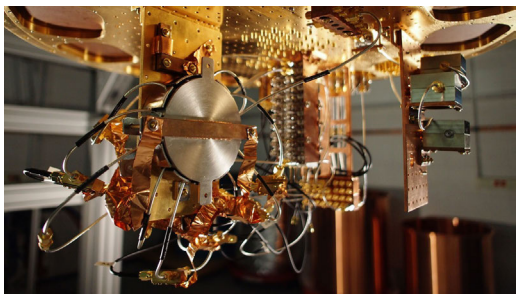


Figure 22.3: Superconducting quantum computer prototype at Google. Image credit: Google / MIT Technology Review.

These approaches are not mutually exclusive and it could be that ultimately quantum computers are built by combining all of them together. In the near future, it seems that we will not be able to achieve full fledged large scale universal quantum computers, but rather more restricted machines, sometimes called "Noisy Intermediate-Scale Quantum Computers" or "NISQ". See this article by John Preskil for some of the progress and applications of such more restricted devices.

22.10 SHOR'S ALGORITHM: HEARING THE SHAPE OF PRIME FACTORS

Bell's Inequality is a powerful demonstration that there is something very strange going on with quantum mechanics. But could this "strangeness" be of any use to solve computational problems not directly related to quantum systems? A priori, one could guess the answer is *no*. In 1994 Peter Shor showed that one would be wrong:

Theorem 22.12 — Shor's Algorithm. There is a polynomial-time quantum algorithm that on input an integer M (represented in base two), outputs the prime factorization of M.

Another way to state Theorem 22.12 is that if we define *FACTORING* $: \{0,1\}^* \to \{0,1\}$ to be the function that on input a pair of numbers (M, X) outputs 1 if and only if M has a factor P such that $2 \leq P \leq X$, then *FACTORING* is in **BQP**. This is an exponential improvement over the best known classical algorithms, which take roughly $2^{\tilde{O}(n^{1/3})}$ time, where the $\tilde{O}$ notation hides factors that are polylogarithmic in n. While we will not prove Theorem 22.12 in this chapter, we will sketch some of the ideas behind the proof.

22.10.1 Period finding

At the heart of Shor's Theorem is an efficient quantum algorithm for finding *periods* of a given function. For example, a function $f : \mathbb{R} \to \mathbb{R}$ is *periodic* if there is some $h > 0$ such that $f(x + h) = f(x)$ for every x (e.g., see Fig. 22.4).

Musical notes yield one type of periodic function. When you pull on a string on a musical instrument, it vibrates in a repeating pattern. Hence, if we plot the speed of the string (and so also the speed of the air around it) as a function of time, it will correspond to some *periodic* function. The length of the period is known as the *wave length* of the note. The *frequency* is the number of times the function repeats itself within a unit of time. For example, the "Middle C" note has a frequency of 261.63 Hertz, which means its period is $1/(261.63)$ seconds.

If we play a *chord* by playing several notes at once, we get a more complex periodic function obtained by combining the functions of the individual notes (see Fig. 22.5). The human ear contains many small hairs, each of which is sensitive to a narrow band of frequencies. Hence when we hear the sound corresponding to a chord, the hairs in our ears actually separate it out to the components corresponding to each frequency.

It turns out that (essentially) *every* periodic function $f : \mathbb{R} \to \mathbb{R}$ can be decomposed into a sum of simple *wave* functions (namely functions of the form $x \mapsto \sin(\theta x)$ or $x \mapsto \cos(\theta x)$). This is known as the Fourier Transform (see Fig. 22.6). The Fourier transform makes it easy to compute the period of a given function: it will simply be the least common multiple of the periods of the constituent waves.

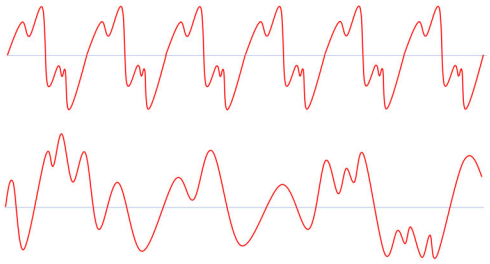


Figure 22.4: Top: A periodic function. Bottom: An a-periodic function.

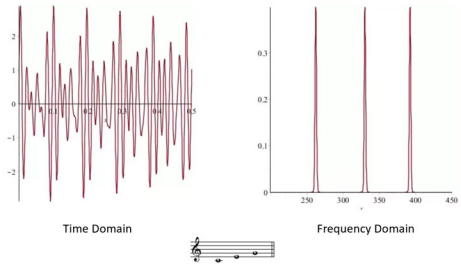


Figure 22.5: Left: The air-pressure when playing a "C Major" chord as a function of time. Right: The coefficients of the Fourier transform of the same function, we can see that it is the sum of three freuencies corresponding to the C, E and G notes (261.63, 329.63 and 392 Hertz respectively). Credit: Bjarke Mønsted's Quora answer.

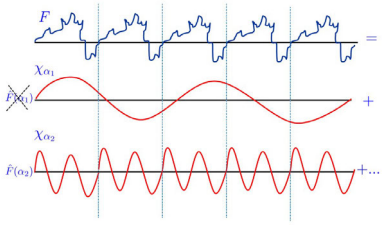


Figure 22.6: If f is a periodic function then when we represent it in the Fourier transform, we expect the coefficients corresponding to wavelengths that do not evenly divide the period to be very small, as they would tend to "cancel out".

22.10.2 Shor's Algorithm: A bird's eye view

On input a an integer M, Shor's algorithm outputs the prime factorization of M in time that is polynomial in $\log M$. The main steps in the algorithm are the following:

Step 1: Reduce to period finding. The first step in the algorithm is to pick a random $A \in \{0, 1 \ldots, M-1\}$ and define the function $F_A : \{0,1\}^m \to \{0,1\}^m$ as $F_A(x) = A^x (\mod M)$ where we identify the

string $x \in \{0,1\}^m$ with an integer using the binary representation, and similarly represent the integer $A^x(\mod M)$ as a string. (We will choose m to be some polynomial in m and so in particular $\{0,1\}^m$ is a large enough set to represent all the numbers in $\{0,1,\ldots,M-1\}$).

Some not-too-hard (though somewhat technical) calculations show that: **(1)** The function F_A is *periodic* (i.e., there is some integer p_A such that $F_A(x+p_A) = F_A(x)$ for almost[17] every x) and more importantly **(2)** If we can recover the period p_A of F_A for several randomly chosen A's, then we can recover the factorization of M. Hence, factoring M reduces to finding out the period of the function F_A. Exercise 22.2 asks you to work out this for the related task of computing the *discrete logarithm* (which underlies the security of the Diffie-Hellman key exchange and elliptic curve cryptography).

[17] We'll ignore this "almost" qualifier in the discussion below. It causes some annoying, yet ultimately manageable, technical issues in the full-fledged algorithm.

Step 2: Period finding via the Quantum Fourier Transform. Using a simple trick known as "repeated squaring", it is possible to compute the map $x \mapsto F_A(x)$ in time polynomial in m, which means we can also compute this map using a polynomial number of NAND gates,and so in particular we can generate in polynomial quantum time a quantum state ρ that is (up to normalization) equal to

$$\sum_{x\in\{0,1\}^m} |x\rangle|F_A(x)\rangle \quad . \tag{22.13}$$

In particular, if we were to *measure* the state ρ, we would get a random pair of the form (x,y) where $y = F_A(x)$. So far, this is not at all impressive. After all, we did not need the power of quantum computing to generate such pairs: we could simply generate a random x and then compute $F_A(x)$.

Another way to describe the state ρ is that the coefficient of $|x\rangle|y\rangle$ in ρ is proportional to $f_{A,y}(x)$ where $f_{A,y} : \{0,1\}^m \to \mathbb{R}$ is the function such that

$$f_{A,y}(x) = \begin{cases} 1 & y = A^x(\mod M) \\ 0 & \text{otherwise} \end{cases} \quad . \tag{22.14}$$

The magic of Shor's algorithm comes from a procedure known as the *Quantum Fourier Transform*. It allows to change the state ρ into the state $\hat{\rho}$ where the coefficient of $|x\rangle|y\rangle$ is now proportional to the *x-th Fourier coefficient* of $f_{A,y}$. In other words, if we measure the state $\hat{\rho}$, we will obtain a pair (x,y) such that the probability of choosing x is proportional to the square of the weight of the *frequency* x in the representation of the function $f_{A,y}$. Since for every y, the function $f_{A,y}$ has the period p_A, it can be shown that the frequency x will be (almost[18]) a multiple of p_A. If we make several such samples $y_0,\ldots,y_k$ and obtain the frequencies $x_1,\ldots,x_k$, then the true period p_A divides all of them, and it can be shown that it is going to be in fact the *greatest*

[18] The "almost" qualifier again appears because the original function was only "almost" periodic, but it turns out this can be handled by using an "approximate greatest common divisor" algorithm instead of a standard g.c.d. below. The latter can be obtained using a tool known as the continued fraction representation of a number.

common divisor (g.c.d.) of all these frequencies: a value which can be computed in polynomial time.

As mentioned above, we can recover the factorization of M from the periods of $F_{A_0}, \ldots, F_{A_t}$ for some randomly chosen $A_0, \ldots, A_t$ in $\{0, \ldots, M-1\}$ and t which is polynomial in $\log M$.

The resulting algorithm can be described in a high (and somewhat inaccurate) level as follows:

Shor's Algorithm: (*sketch*)

Input: Number $M \in \mathbb{N}$.

Output: Prime factorization of M.

Operations:

1. Repeat the following $k = poly(\log M)$ number of times:
   a. Choose $A \in \{0, \ldots, M-1\}$ at random, and let $f_A : \mathbb{Z}_M \to \mathbb{Z}_M$ be the map $x \mapsto A^x \mod M$.
   b. For $t = poly(\log M)$, repeat t times the following step: *Quantum Fourier Transform* to create a quantum state $|\psi\rangle$ over $poly(\log(m))$ qubits, such that if we measure $|\psi\rangle$ we obtain a pair of strings (j, y) with probability proportional to the square of the coefficient corresponding to the wave function $x \mapsto \cos(x\pi j/M)$ or $x \mapsto \sin(x\pi j/M)$ in the Fourier transform of the function $f_{A,y} : \mathbb{Z}_m \to \{0, 1\}$ defined as $f_{A,y}(x) = 1$ iff $f_A(x) = y$.
   c. If $j_1, \ldots, j_t$ are the coefficients we obtained in the previous step, then the least common multiple of $M/j_1, \ldots, M/j_t$ is likely to be the *period* of the function f_A.
2. If we let $A_0, \ldots, A_{k-1}$ and $p_0, \ldots, p_{k-1}$ be the numbers we chose in the previous step and the corresponding periods of the functions $f_{A_0}, \ldots, f_{A_{k-1}}$ then we can use classical results in number theory to obtain from these a non-trivial prime factor Q of M (if such exists). We can now run the algorithm again with the (smaller) input M/Q to obtain all other factors.

Reducing factoring to order finding is cummbersome, but can be done in polynomial time using a classical computer. The key quantum ingredient in Shor's algorithm is the *quantum fourier transform*.

R

Remark 22.13 — Quantum Fourier Transform. Despite its name, the Quantum Fourier Transform does *not* actually give a way to compute the Fourier Transform of a function $f : \{0,1\}^m \to \mathbb{R}$. This would be impossible to do in time polynomial in m, as simply writing down the Fourier Transform would require 2^m coefficients. Rather the Quantum Fourier Transform gives a *quantum state* where the amplitude corresponding to an element (think: frequency) h is equal to the corresponding Fourier coefficient. This allows to sample from a distribution where h is drawn with probability proportional to the square of its Fourier coefficient. This is not the same as computing the Fourier transform, but is good enough for recovering the period.

22.11 QUANTUM FOURIER TRANSFORM (ADVANCED, OPTIONAL)

The above description of Shor's algorithm skipped over the implementation of the main quantum ingredient: the *Quantum Fourier Transform* algorithm. In this section we discuss the ideas behind this algorithm. We will be rather brief and imprecise. Remark 22.3 and Section 22.13 contain references to sources of more information about this topic.

To understand the Quantum Fourier Transform, we need to better understand the Fourier Transform itself. In particular, we will need to understand how it applies not just to functions whose input is a real number but to functions whose domain can be any arbitrary commutative *group*. Therefore we now take a short detour to (very basic) *group theory*, and define the notion of periodic functions over groups.

R

Remark 22.14 — Group theory. While we define the concepts we use, some background in group or number theory might be quite helpful for fully understanding this section.

We will not use anything more than the basic properties of finite Abelian groups. Specifically we use the following notions:

- A finite *group* $\mathbb{G}$ can be thought of as simply a set of elements and some *binary operation* $\star$ on these elements (i.e., if $g, h \in \mathbb{G}$ then $g \star h$ is an element of $\mathbb{G}$ as well).
- The operation $\star$ satisfies the sort of properties that a product operation does, namely, it is *associative* (i.e., $(g \star h) \star f = g \star (h \star f)$) and there is some element 1 such that $g \star 1 = g$ for all g, and for

every $g \in \mathbb{G}$ there exists an element g^{-1} such that $g \star g^{-1} = 1$.

- A group is called *commutative* (also known as *Abelian*) if $g \star h = h \star g$ for all $g, h \in \mathbb{G}$.

The Fourier transform is a deep and vast topic, on which we will barely touch upon here. Over the real numbers, the Fourier transform of a function f is obtained by expressing f in the form $\sum \hat{f}(\alpha)\chi_\alpha$ where the χ_α's are "wave functions" (e.g. sines and cosines). However, it turns out that the same notion exists for *every* Abelian group $\mathbb{G}$. Specifically, for every such group $\mathbb{G}$, if f is a function mapping $\mathbb{G}$ to $\mathbb{C}$, then we can write f as

$$f = \sum_{g \in \mathbb{G}} \hat{f}(g)\chi_g \quad , \tag{22.15}$$

where the χ_g's are functions mapping $\mathbb{G}$ to $\mathbb{C}$ that are analogs of the "wave functions" for the group $\mathbb{G}$ and for every $g \in \mathbb{G}$, $\hat{f}(g)$ is a complex number known as the *Fourier coefficient of f corresponding to g*.[19] The representation (**??**) is known as the *Fourier expansion* or *Fourier transform* of f, the numbers $(\hat{f}(g))_{g\in\mathbb{G}}$ are known as the *Fourier coefficients* of f and the functions $(\chi_g)_{g\in\mathbb{G}}$ are known as the *Fourier characters*. The central property of the Fourier characters is that they are *homomorphisms* of the group into the complex numbers, in the sense that for every $x, x' \in \mathbb{G}$, $\chi_g(x \star x') = \chi_g(x)\chi_g(x')$, where $\star$ is the group operation. One corollary of this property is that if $\chi_g(h) = 1$ then χ_g is h *periodic* in the sense that $\chi_g(x \star h) = \chi_g(x)$ for every x. It turns out that if f is periodic with minimal period h, then the only Fourier characters that have non zero coefficients in the expression (22.15) are those that are h periodic as well. This can be used to recover the period of f from its Fourier expansion.

[19] The equation (22.15) means that if we think of f as a $|\mathbb{G}|$ dimensional vector over the complex numbers, then we can write this vector as a sum (with certain coefficients) of the vectors $\{\chi_g\}_{g\in\mathbb{G}}$.

22.11.1 Quantum Fourier Transform over the Boolean Cube: Simon's Algorithm

We now describe the simplest setting of the Quantum Fourier Transform: the group $\{0,1\}^n$ with the XOR operation, which we'll denote by $(\{0,1\}^n, \oplus)$. It can be shown that the Fourier transform over $(\{0,1\}^n, \oplus)$ corresponds to expressing $f : \{0,1\}^n \to \mathbb{C}$ as

$$f = \sum_{y \in \{0,1\}} \hat{f}(y)\chi_y \tag{22.16}$$

where $\chi_y : \{0,1\}^n \to \mathbb{C}$ is defined as $\chi_y(x) = (-1)^{\sum_i y_i x_i}$ and $\hat{f}(y) = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} f(x)(-1)^{\sum_i y_i x_i}$.

The Quantum Fourier Transform over $(\{0,1\}^n, \oplus)$ is actually quite simple:

Theorem 22.15 — QFT Over the Boolean Cube. Let $\rho = \sum_{x\in\{0,1\}^n} f(x)|x\rangle$ be a quantum state where $f : \{0,1\}^n \to \mathbb{C}$ is some function satisfying $\sum_{x\in\{0,1\}^n} |f(x)|^2 = 1$. Then we can use n gates to transform ρ to the state

$$\sum_{y\in\{0,1\}^n} \hat{f}(y)|y\rangle \tag{22.17}$$

where $f = \sum_y \hat{f}(y)\chi_y$ and $\chi_y : \{0,1\}^n \to \mathbb{C}$ is the function $\chi_y(x) = -1^{\sum x_i y_i}$.

Proof Idea:

The idea behind the proof is that the *Hadamard* operation corresponds to the *Fourier transform* over the group $\{0,1\}^n$ (with the XOR operations). To show this, we just need to do the calculations.

⋆

Proof of Theorem 22.15. We can express the Hadamard operation *HAD* as follows:

$$HAD|a\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle) \; . \tag{22.18}$$

We are given the state

$$\rho = \sum_{x\in\{0,1\}^n} f(x)|x\rangle \; . \tag{22.19}$$

Now suppose that we apply the *HAD* operation to each of the n qubits. We can see that we get the state

$$2^{-n/2} \sum_{x\in\{0,1\}^n} f(x) \prod_{i=0}^{n-1} (|0\rangle + (-1)^{x_i}|1\rangle) \; . \tag{22.20}$$

We can now use the distributive law and open up a term of the form

$$f(x)(|0\rangle + (-1)^{x_0}|1\rangle) \cdots (|0\rangle + (-1)^{x_{n-1}}|1\rangle) \tag{22.21}$$

to the following sum over 2^n terms:[20]

$$f(x) \sum_{y\in\{0,1\}^n} (-1)^{\sum y_i x_i}|y\rangle \; . \tag{22.22}$$

But by changing the order of summations, we see that the final state is

$$\sum_{y\in\{0,1\}^n} 2^{-n/2} \Big(\sum_{x\in\{0,1\}^n} f(x)(-1)^{\sum x_i y_i} \Big)|y\rangle \tag{22.23}$$

[20] If you find this confusing, try to work out why $(|0\rangle + (-1)^{x_0}|1\rangle)(|0\rangle + (-1)^{x_1}|1\rangle)(|0\rangle + (-1)^{x_2}|1\rangle)$ is the same as the sum over 2^3 terms $|000\rangle + (-1)^{x_2}|001\rangle + \cdots + (-1)^{x_0+x_1+x_2}|111\rangle$.

which exactly corresponds to $\hat{\rho}$.

■

22.11.2 From Fourier to Period finding: Simon's Algorithm (advanced, optional)

Using Theorem 22.15 it is not hard to get an algorithm that can recover a string $h^* \in \{0,1\}^n$ given a circuit that computes a function $F : \{0,1\}^n \to \{0,1\}^*$ that is h^* *periodic* in the sense that $F(x) = F(x')$ for distinct x, x' if and only if $x' = x \oplus h^*$. The key observation is that if we compute the state $\sum_{x\in\{0,1\}^n} |x\rangle|F(x)\rangle$, and perform the Quantum Fourier transform on the first n qubits, then we would get a state such that the only basis elements with nonzero coefficients would be of the form $|y\rangle$ where

$$\sum y_i h_i^* = 0 (\mod 2) \tag{22.24}$$

So, by measuring the state, we can obtain a sample of a random y satisfying (22.24). But since (22.24) is a *linear* equation modulo 2 about the unknown n variables $h_0^*, \ldots, h_{n-1}^*$, if we repeat this procedure to get n such equations, we will have at least as many equations as variables and (it can be shown that) this will suffice to recover h^*.

This result is known as Simon's Algorithm, and it preceded and inspired Shor's algorithm.

22.11.3 From Simon to Shor (advanced, optional)

Theorem 22.15 seemed to really use the special bit-wise structure of the group $\{0,1\}^n$, and so one could wonder if it can be extended to other groups. However, it turns out that we can in fact achieve such a generalization.

The key step in Shor's algorithm is to implement the Fourier transform for the group $\mathbb{Z}_L$ which is the set of numbers $\{0, \ldots, L-1\}$ with the operation being addition modulo L. In this case it turns out that the Fourier characters are the functions $\chi_y(x) = \omega^{yx}$ where $\omega = e^{2\pi i/L}$ (i here denotes the complex number $\sqrt{-1}$). The y-th Fourier coefficient of a function $f : \mathbb{Z}_L \to \mathbb{C}$ is

$$\hat{f}(y) = \frac{1}{\sqrt{L}} \sum_{x\in\mathbb{Z}_L} f(x)\omega^{xy} \,. \tag{22.25}$$

The key to implementing the Quantum Fourier Transform for such groups is to use the same recursive equations that enable the classical Fast Fourier Transform (FFT) algorithm. Specifically, consider the case that $L = 2^\ell$. We can separate the sum over x in (22.25) to the terms

corresponding to even x's (of the form $x = 2z$) and odd x's (of the form $x = 2z+1$) to obtain

$$\hat{f}(y) = \frac{1}{\sqrt{L}} \sum_{z \in Z_{L/2}} f(2z)(\omega^2)^{yz} + \frac{\omega^y}{\sqrt{L}} \sum_{z \in \mathbb{Z}_{L/2}} f(2z+1)(\omega^2)^{yz} \qquad (22.26)$$

which reduces computing the Fourier transform of f over the group $\mathbb{Z}_{2^\ell}$ to computing the Fourier transform of the functions f_{even} and f_{odd} (corresponding to the applying f to only the even and odd x's respectively) which have $2^{\ell-1}$ inputs that we can identify with the group $\mathbb{Z}_{2^{\ell-1}} = \mathbb{Z}_{L/2}$.

Specifically, the Fourier characters of the group $\mathbb{Z}_{L/2}$ are the functions $\chi_y(x) = e^{2\pi i/(L/2)yx} = (\omega^2)^{yx}$ for every $x, y \in \mathbb{Z}_{L/2}$. Moreover, since $\omega^L = 1$, $(\omega^2)^y = (\omega^2)^{y \mod L/2}$ for every $y \in \mathbb{N}$. Thus (22.26) translates into

$$\hat{f}(y) = \hat{f}_{even}(y \mod L/2) + \omega^y \hat{f}_{odd}(y \mod L/2) \,. \qquad (22.27)$$

This observation is usually used to obtain a fast (e.g. $O(L \log L)$) time to compute the Fourier transform in a classical setting, but it can be used to obtain a quantum circuit of $poly(\log L)$ gates to transform a state of the form $\sum_{x \in \mathbb{Z}_L} f(x)|x\rangle$ to a state of the form $\sum_{y \in \mathbb{Z}_L} \hat{f}(y)|y\rangle$.

The case that L is not an exact power of two causes some complications in both the classical case of the Fast Fourier Transform and the quantum setting of Shor's algorithm. However, it is possible to handle these. The idea is that we can embed Z_L in the group $\mathbb{Z}_{A \cdot L}$ for any integer A, and we can find an integer A such that $A \cdot L$ will be close enough to a power of 2 (i.e., a number of the form 2^m for some m), so that if we do the Fourier transform over the group $\mathbb{Z}_{2^m}$ then we will not introduce too many errors.



Lecture Recap

- The state of an n-qubit quantum system can be modeled as a 2^n dimensional vector
- An operation on the state corresponds to applying a unitary matrix to this vector.
- Quantum circuits are obtained by composing basic operations such as *HAD* and U_{NAND}.
- We can use quantum circuits to define the classes **BQP**$_{/\textbf{poly}}$ and **BQP** which are the quantum analogs of **P**$_{/\textbf{poly}}$ and **BPP** respectively.
- There are some problems for which the best known quantum algorithm is *exponentially faster* than the best known, but quantum computing is not a panacea. In particular, as far as we know, quantum computers could still require exponential time to solve **NP**-complete problems such as *SAT*.

22.12 EXERCISES

R

Remark 22.16 — Disclaimer. Most of the exercises have been written in the summer of 2018 and haven't yet been fully debugged. While I would prefer people do not post online solutions to the exercises, I would greatly appreciate if you let me know of any bugs. You can do so by posting a GitHub issue about the exercise, and optionally complement this with an email to me with more details about the attempted solution.

Exercise 22.1 — Quantum and classical complexity class relations. Prove the following relations between quantum complexity classes and classical ones:

1. $\mathbf{P}_{/\mathbf{poly}} \subseteq \mathbf{BQP}_{/\mathbf{poly}}$.[21]
2. $\mathbf{P} \subseteq \mathbf{BQP}$.[22]
3. $\mathbf{BPP} \subseteq \mathbf{BQP}$.[23]
4. $\mathbf{BQP} \subseteq \mathbf{EXP}$.[24]
5. If $SAT \in \mathbf{BQP}$ then $\mathbf{NP} \subseteq \mathbf{BQP}$.[25]

[21] *Hint:* You can use U_{NAND} to simulate NAND gates.

[22] *Hint:* Use the alternative characterization of **P** as in Exercise 12.6.

[23] *Hint:* You can use the *HAD* gate to simulate a coin toss.

[24] *Hint:* In exponential time simulating quantum computation boils down to matrix multiplication.

[25] *Hint:* If a reduction can be implemented in **P** it can be implemented in **BQP** as well.

■

Exercise 22.2 — Discrete logarithm from order finding. Show a probabilistic polynomial time classical algorithm that given an Abelian finite group $\mathbb{G}$ (in the form of an algorithm that computes the group operation), a *generator* g for the group, and an element $h \in \mathbb{G}$, as well access to a black box that on input $f \in \mathbb{G}$ outputs the *order* of f (the smallest a such that $f^a = 1$), computes the *discrete logarithm* of h with respect to g. That is the algorithm should output a number x such that $g^x = h$. See footnote for hint.[26]

[26] We are given $h = g^x$ and need to recover x. To do so we can compute the order of various elements of the form $h^a g^b$. The order of such an element is a number c satisfying $c(xa + b) = 0 \pmod{|\mathbb{G}|}$. With a few random examples we will get a non trivial equation on x (where c is not zero modulo $|\mathbb{G}|$) and then we can use our knowledge of a, b, c to recover x.

■

22.13 BIBLIOGRAPHICAL NOTES

Chapters 9 and 10 in the book *Quantum Computing Since Democritus* give an informal but highly informative introduction to the topics of this lecture and much more. Shor's and Simon's algorithms are also covered in Chapter 10 of my book with Arora on computational complexity.

There are many excellent videos available online covering some of these materials. The Fourier transform is covered in this videos of Dr. Chris Geoscience, Clare Zhang and Vi Hart. More specifically to

quantum computing, the videos of Umesh Vazirani on the Quantum Fourier Transform and Kelsey Houston-Edwards on Shor's Algorithm are very recommended.

Chapter 10 in Avi Wigderson's book gives a high level overview of quantum computing. Andrew Childs' lecture notes on quantum algorithms, as well as the lecture notes of Umesh Vazirani, John Preskill, and John Watrous

Regarding quantum mechanics in general, this video illustrates the double slit experiment, this Scientific American video is a nice exposition of Bell's Theorem. This talk and panel moderated by Brian Greene discusses some of the philosophical and technical issues around quantum mechanics and its so called "measurement problem". The Feynmann lecture on the Fourier Transform and quantum mechanics in general are very much worth reading.

The Fast Fourier Transform, used as a component in Shor's algorithm, is one of the most useful algorithms across many applications areas. The stories of its discovery by Gauss in trying to calculate asteroid orbits and rediscovery by Tukey during the cold war are fascinating as well.

22.14 FURTHER EXPLORATIONS

Some topics related to this chapter that might be accessible to advanced students include: (to be completed)

22.15 ACKNOWLEDGEMENTS

Thanks to Scott Aaronson for many helpful comments about this chapter.

VI

APPENDICES

Bibliography

[Aar13] Scott Aaronson. *Quantum computing since Democritus*. Cambridge University Press, 2013.

[AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. "PRIMES is in P". In: *Annals of mathematics* (2004), pp. 781–793.

[Asp18] James Aspens. *Notes on Discrete Mathematics*. Online textbook for CS 202. Available on http://www.cs.yale.edu/homes/aspnes/classes/202/notes.pdf. 2018.

[Bar84] H. P. Barendregt. *The lambda calculus : its syntax and semantics*. Amsterdam New York New York, N.Y: North-Holland Sole distributors for the U.S.A. and Canada, Elsevier Science Pub. Co, 1984. ISBN: 0444875085.

[Blä13] Markus Bläser. *Fast Matrix Multiplication*. Graduate Surveys 5. Theory of Computing Library, 2013, pp. 1–60. DOI: 10.4086/toc.gs.2013.005. URL: http://www.theoryofcomputing.org/library.html.

[Boo47] George Boole. *The mathematical analysis of logic*. Philosophical Library, 1847.

[Bur78] Arthur W Burks. "Booke review: Charles S. Peirce, The new elements of mathematics". In: *Bulletin of the American Mathematical Society* 84.5 (1978), pp. 913–918.

[Cho56] Noam Chomsky. "Three models for the description of language". In: *IRE Transactions on information theory* 2.3 (1956), pp. 113–124.

[Chu41] Alonzo Church. *The calculi of lambda-conversion*. Princeton London: Princeton University Press H. Milford, Oxford University Press, 1941. ISBN: 978-0-691-08394-0.

[CM00] Bruce Collier and James MacLachlan. *Charles Babbage: And the engines of perfection*. Oxford University Press, 2000.

[Coh08] Paul Cohen. *Set theory and the continuum hypothesis*. Mineola, N.Y: Dover Publications, 2008. ISBN: 0486469212.

[Coo66] SA Cook. "On the minimum computation time for multiplication". In: *Doctoral dissertation, Harvard University Department of Mathematics, Cambridge, Mass.* 1 (1966).

[Coo87] James W Cooley. "The re-discovery of the fast Fourier transform algorithm". In: *Microchimica Acta* 93.1-6 (1987), pp. 33–45.

[Cor+09] Thomas H. Cormen et al. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009. ISBN: 978-0-262-03384-8. URL: http://mitpress.mit.edu/books/introduction-algorithms.

[CRT06] Emmanuel J Candes, Justin K Romberg, and Terence Tao. "Stable signal recovery from incomplete and inaccurate measurements". In: *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences* 59.8 (2006), pp. 1207–1223.

[CT06] Thomas M. Cover and Joy A. Thomas. "Elements of information theory 2nd edition". In: *Willey-Interscience: NJ* (2006).

[Dau90] Joseph Warren Dauben. *Georg Cantor: His mathematics and philosophy of the infinite*. Princeton University Press, 1990.

[De 47] Augustus De Morgan. *Formal logic: or, the calculus of inference, necessary and probable*. Taylor and Walton, 1847.

[Don06] David L Donoho. "Compressed sensing". In: *IEEE Transactions on information theory* 52.4 (2006), pp. 1289–1306.

[DPV08] Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani. *Algorithms*. McGraw-Hill, 2008. ISBN: 978-0-07-352340-8.

[Ell10] Jordan Ellenberg. "Fill in the blanks: Using math to turn lo-res datasets into hi-res samples". In: *Wired Magazine* 18.3 (2010), pp. 501–509.

[Ern09] Elizabeth Ann Ernst. "Optimal combinational multi-level logic synthesis". Available on https://deepblue.lib.umich.edu/handle/2027.42/62373. PhD thesis. University of Michigan, 2009.

[Fle18] Margaret M. Fleck. *Building Blocks for Theoretical Computer Science*. Online book, available at http://mfleck.cs.illinois.edu/building-blocks/. 2018.

[Für07] Martin Fürer. "Faster integer multiplication". In: *Proceedings of the 39th Annual ACM Symposium on Theory of Computing, San Diego, California, USA, June 11-13, 2007*. 2007, pp. 57–66.

[FW93] Michael L Fredman and Dan E Willard. "Surpassing the information theoretic bound with fusion trees". In: *Journal of computer and system sciences* 47.3 (1993), pp. 424–436.

[GKS17] Daniel Günther, Ágnes Kiss, and Thomas Schneider. "More efficient universal circuit constructions". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2017, pp. 443–470.

[Gra05] Judith Grabiner. *The origins of Cauchy's rigorous calculus*. Mineola, N.Y: Dover Publications, 2005. ISBN: 9780486438153.

[Gra83] Judith V Grabiner. "Who gave you the epsilon? Cauchy and the origins of rigorous calculus". In: *The American Mathematical Monthly* 90.3 (1983), pp. 185–194.

[Hag98] Torben Hagerup. "Sorting and searching on the word RAM". In: *Annual Symposium on Theoretical Aspects of Computer Science*. Springer. 1998, pp. 366–398.

[Hal60] Paul R Halmos. *Naive set theory*. Republished in 2017 by Courier Dover Publications. 1960.

[Har41] GH Hardy. "A Mathematician's Apology". In: (1941).

[HJB85] Michael T Heideman, Don H Johnson, and C Sidney Burrus. "Gauss and the history of the fast Fourier transform". In: *Archive for history of exact sciences* 34.3 (1985), pp. 265–277.

[HMU14] John Hopcroft, Rajeev Motwani, and Jefferey Ullman. *Introduction to automata theory, languages, and computation*. Harlow, Essex: Pearson Education, 2014. ISBN: 1292039051.

[Hof99] Douglas Hofstadter. *Gödel, Escher, Bach : an eternal golden braid*. New York: Basic Books, 1999. ISBN: 0465026567.

[Hol01] Jim Holt. "The Ada perplex: how Byron's daughter came to be celebrated as a cybervisionary". In: *The New Yorker* 5 (2001), pp. 88–93.

[Hol18] Jim Holt. *When Einstein walked with Gödel : excursions to the edge of thought*. New York: Farrar, Straus and Giroux, 2018. ISBN: 0374146705.

[HU69] John E Hopcroft and Jeffrey D Ullman. "Formal languages and their relation to automata". In: (1969).

[HU79] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages and Computation*. Addison-Wesley, 1979. ISBN: 0-201-02988-X.

[Juk12] Stasys Jukna. *Boolean function complexity: advances and frontiers*. Vol. 27. Springer Science & Business Media, 2012.

[Kar+97] David R. Karger et al. "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web". In: *Proceedings of the Twenty-Ninth Annual ACM Symposium on the Theory of Computing, El Paso, Texas, USA, May 4-6, 1997*. 1997, pp. 654–663. DOI: 10.1145/258533.258660. URL: https://doi.org/10.1145/258533.258660.

[Kar95] ANATOLII ALEXEEVICH Karatsuba. "The complexity of computations". In: *Proceedings of the Steklov Institute of Mathematics-Interperiodica Translation* 211 (1995), pp. 169–183.

[Kle91] Israel Kleiner. "Rigor and Proof in Mathematics: A Historical Perspective". In: *Mathematics Magazine* 64.5 (1991), pp. 291–314. ISSN: 0025570X, 19300980. URL: http://www.jstor.org/stable/2690647.

[Kle99] Jon M Kleinberg. "Authoritative sources in a hyperlinked environment". In: *Journal of the ACM (JACM)* 46.5 (1999), pp. 604–632.

[Koz97] Dexter Kozen. *Automata and computability*. New York: Springer, 1997. ISBN: 978-3-642-85706-5.

[KT06] Jon M. Kleinberg and Éva Tardos. *Algorithm design*. Addison-Wesley, 2006. ISBN: 978-0-321-37291-8.

[Kun18] Jeremy Kun. *A programmer's introduction to mathematics*. Middletown, DE: CreateSpace Independent Publishing Platform, 2018. ISBN: 1727125452.

[Liv05] Mario Livio. *The equation that couldn't be solved : how mathematical genius discovered the language of symmetry*. New York: Simon & Schuster, 2005. ISBN: 0743258207.

[LLM18] Eric Lehman, Thomson F. Leighton, and Albert R. Meyer. *Mathematics for Computer Science*. 2018.

[LMS16] Helger Lipmaa, Payman Mohassel, and Seyed Saeed Sadeghian. "Valiant's Universal Circuit: Improvements, Implementation, and Applications." In: *IACR Cryptology ePrint Archive* 2016 (2016), p. 17.

[Lup58] O Lupanov. "A circuit synthesis method". In: *Izv. Vuzov, Radiofizika* 1.1 (1958), pp. 120–130.

[Lup84] Oleg B. Lupanov. *Asymptotic complexity bounds for control circuits*. In Russian. MSU, 1984.

[Lus+08] Michael Lustig et al. "Compressed sensing MRI". In: *IEEE signal processing magazine* 25.2 (2008), pp. 72–82.

[Lüt02] Jesper Lützen. "Between Rigor and Applications: Developments in the Concept of Function in Mathematical Analysis". In: *The Cambridge History of Science*. Ed. by Mary JoEditor Nye. Vol. 5. The Cambridge History of Science. Cambridge University Press, 2002, pp. 468–487. DOI: 10.1017/CHOL9780521571999.026.

[LZ19] Harry Lewis and Rachel Zax. *Essential Discrete Mathematics for Computer Science*. Princeton University Press, 2019. ISBN: 9780691190617.

[MM11] Cristopher Moore and Stephan Mertens. *The nature of computation*. Oxford University Press, 2011.

[MP43] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.

[NS05] Noam Nisan and Shimon Schocken. *The elements of computing systems: building a modern computer from first principles*. MIT press, 2005.

[Pag+99] Lawrence Page et al. *The PageRank citation ranking: Bringing order to the web.* Tech. rep. Stanford InfoLab, 1999.

[PE76] Charles Sanders Peirce and Carolyn Eisele. *The new elements of mathematics*. Vol. 4. Works of Charles Peirce, collected and edited by Carolyn Eisele. Mouton The Hague, 1976.

[Pie02] Benjamin Pierce. *Types and programming languages*. Cambridge, Mass: MIT Press, 2002. ISBN: 0262162091.

[Ric53] Henry Gordon Rice. "Classes of recursively enumerable sets and their decision problems". In: *Transactions of the American Mathematical Society* 74.2 (1953), pp. 358–366.

[Ros19] Kenneth Rosen. *Discrete mathematics and its applications*. New York, NY: McGraw-Hill, 2019. ISBN: 125967651x.

[RS59] Michael O Rabin and Dana Scott. "Finite automata and their decision problems". In: *IBM journal of research and development* 3.2 (1959), pp. 114–125.

[Sav98] John E Savage. *Models of computation*. Vol. 136. Available electronically at http://cs.brown.edu/people/jsavage/book/. Addison-Wesley Reading, MA, 1998.

[Sha38] Claude E Shannon. "A symbolic analysis of relay and switching circuits". In: *Electrical Engineering* 57.12 (1938), pp. 713–723.

[Sha79] Adi Shamir. "Factoring numbers in O (logn) arithmetic steps". In: *Information Processing Letters* 8.1 (1979), pp. 28–31.

[She03] Saharon Shelah. "Logical dreams". In: *Bulletin of the American Mathematical Society* 40.2 (2003), pp. 203–228.

[She13] Henry Maurice Sheffer. "A set of five independent postulates for Boolean algebras, with application to logical constants". In: *Transactions of the American mathematical society* 14.4 (1913), pp. 481–488.

[She16] Margot Shetterly. *Hidden figures : the American dream and the untold story of the Black women mathematicians who helped win the space race*. New York, NY: William Morrow, 2016. ISBN: 9780062363602.

[Sin97] Simon Singh. *Fermat's enigma : the quest to solve the world's greatest mathematical problem*. New York: Walker, 1997. ISBN: 0385493622.

[Sip97] Michael Sipser. *Introduction to the theory of computation*. PWS Publishing Company, 1997. ISBN: 978-0-534-94728-6.

[Sob17] Dava Sobel. *The Glass Universe : How the Ladies of the Harvard Observatory Took the Measure of the Stars*. New York: Penguin Books, 2017. ISBN: 9780143111344.

[Sol14] Daniel Solow. *How to read and do proofs : an introduction to mathematical thought processes*. Hoboken, New Jersey: John Wiley & Sons, Inc, 2014. ISBN: 9781118164020.

[SS71] Arnold Schönhage and Volker Strassen. "Schnelle multiplikation grosser zahlen". In: *Computing* 7.3-4 (1971), pp. 281–292.

[Ste87] Dorothy Stein. *Ada : a life and a legacy*. Cambridge, Mass: MIT Press, 1987. ISBN: 0262691167.

[Str69] Volker Strassen. "Gaussian elimination is not optimal". In: *Numerische mathematik* 13.4 (1969), pp. 354–356.

[Swa02] Doron Swade. *The difference engine : Charles Babbage and the quest to build the first computer*. New York: Penguin Books, 2002. ISBN: 0142001449.

[Tho84] Ken Thompson. “Reflections on trusting trust”. In: *Communications of the ACM* 27.8 (1984), pp. 761–763.

[Too63] Andrei L Toom. “The complexity of a scheme of functional elements realizing the multiplication of integers”. In: *Soviet Mathematics Doklady*. Vol. 3. 4. 1963, pp. 714–716.

[Tur37] Alan M Turing. “On computable numbers, with an application to the Entscheidungsproblem”. In: *Proceedings of the London mathematical society* 2.1 (1937), pp. 230–265.

[Val76] Leslie G Valiant. “Universal circuits (preliminary report)”. In: *Proceedings of the eighth annual ACM symposium on Theory of computing*. ACM. 1976, pp. 196–203.

[Wan57] Hao Wang. “A variant to Turing’s theory of computing machines”. In: *Journal of the ACM (JACM)* 4.1 (1957), pp. 63–92.

[Weg87] Ingo Wegener. *The complexity of Boolean functions*. Vol. 1. BG Teubner Stuttgart, 1987.

[Wer74] PJ Werbos. “Beyond regression: New tools for prediction and analysis in the behavioral sciences. Ph. D. thesis, Harvard University, Cambridge, MA, 1974.” In: (1974).

[WR12] Alfred North Whitehead and Bertrand Russell. *Principia mathematica*. Vol. 2. University Press, 1912.